Secure Software Systems

Design and Development

Erik Fretheim | Marie Deschene

jbl:\cybersecurity

jbl:\cybersecurity

Secure Software Systems

Design and Development

Erik Fretheim | Marie Deschene

JONES & BARTLETT
LEARNING

World Headquarters
Jones & Bartlett Learning
25 Mall Road
Burlington, MA 01803
978-443-5000
info@jblearning.com
www.jblearning.com

Jones & Bartlett Learning books and products are available through most bookstores and online booksellers. To contact Jones & Bartlett Learning directly, call 800-832-0034, fax 978-443-8000, or visit our website, www.jblearning.com.

Substantial discounts on bulk quantities of Jones & Bartlett Learning publications are available to corporations, professional associations, and other qualified organizations. For details and specific discount information, contact the special sales department at Jones & Bartlett Learning via the above contact information or send an email to specialsales@jblearning.com.

Copyright © 2024 by Jones & Bartlett Learning, LLC, an Ascend Learning Company

All rights reserved. No part of the material protected by this copyright may be reproduced or utilized in any form, electronic or mechanical, including photocopying, recording, or by any information storage and retrieval system, without written permission from the copyright owner.

The content, statements, views, and opinions herein are the sole expression of the respective authors and not that of Jones & Bartlett Learning, LLC. Reference herein to any specific commercial product, process, or service by trade name, trademark, manufacturer, or otherwise does not constitute or imply its endorsement or recommendation by Jones & Bartlett Learning, LLC and such reference shall not be used for advertising or product endorsement purposes. All trademarks displayed are the trademarks of the parties noted herein. *Secure Software Systems: Design and Development* is an independent publication and has not been authorized, sponsored, or otherwise approved by the owners of the trademarks or service marks referenced in this product.

There may be images in this book that feature models; these models do not necessarily endorse, represent, or participate in the activities represented in the images. Any screenshots in this product are for educational and instructive purposes only. Any individuals and scenarios featured in the case studies throughout this product may be real or fictitious but are used for instructional purposes only.

Production Credits
Vice President, Product Management: Marisa R. Urbano
Vice President, Content Strategy and Implementation: Christine Emerton
Director, Product Management: Ray Chew
Director, Content Management: Donna Gridley
Content Strategist: Melissa Duffy
Content Coordinator: Mark Restuccia
Director, Project Management and Content Services: Karen Scott
Manager, Program Management: Kristen Rogers
Project Manager: Belinda Thresher
Senior Digital Project Specialist: Angela Dooley
Director, Marketing: Andrea DeFronzo
Marketing Manager: Mark Adamiak

Content Services Manager: Colleen Lamy
Product Fulfillment Manager: Wendy Kilborn
Composition: Straive
Project Management: Straive
Cover Design: Briana Yates
Media Development Editor: Faith Brosnan
Rights & Permissions Manager: John Rusk
Rights Specialist: James Fortney
Cover Image (Title Page and Chapter Opener): © d1sk/Shutterstock
Printing and Binding: Gasch Printing

Library of Congress Cataloging-in-Publication Data
Names: Fretheim, Erik, author. | Deschene, Marie, author.
Title: Secure software systems : design and development / Erik Fretheim, Marie Deschene.
Description: First edition. | Burlington, Massachusetts : Jones & Bartlett Learning, [2024] | Includes bibliographical references and index. | Summary: “Secure Software Systems Development addresses the software development process from the perspective of a security practitioner”– provided by publisher.
Identifiers: LCCN 2022056380 | ISBN 9781284261158 (paperback)
Subjects: LCSH: Application software–Development. | Computer security.
Classification: LCC QA76.76.D47 S424 2024 | DDC 005.1–dc23/eng/20230109
LC record available at https://lccn.loc.gov/2022056380

6048

Printed in the United States of America
27 26 25 24 23 10 9 8 7 6 5 4 3 2 1

© d1sk/Shutterstock

Contents

Preface
Acknowledgments

CHAPTER 1 Secure Software Systems Development
Introduction
Secure Software Systems Life Cycle
Understanding the Model
System Initiation
Defining and Planning
Design and Decision
Development
Test and Evaluation
Deployment
Operations and Maintenance
Retirement
Variations in the Life Cycle
The Five Ps
Combining the Five Ps with the Secure Software Systems Life Cycle
KEY TERMS

CHAPTER 2 Product and Portfolio Management
Introduction
Product Management
Identifying Opportunities

Product Definition
Product Planning
Bringing the Product to Market
Ecosystem Management
Portfolio Management
SUMMARY
KEY TERMS

CHAPTER 3 Program and Project Management
Introduction
Project Management
Phase 1—Initiation/Conception/Analysis
Phase 2—Define and Plan
Phase 3—Implementation/Execution
Phase 4—Monitor and Control
Phase 5—Project Close
Project Management Summary
Program Management
Strategic Approach
Program Sponsorship
Program Managers
Program Management Office
SUMMARY
KEY TERMS
REFERENCE

CHAPTER 4 Process Management
Introduction
Managing Processes
Identifying Processes
Understanding Processes
Documenting Processes
Process Diagrams
Static Process Analysis
Dynamic Process Analysis
Queueing Theory
Workflow
Process Quality
Process Improvement
Processes and Security
Defining Processes for the Developed System
SUMMARY
KEY TERMS

CHAPTER 5 Managing the Secure Software Systems Development Life Cycle
- Introduction
- Requirements Tracking
- Change Management
- Issue Management
- SUMMARY
- KEY TERMS
- REFERENCE

CHAPTER 6 Security Culture, Responsibility, and Training
- Introduction
- Look and Feel of a Security Culture
  - Leadership Sets the Tone for Cybersecurity
  - Someone with Responsibility and Authority
  - Security Roles and Responsibilities Defined for All
  - Security Is Considered in All Aspects of Products
  - Security Is Widely Discussed and Shared with Customers
  - Security Is Valued
  - Security Culture Maturity
  - Company Policies and Procedures Are Established and Available
  - An Effective Security Culture among Programmers and System Administrators Is Vital
- Responsibility
  - Ethical Integrity
  - Professionalism
  - Long-Term Thinking
- Training the Organization to Be Secure
  - What Training Is Important
  - When to Train
  - Training Style
- SUMMARY
- KEY TERMS

CHAPTER 7 Requirements and Security Requirements Planning
- Introduction
- Types of Requirements
- System Definition and Scope
- Sources of Requirements
  - Stakeholders as a Source of Requirements
  - Intended Security Practices as Security Requirements
  - Outsourcing and Cloud Services as a Source of Requirements
- Requirements Engineering
  - Assessing Requirements for Security
- Requirements Documentation and Tracing

Traceability
Formatting Requirements for Sharing
SUMMARY
KEY TERMS

CHAPTER 8 Compliance
Introduction
Need for Compliance
Laws and Regulations Related to Compliance
Implications of Data Privacy Regulations and Laws
U.S. Regulations and Laws
Canadian Regulations and Laws
European Union Regulations and Laws
China Regulations and Laws
Compliance Models and Frameworks
Control Objectives for Information and related Technology (COBIT)
Capability Maturity Model Integration (CMMI)
Secure Software Development Framework (SSDF) (NIST)
Monitoring
Systems Auditing
SUMMARY
KEY TERMS
REFERENCES

CHAPTER 9 Quality Management
Introduction
Quality Assurance
Processes
Tools
Task Expectations
Training
Auditing and Monitoring Processes
Quality Control
Quality Expectations
Testing
Root Cause Analysis
Corrective Actions
Quality Organizations
SUMMARY
KEY TERMS

CHAPTER 10 Modeling
Introduction

- Choosing a Model
- Modeling Tools
- Modeling Techniques
  - Flow Charts
  - Structure Charts
  - E-R Diagram
  - Object Model
  - Use Case Model
  - Sequence Diagram
  - State Diagram
- SUMMARY
- KEY TERMS

CHAPTER 11 Architecture
- Introduction
- Architecture Views or Models
  - Business or Functional Architecture
  - Logical Architecture
  - Application Architecture
  - Data Architecture
  - Physical and Technical Architectures
- Enterprise Architecture
- Creating the Architecture
  - Architectural Concerns
  - Architectural Principles
  - Security Objectives
- Design Patterns
  - Example Design Patterns
- Refinement Techniques
- Architectural Reviews
- SUMMARY
- KEY TERMS

CHAPTER 12 Vulnerability and Threat Assessment
- Introduction
- Vulnerability Assessment and Mapping
  - The Process
  - The Vulnerability Matrix
- Risk Assessment
- Additional Vulnerability Assessment Tools
- Trust Boundaries
  - Life of Data
  - Other Sources for Analysis

- Threat Modeling
  - Purpose of the System
- Threat Intelligence
  - Building the Threat Model
- SUMMARY
- KEY TERMS

CHAPTER 13 The Development Environment
- Introduction
- Culture of Secure Development Practices
- Secure Development Tools
- Software Sources
- Selecting the Development Language
- Using Libraries in the Development Process
  - Commercial Considerations
  - Excessive Library Dependence
  - Dependency Issues
  - Inherited Vulnerabilities
- Operating Systems and Architecture
- Cloud Environments
  - Cloud Security Considerations
  - Cloud Services Engagement
  - Cloud Development Practices
- Coding Practices
  - Standing Operating Procedures (SOPs)
  - Programming Manuals or Style Guides
  - Fail-Safe Logic and Coding
  - Integration Issues
- SUMMARY
- KEY TERMS
- REFERENCES

CHAPTER 14 Configuration Management
- Introduction
- Version Control
- Configuration Management in Product Development
- Configuration Management in Operations and Maintenance
- SUMMARY
- KEY TERMS

CHAPTER 15 Testing
- Introduction
- Test-Driven Development

- The Test Plan
  - The Risk-Based Test Strategy
  - The Test Team
  - Test Tracking and Communication
  - Test Schedule
  - Test Cases
  - Testing Tools
- Types of Testing
  - Standard Development Test Methods
  - User Test Methods
  - Specialized Test Methods
- SUMMARY
- KEY TERMS
- REFERENCE

CHAPTER 16 Product Release and Deployment
- Introduction
- Preparing for Release
- Release Review
  - Final Security Review
  - Open Issues
- Accreditation for Release
  - Internal Accreditation
  - Government System Accreditation
- Deployment
  - Deployment Roles
  - Training before Deployment
  - Deployment Communications
  - Deployment or Migration Planning
  - Deployment/Migration Window
- SUMMARY
- KEY TERMS

CHAPTER 17 Operations and Maintenance
- Introduction
- Operations
  - Monitoring
  - Security Monitoring
  - User Support
  - Backups
- Incident Management
  - Incident Response Plan
  - Incident Root Cause Analysis

Incident Tracking
Disaster Recovery
Preparation and Practice
Post-Incident Review
Maintenance
Configuration and Software Updates
Functional Fixes and Patches
Security Configuration and Software Updates
Vulnerability Management
Change Management
Maintenance Windows
Environmental Issues
Power Consumption
E-Waste
SUMMARY
KEY TERMS

CHAPTER 18 Retirement or End-of-Life
Introduction
End-of-Life
Planning End-of-Life
Archiving
Strategies When Faced with End-of-Life
Retirement
Communication
Developing a Retirement Plan
Pre-Retirement Audit
Timeline
Removal from Service
Transferring and Storing Data
Decommissioning the Hardware
Licensing Adjustments
Finalizing the Retirement
SUMMARY
KEY TERMS
Glossary
Index

© d1sk/Shutterstock

Preface

The goal of this textbook is to present an approach to secure software systems design and development that tightly integrates security and systems design and development (or software engineering) together. The desire to create the book came from searching for an appropriate textbook for a secure software development course. It quickly became apparent that three types of books were available. The first was a software development/engineering book with a chapter or two of security added at the end. The second was a cybersecurity book with a chapter or two of software development/engineering added at the end. Finally, the third type of book was an A-Z (Ö if you're Swedish) list of all of the potential errors the authors knew about that could be included in a program, with the admonishment "don't do this." None of the alternatives presented a systematic approach to applying security while going through the secure software systems development life cycle.

Security has proven to be a critical part of such software because of the ever-increasing number of attacks. Such attacks are now bold enough to target very important infrastructures such as electrical grids and pipelines. Failure of such infrastructures can lead to many deaths—so security has become more important than ever. That means that security needs to be baked into the software systems development approach, not added at the end, nor it is the provenance of a specific group of security engineers or specialists.

In writing this book, we have focused on the processes, concepts, and concerns of ensuring that secure practices are followed throughout the secure

software systems development life cycle, including the practice of following the life cycle rather than just doing ad hoc development. We have used specific security practices and examples as illustrations rather than a comprehensive list. If you are looking for all of the bugs that can be found in a program implemented in a specific language or tool, a book is not the proper place to look. By the time a book has gone through its publishing cycle, the issue may have already been resolved and a hundred others have taken its place.

Learning Features

The writing style of this text is practical and conversational. Each chapter begins with learning objectives. Illustrations are used to clarify the material and vary presentation. The text is sprinkled with box features to alert the reader to additional information related to the subject under discussion.

Chapter summaries are included in the text to provide a rapid review or preview of the material and to help students understand the relative importance of the concepts presented. Key terms are bolded and listed at the end of each chapter and defined in a glossary at the end of the book. The instructor materials include slides in PowerPoint format and a test bank.

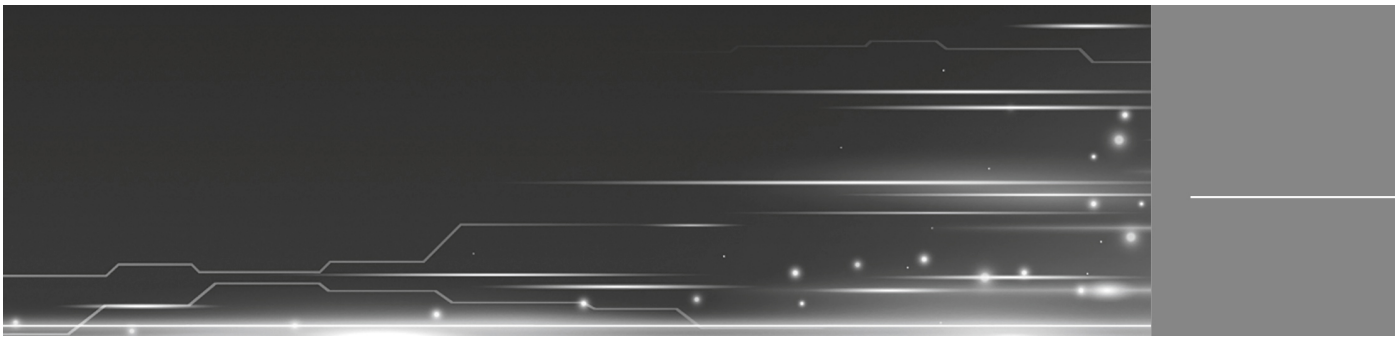
Audience

The material is suitable for undergraduate or graduate computer science majors or information science majors, as well as dedicated cybersecurity and software development programs. This text may also be used at a two-year technical college or community college for students who have a basic technical background or for self-study.

© d1sk/Shutterstock

Acknowledgments

We are grateful to the many colleagues and students who have offered useful comments, advice, and encouragement during the writing of this text.



© d1sk/Shutterstock

CHAPTER 1

Secure Software Systems Development

CHAPTER OBJECTIVES

- Explain how an ordered systematic approach is needed throughout a software life cycle to ensure the development of effective, secure software systems.
- Describe the key phases in the secure software development life cycle (SSDLC), including system initiation; design and planning; design and decision; low-level design and development; test and evaluation; deployment; operations and maintenance; and end of life.
- Examine how no development effort will ever exactly follow the model, but the model remains significant as a guide to order and organize the process.
- Apply the management of processes, projects, programs, products, and portfolios to effectively manage the SSDLC.

Introduction

The objective of secure software systems development is to create software systems that are free of defects and vulnerabilities, built in a repeatable manner, are maintainable, and are well documented.

The concept of a software system is important in this context because software never occurs in isolation. At the very least, it runs on a machine, computer, or processor. More often, especially for interesting or useful software, the software resides in a wide range of environments, processes, and devices. The system can include operating systems, networks, hardware, libraries, application programming interfaces, users, and any number of other items. While it may be tempting to only consider the software itself as the focus of our efforts to make things secure, the reality is that security can only happen if it is applied to the system as a whole. To achieve that we need to apply security to each individual component and it to the system as a whole.

Being free of defects is easy to understand; it simply means that we haven't made any errors. Except, we then must begin to think about what we mean by errors. Do we simply mean that there are no programming errors? That can be relatively easy to check and prevent using intelligent, context-aware editors that perform syntax and other checks on our code as we type it. But those aren't the only kinds of defects encountered during software development. There are also design errors, conceptualization errors, integration errors, security errors, and many other types of errors. Perhaps being defect free isn't so simple after all.

Vulnerabilities also represent a real challenge in identification. Vulnerabilities, unlike defects, are weakness rather that errors; although they can be erroneously introduced, in the right set of circumstances they can potentially be exploited and cause problems in a system. The biggest challenge here comes from that qualifying phrase: in the right set of circumstances. Those circumstances may come from how the components of the software are assembled, they may come from another piece of the system, or they may come from something external. Making the challenge even greater is that vulnerabilities change. A system that appears safe and secure today may not be safe and secure when used in a different location or in a way not originally intended. For that reason, secure software systems development needs to always be aware of the context and circumstances.

Repeatability is another important aspect of secure software systems design. Security should never be a fluke accident that just happened. The methods that make it happen need to occur repeatedly, so the actions that address and contribute to security happen on purpose. That means the methods need to be studied, documented, and applied systematically such that every software system has the best opportunity to be a secure system.

Secure systems are also maintainable systems. Secure systems operate in the environment where vulnerabilities change. That means they need to be maintainable to remain secure. Also, software rots over time. That isn't because the code changes. It is because the environment in which the code runs changes over time, and the software becomes out of date, incompatible with the latest changes in other software, or obsolete. A good example is shown in **Figure 1.1** where a game was built for an earlier version of a PC.

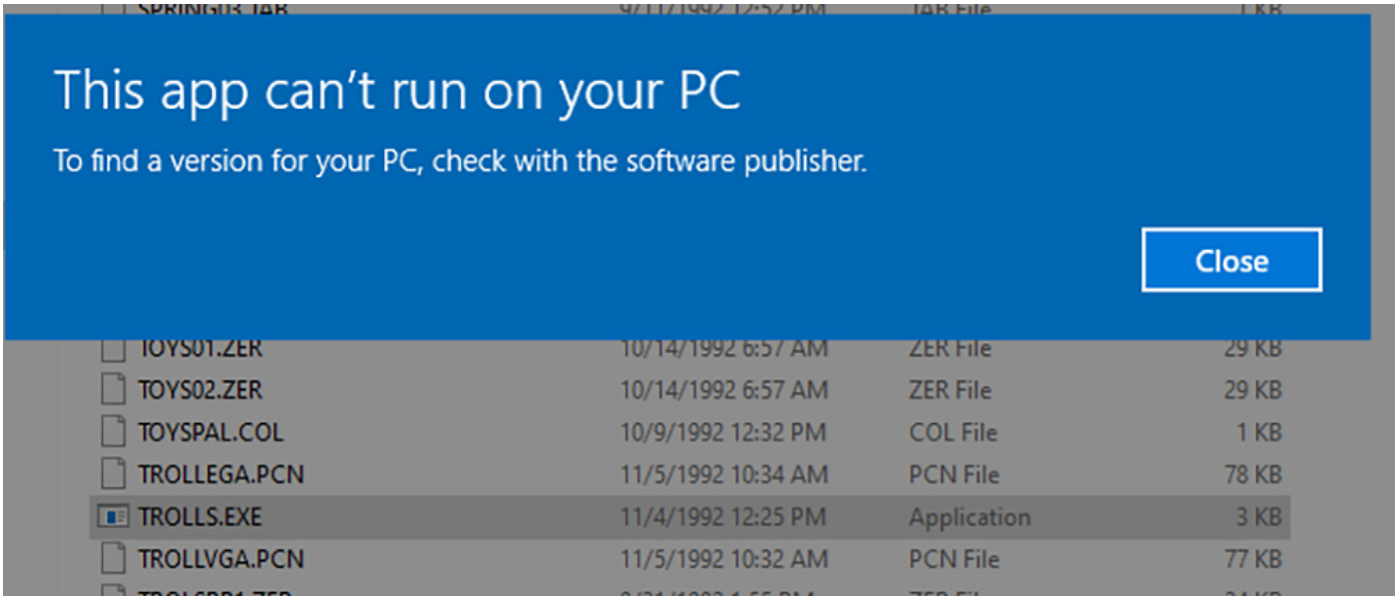


FIGURE 1.1 Software Rot

Description

The 1992 code is still the same. Presumably not a single bit has changed. The code doesn't use a network or application programming interfaces (APIs), just a few simple operating system functions. Those same functions exist in the operating system today, but they don't work the same.

Being maintainable requires good documentation. It means being able to understand the system's function, its context, and its environment. That means having all these explicitly stated in a place where it can be read. Documentation doesn't mean pages and pages of words that bear little if any content. It does mean having significant content available in a manner and format that can be

used to understand the system, be useful in applying maintenance to the system, and help to ensure continuity of intent in the system function.

The development and management of secure software systems is first and foremost dependent on the management of processes, projects, programs, products, and portfolios. Understanding these five Ps and their interrelations allows a distinct advantage to anyone interested in ensuring the security and effectiveness of software. Secure software is not just about memorizing a laundry list of potential threats and vulnerabilities, nor is it about the steps taken to mitigate them. Instead, it is the knowledgeable and effective application of management techniques in each of the five P areas with an appropriate focus on cybersecurity throughout a software system life cycle. Therefore, we begin the study of secure software systems with a study of how to manage the five Ps.

Secure Software Systems Life Cycle

Before we begin with our five Ps though, we first want to introduce the concept of a secure software systems life cycle. And before we can get there, we need to define what we mean by a software system. In the largest sense, we could call a software system any system that depends on software for its operation. Under this definition, that could represent a wide variety of different things: from a small game application that runs on a computer or cellphone, to a toaster that has a software defined controller (whether or not it includes a cloud-based app to tell you the inch-by-inch brownness of your toast's surface), or a business system that tracks inventory, sales, and customer data. Software systems include hardware when that hardware is used to compute things, to sense things, to toast things, or to move things about, as long as that hardware depends on software to accomplish its task or the software depends on the hardware to define the environment.

Software systems have boundaries. Those boundaries may appear to be arbitrary. They are used to define the environment in which the software system exists. We find many places in life where boundaries are fuzzy and ill-defined. For example, is the boundary between North and South America at the Panama Canal because it is a water route from ocean to ocean, at the southern-most border of the southern-most country in Central America, at the middle of the Darien Gap where there is no road from one continent to the other, or is it defined by some other geographic feature? What happens if a canal is built in Nicaragua? Does Panama become a South American country? What happens if a road it built through the Darien Gap? Our definition of the boundary may vary, but whichever the choice, it frames the discussion. Likewise, any software system may be defined in several ways based on different choices of boundaries. If a boundary cannot be determined and defined, then there is no system to be defined.

To give some examples, a game can be implemented as a distinct application on a computing device with an operating system. The boundary between this software system and another is defined through the game's calls to the operating system, and the operating system's commands or returns to the game. If that same game is implemented on a microcontroller with direct inputs to the program from hardware registers or devices, then there is no ability to separate the software from the hardware and define only the programming as a distinct software system.

Being able to define the boundaries of software systems is important for many reasons. Most importantly for now, it allows us to define the limitations within which a software system is secured and within which a software system is managed. Boundaries allow us to set the context for determining where we can consider potential threats as we worry about securing our systems, including concerns about how we keep threats outside of those boundaries, how we keep information secure within those boundaries, and how we create coherence within the bounded system.

Now that we've defined what we mean by a software system, we can begin to consider a software system life cycle, or more properly, a secure software system life cycle. Let's make this clear, there is no difference between a software system life cycle and a secure software system life cycle. There are only degrees to which security is considered. Any secure software system life cycle that purports to be applied separately or distinctly from the basic software system life cycle is inherently not secure and does not address the life cycle of a software system even if it does touch upon it at times. This would be like strapping jet engines onto a Piper Cub and claiming that it performs like an F-16. Even if the plane does manage to match the speed through sheer force alone, it is not going to be able to perform the same turns and capabilities. In fact, it is likely to break up and fall out of the sky because the airframe was never designed from the beginning to be a jet. From this point on, any mention of software system life cycles, software life cycles, software development life cycle (SDLC), or life cycles refers to a secure software system life cycle, unless otherwise specified.

There are many different models for secure software system life cycles. All of them are built as a series of stages, development/life phases, epics, sprints, or some such thing that a software system passes through on its way from idea to retirement. All software system life cycles have loops, even if those are from one end of the cycle to the other end of the cycle (that's why they are called cycles after all). Most software life cycles are valid and useful models that can be used to help plan and manage a software system.

The choice of a particular model, with its particular grammar and vocabulary, will depend on a broad variety of factors, the majority of which boil down to something as simple as "that's what the company has chosen," or "that's what I am familiar with." These are both perfectly valid reasons for choosing a specific model. Others will argue about one specific feature of one model or another or the value of defining the terms in a model in a particular manner. The realities are that most of the distinctive features of any model are created to enable a consultant to sell the model, a student to write a dissertation, or a professor to

develop a textbook. While some features do align more closely with specific business needs in different models at different times, those can be incorporated into any model as it is tailored to the organization where it will be used. Any specific item in any model is nowhere nearly as important as the consistent consideration and application of a life cycle model and the flexibility within the model to tailor it to fit the business needs of the organization.

The secure software system development life cycle model we will use for this text is laid out in **Figure 1.2**.

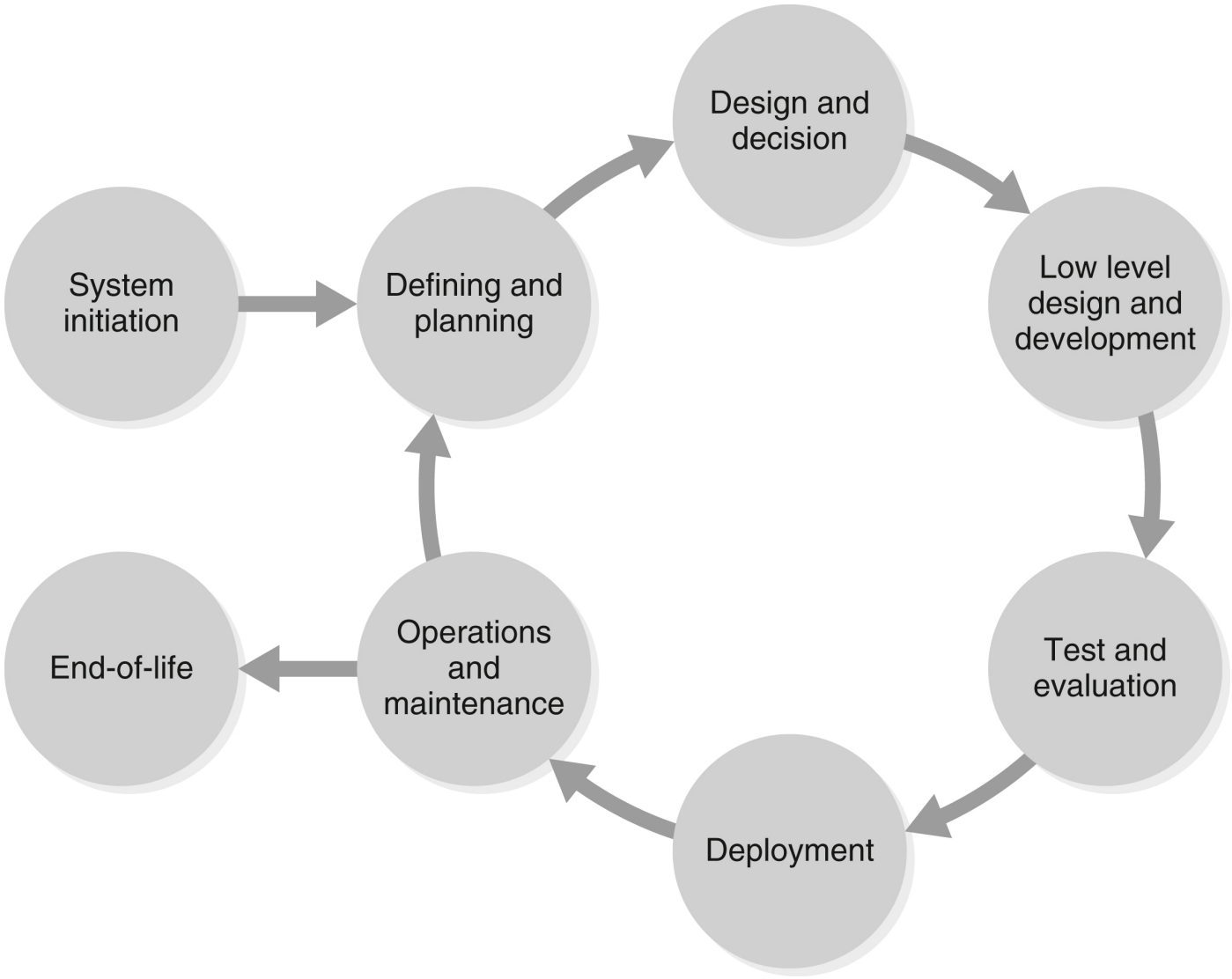


FIGURE 1.2 Secure Software Development Life Cycle Model

Description

Understanding the Model

As we look at the model, at first it will appear that we are about to start a linear process where we begin at the beginning, do each step, and come out at the end with a finished system. The steps of the model all build on each other, and completing essential elements of each step prior to beginning the next is vital, but it is not necessary to fully complete each step or to continuously move forward without circling back to a prior step.

The application of the general model will be different for each development process. If we are using a waterfall process, we start at the beginning, perform each step once, and have a complete finished product at the end. On the other hand, with an Agile process, we loop through defining and planning, design and decision, develop, test, and sometimes deploy, portions of the model many times, and each iteration produces a working software system. As we study the steps in the cycle, consider how each builds upon the other.

System Initiation

The life cycle model begins with system inception or **initiation**. This is where the concept for the software system is first created, or perhaps more accurately, is formalized into a definable, described system. That isn't to say a fully defined, or fully described system, but rather it is something that can be expressed to others as something more than a vague concept or random thought. Often, in an organization there is a formal and defined fashion for this to be done.

Prior to this formality, someone will undoubtedly have mulled over an idea, or a group may have bounced things around, sometimes even over a period of years. But eventually, the concept reaches a stage where it needs to be reduced to a concrete idea. This may be driven by budgets and finances, it may be driven by business needs, or it may be a result of some other organizational process or requirement. In any case, something will trigger an initial description of the need, and the software system will begin to emerge.

Depending on the organization however, it may take some time for a software system to complete the initiation stage. Often requirements include the need for assignment of initial resources, a champion for the system, some defining document, an initial plan, or some combination of all the above. Each organization will specify, either formally or informally, what it takes to initiate a software system. After that point, the system can then start moving into a more regular process where the definitions tend to be more consistent across organizations, even if they are labeled differently.

Defining and Planning

The next step in the life cycle is the **defining and planning** stage. In this stage, the work begins on extending the scoping and definition created in the initiation stage. This may include completing feasibility studies, investigating possible commercial applications, defining the working environment, and addressing a whole host of other issues that may contribute to or inhibit the success of the effort. In some cases, this is an extension of the initiation stage. In others, it may be a preliminary element of the design stage.

Perhaps the most important piece of the defining and planning stage is capturing and engineering the requirements. While the intent of the system may have been created as a part of the initiation, that in and of itself is never enough to describe the full requirements of any system. A requirement gathering exercise can be a very formal and lengthy process that goes into excruciating detail about the most minute particulars about how the system functions. On the other hand, with some extreme programming or Agile methodologies, the initial requirements gathering becomes more an exercise in understanding the environment and scope of the system so that as just-in-time requirements are gathered during the sprints or iterations of the development, those requirements and the subsequent development are restrained within boundaries.

A major concern during this stage is ensuring that the definitions and plans developed are aligned with the mission, objectives, goals, and plans of the business. While business alignment will continue to be critical throughout the process, ensuring it is an imperative at this point will ease the alignment in later stages. This is also the point where major attention to business alignment can fire the imaginations and excite and engage key participants from across the business. This will contribute greatly to the success of the project throughout the effort.

For our purposes, foremost among the requirements are those that are related to security and those imposed upon the development by the threat environment. Also, key at this point is the capture of testing requirements and use cases that influence the initial development of the test strategy.

Design and Decision

Having gathered the requirements and completed the initial planning for system, next is the **design and decision** stage. In this stage, the requirements are transferred into a system architecture, and the designs for the system are established. Decisions are made about what functionality needs to go into

which elements of the architecture, which elements are to be procured externally, and which elements need to be developed. As the architecture emerges, so does the vulnerability map. Based on the architecture and design, decisions will be made at this point about what technologies and hardware need to be implemented. The completed test strategy and a comprehensive test plan emerge from this stage along with the architecture and high-level design.

Development

Following the design and decision stage comes the time for low-level design, build, unit test, and integration, or the **development** stage. This stage is named development, but that is a potentially dangerous name because many forget that the stage contains the critical elements of low-level design, unit test, and other necessary activities. Yet activities cannot be pulled apart and put into their own phases as is often done because this infers that they are distinct activities done serially in isolation.

Done properly, development begins with creating and refining the design of a specific module, submodule, or element based on the stated business requirements, an architectural requirement, or a higher-level design. That design is then analyzed for security, vulnerabilities, structural requirements, and other significant constraints. With an acceptable design, the test requirements and a test bed or harness is put into place. In the process, refinements are fed back into the design. If elements to this module are to be implemented as a submodule, those then go through their individual phases of development.

Once all is ready, if this is a lowest level module, the actual “development” is done, whether that is through programming, configuration, physical build, or other “construction” techniques. Periodically throughout the development, the parts and pieces will be tested to ensure they conform to the design. This piecewise testing is known as unit testing. Unit testing is often the responsibility of the “developer.” though it may also be completed in a more formal manner.

As the parts and pieces are completed and confirmed to be correct (verified), they are brought together into a more complete module. This is a process known as integration. Additional testing, known as integration testing, unsurprisingly, is completed on the modules at each level of integration until the testing is being conducted on the system as a whole.

Test and Evaluation

The last stage of integration testing is referred to as systems testing and brings us to the **test and evaluation** stage. During this stage, testing is used to ensure that the modules and system perform their tasks correctly (verification, functional testing), securely (security testing), and completely as specified (validation).

At the later stages of testing, the task is usually performed by an independent group rather than the developers, often including specialized testers for specific areas, such as security, usability, etc. This adds a different perspective and establishes a space between just making it work and getting it done correctly. Without a separation of responsibilities, the desire to get things done often overwhelms the need to find issues. Also, when developers test, they tend to test using exactly the same use model as they used to build a system, but that is seldom where the issues lie. Testing can involve many tools and approaches ranging from simple inspections to highly automated regression systems involving complex coding.

BOX 1.1 Use Model

A **use model**, sometimes called a user story or use case, is a model developed formally or informally of how a particular product will be used. Often there can be significant variance between a developer's use model and how a product is actually used.

Take the telephone for example. In the development stages of telephony, as the phone companies tried to figure out how much bandwidth was needed for a human voice, they gathered measurements from University of Pennsylvania undergraduates. These resulted in the determination that 3K bandwidth was sufficient.

In making that determination, they made the assumption that all uses of phones would be for business, and since all businesspeople were men, it would not be a problem that all of the undergraduates were also men. This poor development of a usage model still results in more distortions of women's voices on phone calls to this day.

As the testing is conducted, inevitably issues are going to be found involving the system. Whether these issues are fixed or not and whether these are actually "issues" or "features" is going to require some sort of decisions. Most important though is the need to track and manage these issues and others that

will spring up throughout the life cycle. We'll spend some time looking at issue tracking and related areas in a following chapter.

Deciding when the development is complete can be a real challenge in some systems. Is it when the last code is written, when the last test is run, or when the system is up and running? Different organizations are going to have different approaches, none of which need be right or wrong, but that do need to be carefully managed. At some point there is a process within the organization that determines, complete or not, the system is ready to be moved forward, or released. Release is the transition in the cycle from development and test to deployment.

Deployment

With development complete, it becomes time for **deployment** of the system. At first glance this seems to be a simple task. Turn it on, fire it up, and start using it. The reality is far, far different. Rarely is there any system being deployed that is not replacing another system already in use. Sometimes the replaced system is pencil and paper, but even then, significant effort is going to be needed to transition from one system to the other.

Challenges of transition can range from how to transfer data from one system to another correctly, to how to replace management reports used by executives. Among the most challenging are those issues involved with changing the processes in the organization to align with the new system or configuring the system to align with the processes. These are especially difficult when they are entwined with aspects of the organizational culture.

Preparation for deployment of a new or changed software system needs to occur far in advance and should be a consideration already in the early stages of the life cycle. It takes significant time and thought to prepare proper training and to adjust the culture and processes in an organization. Ignored, this can affect the system's ability to perform its function, the security of a system, and the acceptance of a system.

Assuming the training and cultural aspects have been considered, the mechanical aspects of a transition also need significant focus. The first mechanical stage of a transition is the configuration of the system for the specific deployment. If this was an internal development, some of the configuration, or even the majority, may have already been built into the development phase. But even then, the system needs to be configured into the proper deployment environment.

Once readied, a software system is ready for acceptance testing. This is where the system is taken through its paces in an environment as close as possible to that in which it will be deployed in order to ensure all is in order. The formality of this testing will depend on the centrality of the system to the business core, the sensitivity of the system data, the risk acceptable to the organization, as well as many other factors.

One form of acceptance testing is beta testing. This is where a software system is released for customer use under restricted terms, perhaps with liability limitations and lowered expectations, before the organization is fully ready to release to full deployment. In many cases, this can be an effective method, though it does allow for some significant risk.

Other organizations use a dev-ops approach. Dev-ops is a philosophy of development that emphasizes the need for combining team members who are experienced in development and programming with those who are experienced in operations and administration to enable rapid development and deployment of systems. Under this approach, operations heavily influences the development of software systems that are released in a beta-test-like state, often a module or groups of modules at a time. Carefully done, this can spread the risk of a major migration failure over several stages where at any one stage the risk is contained to an acceptable level and mitigated in an ongoing basis. Done carelessly, the risk levels build upon each level until the entire system crashes in a spectacular failure.

Any deployment, however, has risk of failure and challenges in the deployment phase. Evaluating these determines the deployment model that will be used. Other than the beta release or dev-ops models, some other possibilities include hard cut over, parallel transition, and migration by function. All these need to be properly planned and managed, and all of them need some sort of fallback (rollback) planning for the event of a migration failure.

Oh, there is one more cutover model. It's the disaster scenario. The primary or current system fails for some reason. A new or backup system is employed to pull the organization through the disaster. Then, after the immediate issue is resolved, it becomes apparent that there is no effective way to return to the old system. Naturally though, some cases and data just didn't get transferred, so the organization suffers for years from the forced migration. Not a pretty picture.

Operations and Maintenance

Once a software system has been deployed, the next stage of the life cycle is **operations and maintenance**. Software systems are a lot like radioactive material: They have a half-life and decay over time. It would seem reasonable that a program left alone would continue operating consistently the same over a long period. The reality though is that they are not left alone. Instead, programs continuously acquire new data, are used in different ways than originally intended, and are run in changing hardware and network environments. All of this leads to the introduction of newly discovered errors and limitations. The changes in the operating environment also expose the system to new threats, vulnerabilities, and weaknesses. This creates a need for continuous enhanced security. In addition, there is always a requirement for new features. These changes require maintenance activities, such as fixes, tuning, and reconfiguration.

Other operational issues include adding and managing users, managing data backups, and monitoring of system performance. During the operations and maintenance stage, it is easy to fall into habits and activities that effectively reduce the security of a software system without realizing the risks if the security posture isn't visited on a regular basis.

Retirement

The final stage of the software system life cycle is the **retirement** or end-of-life stage. This is the period when a system is taken out of service and decommissioned. Seemingly this would involve simply flipping a switch and shutting the system down. But there is much more to the process. Sometimes this stage requires moving the data to another system or into some sort of a long-term storage scheme. At other times, it could mean scrubbing the system to ensure no errant software or data are available to people with bad intentions.

Scrubbing data includes paying attention to data that may be left on old hardware. Throwing away or otherwise disposing of hardware can put it within reach of people who are interested in obtaining the remnants of data left behind. Cleaning this from the hardware is essential before physical devices are sent off for disposal or recycling. Even the ecologically sound disposal of the system hardware can become a concern. And naturally, just as all the issues are resolved, and you are about to flip the switch that final time, there is going to appear a user with an important task that can "only" be performed on the system about to be shut off.

Variations in the Life Cycle

The preceding discussion makes it sound like the software development cycle is a neat linear process that proceeds systematically through the stages. The reality is far from that. While the cycle serves to provide a structure and helps us to manage the development of software systems, it never works out as simply as laid out. (Although there is a rumor that there once was a software project that followed the life cycle precisely, met all the objectives, and was completed on-time and under-budget. The rumor is always that it happened at a company where a friend of a friend was employed at high wages in a friendly, no-pressure environment, the owners shared all the profit with the employees, and the fairies left chocolates under everyone's pillows each night.)

In practice, there are loops in the cycle where issues and constraints are discovered that require returning to an earlier point. There are discontinuities where new requirements or constraints require a new beginning, middle, or end. There are parts and pieces that are happening in parallel. The cycle appears as in **Figure 1.3**.

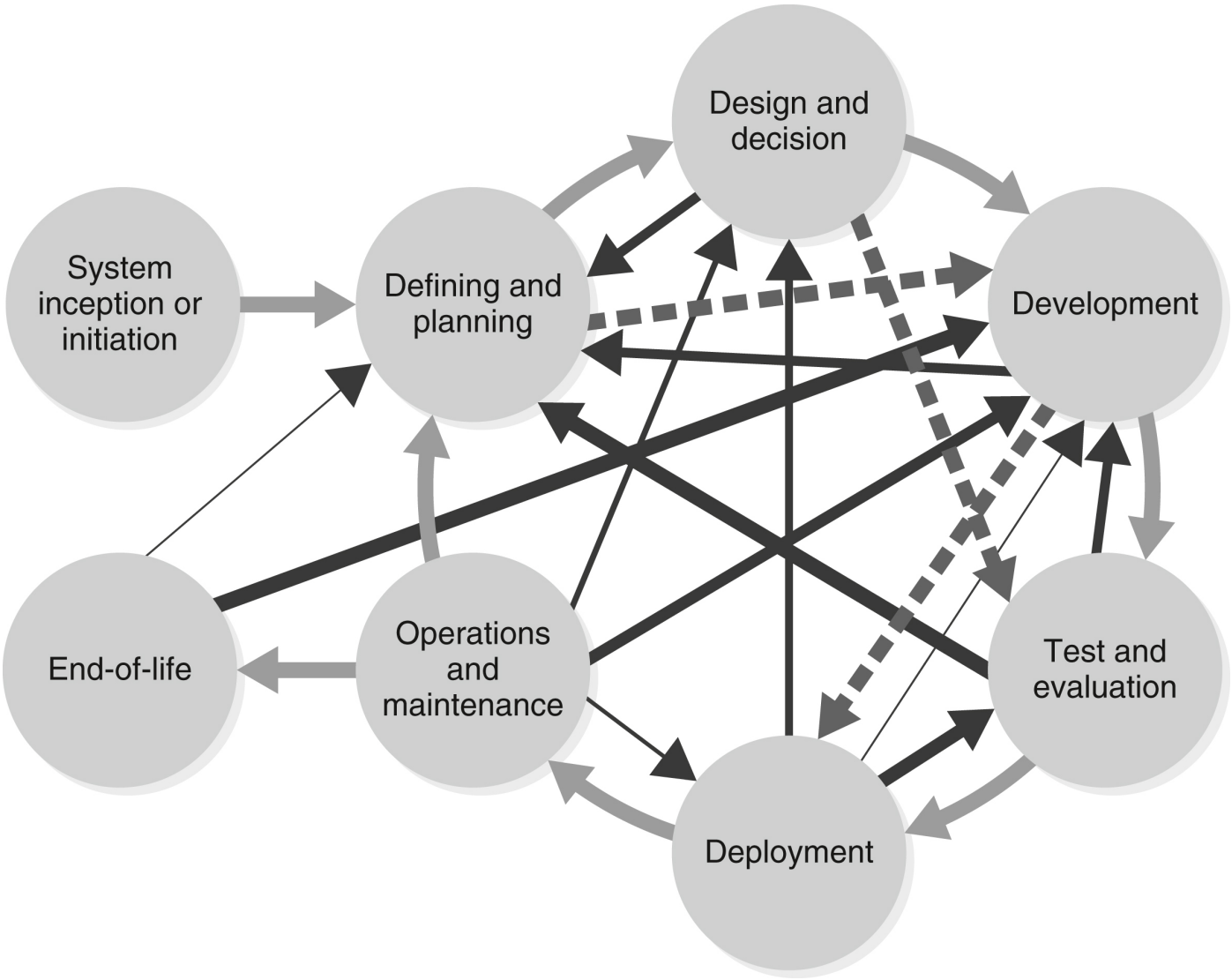


FIGURE 1.3 Secure Development Cycle as It Often Happens

Description

Sometimes people ignore the development cycle altogether. There are developers who charge ahead and begin building before the designs, or even the requirements, are known. Their chart ends up looking much like **Figure 1.4**.

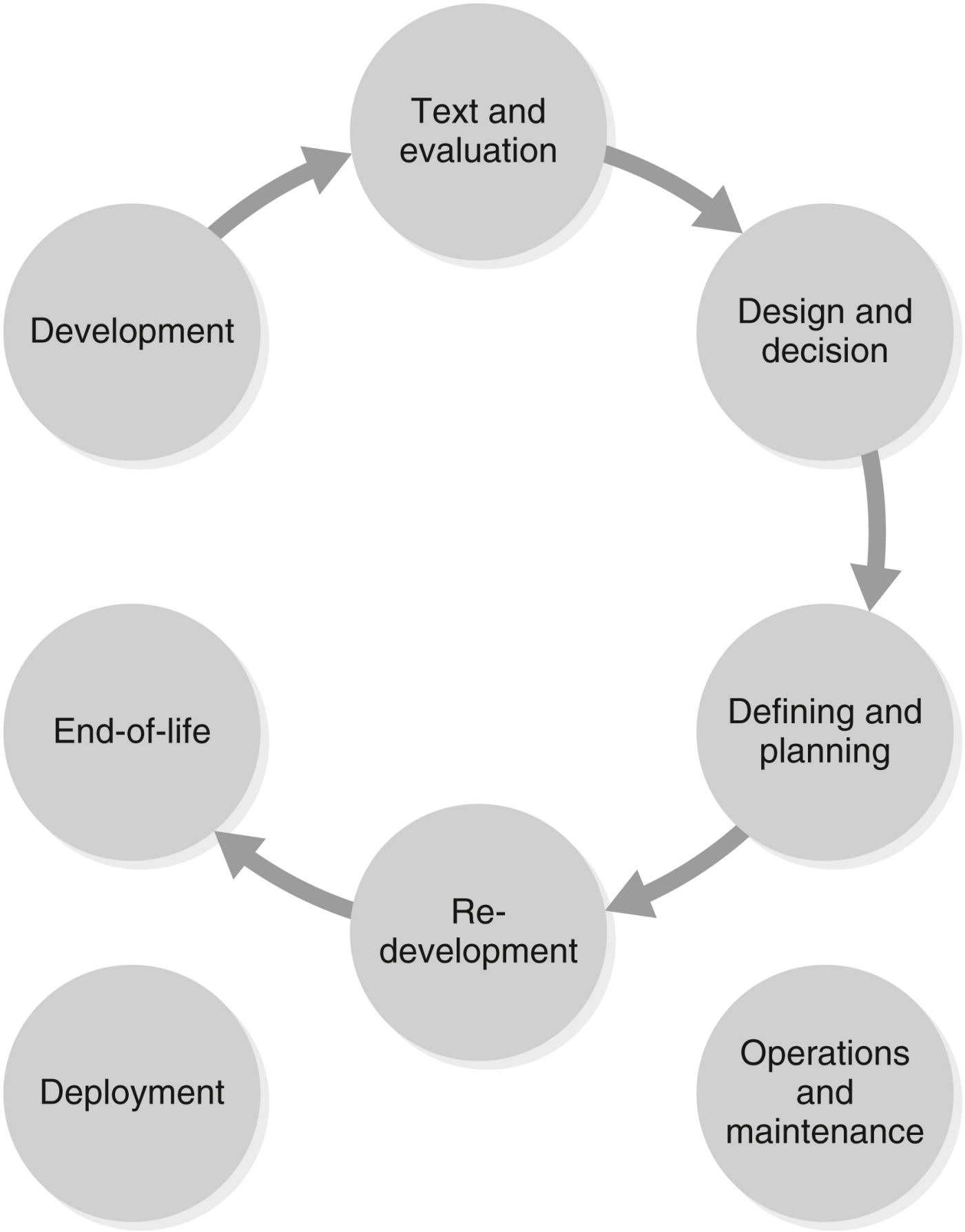


FIGURE 1.4 Unsecure Faulty Development Cycle

Description

The good thing is that through knowing the life cycle, having it as a template, and being able to assess whether the correct issues are addressed at the right stages of the life cycle, we can bring order to chaos and be secure and effective in our software developments.

The model is straightforward and linear even though the process may not be. We can take that model and change the way we portray it to develop models that more closely resemble what we do in reality. For example, we can show a version of the model where we iterate through the model many times, each time adding additional capabilities and complexities to our software system. We can package and sell that as an improved development model. This becomes a spiral model (**Figure 1.5**). It has the same stages; they are just defined to be limited in scope each time they are repeated. That same model, with a focus on people over processes, changes the structure of how processes are applied and at rapid pace becomes Agile development.

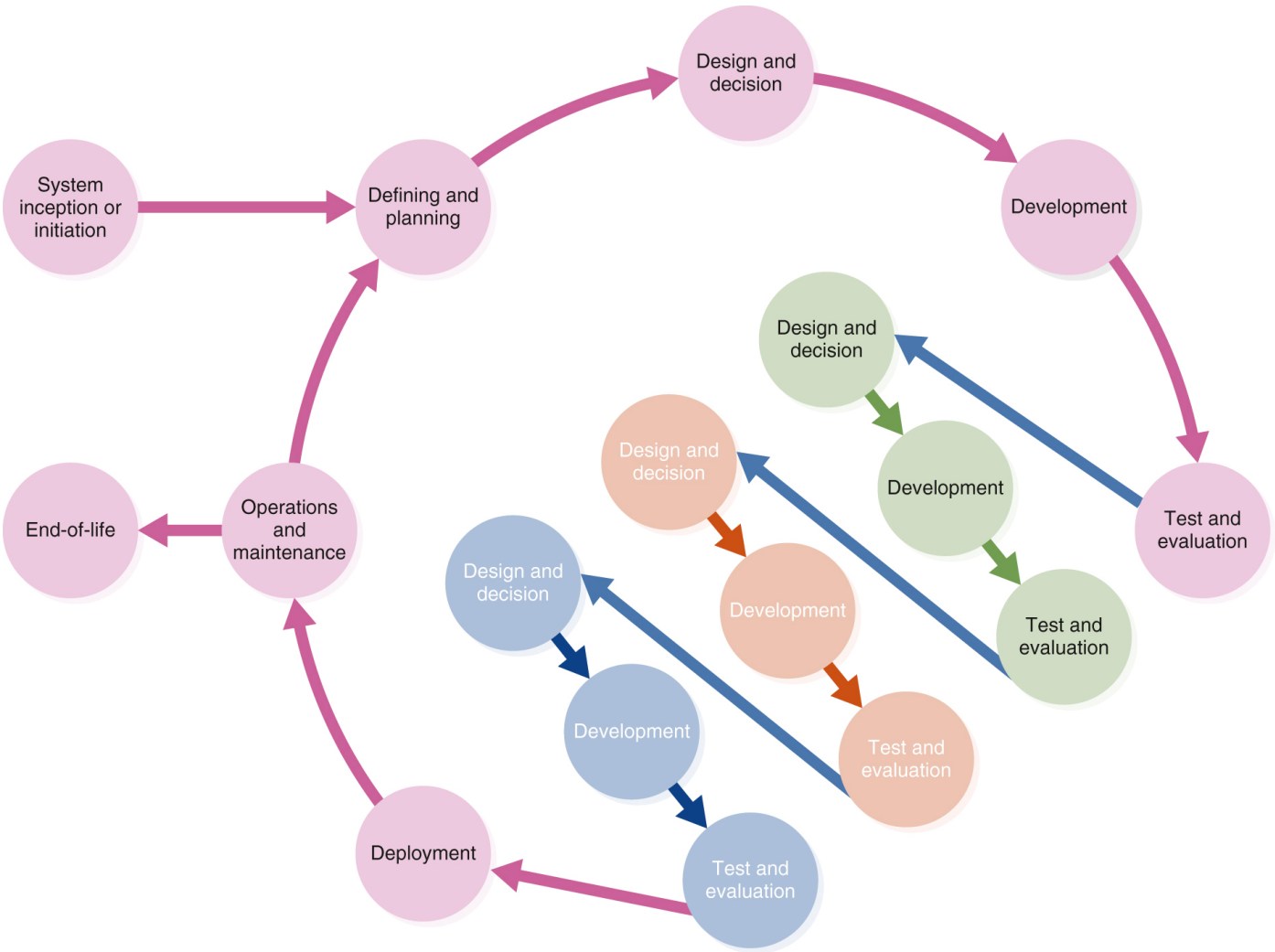


FIGURE 1.5 Spiral Development Model

Description

In another model, we can put an extra development stage ahead of our definition and planning stage in which we try out different concepts and present ideas that are used to stimulate inputs from potential users. This becomes a prototyping model (**Figure 1.6**). We can add other complex pathways through the model to meet specific situations in specific companies and development efforts. This extra development stage doesn't preclude testing; in fact, it is a test of the concepts. If those concepts prove themselves, then as they are incorporated into the design and further developed, they will be more thoroughly tested. A big concern is that of a prototype being adopted without going through proper testing because "it worked as a prototype so it must be good."

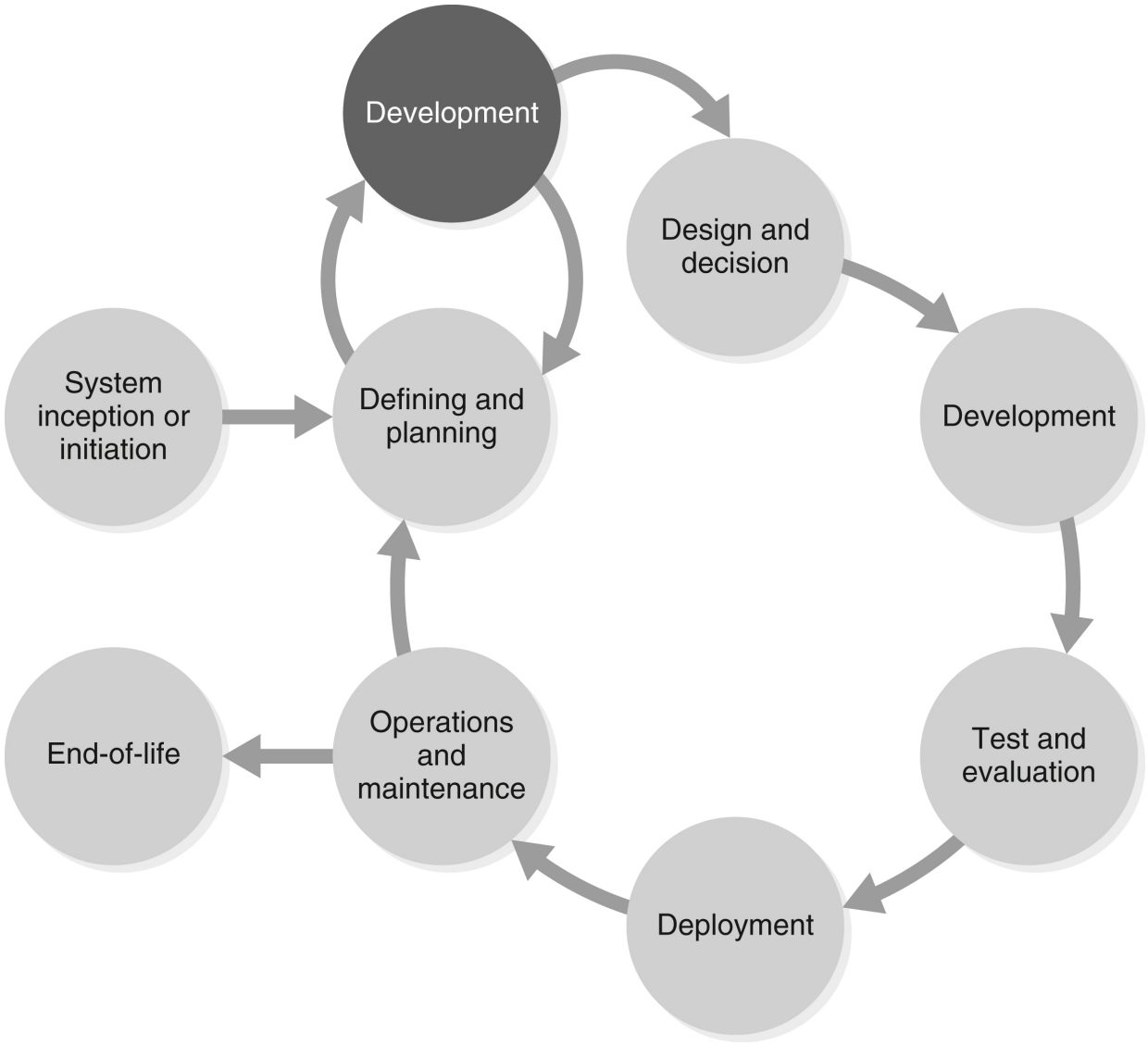


FIGURE 1.6 Prototyping Development Model

Description

Regardless of the seeming complexity of a software development project, the basic phases are always performed, informally or formally. When a model is used as a template, along with a set of rules, for how a software system life cycle is completed and managed rather than as a tool to understand the phases and guide the consideration of what needs to be accomplished in that phase, it becomes a methodology. For example, a development plan with a single linear pass through the model with formal documentation and hand-over at each stage becomes the classic waterfall methodology.

Some **development methodologies** focus on different parts of the model with attempts to parallelize, minimize, or mix the phases in different portions and focuses. Sometimes parts of the model are left off. For example, many variants of Extreme or Agile programming leave out a formal detailed design phase with the collection of requirements left to the development phase, and the high-level design or architecture emerging over time. This can work well when there is already a basic architecture either in the mind of the developers or established as an organizational standard and the problem fits the architectural model. However, when there is no fit, the methodology falls apart. These modified models and methodologies are useful in that they better reflect what is being done in a software system life cycle; however, they too can be limiting if they are rigorously adhered to.

Some of this can be overcome using design patterns, where the developers have a toolbox of known design patterns with which they are familiar. As a problem appears, the pattern with the best fit is applied to the problem and provides a basis for the system design. Provided the problem is understood sufficiently and the selected pattern is appropriate, the pattern could be modified to accommodate those elements that were not a fit, and the approach works well.

The Five Ps

The reader will notice that during the description of the software systems life cycle, we worked very hard to avoid using the terms **process**, **program**, **project**, **product**, and **portfolio**. But now, with a basic understanding of the software systems life cycle it's time to start looking at those terms and where they apply throughout the life cycle.

The first two terms to consider are portfolio and product. A portfolio is a group of software systems that provide a range of capabilities in support of an organization and are managed as a comprehensive set. The key is being managed as a set. If a company simply has a collection of different tools where each is managed independently, where capabilities overlap, where no one has a comprehensive understanding of the capabilities of the systems, and where new functions inevitably results in new systems, without regard to how they integrate with existing systems, that is not a portfolio.

On the other hand, when proper portfolio management is applied, an organization may have an equally broad assemblage of systems; however, each has a specific purpose, the organization understands how the different systems relate to each other, and each function of the organization is supported in the most appropriate system for that function in that organization. Any new functionality needed by the organization is analyzed in terms of the existing systems and assigned to be implemented in the most appropriate place. There may still be overlapping functionality in different systems, but those functions are harmonized to not work at cross purposes. This type of portfolio management is tightly related to information management, and readers who are not familiar with the concepts of information management are encouraged to study it closely. In fact, as we discuss portfolio management in depth in the next chapter, we will inevitably discuss many information management concepts as they are inextricably linked.

Portfolio management is focused on the organization as a consumer or user of software systems. Product management on the other hand is focused on the organization as a supplier or implementer of software systems. The two are the different sides of the same coin. Many times, suppliers and implementers of software systems manage a portfolio of products with overlapping or related capabilities that are focused on a broad variety of customer needs. Product management involves the management of these capabilities in these products, but it also involves the management of each individual product. This includes

making decisions about what to build and sell, how to build and sell, the product life cycle and the associated software system life cycle, and others of the traditional marketing considerations such as pricing, placement, promotion: the 4 Ps, 5 Ps, or 6 Ps, depending on the marketing textbook you read.

Product management is in a book about secure software development because it is critical to understand product management in order to understand the security of software systems. This is a principle that is often overlooked, and as most of the portfolio managed by organizations comes from commercial off-the-shelf purchases or in an integration of off-the-shelf systems, ignoring product management can put the security of any system in peril. In addition, many of the readers of this text are going to find themselves producing a product and need a depth of understanding of product management processes.

Everything an organization does is a program, project, or process. Sometimes organizations ignore this or question whether such a bold statement could be true. The real question is not whether they are, but whether these programs, projects, and processes are managed, or is the organization just doing them. It's time for some definitions.

A project is a task or set of tasks with a specific beginning and an end and set upon accomplishing a specific goal or set of goals. Success of a project can be determined by how well that goal, or set of goals, is achieved within a timeline.

For example, projects can be something as simple as the creation of a tic-tac-toe game for play on a desktop, which could take a week or two to design, build, and test. Or a project could be an internet enabled tic-tac-toe mobile game where the user can compete against others or a computer (which cheats), and where the scores are maintained in the cloud, and along with other complexities takes months to construct, much of it due to the significant vulnerabilities that need to be addressed.

Generally, projects require the coordination of multiple people over a period of time. However, single person projects are not unheard of, although some people refer to them as tasks. For others, tasks are only tasks when they are a specific part of a project. Projects also involve the management of resources, including money and materials, and the management of time and schedules.

Many times, organizations will find themselves in a position where they need to simultaneously manage a group of related projects or a large strategic initiative. This calls for the art of **program management**. The projects within a program may be carried out in sequence or simultaneously or as a mixture of both. Despite the close ties between projects of a program, the key is to

maintain effective boundaries between projects such that they become as loosely coupled as possible.

An example of a program may be the updating of a datacenter, a program where there are processes in place for upgrading each of the applications maintained in the datacenter, processes for upgrading the IT infrastructure (computers and networking equipment) of the data center, and processes for updating the physical infrastructure (air conditioning, new floors, replacing ceiling tiles, fire suppression system, etc.). All these projects need some degree of inter-project coordination. For example, what are server and connectivity requirements for the applications and what are the power requirements for the servers? But the coupling between them is such that outside of the coordination points they do not use conflicting resources and are somewhat loosely connected in time.

Another program would be the development of a new aircraft. This is going to require a significant number of projects and even subprograms to complete the program. This type of a program can continue indefinitely.

In the chapter on project and program management, we will explore tools and techniques to enable the successful completion of programs and projects.

Process shows up in many forms: the processes of production, processes of operations, and the processes of support. Processes are tasks that are performed in an organization on a repetitive basis. Processes can be formal or informal. Informal processes tend to be undocumented and less well defined than formal processes. In many organizations, there are the formal documented processes, and then there are the informal processes, those that are not documented but are still performed. Sometimes these informal processes are the ones that are used, even if they differ from the formal documented ones. This is not good.

A procedure, sometimes referred to as a protocol, is a direction of how to accomplish a specific process. Often organizations will have a set of standing (or standard) operating procedures that document how the processes in a company should be performed. This prevents the people from having to figure out how to navigate a process every time it is encountered. Organizations that follow their established documented procedures are said to be more mature than those who either have no standard procedures or pay little attention to using the established procedures. Organizations that can adjust their procedures as circumstances change, while still maintaining standards, and documenting the adjustments are at yet a higher level of maturity. In a later chapter we will examine methods for managing processes.

Typical processes may range from those used by a help desk, where an employee follows a script with each caller to select a predetermined solution to the caller issue, to something much more complex. Companies often have fixed processes for hiring, on-boarding, and firing employees that specify the steps to take, the forms to fill out, the approvals required, and the order in which this all needs to be accomplished. Processes are also used for software development where the process specifies the tasks that are to be taken to develop a program, the guidelines a developer must follow while programming, and the way in which the results of the development are reported and tested.

Combining the Five Ps with the Secure Software Systems Life Cycle

At this point, we can start to put together the five Ps with the software system life cycle. As we can guess, it isn't going to be straight forward and neat, but we will see patterns emerge. The first pattern we can look at is the role of product and portfolio management in the inception phase of the life cycle. The product and portfolio gaps and deficiencies in an organization are very much going to determine whether a new system is needed (second as a driver only to awed executives who just attended a tradeshow and saw a system "we just need to have") or if an existing system needs revision. These gaps and deficiencies, together with an understanding of the capabilities of other systems in the portfolio, will also serve to help guide the boundaries of the new or revised system. The product requirements or portfolio needs will provide the business or functional requirements for a new or revised system to meet those needs and requirements as well as guiding the technical and development environment requirements.

Current technologies and capabilities help to set the requirements for new ones. For example, an organization that has a large base of programmers in a specific language should be hesitant about beginning a development effort in a new language without carefully considering the challenges and expected benefits of the change. Current technology is also going to affect how the new software system needs to interface with the existing systems.

Another portfolio issue will be how information is managed in the organization, including questions about where it is initiated, processed, stored, and distributed. Which system in the portfolio is the "golden" or reference system for each piece of data?

As the new software system is developed, it will need to be tested in its interactions with each existing system as well as for environmental compatibility. Negative interactions between systems can occur in a production environment. Some systems have issues with existing on the same networks, some systems have dilatory effects on the security of others, and some systems are simply resource hogs that starve others.

Product management is going to be deeply concerned about how a system is deployed. Creating a system that requires significant configuration to be done during deployment can mean that a system has a much broader market, as it can be made to match the differing requirements of a variety of customers.

However, that also makes deployments much more expensive and can chase away potential customers. The challenge is in meeting the balance of the product appealing to the largest number of potential customers while not becoming so complex customers are hesitant to purchase the product, or the cost of configuring the product becomes too expensive.

Unwinding process, program, and project management becomes more challenging. By nature, a software system life cycle is a program or project that has a defined beginning and end and requires the management of resources. Complicating this, however, is that the overall life cycle may be so large as to make some of the tools of program and project management seem irrelevant at that scale. For organizations that regularly develop software systems, this becomes a process. However, the process launches and guides the programs and projects that manage the development. Those projects may in turn make use of processes to accomplish tasks in the development. This is demonstrated in **Figure 1.7**.

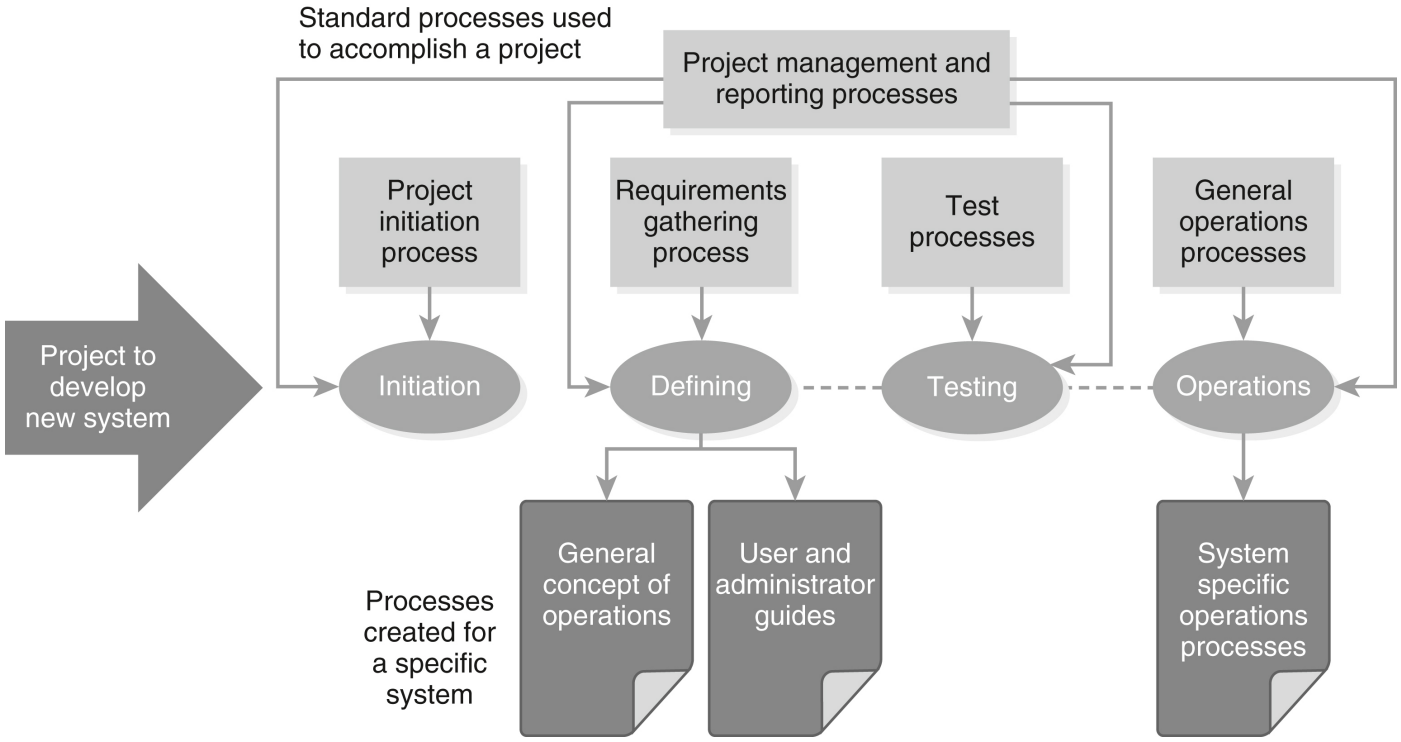


FIGURE 1.7 Project and Processes

Description

Typically, organizations will establish processes for initiating and managing programs and projects. These processes will establish the tools and procedures used, as well as defining established processes to be used during the different phases. For a large development effort, a program effort is

launched, which then has projects launched to accomplish different elements of each phase. The projects to be launched may be determined by the organization's processes, or they may be specific to the needs of the specific program. This is shown graphically in **Figure 1.8**.

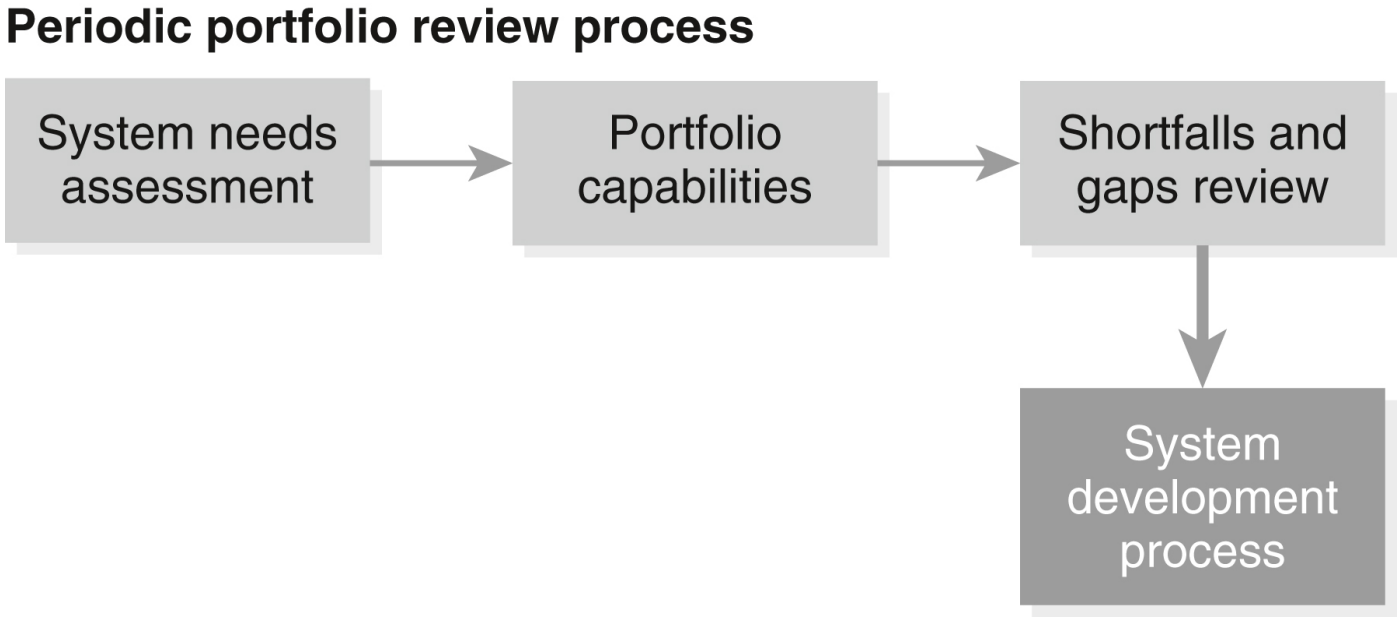
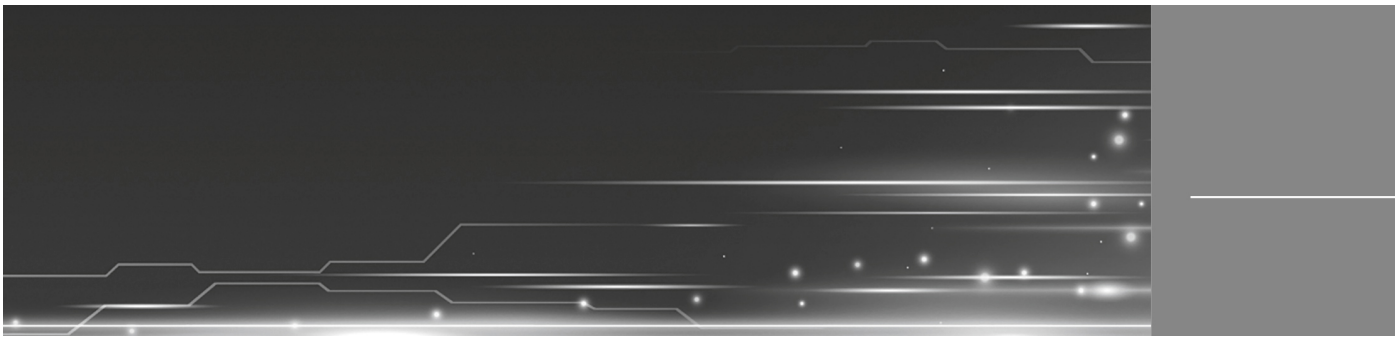


FIGURE 1.8 Periodic Portfolio Review Process

Description

KEY TERMS

defining and planning
deployment
design and decision
development
development methodologies
initiation
operations and maintenance
portfolio
process
product
program
program management
project
retirement
test and evaluation
use model



© d1sk/Shutterstock

CHAPTER 2

Product and Portfolio Management

CHAPTER OBJECTIVES

- Understand how product and portfolio management influence the design and function of software systems.
- Identify how market forces influence the features, budgets, and choices for software systems.
- Trace a typical product life cycle from inception to retirement.
- Describe how a portfolio of software systems is managed to support varied needs.

Introduction

The goal of **product management** is to offer to targeted customers software systems with the right capabilities at the right time to maximize profitability. The goal of **portfolio management** is for an organization to have a set of software systems (versus individual, minimally related systems) with the right capabilities at the right times, orchestrated in a manner to maximize the organization's ability to meet their goals. Sometimes software systems don't align well with organizational goals, creating challenges, conflicts, and a portfolio nightmare. Misalignment can occur when software systems are created by vendors (teams outside the organization) because vendors often are not as familiar with the organization's culture and needs as internal teams are. Such misalignment can cause security as well as other types of functional gaps. An organization that creates its own software systems has greater control over the software system portfolio details and continuity over time.

In this chapter we will look first at product management and then at portfolio management. We will examine how these areas affect security and what can be done as a part of product and portfolio management to enhance security. Attention to security from these perspectives is the most effective way to begin a solid foundation for security. A failure in these areas will result in a never-ending cycle of plugging hole after hole after hole.

Product Management

The product manager, and product management, has the responsibility to shepherd a product from inception, through creation and sales, into the customers' hands, and ultimately into retirement. Product management is at its core a marketing job even though the product happens to be software systems. Yet it is often performed by engineers, either because they have strong technical knowledge to contribute or often because marketing fails to fill the role. Even when done by marketing, engineering is going to have a strong degree of influence over how a software systems product is managed because of the degree of technical demands involved. In some organizations, people rotate back and forth between technical and product management roles. Unfortunately, most people who find themselves in this role have limited knowledge or experience with product management. They end up learning as they go or assuming that the role is something other than what it should be.

Many people associate marketing with creating brochures, advertisements, and fancy graphics. This is only a small subset of marketing called marketing communications. Marketing overall is much broader and covers the 4 Ps or 5 Ps or 6 Ps (depending on the book you read—even saw one with 7 Ps), including price, product, positioning, and promotion, and people, and place (as seen in **Figure 2.1**), and … we highly encourage everyone to take a course in marketing or to read several books on the subject. Product management is concerned about all the Ps, with a focus on the product.

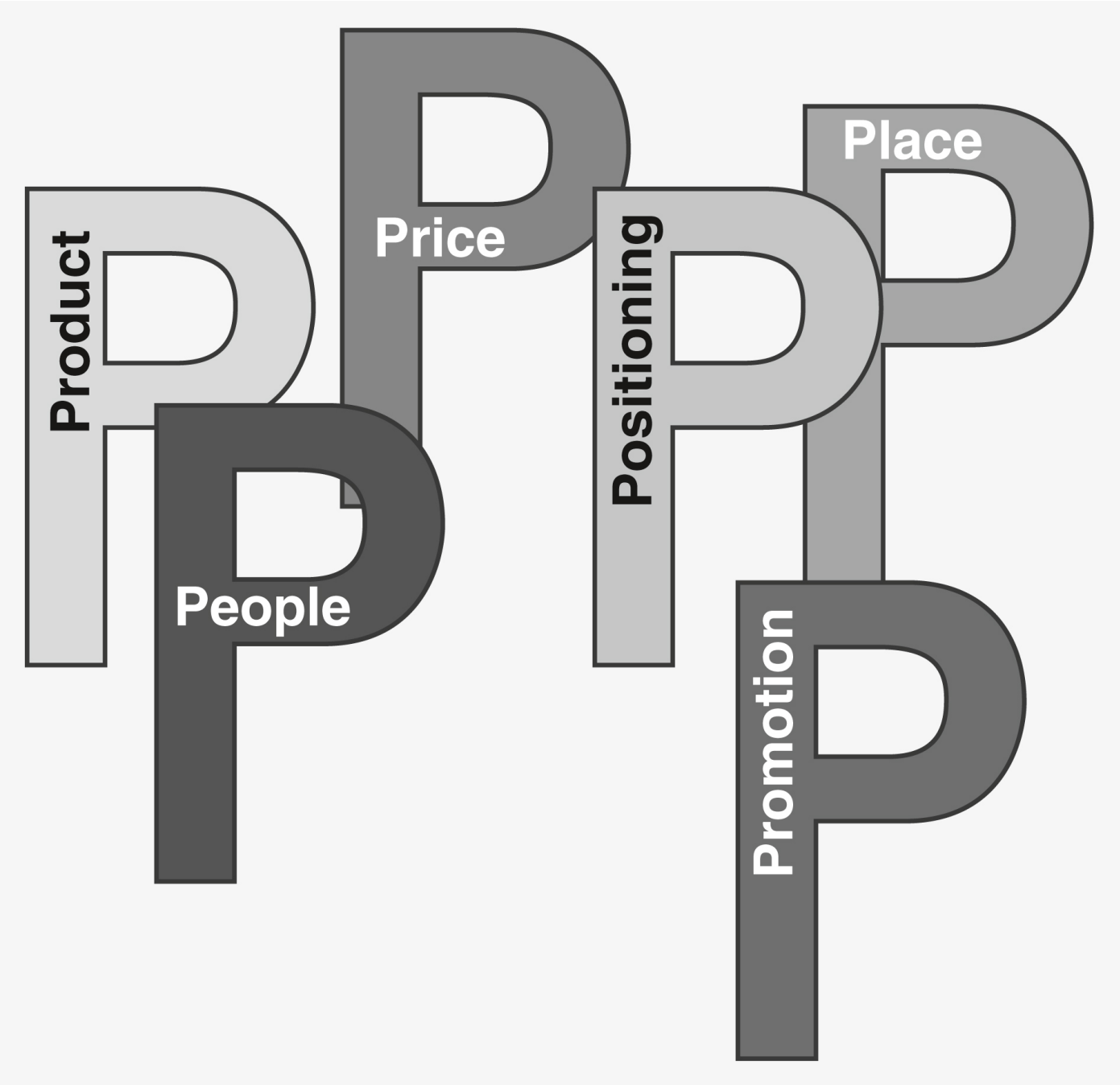


FIGURE 2.1 The Ps of Marketing

Description

The first job of anyone in product management is to understand the market. The market is not just the place where you will sell your product; it is everywhere you could sell your product and everything your product will interact with. A good tool for understanding the market for a product or family of products is Porter's **five forces model** as shown in **Figure 2.2**. Porter created his model as a tool to analyze a company's competitiveness and to help companies develop strategies to enable them to compete.

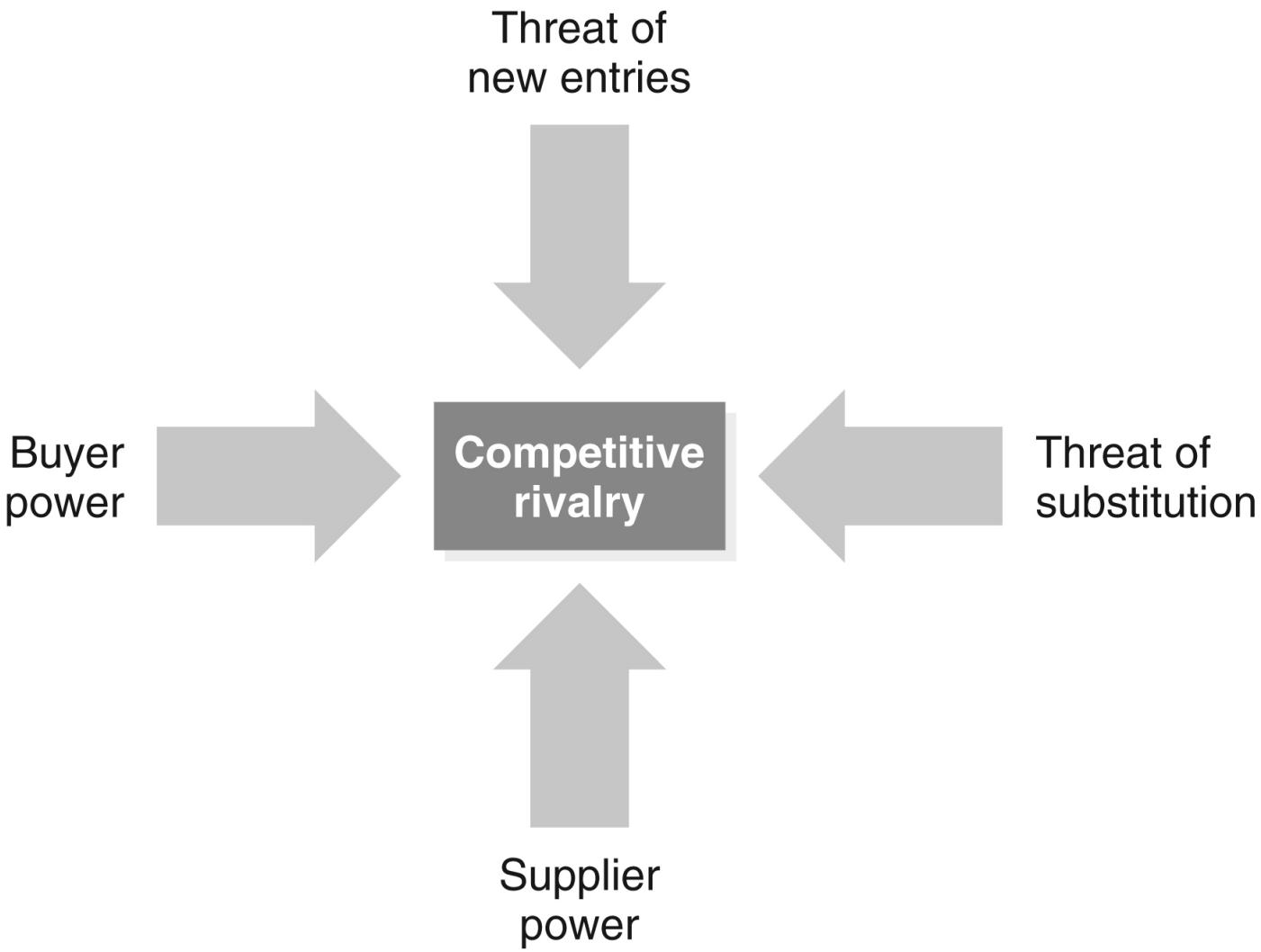


FIGURE 2.2 Porter's Five Forces Model

Description

At the center of Porter's model is the first force: Competitive Rivalry. This is an analysis of the existing rivals in the industry. How many are there? What role in the market does each play? How do the competitors differentiate themselves? If a market is full of undifferentiated competitors, it will be difficult for any one of them to gain market share over the others. Therefore, the competitors are likely to try to gain an advantage through pricing, features, or other key attributes. On the other hand, a market with few competitors has room for growth.

The next force is the Threat of New Entry. This force asks the question of how challenging is it going to be for a new competitor to enter the market? The factors that affect this are barriers to entry, such as development costs, intellectual property issues, and high investment needs. If barriers to entry are low, then a market may not be as attractive as any decent profits in the market

will quickly attract new entrants. With the proliferation of software systems, cloud technologies, and rapid development cycles, many previous barriers to entry have fallen away.

The third force is the Threat of Substitution. This is a competitive force that has been frequently seen in the era of internet business models and disruptive technologies. A product doesn't need to have a direct rival, nor does a new competitor need to bring an equivalent product to market. With substitution, anything that a potential buyer can use instead of the product represents a threat to the original product. Sometimes the threat of substitution was not a serious issue until a company made it so—for example by pricing their product to where it became cheaper to simply complete the process by hand or by pricing labor to where it was cheaper to buy robots than hire people. Other threats of substitution come from unexpected new uses for different products or even from products that were never intended to compete. An example of the latter is the story of what happened to the payphone. These were the magic piggy banks of the telco industry. They would install them out all over the place and simply collect dime after dime, then quarter after quarter. The phone booth providers had no intention of ever getting out of such a lucrative business, but they did want to get into the business of selling mobile phones, and sure enough as soon as the average person had a cell phone, people stopped using payphones. Now it's difficult to find one. The mobile phone hasn't only put the payphone out of business, it has also become a substitute for many uses of computers, for watches, for phonebooks, for maps, and for many other products.

The fourth force is Supplier Power. This force is concerned with the degree of influence that suppliers have on the product and the ability of the supplier to shift value from the product itself to the supplier. Supplier power comes from being locked into a supplier and not having alternatives. In software systems, the place where this is most often evident is with software libraries and toolsets where an organization develops its product using specific, often proprietary, tools or libraries. Changes to these can have significant effects into the product. In some cases, a vendor can reach through the software system directly to a customer and over time convince them that it was the toolset, not the product, that gave them what they wanted, allowing the supplier to walk off with the customer. Another new point of vendor lock in is with cloud services. It is easy to build a new product in the cloud or put data into a cloud service. Moving data out of a cloud provider can be a very different story.

The fifth force is Buyer Power. This represents the power that the buyer has to force changes in the product. This could be done by buyers refusing to use a

product or cutting back on buying the product, such as the American Colonists did with tea just prior to the revolution. Another factor that could give more power to buyers is a market where there are only a limited number of buyers, or even one large dominant one. This allows the buyers to dictate the terms, features, and other factors of the product. With software, as with other products, this influence could have a negative effect on the attractiveness of the software to other customers. An example of this would be software that is tailored to best fit a specific customer's processes. Buyers have less power when there are more of them in the marketplace and have more power when buyers are able to adjust the timing of their purchases.

Because this is a book about secure software systems, we are going to add a sixth force and develop a "security-enhanced five (six) forces model" (as is shown in **Figure 2.3**). This sixth force, as you may have guessed, is Threats to Cybersecurity. It will include looking at how attractive a product is to threats as well as complexity, operating space, and other factors that expose a software system to cybersecurity threats. The ability to operate securely within a specific threat environment increases the competitiveness of a product. The more a product is going to be exposed to a generalized threat environment, the more important proper security is in determining the competitiveness of any product. In general, the more secure a software product is, the more likely it will be desired over similar, less secure products.

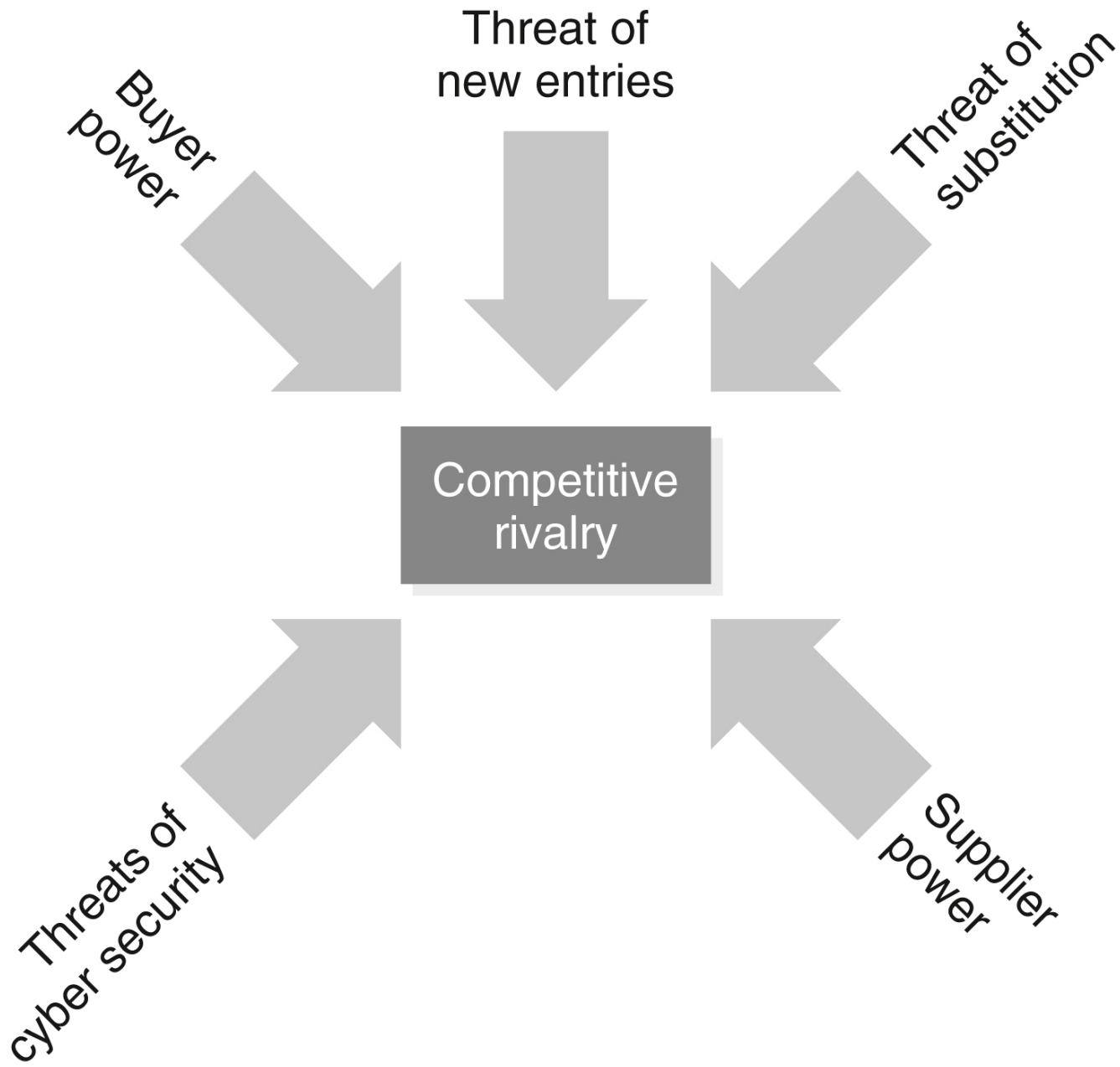


FIGURE 2.3 Security Enhanced Five (Six) Forces Model

Description

A thorough understanding of the market requires not just a forces analysis of our own product, or proposed product, but also the completion of a **competitive analysis**. Competitive analysis is used to evaluate how our product stands in relation to all competitive products in the marketplace. This competitive analysis can be done by applying the lens of the forces model to each of competitors—not just the significant competitors, but all the possible competitors. After all, a competitor who wasn't deemed significant today may dominate a market in a short time, especially if it is underestimated and

ignored. The world of business is filled with unanticipated underdog stories. Of course, none of these stories were unanticipated by the underdogs themselves.

Identifying Opportunities

A natural outcome of a thorough competitive analysis is that **market opportunities** begin to emerge. Market opportunities appear where there are gaps in the offerings in the markets, when there are imbalances in Porter's forces, or, as is often the case, a close look reveals a great opportunity for creating a new market or for introducing an effective substitute for a product.

The competitive analysis will also serve as a base for determining the **market strategy** to adopt for a product. The market strategy is the concept of how we are going to position our product in the market compared to all other competitive products. Strategies that we can choose from include being the low-cost provider, being the high-quality provider, being the most secure product provider, occupying a specific niche, or others. The strategy is guided by the competitive analysis and will have a major effect on the shape of a product. A low-cost product, for example, is likely going to be a low-feature product. A highly secured product will limit choices in how a product is designed. A high-quality strategy will dictate a different focus in design and test and is likely to have a high number of features.

Too often, organizations fail to identify what the **product strategy** will be for a product and as a result end up developing a product that is not appropriate for the intended strategy with disastrous results. For example, a company that wishes to pursue a low-cost strategy but doesn't convey that strategy to the system architects may end up with a system that fails to meet its targets for development or deployment costs. Rather than creating a "cheap and cheerful" implementation of a traffic light controller that can only handle a single four-sided light at an intersection where two small roads meet, the designer may create a fully programmable multilane controller that also handles walk-signs, turn-lanes, and networked signal capabilities. The full-function controller will more than do the job but requires a larger microprocessor and additional electronics, which makes the controller far too expensive for small towns, such as Plentywood, Montana, Monduli, Tanzania, or Portobelo, Panama. On the other hand, if a developer is unaware that a company is pursuing a quality leader strategy, they may put forth a "good enough" effort in an area where it later negatively affects the company's reputation. Ensuring all members of a team are aware of and understand the product strategy is critical for success. Communication and transparency across an organization avoids such issues.

With the opportunities and strategy identified, the company can then start to build a **business case** for a product. These business cases are begun as short documents that describe the market, the opportunity, and the strategy. They then go explore the financial basis for pursuing a product strategy. This analysis includes laying out any assumptions, the costs of the opportunity, and the expected revenues. The end goal of the business case is not the case itself, but rather the ongoing business case refinement with the eventual result of a decision being made to produce or not to produce a product.

In some cases, the outcome is not a single product but rather a family or a portfolio of products, each of which are designed to target a portion of the market opportunity. In developing the portfolio, the company pays attention to fitting each portion of the portfolio to a specific market opportunity and strategy. At the same time, the company needs to consider how the products can be built in a complementary manner.

For example, a company may decide that it is advantageous to go after both a low-cost leader and a high-quality leader strategy at the same time. Rather than developing two separate products, however, the company could use the same basic product, while building in the option to turn off higher-cost features in the low-cost version to match both markets. Because both versions of the product are produced in one product, there is a single development cost and many other efficiencies of scale. As a result, the additional revenue from the low-cost strategy can pay for additional features in the high-quality version without pushing the price as high as it otherwise would be if there needed to be separate development and production efforts for each product. At the same time, having the revenue from the high-quality version can help fund the low-cost version versus developing a separate low-cost version. Here, as with all products, the strategy decision is going to have a significant effect on how the product is developed.

In this example, the software system developed would need to have the ability to activate and deactivate specific features without negatively affecting the core of the product. The system would also likely need to have a capacity to be “themed” (options for different looks and options that users will see) in different manners to allow a distinction between the low-cost and high-quality strategy versions. Failing to do so may result in the approaches being unsuccessful either because customers may not recognize the high-quality strategy version as being effectively different or because the customers may shy away from the low-cost strategy version because it looks too expensive. Much of the perception of differences are passed off as “mere marketing,” but they have significant effects, and readers are urged to delve deeper into the

impacts of such considerations. The key point is that the whole team needs to understand the strategy so that it can be well supported in the product.

BOX 2.1 Methods to Differentiate Basic and Advanced Versions of Products

A guide for marketers who want their cake and to eat it too!

- **Licensing**—Use licensing to enable different features in software. Get the "crippleware" version and then use a license to enable the product.
- **Jumpers/switches**—Install a jumper or switch in the system that slows the system clock. In the early days, IBM charged customers a significant amount of money to do an "upgrade" where the technician removed the jumper.
- **Software updates**—Ship the hardware with different software for basic and advanced versions.
- **Lasers**—Use lasers to disable/cut out advanced technology, such as a coprocessor on a computer chip. The 386SX was a version of the 80396 chip where the math coprocessor was disabled.
- **Leave off connections**—The features are on the circuit; there are just no wires to enable them. Almost all alarm clocks sold use one of two different chips. Both chips have all of the features enabled on them, but simple clocks leave off pin connections to make them work.

As the business case is refined, attention needs to be paid to developing the **product vision**. The product vision is a story about how the product is going to fit into the market. In the vision, the case will not only describe the product itself, but also how the product is intended to be used, who would use it, and how it differs from other products. As a significant part of the vision, we need to look not just how the product will be used, but also how it will be misused. Taking an early look at cybersecurity and other potential areas of malicious or unintended use affords an opportunity to preclude many of these potential vulnerabilities from appearing even before a product is designed.

Determining who the customers will be for a product begins by looking at the overall market for the product. That market then needs to be segmented into groups of potential customers with different characteristics and needs. There

are some classical market segments that divide retail technology consumers into segments such as innovators, early adopters, mainstream, and laggards. These divisions also go to great lengths to describe the characteristics of each segment. That is the kind of segmentation that needs to be done, but in more specific terms, related to the specific product. Together with the strategy and segments, we can begin to identify actual potential customers and what attracts them.

Positioning and differentiation of the product as a part of the vision began with the product strategy. Beyond the basic strategy decision, positioning and differentiation go into depth on how that strategy is going to be implemented. These are the tactical elements to implement the strategy. With product positioning, the company determines how to position the product as meeting each target market segment's needs. Through differentiation, the company identifies meaningful ways for customers to identify differences with competing products. The goal is to generate a product differentiation that exceeds other similar products

Product Definition

With the vision, strategy, positioning, and differentiation determined, the product manager can begin the task of creating the **product definition**. This includes determining the functional features and capabilities of the product, user interface and experience (UX/UI) for the product and its target markets as well as consumer product benefits. Along with this information, the product manager defines an initial level of security and privacy requirements. These requirements will be further enhanced by other members of the team throughout the project or may also be pre-determined, at least in part, by the regulatory or other legal requirements.

However, these are just the beginnings of the product definition, what are commonly called market requirements. The product definition needs to reach beyond these into how a product is produced, sold, delivered, and supported.

Sourcing is concerned with how and where a product is produced. A product can be developed and manufactured/produced in-house, development and manufacturing/production can be outsourced, or there can be some combination of the two. In the simplest terms, sourcing is going to determine the cost of producing the product. Sourcing also has significant effects on many other factors, including time of delivery, product quality, transportation, and other costs.

Sourcing is also a significant concern for security, especially with software systems. If a system is developed in-house, or within the organization, the organization has the opportunity to significantly influence the approach to security. Not all systems, components, libraries, and software development tools are created in-house. In fact, most are not. Those are known as externally sourced. Externally sourced systems and components include those which are bought off-the-shelf, as Original Equipment Manufacturer (OEM) components that are integrated into a product, or custom developed software. Systems that are sourced externally can potentially add significant security risk, either through poor security practices on the part of the sourced company, through misunderstandings between the hiring organization and the sourced company, or through the deliberate introduction of backdoors and other security holes. Extra care needs to be taken with outsourced work.

Another factor that needs to be defined for the product is how it will be delivered. With the maturation of the internet and cloud services, there is an ever-increasing variety of delivery means. Software systems can be delivered as packaged physical or electronic media that the customer installs or as a delivered system including hardware components. The installed software system can be a completed, ready to use system, where all a customer needs to do it to start it up and enter data, or the installed system could be delivered as a platform, where either the customer, the company, or a third-party tailors the application to precisely fit the customer needs.

A software system can also be delivered over the cloud, where the customer is buying **software-as-a-service (SaaS)**. The software is owned and managed by either the company or a third party, and the customer pays for the use of the software system rather than the system itself. Another cloud alternative would be to have a cloud platform strategy; rather than provide the complete service to customers, provide them with a set of cloud-based functions that customers can stitch together into whatever service they need.

For the company, there are several considerations in choosing which delivery strategy, or combination of strategies to use. Naturally, one of the big ones is money; the different strategies will have effects not just on the costs of development, but also on cash and revenue flows. Software system development can be expensive, and some companies are concerned about recouping costs as quickly as possible. For these companies, a delivery model that transfers ownership and recognizes revenues quickly is preferable. Other companies like to focus on the continuing revenue stream that SaaS can provide. A **software-as-a-platform** (SaaP) strategy is where the software provided doesn't perform services in and of itself but rather provides a platform

to enable the rapid development of service. The platform can be provided via media (physical copy or downloaded) or as services from the cloud. This model can also generate substantial revenue from configuration consulting, often many times the cost of the actual software.

Companies also need to consider ongoing support costs in determining their delivery strategy. One-time sales will still require software support for fixing bugs, answering customer questions, and other support. This can be provided for a fee, as a part of the original purchase, or in some combination. Ongoing upgrades and support revenue for business class systems can vary from 10 percent to 40 percent of the initial software costs.

When using an SaaS strategy, the developing company needs to support the hardware and software that customers will use. These costs are substantially higher than supporting development and direct sale of a product to a customer. However, amortizing these increased costs across multiple clients makes the marginal costs per customer significantly less than what the customers would need to provide their own support and provides a regular revenue stream for the developing company.

The choice of delivery method is going to have a substantial effect on how a software system is developed. The delivery of an application where a copy of the application will be provided to a client requires development that is much more functionally complete than the delivery of a platform. In a platform, details, such as specific rates, routing of processes, and formats of reports, can be left to be determined during the configuration process. This configuration capability needs to be built into the platform. As well, SaaS platforms need to be built in a manner that allows a single system to handle multiple customers rather than relying on one instance of the system for each customer. While some software systems have been written requiring one instance per customer, it is not as efficient nor as good financially as having one instance with many different customers.

It quickly becomes evident that the choice of delivery model is also going to have significant implications for system security as well. In a completed application model, security can be baked into each functional process, calculation, and input. That contrasts with a platform approach where the exact routing, inputs, and calculations are not known in advance. As the capabilities for configuration are developed, a parallel effort needs to be undertaken to ensure that security is also identified as a mandatory part of the configuration.

For cloud delivery strategies, one of the big security concerns is going to be the ability to compartmentalize data and business rules from different customers within the same software system. Building a stand-alone system

that only needs to handle the data of a single organization has fewer complications than a system that handles data for tens, hundreds, or thousands of different customers. This is further complicated by the need to manage such a system across customer domains.

Another area of consideration related to product delivery and security is customer expectation management. Naturally, one issue involved is communicating to customers exactly what the delivered capabilities will be. Also important is to communicate to the customer the environment in which those capabilities must exist, either to provide effective usage or to maintain security.

Product definition also includes determining the sales and distribution strategy. Typical strategies can include direct sales, sales through a distributor model, or the use of other channels to market a product. This gets tied back again to the service strategy and is concerned with not only how the product is maintained and updated, as noted before, but also who and where the responsibility for the servicing is managed. Introducing a third party into a service strategy can influence who has access to different capabilities and configuration elements of a system, which in turn influences security. Some issues, such as managing the distribution of encryption keys, can become quite involved. Systems that are not designed to be distributed in a manner that matches the defined product distribution causes complications and security holes to develop.

The final, though not insignificant, elements of the product definition are the product pricing and contribution to revenues and margin. The pricing, and how that contributes to revenues and margin, is tightly coupled to the delivery model, but other significant factors also need to be considered. One factor that naturally influences the price is consideration of product positioning. Higher prices can have the effect of making customers perceive a higher quality for a product. At the same time, higher prices may lead to lower demand, unless the lower price leads customers to believe the product has lower quality and avoid it. Pricing with only consideration of costs could lead to a product either so expensive that no one will buy it, or a situation where the pricing is so low that massive amounts of potential revenue is left on the table. Pricing also sets a perception about the value of a product.

With a pricing model established, the contributions to revenue can be derived by forecasting anticipated sales. The contribution to the profit margin can then be calculated based on the development, sales, support, and other costs. While it is generally good to have a product with a positive contribution to

revenue, there are times when a product contributes to a company's success in other ways.

Product Planning

Product planning is a responsibility of the product manager. Often this begins with the **product roadmap**. This is a representation that traces the planned evolution of the product over time. The roadmap often involves complementary products as well to ensure their releases are well timed and coordinated. In a roadmap, the product manager will cover the different product or family of products versions, including when new version releases will happen and what the key features will be at each release.

The roadmap needs to cover the entire product life cycle, from inception through introduction, maturity, and into retirement. Products at different stages in their life cycles need different approaches to their management. One useful tool to help determine the appropriate steps for a roadmap is the Boston Consulting Group matrix where the product's **market share** and the market's growth rate are plotted into four quadrants as shown in **Figure 2.4**. The market share for a product is the percentage of sales of all competitive products in the market that represents our product. For example, if our product is the only one available, we would have a 100 percent market share. If our product has equal sales with three other competitors, it would have a 25 percent market share.

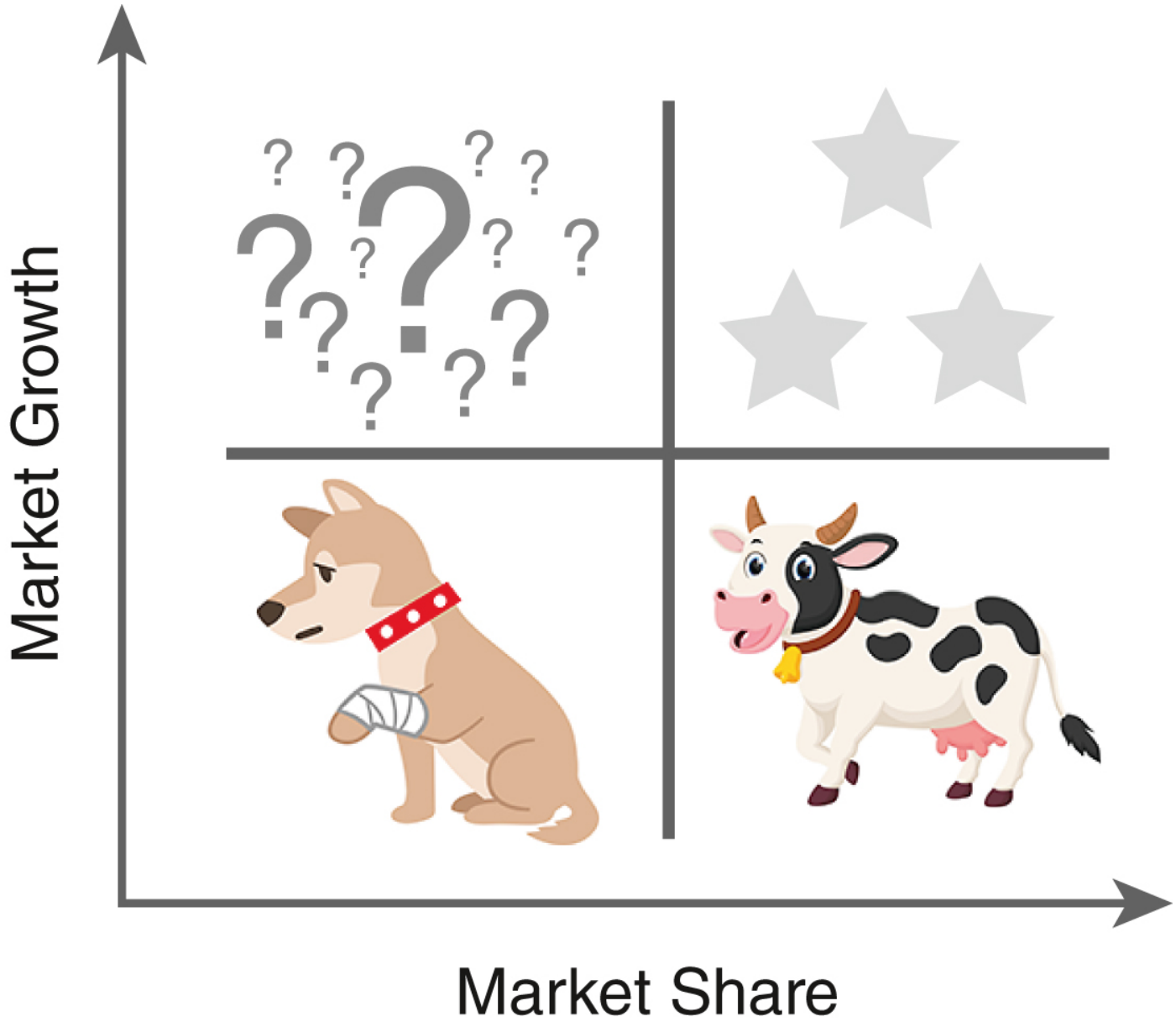


FIGURE 2.4 Product Quadrants

© ankomando/Shutterstock
© lululia/Shutterstock

Description

In the lower right quadrant are products with a high market share in a low growth-potential market. These are considered to be “cash cows” and can be managed successfully with little effort. Products in the lower left quadrant—low share, low growth—are called “dogs.” Without a concerted effort to change things, either by moving into a different market or increasing market share, these products are not going to become successful. Without some compelling

reason to maintain them, these products need to move through the end of life cycle process.

Products with a high market share in a growing market are termed “stars.” Product management for these focuses on maintaining the market share and avoiding risk. The roadmap will be active, but generally not aggressive.

The final quadrant contains the question marks. With a high growth rate in the market, there is room for turning the product into a success. The roadmap for these products is going to be dynamic and aggressive.

The roadmap is important for planning the security of a product. Not only do key security features need to be included in the plan, but a roadmap also allows planners to anticipate future security vulnerabilities. Knowing where the product is going provides an opportunity to design in support for future capabilities as well as for the security features needed to support those capabilities.

Driven by the roadmap, the planning process addresses the specific planning for each release. This release planning involves detailing the processes and steps needed to develop and complete a specific release. As a part of the release planning, the key features from the roadmap are supplemented with other needs identified by the product manager to begin the process of product requirements engineering. These become the exchange point between product marketing and the software system development life cycle requirements (SSDLC) engineering stage.

The specific way that security concerns are addressed is also going to be significantly influenced by the quadrant in which we find a product. As security concerns are discovered, they need to be reviewed in the context of the product quadrant positioning. A security concern in a product that has been classified as a dog is not going to be addressed in the same manner as a star or a question mark. A security concern in a dog product may not be addressed at all, or it may be addressed by simply terminating the product.

Likewise, security concerns in cash cows are likely to only obtain the minimum attention needed to prevent the cash cow from sliding into becoming a dog. Adding security is not going to do anything to increase sales because they are already what they can be in a bounded market. So, any money spent on security simply reduces the ability of the product to generate revenue. If in fact, the cost of adding needed security is high and the potential liabilities if the security is not added are too great, the organization may simply jettison the product. As an illustration, consider a product that is earning $1 million per year in profit but has saturated the market and is starting to slowly decline. If there is a security flaw that will cost $5 million to fix, it would take more than 5 years

to recover the investment (depending on the rate of decline). The company could either invest in the security fix, or it could invest the $5 million in a new product that over time would earn more than $1 million per year. In the meantime, it could continue to collect the $1 million per year from the insecure product. Given the increasing threats to products, this attitude toward system security has been changing.

Evaluating the difference in the treatment of security concerns in question marks and stars is going to be more complex. While addressing a concern properly may be enough to push a question mark into a star, it may also be that addressing the concern properly edges the product toward the bottom of the market or toward noncompetitiveness. Stars also have concerns about whether the return is going to be worthwhile to invest in mitigating a concern if the cost of mitigation is high and the lack of mitigation, or a merely adequate mitigation, is not going to have an effect on market share. One would like to believe, as a security professional, that if a security concern is not mitigated, then the product will fall out of favor and out of use, but reality is that in most cases security issues are not going to cause a product to fall out of use, especially if the creator of the product can push the customers to accepting responsibility/liability for security.

The product manager will maintain a close involvement throughout the SSDLC, maintaining the perspective that this is a product being built, not just a software system. Without this perspective and pull, it is common that decisions are made that are appropriate for a system, but not for a product. The designers and developers would fix every issue and mitigate every vulnerability. They would also add any useful features the users may need. But from a product perspective, this may not be cost effective, or it may position the product somewhere other than is intended. For some engineers and developers, this is a hard perspective to understand.

Bringing the Product to Market

Bringing the product to market consists of those activities that many mistakenly think compose the entirety of **marketing**. This portion consists of product placement and promotion. That is getting the product seen by potential customers in the context in which the product is attractive for them to purchase. Much of this is accomplished by advertising or other marketing communications. This area also is concerned with branding or brand management. Generally, within an organization, this specialty is handled by a

specific group, and the responsibility of the product manager is to coordinate the efforts for the product.

From a security perspective, there is value in promoting the security benefits of a product. In some cases, a degree of "security theater" can actually help deter would-be threats so that would-be attackers perceive the product as being more difficult to attack than another product. It is important to back up that theater with actual controls because otherwise the theater may backfire and generate inordinate attention to a security issue. Extensive and unrealistic claims about how secure a system is can result in a "hold my beer" moment from a malicious actor who feels invited and compelled to demonstrate just how silly the claim was.

BOX 2.2 Security Theater

Remember the story about the hikers who saw a bear? The first hiker stopped and started changing to his running shoes. The second said, "You'll never outrun the bear." To which the first replied, "I don't have to. I only have to outrun you."

Sometimes in security looking like the toughest system is enough to encourage the bad guys to go looking elsewhere for easier pickings. Easy steps, like message-of-the-day security notices, naming conventions, obscurity, posted notices, screening tags on emails, nametags on corporate guests, and other simple steps that "don't really make you more secure" actually do, not always because of the actions themselves, but from the image they portray.

Ecosystem Management

A product ecosystem involves supply chain for a product, product users, the delivery chain, and often a host of third parties that create new businesses, products, and opportunities around a product. The third parties may include consultants who install, customize, integrate, and support a product. They could also include organizations that have built businesses reselling information derived using the product.

One product manager task is **ecosystem management**. That is, to see that the ecosystem grows and develops in the most effective manner for the benefit of the product, and that the diverse needs of the different participants in the ecosystem are met and balanced. Sometimes that will involve nurturing parts of

the ecosystem. At other times, it may require a bit of selective pruning and adjustment.

Often, the creators of a product did not envision the kinds of services and opportunities that these third parties have discovered. In some cases, that has made product creators hostile to the third parties. However, the existence of these third parties is often a sign that a product is suffering from a degree of success, and either there are customers who use the product despite the issues that the third parties overcome, or the product has enabled other successes that in turn encourage more product sales and ultimately result in a fortuitous spiral. The product manager needs to sort apart these types and evolve the product to eliminate the need for selected third parties while supporting those that help achieve the product creator's vision.

The key to ecosystem management is to take a broader picture of the needs and motivations of the ecosystem members. That allows a balanced and productive management approach.

Portfolio Management

The core concept behind portfolio management is mapping needed capabilities to software systems that support those capabilities and actively managing how those capabilities are provided. The historic norm has been for companies to acquire a variety of IT systems in a haphazard manner. As a company discovered it needed a new capability; the company went out and purchased a new IT system or created a new system that could provide that capability. The alternative became to buy a massive system that in theory provided all the capabilities the company would ever need but in practice never did.

A portfolio management approach is different in that it recognizes that there will never be one system that meets all needs and that the needs will change over time. Portfolio management works to ensure that the company's needs are all met through a series of evolving systems that converge toward more effective interoperation. Ideally, active portfolio management will eventually support all needed capabilities through the most efficient collection of systems.

Within that approach, the following are goals of portfolio management:

- The right systems
- The right data
- The right capabilities
- Fewer systems
- Remove redundancies
- Easier integration and interaction
- Flexibility and growth

Which systems are the right systems is determined by the business needs of the organization, which are constantly evolving. These are the systems that are compatible with the **business strategy**, which enable it, and which contribute to business success without becoming a financial burden. The business strategy for a business is a description of how the company intends to conduct its business, the goals of the organization, and how those will be achieved. From a business perspective, they must contain, protect, and make available the data the organization needs to thrive. The right systems provide the

capabilities of manipulating the data, communicating, and processing to match the business strategy. They provide for future capabilities without sacrificing the current health of the organization.

Fewer systems mean the business is able to focus on business rather than which system to use or supporting the resources needed to keep many systems running. This can be compared to restaurant menus. Consider two alternatives, one where the menu has a large selection of entrees and other items. Anything anyone could possibly want to eat appears on the menu. The problem is that in order to be able to make anything on the menu, the restaurant needs to take certain quality shortcuts. For example, less frequently ordered items may be approaching or slightly over their use-by dates. Or rather than using fresh items, the restaurant may use frozen components. In addition, the cooks may not be familiar or practiced with some of the less frequently ordered items. As a result, in their attempt to please the palates of everyone, the results are mediocre and only satisfy those who really don't care what they eat anyway.

On the other hand, a restaurant with a limited number of items on the menu is going to manage to keep a much fresher flow of ingredients. They don't need to store things in a frozen state ready to be thawed into a mush. The chefs in the focused restaurant are going to have a great deal of expertise in preparing all the dishes on the menu. The result not only satisfies the customer but delights their palates.

The same happens with systems. More systems result in less competence and a greater attack surface (mushy software), where getting a quality result becomes increasingly impossible. The attack surface of a system refers to the places in a system where a potential attacker can make an attempt. It includes software, hardware, networks, and any other potentially exposed components. Organizations have attack surfaces as well where their networks and systems are exposed to attack. Think of an attack surface as being in a jungle infested with mosquitos. While shorts and a t-shirt may be more comfortable due to the weather, long pants, long sleeves, a high collar, gloves, and a hat provider fewer surfaces where one can be bitten. Fewer systems result in greater knowledge and experience, diminished attack surfaces, and ultimately, greater capabilities.

Even with fewer systems, there can still be redundancies. These are capabilities or data that appear in more than one system. Sometimes that isn't so bad, if we can manage to keep them synchronized and not be paying for capabilities more than once. The challenge though is that sometimes, while these capabilities may appear to be duplicates, they do not produce the exact

same results. That can have significant consequences. It is much like the big box retailer that sells one model of a brand of a riding lawn mower and sells accessories for that same brand, except the accessories cannot be used on the specific model the retailer sells. It looks good on the surface, but things fall apart and become rotten underneath.

Integration and interaction with systems is an important goal for portfolio management. Having what appear to be the right systems but cannot communicate with each other can result in the infamous swivel-chair interface (**Figure 2.5**). That is where a person sits down with two screens, reads the date from the first, then turns and enters it into the second. While the swivel chair may be the extreme, this issue presents itself in many other ways, including the need for the creation of “glueware” or potential products being abandoned because “we have the data, but we just can’t get it where we need it.”

FIGURE 2.5 Ban the Swivel Chair Interface

Description

Flexibility and growth as a goal mean that when we need to change how we are processing information to support changes to the business strategy, we are

able to do so without having to do a large upgrade to our systems or without having to go through some data gymnastics, such as “this field is going to be used to represent left-handed customers now.” There are two realities that drive the need for flexibility and growth. The first is that no designer or inventor of a system is ever going to be able to fully anticipate how a system will be used or how it needs to be used in the future. The second is that the harder a change is to make, the more likely it is going to result in a security issue or cause the creation of a work-around, which will result in a security issue. Including systems and capabilities for flexibility and growth thus becomes a security imperative.

Portfolio management begins with understanding the current systems and capabilities as well as the capabilities needed by the organization. The needed capabilities are then mapped onto the systems, and shortfalls and overlaps are identified. Where there are redundancies, systems can be consolidated or eliminated. In other cases, needs that are currently unsupported can be implemented on systems with capabilities that were not being exploited.

The shortfalls in capabilities and anticipated or planned new capability requirements can be mapped onto either an existing, an upgraded, or a new system in a manner that is most effective from a cost, management, and control perspective. These capabilities can then be mapped into the roadmaps for the individual system. The plan for how this will be done over time is the portfolio roadmap.

Some of the important considerations when determining which system should be used to provide a particular capability include the current capabilities enabled on a system, the system security level, the cost of implementing the capabilities on particular systems, and the degree of challenge imposed by a system in integrating different capabilities.

Developing the portfolio will face several potentially significant challenges. First is the problem of identifying all the systems in use in an organization. At first blush that seems like it shouldn’t be such an issue. But reality is that there are many shadow IT organizations and functions that spring up in any organization.

BOX 2.3 Shadow IT Organizations

Shadow IT organizations spring up in companies for many reasons. Sometimes, functional departments with a company don’t feel they are getting the support they need from IT. Sometimes, managers want the

power of having their own IT experts on hand; sometimes organization (marketing we're looking at you) have excess money to spend and go buy whatever they feel like.
Shadow IT organizations can take many forms:

- Shadow IT may be regular employees who are into technology and provide informal support in their department, which becomes more formalized over time.
- It may be an accountant who created a spreadsheet to handle tracking of types of spending, and now the spreadsheet has become too large and complex to handle, so they hire an assistant.
- The shadow IT group may be contractors a group hired to perform a specific task, and then another, and another, and suddenly the contractors are semi-permanent.
- The shadow IT organization may be composed of SaaS applications purchased with no coordination with the IT group and disconnected from the company's systems.

In almost all cases, the department with the shadow IT organization feels the need was justified, but at the same time, these represent significant security vulnerabilities.

Issues are as diverse as systems from acquisitions that have never been formally integrated to the desktop spreadsheet application running on someone's desktop that manages millions of dollars of business for the organization or an SaaS application paid out of an advertising budget in a remote regional office. Whatever the reason, these systems exist, represent unknown security risks, and need to be incorporated into portfolio planning whenever they can be identified.

Just as identifying systems is an issue that is harder to overcome than one would expect, knowing what the capabilities and functionality of unknown or even known systems can be a real challenge. The reality is that many systems are poorly documented or have their documentation unavailable, and the people who know, or should know, their true capabilities are the users and operators of the systems. Usually even those people only know about some specific capabilities, limitations, or parts of the system they habitually use. It is not at all unusual to look for a function, only to find it already is present in an existing

system. There is just no one who knows about that capability or has experience with it.

One of the most important parts of portfolio management is the **information management** plan for the organization. Information management looks at the information an organization uses, the purpose of the information, where the information is gathered, and where it is stored. Information management involves managing the people, processes, and technology that provide control over the structure, processing, and use of information in an organization.

Successful information management requires ensuring the quality, accessibility, and utility of every piece of information in the organization. With that is a requirement that for each element of information there is an owner, a custodian or steward, and an authoritative source.

Among the more important concepts in information management is that of properly protecting information. This overlaps with security in that there is concern for the classification of data and applying appropriate controls for that data. Naturally this means that data must be safely stored. In addition, a great deal of the effort in providing proper protection is based upon managing the flow of data from one system to another and the types of vulnerabilities which that may induce. Those vulnerabilities as well as the potential vulnerabilities in any system in the portfolio will play a role in determining which system in a portfolio will support a needed capability.

Overall, the management of a portfolio can be quite complex, especially in the early stages before there has been an opportunity to harmonize the portfolio. There will be many capabilities which are duplicated not just in capacities but also in actual use. There will also be systems in the portfolio that, simply put, do not play nicely with others. Developing an understanding of these issues can lead toward a long-term solution. Along with this are the challenges of migrations and integrations among the pieces of the portfolio, especially as it evolves over time. Were that not enough, challenges come from zealous attachment to systems. These are manifested through those who resist change because “this is the way we’ve always done it.” Or those who champion a deficient, overly complex, or overly costly system because they believe “this system can do anything—yes, really.”

Having a managed portfolio is also important from a security standpoint because it reduces surprises. Because the systems in the portfolio are known, studied, and understood, security personnel can determine the appropriate measures to manage risk within the portfolio. When an application is encountered that is outside of the accepted portfolio, it can be immediately

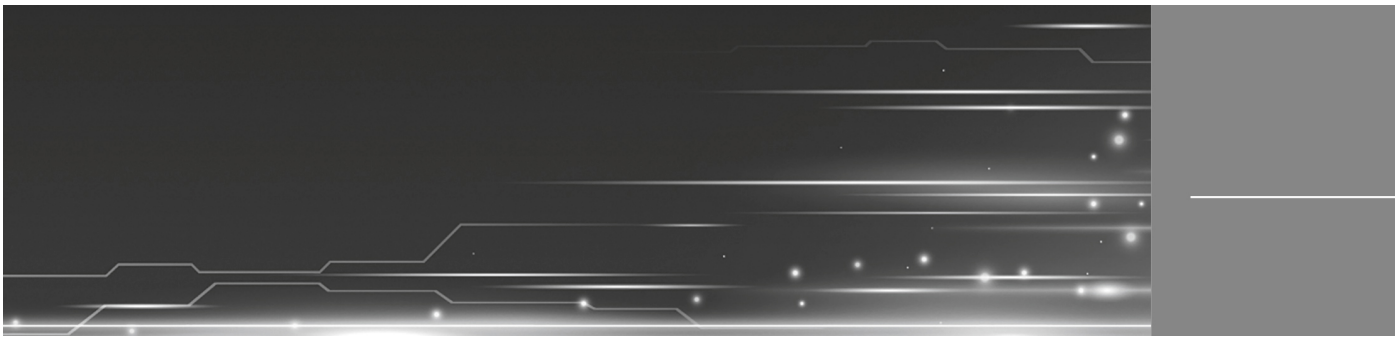
recognized as a security risk as well as a risk to efficient portfolio management and dealt with appropriately.

SUMMARY

The business need for a product or a portfolio of products is the reason for the existence of the product. The specific needs of the business will drive the level of effort put into the development and maintenance of the product. This may not always align with technical views of how a product should function or be developed. That can represent a challenge to people working on the product who don't understand how the market is driving the business requirements. Those who do understand how a particular product fits within a portfolio and addresses a specific market segment are going to better understand how to develop the perfect product for the business.

KEY TERMS

business case
business strategy
competitive analysis
ecosystem management
five forces model
information management
market opportunities
market share
market strategy
marketing
portfolio management
positioning
product definition
product management
product planning
product roadmap
product strategy
product vision
software-as-a-platform (SaaP)
software-as-a-service (SaaS)



© d1sk/Shutterstock

CHAPTER 3

Program and Project Management

CHAPTER OBJECTIVES

- Discuss the importance of managing projects and programs effectively.
- Describe the five phases of project management.
- Explore tools and techniques for project management.
- Create a project plan and other project documentation.
- Examine the strategic approach of program management.

Introduction

Project and program management are fundamental skills to enable the completion of a successful secure software system development life cycle. Program management and project management are quite different but require many of the same skills. A project has a beginning and end, or completion. Multiple projects that are intended to meet a high-level strategic goal are part of an overarching program. Having a program under which the individual projects contribute to a major organizational effort and pull together increases the probability of success of the program and the individual projects. Program management is also used, together with product management, to coordinate ongoing project efforts for a product oversight. This ongoing oversight ensures the product will continue to be managed properly into the future.

BOX 3.1 The Project Management Institute

The Project Management Institute (pmi.org) is a worldwide, nonprofit professional organization dedicated to project management. It sets standards for project management as well as the requirements for anyone who wishes to become a professional project manager.

All projects have a project start and a finish. Projects that span multiple years should become a program broken up into subprojects. Subprojects allow the program to show progress as well as translate the overall program goal(s) into more manageable pieces, which are less daunting and have the added advantage of increasing program visibility.

Program management oversees several projects at a much less granular level than project management to ensure all projects that are part of a much larger goal (or program) are completed on time and meet the required goals so that the program is successful. (See **Figure 3.1**.)

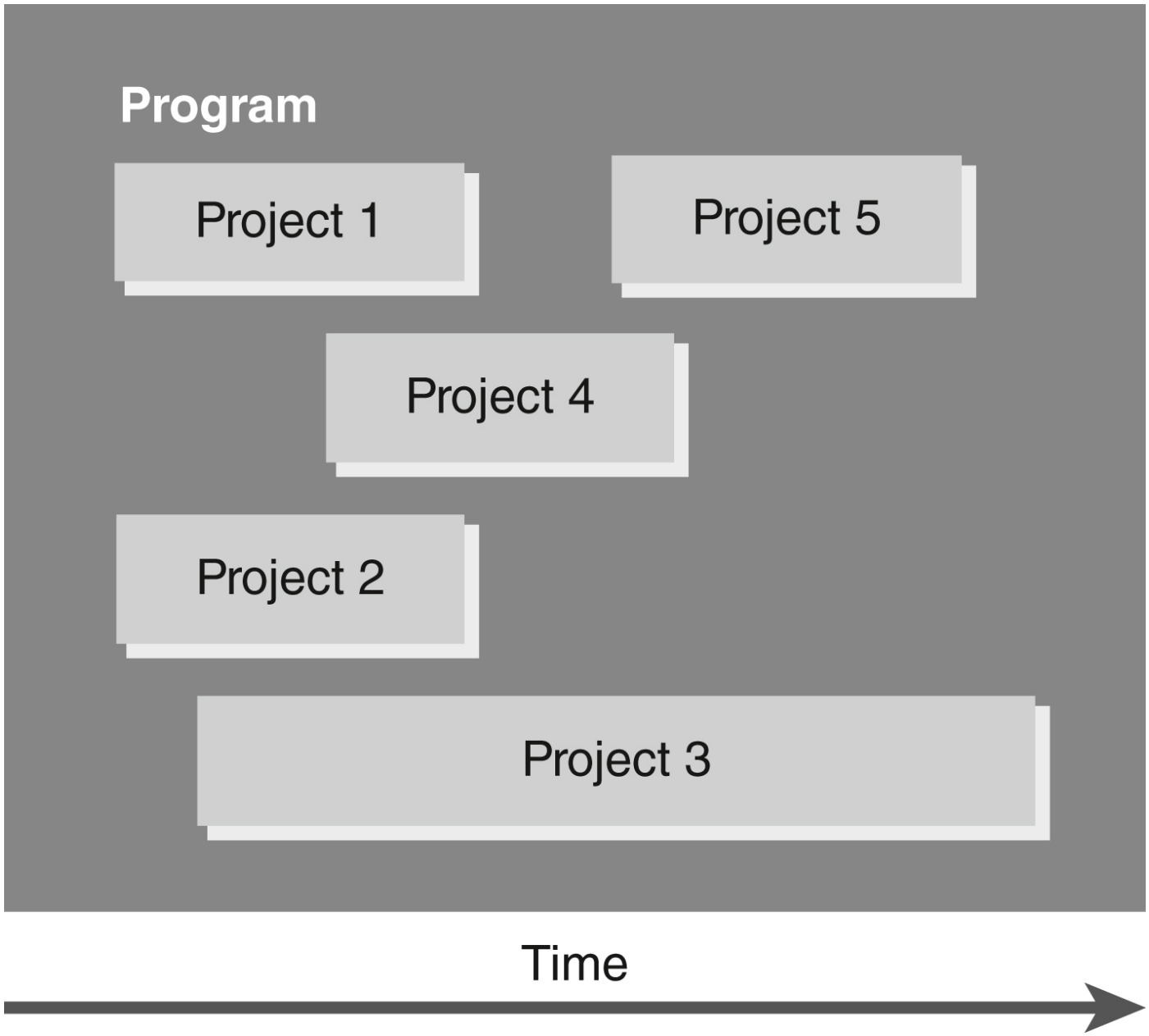


FIGURE 3.1 Relationship between Programs and Projects

Description

Many times, projects that are part of an overall larger goal of the organization are also grouped under the designation of program. These projects need not originate as part of single larger project, nor do they need to be directed toward a single system, but they will often share resources or contribute toward a common need. Program management is used to support the progress of all of these projects in a way that prevents them from interfering with each other and optimizes the use of shared resources.

Project Management

In any type of human endeavor, people perform project management. Basic project management processes are used whether making supper, getting ready for work, or building a high rise. **Project management** encompasses thinking of an idea (such as supper), planning how to execute the project (based on deciding what to have for dinner certain supplies are needed and an order of execution is planned), executing the project (implementing or making the supper), ensuring the project is going as planned (tasting, timing, and reviewing the menu so you don't forget anything), and finally deploying the project outcome (putting the supper on the table and eating it). In summary, following are the five phases of project management (as defined by the Project Management Institute):

1. Initiation/conception/analysis of the project goal
2. Define and plan
3. Implementation/execution
4. Monitor and control (progress oversight) (often part of phase 3, execution)
5. Close project

Project management allows us to break down simple and complex efforts into consecutive phases, which in turn makes all the work needed to produce a product or create a process easier. The phases and efforts needed for a successful project are collected and documented in a project plan. Creative efforts (all projects) are made easier by breaking down the project into phases because it is easier to perform a specific set of defined actions following a well-constructed plan than trying to do everything at once, which can be overwhelming.

Imagine trying to figure out what to have for supper while shopping for ingredients. It is likely the decision on what to have for supper will change many times if the decision has not been made before selecting ingredients because the ingredients depend on the product goal (supper). Effective project management has advantages, such as increasing the probability of completing a project, providing a product the customer expected, and completing a project within the expected time frame and on budget.

Following an established process such as project management also encourages everyone's participation, increases the probability of client satisfaction all through the process and with the final product, and has been shown to decrease overall project costs due to increased efficiency throughout the project execution.

Let's think about building a house. While the specifics of the electrical wiring for the house (placement of switches and outlets) are often unique to a specific design, house building is mature to the point that the pieces of the house have standardized designs, pre-established legal and regulatory requirements, and the components for each piece or part of the house are available for purchase. All houses go through the same building phases yet each one is a separate project.

Phase 1—Initiation/Conception/Analysis

An idea for a product or changes to an existing product is proposed, much as happens with either a new house or renovations to a house. The idea is discussed, tossed around, more details are defined, and people realize it is time to investigate whether to do a thorough assessment of the project and make a final decision on whether to proceed with the project (building a new house), whether to abandon the idea permanently, or shelve the idea for future consideration. The phase in which the decision to proceed or not to proceed is made during the initiation phase. When performed effectively, this will increase the probability of project success because it is this phase that defines the project.

Analysis performed during this phase determines whether the project is feasible (or not) based on analysis of a number of factors. Inaccurate, rushed, or inadequate analysis during the initiation phase leads to projects that are never completed, do not complete on time, or experience cost overruns. When considering whether to build a new house, the idea that it would never be completed is unacceptable given the financial and emotional cost to the owners. The same is true for any software system or service cost overrun. While cost overruns when building a new house (again, this is the same for any project undertaken) are not uncommon, they can have a high impact on whether the house gets completed or, in the worst case, can cause stress in a marriage. A careful analysis of all pertinent factors during the initiation phase greatly decreases the probability of negative impacts or project failure.

First an idea is conceived of and communicated. Effective communication of the project idea or goal can be performed through any number of methods

(although not all are necessarily performed for all projects):

- Informal discussions
- Formal discussions
- Environmental studies
- Analysis of future impacting factors
- Business case document/feasibility studies

Informal discussions include tossing the idea around with colleagues and management. At this early stage of project consideration, it is important to communicate with anyone who may be involved in the project, impacted by the project, or could support the project. These people are called stakeholders as they have a "stake" in the outcome of the project. Without their support, the project will face resistance at a minimum and outright sabotage at a maximum. Being aware of all the possible stakeholders can make the difference between project support and resistance to a project. A previously unknown stakeholder with considerable influence may express a concern with a project at a critical point. This can cause unnecessary project delays while the previously not included stakeholder is brought up to speed on the project or, in the worst case, cause the project to be terminated without completion.

When building a house, informal discussions begin with looking at the current house and where it is and then considering where else might be preferable to live. The decision of whether to build a new house or renovate the current house is based on many factors. If the current house needs so many updates that a new house costs less in terms of both money and time, the decision may be to build new. The same is true of any project. Many considerations have to be reviewed before moving forward to more in depth analysis.

Once the project goal or idea moves beyond informal discussions, the time for more formal discussions begins. Formal discussions get people thinking about the idea as an actual project. This brings up many questions: Would the project increase the company's value? Would it increase productive output capacity? Would it increase company visibility in the world that in turn would increase global business penetration?

In the case of a new house, the question is: What would a new house add to lifestyle and comfort (as well as any other long-term planning considerations)? In all cases, some project value must be seen for the project to move to any concrete studies.

At a high level, is the project feasible at the present time? This question brings up further questions of technical feasibility (which will be determined by a feasibility study), financial feasibility, resource availability, and other key feasibility questions. If the project is perceived to be feasible at this initial stage of discussion, then it is ready for more in-depth analysis.

What is the criticality of the project? The higher the project criticality, the more important it is to build the product in such a way that it has a very low probability of failure. For example, an infrastructure project such as oil pipeline or electrical grid update must be built to a level of quality that ensures very few failures. Large changes to an organization's connections to the internet must include conversations and assessments for reliability, scalability, and the mitigation of possible cyber threats in final product. On the other hand, a project for a device that is not critical, a new type of vacuum cleaner for example, could be completed with a high-risk, high-reward approach.

The result of formal discussions and analyses is a **business case** document. This document brings together the product idea with information on how the project will increase company value, the project's feasibility, the project's criticality, and other factors. Having all this information in one document (or a set of documents) helps the people making the decision to approve (or not approve) the project to make an informed decision.

BOX 3.2 The Business Case Outline

1. Executive Summary
2. Project Definition
   a. Background
   b. Objective
   c. Assumptions
   d. Benefits and Limitations
   e. Options
3. Recommendations
4. Analysis
   a. Market Assessment
   b. Financial Analysis
   c. Environmental Assessment
   d. Technical Feasibility

e. Risk Analysis
f. Additional Analysis
5. Timeline

The project sponsor oversees the business case creation with input from other experts in relevant fields such as finance, IT, and other areas either necessary for the project completion or who will be impacted by the project.

Several initial studies contribute to the business case document. These studies include an environmental study or analysis, future impact analysis, technical feasibility, financial analysis, risk analysis, organizational capability analysis, and others.

An environmental study analyzes the physical environment factors, such as whether a house can be built where desired, ease of access, availability of water, or electricity, or even legal or regulatory requirements. While it might be great to consider building a house on a high mountain ridge, access, restrictions against building, and basic water availability must be considered. Companies need to do environmental studies when building manufacturing plants and other physical structures. Environmental studies also include evaluation of the interaction of human social factors, legal and regulatory factors, current relevant corporate factors, and technical and scientific options. Any one of these factors can preclude going forward with a project. Not considering all factors that could impact a project can lead to failure, delays, or financial overages.

Even software systems may require an environmental study. This would be especially important for projects with control systems, but other impacts of software systems on the environment also need to be considered. An example would be the excessive power consumption of “proof-of-work” cryptocurrencies. These systems were deployed without any consideration of the harm they do to the environment.

One other key analysis that is good to do and provides input into the formal business case document is to look at possible future impacting factors. Are there possible future new technologies or business concepts on the horizon that should be examined that may change all or some of the project approach? To complete such an analysis, external consultants may be asked for expert input in their relevant fields.

All the many analyses provide input into the formal business case document. While a formal business case document is not usually created when a family is

considering a new home, all the discussed advantages and disadvantages (including that the family may just desire a new house) are likely to be documented in some informal form, such as the back of a napkin or other nonstructured document (spreadsheet or other document).

The term “business case” has been used to identify the document that is prepared during project initiation to detail the reasons and justifications for a project. The business case is used to determine whether the project is worthy of completing. The business case document details what is being proposed, the factors behind why the project is being proposed (the why), high-level suggestions on how the project can be achieved, cost benefit analysis, other business, financial considerations (competitive factors, business objectives, and other factors), and feasibility analysis, resource requirements, and an estimate of how long the project would take.

The negative points of the current system or situation need to be included in the business case because these often are the reasons for considering a new project. For example, what are the negatives of the current home that would be overcome by a new house? Is it to build where the family would prefer to live long term? Is it that the current house has so many issues that the cost to update is very close to building new? Perhaps it is simply that a new house has always been a goal and now is a good time to consider going forward with building it. A company must always weigh the benefits of a project against the costs of the project or continuing with what they currently have or are doing.

No one likes to discuss negative factors when trying to get an idea to reality; however, the business case needs to contain risks as well as advantages. Risks to software have become more critical as more and more industries rely on software in their day-to-day operations. Any disruption to operational software causes large business disruption and impacts earnings. With threats from cyber criminals from data breaches and ransomware constantly rising, software security risks need to be discussed early in the project. This initial elapsed time is a very rough estimate as all factors and tasks that have yet to be identified and estimated.

The business case is presented in many meetings, to many individuals one on one, and to many different stakeholders and departments. It is updated with input, and often more analyses are needed before it is considered complete. A well-written business case increases the probability that the decision of whether or not to proceed with a project is the right decision.

With respect to format, the business case document is received more positively if it is brief and concise but clearly conveys the project goal (idea) and the benefits as well as value to the company. It can be in the form of a

slide deck presentation that is used for all meetings, or it may be a written document that is circulated to all stakeholders. Often there will be multiple types of formats for the business case. This variety provides business case reviewers with a communication method that is best for them. Some business case presentations will include audio so that while reading the business case the details are also being heard during any artifact review.

A key question for any project is what problem or issue would be made better by the project. If the goal of the project is not clear, the goal will not resolve the issue or problem the project is intended to resolve, or if too much change or financial input is required, the project should not go forward. While this can be questioned during phase 1, the **initiation** phase, by the time the business case document is completed, there should be sufficient information to decide whether to it is best to proceed with the project and a decision must be made at this point. In some situations, a project gets beyond this point and after financial and personnel resources have been expended, a project is canceled.

During development, the business case document is socialized with all stakeholders. Socialization can take many forms, such as virtual meetings, in-person meetings, widely distributed emails, and posting the artifacts in a central place where stakeholders can access the artifacts. The socialization process must include a method for questions and feedback. The questions and feedback will be included in the final business case.

BOX 3.3 Project Artifacts

Project artifacts are any document or representation that serves a purpose to organize a project, provide visual information regarding the project, or report on project status. Artifacts can include physical models of the expected product (the architect of a new house may build a model of the house to show some of the planned details), minutes of meetings, the project plan, various type of charts, code, lists, spreadsheets, logs, or any other project information sources that are created and used during the course of the project.

All artifacts will be stored together where they can be easily located during the project and in the future. Artifacts are kept for future reference as to how the product was built, particularly if changes need to be made to the product later. They provide a record of what was done during the

project. Often the artifacts are used to evaluate the effectiveness of a project, as well as to reconstruct tasks done during the project.

Once the business case is ready, including analysis and artifacts, it is time to review it and decide whether to proceed with the project in the proposed form, revise (rescope) the project, drop, or delay the project. This decision may be made by a designated person or persons or a selected group of stakeholders. In the latter case, the stakeholders may be asked to vote on how to proceed with the project. Each person's vote is captured either through a formal voting process (email request for vote or by capture of vote in virtual or in-person meetings). The voting results are recorded in a formal document (spreadsheet or ther document) and stored with all other **project artifacts**. It may be the case that while the project does not get sufficient positive votes to proceed, the executive management of the company may have other reasons for the project to go forward, in which case the project will go forward to the next phase. This also applies to building a house. It may be that some members of the family do not wish to move to a new area or house; however, some members of the family may have more input than others and will make the decision to build. Another part of the initiation phase is creation of a **project charter** to set out goals, intent, and boundaries of the project. The project charter is a high-level description of the proposed project that everyone can use to focus on the project goals. The project intent describes the purpose that the project is intended to fulfill and provides a guiding star, which is especially important as the project proceeds and questions arise. A quick look at the project intent can provide guidance on what is really needed or correct for the project. Something could be technically correct, but if it doesn't fulfill the intent of the project, it is the wrong choice. The charter will roughly define the boundaries of the project. It lists the key stakeholders of the project. It includes a baseline of the resources that will be allocated for the project. And it may include other basic information about the project. The goal of the charter is to provide a solid, well-defined starting point for the project.

Where and how the project charter appears and how it is used is going to vary from organization to organization. In some organizations, the project charter is developed as a part of the business case. In others, the business case is appended to the charter. In the first case, the business case approval is considered the formal agreement to start the project. In the latter case, the approval of the charter formally starts the project. Still other organizations use a multistep process and wait to create the charter until after the business case

has been approved. For some organizations, depending on the project type, only the project charter is required because the organization may not see the need for business cases for specific types of projects.

Regardless of whether it is done on the project charter or the business case, it is important to have a signature page or process where everyone has signed off on the project and agreed to the basic structure of the project. This helps to ensure project buy-in, helps to establish that communication about the project has begun, and helps to later be able to reference stakeholders' initial support for the project. If stakeholder later has a question about the direction the project is taking, then the original charter will show the intended direction, and the stakeholder's issues can be addressed or assuaged.

Having a permanent place to store project artifacts is important to allow for project goal/scope and approval review throughout the project as well as provide information possibly decades in the future as to why a project was undertaken, what factors went into the project, and all the relevant information as the project was worked. Furthermore, information from such artifacts can be used in the future when changes to the product are needed.

Once the project has been approved to go forward the project enters phase 2, the define and plan phase.

Phase 2—Define and Plan

In phase 2, **design and plan**, the specific project details are defined. The **scope** of the project is more completely defined, which includes what will and will not be done as part of the project. The scope also includes any constraints, the expected timeline, and what project success looks like. This is critical to ensure everyone understands exactly what will and will not be included at project end. Project scope can be captured in a separate document or artifact or may be part of a larger planning document. It is good to post (and share in meetings, and other communication methods) the project scope (in brief) somewhere that everyone can see so that people do not lose track of what is and is not included. When building a house, scope is particularly important. How big is the house to be? How many rooms, bathrooms, how large a garage, and any other key deliverables are to be included? If items are forgotten, they will increase the cost of the house and probably cause a delay in building. It is even possible that whatever was forgotten will not be possible to add later. Think about forgetting to specify that the kitchen must face south; that would be very difficult to change once the foundation had been laid.

At this point a discussion of scope creep is helpful. **Scope creep** refers to changes in what is and is not included in a project. If, for example, hardwood floors were not included in the initial scope definition of a project, but the customer requests them as house building progresses, this would be considered scope creep and will definitely increase the final cost of the house. The same is true of any project. Requirements added after the project's scope has been defined and approved will likely to increase the project costs as well as cause the project to run longer. Interestingly at the rare times, inventive solutions have led to increased scope being integrated without scope creep and sometimes enhancing the final product. These are very rare however.

How scope creep will be managed on a project is defined at the outset. One method is to clearly state in the project scope document that any changes to the defined scope must be submitted formally, socialized with relevant stakeholders, and the approval or rejection documented. Part of the scope creep process is also to identify what the ramifications of any extra requirements (scope creep) would be, whether a requirement for increased staffing (and therefore increased financing) or an elongated timeline.

Before capturing any tasks to complete the project, the project management team will identify the following:

- What will the team's **operating rhythm** be?
    - How often will the team meet?
    - How often and in what methods will the project team lead(s) communicate with various stakeholders including management?
    - How and with what frequency will the team report progress?
- What systems or artifacts will capture detailed requirements?
- What systems or artifacts will capture individual tasks needed to complete the detailed requirements?
- What systems or artifacts will be used to report project progress?
    - What types of metrics would be useful to show progress? Metric requirements can change over the life of the project.

The team creates documentation with the details for the listed processes and stores that documentation where all team members can access it.

Next, it's time to gather detailed requirements. Requirements are gathered with collaboration between the project team and stakeholders who will use the product or may be affected by the product. Just as homeowners work with an architect to define what they want in the house design and details, the more details they can provide, the more likely the homeowners will get what they will be happy living in. The process of requirements gathering starts with a list of questions that all people involved in the project—clients and team members—will participate in preparing.

Detailed requirements can make the difference between a project that meets client expectations and delivers on budget and on time, and one that fails to do so. The more detailed the original requirements stakeholders can provide, the more likely the development team can deliver a product that pleases them. Provided of course, they do not lose the forest for the trees and become so focused on minute details that the big picture is forgotten. Detailed requirements such as screen mockups and information on specific data elements greatly help the project development teams. Imagine a new house client not being able to specify how big a house they want or whether they want one or two stories.

Requirements provided by people familiar with what the new product or change in an existing product is supposed to do is necessary to help tell the project team what is and is not to be included in the final product. Imagine trying to create a new input form, but no one has a list of the information to be gathered or what the form should look like. Requirements include physical and logical items that could affect the quality and completeness of the final product. No detail is too small.

For each requirement, the client provides the purpose for the requirement or where it fits into the final product. For example, the client specifies a set of stairs between the kitchen and the upstairs to allow the family members to come and go privately when entertaining is happening in the formal living and dining room. Having information on why the requirement is desired will help the implementation or execution team create a better design and possibly allow the execution team to suggest alternatives that are better than the client may have envisioned.

Formal artifacts help capture requirements effectively. A requirements document or artifact can be a spreadsheet, visual representation, or other document type. Some electronic systems can also capture and track requirements.

A particularly useful tool in the capture of requirements is the "use case." A use case is a narrative that describes both the features and how they are used

in context to perform certain functions. Generally, stakeholders provide the use cases. For example, to describe the ideal entryway into a house, the expectant homeowners may tell the story about how they want to arrive home at the end of their workday as, “I want to be able to walk up to my front door and be able to stand on a covered porch while I dig my key out of my pocket without getting wet. Then I’ll open the door, walk in, and set my bag of groceries on a handy table. Next, I’ll hang my coat in the closet, and then sit down on a bench and take off my shoes (we follow the Swedish tradition of not wearing outdoor shoes in the interior of the house), step into my slippers, and then grab my bag of groceries and carry them to the kitchen to put them away.”

Notice how in this “story” the user, without directly listing them, specifies several requirements. The entry must be protected from the rain. The coat closet needs to be close to the door. There needs to be a counter or room for a table in the entry way. The entry way needs room for a bench. There needs to be some clear indicator in the entry way where outdoor shoes versus indoor shoes are worn.

The first requirement is interesting in that while the narrator described a porch with a roof, it was really a way of describing the requirement (protected from rain) rather than the requirement itself. If protection could be provided in a different manner, then it would be equally acceptable. It is not uncommon for users to describe general requirements in terms of specifics. This is because people are most comfortable with specifics and don’t always do well at taking the broader perspective of extracting the broader concept.

The last listed requirement, a clear indicator of space functions, is another requirement that must be interpreted from the use case rather than one which was stated outright. In this case, the user didn’t specify a means of implementation to describe a general concept, but rather gave a general situation from which the requirement could be inferred.

Use cases are provided by the stakeholders who know what the system needs to do. Use cases describe what and how the product will perform specific functions. For the newly planned house, the clients may note a need for doorways wide enough to accommodate a wheelchair for when older family members visit.

This same concept applies to software. The stakeholders describe what type of function the software will perform along with information on data to be collected and manipulated. They will also provide information on what information is preferred on which screens. Very likely the stakeholders will know details such as what reports need to be included in a system or services and what information the report must include.

Capturing requirements can be a very collaborative effort, and many different methods can be used for stakeholders to communicate requirements to the project team. Often project teams find the use of a whiteboard (or some such similar method of either virtual or in-person collaboration) helpful because it allows for discussion of the requirements and their relationship while being able to make changes in real time as thoughts and requirements develop. The information from such sessions is captured and shared with all participants so they can provide further input.

Every industry discipline has its own preferences on what type of documentation to use for requirements. Architects will use a combination of written documents and architectural drawings, including blueprints. Electricians will use electrical drawings and written documents. The written documents specify requirements that are inconvenient to be detailed on blueprints or drawings. Blueprints or drawings create better visual understanding of the requirements. When building a new car design, for example, there will be electronic renderings and small physical mockups of the projected new design.

Project deliverables are an important requirement. These are all of the items and artifacts that need to be provided to the stakeholders throughout and at the end of the project. It is easy to remember that at the end of a house-building project that the physical building and the keys need to be delivered to the client. (Well, they usually remember to provide the keys, but my last builder didn't. Fortunately, I found them hidden in a kitchen drawer.) However, numerous other items also need to be delivered either during or at the completion of a home build. Some of the more obvious of these include the warranties and instructions for all the appliances, materials, and devices in the house, but others are less obvious but still important. For example, the homeowner will need copies of all the permits that were required for the build.

Once the requirements of the project have been gathered and agreed to, they are ready for formal signed off or approval. This means that the clients will formally state that indeed the requirements as gathered represent what they expected in the final product. Both the requirements and the formal sign off decrease the possibility of deliverable misunderstandings between the clients and the implementation team. In other words, if the clients are not sure of what they want and the team is forced to make assumptions, it is highly probable that the clients will not like the final product. When building a house, if the flooring requirements are vague, then the builder will likely install carpeting even if the clients prefer hardwoods or tile because of family allergies. This may seem like a small item; however, any changes after completion leads to increased costs and product (in this case the house) delivery delays.

As noted, clients provide the functional requirements for the project product. They may provide technical requirements such as performance expectations or expected electrical requirements, but in general, it is the implementation team that will assess technical requirements based on the client's stated functional requirements. If a bathroom is required on the main floor as well as a bathroom for each bedroom on the second floor, the architect and plumber may work together to determine the most efficient placement of the bathrooms to decrease water and other plumbing. The same is true for a software system. Software team experts will determine the best technical solution for client requirements. Often there is discussion between the clients and the technical experts to provide optimal solutions.

Once the requirements have been agreed to, further planning can proceed. This includes creating a work breakdown structure (WBS), which subdivides the project into smaller projects, making it easier to manage. Dividing the project into smaller subprojects (such as a project dedicated to electrical requirements) allows project team members specifically required for each subproject to provide details of what needs to be done in their area of expertise. At the same time, this approach means project oversight can more easily see where things may not be meeting the expected timeline and adjustments can be made. Very often this approach also helps clarify the relationship and dependencies of various requirements. Some requirements need to be executed one after the other, and others either can or need to be executed in parallel.

The Project Plan

The **project plan** is the major outcome of the design and planning stage. Project plans provide clear guidance to all stakeholders of the project on the details of how the project will be completed, managed, and reported. The project plan is not completed as a one-time document; it changes as the needs and circumstance of the project changes. Often there will be a formal process for controlling changes to the project plan and the project itself. Other times, the project manager will maintain a more informal update process. The only thing worse than a project plan that changes without control is a project plan that never changes.

BOX 3.4 Project Plan Outline

- Title

- Byline
- Version
- Change History

1. Business Requirements
    - Background
    - Business Opportunity
    - Business Objectives and Success Criteria
    - Customer/Market Needs
    - Business Risks
2. Vision of the Solution
    - Vision Statement
    - Major Features
    - Risks, Assumptions, and Dependencies
3. Scope and Limitations
    - Scope of Initial Release
    - Scope of Subsequent Releases
    - Limitations and Exclusions
4. Business Context
    - Stakeholder Profiles
    - Project Priorities
    - Operating Environment
5. Scheduling
    - Work Breakdown
    - Gantt/PERT Chart
    - Key Milestones
    - Resource Assignments
    - Individual Responsibilities
6. Deliverables
    - Software/Hardware
    - Documentation
    - Key Presentations
    - Other Deliverables

7. Other Appropriate Elements for the Particular Project

Every organization has its own format for a project plan or uses a set of tools that serve the purpose. These project plans normally include a statement of the problem, objectives, scope, deliverables, assumptions and risks, stakeholders, task breakdowns, task structure and dependencies, timelines and schedules, and resource planning, including people, budget, and other resources.

The statement of the problem, objectives, scope, and stakeholders usually comes directly from the project charter or business case with enhancements as each is better understood. The deliverables may be dictated by the project, a result of the requirements process, or based on organizational standards and expectations. The deliverables spell out in detail every tangible product of the project. This may include final products, such as a packaged application, the source code, and the documentation, as well as other artifacts such as finalized test results, product feature sheets, application notes, and user surveys. Additionally, some artifacts will be required earlier in the project, such as requirement documents, test plans, security plans, periodic reports, and others. Each of these should have an associated milestone or schedule.

The risks and assumptions are key elements of the plan as they both highlight what can go wrong and provide the basis on which the plan is built. Plans based on faulty assumptions are not going to work whether that fault was on the part of the planner making wild guesses or the planner being given inaccurate or misleading information. It is critical to identify assumptions immediately so they can be validated and corrected quickly if needed, or the project can be modified if the assumptions on which it was based change. A good example of a faulty assumption is assuming that users are going to access a web application from their desktop computers. While that may have been accurate in the past, more people now access the internet from their phones and other small devices than from a desktop computer. Another faulty assumption may be that an application user is going to have constant internet access. On the other hand, an assumption of regular access to the internet is valid in many parts of the world. Or an organization may assume that software users may have access to a limited set of common operating systems.

Likewise, identifying potential risks early allows for alternative planning as well as careful monitoring in case the risks should arise. Participants in the

project who are aware of potential risks also have a greater chance of avoiding them.

Task breakdowns, structure, dependencies, and timelines are often addressed in a hierarchical manner, especially in a large project. At the first iteration of the project plan for developing software for example, tasks are for design, development, integration, and testing. In later iterations, components within those larger tasks are detailed and refined as the details of what is needed become clearer. The develop or implementation phase may then include individual tasks for the completion of different modules of the application, each with their own schedules and dependencies. Resource planning is also often refined as the project proceeds.

While project plans can be created using many different types of documents or software applications, project managers may choose from many project management specific applications to create their project plans. Such tools have the following features to help plan and execute projects:

- Capture the work breakdown structure.
- List each requirement with all tasks to complete each requirement.
- Assign a length of time for each task in a variety of manners (hourly, daily).
- Assign resources that will perform each task involved in a requirement, including what percentage of their time they will be working on a task.
- Assign predecessors. Assigning predecessors links tasks in specific order to each other. The idea is that a task depends on a previous task (predecessor) in some way as specified. Tasks can be linked to follow each other, to be worked in parallel, or any mix between. Some tasks can have slight overlaps but not be able to be worked in full parallel. It is possible to define a task to start a few days before the end of another task or start a few days after another task has completed.
- Record progress on each task as tasks are completed. Gantt charts translate each task and the task's start and end time into a graphical representation, along with task dependencies. Further, the application can be set up to show progress on each task bar in several different ways
- Build in Gantt, PERT, and other types of charting.

Work Breakdown and Gantt Chart.

All projects have **tasks** that need to be completed. Tasks are the basic unit of work used in a project. Information about each task, such as what the task is, how long it is expected to take, who may work on it, what task goes before and after it, and other details, are part of the work breakdown structure. The term **work breakdown** is understandable if thought of as a breakdown of all the pieces of work or the many tasks that have to be done to be able to build the product and deliver it to the clients.

Whether it is building a house, a new software product, or updating an existing software product, many tasks and phases are involved and have to be done in the correct order by the most appropriate people to arrive at the best product. It is not possible to mentally remember and track all the pieces for a complex project such as building a house or a software project, so various tools have been developed over the years. One such tool is a **Gantt chart** (created by Henry Gantt). An example of a Gantt chart is shown in **Figure 3.2** and displays a project's various phases and tasks. The Gantt chart provides a visual method to understand the tasks, their relative lengths, the dependencies between tasks, and other important information about a project. A Gantt chart can also be used to track progress on a project.

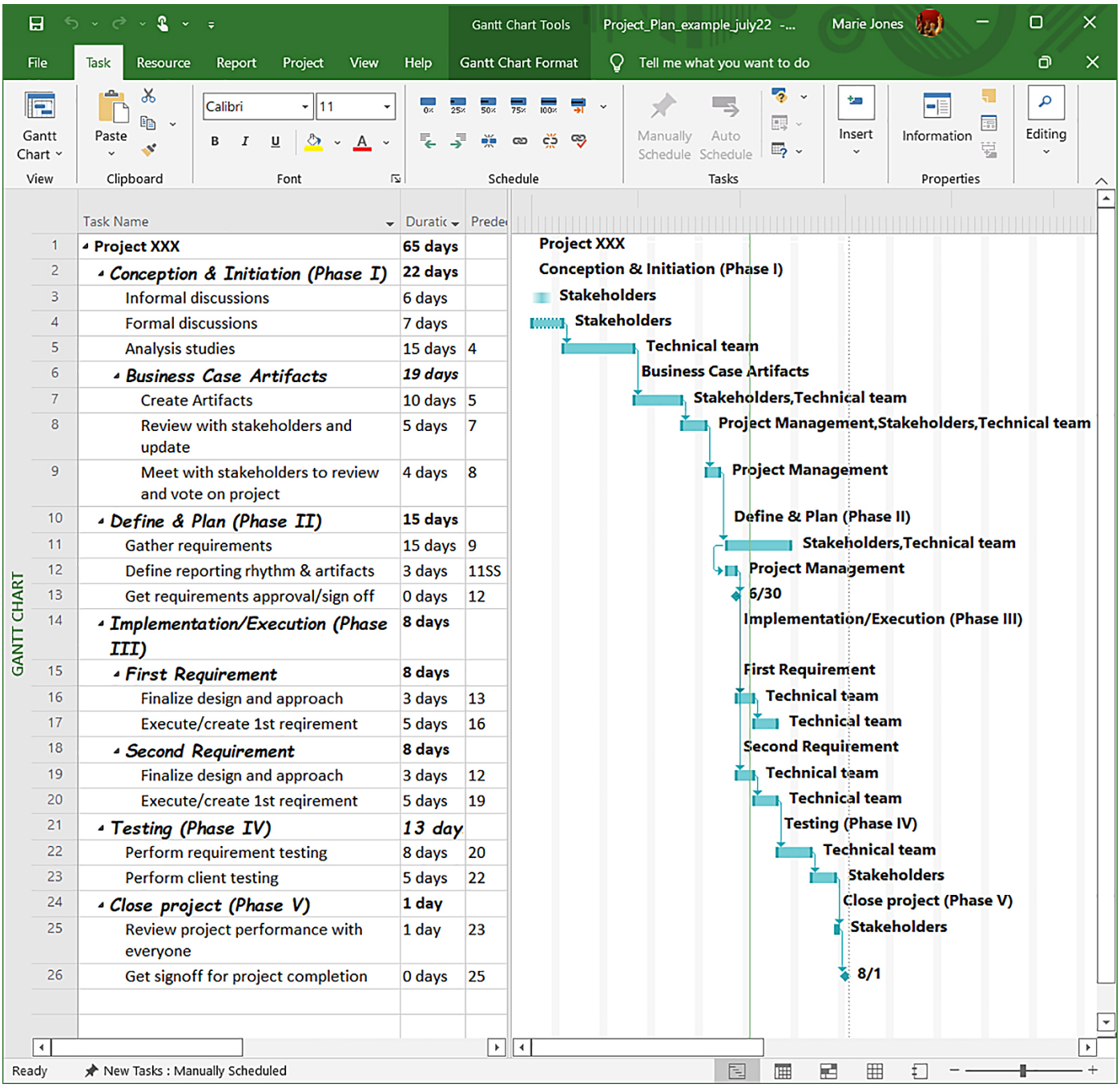


FIGURE 3.2 Work Breakdown and Gantt Chart Example

Description

The timeline shown is only a sample. Expenses for all resources and many other project management tasks can be captured in project management software applications. Note the order of the tasks is shown in the chart; some tasks start when a prior task is completed, and some are worked at the same time as other tasks.

All project management applications have advantages and disadvantages that are unique to each. Being able to track tasks and task durations is a basic

requirement of such applications. In the sample timeline, progress would show up as a darker line within the larger block that shows the expected length of time to complete the task.

Milestone Chart.

A **milestone chart** (sometimes called bird on a wire chart) is another visual tool used to communicate project task timelines and task status. Often, they are used to communicate high-level milestones or goals as a part of regular status reporting. **Figure 3.3** demonstrates the high-level information often shown on a milestone chart.

FIGURE 3.3 Milestone Chart

Description

Such charts are useful to help management see the overall project goals and progress. As each milestone is completed, the diamond format indicates whether a milestone is completed or yet to be completed. A filled in (black in this case) diamond means the task is completed. A diamond that is not filled in is a task that is not yet completed. A number of different indicators can be added to a milestone chart to show such details as when a task finishes early (slides to the left) or is late (slides to the right). The goal of a milestone chart is to capture the most critical tasks and their completion to give a succinct overview of project status.

Metrics Sample.

Metrics are measurable values that are used to monitor and evaluate a project. Metrics are often used consistently across projects. This allows comparing progress and effectiveness on different projects. Typical metrics may include measures such as the percentage of work completed, the number of hours worked, hours worked per team member, the lines of code written, etc. Metrics need to be selected with care because when they are used, project teams tend to work toward optimizing the metric, even if it is to the

general detriment of the project. Therefore, the best metrics are those that motivate successful completion of the project.

Metrics graphs are any type of visual representation of any project item that lends itself to such a presentation. Most often they are used to show project progress across elapsed time. **Figure 3.4** is an example of a burn-down chart. Note how the trend line is optimal project execution, while the actual project execution usually does not exactly follow the preferred trend line.

As can be seen in the metric in **Figure 3.4**, there were 1,589 requirements to start with. The red line indicates a projected trend (best possible burn-down progress) line. The optimal number of requirements worked each month do not meet the optimal trend line for expected worked requirements until late in the project (October). This can be expected on many projects because as the team(s) become more familiar with the tasks of each requirement, then team members work more effectively with each other.

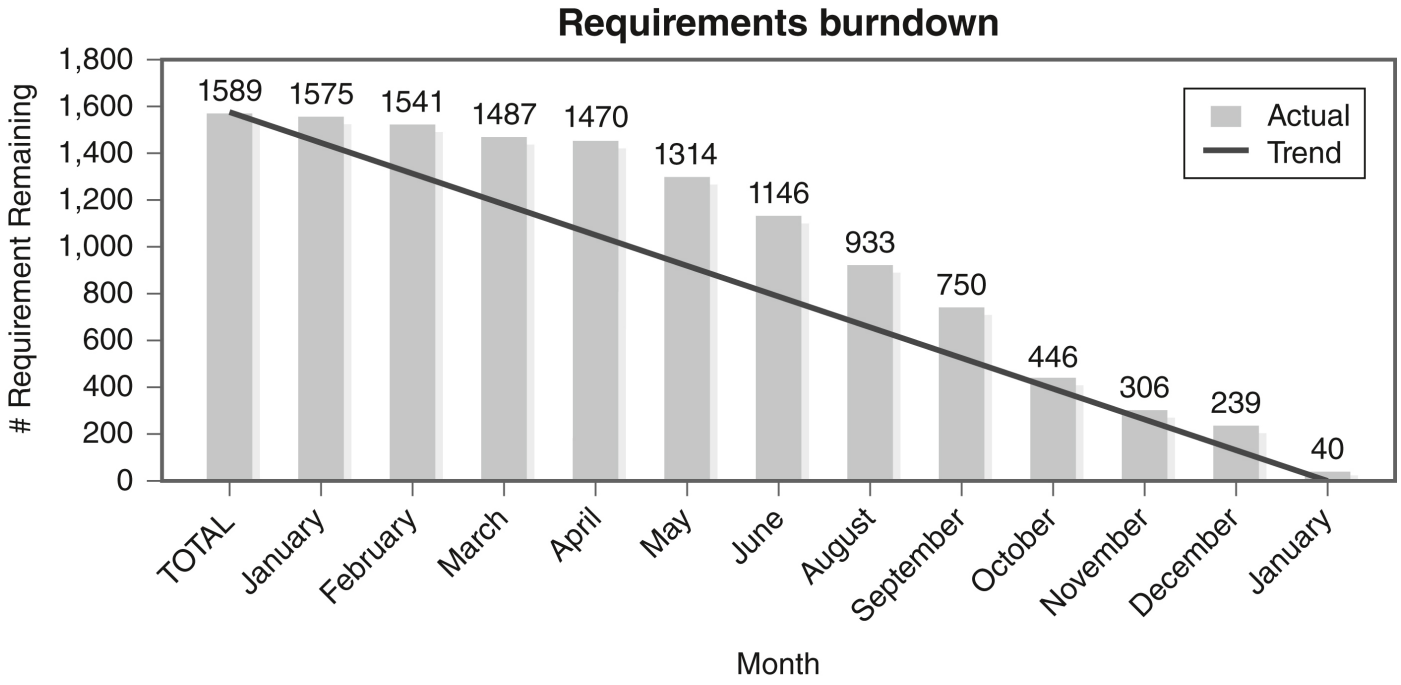


FIGURE 3.4 Metric Example

Description

Projects can use many other types of metrics to show project progress and status. Metrics are a picture, and a picture is “worth a thousand words.” They should be used frequently to convey status. Yet, too many metrics can be confusing, so selecting the best and most representative metric for what is being communicated should be considered carefully.

PERT Chart.

Program evaluation review technique (PERT) charts are used to show the relationship between tasks in a project in a different way than a Gantt chart. While both Gantt and PERT charts show expected project length, the PERT chart more directly displays how all the tasks relate to each other and can help provide a realistic timeline for a project. PERT charts help to visually organize and coordinate project tasks. Elapsed dates or task duration for each task can be added, as shown in **Figure 3.5**.

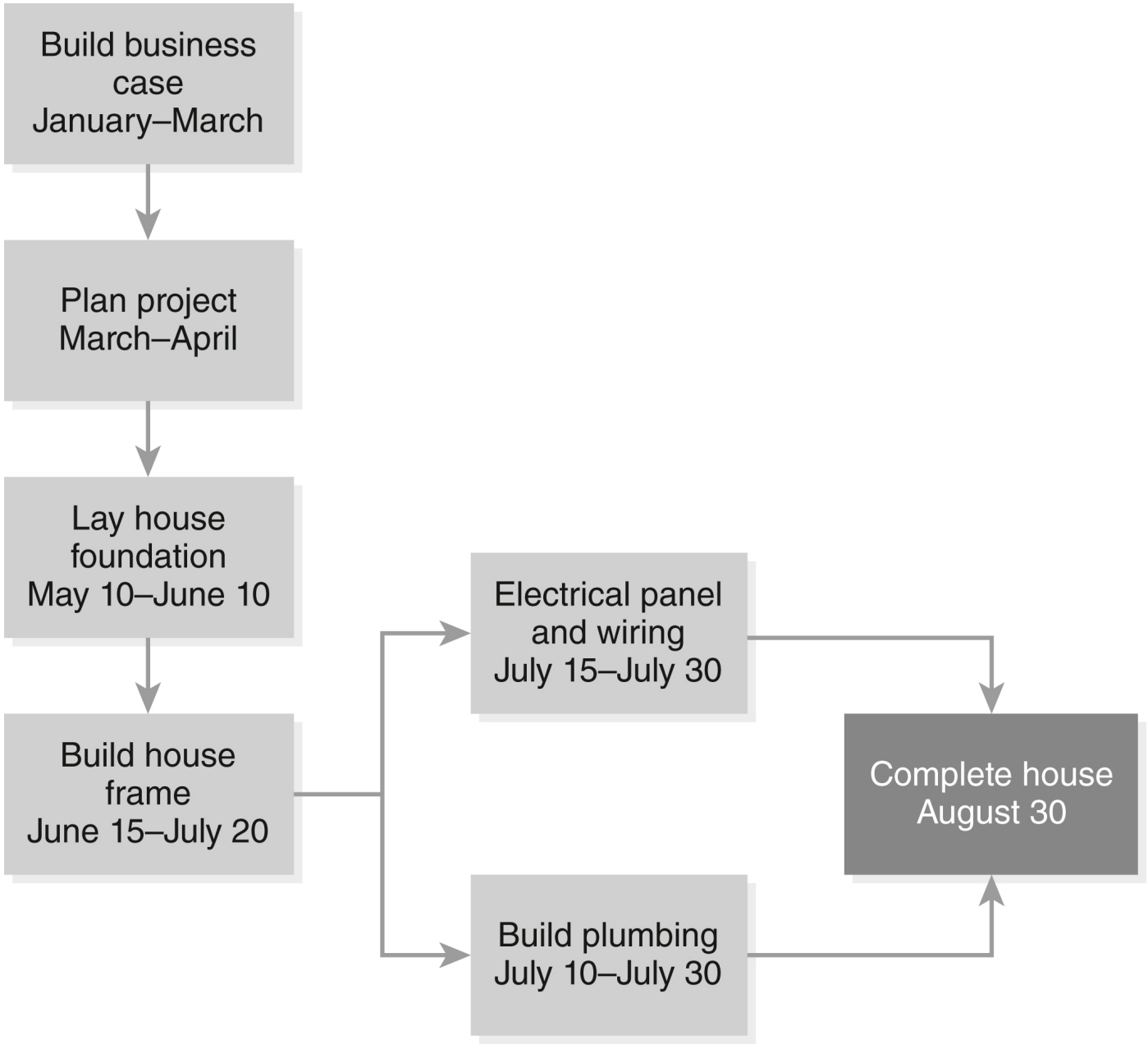


FIGURE 3.5 PERT Chart Example

Description

A PERT chart allows a planner to quickly and easily spot tasks that can be completed in parallel and those that must be completed serially. In critical path analysis, a planner can identify the shortest possible path through tasks. Provided sufficient resources are available for all tasks, the shortest path represents the shortest time to project completion. Anything that impedes timely completion of the tasks on that path will delay project completion.

With the critical path in mind, the planner can then focus on assigning resources and planning the timeline. As resources are assigned, the planner needs to consider whether resource-related delays or other issues will move another task onto the critical path. In many cases, this cannot be avoided because sufficient resources are simply not available or are not available at the appropriate times. An increase in the critical path may become unavoidable, but through analysis, the planner can optimize the critical path to become the shortest possible within the constraints.

The identified critical path is also important in the management of the project as monitoring adherence to the critical path is a great indicator of project process. Additionally, the project manager can pay particular attention to issues which may impact the critical path. As the project proceeds, the critical path may be recalculated based on the evidence on the ground.

There are several different formats for a PERT chart. Project management practices or tools determine which format will be used for a particular project. Most project management applications provide support for PERT charting. Many of the applications automatically change the view back and forth between PERT and Gantt charts or may combine characteristics of both.

Project Dashboard.

One other artifact that is useful when performing projects is a project dashboard, often implemented as a website, a wiki, or other easily accessible media. The purpose of a **project dashboard** is to provide one place that anyone can go to learn about the project. This “one-stop shopping” site will also serve other goals, such as providing team members with new information as it comes out in one central place and a consistent site from which to share information.

While all project dashboards are likely to contain different information that is relevant to each project, some key information is common to all projects:

- High-level project goals
- Project timeline

- Links to key project artifacts
- Link to a group email box or other common messaging platform, such as Slack or Discord, where people can ask questions that are not answered on the dashboard/website
- Relevant metrics (this is particularly useful as if the metrics are real-time dynamically updated from a database source leadership can get their data from this site rather than having to request information through email)
- Method to report issues with the dashboard or dashboard information

Phase 3—Implementation/Execution

The project deliverable is built in the **implementation/execution** phase. In this phase, project management oversight (usually a project manager) seeks to keep team members focused and delivering their assigned tasks on time and with expected quality. At the beginning of this phase, teams for specific deliverables are established. Project management oversight with the help of deliverable experts (plumbers or carpenters for a house or car engine experts if building a car) establishes the number of resources and the required skills to complete specific requirements.

Tasks

The teams required to execute the tasks to build the product were assigned during the planning phase. Team members now begin performing the tasks as assigned. Tasks must be completed in the specific order and time frames defined in the project plan. As noted in the discussion of Gantt and PERT charts, tasks can be assigned to be done one after the other, at the same time, or some other time frame. Sometimes as teams work, a project task length may change and so might task order. Hopefully this does not happen too often because it leads to delays as replanning is needed. The goal is to understand the tasks and their necessary order at the start of the project so that this doesn't happen. For example, electrical wiring cannot be built into the house until the framing is completed.

Effective project management oversight will bring experts onto the project just before they are needed and then roll them off once their section or tasks are completed. In each team, a minimal number of experts will be on the project before the implementation/execution phase, and a minimal number of experts will remain until the end of the project and the product is delivered to

the clients. These specific experts in their field help to plan the work breakdown structure (tasks) and remain in case any remediation is required. For a house, some additional electrical outlets might be needed that were not originally planned for, and the remaining electrician(s) will perform those final tasks, while the majority of the electricians can be released to work on other houses or projects.

Recommunication of the project scope, customer expectations, and timeline to all project team members at the implementation phase once again increases the probability of project success and builds project team cohesion. Projects can have a disadvantage if many different experts and people come together to work on the project but do not have a prior working relationship.

Communication

A comprehensive and flexible communication plan helps everyone understand what is to be delivered for the project to be successful. Obviously, the project dashboard is included in communications planning, but it is not the only method employed.

Meetings are one communication method that increases everyone's understanding of how the project is going, what the status is, and what risks or opportunities the project may be facing. Meetings are an excellent communication vehicle, but to be effective, they must be carefully planned. Meetings with management would be conducted differently from meetings with the execution team. The same is true for meetings with the clients.

All meetings need to have specific elements to be successful. Other elements may be included, depending on the type of project, personal preferences, and corporate expectations. The minimum elements required for a meeting include the following:

- Meeting invitations. A meeting invitation should include the following:
    - The appropriate individuals to the type of meeting to be held.
    - A meeting time when all, or at least essential, invitees are free.
    - A short but clear title for the meeting in the subject line. That way everyone will know at first glance what the meeting concerns.
    - A specific agenda, and if possible, assignments for who will speak to each agenda item. By providing an agenda for the meeting, invitees

will (hopefully) come prepared to the meeting to discuss items on the agenda relevant to them.

- Some method of note or minute taking during the meeting. Note taking has many advantages, from recording what was discussed so that everyone has a record of what was said, for people who could not be at the meeting, as well as to provide a historical record of decisions. Note or minute taking includes the assignment of action items that are expected to be completed by a date specified in the meeting. Recording such assignments in a formal document that is distributed to all meeting attendees provides a reminder to those assigned an action item.

  A product useful for note taking is Microsoft OneNote. It has a few advantages, including being able to interact with Microsoft Outlook. It provides one place for all meeting notes because many different tabs or folders can be set up in a single OneNote file. This means there is only one file for each project, and that one file can contain the notes or minutes for all meetings. There are many other commonly used products as well.
- A means to conduct the meetings virtually, without needing all attendees to be physically present. This is increasingly important as more teams are global or otherwise scattered across long distances, and gathering all attendees physically may not be possible. Several products are available to facilitate this requirement, such as Skype, Zoom, TeamCenter, and WebEx. Many of these products provide extra features, such as a virtual whiteboard, the ability to have a video meeting, and the capability to record the meeting.

Team meetings may focus on specific tasks related to the overall project execution, the meeting may be more narrowly focused to work with specific experts, or the meeting may address a specific issue. In regular team meetings, team members provide information on the progress of their assigned tasks. The team members may also report any new risks or issues. These new risks or issues are discussed in the meeting to determine what possible things could be done to avoid a risk or resolve an issue. All risks and issues are formally captured in a format such as a spreadsheet, on paper, or some other trackable system.

Client meetings often are built around a presentation with information on project progress, risks, and any issues encountered. When a client building a new house meets with the contractor (the person overseeing the work and can be called the project manager), the client wants to hear about progress on the house. The contractor might report on completion of the foundation and framing. The client and the contractor may meet at the site to see the progress. At the same time, the contractor would discuss the next tasks or milestones to be completed, such as the plumbing or the electrical wiring. During these meetings, the client or the contractor (project manager) may bring up some suggestions for changes or improvements.

Meetings with management or corporate executives often require some type of formal presentation or documentation. As with the client, the presentation, whether formal or informal, usually addresses project progress, risks, and any issues encountered. Some meetings will be informational, while others will be asking for specific decisions regarding the project. Status reporting provided at the meeting focuses on attendees and provides a historical record of the project status throughout the execution phase. The documentation can be in any type of format; however, it will include the following:

- High-level review of project goals and scope.
- High-level review of requirements and/or tasks.
- Summary of requirements and/or tasks asks completed.
- Next goals.
- Metrics (as appropriate to the project) showing overall progress to date and possibly projected progress. Metrics can include information on finances or team staffing levels or many other items.

If the meeting requires the participants to make a decision, providing additional information prior to the meeting is beneficial. Such information would include the status as well as other information pertinent to the required decision. Ideally, in the meeting itself, the majority of the time will not be spent covering information concerning the decision but rather debating and analyzing the results of the decisions to be made.

While meetings help communicate project goals and progress, other communication methods help keep the project on target. Messages (such as email messages) discussing project progress help keep the clients and team members engaged between meetings. Personal communications through phone

calls or other methods to discuss risks, issues, or alternative approaches to various tasks help the team get to know one another and increase inter-team trust. Hallway and water cooler discussions are also a great informal communications method but need to be used with care and should not take the place of formal communications processes.

Phase 4—Monitor and Control

Monitor and control, although given as its own phase, occurs simultaneously with phase 3, execution. Monitoring and controlling a project is difficult because it entails bringing people, physical resources (nails, wood, plumbing pipes to build a house; servers, network connections, computers when building software) together to create a product that meets the client requirements, is produced within a budget, and meets a specified timeline. Yet, when it is performed correctly, a project meets all three of these requirements. Such success is achieved through effective monitoring and control.

Monitoring and controlling a project include the following:

- Controlling the schedule
- Establishing an operating rhythm
- Tracking all tasks at a granular level
- Performing regular audits of the quality and conformance
- Capturing and managing risks
- Recording and resolving issues
- Monitoring the project for scope creep
- Monitoring and controlling finances and the budget
- Managing stakeholder expectations

Maintaining the Schedule

Controlling the schedule is one of the primary project manager tasks throughout the project. During the earlier phases, the task is fairly straight forward. But during the implementation phase, the task becomes considerably more complex. Not only does the project manager need to be effective in ensuring that everyone understands and complies with the schedule and that resources are made available at the right time, but the project manager also needs to

anticipate challenges that may threaten the schedule. These challenges can be both internal and external to the project.

Operating Rhythm

The operating rhythm for a project team is an established pattern for how the team functions. Teams without an established operating rhythm often jerk from one task to the next with awkward pauses in between, or rapid sprints to keep pace with what should have already been done. A team with a solid operating rhythm moves smoothly from one task to the next and greatly eases project management.

Consideration of the operating rhythm for a team includes a broad variety of factors, including how team members communicate, how they hand tasks from one member to the other, and what the daily and weekly expectations are for efforts and efficiencies. Establishing an effective operating rhythm includes setting regular meetings to capture status, including any issues that may have come up, and where and how progress will be reported.

As the project manager and other project leads establish a rapport and regular communications with team members, with management, and with the client, they build expectations and eliminate surprises. This creates an environment where work and tasks flow smoothly. While disruptions to the rhythm may occur from unexpected circumstances, weather, unexpected changes, teams that have established solid and effective rhythms quickly overcome the challenge and fall back into their established rhythm, while teams that haven't established an effective rhythm tend to crumble.

Task Tracking

Monitoring requires methods to track all tasks at a granular level so that progress (or lack of progress) can be easily seen. The lack of progress alerts project management to bring the project back on track.

Quality and Conformance

Part of the implementation/execution phase is to ensure that each task completed meets the stated requirements and quality. Each expert specialty is expected to test their work. Electricians are expected to perform their work to high standards and test each completed piece of work as they complete it. The same is true for software development personnel. Anyone who write software is also expected to test it to make sure it does what it was expected to do and to the stated technical specifications (such as performance and other expectations).

The building trades have many regulatory requirements. As a particular trade is completed in a house, often a city or other government body will send a representative to approve the work. This is done to ensure all house building is performed to expected standards, which in turn decreases the probability of the house having issues, such as an electrical fire because the electrical work was not performed appropriately. Regulatory trade oversight is a form of auditing.

While no specific regulatory trade oversight representatives will audit (test) software projects for quality and conformance to the original requirement's attributes, many software projects have separate quality auditors on the team, called a test team. These quality and conformance team's sole function is to test, evaluate, and report back to the expert team leads and project management on whether modifications are needed for the task completed to meet expectations. Testers use formal documentation (written, electronic, visual [such as pictures], or other means) to capture the results of their audits or testing. Any deficiencies need to be quickly addressed.

Risk Monitoring and Management

Risks must be captured, mitigated, and tracked. Risks can have a negative impact on a project. If this occurs, they then become issues. Risks to a project should not be allowed to develop into issues.

Recording and Resolving Issues

Issues are recorded and must be resolved wherever possible. If an issue cannot be resolved, its impact will need to be discussed with the client, project team, and management. As can be imagined, an unresolved issue that is large enough can cause a project to be canceled. This can be anything from funding to a (currently) unsurmountable technical issue.

Scope Creep

Monitor the project for scope creep. Processes are put in place at the beginning of a project to manage any scope creep. Scope control is performed through careful requirements gathering along with processes to review any requested changes to project scope. Such processes are detailed in how a scope change will be documented, who will participate in reviewing any scope change, and what steps will be taken if a scope change is approved.

Part of a scope change process (sometimes called change control process) is identifying what the ramifications of a change are, whether that is more time or increased budget for the project. In some rare cases, a change may

decrease the time a project takes because the change generates a more efficient way to meet a requirement.

Finances

Monitor finances and the budget throughout the project by capturing expenses in a timely manner so that runaway expenses do not jeopardize completion of the project. If the budget is exhausted before the project is completed, the project will need to request more funding, or it may be stopped for now. Neither situation makes anyone involved happy.

Stakeholder Expectations

An aspect that is not often discussed is the need to manage **stakeholder expectations**. As noted, if clear requirements are gathered and approved by the clients, then there should be no questions of what the final deliverables will be. However, as time goes on and product deliverables (requirements) are reviewed, clients often think of new additions or changes to what has been implemented (created). As with other parts of the project, the best way to manage stakeholder expectations is to ensure frequent communication of progress on project deliverables, linking the progress to the original requirements. As well, a good relationship with the stakeholders to the point that they are comfortable bringing up issues (which can then be addressed quickly) helps greatly to control stakeholder scope creep and increases stakeholder understanding of the project progress. It also helps to create a more collaborative (team spirit) environment between the stakeholders and the project team.

Often clients realize the stated requirements are not exactly what they would prefer, regardless of agreeing to them before development was started. Stopping to capture updated requirements and restart development causes project budget overruns and delays. Once again, frequent communication with clients reporting on progress and reminding them of the advantages of the project based on original requirements helps focus everyone on the final project product. Future changes to the current product under development can be made once the clients have a chance to use the completed product to evaluate possible improvements. Changes will always be wanted either due to changing expectations, technology, or business needs.

Phase 5—Project Close

At the end of a project, or **project close**, the product functionality and stated technical requirements are tested to prove that the product does what the client and the team agreed the product would do. The product must meet expectations as defined in the requirements.

Delivery and Acceptance

Once a new house has been completed, the homeowner reviews the house with the contractor. All is as they expect, and they are feeling quite excited because now they can finally move into this new house, which has been a long-time dream. A formal document is used to record acceptances and deficiencies. During the review, the owner notices a few minor items such as a wall that is scuffed and needs cleaning, a lack of caulking around one of the bathroom sinks, and a back porch light that is not working. The contractor agrees to resolve these final items within a week, and the excited new homeowner starts to plan the move into the new house. Once the deficiencies are corrected and signed off as completed, the owners can move in and start living in their dream home.

Every project validates that all requirements (including changes made to requirements during the project) have been delivered as part of the project execution. No requirement must be left undone. If during project close a requirement is found to have been forgotten, a decision is made, with the client, as to whether the requirement can be left undone or incomplete; whether the requirement will need to be done and therefore the project close postponed; or whether the requirement can be done at a later date under a new project with the current project allowed to be closed out.

Project Review

Part of phase 5, the close of a project is to perform a project review. During the close project review or **after-action review (AAR)**, team members and clients discuss what was and was not effective during the project. To be effective, a review needs to have everyone involved with the project collaborate to provide input in open and honest communication about what went well and what did not. For those items that did not go well, finger-pointing and blaming is to be discouraged because it does not contribute to solving problems. How these reviews are conducted is important. If completed in an adversarial manner, the participants will get defensive and not contribute significantly. Rather than being a blame session, the purpose of the AAR is to investigate what went according to plan, what didn't, and how improvements could be made to either ensure the plan is met in future projects or that the type of

obstacles encountered in pursuing the plan are accounted for and avoided in the next plan. This requires open and honest communication. Everyone participating in the AAR should be able to contribute freely without fear of condemnation or reprisal.

In anticipation of the AAR, everyone should be encouraged to write down any issues or concerns encountered as the project progresses. Even when these haven't resulted in problems, changes, or failures during the course of the project, knowing and remembering what these issues and concerns were, and how they were resolved if they became problems, can help in planning and preparing for future projects. Organizations that accomplish this successfully grow to become better organizations.

The lessons learned from this review are documented and stored with all other project artifacts for historical purposes. Lessons learned have an added advantage because they are used in future projects to increase project success. In particular, resolutions to problems discovered during the project are likely to be useful in future projects.

Maintenance Requirements

One of the last parts of any project is to establish any maintenance requirements. Does the house furnace require regular servicing? If so, how should the servicing be done and on what schedule? How are the floors to be kept up? The same is true for software products and services. Are there expected regular reviews of log files for warnings or issues? Should the database have some specific types of maintenance? When a project ends, many things pass into maintenance. All maintenance items are to be documented, the necessary maintenance reviewed with the client, and the client should formally state an understanding of the maintenance requirements for the product.

Finances

Finally, the project finances are closed out, the final invoices paid, and a final accounting report created. A final review is done with the client and management. All accounting reports are also stored with the project documentation.

Celebration

Once all the above have been completed, a celebration is held. This celebration is to celebrate team achievements and the delivery of the project product, policy, or standard; reward team members for a job well done; provide some

relaxation before the next project; and mark the end of the project in such a way that all stakeholders and team members recognize the project closure.

Project Management Summary

Project management phases, as shown in **Figure 3.6,** include initiation, define, design, implementation or execution, monitor and control, close and maintain.

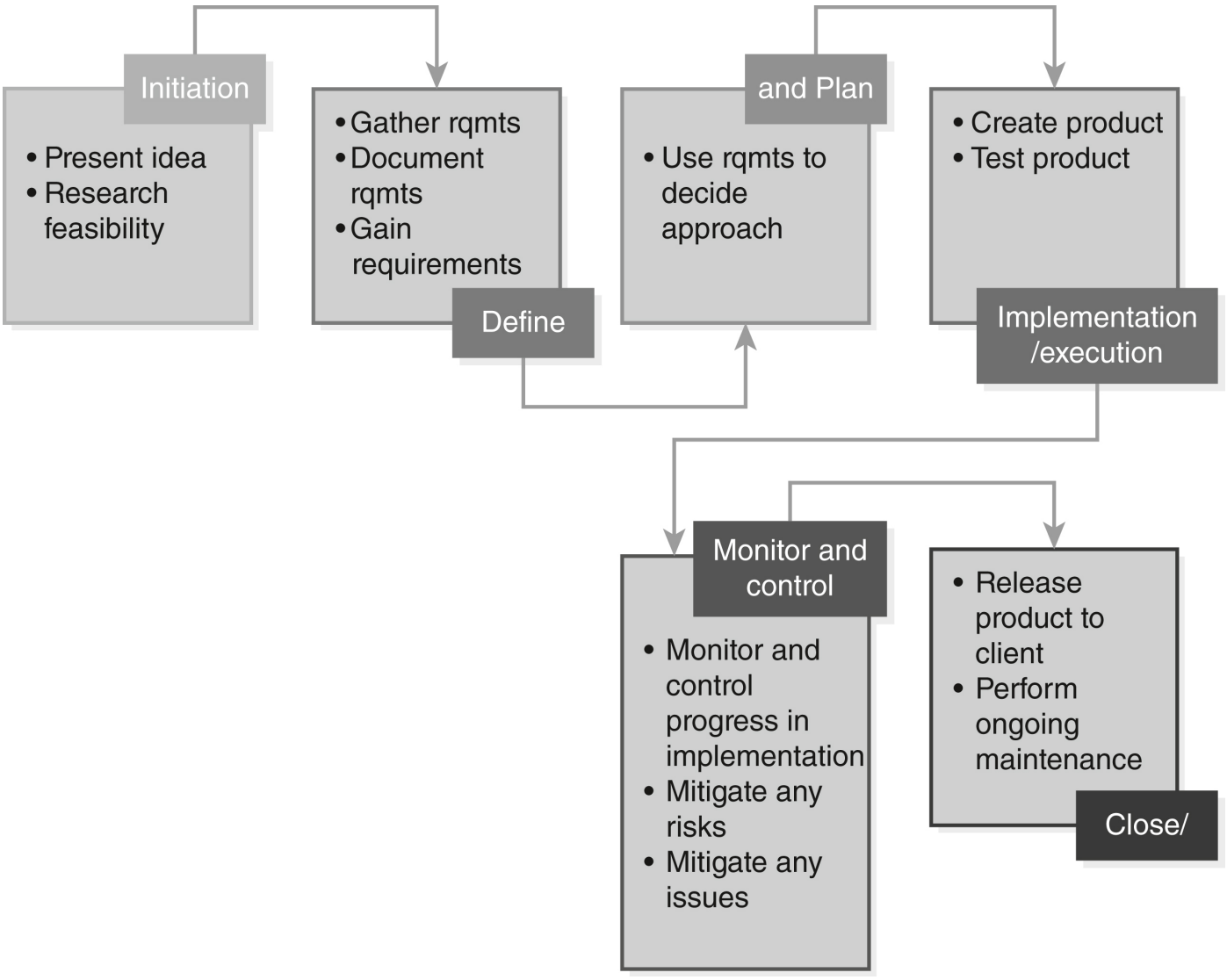


FIGURE 3.6 Project Management Phases

Description

Effective project management increases the probability of project success. It creates effective teams as well as effective client and execution team collaboration. It uses several different tools, including work breakdown structure, project plan with tasks assigned and dates for tasks to be

completed, and communication methods, to increase the probability that the date the project needs to be delivered is met.

Program Management

Program management is a strategic activity. It is focused more on the broader management of resources, planning, and coordination rather than the direct accomplishment of specific tasks.

Program management oversees many related projects in a way that provides benefits for and from all projects greater than the benefit that is gained for or from any one project. A project's primary goal is to deliver a specific product, process, or standard; a program's primary goal is to deliver the benefits accrued by the set of projects. Often a program will have multiple goals it is trying to achieve by managing the set of projects within the program. Bringing multiple projects under one high-level oversight may occur because a single, large project had to be broken down into smaller, more digestible projects, or it may be to share resources working more efficiently toward common large goals. On the other hand, the projects may have nothing more in common than that they all need to be monitored and managed, and it is easier to do it as a group. A project is an effort with a start and an end, but a program may continue indefinitely if there are projects within the program portfolio. Over time, projects will be finished, and new projects will appear, but the program will remain as ideas for new projects or desired changes to existing systems and services are proposed. Or the program may follow a product life cycle.

Strategic Approach

The strategic focus of program management includes managing the set of projects in the program. This would include ensuring that projects enter and leave the program as required, reviewing potential changes in scope, timing, and resources of projects in the program, and analyzing the progress of projects in the program.

As each project is established, the program is not concerned about the particular details or the specific requirements. Instead, it is concerned about the project outcomes, definition, and scope. These define the bounds of a particular project. The program, working with product and portfolio management when appropriate, will ensure that the projects mesh together well and do not overlap in their efforts. The program will maintain uncovered needs that are out of scope for a specific project and assign those to others as

needed. The program will also ensure that the project definitions cover the needs of the organization.

Program management coordinates collaboration between projects, monitoring interdependencies and adjudicating the allocation of shared resources. Where there are interdependencies, it makes sure that the affected projects are aware and are communicating and coordinating effectively. When resources are shared, program management prioritizes needs and also works with project managers to shift the timing of requirements of overall optimization of resource use and project scheduling. Program management will monitor changing needs for resource allocation between projects.

Program management establishes common metrics and communications requirements for all projects. This increases the ability to manage across multiple projects.

Program management does not need to perform day-to-day oversight of projects because that is done by project management teams. Project monitoring is done by collecting information about a project's current status as well as improving future project performance. Often, a program will have formal periodic reviews of each project or of the set of projects. All projects within a program are expected to provide regular, standardized reports to the program management office.

Program Sponsorship

Just as with a project, all programs have a sponsor, a person or persons who support the program. Sponsors support and approve (authorize) the project based on the business case. They will appoint someone to see that their interests are met, socialize the program with senior stakeholders, and review the program on a regular basis to track program progress in meeting the desired strategy and objectives of the organization. The person overseeing the portfolio for the sponsor(s) is more involved in day-to-day program activities and is responsible for reporting back to the project sponsor and senior stakeholders.

Program Managers

All programs have a program manager who is usually a senior project manager by experience because they must be experienced in leading many large projects at the same time. Program managers communicate processes that the program expects to be followed to the project teams. This creates a unified

approach by program project teams. A key responsibility of the program manager is to ensure all projects follow similar, if not exactly the same, processes to make sure the projects (and their interdependencies) are coordinated effectively. The program life cycle is determined by the program manager, in collaboration with the senior responsible owner and project managers.

Governance (control) processes, budget processes, monitoring, change processes, communication methods, risk, issue and opportunities processes, and resource utilization are all overseen by the program manager. The skills required for program managers include being able to engage stakeholders, effective communication skills, change management skills, analytical and integration skills as well as leadership skills. Having experience in the field relevant to the program aids a program manager's effectiveness.

Program managers do not provide detailed oversight of projects or project managers. They are looking for the outcome, not the details of planning, execution, or project closure. Project managers will communicate issues and risks that may impact delivery of a project that in turn will impact success of a program. Collaboration between program and project managers is important to the success for both projects and programs. Program management, through high-level plans, tracks the interdependencies and progress of projects or subprograms to ensure the program meets expected organizational deliverables.

Program Management Office

Many organizations establish a **program management office (PMO)** as a function to manage their programs and projects. A program management office may exist for the length of a corporate initiative, or it may be a permanent office. Program management offices may be established to manage projects that may or may not be related.

When program management offices are established to centrally manage projects that are not related, the primary goal is to provide a strategic perspective to the management of programs and projects. The benefits include establishing consistent project management processes, sharing resources for planning, budgeting, management, and reporting, and distributing resources across programs and projects.

The strategic direction of a program may change as subprograms or projects are completed or as the organizational strategic directions change. The outcome of a project may indicate a change is indicated in the program's

approach to meeting a goal. The program management office oversees such changes in direction as the program management office tracks how the projects are performing and whether they are still contributing to the corporate organization goals of the program.

By using consistent project management processes across an organization (company), the organization benefits from being able to have project managers and team members move smoothly from project to project. This reduces “drag” in the organization, which would be incurred as the members adjust themselves to moving from one style of project to another.

The consistency of approach also allows for the consistent collection of reports and other data from the project. That in turn makes it possible to make consistent comparisons across different projects. It can answer questions about the productivity of teams and locations, the resource consumption rates, the number of issues arising, differences in costs, and many other factors. This also allows for consistent risk management across the organization.

Despite the value of using a consistent approach across all projects, in some organizations each project is allowed to select a different project management methodology based on perceived value to the projects. While this can be justified in limited instances with special types of projects, in general, it is not a good idea. If it is allowed, then each project team would still need to ensure that they follow the best practices of whichever approach they are using. Another approach where teams feel they need different approaches would be to accommodate the PMO’s processes with acceptable variances within the core processes.

A centralized PMO also allows for the application of resources to projects in a manner consistent with strategic objectives. The office can manage to the priorities of the organization and, as those change, adjust resource assignments accordingly. This management of resources among projects is particularly important when it comes to scarce or expensive resources. Ideally, these will always be fully engaged, but not overcommitted. Through strategic oversight, the program management office can adjust to the planning and scheduling of projects to ensure that specific resources do not become a bottle neck or do not sit idle waiting for a need to arise.

Because one of the tasks of a central PMO is to develop and maintain the project methodologies and expertise within the organization, the skills of the project managers assigned to the various projects are developed through the PMO. Skill development may include training, conferences, professional development opportunities, and mentoring. A part of the developmental effort includes matching the skill level of project managers to the level of skills

required for any project. Over time, that allows for project managers to increase their capabilities and allows a career path for members of the PMO without them having to seek out new opportunities elsewhere.

Organizations have different methods for organizing how their PMO and program managers work and report. Different models work for different organizations, with no one model being the preferred, though each model will have a supporter who will argue for that model.

The first model to consider is the **matrixed organization**. In this model, the PMO and technical managers in the organization have distinct reporting structures as can be seen in **Figure 3.7**. Project managers report directly to and are controlled by the PMO, and each of the technical managers has a team of specialized technical members. For example, there may be a team of system architects, a team of software developers, a team of marketers, and a team of testers.

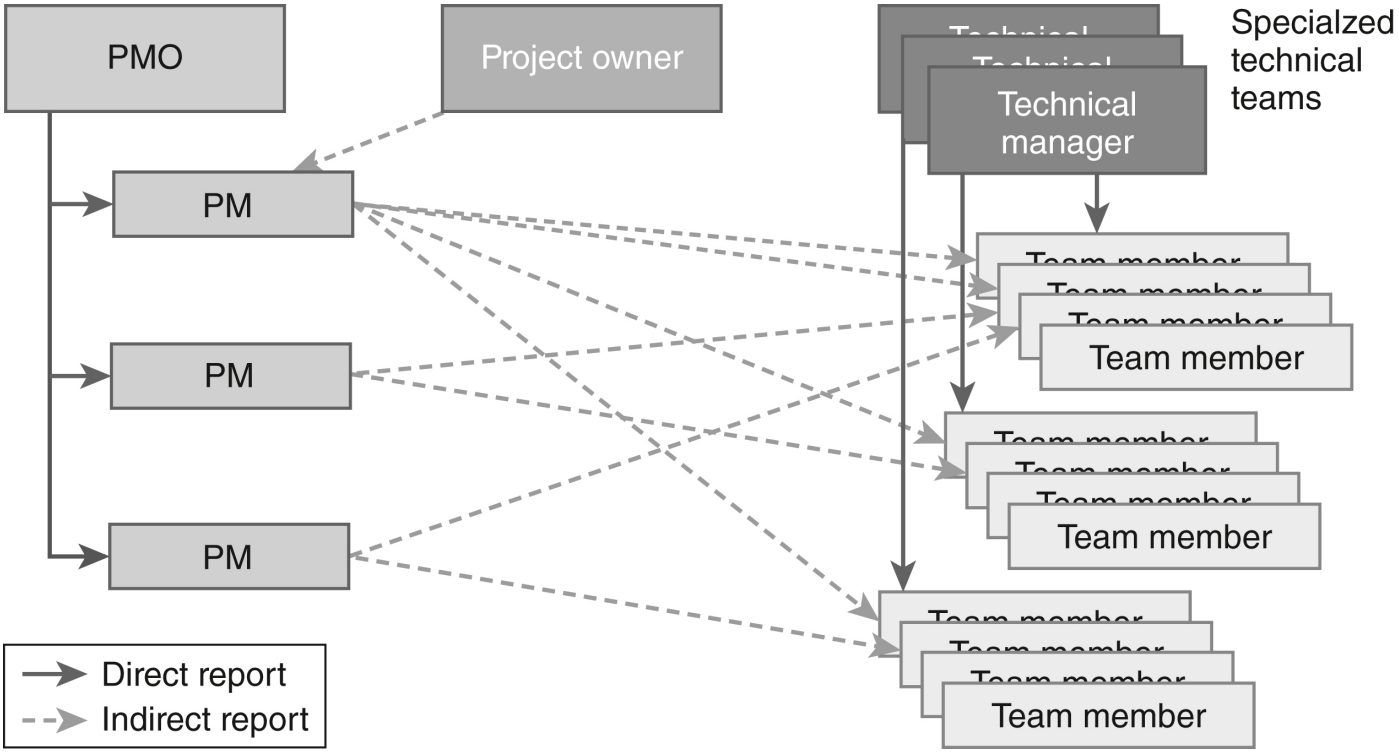


FIGURE 3.7 PMO in the Organization, Matrix

Description

This structure allows managers to focus on developing the technical skills of their teams. The technical managers also have responsibility for personnel issues.

As each project is developed, a program manager is assigned to the project owner with an indirect reporting relationship. A project team is then drawn from

the needed specialist teams. This team reports indirectly to the project manager. That is, they take their instructions involving the project from the project manager but are hired, trained, supported, and fired by their technical manager. The project team changes over time as project needs change.

In a matrixed approach, a technical manager may have members of their team supporting many different projects at any one time. Any team member may be assigned to multiple projects as well. The technical manager worries about the individual projects only so far as the projects involve their team members, and technical managers provide support for the team member's needs, ensure the team members have the needed skills, and manage team member availability. Team members themselves perform their tasks for their assigned projects, balancing their time across various assigned projects (with project managers resolving conflicts) and looking to their technical manager for their other needs.

Challenges and conflicts can arise out of the matrix organizational model. In some cases, functional experts may feel a tension as they feel pulled between the needs and demands of the technical manager and the project manager. In other cases, conflicts can arise between the needs of different projects where the project managers have conflicting needs and priorities but have not resolved these issues among themselves. Sometimes conflicts over roles arise between the technical managers and the project managers, or the technical manager may not feel that the project manager has sufficient technical knowledge to make decisions about the project. In these types of organizations, clear and open communication and trust between the project managers and the technical managers is a real need.

Often it is useful to have a technical expert (who will act as a technical lead) assigned directly to the project manager where the project manager and the technical lead work together as a team managing all their projects. This project management team then has additional credibility and capacity.

Another variation of a PMO organization is the assigned model, as shown in **Figure 3.8**. In this model, the project managers all report to the PMO, as they did in the matrix organization. The technical managers have teams with more broadly varied skill sets. Projects are assigned directly to the technical managers. They in turn have a project manager assigned to them as an indirect report. A technical manager may have several different projects assigned to them.

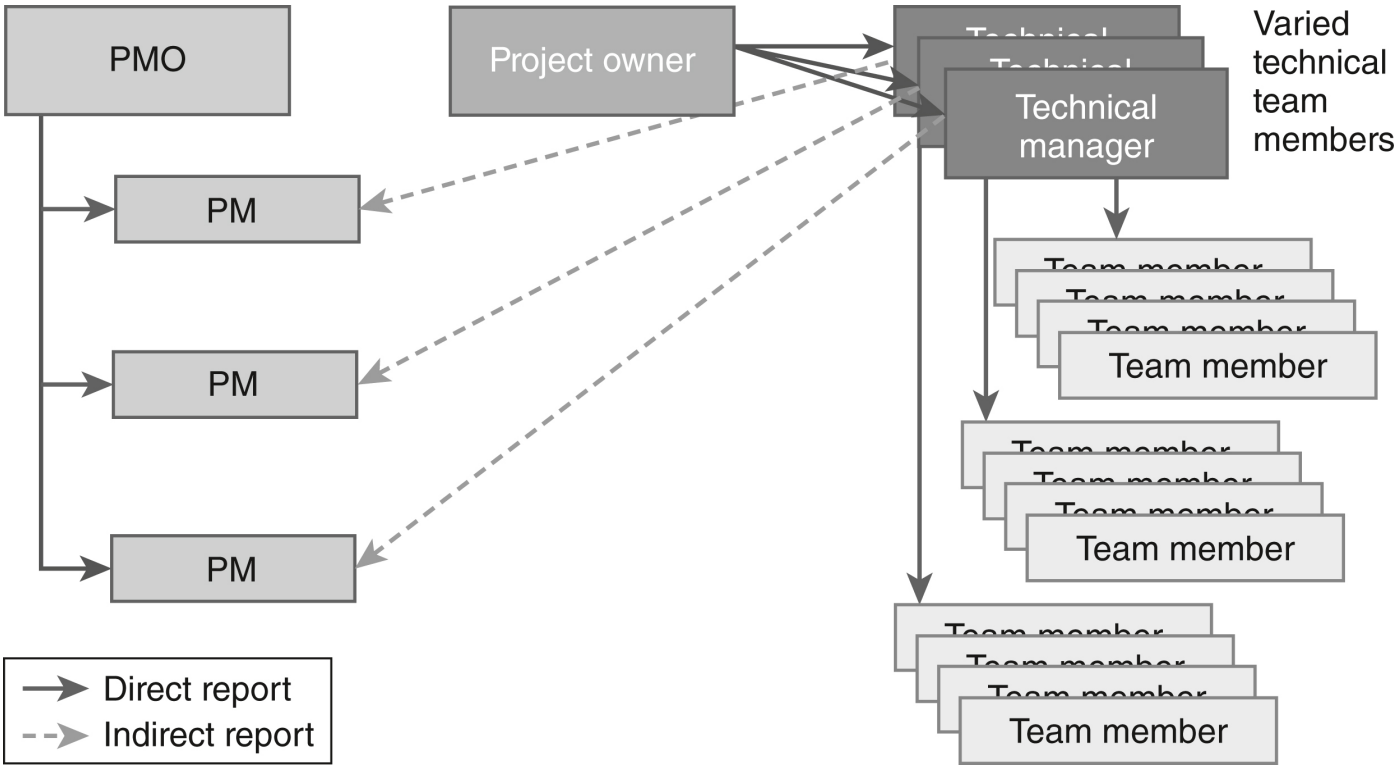


FIGURE 3.8 PMO in the Organization, Assigned

Description

The project manager works with the technical manager to manage the project, with the project manager often handling aspects such as budget, schedule, reporting, and coordination. The technical manager often handles priorities and resources as well as personnel and other issues. Both share responsibility for the completion of the project, but ultimately it is the technical manager with the greatest share. In some cases, specialized team members may be shared from one group to another.

As with the matrix organization, this model is not without its challenges and conflicts. In this model, the project managers may feel torn among the needs of the PMO, the project, and the technical manager. There can also be a tendency for the technical manager to become the de facto project manager with the project manager being reduced to the role of project coordinator. If the technical manager has the proper skillset, this type of relationship can work well; however, the project manager and PMO may not be satisfied with the arrangement. In this latter case, the technical manager often experiences challenges between technical solutions, the budget, and the expected delivery date. In the ideal situation, the project management responsibilities are maintained by the project manager, while the technical manager works with the project manager.

A third model is the embedded model. In this model, the project managers are assigned to report directly to the technical managers. They also report indirectly to the PMO. This indirect reporting link is used to ensure that program managers within the organization all use the same project management and reporting methodologies. The direct reporting link ensures that technical managers have operational control of the project managers whom they are depending on to complete the assigned projects. As shown in **Figure 3.9,** having the project manager reporting to the technical manager can cause conflicts as a project proceeds.

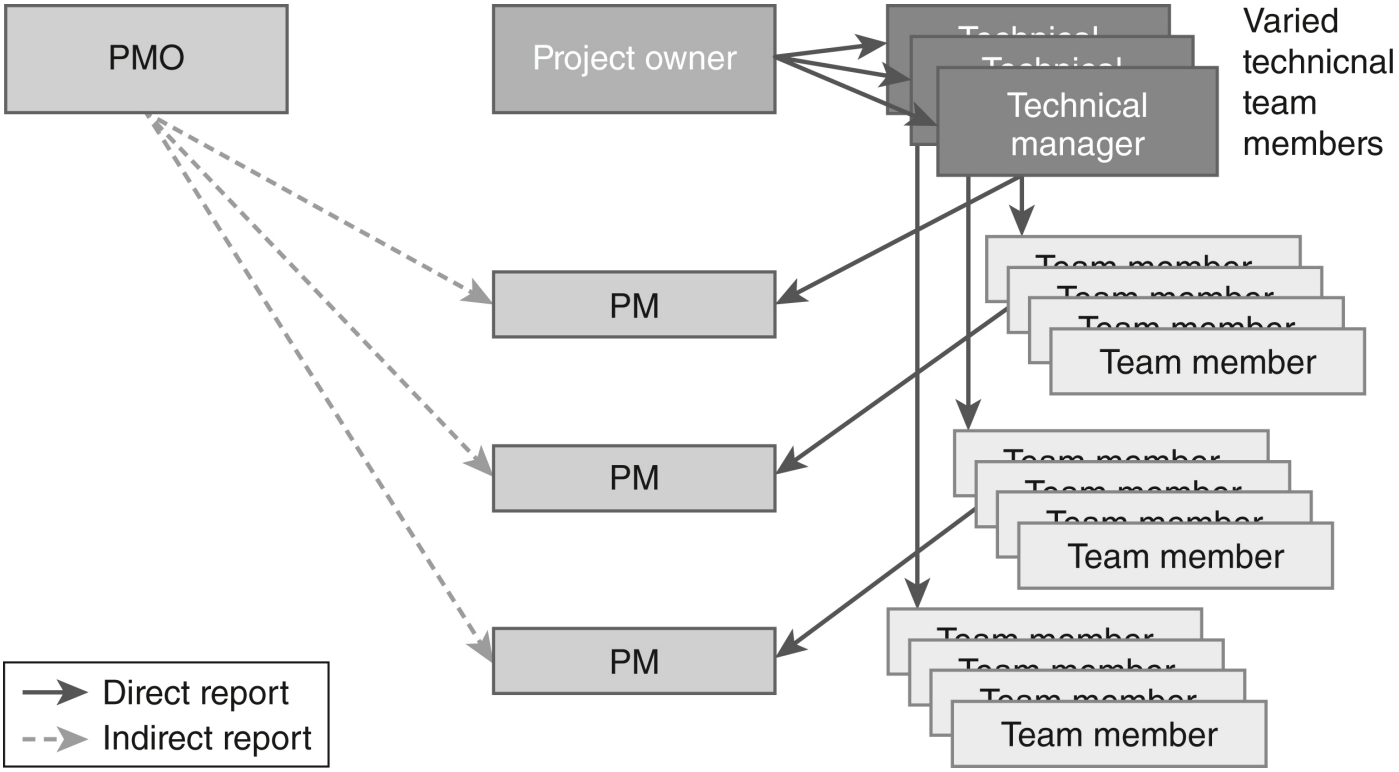


FIGURE 3.9 PMO in the Organization, Embedded

Description

The project manager's goals and mandates can be at odds with the technical team's abilities and approaches. The project manager position can also be reduced to more of a project coordinator role, as with the assigned model. Technical managers, however, tend to favor the embedded model over the assigned model because they have more direct control. In the past, this has also led to a focus on technical updates over delivered quality, which resulted in unhappy clients.

The other option is the traditional assignment of responsibility for both project management oversight and technical oversight to a single management

structure. This type of structure requires that management be skilled in both project management and technical management. It is rare for one person to have sufficient project and technical management skills, which can lead to challenges on a project using this type of structure.

There are alternative approaches to how to distribute responsibility for project management, with the debate over what is the best method being frequently discussed and revisited.

SUMMARY

Project and program management are two areas that are very tightly coupled together but differ in their focus. While project management is focused on the tactical aspects of getting a project begun, planned, and completed, program management has a strategic focus where it looks at oversight across multiple projects, resource allocation across projects, consolidated reporting, and consistent practices. The two work together at the micro and macro scale to ensure the right projects are successfully delivered at the right time.

Whether a particular software system is completed in a single project or as a program is determined based on the scale and complexity of the system. A software system may also be integrated into a program as a single project or as multiple projects, where the program also include other projects related to the system being developed.

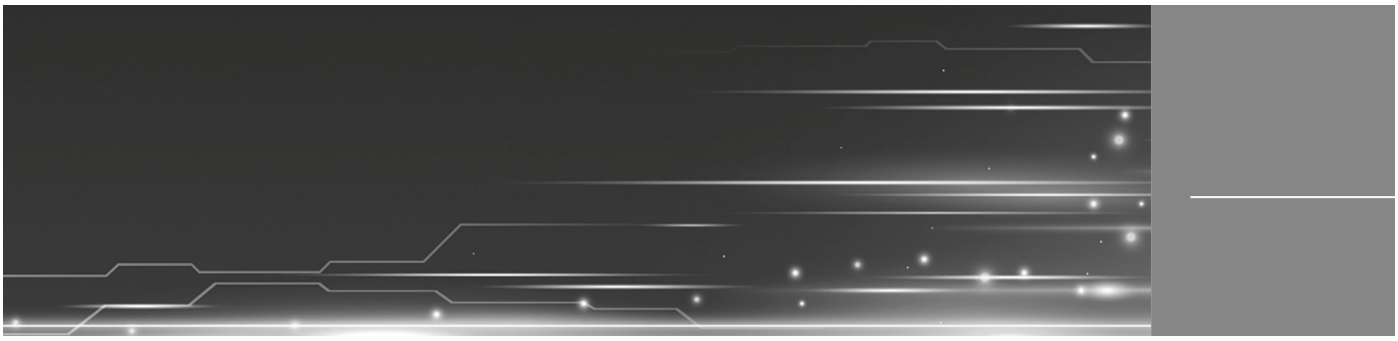
Many organizations use a PMO to coordinate their programs and projects and to allocate resources, including qualified project managers as needed across the organization. How this office works with other departments in the organization can vary considerably and needs to be carefully chosen.

KEY TERMS

after-action review (AAR)
business case
design and plan
Gantt chart
implementation/execution
initiation
matrixed organization
metrics
milestone chart
monitor and control
operating rhythm
program evaluation review technique (PERT) chart
program management
program management office (PMO)
project artifacts
project charter
project close
project dashboard
project management
project plan
scope
scope creep
stakeholder expectations
tasks
work breakdown

Reference

Project Management Institute. *A Guide to the Project Management Body of Knowledge*. 6th ed. Newton Square, PA: Project Management Institute, 2017.



© d1sk/Shutterstock

CHAPTER 4

Process Management

CHAPTER OBJECTIVES

- Identify the importance of established, documented, and communicated processes.
- Document processes using appropriate methods.
- Apply techniques for process management and process improvement.
- Discuss how properly managed processes contribute to cybersecurity.

Introduction

Processes are often defined as anything an organization does on a planned repetitive basis. Everything else is either a project or a mess. Note, we don't use the terms "emergency" or "disaster" because the reactions to those can be planned. **Process management** involves the management of people, resources, and flow in order to optimize the effectiveness, efficiency, and quality of processes. In this context, effectiveness is defined to mean that the process accomplishes the given tasks in a complete manner and involves the right people. Process efficiency refers to the minimization of the time or resources needed to accomplish the process. Process quality is more challenging to define because it refers to both the effectiveness and efficiency as well as other factors related to the outcome of a process.

Managing Processes

Identifying Processes

In general, the first step in managing processes is knowing what the processes are. Every process has a goal or objective of what the process will accomplish. For example, a typical human resources process would be to onboard a new employee. The goal of this process would be to have a new employee's paperwork and training completed, with assigned workspace and tools, and all other elements needed for the new employee to begin productively working for the company. A systems process example may be the implementation of a new server, where the goal is to have the server physically installed and connected, the operating system set up, including IP addresses, users, and security, and any applications installed.

Understanding Processes

Processes are composed of steps, which are organized into flows. Each process step is a single discrete action, procedure, or set of actions or procedures that are done to accomplish a part of the overall goal. Often the steps in a process must be executed in a specific order. This effectively means that prerequisite steps must be executed prior to continuing to the next step within the process. Some steps in a process are decision steps. Based on specified inputs, different subsequent steps will be executed next.

For processes, tasks, items, or groups of things traverse the flow as an entity. These are known as **transactions**. An example of a human resources transaction would be the hiring task for a single employee. The entire hiring task for a single employee is independent of the hiring task for another employee, and the hiring task for any one employee is not considered complete until all steps have been performed for that employee. A bank deposit is another example of a transaction. A process is completed for each deposit, which starts with the handover of cash or checks and ends with the amount being credited to the depositor's account.

BOX 4.1 Email Queues

Many people never think about the different queues involved in processes. A good example of a common queue is an email system. A worker may receive emails throughout the day requesting participation in different processes. These email requests are stored in a common queue—the inbox.

Periodically throughout the day, workers will switch their focus to their email inbox, and then start reading the emails, usually beginning with the most recent (stack). As workers read and react to the emails, they are removing an entry from the queue and moving the transaction through the process. Unfortunately, this approach has several problems. Obviously, the ordering of working with transactions is off because the earliest transactions in the process are not addressed first.

Then there is a prioritization issue. All emails receive the same priority, regardless of which process they refer to. The result is lower priority processes may receive as much or more attention than high priority processes.

Fortunately, there are ways to address these issues.

Another common feature of processes are **queues**. Queues are where the tasks or transactions in a process wait for an action to be performed or a decision to be made. Queues can be formal or informal. For example, a queue may be an inbox on someone's desk, or it could simply be a pile of papers somewhere. A queue can also exist electronically. Some types of queues are more efficient than others for specific uses.

Queues may be ordered or unordered. In an ordered queue, tasks are handled in a specific order. For example, **first-in first-out (FIFO)**, **last-in first-out (LIFO)**, or **highest priority first**. A FIFO queue is the normal expected line at a cash register or the line to go on a ride at an amusement park. The customers line up in the order they arrived, and those who arrived first are handled first. A LIFO queue is also known as a stack. The common example given is that of a plate warmer at a restaurant where plates come out of the dishwasher and are placed onto the stack of plates. As plates are needed for orders they are taken off of the top of the stack. Highest priority first is often seen at airports, where families with small children get on the plane first, then the high-level frequent flyer members, and so on. Queue ordering may also be mixed with priorities considered first and then arrival times. A **preemptive queue** is one where a higher priority transaction will cause processing to stop on lower priority transactions until the higher priority transaction is completed.

This happens in loading airplanes when a high-level frequent flyer shows up at the gate as all of the common passengers are being loaded. The frequent flyer goes through a separate lane and boards ahead of everyone who has been standing in line and waiting.

A queue may have a limited length. In this case, any transactions arriving after the queue is full are dropped. When transactions are processed at a fixed rate, this is called a “leaky bucket.” Another interesting queue is when, if the queue is full and another transaction arrives, instead of it being discarded, a transaction that arrived earlier is discarded. This can happen for instance with a circular queue.

In an **unordered queue**, there is no required order for the tasks. The queue processor may do a random selection of the next task or perhaps favor the easiest, the nicest colored, or the one with the bribe (well, maybe that is a means of applying a priority).

To manage processes then, we need to manage how tasks or transactions flow through the actions, how and by whom the actions are performed, how and by whom decisions are made, and how queues are managed. At times it is also useful to manage what a transaction looks like.

Documenting Processes

Using information about processes allows the manager to study and describe the process. To be useful, that information needs to be documented in some manner.

A simple and straight forward way to do this is to simply write down the process. For simple processes and those that are infrequently used, writing it down is often sufficient documentation. This written description of the process can be done based on the actual process being completed, or it can be done to plan how a process should be completed. If it is done as a plan, the process should be tested to ensure that it actually effectively accomplishes the desired result.

A written description of a process can be used as a guide for how to complete a process. At that point it becomes a **standing (or standard) operating procedure (SOP)**. A procedure is the steps a person takes to complete a process. SOPs are great for helping new people figure out how to perform a process or for remembering all of the steps in a process when that process is not used frequently. An SOP should also be used on processes that are performed on a frequent basis. Performing the process in this manner makes it self-correcting. If the person performing the process varies from the SOP, then either the person needs to mend his or her methods, or the SOP needs to be updated to match the actual process.

Process Diagrams

Once a process gets complicated, mapping it out is not as simple as simply writing it down. It can be difficult to visualize the ordering and connection of all of the steps in the process. The **process diagram** is used to overcome this challenge by carefully diagraming the steps being used, the relationships between the steps, and other valuable information. Several tools can be used to do this. Some of these tools are for general use; others are industry specific because process diagraming is done across many industries and have developed their own specialized symbols and grammars. Using a basic graphical tool, the process diagram can provide useful advantages in understanding and managing processes.

For the process diagram, each step in the process is mapped using a specific symbol. The actions that start a transaction and are not dependent on

any previous actions are noted as process entry points or start states. Actions that complete a process are marked as terminal points or end states. Each action in the process is marked with a symbol. Sometimes different actions are marked with different symbols. Often these are specific to a particular industry or domain. These special symbols aid in quickly understanding the process for those who are familiar with the special symbols. In basic diagraming, tasks special symbols are not required, and labels can be added to the general symbols to indicate different actions.

Arrows are used to show the **process flow** from one step to another. The process flow is the ordering of steps in the process. It can be a straight linear flow from one step to the next, but it usually involves branches and iterations. See **Figure 4.1a.** Tasks in a process traverse the process flow and may either split or combine at different points in the process. This is shown by having multiple arrows departing or converging on an action. These may use specific symbols to help identify those points. The circle with an X in it is often specifically used to designate that multiple transactions are combined together at that point rather than it being a point where flows from multiple paths combine.

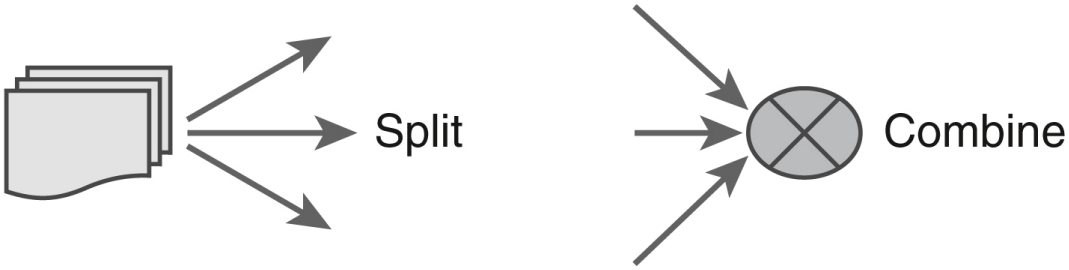
Another specific type of symbol is often used for decisions, and different symbol type is used for queues, as seen in Figure 4.1a. These are points where a transaction may take one path or another, but not both. Decisions have some sort of a criteria upon which the choice of paths is based. The decision may be automated, or it may be left to a person.

Components of processes

- **Steps organized into flows**
  - Step – single discrete action, set of actions, procedure, or set of procedures.

  - Flow – the organization and ordering of the steps of a process.

- **Transactions**
  - Tasks, items, or groups which traverse the flow, or portions of a flow, as an entity.



- **Decisions**
  - Points where the flow changes for individual transactions based on intrinsic or extrinsic factors.

- **Queues**
  - Where transactions wait for an action to be performed or a decision to be taken.

FIGURE 4.1 Components of a Process

Description

Queues are also included in a process diagram. Normally queues are included prior to an action in a process when the action cannot be performed at the same rate as the transactions are arriving. Queues are similar to the inbox on a desk or to a line of cars waiting to be able to enter a car wash. Only a limited number of cars can be washed at any one time; the others wait in a line for their turn. Without a queue, the process comes to a halt as each transaction waits for the next action to become available. For example, cars entering the car wash would back up onto the street if there is nowhere for them to wait.

A typical process diagram is given in **Figure 4.2**. Notice that the example process diagram uses "**swim lanes**." Swim lanes group parts of the process by the organizational unit responsible for the activities shown. Often this is a useful means for understanding and analyzing a process because it provides a quick graphical representation of the parties responsible for each of the steps in the process.

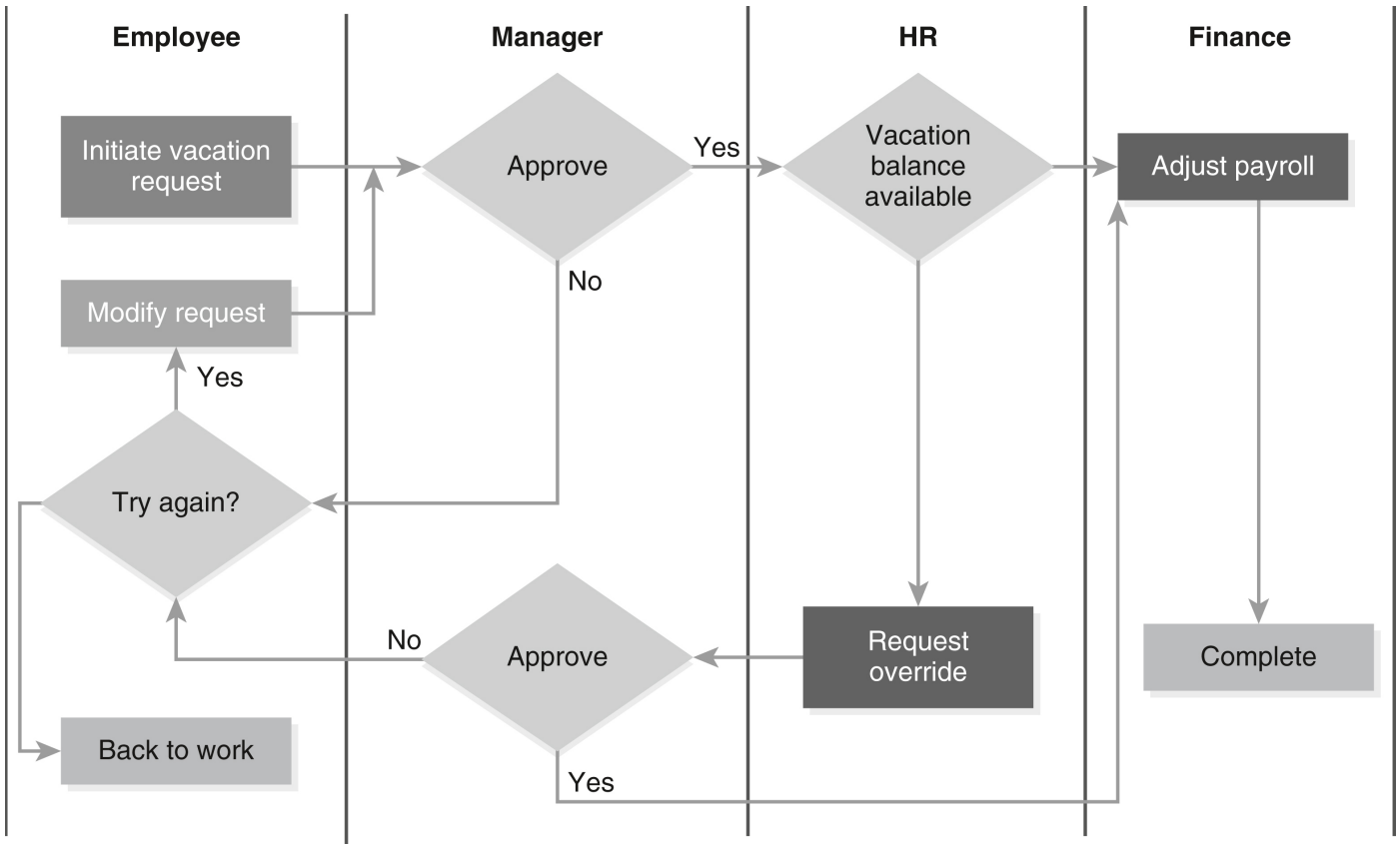


FIGURE 4.2 Typical Process Diagram

Description

The diagram is also color-coded to help the user understand the parts of the process. Often the time and resources needed to complete an action are also noted.

In many ways, the process diagram is similar to the PERT chart used in project management, where it starts at the beginning, follows the transitions, and goes through the specified actions. The difference is that rather than being followed a single time, as with a PERT chart, multiple transitions will go through the process, and multiple simultaneous transactions may follow independent paths through the process.

Static Process Analysis

With a process diagram it is possible to trace the expected path of any transaction through the process. **Static process analysis** is done using the standard expected values, expected decisions, and the path that would be followed by a single transaction without considering the effects and interactions of multiple transactions being handled in the process at the same time. This type of analysis can provide a lot of information to the process manager. The first useful information would be the path taken and the decisions required to navigate that transaction path. With that information, the manager can predict the expected cost and time for a transaction by adding together the resources and time required at each action.

Another useful analysis result available from the process diagram is the ability to identify bottlenecks and resource constraints. These become apparent when multiple paths converge on a single node, especially one which takes a significant amount of time. These bottlenecks can be reduced by adding parallel nodes and distributing the transactions to the different nodes in a fair fashion. An example of this is seen in **Figure 4.3**. In considering a change such as this, the cost of adding a parallel node needs to be compared to the cost introduced by the delay without it.

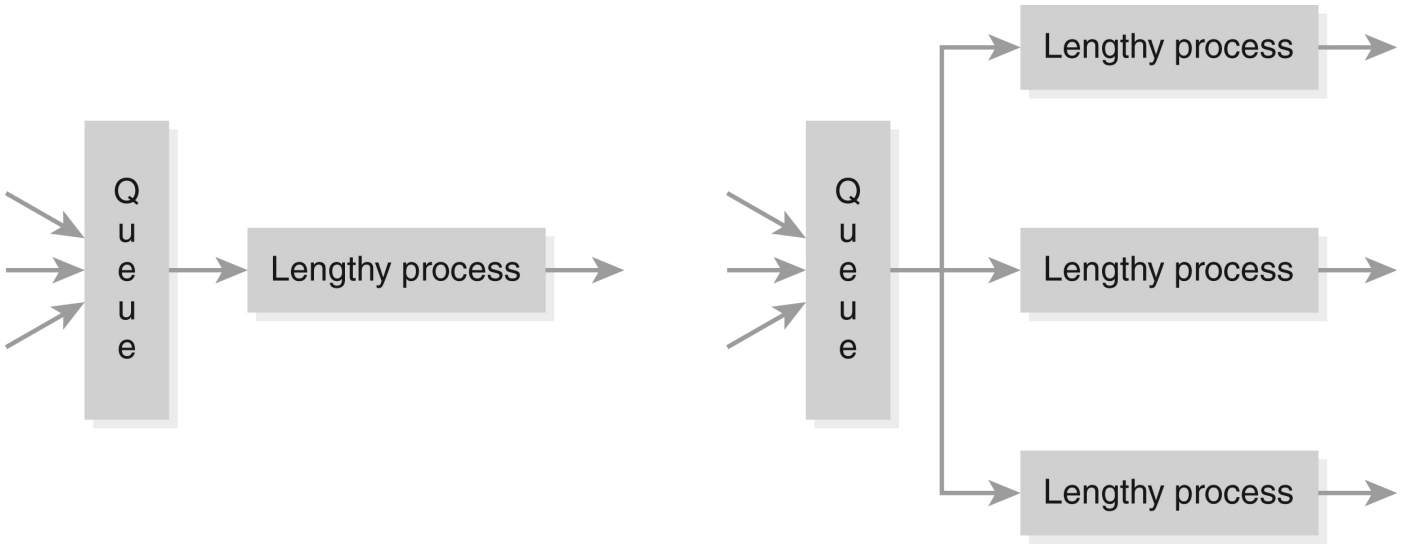


FIGURE 4.3 Adding Parallel Capacity

Description

Loops within a process are a potential issue. Any looping should be done on an exceptional basis because looping transactions end up both spending more time waiting in queues and delaying new transactions entering the system. Looping often results from how decisions are made about transactions in the process. One problem occurs when decisions are made before there is enough information to make a final decision. Another problem occurs when decisions are overly centralized. Frequent looping may also be an indicator of a quality problem.

Dynamic Process Analysis

The size of queues in a process isn't immediately apparent in a process diagram, but the queues can be measured from the actual process, or they can be predicted through modeling. This is called **dynamic process analysis**. Often it involves both modeling and observing the actual process to determine the difference between the two. Software packages are available to create, model, and simulate process diagrams, or the adventurous could use a simulation language and program their own model.

Dynamic analysis can allow the observation in many critical areas. For example, we can observe the queue lengths, both actual and predicted. The queue lengths and time spent in queues can allow us to locate bottlenecks and other process inefficiencies. The differences between predicted queue lengths and actual time spent in queues and values that are actually observed may inform us about flaws in our model, or more often, they help us to identify tasks

that are not being performed as expected, resources that are not as available as they should be, or perhaps even that portions of the process hadn't actually been previously identified.

The routing of transactions in the model and the in-flight process is another area that is of interest. In our static mapping of a process, we often have a concept of how transactions are going to flow through the system and what constitutes a normal route. Sometimes though, when we start dynamic analysis of the process, we discover our assumptions were incorrect: What we thought was the "normal" flow is a route rarely taken, and what we expected to be occasional "exceptions" turn out to be the standard route through the process. This type of event can occur for several reasons. It could turn out that the mix and content of transactions was far different from what we expected. For example, we may be looking at an ordering process, and we expect all orders to be for standard off-the-shelf products, but most orders contain products that needed custom adjustments. Perhaps we had designed the process to quickly complete standard orders, but push those pesky, time-consuming customizations off to interns to complete in their spare time. Obviously, this was a poor choice.

The process problem may not be with the nature of the transactions themselves. Instead, it may be an issue with the quality of the decisions being made. Perhaps too many decisions are being routed to an overloaded resource, insufficient information is being provided to make accurate decisions, or the criteria for making decisions are incorrect. Any of these could lead to a minor path with insufficient resources becoming the standard path through the process. Fixes would include anything that could improve the quality of the decision-making.

Queueing Theory

At the heart of modeling processes is a branch of mathematics called queueing theory. **Queueing theory** has been applied to many different areas besides processes because it offers a way to quantitatively study how many different things work in the real world. Queueing theory can be applied to areas such as managing traffic, determining where to locate restaurants, programming elevators in a skyscraper, or managing lines in an amusement park. Queueing theory is the study of lines and waiting.

Key elements to consider in modeling a queue include the arrival process. That is how transactions arrive in a queue, including such factors as whether they arrive alone or in groups, the inter-arrival times, including the average and

their distribution, and time of day patterns. A system with a hundred transactions a day that all arrive at the beginning of the day is far different from one where the same hundred transactions arrive at noon, or where they are distributed evenly through the day. Assuming all transactions are handled in the day, with the latter, the average time for a transaction in the system would be fairly short, while in the first it would be approximately half of a day.

Another factor in a queue is the **transaction behavior**. While presumably every transaction would travel through the system until being completed, the reality can be different. Sometimes, transactions that are begun will time-out or leave the queue if their wait becomes too long at any point. This in turn can result in additional transactions being created to replace the timed-out transactions. Sometimes the timed-out transactions can even leave uncompleted actions dangling, further complicating the processing. Another issue is that transactions waiting in a queue can sometimes require servicing while in the queue. This may be through the need for reporting, due to ongoing maintenance requirements, or even a requirement to placate people who are anxious about the waiting.

Queues also need to be concerned about the waiting room, that is, how many transactions can be accommodated in the queue. A queue that is overloaded may need a secondary queue, or it may result in transactions being delayed in progressing from the previous stage in the process. This could eventually back up to the beginning of the process.

How the transactions are selected for completion from the queue refers to the **service discipline**. Considerations for the service discipline may include the priorities of the transactions, whether transactions are serviced in the order they arrive (first-come first-served), or the most current transaction being serviced first (last-come first-served), as in a stack. Service discipline also considers whether a single queue is used for multiple processors or whether each processor has its own queue. Closely related to the service discipline is the method for pulling transactions from the queue. In some cases, one transaction at a time may be pulled for attention. In other cases, a group of transactions may be pulled and serviced together in a batch. The servicing may be done by one or more resources each of which may be a generalist, may have a specialty of the type, or types, of transactions it handles, or some combination of generalist and specialist abilities.

In a queueing theory model, each of these elements are modeled at each queue point in the process, and in turn, the series of queues in a process can be linked together in a network to model a complete process. Using either math

(for those of you who are so inclined) or simulations, using the model provides a way of evaluating the performance of the process.

Workflow

Another way of managing processes is with a **workflow system**, which provides an automated means of routing transactions in processes from one step to the next. The steps themselves can be either manual or automated procedures. Workflow systems are efficient in that they can link all the information about a transaction such that anyone performing a task has easy access to that information. Workflow systems can either be a stand-alone system or be embedded as a part of another system.

An example of a workflow system in action would be as a part of an insurance policy management system. In such a system, a customer may make a request for a change on a policy. The request could be initiated via mail or through an online interface. When the request is made, the system automatically routes the request through an underwriting process to determine if the company would allow the change. If so, the request would next be routed to a rating system, which would determine any adjustments to the premium from the requested change. Next the request would be routed to insert the changes into the policy and print (virtually or physically) the policy and issue any billing requests. Finally, the change request would get routed back to inform the customer, either via mail or online, that the change had been made. There would of course be different routes through the process based on the particulars of the change and decisions made during the process.

Among the advantages of this type of a system is that it allows the manager insight into various aspects of the process, including queue lengths, processing times, flow rates, and other factors that can affect the process. Most systems include tools to help design and manage processes,

Once a stand-alone tool, workflow systems are most often found incorporated into other specific systems to make them more effective.

Process Quality

Process quality is a concept that has been addressed in many ways. Because it considers other factors in addition to the effectiveness and efficiency of a process, quality definitions can vary among process implementations. At the simplest, quality can be defined as a lack of defects. However, that definition isn't really very satisfactory because other factors may be involved. For example, a process may produce parts that consistently fall within the provided specifications yet vary within those specifications to the point where some perform better than others. Or a process that involves interaction with customers may correctly collect all information and complete the desired transactions, but the customer receives a distasteful feeling from the way the conversation was handled. Perhaps the person talked with was too brisk, or perhaps the customer was asked to repeat the same information numerous times.

Process quality is a key part of **quality assurance**, the formal process of ensuring that the quality of a product is kept to a specified level. It is often confused with product testing. QA does get concerned that testing was conducted with the proper outcomes, but that is only an artifact of ensuring the quality of processes, inputs to the processes, and proper attention from management.

Many attempts have been made over the years to increase the quality of processes. Among the notable efforts in this area has been the ISO 9000 and 9001 series. This has been an international effort to improve quality by ensuring that companies have documented and followed established processes.

ISO 9001 is based on seven general principles, including strong customer focus, motivated and involved top management, engaged employees, a process approach, continual improvement, evidenced-based decision-making, and relationship management. These work together to ensure that the company delivers consistently high-quality products and services to its customers.

Another component of ISO 9001 is to take a risk-based approach. This works together with evidence-based decision-making and a motivated top management to create an environment where decision-making becomes informed, consistent, and repeatable. This risk-based approach will show up later as we consider the security of our software systems.

It is perhaps worthwhile to briefly explore precisely why risk-based decision-making is so critical. **Risk-based decision-making** requires an organization to

be prepared to accept a specified level of risk and understand and be prepared for the consequences of accepting that risk. This is basically what all drivers do when they buy auto insurance. Drivers know that there is a risk of an accident any time they drive. They judge the level of risk based on their experience, the amount of driving they do, and other factors. By buying insurance, they reduce their risk exposure to equal to the amount of their deductible. Some drivers are confident in their driving and have sufficient funds such that they are willing to accept the risk of a high deductible, for example $1,000. Other drivers have less confidence, or find they are not able to accept such high consequences and purchase a lower deductible policy, such as $250. In either case, the choice is made based on their informed opinion about their abilities to drive, to pay, and to be comfortable accepting a particular level of risk.

Risk-based decision-making in business works the same. The organization understands and accepts the level or risk that it is comfortable with as an organization. It uses evidence to calculate the risk and consequences of a decision and then makes the decision in a calculated manner.

The alternatives to a risk-based approach would be for an organization to make decisions based on management's "gut-feelings," best-case promises and projections, or any of a myriad of other instinctual or feelings-based approaches. Even a pure evidenced-based approach not informed by risk is bound for failure because perfect information is never available nor does it point to specific outcomes with total certainty.

Returning to ISO, included with the ISO effort is ISO/IEC 90003, which is an application of the ISO 9001 quality program to software engineering. This is not a specification, but rather recommendations and guidelines in how to apply the 9001 criteria to the specific field.

One of the features with ISO 9001 is that companies can become certified as ISO 9001 compliant. This lends a degree of assurance that a company has adopted quality practices in the management of their processes.

Another method of judging the quality level of a company is through **capability maturity modeling (CMM)**. This concept came out of the Carnegie Mellon Software Engineering Institute. Originally designed as a method for determining how capable contractors were of handling government programs, the concept has been expanded to cover development, integration, and other areas as well.

The concept behind CMM is that as an organization become better or more mature in how it handles its processes, the organization can produce higher quality products and perform better as an organization. CMM generally divides organizations into five levels of maturity and recognizes several capabilities, or

key process areas, within an organization, any of which can be at its own level of maturity. The maturity level of an organization is an unspecified weighted average of its maturity across all relevant capabilities within the organization.

The maturity levels begin with an Initial level. At this level, processes within an area in an organization are chaotic and ad hoc. They may be dominated by individual decisions from different managers where the same process differs across the organization and varies based on the experience and preferences of individuals. Processes have little or no documentation.

At the next level, Repeatable, some processes are documented or at least defined, though perhaps not in detail. Described processes are not rigorously adhered to, but the same process will be roughly followed.

As more processes are documented and details are included, the organization can reach the Defined level. At this level, evidence shows that processes are actually being followed, at least on a somewhat more consistent basis. There is also an emergent method for providing feedback in the definition of processes. The processes are more considered and valid across a broader variety of circumstances.

A Managed or Capable level is achieved when the defined capabilities are maintained and applied on a regular basis by the organization. The application of processes is consistent across the organization, and process users are experienced, supportive, and proficient with the processes. The organization is capable of applying and adapting the process across a wide variety of circumstances, and different conditions are anticipated and accounted for in the process. The organization performs with increasing process quality.

The Optimizing level is the peak of the CMM. At this point the processes in the organization are not only rigorously documented and applied, they are also being continuously refined as the organization learns and circumstances change. A mechanism is in place for transferring the process learning rapidly across the organization, and support for it is enthusiastic throughout the organization.

As with ISO 9001 companies can become certified in their CMM level.

Process Improvement

CMM and ISO 9001 are concerned with documenting or describing process quality or capabilities. They are not directly methods or methodology for pursuing quality processes. With these the organization improves its processes as a result of pursuing the attainment of a certification or level. While working through CMM levels or applying ISO practices has the effect of improving processes, it is not accomplished by any specific means. A different approach is to specifically apply a process improvement methodology or program. **Process improvement** has the objective of changing the process to raise quality, shorten processing times, or to make another quantitative or qualitative improvement in the process.

Several process management approaches are focused on improvement. Some of these stand alone as a process management approach, while others are applied to supplement an organization's process management efforts. The best approach to choose will depend on a few factors, including organizational preferences, the experience of those implementing the approach, the nature of the processes being improved, and several other factors. Additionally, it is rare that a process management approach in its purest form is a perfect match for any situation, project, or company. Any approach needs to be tailored and fitted to the specific circumstances.

The real gains come not so much from the specific approach to process management but from the fact that a structured approach toward improvements is being taken. Sometimes a practitioner or organization will forget this truism and credit any improvements purely to the approach being taken rather than to the fact that an active structured approach to improvements was being taken. They'll then apply the same approach to any problem, without considering the circumstances, and then blame the approach, or the people the approach was foisted upon, when the anticipated improvements failed to materialize.

Among the process improvement approaches that have been developed over the years is one called **Six Sigma**. The core to the Six Sigma approach to quality is data driven, where the goal is to improve processes through increasing consistency. Consistency is defined as the number of defects lying outside of a six-sigma deviation from the mean—hence the name. In practice that works out to be three or four defects per million opportunities. Processes are analyzed using an approach called DMAIC (define, measure, analyze,

improve, control) to find methods to move them closer to obtaining the desired success rates. New processes are created using an approach called DMADV (define, measure, analyze, design, verify).

Six Sigma is big on using software and other tools to perform measurements and statistical evaluations. As one may guess, identifying and locating three to four issues per million attempts can use a lot of data and a lot of analysis. Commonly, Six Sigma practitioners receive significant training in the use and application of these tools, many of which are proprietary. These “Black Belt” certified practitioners specialize in the application of Six Sigma methods and tools either in specific industries or in general.

Another popular process improvement technique is **Lean**. This grew out of the study of an approach to manufacturing by Toyota in the late 20th century. At its core, Lean is about creating value with less waste. Waste comes in many forms. In manufacturing, it obviously can mean scrap, such as excess steel or other commodities, which of course have physical value. But waste can also come through people spending time on inefficient or unnecessary processes. Waste can also come from redoing processes rather than having them completed once and being done with.

Put into action, the Lean process identifies the value that is being produced, identifies the steps in creating that value, synchronizes those steps to make them flow smoothly, and times the process to be pulled by need rather than pushed. Finally, the whole process is revisited again to ensure it is reformed in a continuous fashion. Lean also involves other activities, such as involving those in the process in determining how the process can be improved.

Like Six Sigma, Lean tends to develop a cadre of trained people who specialize in the application of the Lean approach. Lean also has several different frameworks that can guide deployment in different circumstances. One of the more interesting things about Lean is its penchant for using Japanese terms for activities and principles. Some of the Japanese terms are not clear to those who speak Japanese because they are used in an unfamiliar context. It would be like using the term “peanut butter” for spreading information across a spreadsheet, as in, “I am going to peanut butter the costs across the clients.” This use of special language can tend to disorient or exclude people who do not feel welcomed into the club and can lead to reluctance to apply or embrace elements of the methodology even where they would otherwise have value.

In some cases, the value and waste focus of Lean are paired with the tools and analysis of Six Sigma to produce a Lean Six Sigma approach. This

approach tends to be more collaborative than Six Sigma, but with a more analytic approach than Lean.

Agile process management, much like Agile project management, seeks to create flexibility and responsiveness in processes through breaking them down to their most elemental requirements, incrementing frequently, and using inspection and adaptation to ensure the process is meeting the needs of the organization. Agile processes are operated on principles more than on details, with the outcomes and guidelines specified. Initially applied principally to software development, Agile processes are being adapted to many other business processes as well.

In a typical Agile process, decision-making is devolved down to the lowest level employee possible while providing guidelines to assist in making the decisions. The concept is that pushing decision-making lower and distributing authority allows an organization to be more responsive to customer needs and rapidly evolving markets. This kind of distributed approach requires a high level of communication and buy-in for organizational principles and goals as well as a high level of trust in organization members.

There are several other approaches as well including total quality management, just-in-time, process reengineering, and others. Each organization needs to adopt an approach that works best for their particular circumstances, learn to effectively apply that approach, and stick with it. Nothing is more disheartening in an organization that the “management philosophy of the month” approach to process management.

Processes and Security

Security thrives on well-managed processes. Consistency and continuous improvement are the cornerstones of good security. The biggest issues in security come from chaotic situations and human weaknesses. These are the very same challenges that process management seeks to overcome. The more tightly processes can be managed or controlled, the less room there is for an error to creep into the process, which is going to expose a vulnerability. Even when a vulnerability has been exposed, good processes help to minimize any damage that can result from the exposure and to identify when an exposure has occurred. Continual improvement is an essential element as well because processes that are not adapted become stale, moribund, and inappropriate, leading to induced vulnerabilities.

There will always be unidentified vulnerabilities in complex software systems given the nature of such systems. However, applying continuous improvement methods helps to create refinements in processes, which will increasingly mitigate risks resulting from unknown or recently discovered vulnerabilities and threats. It will by no means make a system invulnerable. But it will reduce risk.

Some of the important processes for secure development include the following:

- Security requirements engineering
- Threat and vulnerability mapping
- Security architectural reviews
- Security analysis of libraries and components
- Security testing
    - Code review
    - Static analysis
    - Dynamic analysis
- Final security review/certification/accreditation
    - Approval to operate

These processes are discussed in appropriate chapters of this book.

Defining Processes for the Developed System

Finally in this chapter, it is important to remember that in addition to considering processes from the perspective of managing the development and life cycle of a software system, we also need to consider processes from the perspective of the processes that the software system will support. Any software system is a part of a process or of multiple processes. When that has been considered from the beginning and support for those processes is baked into the system, the quality of each implementation of the system is going to be easier to maintain. If the processes are required to continue despite the system, then the quality and efficiency of the processes is going to suffer. This applies to support processes for the system as well as to primary business processes. In security, this can be a problem because problem processes are problems for security. For this reason, it is essential to understand how processes are managed and improved.

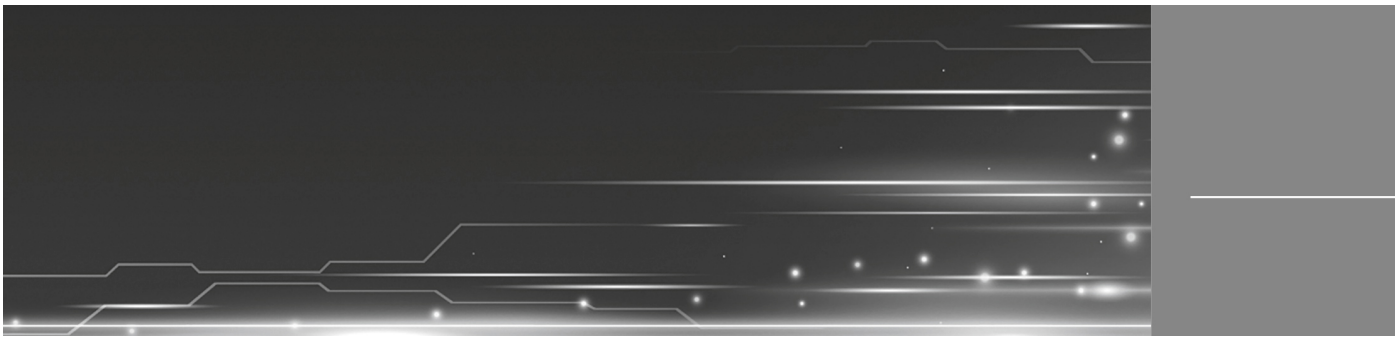
SUMMARY

Understanding processes is a way of understanding the work that is done in an organization by people and systems. Describing and documenting the processes allows those processes to become understood and repetitive. This in turn allows for process improvement as well as for designing software systems that effectively implement and support processes. Understanding processes is also a significant step toward ensuring the security, implementation, and the data used in those processes.

KEY TERMS

Agile process management
capability maturity modeling (CMM)
dynamic process analysis
FIFO (first-in first-out)
highest priority first
Lean
LIFO (last-in first-out)
preemptive queue
process diagram
processes
process flow
process improvement
process management
process quality
quality assurance
queueing theory
queues
risk-based decision-making
service discipline
Six Sigma
standing (standard) operating procedure (SOP)
static process analysis
swim lanes
transaction behavior
transactions
unordered queue

workflow system



© d1sk/Shutterstock

CHAPTER 5

Managing the Secure Software Systems Development Life Cycle

CHAPTER OBJECTIVES

- Compare and contrast different software methodologies and how they are managed.
- Trace how requirements are tracked throughout the development process.
- Describe typical change management processes.
- Examine the role of issue management in the secure software systems development life cycle.

Introduction

Managing a secure software systems development life cycle is a complex and challenging prospect that is based on applying product, portfolio, process, program, and project management in the right portions at the right times. Each of the management areas is applied at the right level, and elements of the area support, drive, and inform the elements of the other areas. The key to making everything work effectively is communication.

Different organizations will often have differing opinions about which management area is primary, but the key is that all are needed, and neglecting any one area from consideration can contribute to failures in the organization. **Figure 5.1** displays how different management areas overlap. When they overlap, that does not mean that one management approach needs to be selected over the others, but rather the various approaches should be applied in a manner that complements each other.

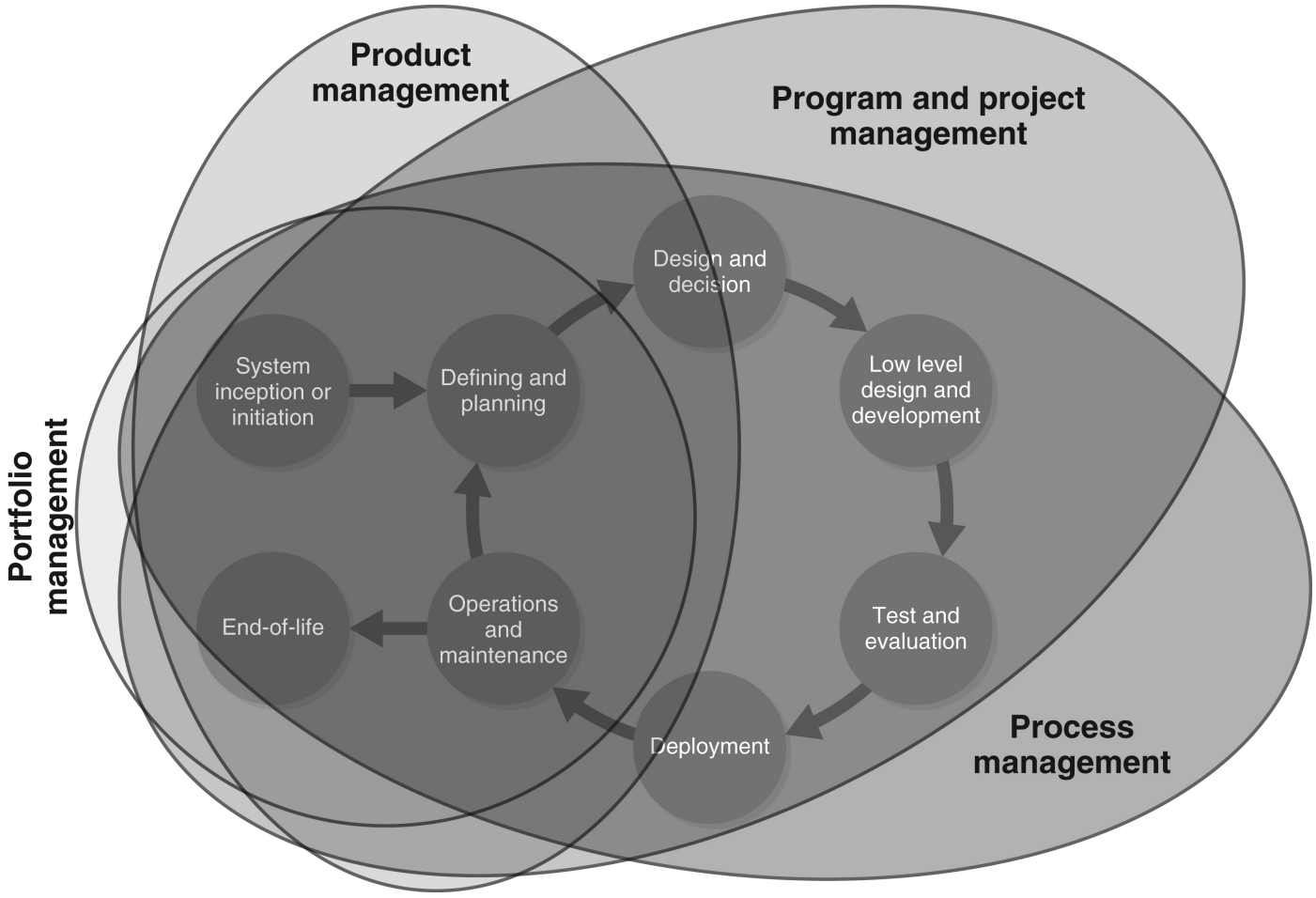


FIGURE 5.1 The Secure Life Cycle with Supporting Concepts

Description

Because they recognize the need to actively manage their software development activities, organizations put **development methodologies** into place for their management activities. Here the term "development methodologies" means a process for or standard method of management. Most often, these methodologies are adopted from some standard model and then applied or, ideally, adjusted to fit the circumstances and applied to the organization.

Hundreds, possibly thousands of methodologies are out there, available to be selected or bought. Thousands, maybe millions, of consultants are willing, for a fee, to implement their flavor of methodology within any organization. Millions of books are sold every year with new methodologies or rehashed old methodologies for different aspects of managing the organization or activities of the organization in the "perfect" fashion. Not all of these are bad. Adopting a sound methodology can be beneficial to the organization if done correctly, in a manner aligned with the organization's goals, and not to the point of exclusive focus, ignoring or slighting the other management areas. For any methodology, the most important thing is a thoughtful and deliberate application, not a hurried forced compliance with the latest popular book or rushing headlong into the methodology proposed by the slickest talking consultants.

Just as with all other management areas, software development has methodologies, which are generally a means by which aspects of project management are applied to a software development life cycle. Among the most common of these are the waterfall model, spiral model, and Agile methodology.

The **waterfall model** is the most classical of all development methods. It is named such because development falls from one stage to the other, without loops or feedback, toward an inexorable outcome. It demands that everything along the path be completed and completed correctly before moving into the next stage. This leads to requiring high levels of documentation throughout the process in a traditionally focused waterfall model. In reality, the full model is seldom actually used in such a strict manner, and the waterfall is held up as a strawman to compare to other methodologies.

The **spiral model** is an approach that also tends to strictly follow the secure software systems development life cycle, at least notionally. What differentiates it from the waterfall is that the development is not expected to be completed in a single pass. Instead, multiple cycles are expected, each with an increasing number of requirements, and possibly increasing expectations for performance, accuracy, or other tangibles. Because there is no one formal

definition for what constitutes a proper execution of a spiral, it is subject to opinion; however, the strict application and completion of the steps in order is emphasized. Again, there is a tradition of a focus on documentation associated with the spiral model.

Many variants of the waterfall and spiral model focused on the pathway discipline and documentation of the steps. This led to a bit of a backlash in the way people responded to the models, as many felt that the strict adherence to these models led to poor decisions being made to meet the form, that the models inhibited as much work as they aided, and that the models did not conform to how programmers actually worked. Most notable of the objections was the **Agile** movement (see **Box 5.1**). This was one of the first of a variety of movements to create processes, paradigms, and methodologies which were more productive, less constraining to developers, and still maintained a high level of quality.

The reality is that these did not really eliminate the waterfall or spiral. Instead, they changed the emphasis, formalized work arounds, or addressed specific portions of the secure software systems development life cycle. Often this was done in response to a specific implementation of a methodology but proved to be useful for other organizations with other specific methodologies as well. Unfortunately, many interpreted the Agile manifesto to be that there was no value in processes, documentation, contract negotiation, or planning; rather, these were still essential, just not the focus. This is not what the Agile manifesto says. In addition, different approaches to Agile have added their own forms of dictates for their own style of processes, documentation, negotiations, and planning.

BOX 5.1 The Agile Manifesto

We are uncovering better ways of developing software by doing it and helping others do it. Through this work we have come to value the following:

- Individuals and interactions over processes and tools
- **Working software** over comprehensive documentation
- **Customer collaboration** over contract negotiation
- **Responding to change** over following a plan

That is, while there is value in the items on the right, we value the items on the left more.

https://agilemanifesto.org/

Some of the popular Agile-style methodologies introduced include extreme programming, scrum, lean, kanban, and dev-ops. Each addresses a subset of activities within the secure software development life cycle. Each can contribute to successful development. As with all methodologies, each has been abused by being forced on developers in an inappropriate situation.

In conjunction with the methodologies chosen and applied, some key management processes need to be implemented as a part of the secure software systems development process. These include, among many other processes, requirements tracking, change management, and issue management.

Requirements Tracking

The goal for **requirements tracking** is that every requirement is recorded and traced throughout the development process, as **Figure 5.2** shows. This process will include information about where the requirements are implemented, specific information about how the requirement is implemented, and when in the development process the requirement is tested—verified and validated. Details of the requirements planning and tracking processes are provided later.

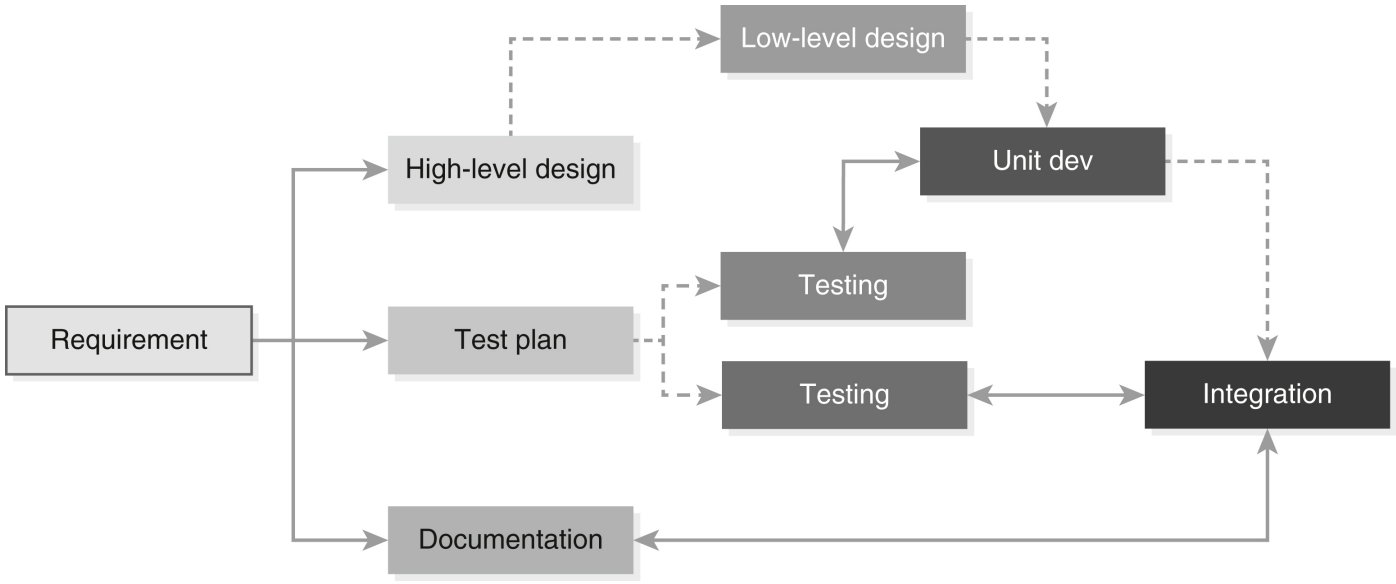


FIGURE 5.2 Requirements Tracking throughout Development

Description

Change Management

Change management is the systematic management of changes that will affect the scope, deliverables, or other factors in the development process. It often involves changes to requirements directly or indirectly through changes in budgets, milestones, or other factors. Such changes lead to scope creep, which usually means more funding and time will be required. Unmanaged change can be extremely detrimental in the development process because it may result in work not being done, work being completed unnecessarily, or even in extreme cases, two parallel efforts working in different directions as one party is aware of some change or changes and the other is not. Lack of change management can also result in changes being made without consideration of the overall impact of the change on the development or changes that cancel each other out.

BOX 5.2 Change in Context

Change in the context of secure software develops life cycle is not

- process change management,
- organizational change management, or
- system change management.

The goals of change management are to ensure that each change request is actively managed from initiation to closure, that changes are process based on direction from proper authority, that all changes and their impacts are communicated to appropriate personnel, that the change is performed with quality, and that the overhead in managing changes is appropriate to the size and scale of the change requested.

Just as with tracking requirements, tracking changes is important. Information that needs to be tracked includes the source of the requested change, factors that drive the need for the change, approvals and approvers of the requested changes, impacts of the change, requirements that derive from

the change (those requirements, as they are tracked, will also have links back to the change as their source), and any other relevant information.

As a part of the change mechanism, there needs to be a process for acceptance of change requests where changes can be approved by a proper authority once their impact has been assessed. The process needs to both define the level of authority for changes of differing impacts and the depth of the review for impacts.

Authority for differing types of changes may be delegated to different responsible parties. For example, changes to colors, images, and wording may be designated as needing marketing approval, while changes in uses of different third-party tools or application programming interfaces (APIs) may need approval of a system architect. Changes with a broader impact will often be required to be submitted to a change board or committee, which considers the impacts of a proposed change from a variety of viewpoints, and then approves or recommends approval for the requested change if appropriate.

It is important though that the complexity and responsiveness of the process for approving a requested change needs to be appropriate to the degree of impact of a change. Small changes should be managed with a minimum of overhead. Failing to do so can result in a situation where people are hesitant to recommend minor changes because it is just too troublesome, with an end result of a thousand small, irritating “features,” no one of which was worth the effort to change but together make for a miserable result.

Communicating during the change process is vital, both for communicating the impacts of requested changes and for communicating that the changes have been approved or disapproved. Presumably, everyone in the development process is familiar with the original requirements, schedule, and budget. These have set the focus and drives everyone working toward their goals.

Without some systematic effort to reach out to everyone impacted by a change, people are going to keep on doing what they believe is the right thing. To change, people need to be informed. Change is difficult for most people. It is impossible if they don’t know they need to change. Of course, making sure that people understand the reason for the change is also useful in making the change. Understanding the reason for a change is also important because it provides the opportunity to ensure that the change is correctly made and to recognize any other impacts of the change.

The **change process** can be formalized in the following steps:

- Identify (identify and document the required change)

- Validate (verify the change is valid and requires management)
- Analyze (analyze and record schedule, cost and effort impact of change)
- Control (decide whether to execute the change)
- Action (execute decision, including revision to project plans if necessary)
- Close (verify that action is complete and close change request)

Issue Management

A key element in managing the development life cycle is issue management. **Issue management** is the process of managing issues related to the product or project in a disciplined manner. The reality of the world is that things don't always go perfectly or the way they are planned. Stuff happens. When the stuff that happened needs to be addressed, it is an issue. If that stuff gathers in a heap, it gets smelly, so it is best to organize and manage it properly.

The first consideration in managing issues is to track them so they don't become lost. Lost issues represent missed opportunities, times when the issue could have been quickly and easily resolved or recognized and avoided so it didn't need to be discovered again the hard way. The funny thing about issues is that they usually arrive in flocks, so if they are not recorded and tracked, the amount can quickly become overwhelming.

Once an issue has been recorded, the next step is to evaluate the nature and risks associated with an issue. Issues that represent greater risks need to be evaluated and resolved more quickly. Issues can come in many different forms. Bugs in the code being developed are an obvious example. But issues could also include delays in delivery of components, a software tool or library used in the development becoming outdated or unavailable, a security vulnerability discovered in an operating system, or any other form that might affect the development life cycle.

Once an issue has been identified and evaluated, the next step it to route the issue to the proper place for resolution. Resolution may include sending a bug to developers for fixing, involving procurement in purchasing new licenses, a systems architect reviewing the material to find a substitute library, or having system administration apply patches to resolve a vulnerability.

Sometimes an issue could be resolved multiple ways. An issue could be resolved through development, writing code, or even through fixing problems in the code. But it is possible that an issue could also be resolved through a process or process change or through a combination of code, process, and other means. Just as with changes, the issue process needs to have defined patterns of authority for making decisions about how issues should be resolved. Some issues can be handled routinely; other issues are going to need a defined escalation process to get them quickly to the proper levels of authority to make the necessary decisions or to provide awareness to anyone and everyone who may be affected by the issue.

Naturally, resolving the issue is not sufficient; instead, the resolution needs to be tracked and recorded. Simply tracking that the issue was resolved is insufficient. While tracking allows us to know that the issue has been resolved, which is important to know, it doesn't help us the next time the same or a similar issue arises. Without recording how the issue was resolved, it is likely we will end up having to rediscover how to resolve it or will apply different and potentially conflicting resolutions if the issue reemerges.

Because of the complexity of issue management, a number of different issue tracking tools are available. These tools can help with the recording, routing and escalation, and review of issues. While many of the issue tracking tools are designed in a fairly generic manner, each of them is designed around a paradigm and method for managing issues in different contexts.

Finding the right issue tracking tool for a specific organization can require a significant amount of time and investigation. It is going to depend on many different factors, including the organization's processes and workflows, the level of integration of the tool with other tools, the type of information to be managed with each issue, the cost of the tool, and many other factors. Most important though is the organization's commitment to using the tool regularly and consistently. Any tool that is only partially deployed or used only for selected issues is never going to be effective.

SUMMARY

Managing a secure software systems development life cycle is a complex task that requires bringing perspective and skills from a variety of management areas together to successfully accomplish the task. The more team members understand how each area influences the success of the product, the more effective they will be in contributing to delivering the product that most effectively meets the customer's requirements.

Different development methodologies attempt to wrap the different management areas together into a comprehensive approach that can be applied throughout the life cycle. Because of the nature of software systems, some of these methodologies are more effective for particular development efforts than others. On the other hand, sometimes methodologies, or the way they are applied, neglects aspects of the management areas to the detriment of the development effort.

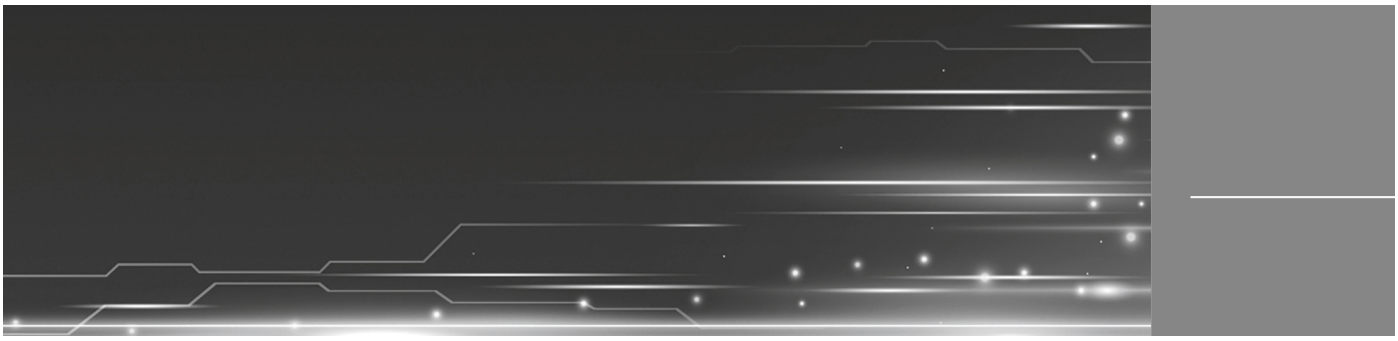
Requirements tracking, change management, and issue management are examples of processes and challenges which need to be managed throughout the secure software systems development life cycle.

KEY TERMS

Agile
change management
change process
development methodologies
issue management
requirements tracking
spiral model
waterfall model

Reference

Beck, Kent, Mike Beedle, Arie va Bennekum, Allistair Cockburn, Ward Cunningham, Martin Flower, et al. “The Agile Manifesto.” *Agile Alliance.* 2001. http://agilemanifesto.org/



© d1sk/Shutterstock

CHAPTER 6

Security Culture, Responsibility, and Training

CHAPTER OBJECTIVES

- Examine the effect of an organization's culture on security.
- List and describe the elements of secure culture.
- Relate the role of personal responsibility within a secure culture.
- Evaluate the need for effective training methods and styles.
- Specify the training needs for people in varied security roles.

Introduction

Without a doubt, the most important factor in determining the state of security in an organization is its culture. Since security depends so highly on human factors, the only way to effectively manage those factors is through the influence of culture. Simply defining and describing what everyone must do is not sufficient. First, one cannot ever adequately describe everything that must be done, and further, even if everything could be defined, read, and memorized, that is not adequate insurance that anyone in an organization would ever follow the guidelines.

A key element of any organization is the **organizational culture**. This is a set of norms and expectations commonly held by the majority of people within an organization. Organizational culture forms over time from both what is explicitly encouraged and rewarded in an organization and from what is implicitly condoned or supported by silence and apathy. Organizational cultures can take many forms, both positive and negative.

Organizational culture is going affect security in three major ways: through buy-in, impedance, and risk appetite. **Buy-in** is the degree to which the members of the organization accept security as a key and fundamental value of the organization. **Impedance** is the degree to which changes, ideas, and values are slowed in their implementation and conversion to action. The **risk appetite** of an organization is the degree to which the organization is willing to engage in actions and activities that may introduce risk, the number of resources the organization will expose to risk, and the number of resources an organization will expend to avoid or transfer risk.

When examining the buy-in of an organizational culture, the best place to truly understand how well a desired culture has permeated an organization is with the **first-line supervisors**. It is one thing to have executives who are all enthusiastic about and are citing platitudes from a cultural vision. It is quite another to see that vision lived out throughout an organization, and the first-line supervisors are where that buy in happens. It is the first-line supervisors who choose who to hire and fire and which company policies, principles, and slogans to follow. The first-line supervisors directly affect the attitudes and actions of the employees who directly face the customer, build the product, and create value for the company.

If first-line supervisors embrace a cultural initiative, they are not only going to be enthusiastic about it themselves, they are also going to see to it that their

direct reports live that culture. Take something as simple as wearing safety vests on a shop floor. It would seem simple enough to send down a directive that all employees will all wear safety vests at all times. If the first-line supervisors have a safety-minded approach and agree that the policy is sensible and workable, they will step in to enforce it and see to it that employees wear their vests. However, if the first-line supervisors don't see a reason for the policy, don't feel that it applies to themselves, or see the policy as an intrusion into their area of influence, then the policy will not be enforced or only enforced when certain people are watching.

It would then seem that the easy way to create a security culture in an organization would be to jump directly into **training** those first-line supervisors. However, this is where impedance in an organization starts to come into play. Jumping directly to first-line supervisors without first bringing along intermediate supervisors can create another challenge because those intermediate supervisors feel left out of the culture. These are people who have worked hard to obtain their positions and feel the need to have a level of control over what happens in their organization.

Also important is the need for the executives to be fully committed to an effective security culture. Their involvement, directives, and tone determine whether the organization is really going to have a secure culture or not. If the executives in the organization are not committed and stepping out as first movers in developing a security culture, few, if any, managers are going to feel inclined to risk their careers to push cultural change.

The key then is going to be discovering that pathway though the organization that engages on all levels without creating barriers. The means for doing this is going to vary greatly within organizations and requires a certain degree of study, aptitude, and politics to discern and chart a course.

Similarly, the appetite for risk is going to be unique to each organization. It would be convenient if each company had a nice little dial sitting by the front desk that showed the level of acceptable risk. Or if the organization passed out little yard sticks with a degree of risk marked on them, and one could simply measure the proposed risk against the mark on the yard stick and see where it fell on the company scale. Life is never quite that simple.

In general, organizations tend to be willing to take more risk in areas in which they feel proficiency as an organization. Note the "as an organization" clause. Simply having an individual with expertise in an area is not going to increase an organization's risk appetite if that organization has not first found a way to incorporate that individual into its rhythms and culture. Too often, people feel that because an organization has hired someone with skills in a particular area,

then that is going to make all of the difference. It can, over time, but only if that person is not left as an island of special skills, independent from the function and flow of the organization.

Likewise, organizations are less willing to take risk in areas where it feels less skilled and knowledgeable. The solution seems simple enough: If you want to encourage an organization to undertake more risk in a particular area, then educate the organization in that area. The challenge though can be that the opposite happens as a little knowledge can be dangerous. As the organization reaches a point where it begins to understand that risk exists, which it may not have previously acknowledged, it may shy away from any risk in that area.

Nor is factual knowledge simply enough. With knowledge there needs to be an organizational cultural understanding to enable it to act upon knowledge. Part of that is influenced by the way the company reacts to new ideas, and part of it is based upon how concepts are distributed throughout the organization.

Look and Feel of a Security Culture

We can ask, “What does a mature security culture look like?” That is, how do we recognize that an organization values security and makes it a priority? Just as there are no gauges to tell us the risk appetite of an organization, there are no dials to show the maturity of security culture within an organization. However, we can look for certain signs and signals that will give us a feel for the maturity of security within an organization.

Leadership Sets the Tone for Cybersecurity

In an organization with a good cybersecurity, culture leadership and management set the tone. **Leadership** is providing the vision, motivation, resources, and energy to move the organization in a direction. Leadership can be formal or informal. Formal leaders are those who are appointed to their positions in the organization. Informal leaders are those who, despite their positions, motivate and inspire others to move the organization in the direction they intend. The importance of security needs to be embraced at the top and adopted throughout the chain-of-command. Both the formal and informal leaders in the organization need to inspire adherence to good cybersecurity principles.

Someone with Responsibility and Authority

One of the first things to look for is whether there is someone in the organization with overall **responsibility** for security. When things are important for an organization, someone is charge of it and will be held accountable if things do not go right. It is usually a quick and easy task to find out who it the head of sales, marketing, and production. Companies that value people have someone in charge of human resources. Companies that value security have someone in charge of it. Increasingly that person is called the **chief information security officer (CISO)**, and cybersecurity is that person’s sole job responsibility, not an extra duty of the chief information officer (CIO) or someone else.

It isn’t enough, though, to have a person designated as responsible for cybersecurity; that person also needs to have the **authority** and independence to be able to actively influence security. Often, authority comes along with

direct reports, but not always. What authority does require, however, is it that the person can make decisions and take action. Nothing is worse than to have responsibility, but no authority, being held accountable for a problem that you are not allowed to fix.

How authority is exercised depends on the organization, and it can often depend upon where the individual is placed in an organization **(Figure 6.1)**. For example, a CISO who reports to a CIO may find that his or her voice is filtered from the larger organization and any conflicts in system security are decided before they are considered across the business. While this may seem efficient, it may also preclude the full consideration of issues.

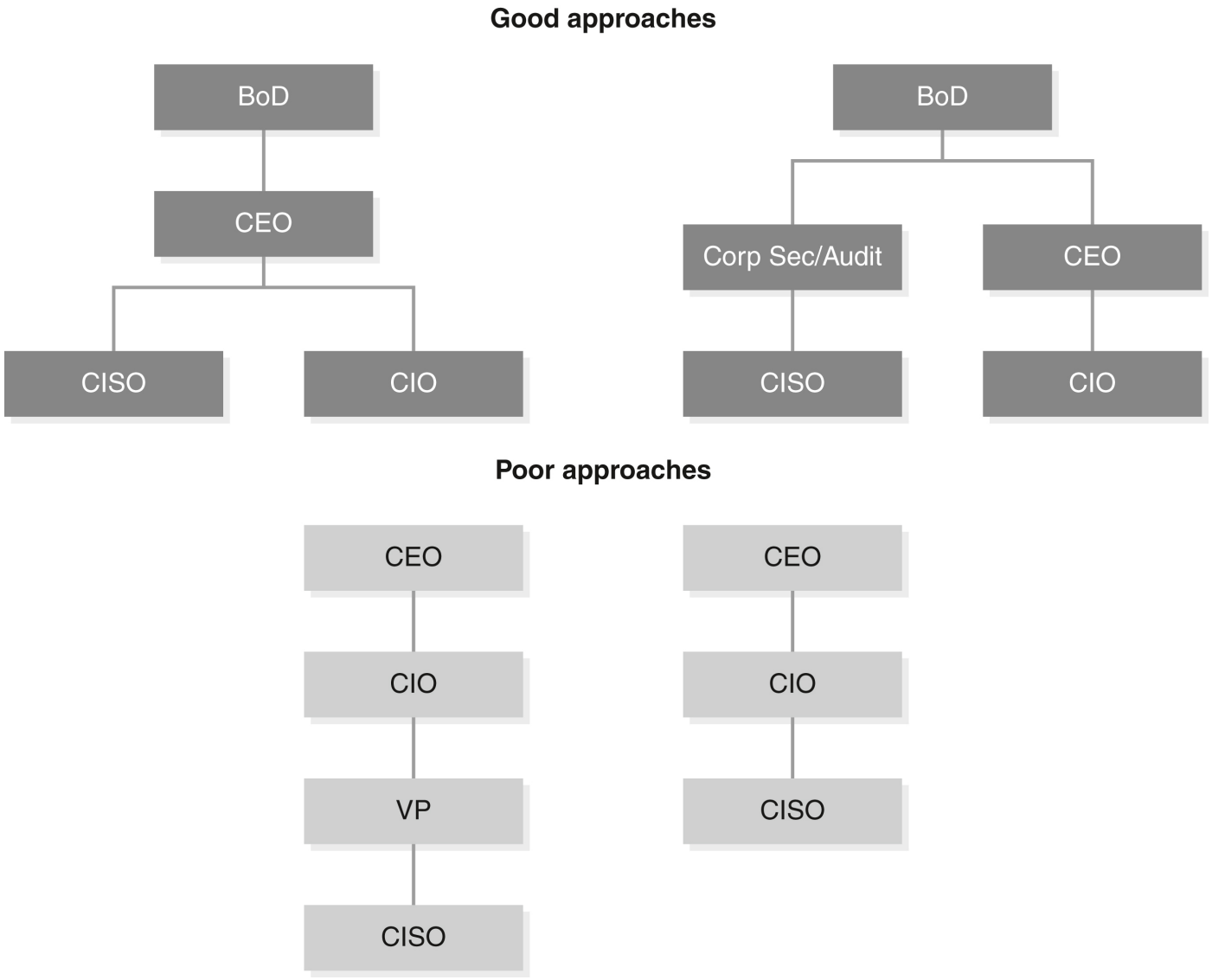


FIGURE 6.1 CISO Reporting Structures

Description

For these reasons and the similar nature of their tasks, security is increasingly being found tied with the corporate audit function. This is not the only, or even always the desired solution, but one of several possible, the keys being authority, autonomy, and ability to work across the organization.

Security Roles and Responsibilities Defined for All

Another sign of a mature security culture is that security is not left to the security professionals. It is too important for that. Everybody in an organization has a *role to play in security*, and everyone in the organization should have their security responsibilities defined for them. **Table 6.1** provides information on corporate roles and how each role can contribute to security. Even the janitor plays an important role in security. The janitor's role may be mostly focused on physical security, but that is key to information security.

TABLE 6.1 Responsibilities for All Positions

Position	Security Needs	Security Roles
CISO	Deep knowledge	Oversee and evaluate
Software engineer	Secure develop	Implement, practice individual security
Shipping clerk	Awareness	Cross check shipments, practice individual security
Janitor	Awareness	Physical security, practice individual security
Chief executive officer	Broad background	Security decision-making
...	...	...

Another example of a less expected security role would be that of the human resources person responsible for monitoring leave balances. This person should be aware of people who haven't taken continuous blocks of leave in a significant amount of time. That may indicate someone who is perpetrating a fraud or an internal cyberattack on the organization. They aren't taking leave because they fear that whoever takes over their duties during their absence may discover the fraud or attack or that the action may collapse without their presence. Even a cook in an on-site café contributes to security by observing unusual behaviors and activities of people in the building. Or the cook may notice a high level of careless discussions in the café and report his or her concerns.

As a part of their normal duties and responsibilities, everyone in the organization needs not only to know their role in security but also to be trained for it.

Security Is Considered in All Aspects of Products

Because this is a book about secure software development, we could perhaps phrase this as, “Security must be considered in all stages of development from requirements gathering, to design and development, and on into testing, deployment, and support.” **Figure 6.2** lists some of the areas where security needs to be included. However, it is far more than that because security must be considered even before the development cycle begins.

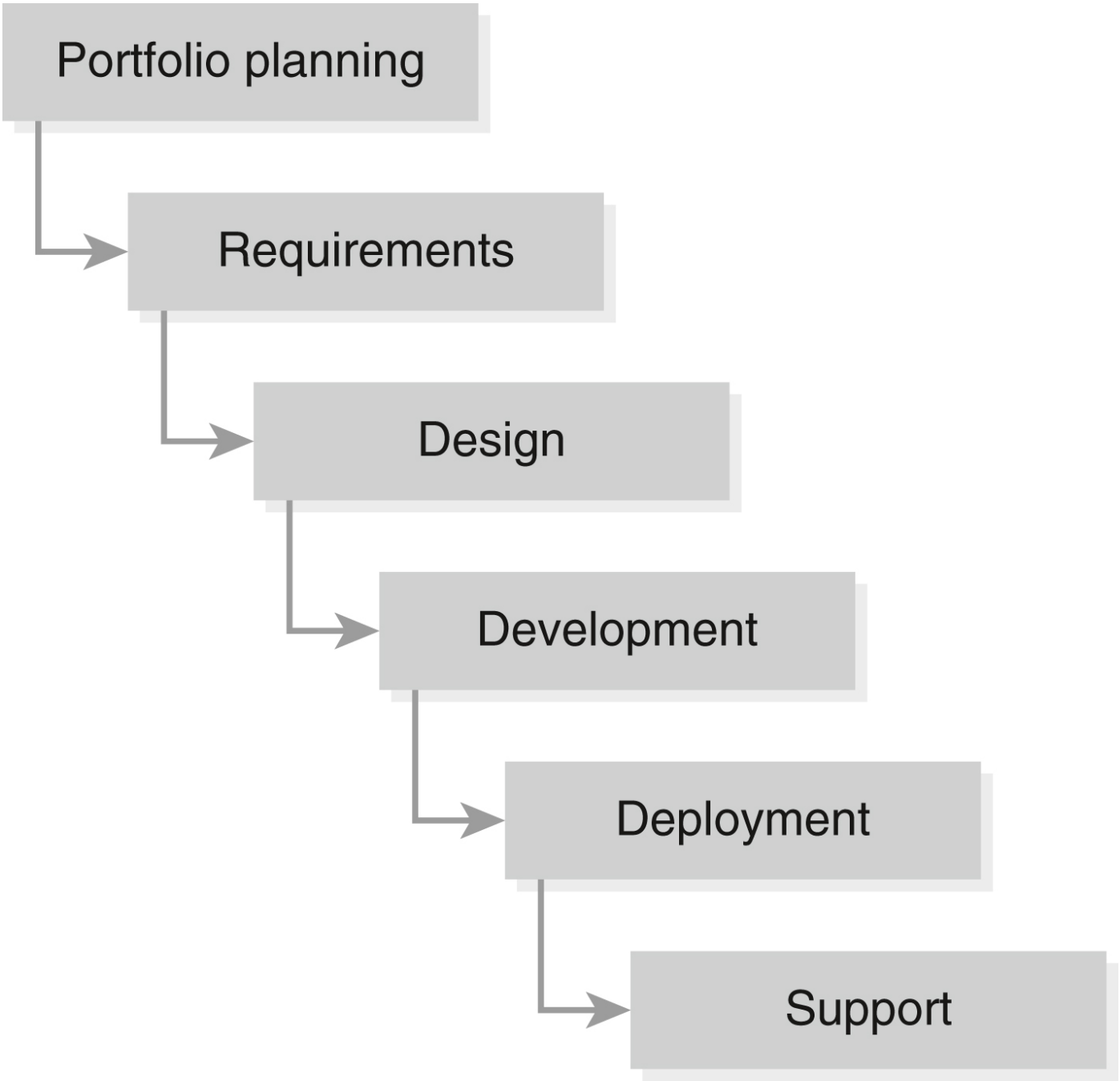


FIGURE 6.2 Areas to Consider Security

Security Is Widely Discussed and Shared with Customers

In a mature security culture, it should not be uncommon to wander down the hallways and hear random snippets of conversation about security. The level of comfort and degree to which everyone is aware of their role in security in a mature organization should naturally lead to discussions in the same way that other product features and company culture generate discussions. If there were never any discussions in the hallways and cafeteria about security, then one would question whether it is of much importance to the organization.

Just as security is a common topic of discussion among coworkers, it should also be a common topic of discussion with customers. Everyone should have something to say to the customer about security. That is not to say that everyone needs to be an expert about every aspect of security, but they do need a level of awareness and an ability to direct questions beyond their level of awareness to the right person. Nothing inspires a lack of confidence in a customer than when they ask an employee about security and receive the answer, “Yeah, I think we have someone who handles that. I’m not sure who, but we’ll see if we can’t figure it out.” Worse yet: “Of course we have someone who does that. I don’t know who, but don’t worry, they take care of everything. We won’t worry about it.”

Security Is Valued

In a mature security organization, security is valued. Not in a cultural “yes, we value security” type of manner, but actually valued in terms of dollars and figures. Mature organizations know how much they are spending on security in dollar terms, personnel, and time. They also know how much customers value security, the cost security adds to their products, and the return on investment from their security practices.

Numbers are important for organizations. Numbers are how they report their status, earnings, and everything else significant to their shareholders or other stakeholders. Not to number something is to say that it is *not* important. If an organization values security, then there are going to be numbers related to it in their annual report and anywhere else they can put them. It is like the old saying “If you can’t count it, it doesn’t exist.”

An organization with no numbers attached to security places no value on security. “But numbers in security are hard!” Sometimes things are difficult to

define in terms of numbers, or an organization does not have the processes and systems in place to capture numbers. This does not have to mean that there can be no numbers. Organizations can start small. Perhaps the first numbers are only qualitative, given as approximations of the organization's security posture.

The CISO could look at the organization, judge that there are no known security breaches, but organizational knowledge and interest are low, so security is a "5." That number provides a starting point. A clear statement that the organization values security enough to assign a number. It provides something to measure against: "We did a companywide training event and now security is a '5.7.'" If provides a target, and it gets visibility. "Why is security only a "5"? We want it to be at least an "8." Arbitrary measurements are not really where anyone wants to be, but they are a starting point and can be a driver toward long-term accuracy and awareness.

Security Culture Maturity

As with anything else, a mature security culture cannot simply appear overnight. It is something that develops over time with the careful, thoughtful guidance of senior management, attention from leadership, and increasing buy-in from within the ranks of an organization. Much like with process maturity, a company can expect to go through different levels of security culture maturity. **Box 6.1** presents some expectations for each level of security culture maturity.

BOX 6.1 Security Culture Maturity

- Leven 1—Initial
  - Security is unpredictable, poorly controlled, and reactive.
- Level 2—Managed
  - Initial practices are in place, basis measures are used, and is still fairly reactive.
- Level 3—Defined
  - Practices are defined and implemented in most areas. Systemic approaches used.
- Level 4—Qualitatively Managed

    - Risk-based analysis and implementation of controls and practices are used. It is measured.
- Level 5—Optimizing
    - Focus is on continuous security improvement.

The lowest level would be a company with little or no thought for or understanding of security. Next would be a company with an emerging realization of the importance of security and some haphazard, nonsystematic application of security efforts. A moderately mature organization would have pools of more intense security awareness and the beginnings of systematic application of security processes and activities.

A mature organization would include a more pervasive consideration of security throughout the organization and the consistent application of security practices. Finally, a fully matured organization would have security as a normal part of everyday thought and action, with the personnel actively engaged in learning and applying more. The organization has an active and effective training program. Active metrics are used to assess current and developing security. All processes and activities would have security baked in. And all those security practices would be evolving in an organic process to address new concerns.

Company Policies and Procedures Are Established and Available

It is impossible to maintain a consistent corporate culture when policies and procedures are not clearly established, documented, and readily available. Establishing and documenting policies and procedures are useful because they can establish a mindfulness about them if done carefully. Rather than simply writing policies for the sake of having written policies, the process allows for an opportunity to reflect and evaluate the quality of the process. This prevents having policies that are “the way we’ve always done it.”

A policy that exists but is not available and accessible is worse than having no policy at all. A policy for form’s sake sitting on a shelf cannot be followed, and in absence of the knowledge of the content of the policy, employees are going to base their actions on their individual judgment, on the way things appear to be done, or on what actions would appear to provide the best cover. Only coincidentally would any of these be what the policy intended. As the

actual actions diverge from policy intent, the difference between what is occurring and what was planned increase. This causes continuing responses to become even more ineffective.

In not knowing the policies, the organization is also in a position of noncompliance where an audit or inspection would reflect the difference between the policy and the actions taken. Even if the actions were effective, the noncompliance still exists and can lead to problems. If the actions were not effective, the organization could potentially find itself in a serious breach with potentially significant liabilities.

Even when a team is familiar with policies and procedures to the extent they feel the manual can be left on the shelf, the culture shouldn't allow it. Even then the documentation can serve as a checklist to walk through, just to ensure the right steps are taken. Would anyone be comfortable flying in airplanes where pilots felt they had the preflight process down to the point that they no longer needed to use the preflight checklist? If the reasoning is that the policies or processes are too onerous to follow, then there is a problem and there needs to be a revision.

Complicated, onerous, and obscure policies and procedures do not promote good cybersecurity, but they do create a culture where cybersecurity will not be respected. Clarity, focus, and conciseness create effective policies that can be easily followed and an environment where people are willing to follow them.

An Effective Security Culture among Programmers and System Administrators Is Vital

While a security culture throughout an organization is important, one of the most critical areas for having an effective culture is with an organization's programmers and systems administrators. Programmers and systems administrators are in particularly sensitive positions where their daily tasks directly affect security throughout an organization. They also have a significant amount of pressure on them to produce results rapidly. Often for them the choice is between the quick way and the right way.

The approach of these critical personnel to cybersecurity also helps set the tone for cybersecurity throughout the company. Even if not in formal leadership positions, these personnel are among the culture leaders of the organization. A casual attitude about security from them sets a tone of casual unconcern about security throughout the organization. A system administrator who regularly bypasses security to help others and "get things working—we'll go back and fix it later'" sets a perception that security just isn't important. With key people in

these positions setting that tone, the lack of concern for security will run throughout the organization even if the company leadership is trying to push a different attitude.

Responsibility

Managing the development of a secure software system involves a great deal of responsibility beyond the complex tasks involved in managing the processes, projects, and programs. A healthy organizational culture reaches beyond just security and also embraces ethical integrity, professionalism, and long-term thinking. While these traits can be supported by the culture, they need to be individually cultivated.

Ethical Integrity

Ethical integrity is being consistent with the ethics professed and actions taken. The professionals involved in secure software development are often placed in positions where they need to make choices about how things are done and how issues are addressed. They are asked to report about status, capabilities, and accomplishments. The easy part of ethical behavior is to simply say that the truth about the statements they make should be beyond reproach and the code they write should be flawless. The harder part is to understand the ethical implications of the systems they are developing and the decisions they make.

In some cases, questions about what a software system can and cannot do are baked into laws and regulations. These requirements or restrictions dictate the kinds of uses and functions of systems. While these are great for the politicians, bureaucrats, and lawyers to debate and pursue, a more significant influence on how systems can be used is how they are built. These systems can change or support the way society functions.

For example, some systems are built such that their logging and processes allow for easy tracking of who completed what functions within the system. While there may be any number of good reasons for doing this in a system tracking financial transactions, the same capabilities could be abused in a system used for reporting fraud, waste, and abuse where a user may be seen as a threat. Other ethical dilemmas may come from whether we should include enticements to children to make in-app purchases, or even what we allow people to answer in response to a simple question, such as allowing only a simple radio-button as a response to asking a user's race. If the choices are only white, black, Asian, or "prefer not to tell," we have ignored and offended those who have a multiethnic background.

Another ethical concern is whether we are collecting data that are unnecessary, especially as they may disclose information of a personal nature.

Professionalism

Sometimes professionalism and ethical integrity are convoluted into being the same thing. They are related, but they are different. While it encompasses ethical integrity, professionalism reaches more broadly. **Professionalism** revolves to a high degree around how we interact with and treat others. It invokes not just the golden rule of treating others the way we would like to be treated, but it also calls for ensuring that our interactions with others are clear, concise, and that we have done our best to be effective.

Professionalism also demands that we perform at our best in our endeavors, ensuring that we put the proper effort into all aspects of our tasks. It means that we follow processes correctly and put the same effort into testing and documentation, even if we enjoy coding a whole lot more. Professionalism also means that we educate ourselves properly about what we are doing and don't undertake tasks that are beyond our ability, for which we are not qualified, or that we know we cannot properly perform. That doesn't mean that we don't stretch ourselves and reach beyond our current capabilities, but rather that when we do so, we do it thoughtfully and carefully with full disclosure and a plan for how we are going to acquire the skills we need.

Sometimes people mistake professionalism for close adherence to specific details and standards. These are things such as putting the exact right words into a form, being at meetings 5 minutes early, wearing a suit, or using the right template for a slideshow. While paying attention to details is great, rigid adherence at the cost of lack of consideration of others, failing to attain goals, or being intolerant is not true professionalism.

Long-Term Thinking

Long-term thinking is another component of responsibility. As a long-term thinker, we consider not only the immediate consequences of our actions and the systems we build, but also future consequences. Some software was written many years ago yet is still in use. As an example, I once had a programmer working for me who retired at the end of a 40-year career with the company. His first assignment when he began was maintaining a specific COBOL program. His assignment just before he retired was maintaining that same program. Part of the challenges he faced in his career were that the

software had not been designed to survive the year 2000, or even the decade-to-decade changes, such as 1980 to 1990. The year field was only a single number. Had the program been designed in the beginning with an expectation of being used for many years, this programmer may have been saved from a significant bump in his workload every 10 years. Had he himself taken a longer-term viewpoint, the first time he encountered the problem, he could have put in a fix that would have eliminated the problem as future decades approached, including the change of the millennium. In that case, his career may have extended longer instead of him retiring before having to deal with the change—then again of course, the looming workload may have been just the excuse he was looking for to retire.

Building software systems that will endure requires foresight. This is not expecting the architects, designers, and developers to be clairvoyant and predict the needs of the future, but rather it means building systems in a manner that allows them to adapt to future needs. Much of what is required for this is good practice for the present as well as modularity, concentration of function, clear documentation and readability, a single place for each function, and other design principles.

Additionally, long-term thinking requires considering the effects of the system on society and how the system will be used. The latter is an especially challenging task. People expect others to use a system in the same way they do or in the manner it was designed to be used. Often though, users don't. Trying to anticipate how they actually will use a system and enabling the system to be effective or functional even when it isn't used as expected is important.

Training the Organization to Be Secure

One of the key means to influence the security culture of an organization is through training. Not just any training though, but the right training given at the right time. To be effective, training must be designed and built with the specific needs of the people who will receive the training and must be delivered when it can be most effective, in a manner that is most effective for them. Because people in an organization have different roles and responsibilities for security, each person needs a training program tailored to those roles and responsibilities. Too much training in the wrong areas and at the wrong times can be just as damaging to security as too little training. The principle is to provide appropriate training to the appropriate people at the appropriate time.

What Training Is Important

Eager to get security training done, some companies dive in quickly into in-depth security issues. These companies teach all their employees about phishing, spear phishing, and whaling and the differences between the three. They add curriculum about buffer overflows, denial of service attacks, ping-of-death, ack backs, as well as whatever flavor of the month the CISO ran across. The result is bored employees who don't understand what they were just taught and who really don't care because they will never look at anything at that level or have any real reason to name the specific attack that caused them problems. Other companies take the buzzword bingo approach, trying to get employees familiar with the language of cybersecurity rather than the concepts. The effect is the same.

Some important areas for consideration when training including training for awareness, training on policies and procedures, and training for specific types of users, such as management, designers and developers, and users.

The reality is that not every employee needs to know all the details about security. Most employees need **awareness** training rather than specifics. Employees only need specifics about things that will directly affect their use of technology, such as how to create and manage a secure password. For the rest of security, most employees need far fewer details, particularly about specifics. What they do need is to know how to recognize potential security challenges, apply general security principles, and find help and guidance when

they encounter an issue. This type of training is going to be far more effective than training focused on details, which can never cover all potential details.

What each employee does need to know is how to recognize potentially compromising emails, signs that there may be an attack or issue in progress, and how to conduct oneself day to day in a manner that doesn't lead to security compromises. Of importance is creating in employees a sense of the importance of following secure practices. Often understanding what can happen when security fails is lacking.

Specialized training is going to be needed for people in specialized roles. Developers and designers for example need a significant amount of training in how to design and build secure systems. The depth of their need to know is going to reach far beyond the common user in these areas. At the same time though, one can't forget that they are still users and need appropriate training in that area too. Likewise, training for a manager should be more thorough than training for the average user, while still ensuring managers receive training at the user level.

Besides training on what is needed for security, it is also important to train on the limitations of security. While it is important to know what needs to be done to maintain security, employees need to know that even with the best security systems, processes, and procedures, there are limitations to the degree of security these can provide. This can both help them to avoid going beyond those limitations and to identify with security has not been a hundred percent effective.

Part of training is to provide an awareness of what happens when security fails. The most meaningful way to do this is to show actual, relevant examples of what has happened in the past. But these can't just be a flood of horror stories. The cyber horror show is an easy thing to assemble and will certainly impress the audience; however, it quickly becomes ineffective. When the examples are too extreme or are not related to the actual environment of the employee, they are easily dismissed. Extreme examples could also become numbing. Employees throw up their hands and decide there is nothing they can effectively do despite their best efforts, and the bad guys will win. So limit the extreme cases, focus on relevance for the situation, and show examples of where everyday employees have effectively recognized and thwarted cyberattacks. With this done, trainee attention is captured, they become engaged, and the training is going to be meaningful.

When to Train

A carefully constructed training plan is not simply a single class, but rather a plan for how the company will deliver information that employees need when it is needed. It would include formal training courses, insertion of cybersecurity training into other training events, awareness activities, and other methods for constant consistent delivery of the security message in a meaningful context. The delivery mix is designed to avoid being a big bundle of boredom where the primary response of the employees is to avoid it rather than embrace it.

Deciding when to train on which areas of security is important. There are some indicators for the organization that can be used to focus the timing. An important part of that is to have an awareness of what people are doing both in the organization and in the environment around the organization. This awareness may result in the identification of specific trends and threats that merit training in specific areas of security or a need for a general awareness refresher.

Being aware of upcoming events with specific, related vulnerabilities or distractors can provide a source for planning specific training needs. In fact, any time an organization undergoes significant change, focused training about the associated vulnerabilities is needed. Just as organizational changes require training, individual changes also need training. Appropriate times for training would come from moving to new jobs through promotions and transfers, undertaking new responsibilities, or in returning from extended absences.

Naturally, with such an approach, security training isn't a one-time event. It is a process. As the process happens, it needs to be planned so that it serves to always keep employees at a sufficiently high level of awareness that they do not become a weak spot in the organization.

Training needs to be refreshed periodically over time. Too long between refresher sessions causes big dips in security awareness and skills. Lengthy periods require that the training sessions be longer and more intense. This may also result in decreased retention. Frequent, small, focused sessions can retain a high level of security awareness and skills while not being a pain for the people who consume the training (see **Figure 6.3**).

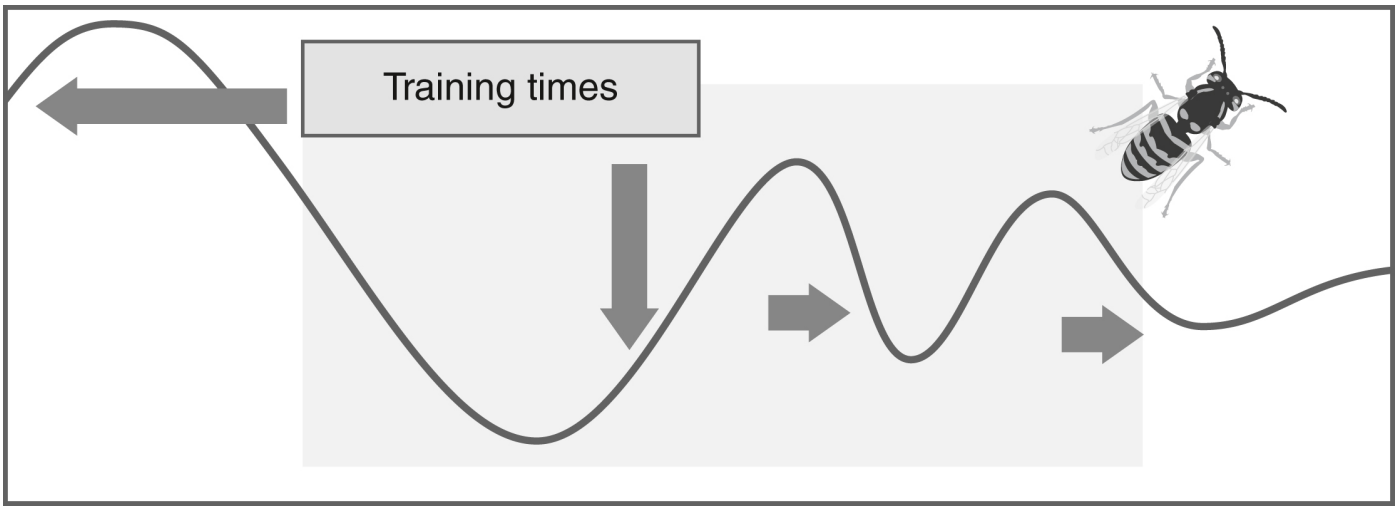


FIGURE 6.3 Training Refreshes Awareness over Time

Description

Training Style

Where to train for security is important to consider. Some training is best suited for the classroom, other types online. Many times, training on-site is the best option because it is where the security situations are encountered, and having been trained on-site, the team member is attuned to encountering and overcoming threats to security in that environment rather than if they have to translate some sort of a vague academic concept into their workplace.

Security training should also contain as many hands-on active components as possible. A currently popular training technique is for the company to conduct a “phake-phishing” campaign. In this exercise, the company sends out fake emails to see how many of the employees respond in an improper manner. Those who do are then trained how to recognize phishing emails and respond correctly. If done properly, this training can be effective in both building awareness and in training how to respond. If done poorly, where management reacts negatively to employee responses, then it can become a demoralizing incident. The objective of training exercises is not to penalize failure, but to identify weaknesses and build strengths.

Management needs to recognize that cybersecurity training is not just a formal training program pushed out independent of whatever else is going on in the organization. Training is also a result of day-to-day attitudes, responses, and management actions. These need to be consistent with the ongoing formal and informal corporate training.

SUMMARY

The organizational culture is going to play a significant role in how secure an organization and the software systems it develops are. Organizational culture begins at the top; however, efforts and wishes for a secure corporate culture are all for naught if they do not focus all the way throughout the organization with a specific focus on ensuring that the first-level supervisors are onboard. At some point, a healthy corporate security culture will be readily apparent from observing the organization and how it behaves.

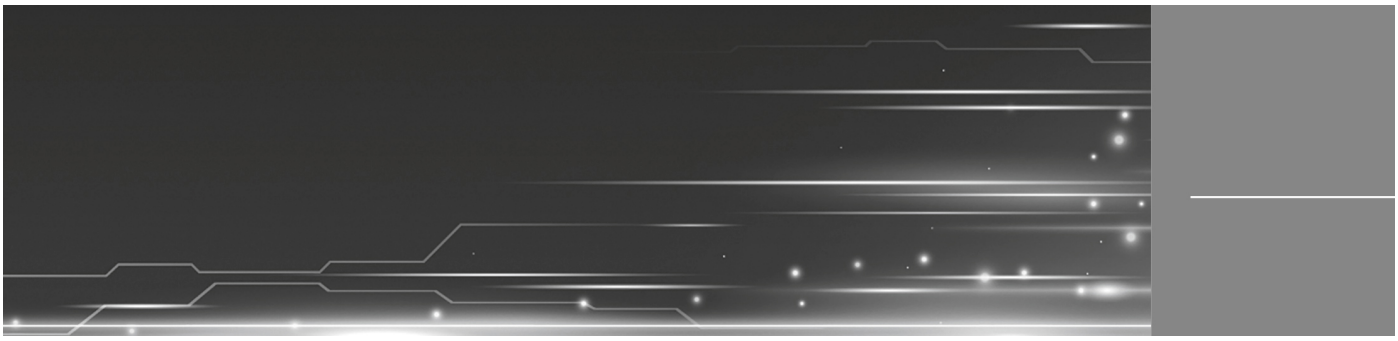
Naturally, a key part of security in any organization is having members who exhibit personal responsibility. This foundation is built with ethical integrity, professionalism, and long-term thinking.

Another key area in developing and sustaining a healthy, secure organizational culture is a well-planned, well-executed program of security training. This training needs to be designed to provide people throughout the organization with the training they need in an effective manner in a setting where it can be applied.

Organizations with members who act with personal responsibility and are properly trained naturally develop a secure culture. Organizations with a secure culture naturally attract members with a sense of personal responsibility and train them appropriately.

KEY TERMS

authority
awareness
buy-in
chief information security officer (CISO)
ethical integrity
first-line supervisors
impedance
leadership
long-term thinking
organizational culture
professionalism
responsibility
risk appetite
training



© d1sk/Shutterstock

CHAPTER 7

Requirements and Security Requirements Planning

CHAPTER OBJECTIVES

- Examine and apply methods for collecting requirements for a software system.
- Compare different sources of software system requirements.
- Trace how requirements are traced through the secure software systems development life cycle.
- Apply requirements analysis and engineering techniques.
- Compose documentation for requirements.

Introduction

A significant portion of the defining and planning stage of the secure software systems development life cycle is defining the system requirements. This is a real challenge that to some extent is both an art and a science. That is to say, there are rules and methods (the science part) to collecting and managing requirements, but knowing how, when, and which of the rules to apply is a challenge that requires careful consideration, judgment, and style (the art part). Requirements are not simply the general concept of what the software will do, but the specifics of how that concept will be accomplished.

Types of Requirements

Requirements come in two flavors. **Functional requirements** are what the system must do in order to accomplish its purpose. **Nonfunctional requirements** qualify or constrain the way the system is developed, system performance, or how it completes a functional requirement. Even though they are not specifying what a system must do, nonfunctional requirements may be just as or even more important than functional requirements.

From the vantage of secure software systems development, security requirements are another critical aspect. **Security requirements** are not only those protections that are to be put in place in the system, but also those that need to be put in place and followed during the development of the system. This is essential because without the protections being in place during development, the opportunity exists for the system to be compromised even if it is being built to be thoroughly secure.

An important concept is the **secure functional requirement**. Under this concept, as standard functional requirements are developed, each functional requirement is considered in terms of security. Does the requirement have associated potential vulnerabilities, or could the requirement could be abused in some manner? As the requirement is reviewed, modifiers are added to ensure it is performed securely. Often these modifiers specify what should not happen. For example, if a requirement calls for collecting the amount of a deduction from a bank account, the requirement may be modified to include that the amount must be a positive number (otherwise it could become a deposit into the account rather than a deduction).

Functional security requirements are used to describe the security that must be supported by the system as a part of its mission. These requirements reach beyond modifying the other requirements. Examples of functional security requirements include implementing user accounts and account security, creating distinct domains, allowing administrative functions distinct from user functions, providing encryption for data and communications, supporting specific security protocols, etc. These are features the users are going to see and explicitly use.

BOX 7.1 Secure Functional Requirement versus Functional Security Requirement

Though similar in name, these are different.

- A *secure functional requirement* refers to a standard functional requirement that is made to be secure.
- A *functional security requirement* is for a specific security function.

Other security requirements are nonfunctional. Those would be requirements that need to be implemented in the system in order to support security but are not viewed as a part of the system function. Examples of nonfunctional security requirements would include logging, audit trails, and graceful degradation. Other nonfunctional security requirements may specify the way security protocols and standards will be implemented in the system or data classification guidelines. Or, as we will see later, security requirements may be for any libraries or application programming interfaces used in the system.

In addition to the requirements for security in the system itself, it is necessary to have security requirements in the development of the system. Those requirements may include requirements for development, testing, and certification processes, requirements for tracking security concerns, and even requirements for security checks of the personnel working on the development. These will also include security requirements for tools used in the development or coding guidelines.

A final set of security requirements will be security training requirements. These requirements specify training for developers, testers, administrators, and users of the system. Training is an important part of any security program, and each system needs to have its specific security training requirements clearly identified.

System Definition and Scope

Defining the system is closely tied to the requirements process. The system definition was begun in the system initiation stage and is continued being refined in the defining and planning stage. The full definition often isn't completed until the requirements are complete because the requirements process can change the system definition as the understanding of what needs to get accomplished changes. The system definition is closely related to the scope of the development. While the definition establishes what the system is, the scope establishes the work that will be done on the system during the development.

The two core parts of defining a system are figuring out what the key functions of the system are and what the system boundaries are. **Key functions** are what the system needs for it to perform its function at the basic level. The key functions are often relatively well understood, at least at first. But then as we go deeper into investigating the requirements, we may find there are key functions we didn't realize needed to be included or without which the system isn't able to perform its intended purpose.

For example, one time I was asked to develop a system to collect orders from customers for a particular kind of telecommunications service, but then discovered that in order to provide that service, a method was needed to update the service needs into the switches. The intent of the requester had been to have that manually completed, but given the complexity of the data required and the delays in implementing the service using a manual process, it rapidly became apparent that the system needed to be able to update the switches. This change in the definition of the system involved some additional complexity, but because the ordering system already had the needed data, the additional cost of automating the system definition was minor compared to the return on investment.

The **system boundaries** specify what functions will be performed within the system. Boundaries include key functions as well as any extended functions. Establishing the system boundaries is just as important, if not more so, that determining what a system will do. Without effective boundaries, the system definition eventually expands to include everything. Not only will it perform its core tasks, but it will also include a new, highly optimized operating system; a new, optimized processor design; and to support that, a new method and factory for fabricating chips. Not every system definition grows quite to that

extent, but it is really easy for a system to grow quickly beyond the original intended bounds. That isn't always bad, as in the previous example, but it needs to be done intentionally and for specific reasons.

Just as with the system boundaries, the scope boundaries of a specific development effort need to be clearly defined. The **scope boundaries** will include defining what functionality will be added, what portions of the system will be touched or left alone, and the level of effort that will be available to develop the system.

Often when we are defining the boundaries of a system or the scope of development, we will very intentionally say what the system or scope will not do or will not include. There can be two different lists, one that says what is included in the system (**in-scope**) and another that says what features, functions, and changes will not be included in the development effort (**out-of-scope**). These could be fairly long lists. The earlier these boundaries are defined, the easier it is going to be to stick within them.

Sometimes we will want the system to be able to do functions later, in another version or release. For these functions, we want to clearly state them so that during the definition of our system we do not preclude the ability to later add those features or make it too difficult to do so effectively. Often these features appear as we go through our requirements gathering and analysis process. In that process, these can get tagged as future features or requirements.

We will encounter other features and requirements that are important, nifty, or otherwise great, but not for this system. Rather than expand the scope of the system, these can be gathered for consideration for inclusion in a different system. Features and requirements that fall outside of the system boundaries or scope and haven't yet been determined to belong to a future version of the system or to another system or may be reconsidered later can be placed in a "parking lot" to be addressed later.

Sources of Requirements

There are going to be multiple sources for gathering system requirements. The first task of the process, once the system and scope definition are completed, is to identify the sources. With a variety of sources, each will have contributions.

One of the first places to look for requirements is the system definition, scope, and intent. Based on these, a number of either directly stated or derived requirements will be found. Another source from which we can derive requirements is from the mission statement of the organization. That combined with the system and scope will imply certain requirements that we must meet to stay consistent with the organizational mission. After all, what good is a system if it does not support or if it contradicts the mission of the organization it is built to serve?

Another primary source of requirements will be based on laws, policies, and standards related to the system and the organization. These sources are collectively grouped as compliance and will be discussed in another chapter.

Similar to compliance sources, but differing slightly, is the use of industry best practices as a source of requirements. Looking at what others in the area are doing is a good gauge of what works well. There are a few issues of concern though because what works for one organization may not work for another and just because that is the way it is done, does not mean that is the right way. Also, to be considered is looking at how other organizations in other industries do things. Too often people think their industry is unique and needs to do things differently. The reality is that 90 percent of everything done is done the same across all industries, and only the final 10 percent needs to be done in an industry unique manner.

Some requirements will come from the other systems with which our system will interact. Those systems may have security, format, and protocol requirements. It may even turn out that we do not really want to meet those requirements in our system and instead we develop requirements to put another system or intermediary in between to act as a buffer.

Stakeholders as a Source of Requirements

Another source of requirements will be from the stakeholders. These requirements can be identified from knowing who the stakeholders are or

provided by the stakeholders themselves. **Stakeholders** are those who have an interest in the system being completed correctly. Primary stakeholders will be users and administrators of the system. Any system may have multiple types of stakeholders.

For example, a web-based ordering system will have administrators who manage the system, corporate-based users who add products to the system or receive the orders from the system, and consumers who make orders online. Each of these stakeholders will have specific needs they will want to be supported in the system. What is important to one type of stakeholder may not have any significance to another.

Secondary stakeholders will not use the system directly but will be recipients of the functions of the system. They might be owners of another system with which the current system will interact. It could be a consumer who receives a bill in the mail that has been produced by the system. Just as with primary stakeholders, secondary stakeholders will have different requirements based on their own perspectives.

One key stakeholder who always needs to be considered is the project sponsor. After all, they have the money and key decision-making authority. That makes it imperative to consult them for their versions of requirements.

Scatter Analysis of Stakeholders

Beyond simply identifying the stakeholders, it is often useful to perform some analysis of the stakeholders. This can help in deciding where to focus attention in gathering requirements and later in their analysis. One type of analysis that can be useful is a scatter analysis. In the scatter analysis, each stakeholder or group of stakeholders is evaluated for their level of interest in the project and their level of influence. Often the best approach is to evaluate most stakeholders in their groups, but selected stakeholders individually.

The level of **interest** a stakeholder may have can vary greatly based on whether they are a primary or secondary stakeholder, whether they have time to become involved with a project, or based on their attitude. For some stakeholders, the particular project may simply be one of many with nothing in particular to hold their attention. At other times, a stakeholder may have significant interest in a project simply because it intrigues him or her.

Influence is the ability of a stakeholder to affect the project. In evaluating the influence of a stakeholder, many kinds of influence should be considered. The first of course is money and resources. Some stakeholders have control over these. Others will wield influence on a project due to their management positions and their ability to support a project in the organization. Still others will

have influence as a result of their ability to sway the opinions of others regardless of their own position.

A good example of informal influence leading to being rated as a primary source of requirements is the help desk technician. The techs have a strong interest in the system because they will need to support it in the future, and the better the system is built, the less work it will be for the techs to support it. At the same time, the techs have no money or resources to apply to the project. They have no managerial influence or control in being able to support a system in the organization. But, despite all of that, they have significant influence in how a system is viewed and liked within an organization. It is their attitude toward the system that will influence the users to either view it as a "good system with a few idiosyncrasies" or a "total failure of a system that fails to support the needs of the user."

Once we have plotted where the different stakeholders appear on our scatter diagram, we can begin to do some analysis. For a really rough level of analysis, we can group the stakeholders into four quadrants, as is diagrammed in **Figure 7.1**. The upper-right quadrant is where we find the stakeholders with both a high level of influence and a high level of interest. These are the stakeholders we want to make the primary focus of our requirements gathering efforts. But we still don't want to ignore all of the other stakeholders.

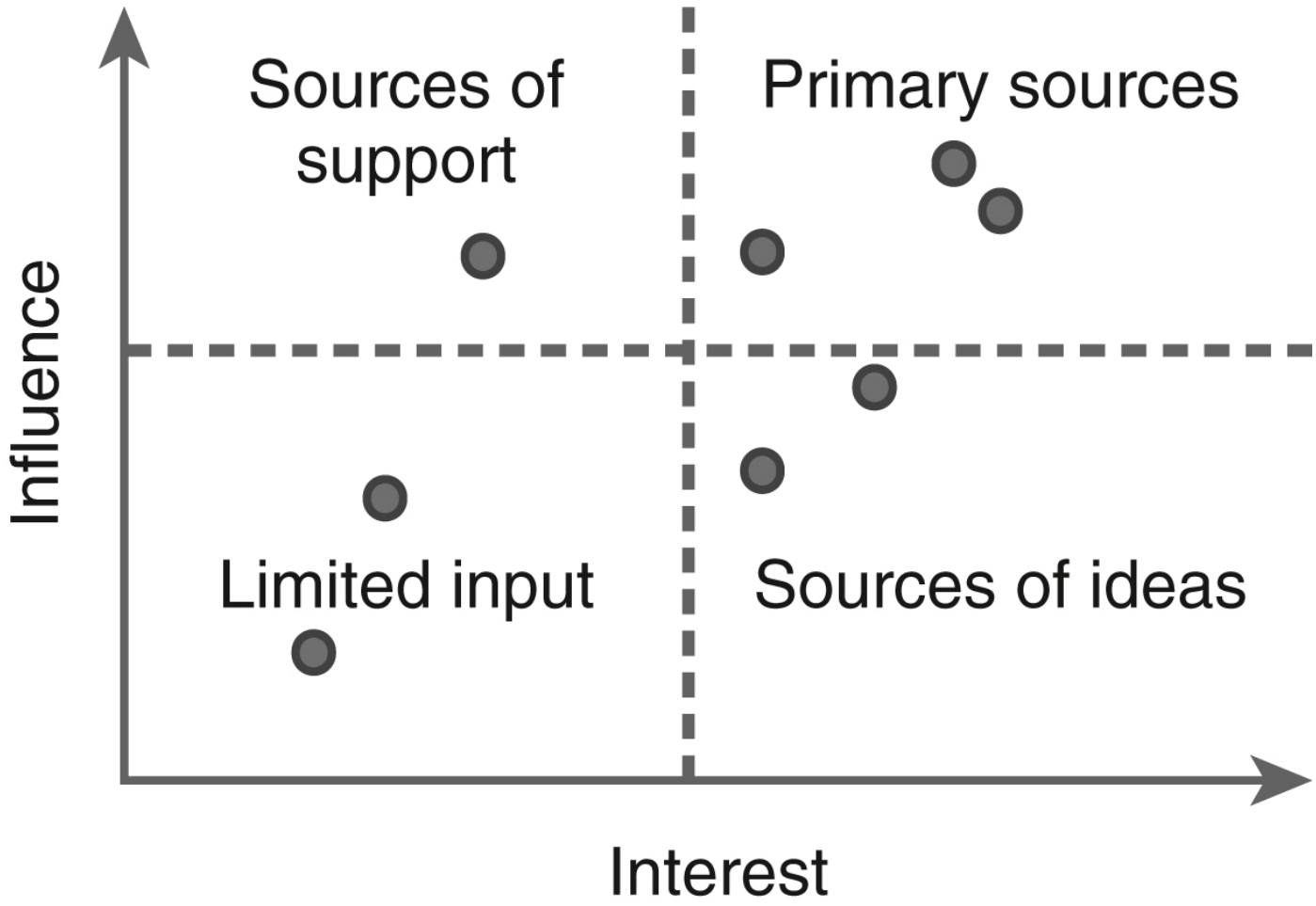


FIGURE 7.1 Scatter Analysis of Stakeholders

Description

Those who are in the upper-left quadrant are stakeholders who still have a high level of influence. We forget them at our peril. But because their level of interest is low, they are not likely to become major sources for requirements. Instead with them we will want to focus our efforts on getting their buy-in and support for requirements we have acquired from other sources. Ideally, we can do this using methods that are a low barrier for them and keep their influence positively directed. Simple things like frequent updates and key feature reviews can be useful for this.

Likewise, those in the lower-right quadrant can be useful as well. While they may not have a great deal of influence over the project, they have a high interest in the project and are going to be willing and able to provide a good number of ideas for different requirements. Because of their low influence, many of these ideas will not be inhibited in ways that others may be. The stakeholders in this quadrant are often a good source of ideas. Because of their lower levels of influence, they also often don't have tight ownership of their

ideas and are willing to let them go or change them as needed. Many of the "left field" ideas that make a system truly innovative can spring out of stakeholders in this sector.

Finally, while we won't want to put much focus on those in the lower-left quadrant. While they have little interest and little influence, they still matter and can benefit from being kept informed.

Gathering Stakeholder Requirements

Having identified the stakeholders, the next step is to determine how to collect the system requirements from them. Usually there is no single way in which this is done, but rather a variety of different approaches are used for different stakeholders and groups of stakeholders and then the results are combined. Different methods work better with some stakeholders than others, and each method will result in different combinations of requirements, so using multiple methods is essential.

Methods for **requirements solicitation** can be categorized as individual or group methods. There are reasons for taking both approaches. As individuals, stakeholders are going to be less affected by others' ideas and won't be concerned that others may judge the qualities of their ideas. Also, with an individual, we can focus on drawing out a depth of focus on requirement ideas that we may not be able to in a group. However, it can take longer to solicit ideas from each stakeholder, the results can be very repetitive, and the stakeholder doesn't benefit from the ability to hear the others' ideas and build on those in a collaborative manner.

Individual methods can include sending a solicitation to the stakeholders, a request for inputs. This solicitation can be formal or informal and may be done via email or through some other communications channel. Usually, it is best to provide the most complete system description possible, including the boundaries, and any other information that can inform the stakeholder about the system. The solicitation may also include prompts about areas to consider for requirements. This helps stakeholders frame their ideas and think across the spectrum of potential requirements.

Another common method to solicit requirements would be using a survey. The survey can provide a more structured approach and take less time. It allows stakeholders to rate the importance of different potential requirements and to select among different possibilities. The survey approach is often good for the upper-left quadrant of strong influencers with limited interest because it allows them to quickly give their opinions without taking a lot of their time.

If there is an opportunity, a great method for discovering requirements is to sit and watch how the work is currently completed. This method is often referred to as “observation.” This gives both an opportunity to match the descriptions of how the work is done with the reality and the chance to observe details that may seem too trivial to report but are actually essential to completing the tasks properly. This type of observation can also allow the discovery of inefficient and ineffective steps in the current processes. A good means of applying this method is to have people doing the work describe what they are doing as they go through the process. It can be really interesting and informative, not only for the observer, but also for the people performing the process. More than once while doing this, individuals have stopped mid-sentence, questioned why they are doing something the way they are, and then made changes and corrections to the process, which resulted in immediate improvements even before the new system was built. These improvements can then be incorporated into the new system, adding increased effectiveness. As with any other method, it is important to remember that observation has its limits. For example, it may miss periodic activities like a monthly reconciliation or other activities that didn’t occur when the observation was taking place.

Finally, requirements can be gathered through one-on-one interviews with stakeholders. This is the most time-consuming option and should only be selected in a limited number of instances. However, it allows the most detailed capture of requirements and allows for following rabbit trails of hints of additional requirements. To use this technique, the interviewing skills of the requirements collector are important.

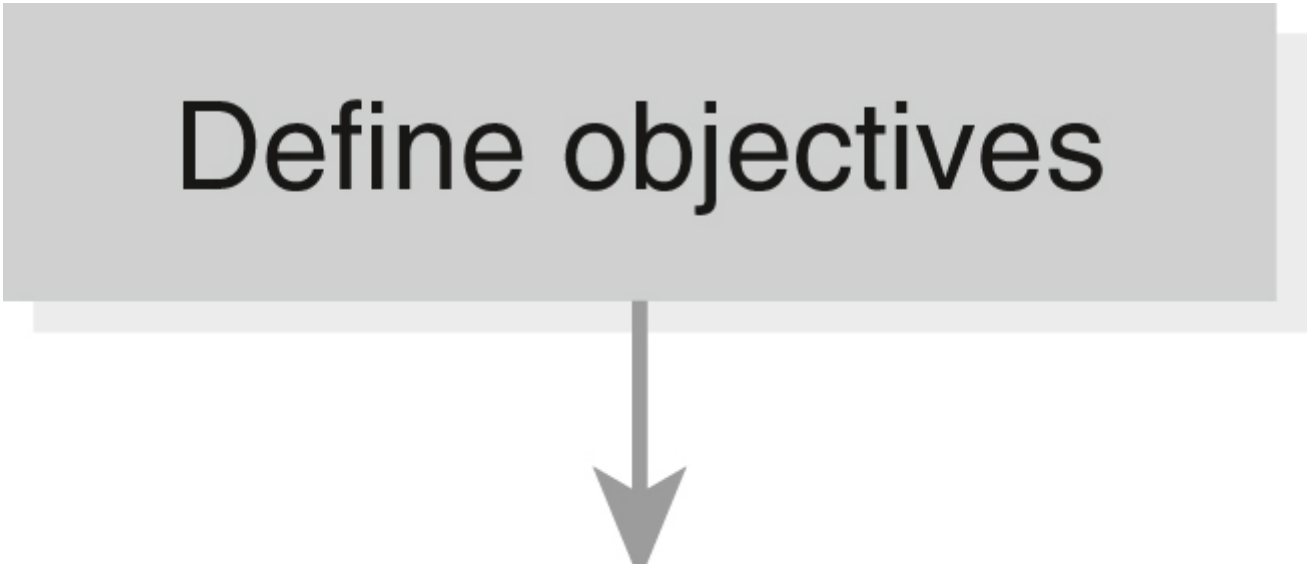
Soliciting requirements from a group of stakeholders overcomes some of the challenges of soliciting requirements from individuals. It is generally quicker, and it allows different stakeholders to build on each other’s ideas. Additionally, it often can result in the coverage of a broader set of requirements than would come from any one individual. The challenges with group solicitations on the other hand include a tendency toward group think, the potential dominance of group interactions by a small subset of the group, the hesitancy of many to open up their ideas in front of a group, and possible negative reactions to ideas presented. Because of these drawbacks, it is really important to consider the types of group sessions used, how those sessions are run, and which stakeholders are included within any particular group session.

Even with those limitations, group sessions are perhaps one of the most common methods used. When used in combination with selected individual sessions, they can improve the individual sessions with the stimulation of ideas and by probing into different areas.

Among the common types of group sessions used are focus groups. These bring a selected group of stakeholders together to consider aspects of the system in one, or a few, areas at a time. Multiple focus groups can be used to cover a range of different areas. For example, one focus group might consider the user interface, another the administrative interface, another data requirements, and another consider which systems the new system needs to interact with. Some stakeholders may be members of multiple focus groups, while others may only be involved in a single group.

Another common type of group solicitation is the brainstorming group. In the brainstorming session, members are asked to throw out as many ideas as possible about what the requirements might be. No idea is judged as it is presented. Only after the ideas are all out on the table does the group consider which ideas should be included as requirements. Usually as a part of the process, a facilitator guides the group through different areas of potential requirements.

A similar but more structured approach is a joint application design (JAD) session, as shown in **Figure 7.2**. In a JAD session, a broad representative base of stakeholders is brought together and walked through a highly structured process to solicit and refine requirements. One of the goals of the JAD session is to reach a consensus on the requirements and have a clear focus on the needs of the system. Ideally, stakeholders with different viewpoints should be able to reach consensus during the session, or at least understand each other's perspectives. Because of its structure and goals, JAD sessions are usually late in the requirements gathering phase with considerable work done to lay the groundwork for the session. Without this groundwork, the likelihood that the session outcome will incomplete or otherwise disappointing is high.



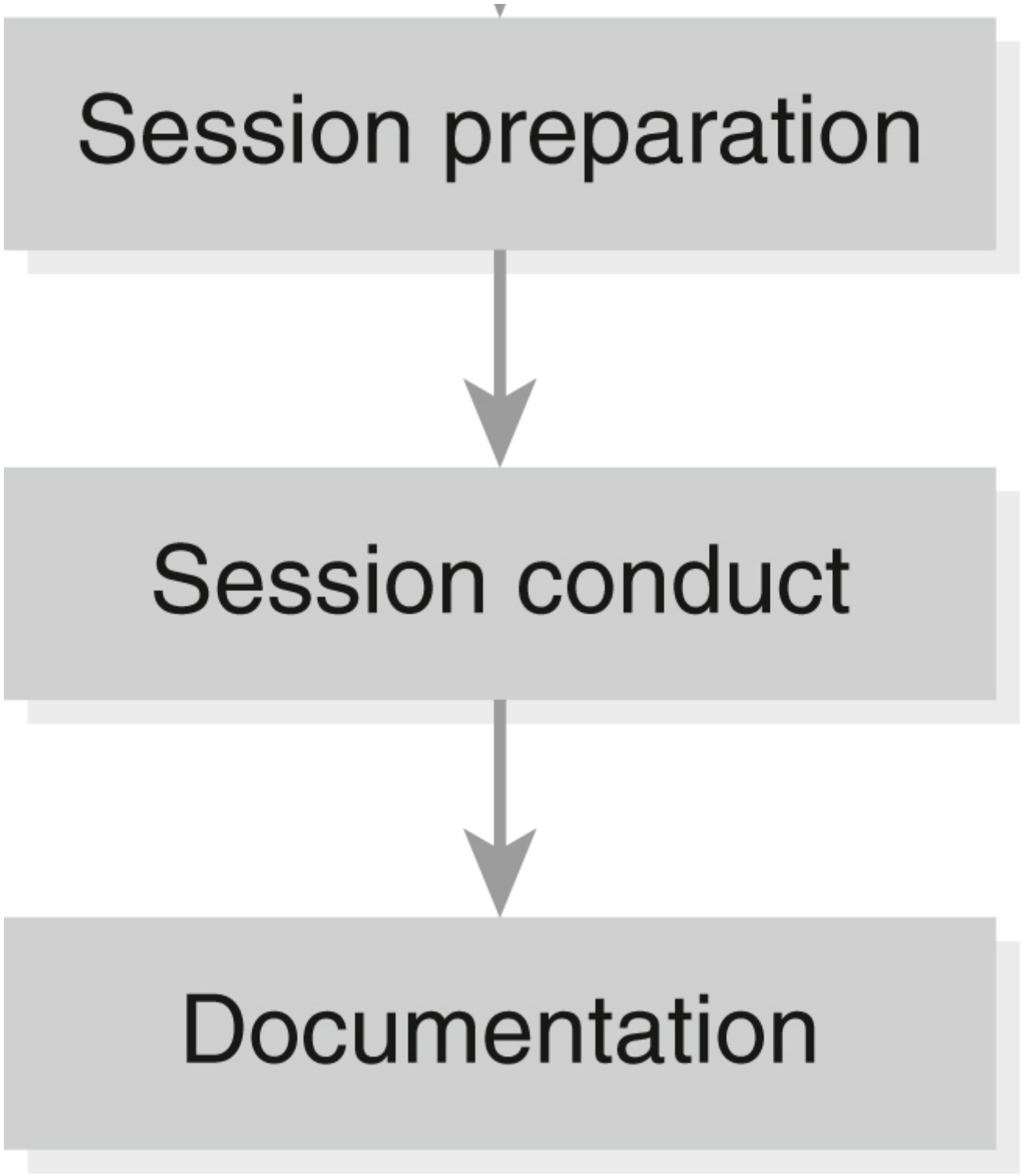


FIGURE 7.2 JAD Phases

Description

Working with Stakeholders

However, we choose to solicit requirements, we will encounter challenges in working with stakeholders. One of the greatest of these challenges is decoding what stakeholders mean when they give us their requirements. While our ideal

is to get precise, unambiguous technical specifications, this isn't always or often going to happen. Part of the problem is that stakeholders tend to phrase requirements in terms of what they know and are familiar with. We see this in everyday life when someone will ask for a "Kleenex" and they really mean a facial tissue; it doesn't need to be a specific brand. Or in some parts of the country, when a person asks for a "Coke," the proper response is to ask them, "What kind?" not to bring them a bottle of Coca-Cola.

Stakeholders also often use broad general metaphors to describe things. When an end user asks to have a system in the cloud, they are not actually asking to have the system hosted on Amazon Web Services (AWS), Azure, or some other public cloud and provided as software as a service. Generally, stakeholders have no real idea what that actually is and what it would take to provide it. What they really mean is that they want the system to be available through a browser or mobile interface and available wherever they happen to be. Whether it is actually run in a corporate data center on a dedicated server or is on a virtual machine spinning in a public cloud somewhere isn't even a factor they consider.

This use of specifics to describe generalities and metaphors can result in some significant problems if they are not seen for what they are. It is important for the person capturing the requirements to dig into the real meaning and intent behind what is expressed.

Another problem that often happens in the process of soliciting requirements is that requirements are limited by the known. Stakeholders, especially those who are currently dealing with older systems, have no idea of what is possible. Ideally, this shouldn't happen and instead of letting the stakeholders limit their requirements by their conception of what is possible, the ideas should flow freely, and the requirements limited later by what is actually in the realm of possibility.

One approach to overcoming possibility limitations is to expose the stakeholders to different systems and the functions they provide. This can open their minds to new ideas. The danger is that they may become enamored of the system used to demonstrate those possibilities even if it doesn't meet many significant needs. Part of the solution is to expose them to multiple different systems featuring different functionality, or to only demonstrate specific, or notional pieces of functionality.

Another method to help stakeholders figure out what their requirements are is to have them tell stories about how they are using the system. In their imagination they can visualize and then describe what they do and how the system interacts with them. By walking them through several potential

scenarios, these stories can cover a broad variety of different areas of requirements. Those storytelling sessions could be coupled with the description of different conceptualizations of potential systems or by graphics including pictures and storyboards that set the stage for the story.

Ideally, the variety of methods are applied in an iterative fashion where the results of each method, both traditional and nontraditional, inform the way the next is applied. Working through the different means should result in a fairly exhaustive and complete set of requirements, ready for the requirements engineering phase.

An issue frequently encountered when working with stakeholders is differentiating requirements from design. Requirements are what needs to be captured, not how to get that accomplished. In trying to describe what needs to be done, it is a natural tendency to describe how it should be done. The challenge then for the requirements gatherer is to unsort the elements of design from the requirements. It may turn out later that the given design is in fact the best way to implement the requirement. But it very often turns out that solutions for the requirement could be successfully implemented multiple ways. Sometimes the given design isn't even possible to accomplish or has some other significant problems.

This doesn't mean that we never allow stakeholders to provide requirements in the form of a design, but we do want to keep them aware of the differences between requirements and design. Having users produce screen mock-ups can be very informative about the types of functionality they include and how those functions are grouped together in their screens. Users may include other details that give additional clues about how they perform their tasks and other requirements. When using this technique, we do want to ensure that there is no expectation that the design elements will become requirements or be adopted as the official design.

There may be a few exceptions to the rule of separating requirements and design. Those exceptions generally occur when the stakeholder is a systems architect or if there is a corporate mandate for a specific design style.

Intended Security Practices as Security Requirements

One source for gathering security requirements is through examining the **intended security practices** for the system under development. This will not be the sole source for security requirements, but it does provide a good structure for a beginning.

A deliberate look at intended practices begins with an in-depth evaluation of current practices. During this evaluation, the security practices are reviewed and those that are effective and will be maintained are collected. Other practices that are not relevant to the new system may be put aside, while those that have been shown to be less than totally effective are noted for improvements.

The next step is to evaluate the **separation of duties** and **privilege levels** in the system. Separation of duties is a standard security practice to ensure that no single user is able to fully compromise and control the system. Separation of duties implements a system of checks and balances. Likewise, separating privilege levels is a means of implementing a least privilege and ensuring that any user on the system is only able to perform those tasks needed for their job. Implementing it requires identifying what the different roles will be in the system and then setting up a mechanism for enforcing the privilege structure. An important part is to also consider how to structure administrative roles and privileges. All too often, the default is to give full authority to all administrative users. The results can be disastrous.

A further step is identifying sources for user authentication and authorization. Not every system needs its own processes for user authentication and authorization. Most organizations already have authentication and authorization systems in place that act or are capable of acting as a single source within the organization. Leveraging pre-existing authentication systems alleviates the need for more complexity. Even when a new system needs to include authentication and authorization processes, using the roles and privileges established around the existing system allows for clear and unambiguous specification of authentication and authorization requirements and will avoid a situation where compatibility needs to be tacked on later if the organization later consolidates to a single system.

The next step in the evaluation of intended practices is to identify the logging requirements for the system. The logs are records of significant events that are recorded, along with metadata, to allow the identification, tracing, and reconstruction of those events. The logs will be used for evaluating potential security breaches, tracking potential misuse of the system, investigating issues upon discovery, and in reconstructing missing or mangled data. With this, a review of where “interesting” events occur in the system can indicate potential logging points and may provide clues of required logging levels. As a minimum, logs should cover user and user privilege changes (logins, logouts, privilege escalations, etc.), significant transactions (major changes, major data additions or deletions, etc.), financial transactions (all kinds), and key communications

(session initiations, large data transfers, remote accesses, etc.). As a general rule of thumb, it is always easier to begin with more logging and later reduce it than to begin with too little and later figure out how to best capture it. Allowing flexible logging management is a good approach, as is allowing logs to be stored remotely. Naturally, all logs need to be stored securely, with controlled access.

Finally, in considering intended security practices, remember to look at the best practices of other organizations as well as any regulatory, legal, or other compliance issues. While this provides a baseline of security requirements, others will show up throughout the design process, particularly as the vulnerability assessment and threat modeling are completed.

Outsourcing and Cloud Services as a Source of Requirements

If our development is going to be outsourced or hosted on cloud services, there is another area to examine for requirements. This would include investigating any special security requirements resulting from the arrangement. One source for these requirements would be to explicitly examine the security **responsibility boundaries**. Security responsibility boundaries are where responsibility for providing, monitoring, and managing security transitions from one organization or organizational component to another. The area around these boundaries can easily become a security gap as each side of the arrangement assumes the other takes responsibility for particular details. Even when this is the case though there may be a need for overlapping monitoring and management of security issues. Any security quality of service or service-level agreement (SLA) from the provider needs a thorough review to identify security requirements that may otherwise not be considered.

Another concern with outsourcing and cloud services is to what degree independent security auditing is needed and how it will be implemented. Some level may already be available in the service parameters or the SLA, but that may not be sufficient.

Requirements Engineering

Simply gathering requirements is not enough. As requirements are gathered, a significant amount of work will need to be done. This is **requirements engineering**. It involves categorizing, clarifying, deconflicting, and analyzing each requirement.

Categorizing requirements can be done in several different manners. They can be categorized as functional and nonfunctional requirements. They can be categorized by their area—user interface, security, data, etc. They can be categorized by source or by any other method that will help organize and analyze the requirements. Requirements can be categorized and cross-categorized in many ways.

The next task is to clarify the requirements. To accomplish that, we review the requirements and make sure we understand exactly what the requirement is saying. If there is any ambiguity or possibility of different interpretations, then the requirement source is consulted to help with clarifications. Sometimes subject-matter experts need to be consulted to gain a full understanding of what a requirement means.

Requirements are also going to need to be deconflicted. Because requirements come from multiple different sources, it is inevitable that there will be conflicting requirements. The challenge is which of the conflicting requirements need to be included. On rare occasions it is easy to sort out because one requirement comes from a mandatory source, for example a legal requirement, and another is more of a request, such as “all rounded amounts go into my bank account.” But this is rare. More often we are going to either look for an acceptable manner to support both requirements or go back to the sources, explain the conflicts, and see if we find an agreement among them as to what the actual requirement should be. An example of finding an acceptable compromise would be if one stakeholder said the primary interface color should be red and another required green. Both could be met by allowing the color to be set as a configuration parameter. On the other hand, if one of the stakeholders was marketing and wanted the color to be blue because it aligned with corporate marketing strategies, and another a stakeholder wanted the color to be orange, because it is their favorite color, the second would be overruled.

If the sources of the requirements cannot come to an agreement, then it may be necessary to go to a higher authority to determine the priority for the

requirements. This may be the sponsor or some other authority. Ideally, conflicts in requirements wouldn't be resolved this way as it can cause issues among groups and stakeholders, but sometimes there is no choice.

Requirements also need to be assessed to determine whether the fall within the scope of the effort. As we previously discussed, those which do not are relegated to another system, a later development, or the parking lot.

Additional analysis will look at why each requirement is included. If a requirement doesn't provide a specific advantage or meet a specific need, if it should actually be included needs to be given significant consideration. Meeting requirements costs in money, time, people, and other resources. Those expenditures need to be justified. Sometimes, requirements that cannot be fully justified can be reduced from a "required" to a "desired" status, where they are only implemented if it can be done without an additional expenditure in resources.

Requirements need to be assessed for constraints and dependencies. Constraints are limitations that will be placed on the requirement or that limit the ability to comply with a requirement. Dependencies are other requirements that must be met in order to be able to meet a requirement. The discovery of dependencies that have not already been captured as requirements end up as another source.

Assessing Requirements for Security

Security requirement assessments are done on a requirement-by-requirement basis. Usually, this assessment is performed by someone who has strong competency in security. The assessor should also have a good understanding of the development and deployment processes as well as how the software will be used.

Each requirement is examined from a risk management perspective. The risk assessment looks at how the requirement could fail, either during development or when deployed, and what happens if it fails. Failures examined could include entire system failures resulting from incorrect implementation or application of the requirement or failures of the requirement to be implemented. In the risk assessment, particular attention needs to be paid to exceptions to normal cases and how a feature from a requirement could be misused or abused.

If a requirement is identified as having a potential for failure, then the results of that failure should be assessed for their impact on the system. In some failure cases, the impact may be limited through graceful degradation, limiting the propagation or the spread of the impact, or by having a fail-secure mode.

The requirements risk assessment is also a great opportunity to identify potentially sensitive information and begin the process of ensuring it is properly protected.

The results of the risk requirement assessment can be that a requirement is reconsidered for inclusion, that a requirement is modified to reduce security impacts, or that additional requirements are generated to mitigate potential impacts of a failure. The results are also used to inform the vulnerability analysis of the system.

Requirements Documentation and Tracing

An important consideration for requirements is the methods we use for documenting them. Although better than nothing, simply tossing the requirements serially into a simple document does not provide much advantage. Keep in mind that we are going to want to be able to sort, categorize, and otherwise manipulate the requirements. That means using a tool that gives us the flexibility to do so. The tool is known as the requirements repository.

Something as straight forward as using a spreadsheet as the requirements repository can offer a significant advantage over a text document for recording and tracking requirements. We can add columns for marking the different categories, tags, and metadata for the requirements (provider, priority, etc.). However, maintaining requirements in a relational database allows additional information, such as links to related or prerequisite requirements.

Eventually, we may find justification to expend the money, time, and effort to obtain, install, and learn to use a requirements management software package. Well done, these packages can be used to assist in the requirements engineering process. This is particularly useful for large development projects where managing and tracking requirements by hand becomes increasingly challenging.

Traceability

One of the most important areas requirements management systems can assist with is **traceability**. **Figure 7.3** demonstrates the end-to-end traceability of a requirement from initial capture through design and development, until the requirement is verified, validated, and deployed.

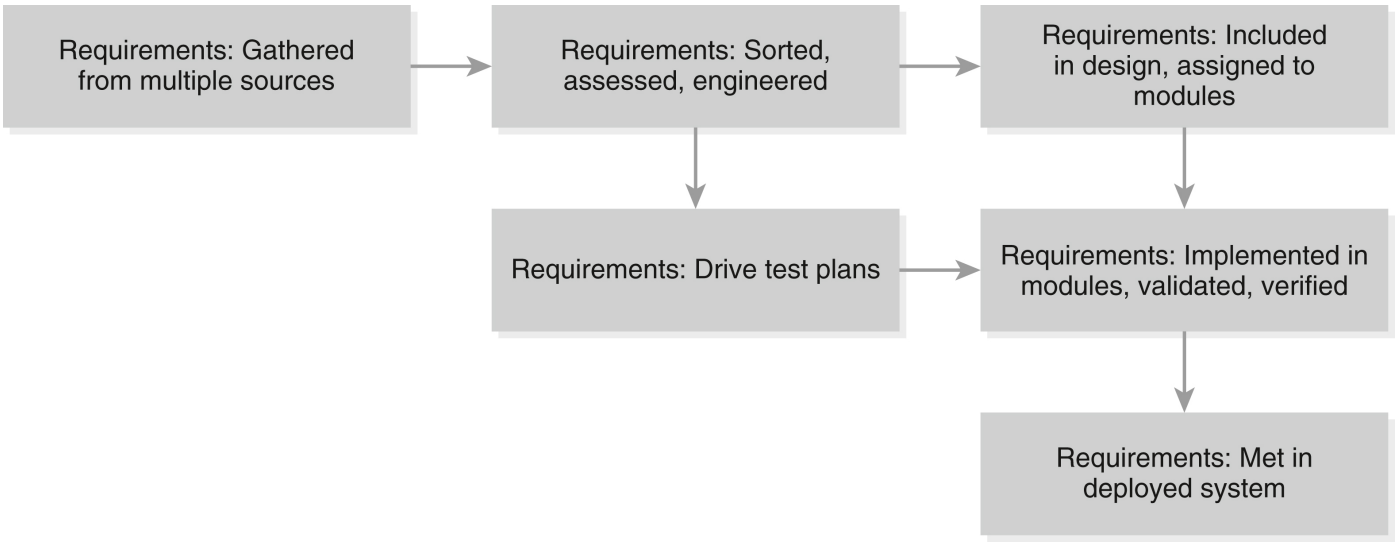


FIGURE 7.3 Requirements Traceability

Description

The first step of traceability is to be able to know where the requirement originated. This plays a part in our evaluation of requirements where the sources of some requirements may be more authoritative than others, for example, legal restrictions implemented as requirements or sponsor set requirements. Tracing the source is also useful in the clarification and deconfliction of requirements. At the end of the development, we can go back to the requirement originator and ask if our implementation is what they originally had in mind when they submitted the requirement. Of course, hopefully, that is not the first time we ask them that question, as it would have been easier to account for deviations earlier.

In the requirements engineering phase, we also want to keep track of all the requirements that are altered, merged with another, dropped, or moved to another effort. Maintaining reasons for these actions allows us to defend those actions later if questioned and ensures that a requirement is not met because it was simply forgotten.

Likewise, we need to be able to track the modules or development steps to which each requirement has been assigned. This information is going to be used by the designers, developers, and other team members who need to know what needs to be done for each component or step of the development. It is also going to be used in creating the vulnerability assessment and in test planning. Having the test plan developed based on what is required, rather than simply what was designed and built, provides a much higher level of confidence that the system becomes what we want it to be.

Formatting Requirements for Sharing

Another important consideration is how to format requirements so they can be shared. While for many, it will be sufficient to access and read the requirements through the system in which they are stored, for others a well-structured document is important. Having a well-formatted document provides clarity and focus, which is not always as well provided in a general system where a reader may need to search out and self-organize those requirements of interest. The requirements document may also be used as a part of the requirements approval process.

The Software/System Requirements Specification (SRS)

The **Software/Systems Requirements Specification (SRS)** document is a written format used to document the requirements in a structured manner and obtain buy-in and approvals for the requirements. Many projects, especially those that are larger, may not have a single SRS. Instead, one may be created at a systems level, and then multiple documents created for components of the system. These lower layer SRS documents may be included with requests for proposal (RFPs), which we address later, for systems that have an off-the-shelf or outsourced component. As with everything else in the secure software development life cycle, the creation of the SRS is often a cyclical process.

The exact outline or specifications for the SRS will vary greatly from organization to organization. No single agreed format is in common use. More important is that a consistent format is used within each organization, that the selected format meets the needs of the organization, and that people within the organization apply the format in a consistent manner. Using the same format, however, does not mean that every SRS in an organization will be exactly the same, especially in listing the detailed requirements, because every SRS also needs to address the requirements specific to the system it is used for.

An SRS generally begins with a statement of intent. This is usually the same intent that was begun in the project charter or equivalent document in the initiation phase. That intent will have been refined and expanded in the creation of the project plan. The same intent will now be focused to the software system effort of the project and then enhanced to give a good vision of what the requirements expect to accomplish.

The next task of the SRS is to establish the context in which the development will be taking place. As with a project plan, a scope is often included. This scope provides both information for those who do not have the scope from the

project plan and an opportunity for defining a more limited scope if the SRS is specific to a component of the system.

In addition to the scope, the context will describe the general environment in which the system will operate. The context can include descriptions of related systems and how they function, the overall system, if this is a component, and other information that will make the developer aware of potential issues.

As a part of setting the context, the SRS may include user stories, descriptive use cases, or other descriptions of the system in action. While these are not referenced as detailed requirements, they help the developers to understand how the actual requirements interact to produce the desired system. These help to illustrate how the intent is accomplished and put context around some of the requirements that may otherwise be obscure or unclear.

The context may also include a list of references used in the requirements or provide additional context (see **Box 7.2**). This could include items, such as a standard that must be adhered to, a drawing, a description of a similar system, or other sources of information about the application domain. If the number of references is a significant, it can be best to provide key references here and then include a pointer to all others in an appendix.

BOX 7.2 Reference Example

The following excerpt from an RFP, discussing requirements for user help on a system, provides an example of a labeling scheme for requirements:

References:

For Floating Point see - “IEEE 754-2019 - IEEE Standard for Floating-Point Arithmetic”. *standards.ieee.org*.

…

Specific Requirements

…

(R 1.17) All floating-point operations must follow the IEEE 754 Standard for Floating-Point Arithmetic

If the system is designed for specific types of users, it is sometimes useful to give a description of the expected users and their characteristics in this section. That helps the designers and developers to understand the context in which they are working more fully.

With the context established, it is then time to start into the requirements. This begins with general requirements that will be maintained throughout the development and need to be included in every component. Such requirements would include specifying a format for function calls, general adherence to specific standards, and expectations for coding standards (or a reference to an organizational best practice document).

The general requirement section may also include an overview of the requirements. This is not to be used as specific requirements but to assist someone reading the document understand the structure of the requirements.

General constraints are also detailed in this section. This can include items such as limitations on memory, processing, or storage capacity. It could be hardware or systems limitations. Or it could be any other general constraints that will impact the process.

Following the general requirements are the specific requirements. These are usually presented in different groupings appropriate to the development effort. They may be broken apart by functional and nonfunctional requirements, which are then grouped internally. Another alternative is to group by interfaces—user and automated–internal functions, data handling, and other layers. The needs of the development and organizational convention will dictate the specifics. **Figure 7.4** contains some examples of the various types of documents and artifacts that may be created early on in the life cycle.

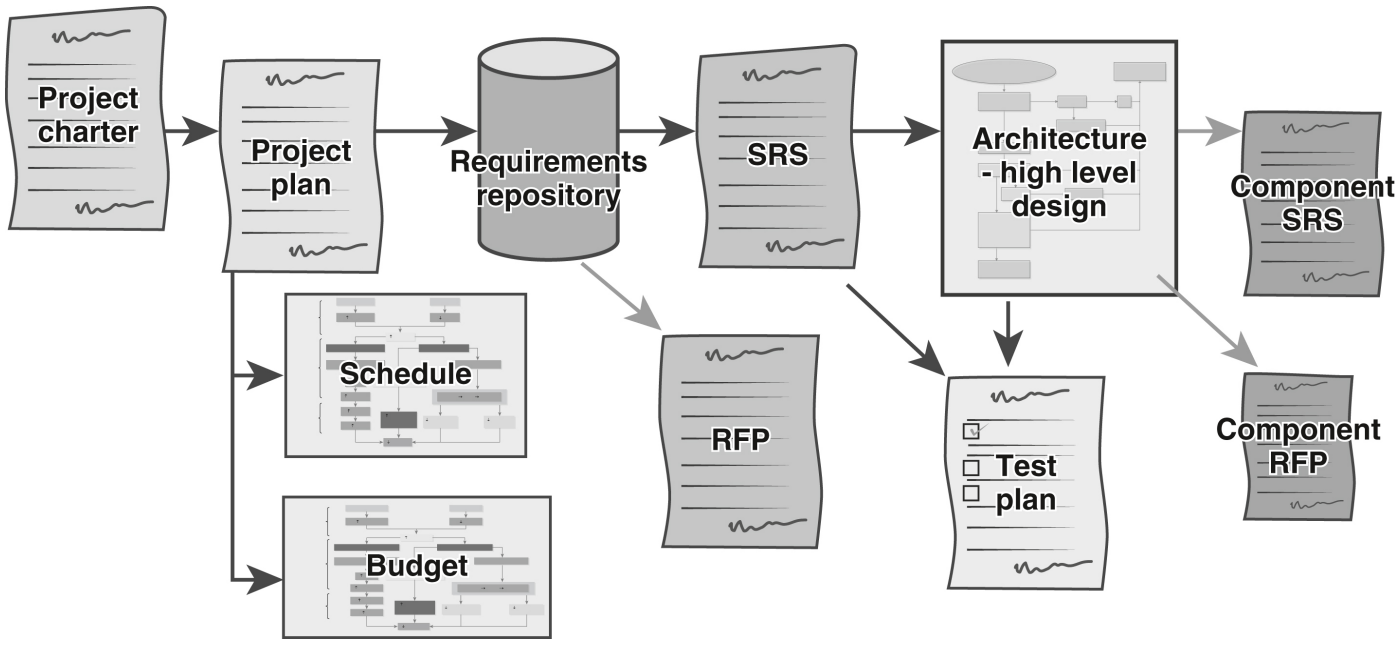


FIGURE 7.4 Flow of Documents in the Early Life Cycle

Description

This organizational grouping, and often the level of detail of the requirements, may change depending on the development cycle the system is in, whether this SRS is for a system or component or when different development paradigms are used. For example, in some Agile development methodologies, the initial SRS may be fairly high level with few specific requirements and many aspirational not-quite requirements. Further iterations through the cycle may add more specific requirements and change the content or character of some of the aspirational requirements.

Naturally, the requirements of the SRS should have a good identification scheme. This often can come out of the identification scheme used in the requirements repository database. Other schemes may be used, such as the one described next to be used with RFPs.

The completion of the SRS is frequently used as a milestone. The SRS is reviewed, revised, and approved by any number of designated reviewers and approvers. When used in this manner, once approved, the initial SRS is then frozen as a deliverable artifact of the development effort. Any subsequent changes that are made to the substance of the approved document are subject to a formal change process. The changes are processed and approved as needed by protocol, and changes and reapprovals are noted on the document.

As the secure software systems development life cycle proceeds, additional SRS documents may be created at a component level. These need to stay consistent with the approved or revised and reapproved SRS but will generally include more details or different groupings of the detailed requirements around the components of the created architecture or high-level design.

Request for Proposals

A good example of needing a different format for requirements is in a **request for proposal (RFP)**. This is a document that is generated, based on requirements, when a vendor or multiple vendors are going to asked to submit proposals for pre- or custom-build system to meet the needs for a system or a component of a system. The RFP is going to begin with an introductory section that paints an overall picture of what is being asked for and how that is going to fit into the big picture. This is very similar to the SRS intent and may be a modified version.

An RFP then has sections that group the requirements in a structured manner. Each of these sections will have some sort of requirements listing, often with explanatory material that aids the reader in understanding the meaning of the requirements. Each requirement needs to be labeled specifically as a requirement. Ideally, the label for each requirement is unique. This allows two things to happen. First, requirements will not be overlooked as simply being a part of the narrative. Second, the structure and identification of the requirements can allow easy assessment of different proposals as they are returned both in terms of how well they meet the requirements and how the proposals compare to each other.

BOX 7.3 Example Labeling Scheme

"**(R 1.2)** The system will include the ability to right click for help for each button. **(R 1.3)** Users should be notified that help is available in the main help section for the device. **(O 1.1)** Alternatively, the notification could be periodically included on a scrolling information bar. **(D 1.1)** The help provided should not only be sensitive to where help was requested, but also to who requested the help. **(F 1.1)** The system in future versions will include context sensitive help for other screen objects."
This labeling scheme differentiates the types of requirements:

- R—Required—Those items which are definite requirements.
- D—Desired—Items that are good to have but not essential.
- O—Optional—An alternative capability that could replace a requirement or set of requirements.
- F—Future—Not a present requirement but will be. The current implementation should not preclude future support for this requirement even if it isn't supported now.
- The numbering with the scheme supports the tracking of requirements, and by providing a section number as a part of the scheme, both allows the identification of a requirement with a section and prevents the need for extensive renumbering if requirements are added to a section.

The thought and care put into the format and presentation of the requirements shows to a large degree the thought and care that will go into the

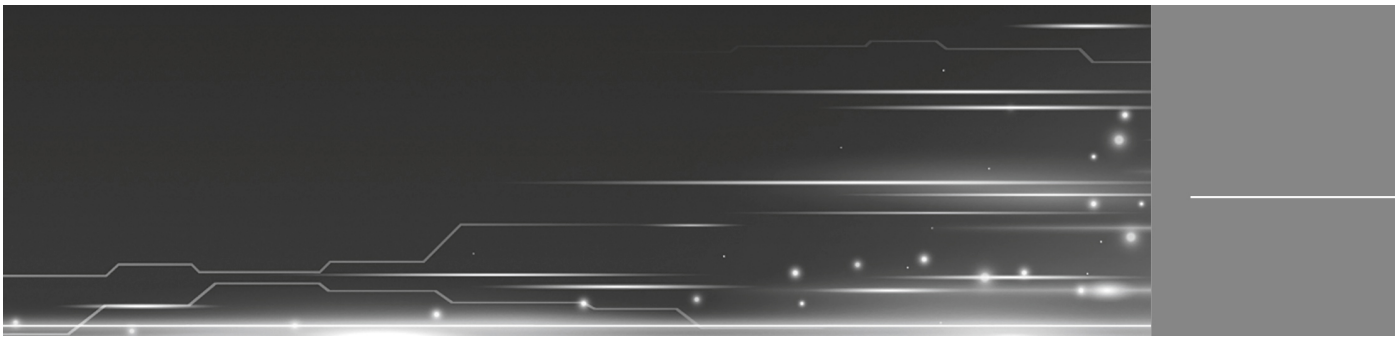
rest of a secure software development effort.

SUMMARY

Knowing and understanding the requirements for a software system seems to be straightforward. However, it is more often a significant challenge for many reasons: users don't know or understand what their requirements are, developers want to just start developing, and the world doesn't stand still while a system is in development. If this weren't enough of a problem, at the end of the process each requirement needs to be verified and validated. Because of these factors, developers need to put significant effort into collecting, engineering, documenting, and tracking requirements.

KEY TERMS

functional requirements
functional security requirements
influence
in-scope
intended security practices
interest
key functions
nonfunctional requirements
out-of-scope
privilege levels
request for proposal (RFP)
requirements
requirements classification
requirements engineering
requirements solicitation
responsibility boundaries
scope boundaries
secure functional requirement
security requirements
separation of duties
Software/System Requirements Specification (SRS)
stakeholders
system boundaries
traceability



© d1sk/Shutterstock

CHAPTER 8
Compliance

CHAPTER OBJECTIVES

- Explain why compliance is needed.
- Review sources of compliance requirements.
- Compare and contrast standards, frameworks, and laws and regulations.
- Examine the roll of monitoring in assuring continuing compliance.
- Summarize needs for auditing of systems.

Introduction

In every field of human endeavor, processes and rules need to be followed to make sure the technology is used as designed and in a responsible, safe manner. Usage of software systems and services is the same as any other human technology. There are best practices for the use of the technology as well as, very often, laws and regulations that influence or, in some cases, dictate, how technology is to be created and used. This is not just for electronic systems, but any type of technology that gets created.

Need for Compliance

Compliance is the act of ensuring that our systems, processes, and deliverables meet specified requirements. Compliance of software systems and services is the same as compliance in any other industry or field, such as compliance to the rules of the road when driving. In the field of IT, compliance ranges from how to install the products (often dictated by the product manufacturers) to ensuring relevant regulations, laws, and standards are met by the software systems and services. Often not complying with expectations, regulations, standards, or laws has negative consequences.

Any system is going to potentially be affected by many different compliance sources. Each potential source is going to need to be evaluated to determine if it applies to the specific system and often to determine which of multiple competing sources are most important or have precedence in the event of a conflict between sources. When we are going to evaluate or compare different compliance sources, we need to consider several factors. These factors include the type of the requirement, the governing body, whether adherence is mandatory or voluntary, how compliance is measured and enforced, and how compliance is recognized.

BOX 8.1 Evaluating Compliance Sources

In evaluating compliance sources, several factors should be considered:

Type of Requirement: Laws or regulations, standards, frameworks

Governing body: Country, state, or local jurisdiction; official international committee, or organization; corporate established; industry organization; open standards; tradition

Adherence Type: Mandatory, voluntary, blended

Compliance Measurement: Internal or external audits or reviews, self-certification, exceptions based, public disclosures

Consequences of Noncompliance: Fines, imprisonment, sanctions, exclusion, loss of endorsement, embarrassment, fees

Recognition: Special symbols and marks, certifications, listings, basic expectations, ceremonies, and awards

Laws and regulations provide directions from a government on specific topics. Laws are a legislative function of a government and tend to be more general or less technical in nature. Often, they provide the intent. Laws are created by a legislative body, such as the U.S. Congress or a state legislature. Laws may also be developed out of treaties among countries. Regulations are a bureaucratic function. They are created by administrators and technocrats under the directive of a law to provide details and specifics or to add structure to the law. If the two are in conflict, the law is typically considered definitive over a regulation.

Exhaustive review of all laws and regulations for any type of business is not possible in one written resource, both because each industry has specific laws and requirements and the amount of space necessary is not possible in one resource. As well, an exhaustive study of the regulations and laws for all business types is better left to books and classes focusing on the legal aspects of IT compliance. The organization will need to prove (through monitoring results and regular audits) that they meet the laws and regulations applicable to its industry.

Certification of compliance is recorded and tracked with the applicable government agencies. The company receives approval from the agency granting certification and is able to claim compliance. A certified company is also able, as approved, to display the designed certification symbol indicating they have met the certification requirements.

Standards provide specific approaches to implementing or delivering devices, systems, and services. Depending on the industry, standards may be voluntary or mandatory. While voluntary standards may not have the same legal and regulatory consequences, they are necessary for the effective, efficient, and seamless operation of systems and services that have become critical to the running of modern businesses. It is difficult to think of a business today that does not use some sort of software or electronic service to keep it running. Many standards have arisen over time as it was realized that certain things need to be done on a regular basis and certain things that should *not* be done. Because standards are not always written into either laws or formal regulations, fines or jail time are not potential consequences of noncompliance. The consequences that do exist are that when systems and services fail, it negatively impacts business.

To assist with meeting requirements based on standards, regulations, or laws, various structured guidance frameworks have arisen over the years since software development began. **Frameworks** provide broad guidance, which may be implemented in a variety of ways, for building, managing, or maintaining a system or a set of processes. Frameworks exist in many developed fields of human endeavor. Frameworks provide benefits as they provide a defined and structured approach, often based on industry experience (best practices) to creating, managing, and keeping systems and services working effectively and securely. Multiple frameworks exist. Frameworks can be used by themselves, or several frameworks can be combined into a hybrid approach. Each company will define/discover which framework or hybrid of different frameworks is best for it. This decision is based on the type of business, size of business, how much technology use there is in the business, and the dictates of business relevant laws and regulations. Each framework defines compliance requirements. Often compliance is broken up into multiple levels. Accreditation can be claimed when a business is compliant, along with the right to display the framework specific symbolic indicators proving the business is compliant with the specific framework. Some frameworks provide listings of company compliance by level of compliance. These types of listings of compliance demonstrates to business customers and regulatory agencies that the business has met a specific standard of compliance.

Knowing and using both frameworks and standards allows personnel to keep critical systems and services running as expected and kept secure from intentional or unintentional misuse.

Every compliance source has a **governing body**. The governing body is the organization or entity that is responsible for determining the content and requirements. For laws and regulations, this is a national, regional, or local government. Or it may be an international body or treaty subscribed to by different nations, for example, the Law of the Sea. While standards and frameworks may be managed by governments, this is often not the case. Some are governed by official international bodies, such as the International Organization for Standardization (ISO), which has official representatives from different countries as well as approved volunteer members. Another example is the International Telecommunication Union (ITU), which is a United Nations agency that sets telecommunications standards.

Some standards bodies are sponsored by industry and trade groups. These bodies often require participant companies monetarily contribute to the organization. An example is the National Marine Electronics Association (NMEA), an organization that despite its name is an international organization

that sets standards for electronics systems used on boats, ships, and other marine systems.

Many governing bodies are less formal and are often **open-standards** bodies. Open standards are standards and frameworks to which anyone with an interest in the subject area is free to contribute. Many of these standards have attracted significant participation because of their openness and have become foundational in many areas. One of the best examples is the Internet Engineering Task Force (IETF). Everyone is free to participate and present their ideas in open forums. The standards are set through consensus and only after have multiple demonstrations of successful implementations.

In some cases, standards are set by a dominant or early player in the industry. The initial personal computer (PC) standards were a good example. IBM introduced its version of the PC and released its standards. Because of its dominate position in the world of computing at the time, this quickly became the standard from which an entire industry emerged.

Finally, some standards are standards just because that is the way it has always been done and everybody expects it to be done the same way. These standards are firmly rooted in tradition, sometimes to the point where eventually they are incorporated into laws and regulations.

The type of **adherence** required will vary from source to source. Some compliance sources require strict adherence to all elements of the requirements. In other sources, the implementor is free to choose which elements are important for their needs. Often there is a blend of items requiring strict adherence and those that are optional.

Regardless of the industry relevant laws and regulations, a part of all standards and frameworks is **compliance measurement** to confirm systems, services, and personnel meet the requirements given through defined and documented monitoring and auditing processes and procedures. **Monitoring** is an ongoing set of activities collecting information and checking compliance. It is intended to be proactive. **Auditing** is a reactive process that identifies where companies may be failing to implement the laws, regulations, standards, or identified framework. The monitoring or auditing of compliance may cover many areas, both technical and nontechnical:

- Access to systems and/or services
- Access to data
- Permitted activities on a system or service
- Physical security required for systems and/or services

- Cybersecurity requirements for systems
- Adherence to business and technology relevant regulations and laws

The worst consequence of noncompliance would be shutting down the business, possibly with fines and even potential jail time for company executives. The least consequence would be to replan and reimplement compliance efforts, processes, and procedures, which is very expensive because of delays in systems and services needed to run the business.

Those who promulgate standards and frameworks do not have the ability to access the same coercive mechanisms that governments do; however, that doesn't always mean they are without teeth. Standards often have a method for certifying compliance. Failing to meet the requirements can mean that an organization doesn't receive certification. This can result in lost sales, even if the product works well. Of course, if the system doesn't meet the standard, then the product is not likely to work well anyway.

In some cases, standards bodies are membership based, and those who don't comply with the standard could be subject to sanctions, such as exclusion from participation or a fine. Even when there isn't formal membership, participants who are noncompliant may find that they are shunned by the community.

When systems and services do not comply with the required laws for the business, the consequences can include warnings, small to large fines, and even possible jail time! Therefore, it is critical for personnel to be aware of the requirements for the systems and services they create and support to meet business required regulations and laws. The consequences of noncompliance are specified within the applicable laws and regulations (some of which are discussed next).

The ability to recognize compliance with laws and regulations, standards, and frameworks is highly important. Recognition is not only about internal acknowledgement of compliance, but also about letting the rest of the world know that a product or organization is compliant. The recognition is valuable for marketing and organizational reputation. Often it plays a significant factor in potential customers' decisions. One of the many different means to provide recognition are special marks, such as the Windows™ Compatible sticker found on many PCs; certificates that are displayed by an organization or accompany a product; or inclusion in a list of compliance organizations. Often the designation of compliance is accompanied by a ceremony or an award.

Laws and Regulations Related to Compliance

Many laws and regulations are related to compliance and differ from country to country and region to region. However, software generally solves the same problems regardless of country or region. Businesses rely on vendors to create systems that not only meet their needs but are also compliant in the country or region in which the business operates. Many products are deployed from relatively few companies (such as Microsoft or Oracle). Widespread use of products from a few primary vendors means that businesses that use those products must rely on the vendors to meet laws and regulations for all countries in which their products are sold. The same is true of in-house developed software—it too must meet the laws and regulations of each country in which the software will be used. The best approach to meeting a wide variety of compliance requirements is to select the most rigorous requirements from all the requirements across all countries and develop software to meet those requirements. In this way software development teams can be assured that they are meeting the complex set of compliance requirements worldwide. One such example is the privacy requirements of the European Union's **General Data Protection Regulation (GDPR)**. Currently GDPR is the most comprehensive data privacy and software security law. Many worldwide software vendors have modified their software to meet the GDPR requirements. In doing so they are then able to provide higher than expected data privacy and software security to users in non-EU countries. This generalized approach to GDPR compliance is why we see the same pop-ups asking which cookies we allow on our browsers even when we are in a place where GDPR does not legally apply.

Technology-related compliance regulations and laws are expressed in the expected outcome of the products and services. These regulations and laws do not state which product to use, how to configure a product, or how to comply with the regulations and laws explicitly. It is up to software development teams and the teams deploying and supporting the software products to define policies and procedures to meet the regulations and legal requirements. Frameworks and models have been developed by a variety of organizations and institutions to assist with meeting compliance requirements.

Implications of Data Privacy Regulations and Laws

All laws and regulations related to data are intended to encourage trust in data systems (both electronic and nonelectronic). Without such trust, systems will not be accepted for use in business. The basic concepts any country's data laws and regulations are intended to meet have been discussed previously in this book and are summarized here:

- Confidentiality: trust that only those who have a right to the data will have access
- Integrity: trust that the data will always reflect the truth and that data cannot be changed except by people with appropriate authorization
- Availability: trust that the data will be available as and when desired
- Nonrepudiation: trust that actions performed in any system were performed by those who are authorized (and therefore changes to data performed by the authorized person cannot be denied)

As noted, data privacy laws, regulations, or directives of a country can be different from other countries. This creates a complexity for global businesses as they must meet data privacy requirements of each country where the business has a presence.

U.S. Regulations and Laws

Many U.S. regulations and laws are related to data privacy. Personal data in the United States is considered to be the property of whoever collects the data. The United States does not, at the point of writing, have nationally comprehensive laws regarding data privacy, but rather, a few federal laws as well as some specific state laws are relevant. Some examples are discussed next:

- Sarbanes-Oxley (SOX) Act
- Electronic Communications Privacy Act (ECPA)
- Computer Fraud and Abuse Act (CFAA)
- Health Insurance Portability and Accountability Act (HIPAA)
- Health Information Technology for Economic and Clinical Health (HITECH)
- Fair and Accurate Credit Transactions Act (FACTA)

- Payment Card Industry Data Security Standard (PCI DSS)

The **Sarbanes-Oxley Act (SOX)** was enacted in 2002 in response to corporate accounting issues and, of more concern, financial fraud. It was felt that the government needed to hold the people in power in corporations responsible for ensuring that accounting processes and financial statements could be trusted. Investor and client trust had been eroded due to a number of organizations presenting inaccurate financial statements and using questionable accounting processes.

Section 404 of the SOX Act most directly applies to cyber products and services. This section of the SOX Act requires a regular review of internal controls on everything involved with financial reporting. This review includes business processes and IT system processes and security. Support personnel must ensure they have robust processes and procedures to ensure the financial hardware and software under their control meets the highest levels of integrity and security. This is done through hardware and software configurations as well as effective processes and procedures.

The consequences of being found in breach of SOX range from a company being de-listed on all public stock exchanges as well as directors and officers (D&O) insurance policies being cancelled. Company executives found to have deliberately submitted incorrect SOX audit (financial) results face fines of up to $5 million and possibly 20 years in jail.

The **Electronic Communications Privacy Act (ECPA)** was enacted in 1986. It has been extended a number of times to now include any unapproved access to electronic systems (this includes telephones, computer systems, and data in transit) by anyone, including unauthorized government tapping of electronic systems. ECPA breaches can result in fines as high as $250,000 and up to 5 years in prison.

The **Computer Fraud and Abuse Act (CFAA)** was enacted in 1986. This law also has been amended a number of times since it was enacted. Essentially it states that the use of any electronic system in a way that is against the end user license agreements (EULA) that all IT products and services. Very often the EULAs that we agree to in order to use websites and software are quite long, and as has been noted, most users do not read them in detail (literally no one has the time to review all the EULAs related to the numerous sites we go to). For example, if a user exceeds the number of Google queries allowed, they may be in breach of the CFAA. This law does protect against such situations as someone breaking into another person's

computer or accessing any system (database or website) they are not authorized to access.

Violating CFAA can lead to fines of up to $1 million and up to 30 years in jail (as has recently been seen with the case of Aaron Swartz). While the law is intended to protect data and systems, it has been accused of being interpreted by the courts in an overly broad manner, leading to prosecution in situations that are of minimal impact and resulting in legal penalties that have been seen to be in excess of the situation.

The **Health Insurance Portability and Accountability Act (HIPAA)** was enacted in 1996. Among the many items that HIPAA covers, it defines standards for electronic health records usage (including data storage or in transit). Patient **protected health information (PHI)** includes a wide range of people's meta-data from their **personally identifiable information (PII)** such as name or date of birth to past medical history. Basically, anything that could be used to identify a person is covered by HIPAA.

Anyone working with patient information or supporting systems that process or store patient information must know the definition of PHI. They are responsible for ensuring the data are protected from either intentional or unintentional disclosure whether the data are at rest (stored in a database or any type of storage) or when data are in transit. Data in transit means data are being accessed by a person or another system and so the data travels from where it is usually stored to a new destination. At the new destination may be another data store, or the data may be displayed for a user to either view the data or update it. Data security at rest or in transit is a key requirement of HIPAA. All people who may interact with HIPAA data are required to be trained and understand how to keep such data private. This requirement includes medical staff, electronic system developers and support personnel, and any other people that may have a need to have access to medical related data, systems, and services.

Consequences of a violation of any aspect of HIPAA can be from $100 to $50,000 per violation to a maximum of $1.5 million per year for repetitive violations. In addition to the financial repercussions, prison sentences of up to 5 years can be given to an organization's management.

The **Health Information Technology for Economic and Clinical Health (HITECH)** Act was enacted in 2009. The goal of the HITECH Act is to encourage increased use of electronic health records (EHR) in the United States. This law increased the breadth of HIPAA data privacy and system security protections. It had five central objectives:

- Coordination of care improvement
- Improve public health
- Increase the quality, safety, and efficacy of electronic PHI
- Encourage patient and family acceptance of electronic healthcare systems and records
- Make sure PHI data privacy is projected and has the highest security

Similar to penalties for HIPAA, violations can result in fines between $50,000 and $100,000 (or more) as well as imprisonment for 1 to 5 years.

The **Fair and Accurate Credit Transactions Act (FACTA)**, enacted in 2003, was an amendment to the original Fair Credit Reporting Act (FCRA). Its primary purpose was to define how consumer account information is managed (and used) with the goal of reducing the risk of identity theft. It also permits consumers to request a free consumer credit report once every 12 months from any of the three credit companies (one of which has already been breached in one of the biggest modern-day hacks—Experian). Some aspects of the act include allowing consumers to place a fraud alert on their files with each reporting agency for at least 90 days, requiring security measures for credit card information (such as limiting printing five credit card digits on a receipt and no printing the card expiration date on a receipt), and finally, a number of requirements to prevent identity theft related to financial institutions as well as creditors.

Companies in breach of FACTA can face fines between $100 and $1,000 for each person affected, and if identity theft results from the breach, the company could be held liable for any damages resulting from the breach. (At this time Experian is facing billions of dollars in civil suits for the loss of data control for more than 240 million people.)

While not passed into law at the federal level, the **Payment Card Industry Data Security Standard (PCI DSS)** standard has been passed into law in some states. The PCI DSS standards were originally proposed (an accepted) in 2004 by a consortium of the credit card companies. However, the initial standards go back to an increase in credit card fraud between 1988 and 1998. Visa and other credit card companies created the initial standards to try to combat credit card fraud.

Currently 12 requirements are included in the standard. These 12 requirements are very specific things that need to be done, such as installing and maintaining firewalls, changing the default passwords that come with IT

systems, protecting any cardholder data, encrypting data in transit, keeping anti-malware software up to date, limiting both remote and physical access to cardholder data, obfuscating the cardholder information, testing the security of systems as well as processes, and finally setting up and enforcing a robust information security policy. When creating software systems, the team needs to plan for and build in (code) the ability to meet the 12 requirements (at a minimum) to keep credit card information and credit card systems safe. In the past, there have been a number of successful attacks on payment card systems as well as backend server databases. One particular attack was against a credit card intermediary processing agency, Heartland (Jackson-Higgins 2018).

If a company is found to be noncompliant with PCI-DSS requirements (as noted by an external auditor or in the case of a reported breach), the consequences can range from $5,000 to $10,000 per month. The potential financial impact of a breach is likely to be much larger if consumers are affected because consumers can sue in civil court. Finally, the ability to accept credit cards can be revoked if a company is not PCI DSS compliant.

Canadian Regulations and Laws

Canada has one primary federal law to protect people's personal information, which is the **Personal Information Protection and Electronic Documents Act (PIPEDA)**. PIPEDA defines the collection, use, and disclosure of personal information by private companies. Provincial laws related to personal data privacy can only override PIPEDA if the provincial law is more comprehensive. PIPEDA has 10 principles, including the following:

- Limitations on collection of personal data, disclosure, use and retention
- Accuracy of data collected
- Specific data safeguard expectations
- Corporate (and assignment of specific corporate individuals for) accountability for data collection and retention
- Ensure data owner consent to data collection and limit data collected to only what is needed
- Provide information regarding personal information management policies upon request

- Allow people to discuss privacy concerns with specified individuals of a company
- Report breaches in a timely manner to affected individuals

The primary difference between PIPEDA and U.S. laws such as HIPAA is that PIPEDA covers *all* data related to an individual, whether collected by a private or public organization. Further, a person's data can be sold to other organizations under HIPAA; however, under PIPEDA, PHI data cannot be sold without specific consent. Data are considered to be the personal property of the individual and no one else.

Both fines and potentially prison sentences can be the result of breaching PIPEDA, depending on the severity of the breach.

European Union Regulations and Laws

The GDPR European Directive applies to all members of the European Union (EU). It is a set of data privacy goals that each member country of the EU is expected to meet through their own regulations or laws. Central to the GDPR is the concept that individuals are the sole owner of their data and as such have approval on what data are collected, the use of their personal data, and how long their data are kept. Included within the GDPR are the following concepts:

- Individuals own their data and have the right to see their data at any time.
- Individual data are to be anonymized (modified in such a way that who it belongs to cannot be determined; this protects individuals from being persecuted for any specific personal traits).
- Individual data are to be processed within the EU unless covered by agreements (such as the Safe Harbor agreement with the United States).
- Data breaches must be reported to both an overarching Supervisory Authority with 72 hours as well as to impacted individuals.
- Individuals have the right to request to be "forgotten" by any business, which means an individual can request to have specific personal information deleted from any data source.

Given the philosophical differences between the data privacy laws and the GDPR, an agreement of how European data will be managed by U.S. businesses was agreed to (called Privacy Shield).

The consequences of not meeting the GDPR requirements can be as high of 4 percent of a business' revenue (up to 20 million euros) and members of the business could face prison time.

China Regulations and Laws

Within the past few years China has added a few data privacy laws into its body of criminal law. Similar to other countries, the new data privacy law includes the following:

- A business may not illegally sell or publish a person's data.
- A business must have the user's consent to collect personal data (which is a change from the past).
- Any personal data collected must be legal, required, and proper.
- Businesses must explain why the data are being collected, and any personal data not required for the business purpose must not be collected.
- If a data breach occurs, the data owners and the relevant government agencies must be promptly advised,
- If data have been collected illegally or without authorization, affected individuals can request the data be deleted.
- An individual may ask for incorrect data to be corrected.
- Businesses must perform annual security risk assessments on their cyber infrastructures.

Compliance Models and Frameworks

Organizations can use a number of models or frameworks to help them meet electronic compliance expectations. The models can be modified, depending on the business requirements, type of industry, and the technology being implemented. Models and frameworks are intended as guidance; they do not specify detailed procedures. Interestingly, all frameworks seek to incorporate security as early as possible into system or service development. To do that, security gets discussed as early as initial requirements gathering. Frameworks also specify expectations regarding corporate culture, identifying who does what (roles and responsibilities) to make sure created electronic systems include security. Documentation is another important part of any framework because recorded knowledge helps assess further problems as well as establishes an ever-improving security baseline.

Using a framework has a number of advantages:

- It provides suggestions of what aspects of compliance to cover and, in particular, reminds organizations of items they might otherwise overlook.
- A business whose IT organization uses a standardized framework is recognized by internal and external auditors as being more likely to meet the necessary compliance for their industry because they comply to the framework requirements.
- Being certified in one or more of the frameworks indicates to investors, and regulatory agencies, that a business's IT organization is meeting the expected compliance to keep their systems and data secure.

Compliance models or frameworks include the following:

- Control Objectives for Information and related Technology (COBIT)
- International Organization for Standardization/International Electrotechnical Commission (ISO/IEC 27001:2005)
- IT Infrastructure Library (ITIL)
- Capability Maturity Model Integration (CMMI)

- Secure Software Development Framework (SSDF)

Control Objectives for Information and related Technology (COBIT)

Control Objectives for Information and related Technology (COBIT) is a high-level governance framework consisting of processes and control objectives to assist corporate management (such as board members, managers, and auditors) to define expectations for the internal IT organization. It was created by the ISACA organization. There are 34 IT processes and 318 control objectives defined in COBIT. ITIL and ISO/IEC 27001 define how to implement the high-level COBIT processes and control objectives.

COBIT helps businesses align business goals with IT goals, leading to greater efficiency through bilateral understanding as well as concrete deliverables such as metrics. **Figure 8.1** displays the six key concepts underlying COBIT version 5. Fundamental components include the following:

- A framework that sets out governance objectives and best practices for IT while aligning them with business needs
- High-level control objectives for IT to use when defining how to effectively control IT processes
- Management guidelines help to identify objectives, performance expectations, and how IT processes relate to each other
- Assists both IT and businesses to evaluate IT processes maturity and capabilities (and to identify and resolve gaps)

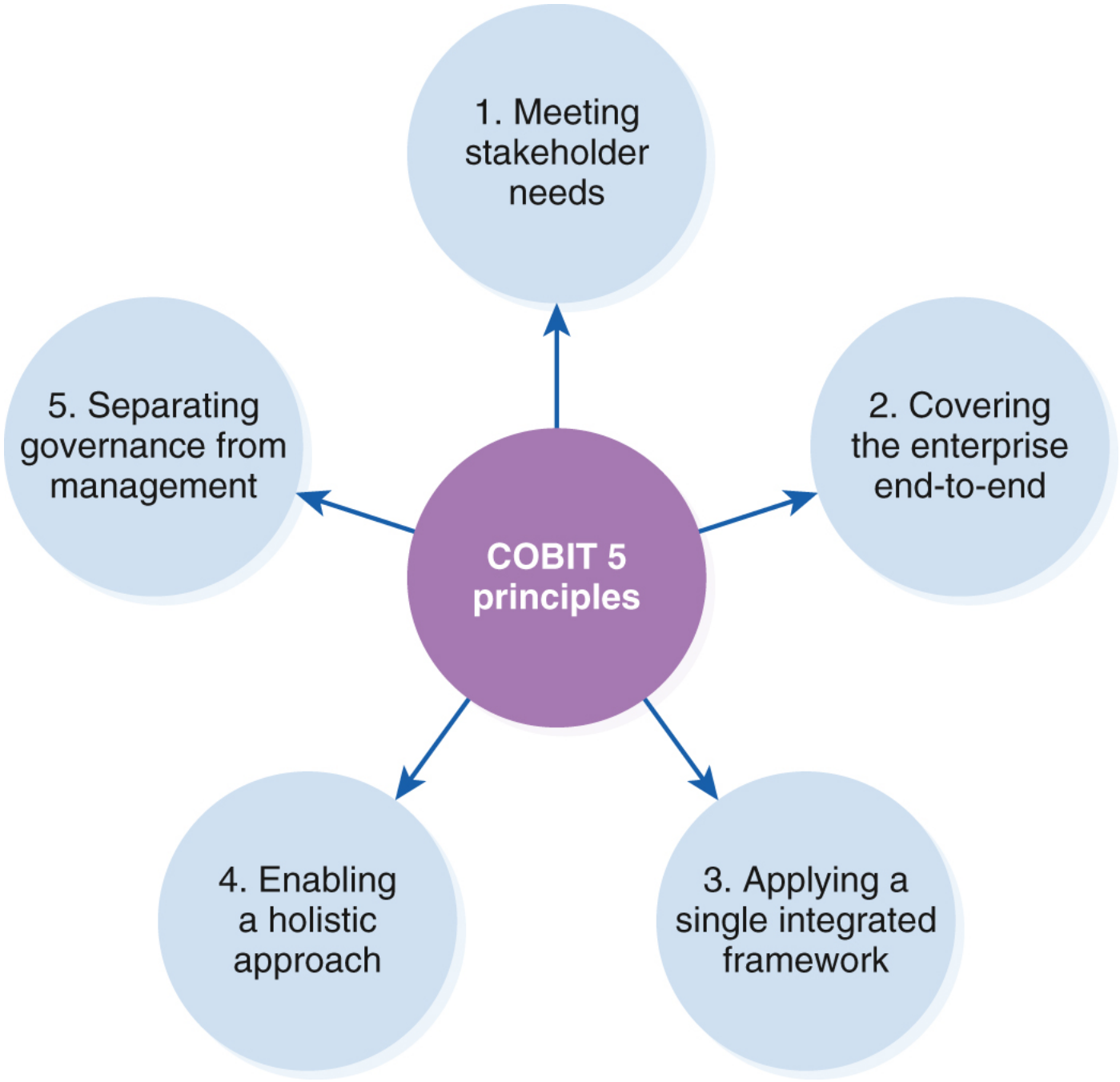


FIGURE 8.1 COBIT 5 Principles

Description

ISO/IEC 27001

ISO/IEC 27001 sets out both best practices for IT as well as information security. Becoming certified under ISO/IEC 27001 shows external agencies and parties (such as investors) that the company is meeting the suggested best practices to secure IT systems, products, and data. This framework covers a broad spectrum of IT related controls:

- Secure software development processes
- Separation of privileges for IT products and services
- Implement security testing for all IT products and services
- Establish security requirements when working with suppliers
- Define security incident responses
- Identify and implement IT usage guidelines for company employees and other users of company IT products and services
- Define IT products and services acquisition, development, and upkeep processes

Each of these controls has multiple subcontrols identified, resulting in 14 groups, and 35 control objectives resulting in a total of 114 controls.

ITIL

Information Technology Infrastructure Library (ITIL) also sets out IT best practices in the form of processes and procedures as well as specific tasks and checklists. These are not specific to a particular industry but are detailed enough that IT organizations can use the framework to deliver the highest security and best solutions for their products and services. There are a total of five core sections (service) including strategic approach to IT service, designing IT services, transition of IT services (from evaluation through testing, into deployment in production, and finally maintenance), the operation of IT services, and lastly, continuous improvement of IT services.

ITIL uses the idea of sections (services) rather than specific IT processes and procedures because each one includes a broad set of categories related to the section (service). Categories covered within each section (service) include design, availability (of the IT product or service), performance and capacity management, security requirements, supplier management, change and knowledge management, and IT product testing and validation. Finally, how IT serves the business clients, including how incidents are managed, and the communication with business users along with looking at how to continually improve are all aspects of IT.

Capability Maturity Model Integration (CMMI)

CMMI Institute oversees the administration of the **Capability Maturity Model Integration (CMMI)** model, developed at Carnegie Mellon University. ITIL and the CMM model are similar in its approach to optimize and secure IT products and services. The framework consists of 17 central process areas, including the alignment of IT to business needs, user training, incident management, project management, risk management, and supplier management.

Five levels of maturity are defined within the model with each successive level requiring increasingly consistent processes and security. Level 1 is assigned for organizations with minimal organizational consistency and processes (more like the organization doing what it feels is necessary at the moment rather than having a structure that is repeatable). In Level 2, the organization demonstrates effective management of processes, service delivery, work scope, and configuration management. Level 3 identifies a company's organization as being managed and having defined higher-level requirements that better align technology with the business, such as system availability, incident management and prevention, training for all users and support, and performing risk management for systems and services. Level 3 tends to be more proactive than reactive (as was Level 2). Level 4 includes the requirements and efforts of the prior three levels but expands expectations to include the quantitative evaluation of how effective the organization is both in performance from a technology perspective and also in meeting business needs. The final level, Level 5 adds performance management of the organization as well as in-depth analysis and resolution of opportunities for improvement of products and services. In this final level, the organization is a stable, mature organization that can now begin to focus on continuous improvement of its products and services.

CMMI (as seen in **Figure 8.2**) is a formal appraisal system used to assess the maturity of an IT organization's practices. There are three classes of appraisal (A, B, and C). Only a class A appraisal is formal and can result in an identified CMMI level for a company. Each increasing level of maturity has successfully higher expectations of control.

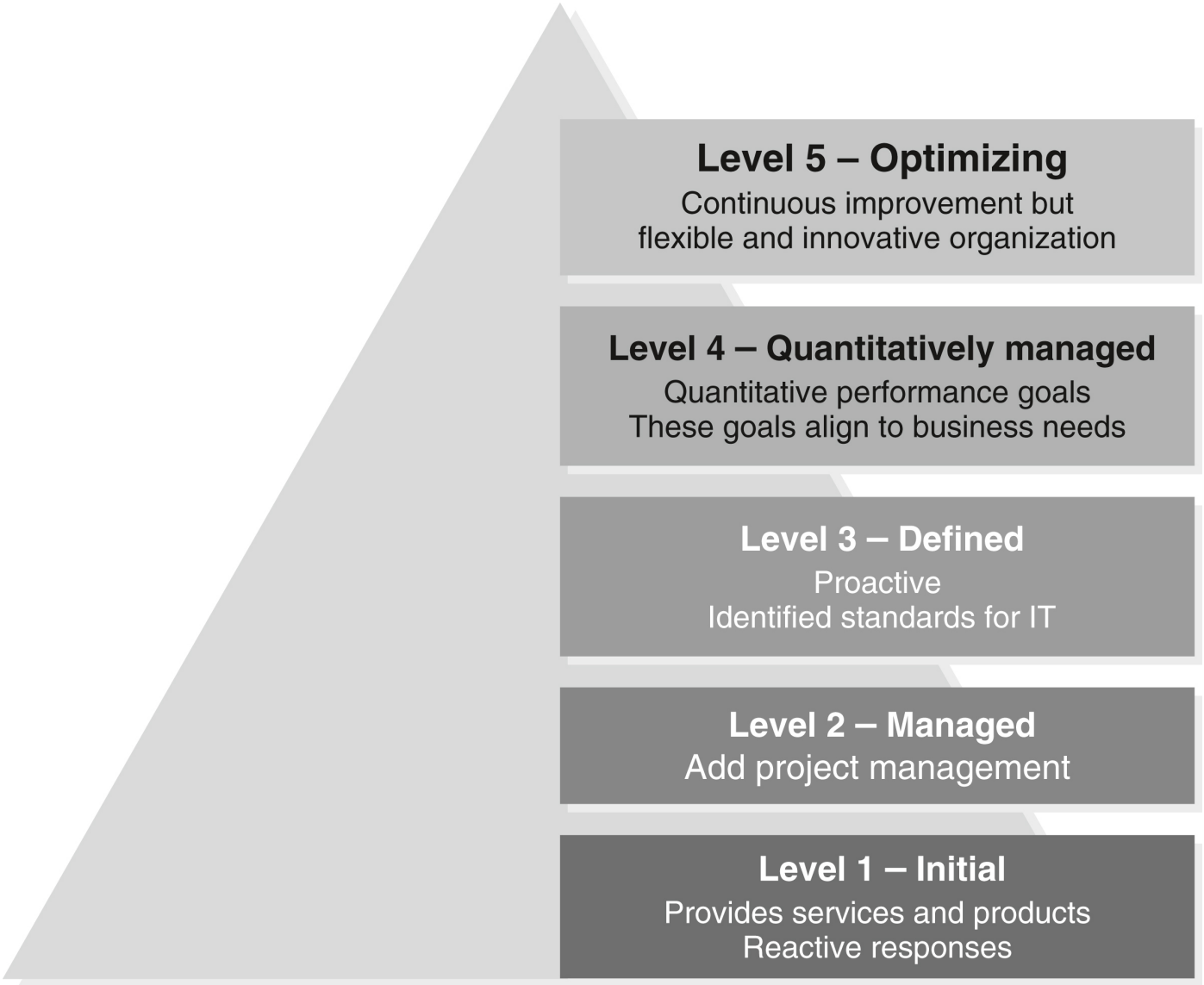


FIGURE 8.2 CMMI Framework Progression

Description

Secure Software Development Framework (SSDF) (NIST)

NIST has published its **Secure Software Development Framework (SSDF)** guidance. The framework suggests the same initiatives and processes as other secure software frameworks. Throughout the framework both education of all people involved and up to date documentation is emphasized. These initiatives are broken up into four main categories:

- Prepare the Organization (PO): To prepare an organization to embrace secure software development, first the system requirements have to be known, including security requirements (PO.1). Policies include how to

securely store software or how developer workstations will be protected against unwanted intrusion, how to meet legal or regulatory requirements, and regular vulnerability review of software. Of particular importance is defining who is responsible for what part of any software development cycle (PO.2). Have a set of tools to help automate verification, validation, and security checks (PO.3). Create policies to perform software security checks to make sure the software meets the security requirements (PO.4).

- Protect the Software (PS): While protecting the software may seem obvious, let's consider software providers (vendors) that provide software to a wide number of clients and may even push updates to clients without clients doing their own security evaluation of the updates. Define a set of policies to make sure software code is securely stored in a code repository and only those who need access have access (least privilege concept) and that code is signed by those that create it (PS.1 and PS.3). We want to ensure that any third-party software received is as the provider created it so the ability to securely compare what was received to what was provided is needed (PS.2).

BOX 8.2 Software Supplier Risks—SolarWinds

In 2020, SolarWinds, a software company, suffered a cyberattack. The attack injected a backdoor into its software product.

Because SolarWinds provided software updates and patches to tens of thousands of customers, they were infected by installing the (unknowingly) corrupted software patch from SolarWinds. This allowed attackers to access the systems running the SolarWinds products and infect them with further malware.

If your supplier does not secure its software, you are at risk.

Malware in a supplier's code is easily inserted into your environment: https://www.csoonline.com/article/3601508/solarwinds-supply-chain-attack-explained-why-organizations-were-not-prepared.html.

- Produce Well-Secured Software (PW): To produce secure software provide ongoing training, do vulnerability reviews (manual and automated) (PW.1), and perform code reviews and make updates (PW.2). Define security requirement policies that external software providers must meet before using their code and have a way to test the code (PW.3). Use existing code (external or internal) where it meets secure requirements and create a controlled central place for such code (PW.4). Require teams creating code to follow policies to create secure code through user input validation (UIV), careful error messaging, and security requirements specific to each coding language (PW.5). Configure code compilers to look for known vulnerabilities as code is being created and require code reviews for security issues (PW.6 & PW.7). Decide what functional and security testing needs doing, perform it, fix issues, and document (PW.8). Finally, figure out secure configuration settings, and document information about them including how to set them up (PW.9).
- Respond to Vulnerabilities (RV): Vulnerabilities will still occur in software, so we need a way to react when they do. First have a set of processes to get vulnerability reports and confirm the vulnerability (RV.1). Next determine its priority and a policy on fixing the vulnerability (RV.2). Do root cause analysis and document everything learned and use to update SSDF processes as well as any knowledge source (RV.3).

Monitoring

In any industry, there is a need to monitor or oversee the equipment being used as well as how well the business processes are functioning. Technology also requires a variety of monitoring efforts. The goals of monitoring are to make sure systems are meeting both technological and business expectations. Just because a system is available does not mean that the system performance is as expected. Monitoring both system health and system usage is required.

System health monitoring includes system availability (is it able to be accessed?), performance monitoring (are all operations of the system available and performing as expected?), and abnormalities that may indicate either a service is degrading or a failure is imminent. Monitoring of each layer of the modern complex IT environments is needed as well as the integration of all layers (sometimes called tiers of the technical stack). Only then can organizations be ensured its systems are providing the business with the highest level of functionality.

Much of the system monitoring can be performed automatically with the use of monitoring scripts (code written for a specific monitoring purpose). There is also Commercial-Off-The-Shelf (COTS) software that can be used to perform monitoring. Several monitoring COTS tools are available in the market. Often these tools (software applications to monitor hardware and software) are only able to monitor specific layers of the complex, multilayered, environment. Very few monitoring systems can monitor all the layers of the environment. This is partially because the monitoring systems must be created to be compatible with specific software or hardware and partially because such a comprehensive monitoring system would be very large, complex, and difficult to keep up to date because the software being monitored changes rapidly.

Finally, often people still need to review the results of the monitoring for things that the automatic monitoring systems may not be programmed for. By reviewing the system logs and other system information, experts can detect new issues before they impact system performance and negatively impact the business.

One of the biggest challenges in modern complex business technology environments (cyber ecologies) is how to monitor the business effectiveness of the systems. This is because, to the business, it is the functionality that is required and what the IT organization is being financially supported to provide. Collaboration between the IT organization and the business is needed to set

the performance expectations of the functions the software and hardware provide. Often this performance is a combination of many layers (tiers) of the current complex environments.

Monitoring system security is another key goal for organizations. Security monitoring is quite different from system health monitoring. Security monitoring looks for inappropriate use of the systems, whether internal unapproved usage (such as accessing data someone is not authorized to access) or unauthorized access from external agencies (human beings or other computer systems). Every layer of the environment (hardware and software) is monitored for situations such as too much data being downloaded from a data store (database), too many requests for connection coming from outside the company (indicates an attempted break in), internal people accessing either systems or data they are not authorized to access, and files that do not belong or are not approved to be on the hardware (which could indicate malware of various types).

Monitoring systems is good; however, once a pre-defined issue being monitored for is detected, someone must be notified. In today's modern systems there are copious logs (files containing information on system health based on pre-determined scenarios) for most systems (hardware and software); however, they need to be reviewed. Many monitoring systems have dashboards that provide a visual representation of system health. Monitoring systems can be set up to email, text, or page workers to notify them of an issue. All the information regarding any anomalies is also recorded (logged) in files. The management of these files includes making sure the monitoring logs are not able to be tampered with or inadvertently deleted before they can be reviewed and any necessary actions taken to fix a situation is reported in the logs.

Monitoring, whether manual or automated, is intended to provide ongoing demonstration of meeting the organizational compliance requirements.

Systems Auditing

Auditing has been a part of human business for a very long time. Every time a shepherd counted his sheep to ensure they were all present is a form of auditing (although a very basic form). Financial auditing, looking at financial records to determine whether they are accurate or not, has been a part of business for a very long time. Auditing of systems is much more complex in that many more factors need to be audited than either counting sheep or performing financial auditing.

To audit something is to check whether the system activities, system security, or user activities are in compliance with the defined expectations for the particular organization within a company. Auditors are looking to confirm or refute that the systems and processes are meeting identified corporate goals. As discussed before, the primary corporate goals revolve around systems working as expected, protecting business assets (such as intellectual property), and ensuring data integrity.

Similar to monitoring, automated auditing can be set up as only accessible to either internal or external approved auditors. Regular internal audits by many levels of personnel are performed. The monitoring efforts as noted in the prior section are a type of auditing that is performed by those supporting the IT systems. However, as those supporting IT systems are suspected of possibly being biased, the majority of medium-to-large companies will designate a particular group of internal non-IT system supporting people to identify what should be audited as well as how often each item is to be audited. These internal auditors will create and provide reports to all types (business and IT) and levels of corporate management. Along with the audit reports these internal auditors will work with the business and IT to make suggestions on processes and system goals.

Regular audits by external auditors (agencies) are to be performed on a regular basis. Generally, a team of auditors is used that consists of people who are experts in specific fields of IT auditing, whether that SOX requirements or a specific IT system. The use of external auditors (agencies) to perform regular audits increases trust in the processes and systems as external auditors are expected to be objective.

SUMMARY

Frameworks and models are the minimum needed to meet expected compliance requirements. Beyond meeting the expected compliance requirements, organizations are encouraged to always be seeking methods to increase the security of the deployed products and services.

KEY TERMS

adherence
auditing
Capability Maturity Model Integration (CMMI)
compliance
compliance measurement
Computer Fraud and Abuse Act (CFAA)
Control Objectives for Information and related Technology (COBIT)
Electronic Communications Privacy Act (ECPA)
Fair and Accurate Credit Transactions Act (FACTA)
frameworks
General Data Protection Regulation (GDPR)
governing body
Health Information Technology for Economic and Clinical Health (HITECH)
Health Insurance Portability and Accountability Act (HIPAA)
Information Technology Infrastructure Library (ITIL)
ISO/IEC 27001
laws and regulations
monitoring
open-standards
Payment Card Industry Data Security Standard (PCI DSS)
Personal Information Protection and Electronic Documents Act (PIPEDA)
personally identifiable information (PII)
protected health information (PHI)
Sarbanes-Oxley Act (SOX)

Secure Software Development Framework (SSDF) standards

References

Jackson-Higgins, Kelly. “Russian Hackers Sentenced in Heartland Payment Systems Breach Case.” *Dark Reading*, February 16, 2018. https://www.darkreading.com/attacks-breaches/russian-hackers-sentenced-in-heartland-payment-systems-breach-case/d/d-id/1331080

GDPR. 2018. “General Data Protection Regulation (GDPR).” General Data Protection Regulation (GDPR). 2018. https://gdpr-info.eu/

Congress.gov. 2019. “H.R.3763—107th Congress (2001–2002): Sarbanes-Oxley Act of 2002.” Congress.gov. 2019. https://www.congress.gov/bill/107th-congress/house-bill/3763

“H.R.4952—99th Congress (1985-1986): Electronic Communications Privacy Act of 1986.” 2019. Congress.gov. 2019. https://www.congress.gov/bill/99th-congress/house-bill/4952

Hughes, William J. 1986. “H.R.4718—99th Congress (1985–1986): Computer Fraud and Abuse Act of 1986.” Congress.gov. October 16, 1986. https://www.congress.gov/bill/99th-congress/house-bill/4718. https://www.congress.gov/bill/99th-congress/house-bill/4718

CDC. 2018. “Health Insurance Portability and Accountability Act of 1996 (HIPAA).” Centers for Disease Control and Prevention. 2018. https://www.cdc.gov/phlp/publications/topic/hipaa.html

Health Information Technology for Economic and Clinical Health, Pub. L. No. 111-5, 123 Stat. 226 (2009). https://www.hhs.gov/sites/default/files/ocr/privacy/hipaa/understanding/coveredentities/hitechact.pdf

Fair and Accurate Credit Transactions Act, Pub. L. No. 108-159, 117 Stat. 1952 (2003). https://www.congress.gov/108/plaws/publ159/PLAW-108publ159.pdf

"Office of the Privacy Commissioner of Canada. 2019. “The Personal Information Protection and Electronic Documents Act (PIPEDA)—Office of the Privacy Commissioner of Canada.” Priv.gc.ca. 2019. https://www.priv.gc.ca/en/privacy-topics/privacy-laws-in-canada/the-personal-information-protection-and-electronic-documents-act-pipeda/

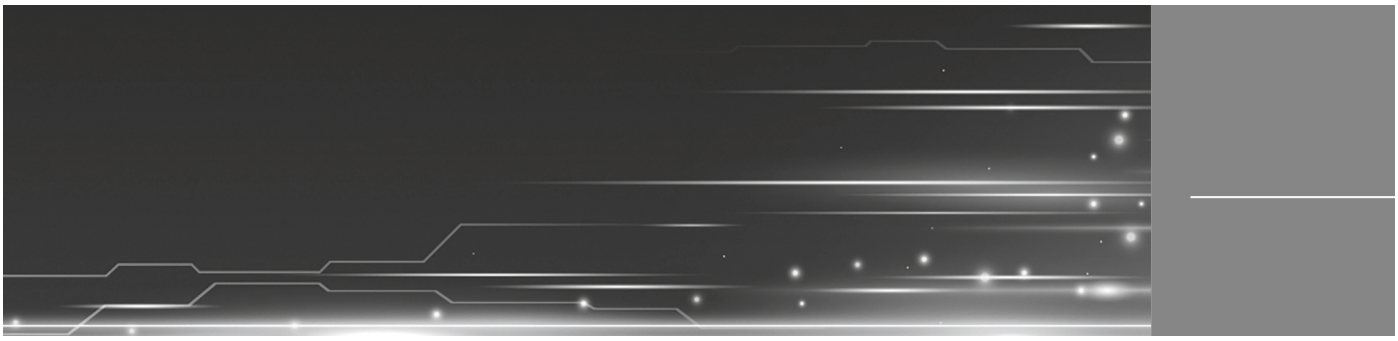
COBIT 2019 Framework: Introduction and Methodology from ISACA

Kempter, Stefan. n.d. “ITIL 4 | IT Process Wiki.” IT Process Wiki—the ITIL® Wiki. https://wiki.en.it-processmaps.com/index.php/ITIL_4.

“CMMI Institute—Home.” 2021. Cmu.edu. 2021. http://www.sei.cmu.edu/cmmi/.

Computer Security Division, Information Technology Laboratory. 2021. “Secure Software Development Framework | CSRC | CSRC.” CSRC | NIST. February 25, 2021. https://csrc.nist.gov/Projects/ssdf

Constantin, Lucian. 2020. “SolarWinds Attack Explained: And Why It Was So Hard to Detect.” CSO Online. December 15, 2020. https://www.csoonline.com/article/3601508/solarwinds-supply-chain-attack-explained-why-organizations-were-not-prepared.html



© d1sk/Shutterstock

CHAPTER 9

Quality Management

CHAPTER OBJECTIVES

- Outline how quality management contributes to project success.
- Explain the difference between quality assurance and quality control.
- Examine the purpose of root cause analysis.
- Describe the place of the quality team in the organization.

Introduction

Quality. We all expect it, but what does it mean? How can anyone creating products important to human life be assured that products do what they are supposed to do, when they are supposed to do it, for as long as they are supposed to it, and with the security they are supposed to do it with? Quality management is the process of ensuring that the products, services, and processes of a company are produced and followed in a consistent manner, ensuring that the organization is producing product and results as free of defects and deficiencies as possible. Quality management includes everything that is done when creating or updating a product to make sure what results meets all measures of quality, as determined by the client, and the industry. A product that is as free of defects and deficiencies as possible has many benefits from happier and more satisfied clients, development teams that better understand quality management processes and follow them, to less time and money fixing issues.

Quality management consists of two subsets, **quality assurance (QA)** and **quality control (QC)**, each with multiple parts. Quality assurance focuses on processes and tools used to ensure quality is baked into products throughout the secure software systems development life cycle. It includes establishing quality goals. Quality control focuses on the product and includes processes and procedures that directly measure and ensure that quality goals are being met.

Quality Assurance

The focus of quality assurance is to make certain that the environment in which a system or other product is being developed is conducive to producing a quality product. As a part of that focus, quality assurance will apply processes and tools, establish expectations for how tasks are completed, ensure that training in the proper processes is completed, and audit and monitor processes adhere to the required standards.

Processes

Quality assurance (QA) processes and oversight give customers, regulatory agencies, and company management confidence that the product will meet expectations by ensuring that the processes produce a quality product. QA relies on consistent, repeatable, documented, and communicated processes to make sure that everyone involved in the creation or update of any product (software or otherwise) follows the defined processes. By following the defined processes, product creation or updating is less likely to include inadvertent errors or miss expected requirements. It is a proactive management approach with the goal of limiting issues (bugs) being introduced while a product is being created.

QA processes have developed over time from thoughtful people getting together to plan how to create something and, unfortunately, from learning from past mistakes when building a product. We all learn from our past experiences and how to achieve quality is no exception. Each past experience helps us add to the plan when creating a new product or updating an existing product. Planning for quality makes us more likely to meet the **quality expectations** of customers, clients, users—anyone who will use the product or service we're creating. As with other human efforts, past experiences and planning have resulted in a defined life cycle for QA (as shown in **Figure 9.1**). Quality assurance processes are closely related to many process improvement efforts.

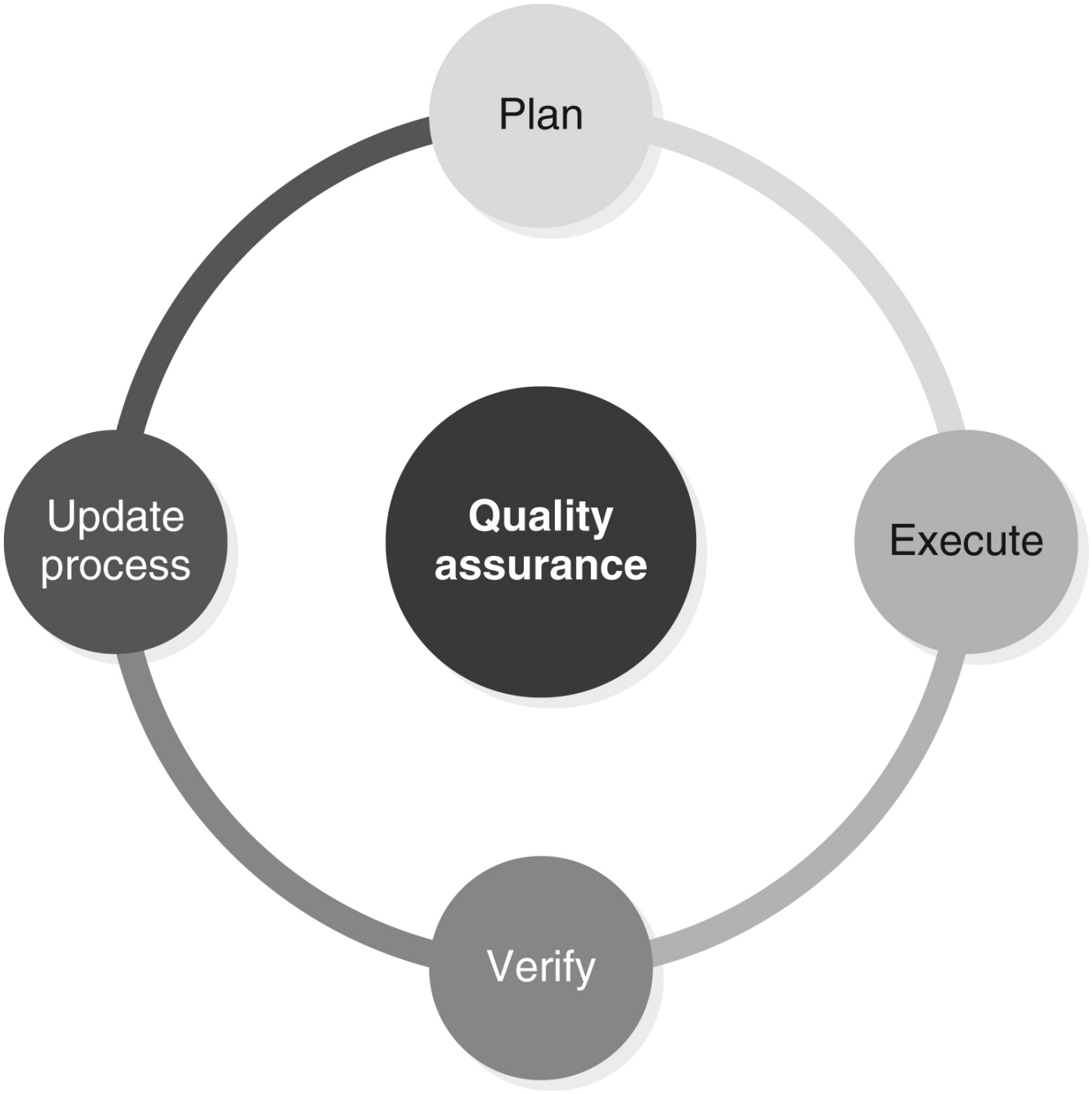


FIGURE 9.1 QA Life Cycle

Description

QA processes may be determined by the company for all projects, by a central program or project management office, or may be specific to the project but based on any of the many different available quality assurance frameworks (such as ISO 9000 group). It is better if QA processes are established for the entire company, but at least for specific product types. If building cell phones, then any area of the company that builds specific cell

phone parts must use the same processes, there are no changes to quality processes based on culture or geographical area. To effectively perform quality assurance, adequate time is needed for all processes, just as adequate time is needed to produce quality software.

At the same time, while the focus of QA is on customer satisfaction, often products must meet standards, regulations, and legal obligations. Electrical products such as toasters or house wiring have very stringent regulatory quality requirements to make sure no one gets shocked or a fire is not started. These regulations are in place because of bad things that have happened in the past causing terrible damage to property and people. Such standards, regulations, or legal requirements must have quality processes defined before any development project begins. Otherwise, the development team may not design what is needed for such requirements.

As noted, frameworks are available (such as the ISO 9000 group or test maturity model) to help organizations develop their QA processes and tracking. Many of the frameworks also provide certification in their specific framework. Companies are certified as effectively using the framework. QA certifications, like any certification, helps to reassure customers that the product has been created or updated using the best possible quality approach. Such certifications also reassure company management that they are following well-defined and tested QA processes.

Tools

Many different software tools are available to help teams monitor and audit their product quality. As well as QA-specific tools, many development tools discussed in this book are also able to perform QA functions, including some automatic monitoring and auditing to make sure development teams follow the agreed upon QA processes.

Among other tasks, QA tools automate and systematize the collection of data to determine if process steps were followed and addressed in the proper order. As each step in a process is completed, the person performing the step logs the action in the system. Ideally, this is done as electronically as possible, such as through the scanning of an RFID tag or bar code. The system would record the ID, the person completing the task, the task completed, and any other relevant information.

Automated systems for tracking system development are available as components, add-on applications to source control systems, or other integrated development environments. These systems not only track code being checked

into and out of the system, but they also maintain data about what was done with the code while it was checked out, such as adding support for specific requirements, fixing bugs, unit or integration testing, and code review. With these tools in place, organizations not only know who accessed a particular piece of code, they know whether the right processes were applied and who completed the task. Using such a tool, it is quick and easy to check to see which modules of a system have gone through a security review.

Task Expectations

QA is also responsible for ensuring that established and recorded expectations or standards for tasks will be completed. This is an important part of processes that is often neglected, often because it simply seems unnecessary. Suppose someone is asked to go get some water. How much water do they need to get? What are they supposed to use to get it? Where should they put the water upon their return? Without standards for how tasks are completed, it is impossible to ensure consistent effort quality and therefore consistent product quality.

Training

Another role of QA is ensuring that training is available for every task and that those who are completing the tasks have received that training. The QA role is not the development of the training, but rather planning and monitoring to ensure that training happens.

Auditing and Monitoring Processes

The final role for QA is the auditing and monitoring of processes. The focus on these audits is not the outcome of the process. Instead, the audits and monitoring focus on ensuring that the processes are being followed as documented, that any changes to the processes are made in accordance with established methods, and that the changes are correctly incorporated into the processes and followed.

Asking people, “Do you follow the defined processes,” does not objectively prove that expected processes are in fact being followed. Of course, everyone would respond with an emphatic “YES” to such a question. It takes reviewing the quality of process artifacts (such as documents), including accessibility, and confirming they are up to date.

Product development teams need to be able to assess whether they are following the QA processes that were set out at the beginning of the project. These tell the team to what degree the team is performing the actions necessary to produce a quality product. When the audits identify that areas of QA processes either are not being followed or only somewhat followed, the development team needs to be notified. When quality processes are not being followed, action needs to be taken to get back on track and avoid future lapses.

While QA audits and monitoring can be performed by internal people, there is value in having external people come in to at least perform regular audits. Having external audits performed are a confirmation of any internal audit results and provide increased credibility for the audit results. Engaging external people to do such work also frees up internal people (familiar with the company, its culture, and business needs) to work on other business contributing tasks.

Quality Control

If QA is a proactive effort defining expected outcomes and processes to achieve those outcomes, then it follows that QC verifies QA was effectively followed and the final product meets quality and business functional expectations.

Quality Expectations

The first step in quality control is to ensure that **quality expectations** are established for the product or system being created. These quality expectations are identified as a part of the requirements phase of the project and are themselves requirements. As the requirements are established, the quality control team is going to ensure that those requirements selected to evaluate the quality of the product are measurable. These requirements are labeled as **quality measures** or **metrics**.

If appropriate, the team will also establish the tolerances for the measurements. While for a physical device, this may mean it has a dimension of 1mm +/– 0.05 percent, for software, the tolerances are often measured in terms of the numbers of defects (bugs) at various levels of criticality, the percentage availability the system can achieve, or the ability of the system to operate in a high stress environment.

Testing

Testing, as discussed in a separate chapter in this book, is a major part in quality control. Monitoring and auditing whether product team members are following the pre-defined QA processes tells everyone that processes are being followed but does not establish if the product meets the expected quality. Testing using defined quality expectations demonstrates that the product meets all functional, performance, security, quality, standards, legal, or regulatory requirements. The testing is not performed by the quality team. Instead, the results of testing get documented and reviewed.

The QC team will monitor and periodically audit the progress of testing.

Root Cause Analysis

Another concern of QC is the application of **root cause analysis**. This involves looking behind issues identified in testing, monitoring, and auditing to identify not the specific issue itself, but rather the fundamental reason that the issue occurred.

For example, a quality team may identify that several buffer overflows were present in a piece of software. Naturally, these all need to be identified and resolved, and additionally, an effort should likely be made to identify any similar issues. Moreover, the team needs to investigate and identify why the buffer overflows made it into the software to begin with. Potentially, the programmers have not been properly introduced to the concept of a buffer overflow and are making the errors because they are not aware it is an issue. However, rather than the programmers, the buffer overflows could be the result of a switch not being properly set on a compiler or a bug in the compiler. Ultimately, it is even possible that the buffer overflow is being induced by malware in a third-party tool in an attempt to create a vulnerability. Whatever the reason, the quality team needs be persistent in their efforts to uncover the ultimate or root cause of the issue.

Corrective Actions

Once an issue has been identified, and the root cause of the issue investigated, the quality team needs to monitor to ensure that **corrective actions** are identified and applied. It isn't enough to know what went wrong, why it went wrong, and how to fix it. Unless a fix is actually applied, the issue remains. It is also important that the appropriate follow-up testing was completed to ensure the corrective actions were properly applied and caused no new issues to appear.

Quality Organizations

The quality function appears in different areas in different organizations. This is a result of how different organizations view the quality function, legal or regulatory requirements, and organizational culture. In some organizations, the quality function is lumped together with testing in the belief that the two functions are accomplishing the same task. They are not. Ideally, the quality function, much like the chief information security officer or internal audit, reports as directly as possible to the highest layer of the organization, either the chief executive officer or the board.

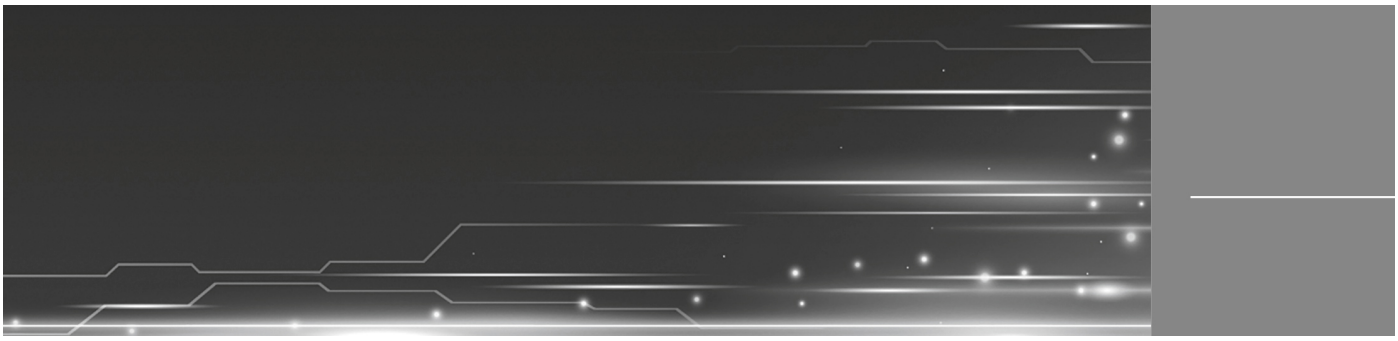
However the reporting structure, the quality organization needs to be able to monitor across the organization, not be distracted by the chore of running tests, and be able to identify quality issues and provide input to all.

SUMMARY

Ensuring the development of a quality product is vital in any organization. The ability to ensure that happens depends on the careful management and monitoring of quality throughout the development process. Quality management is not the task of conducting tests but rather a process of ensuring processes and procedures are documented, followed, and improved as needed. Quality requires not just identifying defects and issues, but also identifying the root causes from which issues arise.

KEY TERMS

corrective actions
quality assurance (QA)
quality control (QC)
quality expectations
quality measures or metrics
root cause analysis



© d1sk/Shutterstock

CHAPTER 10
Modeling

CHAPTER OBJECTIVES

- Discuss how models are used to analyze problems and to develop and communicate solutions.
- Explain differences in the degrees of completeness and formality of models.
- List and describe some common modeling techniques and circumstances when they may be chosen.
- Demonstrate the application of a model to a problem.

Introduction

Modeling is a tool that is used frequently in the secure software development process. Modeling is a process where we create a graphical representation of a system or a portion of a system. Modeling has distinct advantages because people are naturally visual. Our minds are accustomed and attuned to interpret information from visual presentations. We can naturally spot patterns in images and are drawn toward unique or interesting items that stand out clearly. Additionally, the process of making a drawing engages a natural organizing mechanism in our brains. Modeling lets us take advantage of our natural abilities in a number of ways.

Because of the positive attributes, modeling will be used throughout the secure software development life cycle. In the early stages of the cycle, we will often use rough informal models to inform our thinking about what we want to build. There is a reason why so many systems first begin as drawings on napkins. As we try and define what we need, drawing a model becomes a way of organizing our thoughts so that we can then translate them into the prose that will become the basis for the initial system description.

Later in the secure software systems development life cycle we will use other models to help in the requirements gathering and engineering processes, in the architecture and design efforts, in developing a vulnerability map, and in planning for testing. Models are also used throughout the life cycle in documentation because they are useful in helping to clarify what the text really is trying to say.

The section on processes and process mapping gave a brief example of modeling used to create **process flow diagrams**. These, like any other model, are graphical representations that help us to understand, study, and design the processes we are concerned about. While the processes modeled may be those that the organization is using for the secure software systems development life cycle, for administering the business, or for interacting with customers, there will also be processes involved in the tasks that are completed by the software system being built. These processes can be modeled in the same manner.

Sometimes models can be used in the requirements phase to help stakeholders understand and communicate their inputs into the requirements. Often these models are crude, such as the back-of-the-napkin types (see **Box 10.1**), but they serve an important communication function. Modeling is also

used during the requirements engineering phase. Here it helps to clarify and understand the meanings of gathered requirements. It can also be helpful in understanding the relationships among different requirements.

BOX 10.1 The Famous Napkin

We often hear about back-of-the-napkin designs and perhaps are wondering about the term. Not all design work and insights into design challenges happen in the office. In fact, a significant number come about during casual conversations between colleagues. Frequently, this happens during lunch or dinner. As they begin to talk about what they are up to, one thing leads to another, and they start discussing their design challenges. Suddenly, the one colleague has an insight into how to approach the problem, or just by talking the issue through, the first one comes to an insight on a solution. Naturally, not wanting to lose the insight, somebody reaches for the first available item to draw or write the solution. Often, this is a napkin (sometimes used – hence they turn to the backside).

These are similar to the casual conversations about "can this be done?" that lead to "hold my beer…" or "I wonder how much it would take?" that lead someone to grab the nearest scrap of paper, often an envelope, and create some "back-of-the-envelope" calculations for how many servers it would take to run a system that supported 24X7 monitoring of egg production by chickens in facilities spread across 14 states.

Sometimes, the modeling done for requirements engineering can become the first step for the design process. That first step of design using modeling can be begun even if none has been done for requirements engineering. There remains the need for establishing a clear understanding and clarification of the requirements. The modeling will show the inner relationships of the parts and pieces of the system being designed.

Modeling at this early point can be used to explore different design concepts and perform a systematic inspection and analysis of different potential solutions. The model can be used to explore how the components interact while highlighting potential issues. These early models can be used to drive the potential design as well as to begin the system documentation.

When begun on these early models, vulnerability mapping, which will be explored in a future chapter, allows the examination of potential vulnerabilities that can plague a system. Done early, the results of vulnerability mapping can

be used to drive the shape of the design to eliminate or limit many of the potential vulnerabilities. Delayed, vulnerability mapping leads to later recognition of vulnerabilities that may not allow as much control over how the vulnerabilities are addressed in developing the system. Still, delayed vulnerability mapping does remain useful in limiting impact of vulnerabilities that are discovered when it is no longer feasible to avoid them. Of course, the best solution is mapping vulnerabilities repeatedly as the design evolves.

Some will argue that there is never enough, and certainly never too much, modeling that can be done. The reality though is that resources are limited and that includes the resources that are available for modeling, including time, talent, and tools. As a result, the best approach is to apply modeling where it is the most efficient and effective and where it will give better results than other tools and techniques. Some of the areas for which those conditions apply include user interactions, interfaces, actions or processes, and components.

Choosing a Model

A great variety of types and tools are available for creating models. The choice of what type of model to choose for a particular task is based on many factors. Some types of models are better suited for some situations than others. This can be because they are more easily translated into the intended solution space (programming language, database, enterprise architecture, etc.) or they are simply a better fit for the problem space. The choice of the model type used can have a significant effect on the design outcome.

The choice of the model type used can affect how a problem is understood, but at the same time, understanding the problem can help with the selection of appropriate model types. Because of this, it is useful to apply different model types to a problem in order to broaden the understanding of the model as well as to find the model types that are most appropriate. For example, we may start building a functional model to understand how the different parts of a system will work together, but then realize we don't have a good understanding of how data elements passed about are related and grouped. At this point, we can switch to data modeling and, then once we sufficiently understand the data, return to our functional model.

No single model type is going to appropriately capture all elements of the problems we are modeling. Therefore, as different models are applied, no single model will need to be selected, but rather a set of models that, as a composite, will provide the best coverage of the problem. Each model will contribute to the overall understanding of the system as well as providing useful insights to be applied to the other models.

Not all modeling will be done at one time. Because models are used for different reasons and models can be applied at different levels of detail, different models will be constructed at different times. In addition, those models that are already built will often need to be updated to reflect new information, new requirements, or to gain new understandings.

Modeling Tools

When creating a model, no matter the type chosen, generally many tools are available to assist with the modeling. These tools can range from simple drawing utilities to applications that automatically generate a model from text or other inputs. Some of the important considerations in selecting **modeling tools** include support for the type or types of models desired; the amount of automation, such as restructuring and annotations; the style and form of the graphics; the ability to accept different inputs; support for mapping between different model types; and flexibility. Additional considerations are the cost, the learning curve, and availability.

Many modeling tools also have drawbacks, especially during the initial creation phase. The first of these is that computer-based modeling tools only have a limited ability to display a broad view of a model. What can be shown is limited by the physical size of the display and by the resolution of the display. For a large model, even with a huge monitor, the detail gets fuzzy for the individual parts as the view zooms out. Eventually, the whole thing can just become a blur. Some modeling tools try to resolve this by using different levels, but the solution is not always satisfactory.

Another modeling tool problem is that in the early stages of model development, there are often frequent significant changes to sections of the model. Items that were in one place may be moved wholesale to another place, or the items may be divided apart and placed in many different areas. These types of changes can be really difficult for many modeling tools to gracefully accommodate. As the model becomes sufficiently complex, changes take increasing time to process, and the routing and rerouting of connections can overwhelm the system.

A challenge with the display on many modeling tools is that it can be hard to compare different regions of a model. This challenge can be overcome somewhat by having a number of different displays side by side, but that is not always a good solution.

The limitations of displays on tools also extends to when several people are trying to work on a model together. A standard display is difficult for everyone to see, but even a large projected display may not be satisfactory, especially given resolution issues. Then when multiple people are trying to contribute, there is a constraint of a single keyboard for entry. Some tools allow multiple users to provide input to a drawing or document at the same time. However,

even with these types of tools, there are limitations in how they can be accessed and how to manage the discussion around the effort. Also the tools are prone to becoming unsynchronized with one or more to the users getting their inputs locked out of the system. Even with these issues, these tools can be useful for distributed teams when they are unable to physically gather.

An effective solution for when a team can be assembled, especially appropriate at the early stage of modeling, is to find a large surface, such as a white board, a wall, or a window. (I once had an office with four glass walls, which was perfect.) Then using a dry erase marker (great for windows and white boards), sticky notes or 3-by-5 cards and yarn (different colors for different meanings), or a combination, lay out the model. The bigger the space the better. Drawn circles, the sticky notes, or 3-by-5 cards become the items, objects, entities, components, or states (depending on the language of the modeling types), and dry erase lines, colored yarn, or string becomes the connections, relations, paths, or routes between them.

With such a model, it is easy to see and understand the big picture, while zooming in on details by walking closer to particular sections. Multiple people can be studying and working on different parts at a time, and it is relatively easy to manipulate the model by moving sticky notes, erasing and redrawing lines, and putting up more yarn. Once the model is completed, then it can be transferred from the wall-mounted model into a computer-based representation. Sometimes the large-scale model proves so useful it may be kept as it is. I once had a model that covered a 40-foot wall, floor to ceiling, that we kept for more than 3 years before finally removing it when the offices relocated.

Sometimes, though, hand modeling techniques do not work as well as digital tools because the latter can make parts of developing a model easier. For example, a digital tool used to generate a state diagram can check that all conditions are covered when considering transitions or that the system always reverts to a safe state. This is something that can be done by hand for a simple system but becomes increasingly difficult to track as the complexity grows. Modeling tools are also useful with a distributed team where not everyone can gather at the same location. For these situations, the communications methods that accompany the tools—maintenance of lists, tracking of changes, highlighting abilities, user privilege management—can be important considerations in selecting which tool to use.

Some modeling tools can create simulations based on the models developed. These can be used to help find challenges and omissions in the model.

Some modeling tools have an additional advantage in that they can generate code directly from the model. The quality of the code generated is directly related to the quality of the modeling done, with sloppy or poor modeling leading to poor quality code. Often the tools allow for some degree of control over the code produced with options to optimize for performance, optimize for code size, or optimize for some other factor. Tools that produce code often have their own constraints on how the modeling technique is applied for the tool to be able to correctly produce code.

While the code produced is normally syntactically and functionally correct, it may not provide full coverage, it may not be totally free of security issues, and it may not address the solution that the modeler thinks it does. The code, like any other needs to be thoroughly reviewed and tested. A frequent challenge to this though is that much of the code produced in this manner is challenging for people to read and follow and may not be well documented.

Modeling Techniques

This section will examine several different **modeling techniques** in addition to the process modeling already previously discussed. These are descriptive discussions and are not meant to be definitive detailed guidance on the use and application or a formal definition of each tool. If this is necessary, the specific documentation for the technique or for the tool specific implementation of that technique should be researched. Yes, that is correct, while each technique may have a formal specification, tools that implement that technique may or may not fully or only partially meet that specification or may include its own specification or extensions and limitations of the technique as implemented in the tool.

It is important to remember that it is not always essential to apply the exact rules and methods of any technique to create a useful model. It is generally a good idea to follow basic conventions and usages to allow others to understand the information more readily being conveyed in the developed model, but usefulness triumphs over form. Further, some techniques are so strongly specified that the study of the technique overwhelms the ability to apply it. So, for example, only two people in the world use the Unified Modeling Language (UML) exactly as it is specified, and one of them is retired.

Flow Charts

One of the most common modeling techniques used in programming is the **flow chart**. Flow charts are used in low-level modeling to capture the implementation of complex algorithms. For a time, they fell out of favor as everything in programming became “object oriented,” and the flow chart was seen as an old-fashioned obsolete procedural tool. As an overall module of an entire program, this is undoubtedly true. At the heart of each object are the methods (along with the attributes, properties, or whatever they are called in the specific implementation). Among the methods are many that require little if any modeling, some which can best be modeled through different means, and a not insignificant number that require the implementation of a complex algorithm. This is the place where flow charting is appropriate, and often in fact essential.

Flow charting is done with a symbol library. The basic symbols are for decision, iteration, and action, along with a start and a stop state as can be seen in **Figure 10.1**. There is a complete library of other specialized symbols, most of which are rarely used, especially in flow charting based on complex

algorithms. Each of the symbols are joined by unidirectional lines. The path through a flow chart follows the path from the beginning, through the various symbols, and eventually reaches an end. An example of a flow chart is given in **Figure 10.2**. Figure 10.2 features a flowchart for a bubble sort algorithm.

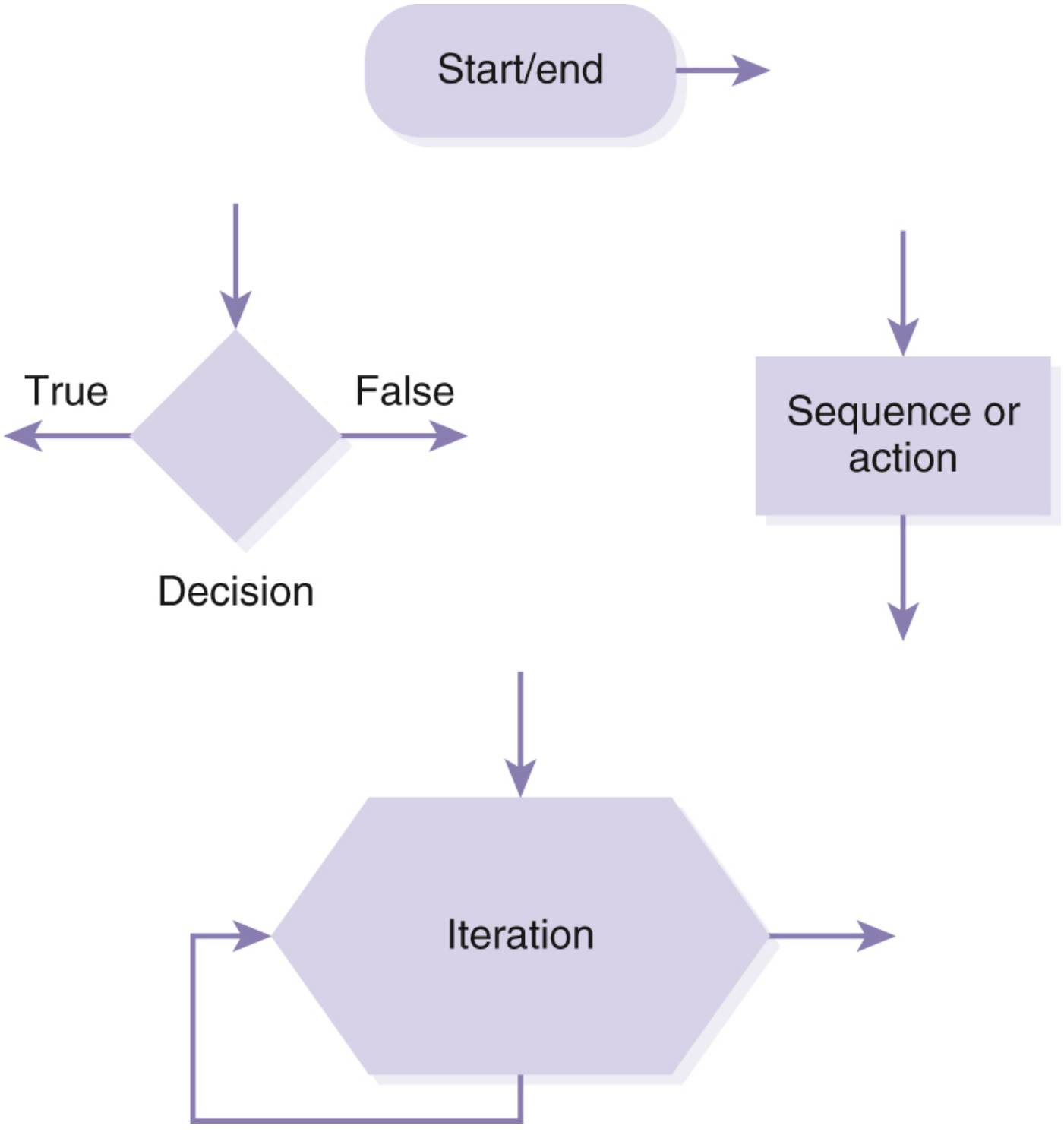
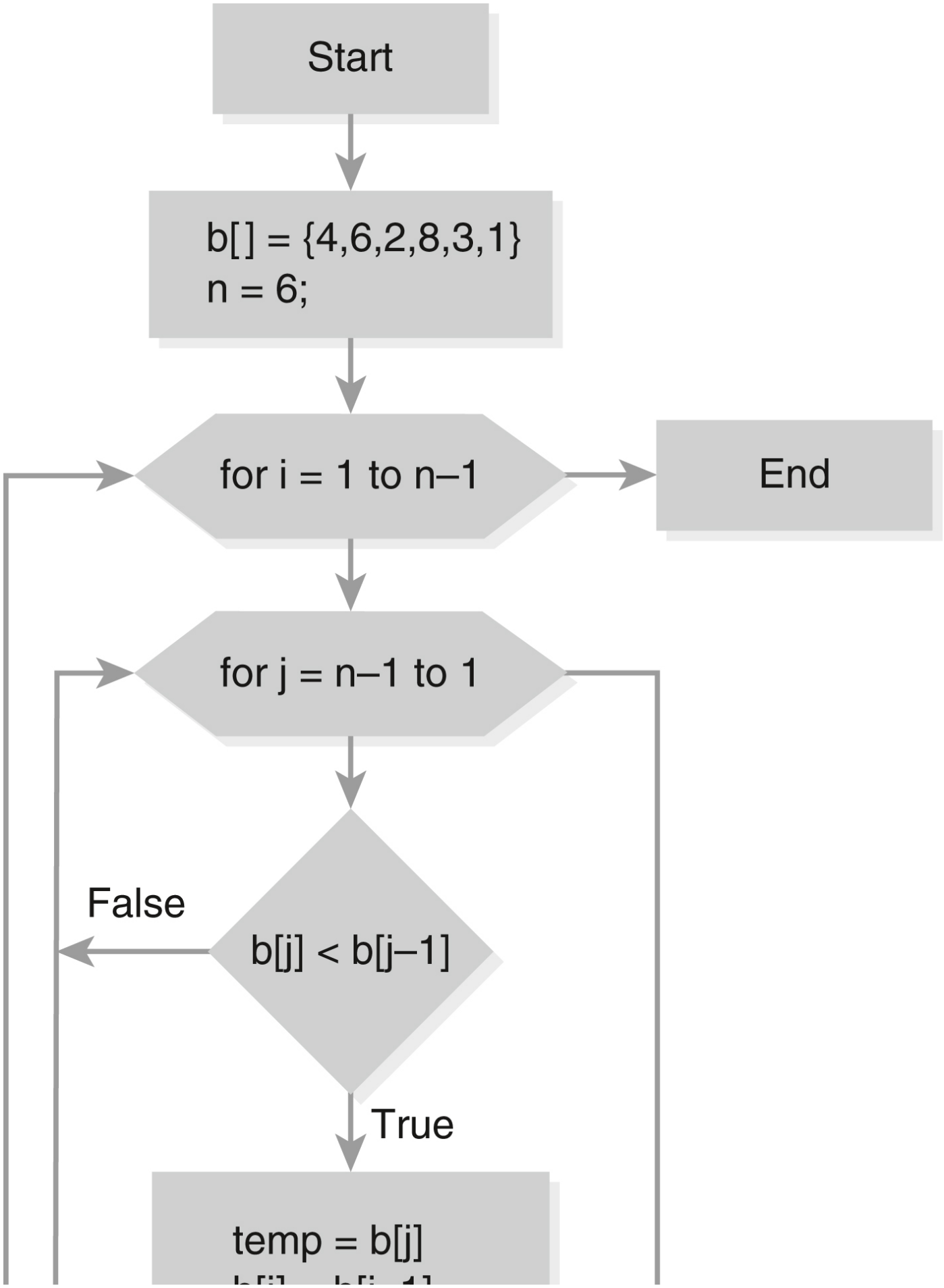


FIGURE 10.1 Example Flow Chart Symbols

Description


Start
b[] = {4,6,2,8,3,1}
n = 6;
for i = 1 to n–1
End
for j = n–1 to 1
False
b[j] < b[j–1]
True
temp = b[j]

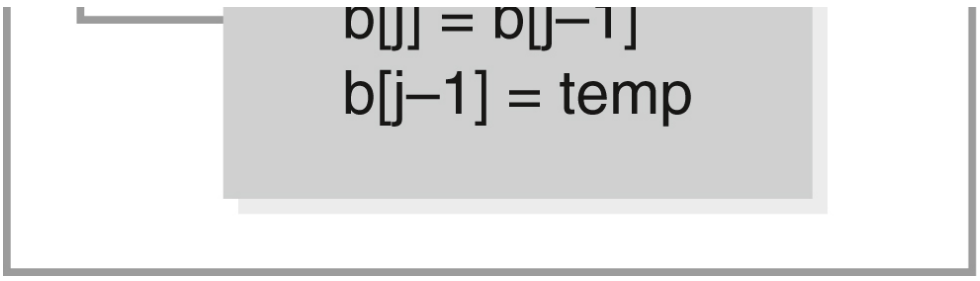


FIGURE 10.2 Example Flow Chart for a Bubble Sort

Description

At a decision point, represented by a diamond, the path will branch in one direction or another. Each direction is labeled (“true”/”false”), (“>60”/”<=60”), etc. Multiple possible paths, such as with a case or switch, can be constructed from subsequent decision points.

Iteration points, represented by an elongated hexagon, will include control information in the symbol, such as the pre/post calculation, or number of iterations, which indicates how the looping is controlled. In addition to the initial path, or paths, into the iteration point there will be at least one path returning to the symbol from the internal loop path. These paths may be obvious from the context, may be explicitly indicated on the chart, or may follow a specific convention, such as the initial path in is at the top, the iteration path exits the bottom and returns on the left, and the path exiting comes out the right side. Any iteration can also be decomposed into its constituent decision and sequence steps.

A sequence or action step is used whenever any action is taken that is not a decision or iteration. This could be a calculation, a function call, instantiation of an object, input or output (which also could use their own special symbols, some of which are dated, such as reading from a paper tape), or setting a value.

Another common symbol is a circle, which is optionally used to combine multiple paths. This can also be accomplished by having the lines converge on the appropriate symbol for the next step or by simply joining the lines together.

As the model starts, an array is defined with values that will be sorted, and an index is set to the number of elements in the array. The model includes two loops, an outer and an inner loop. Inside of the inner loop, the current element is compared to the previous element. If the current element is smaller, the position of the two values is reversed. After both loops have been completed, the array will be sorted in order, smallest to largest. This is a relatively simple example. Some flow charts end up very complex but provide a means for the

developer to visually check that the algorithm executes as expected. In some cases, automated tools can both execute the algorithm graphically and can convert the flow chart into code.

While there was a time when flow charts were used to model an entire program at a very detailed level, current application is to create specific detailed flow charts for critical pieces of a system or to create a higher-level generalized flow chart, often with detailed subcharts in specific areas.

Structure Charts

Another programming-based model is the **structure chart**. The idea behind the structure chart is that people are not always good at solving great big problems. Instead, the problem can be broken apart into smaller problems. These smaller problems, if they are still challenging, can again be broken apart into another layer of smaller problems. Eventually, instead of a large problem, there is a set of smaller, solvable problems. These are solved, and then put together to provide a solution at the next layer up. Done repeatedly, the big, hairy, nasty problem is solved.

The goal of the structure chart is to define the components within a system, where components are modules that define the solution to a problem as a specific task or set of tasks. The components are defined in a hierarchical manner. At the highest level, the components represent groupings of generalized high-level tasks. Each of these is then decomposed into smaller, more specialized groupings, until at the lowest level, each component represents a single simple function. Simple here meaning not decomposable because it may actually be a fairly complex function, but one that can't be further broken apart. These simple functions may be common to multiple higher-level components, in which case they are reused rather than rebuilt.

The goal of the decomposition is to create a high level of fan out at the top layers. That is to break the problem into the largest number of smaller components possible while still maintaining logical coherence of function at that abstract layer. At the lowest layer, the goal is to create a high level of fan in. This requires that common functions are identified and reused to the greatest extent possible.

Structure charts can be used in different programming paradigms. The chart does not define the programming paradigm but rather the structure of the tasks needed to solve the problem. The result can be mapped to an object-oriented, a procedural, or a functional program. An example of a structure chart is seen in **Figure 10.3**.

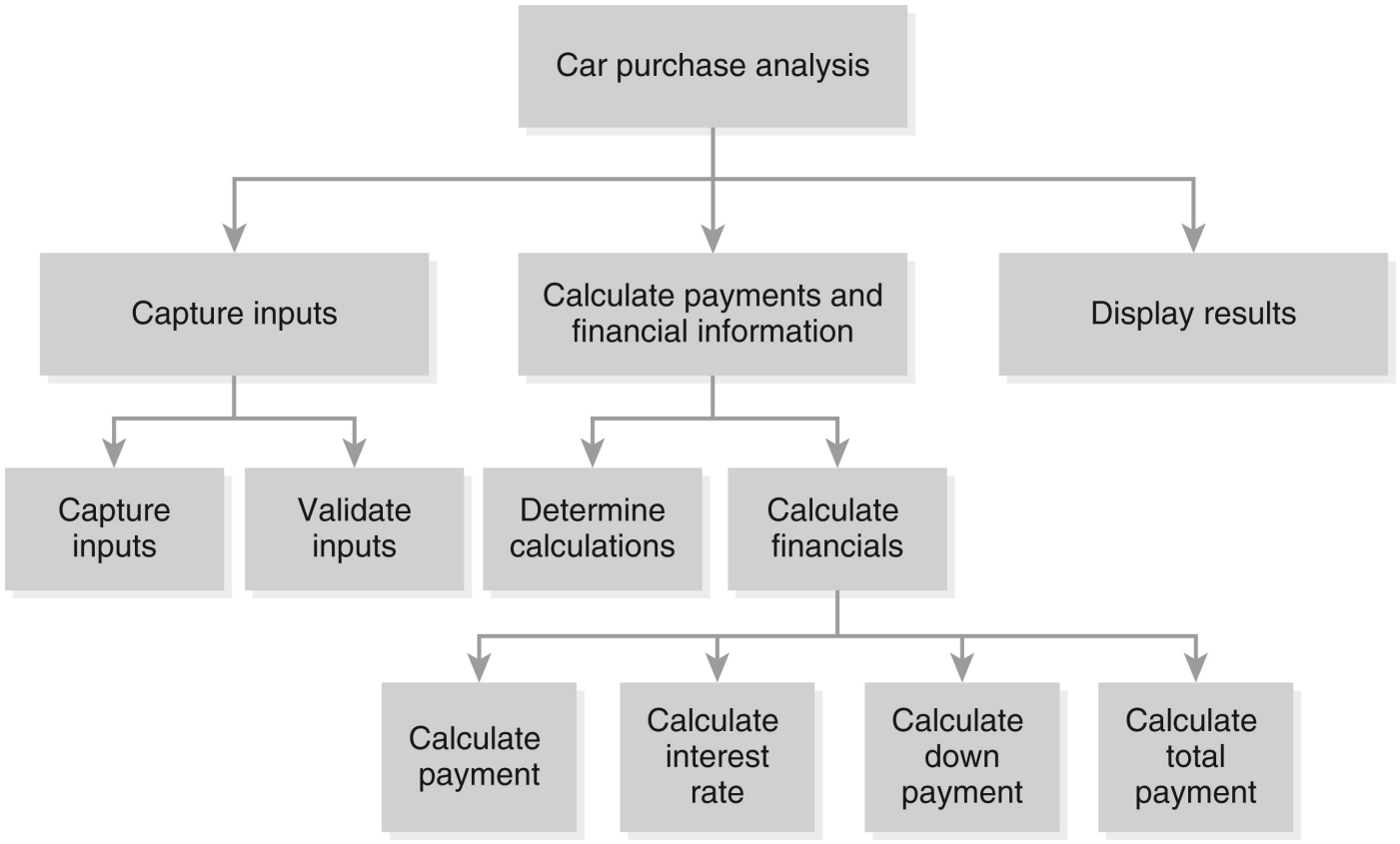


FIGURE 10.3 Structure Chart for Car Purchase Analysis Program

Description

This structure chart is for a small program that takes inputs, such as the cost of a car, a down payment, an interest rate (or a target payment), and a length of the loan. Based on that data, the program would calculate either the payment amount or an interest rate and down payment needed to meet a desired monthly payment. After making the needed calculations, the program displays the results. While a simple example, this helps to illustrate how the model develops the requirements into coherent blocks that organize the program.

E-R Diagram

Entity–relationship (E-R) diagrams are commonly used for modeling data and how the data are structured. In an E-R diagram, an entity is a block containing a set of like data sets and attributes between blocks. A data set includes at least one attribute that must be complete and as many other attributes as are needed. As an example, an entity may be “cars,” where the included attributes are “make,” “model,” “year,” “VIN,” “color,” “tires,” and “location.”

Generally, a set of primary attributes can uniquely identify each set of data in the entity. The other attributes are secondary attributes. All secondary attributes should have a direct dependence or relationship with the primary attributes. In our example, we would not expect to find an attribute of “tread width” because, while it may be related to the tires, it would not have a direct dependence on anything else about a car. Nor would an attribute such as “road sign.” If there is no unique primary attribute, a group of attributes maybe can be combined to create a key, or the designer can add a unique key, such as an ID number.

Each entity is drawn as a block. A label for the entity is placed above, or inside, the block. Internally, the block is divided with the primary attributes listed in bold above a short line. The secondary attributes are listed below the line. Each attribute may be listed with its storage type. Examples of attributes would be integer, float, text, char[60], etc.

Lines are drawn between entities to indicate relationships. Relations are connections between one entity and another. Relationships can be of many different types. Some common relations are “contains,” “is a type of,” or “uses.” Any relationship word may be appropriate. Relationships can be one-to-one, one-to-many, or many-to-many. A one-to-one relationship is one where for each data set in the first entity, there is one and only one data set in the second entity, and that second entity data set has no other data set in the first entity. An example of a one-to-one relationship would be between a person and a set of fingerprints. Each person has a unique set of fingerprints, and no two people share the same fingerprints.

A one-to-many relationship is where a data set in an entity may have multiple data sets in a related entity that are specifically related. Those data sets in the second entity would not have relationships to any other data sets in the first entity. A typical one-to-many entity would be between a car and its tires. A car will have four (except for some older British models) tires. Each of the four tires is only on one car.

Relationships can be drawn from the entity or from the specific attributes of one entity to another. The line is labeled with the relationship. If the line represents a one-to-many relationship, then the end of the line that can be many is marked with a fan out. In a many-to-many relationship, both sides of the line are marked with fan outs. (See **Figure 10.4**.)

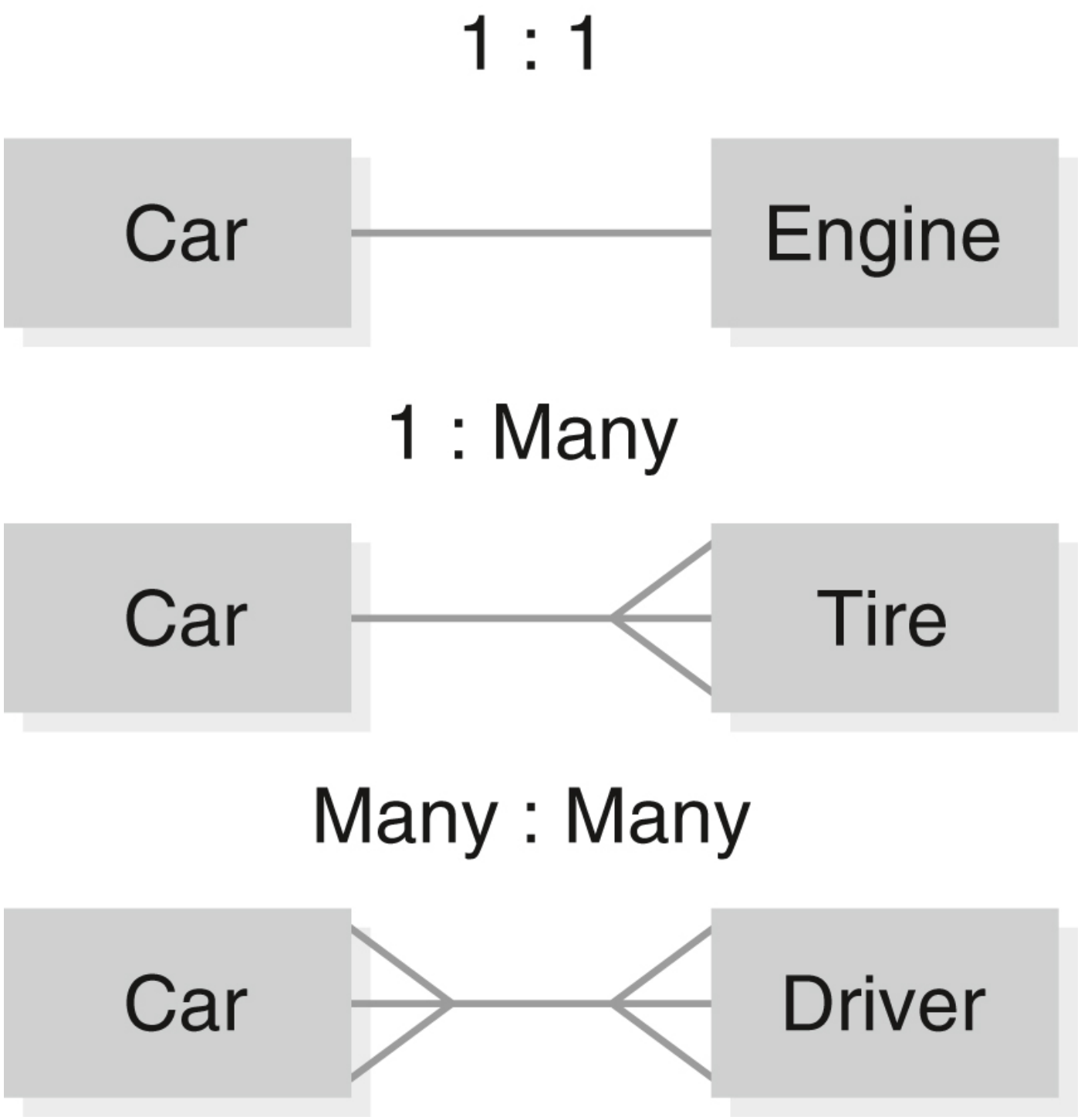


FIGURE 10.4 E-R Relationship Types

Description

The construction of an E-R diagram can be much more strictly formalized with the inclusion of additional symbols and differentiation of the styling for the labels and lines. For most uses, this is not needed. The degree of formality and depth of construction of the diagram will depend on policies in place. In

everyday use, the purpose of the diagram is well served with the simplest construction that helps to capture and convey information.

In constructing an E-R diagram, the designer needs to take an in-depth look at the data that will be used in a system. The specific types of data get grouped into sets of data that appear to be related. The grouping may need to change multiple times as the designer considers the problem. These groups start to form the basis for the entities. The designer identifies potential relationships between the groups. When these relationships make sense, the model is on the right path. However, if the relationship seems forced, then it is likely that some of the data are not grouped appropriately.

Once the designer is satisfied that the groupings are generally good, then it is time to look at different ways that pieces of data can, will, and are likely to be used. For this, the designer looks to see if the relationships can easily linked through to make pieces of data fit together. For example, when working with cars, there would appear to be a natural need to be able to link a driver with a car, or the owner of a car with the tires (as purchaser), but it is unlikely that there is a need to link the driver to the tires. In reviewing the model, the designer needs to check that making the first link is relatively straight forward. The second link could be ignored. If a link that is needed is not straight forward, then there are three possible solutions: add a relationship, add an attribute to the model, or change how the data are grouped into entities. Once designers are satisfied that a model fulfills all their needs, they can “lock down” the model and formalize it to the extent needed.

A completed E-R diagram usually translates almost directly into a database model. Each of the entities becomes a table. The relationships between the entities can be created using SQL JOIN statements or may be built into the database management system. For many-to-many relationships, a helper table needs to be added to map the bidirectional relationships.

The E-R diagram is also useful in determining data structures to be used in a program as well as helping to identify some needed functions. It can also be used in helping to understand how to divide functionality among different components of a system. The grouping can be done to help maintain all manipulations of an entity in the same component. **Figure 10.5** is an example of an E-R diagram.

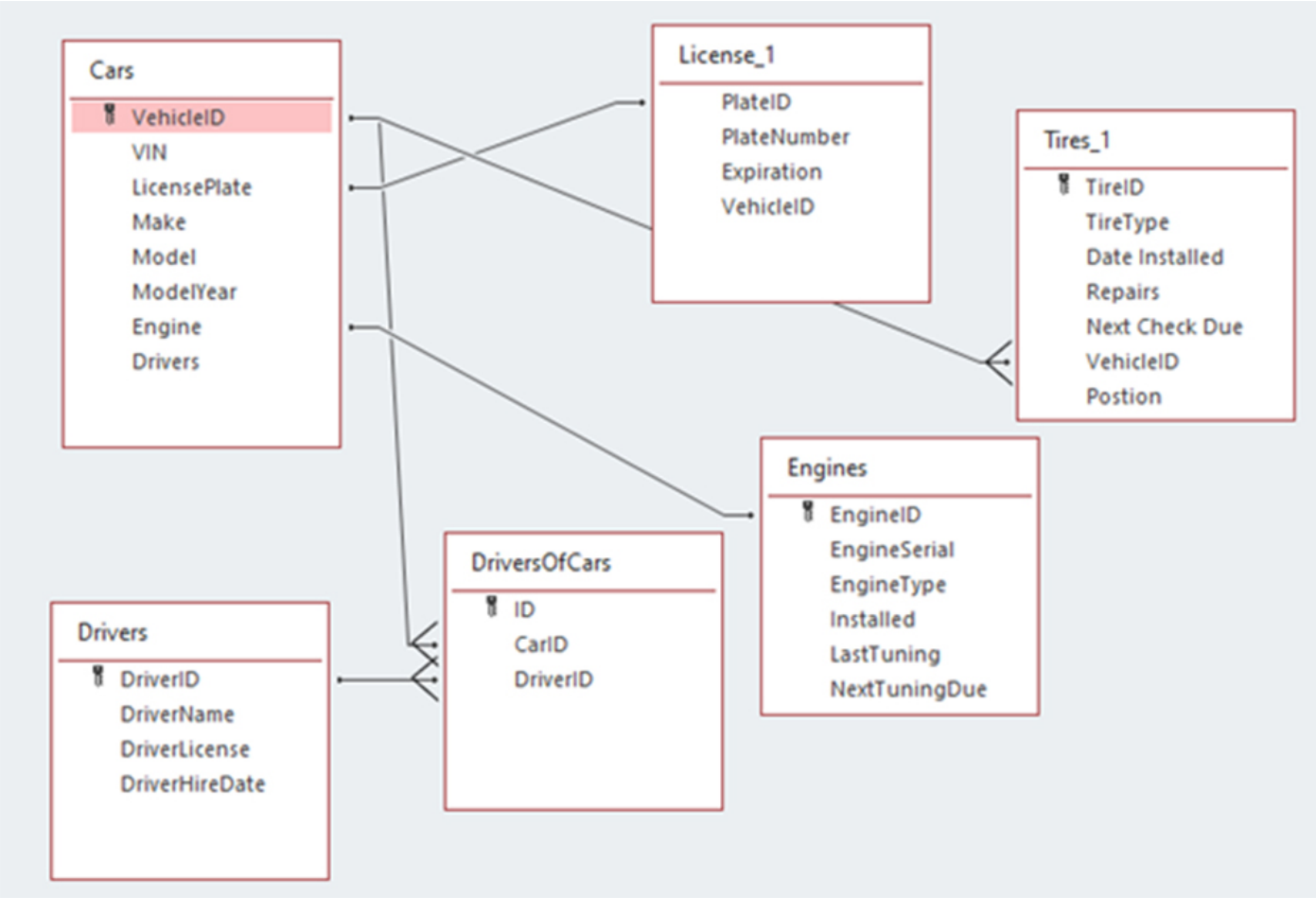


FIGURE 10.5 Example E-R Diagram

Description

The example was generated using Microsoft Access®. This diagram is used for information about cars in an organization that has multiple cars and drivers. The organization will use the database to track which drivers are assigned to cars as well as to track maintenance and other information about cars. Note the 1:1 relationships from cars to licenses and engines; each car has only one of these at a time. However, each car has four tires (five including the spare), so there is a one-car to many-tires relationship. The relationship of many-drivers to many-cars is built using a helper table, which has one-driver to many-driversOfCars and a one-car to many-driversOfCars relationships. In the helper table, each car can be paired with as many drivers as needed.

Object Model

An object model is in many ways similar to an E-R diagram, although some purists will disagree with that statement. The first major difference in the object

model is that rather than starting by grouping together pieces of data to create entities, in an object model we try first to identify objects.

Objects are physical or abstract things that have properties and can act or be acted upon. Identifying what “things” are can be a real challenge at times. Physical things tend to be easier. Physical things can be a car, a tire, a fish, or anything else you can touch. There is still a problem though with things in deciding what level of an object is appropriate. Is the car the right level for the problem we are addressing, or do we need to break it into smaller component objects, and if so, are they the engine, tires, and body? Or do we need to be concerned about the passenger compartment, trunk, and glovebox? Often, it will take some working with the model and manipulation as well as a deep understanding of the problem to determine which is best.

Moving beyond physical objects, we can create abstract objects, such as a telephone call, a video download, or a feeling. Just as with determining the proper layer of abstraction for physical objects, identifying what fundamental objects are inherent in the problem we are trying to solve is a challenge. Some of the objects we create will be container objects. These are objects that gather or contain other objects. Sometimes a container object represents a physical or abstract object; for example, a car could be a container of the parts of a car. On the other hand, there could be a container that had car parts but no other information and perhaps no need to have all the parts needed for a car to run.

Objects have attributes. Just like the data sets in the E-R diagram, the attributes for an object all need to be related to the object. It is helpful to think of the attributes of an object as the description. For example, the attributes could be the color, size, and top speed of a car. Some of the attributes will be other objects. This is how objects build relationships to each other. A car object may have an attribute of location, which is an object that has an address, coordinates, or other information related to the location. An attribute can also be a list. For example, it could be a list of pairs of locations and time. That could be used to give a travel history for the car.

Objects also have a special relationship created by inheritance. This occurs when a new object type is created to enhance, extend, or alter an existing object type. An example of this occurring would be objects created for animals. A base object would describe the core features that all animals possess, for example, size, movement, and cells. A new set of objects could then be created for different classes of animals, for example, fish, mammal, and bird. These objects would all share the core features of animals but would have their own special features unique to the class of animals. A further level of

inheritance could extend the mammal as dog, cat, and platypus. These new classes would have their own unique attributes and, in the case of platypus, may actually remove or override some common attributes of mammals.

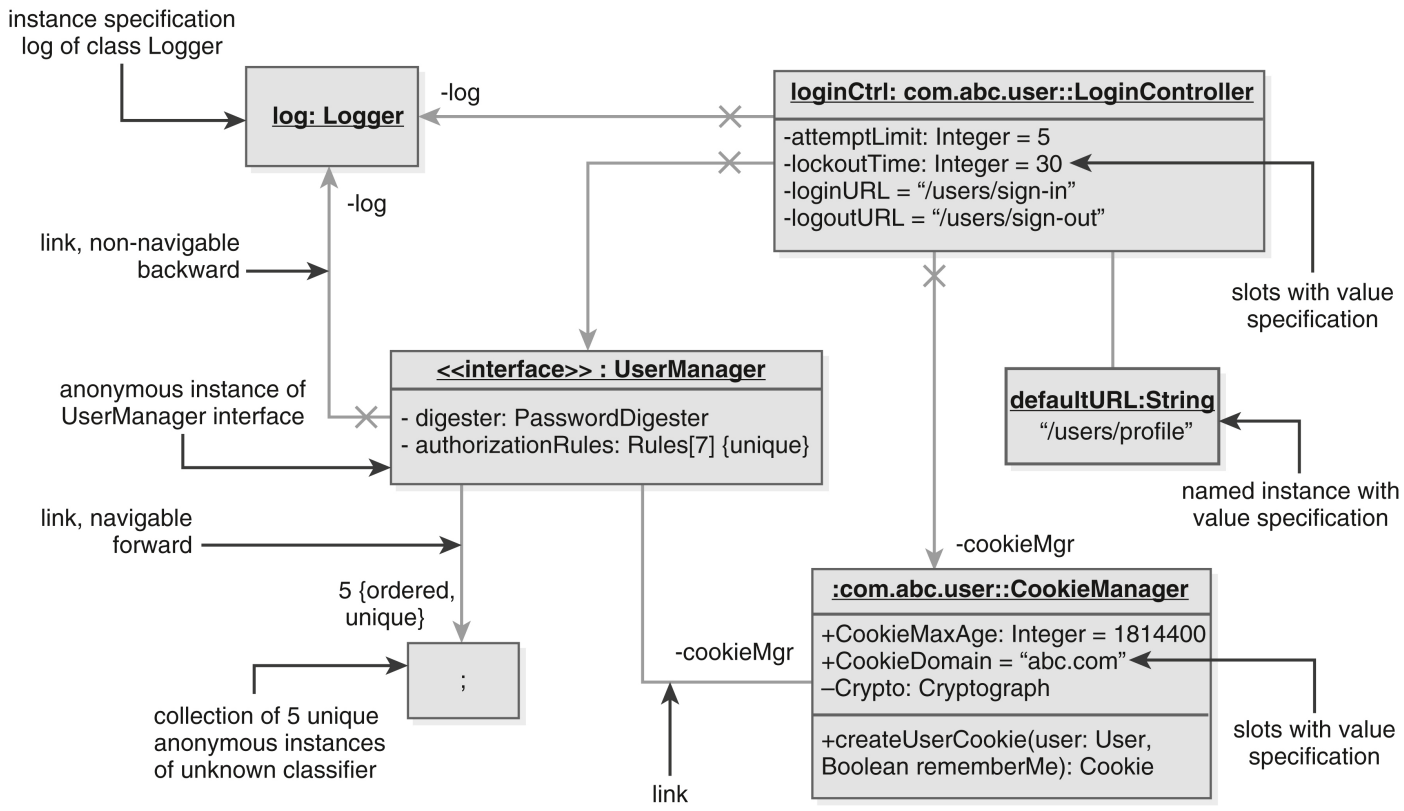
In addition to their attributes, objects have methods. Methods are functions that perform actions on or with the objects. These could be simple, such as providing the value of an attribute of the object. The methods could also be of more complex nature, such as determining the future value of a monetary value maintained by the object. The methods can be private or public. Public methods are accessed externally. Private methods are used internally within the object.

In theory, there could be one set of objects for everything in the world, and the objects are reused any time they are needed in a software system. The challenge is that this would become very inefficient because objects would have attributes not needed in a specific system or the system would have excess public methods. Using existing objects can also be risky from a security perspective. If the object has been built elsewhere, there is a risk that is unsecure either by intent or complacency and disregard for potential issues. Instead, the designer will either describe or select the specific objects appropriate for a system.

An object model is built using one of many different methods to graphically show the objects, their methods and attributes, and the relationships between objects. Commonly, inheritance and other relationships are not shown on the same diagram, but sometimes they are.

Most formally, object modeling is done using UML. This language has been adopted as a standard by the International Organization for Standardization (ISO). UML provides standards for symbology to be used in modeling as well as styles for different types of diagrams. The language is known for being exceedingly complex. Most designers use a selected portion of the language and often do so in their own style as well, while keeping that style close enough to UML to allow general recognition of their intent. **Figure 10.6** is an example of a UML object model diagram.

Figure 10.6 is annotated to show different elements of the UML specification and how they are used in a diagram. As can be seen, the UML can become complex and challenging for a novice to understand. Extracting and applying only those items relevant to the project is encouraged. Over time, users will develop more expertise as well as their own style.



Object diagram overview - instance specifications, value specifications, slots, and links.

FIGURE 10.6 Object Diagram Overview

Description

Use Case Model

Use case modeling is used to model interactions between users and a system. The model is used to document the actors in a system and the cases in which they will use the system. The use-case model is valuable in identifying what interfaces are needed to enable an actor to work with the system as well as to group the controls on an interface and their support in the code to best support the expected uses of the system. An example of a use-case model is given in **Figure 10.7**.

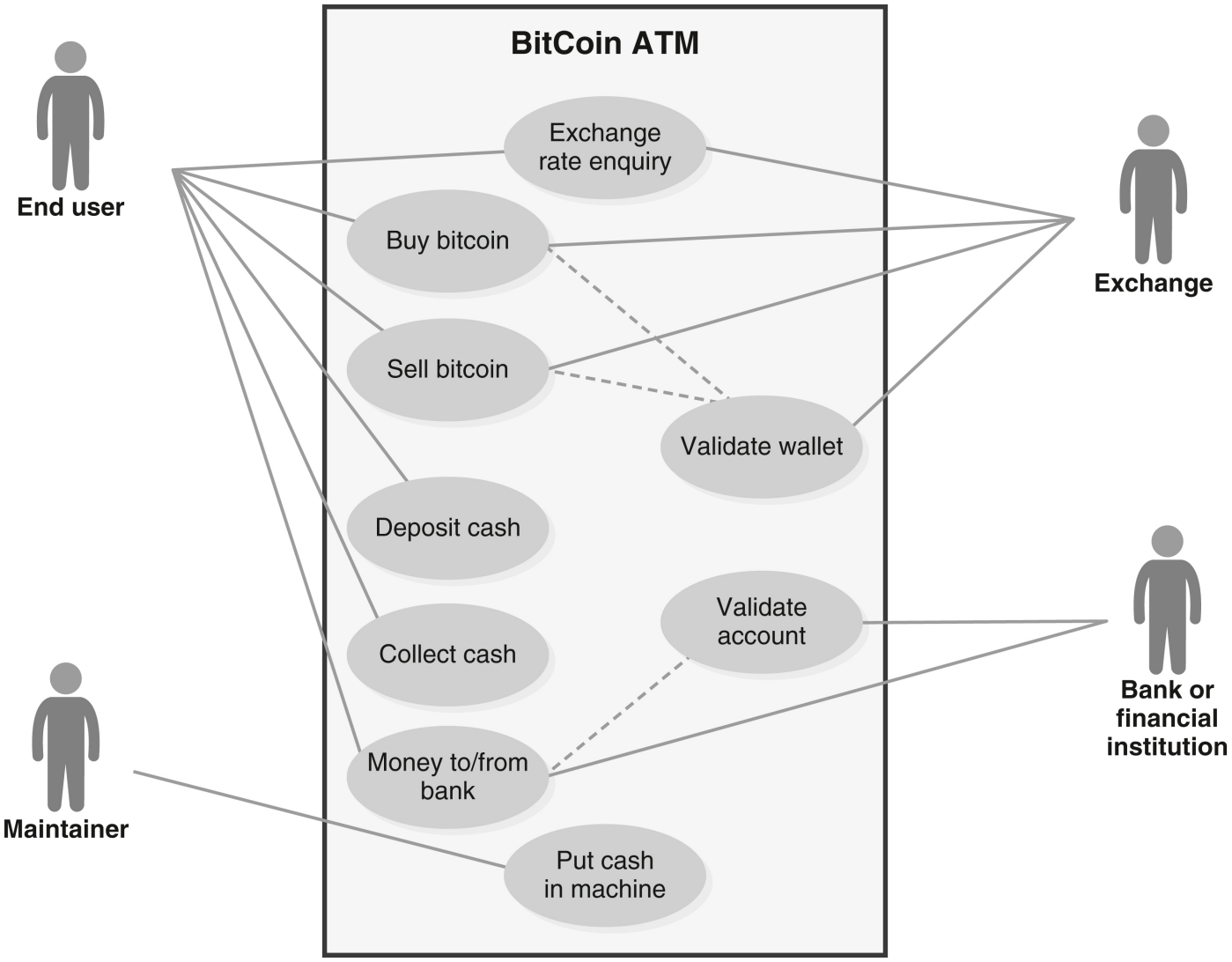


FIGURE 10.7 Use Case Model of a BitCoin ATM

Description

This model presents use-cases for a BitCoin ATM. The box is the boundary of the system. For this system, four different actors have been identified. There is an end user, someone who wishes to buy or sell BitCoin. The end user may also make inquiries about the current exchange rate for BitCoin, deposit or withdraw cash from the machine, or move money in and out of an account with a financial institution.

In the cases that involve BitCoin, the ATM will need to interact with another actor, the BitCoin Exchange, to make an inquiry about the current rate of exchange, and in the case of a sale or purchase, the ATM will interact with the exchange both to validate the user and to complete the transaction. Notice that the Deposit and Collect Cash cases don't involve any actors other than the end user. If the user deposits cash, the cash simply sits in the machine until the

user uses it to buy BitCoin, moves it to a bank or financial institution, or simply collects it back from the machine. The bank or financial institution is an actor who interacts with the ATM both to validate users and to complete financial transactions. Cash withdrawn from the bank or financial institution and cash from the sale of BitCoin, as with cash deposited, remains in the ATM until the end user completes a case to do something with it. The final actor is the maintainer who needs to be able to periodically add cash to the machine (and possibly withdraw it) to ensure cash is available for withdrawal.

The diagram is fairly simple, but useful in working through how the system should interact with different actors. It can also be used to imagine different scenarios, such as examining which cases would be needed to support working in different currencies, both specie and crypto.

Sequence Diagram

A **sequence diagram** is used to track the order of synchronized interactions among elements of a system. Each portion of the system is represented as its own vertical line, labeled with the name of the element. Arrows are used to show messages passed from each element to others. The ordering of the elements is important. If an arrow appears below another arrow, that message can only be in response to the previous message, unless annotated otherwise. Often, but not mandatory, the period when an element is active is shown as a broad section of the vertical line. **Figure 10.8** is an example of a sequence diagram.

In cases where different sequences can be followed by the same elements, they are shown on a page (or multiple pages) with breaks in the vertical lines. **Figure 10.9** shows the gap between sequences where A, the element continues as active, and B, the element terminates and becomes active with a new sequence.

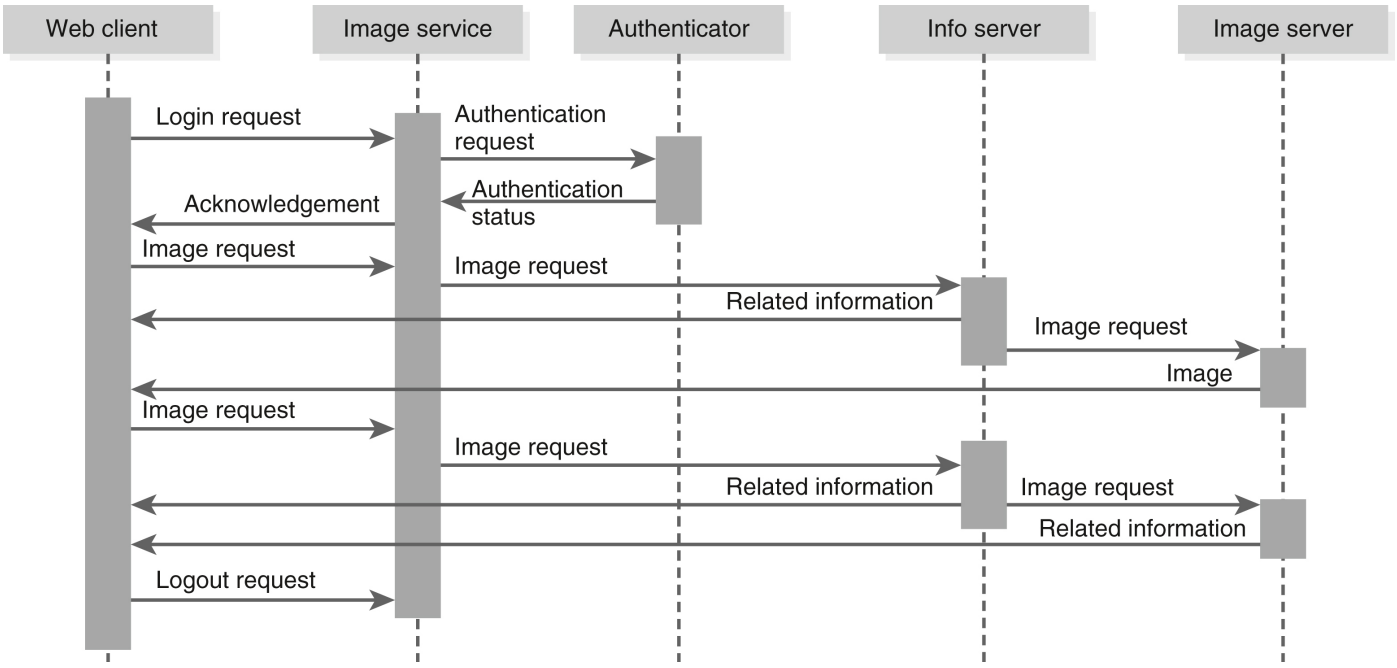


FIGURE 10.8 Sequence Diagram Example for a Web-Based Image System

Description

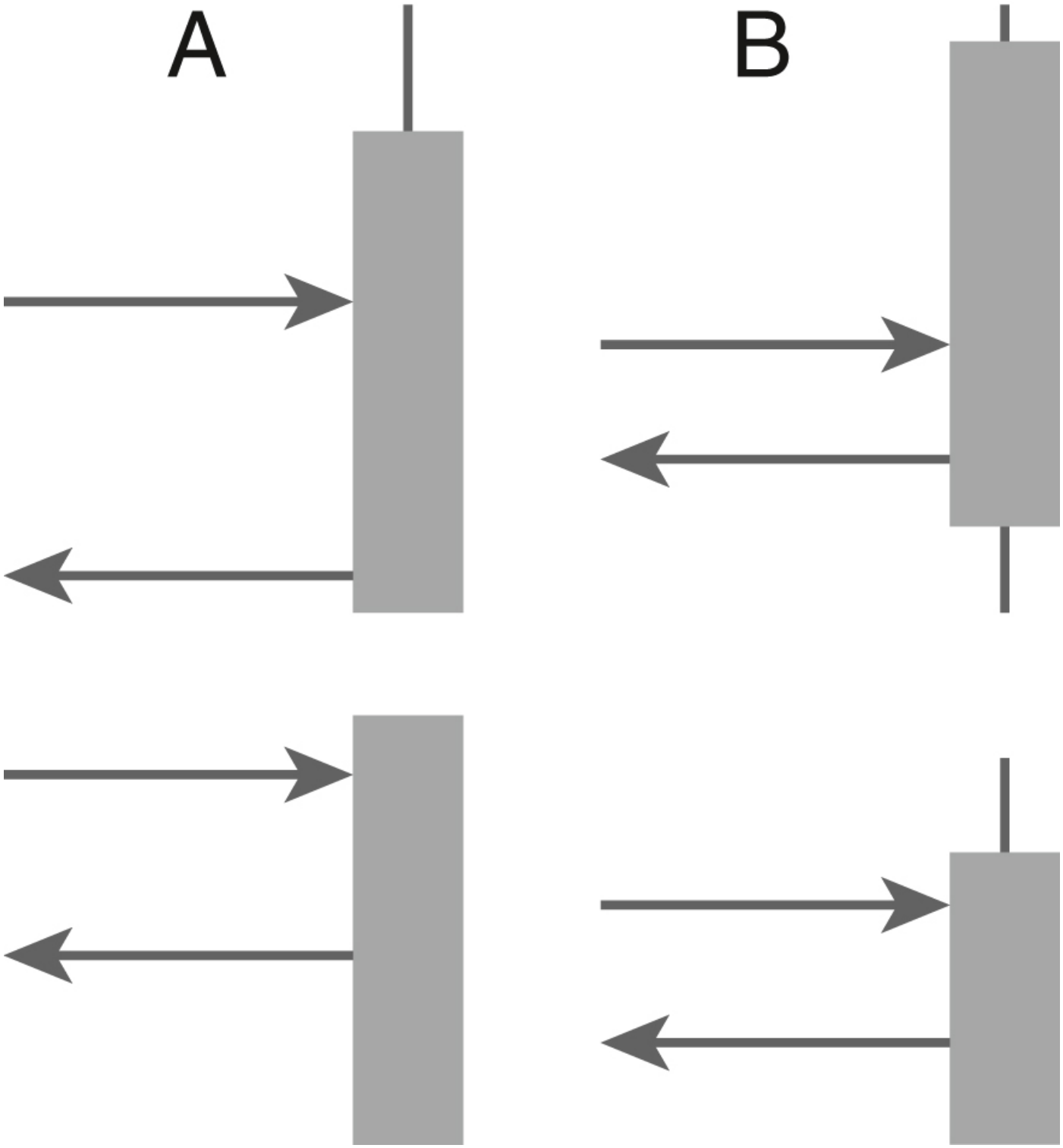


FIGURE 10.9 Detail with Gap in Sequences

Description

The example in Figure 10.8 demonstrates an image delivery service where each image is delivered with a significant amount of information contained on a different server from where the image itself is maintained. Perhaps this information server is a database with information about how often the image

has been accessed, or coordinates for significant features on the image. In this diagram, the web client first logs into the image service. The service uses an authentication system to verify the identity of the user. Once a user is authenticated, the image service provides an acknowledgement and is ready to receive requests for images from the user.

As each request is received from the web client, the service passes the request first to the information server, which provides the information to the web client, and then in turn passes the request to the image server. The image server then provides the image to the web client. Note that these last two processes are only active while the request is being handled. The image service process remains active throughout the whole session, perhaps providing information about what images are currently available or performing other processing. Once the image service receives a logout request from the web browser, it terminates.

As with any of the other modeling tools, sequence diagrams can be used either formally or casually as is applicable in the situation. The more formal application is generally used when communicating precise directions for how messaging should be sequenced. The more casual application is often used in developing concepts and creating general guides.

State Diagram

State diagrams are a useful modeling tool for understanding and documenting process-based computing and other computing processes where there are distinct sets of values or states that are connected through pathways that are traversed based on specific conditions.

The state diagram begins with a collection of states, or fixed conditions in which the system may appear. Associated with each state are outputs related to the system being in that state, variables that provide the state, or other actions or activities that occur when the system is in that state.

The state diagram also shows transitions from one state to another. The diagram should show every possible condition from each state to another, including when it returns to itself. The conditions for transition are often written as logic equations but may also be expressed in other ways. If multiple transitions are possible from any one state, then the decision of which transition to take under any condition must be clear and unambiguous.

In more formal state diagrams, the transition equations must cover all possible combinations of terms without overlap. As with other models, the formality and coverage of the model is adjusted to the goals of what the model

is trying to communicate. **Figure 10.10** provides an example of a state diagram.

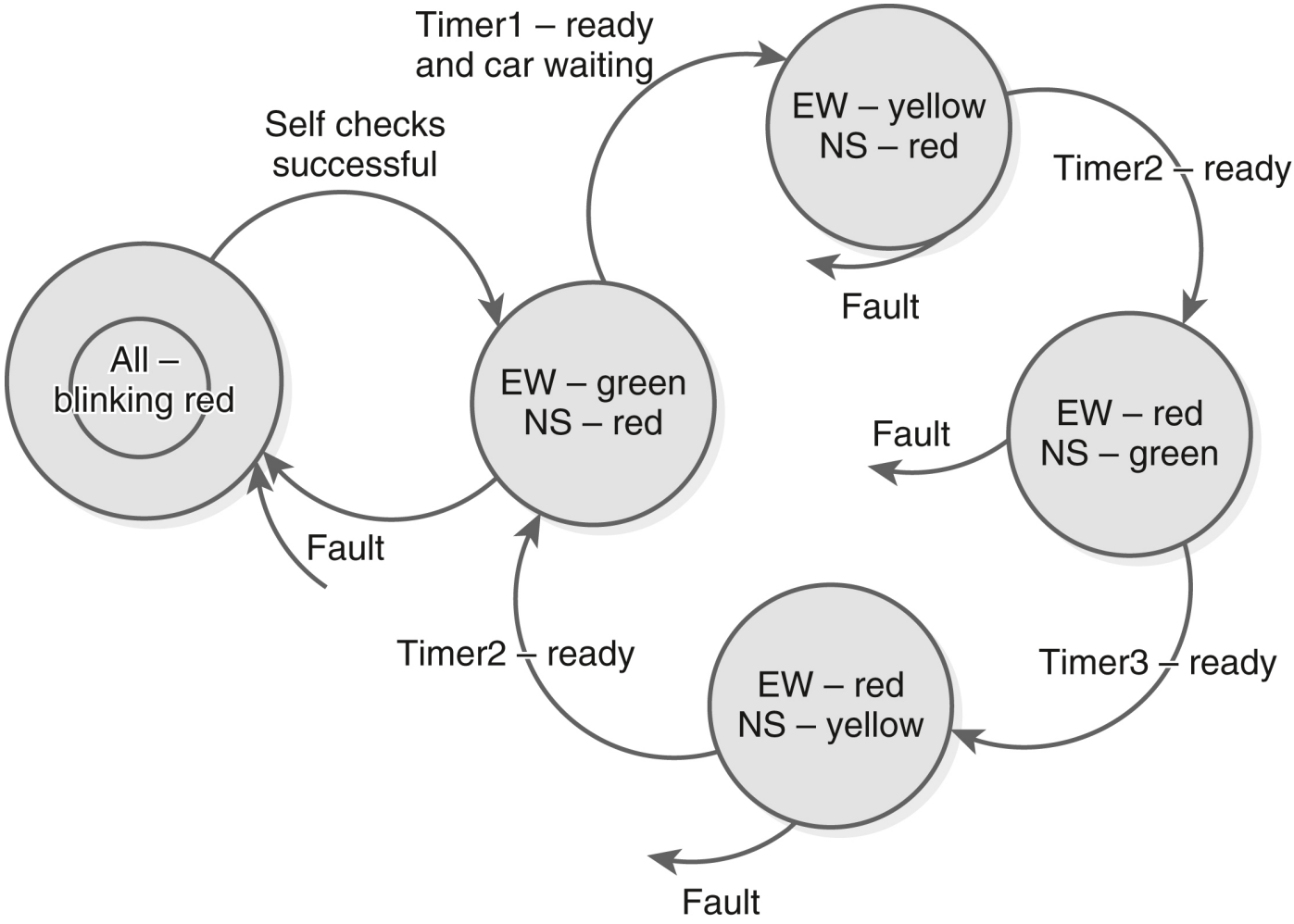


FIGURE 10.10 State Diagram for a Simple Traffic Light Controller

Description

The system being modeled in this diagram is a simple traffic light controller. The controller is expected to be used on a basic intersection where a busy road is crossed by a less frequently used side street. The initial state on the diagram is represented by a double circle. Because it is the initial state in a critical system, the system needs to be built in a manner to ensure that this is always the first state the system enters. In software, this would be done by setting the initial values. If the system were initiated in hardware, the hardware would be designed to ensure this happens. In fact, for a traffic light controller, it is likely the system would be designed in a way that the hardware always initiated the system into this state, and the software only took control of the system once the software had also ensured it was also in the initial state.

For the traffic light controller, the initial state will also be the state used whenever the system detects an error. Note that in this initial state, all traffic light faces are in a blinking red mode. That ensures that if the system is not in control, all traffic treats the light as a four-way stop.

While in the initial state, the traffic light controller will complete a number of self-tests. This may include checks of all of the bulbs to ensure they are not burnt out, checks of the relays, and any other tests deemed appropriate. The transition out of the initial state is completed when all self-tests have been completed successfully.

The first state the traffic light enters is allowing traffic to flow on the east-west (EW) or primary road. This state is defined by having the EW lights green and the north-south (NS) lights red. The light will remain in this state for a minimum amount of time determined by Timer1. At some point after that time, if there is a car waiting on the NS road, the controller will transition to the next state. In that state, the EW lights turn yellow. The NS lights remain red.

The time of the yellow light is determined by Timer2. Once the timer is ready (completed), the controller transitions to the next state where EW lights are red, and NS lights are green. It transitions again after a period determined by Timer3. This will be a different period of time than Timer1 because not as many vehicles need to move through the intersection on the NS road. The period for all yellow lights is the same, so the next transition is also determined by Timer2. At this point, the controller has returned to the second state. From here it will continue to make the transitions as described, until a fault occurs, or the system is shut down.

Sometimes states are given names or identifiers to make them easier to describe. For example, here the states could have been named: Init, EWG, EWY, NSG, NSY. Or they could have been numbered: State0, State1, State2, State3, State4.

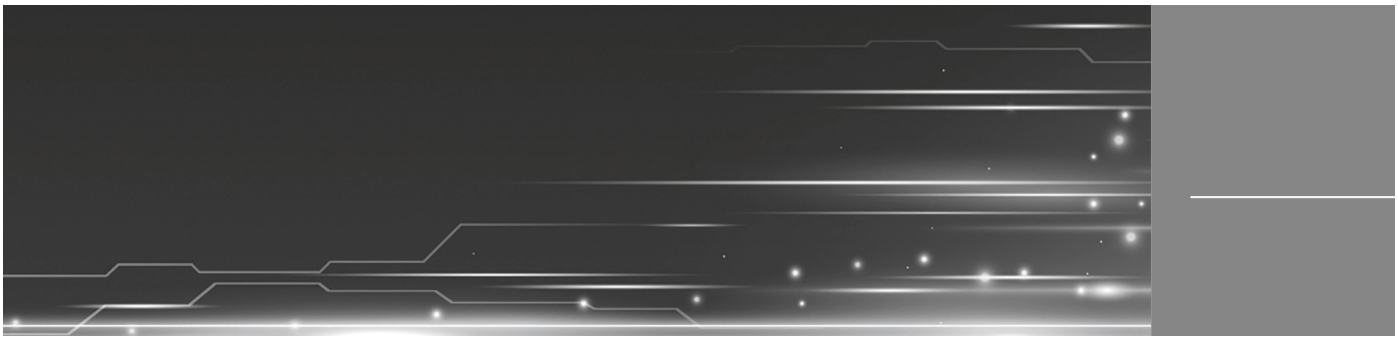
SUMMARY

Applying modeling to a problem is a valuable tool both in helping to understand the problem and in developing a solution and communicating that solution to others. Because people are visually oriented, models can communicate ideas and facilitate understanding in ways that are difficult, if not impossible to achieve through other means. Sometimes developers shy away from the use of models because of the complexity that can be involved in developing models in a formal manner. This is unfortunate because not all applications of models require a high level of formality, and even when some formality is desired, it need not be excessive. The goal is to clearly communicate the concept, not to strictly follow a formula.

Another challenge with modeling is choosing what should be modeled and when. Here again, the goal is to achieve usefulness. A quick model can clarify much or link together often disparate concepts. Finally, it is valuable to choose the modeling technique that is appropriate for the problem or domain being addressed. Often no single model will suffice, but rather a combination of models work together, each adding its own value. There are many different models to choose from, some of which have been discussed here. There are many other types of models as well, and different techniques can be combined together in a single model.

KEY TERMS

entity–relationship (E-R) diagrams
flow chart
modeling
modeling techniques
modeling tools
process flow diagrams
sequence diagram
state diagrams
structure chart
use case modeling



© d1sk/Shutterstock

CHAPTER 11
Architecture

CHAPTER OBJECTIVES

- Outline common architectural concerns, principles, and security objectives that drive system architectures.
- Describe how system architecture contributes to system security.
- Review how design patterns are applied in developing an architecture.

Introduction

The architecture or high-level design of a system can play a significant role in determining the security, reliability, quality, and usability of a system. Unfortunately, many systems are crippled from their beginnings with a poor design that serves more as an obstacle to easy secure development than a help. Sometimes this is because people do not stop to think about what the architecture of a system should look like. They simply begin development or use the same basic architecture they use for every other system. Other times, there is a rush to skip the consideration of the architecture because “this is an Agile development.” That is a poor excuse. Nowhere do Agile practices preclude selecting or creating a sound architecture. In fact, the whole Agile concept is based on beginning on a solid architectural foundation. Then we can start the rapid delivery of components developed in close consultation with the end users.

An architecture is the high-level plan of how all the functional components of a system will cooperate to produce the desired result. An architecture does not look at the details, although they are often considered in developing an architecture, but rather focuses on the big picture of workflows, connections, assignment of function, and other considerations. When planning a system’s architecture, security is included so that the architecture does not have to be later changed to make sure it will meet all the security requirements. Security requirements such as how to design a database to keep information secure and what does and does not need to be encrypted can affect which type of database to select as part of the architecture. There may be a need for the system to be both secure and constantly available, which will drive a decision on the type of hardware, how much hardware, and necessary configuration of the hardware.

Architecture Views or Models

One of the challenges with architecture is that many things need to be considered that it becomes hard to view everything from all perspectives at once. Trying to capture everything into a single drawing or plan creates a chaotic, cramped document that loses focus. **Architectural views** are used to focus attention on key elements of the architecture, often as seen from different perspectives. We see the same thing in building architecture. No single drawing shows everything. External perspectives show the outside of a house; floor plans show aspects of the interior; wiring and utility plans provide yet other views. The same approach is use for software systems design. In creating the architecture, there will be multiple views or models, each focusing on a different aspect of the system. The different views will cross reference each other to show where aspects of the architecture from another view are reflected on the current view.

Business or Functional Architecture

The **business or functional architecture** provides the view of how the system is put together and functions from the point of view of the users. This view focuses on the functional modules, the implemented processes, and the points of interaction between the users and the system. The business or functional architecture is also concerned with the big picture aspects of how the system with interact with other systems.

In developing the business or functional architecture, key requirements to consider are the functional aspects, the business goals and objectives, and how the system will be used. This view of the architecture will group the functionality into logical units from a user perspective. Security considerations in this view include considering issues such as separation of duties, user roles and capabilities, as well as other functional security needs. **Figure 11.1** provides a business architecture example.

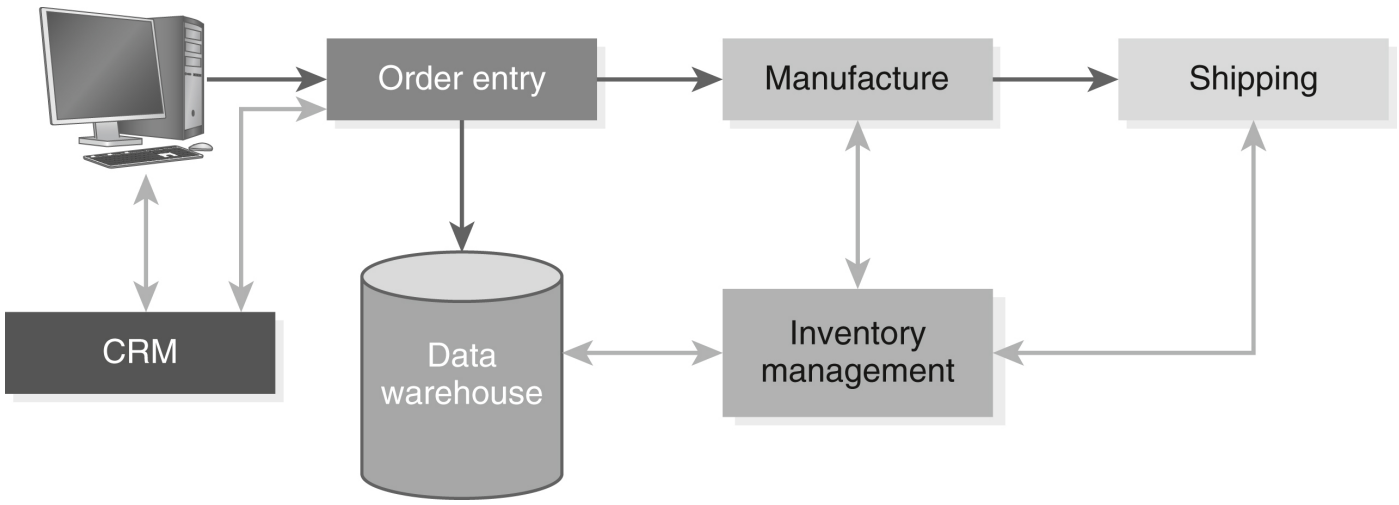


FIGURE 11.1 Business Architecture Showing Relationships among Systems

Description

This architecture identifies the systems used in accepting and delivering an order from a customer. In this view, the systems involved are identified, as are the interfaces between the systems. The details of the systems and their interfaces are not relevant for understanding, at a high level, the flow of data from one system to another.

Logical Architecture

Logical architecture differs from the functional architecture in that it is going to group the implementation requirements together into where the functions will be performed rather than how they are used. The logical architecture is usually more detailed than the functional architecture. In addition, it will consider more of the nonfunctional requirements. **Figure 11.2** provides an example of a logical architecture for a water data collection buoy.

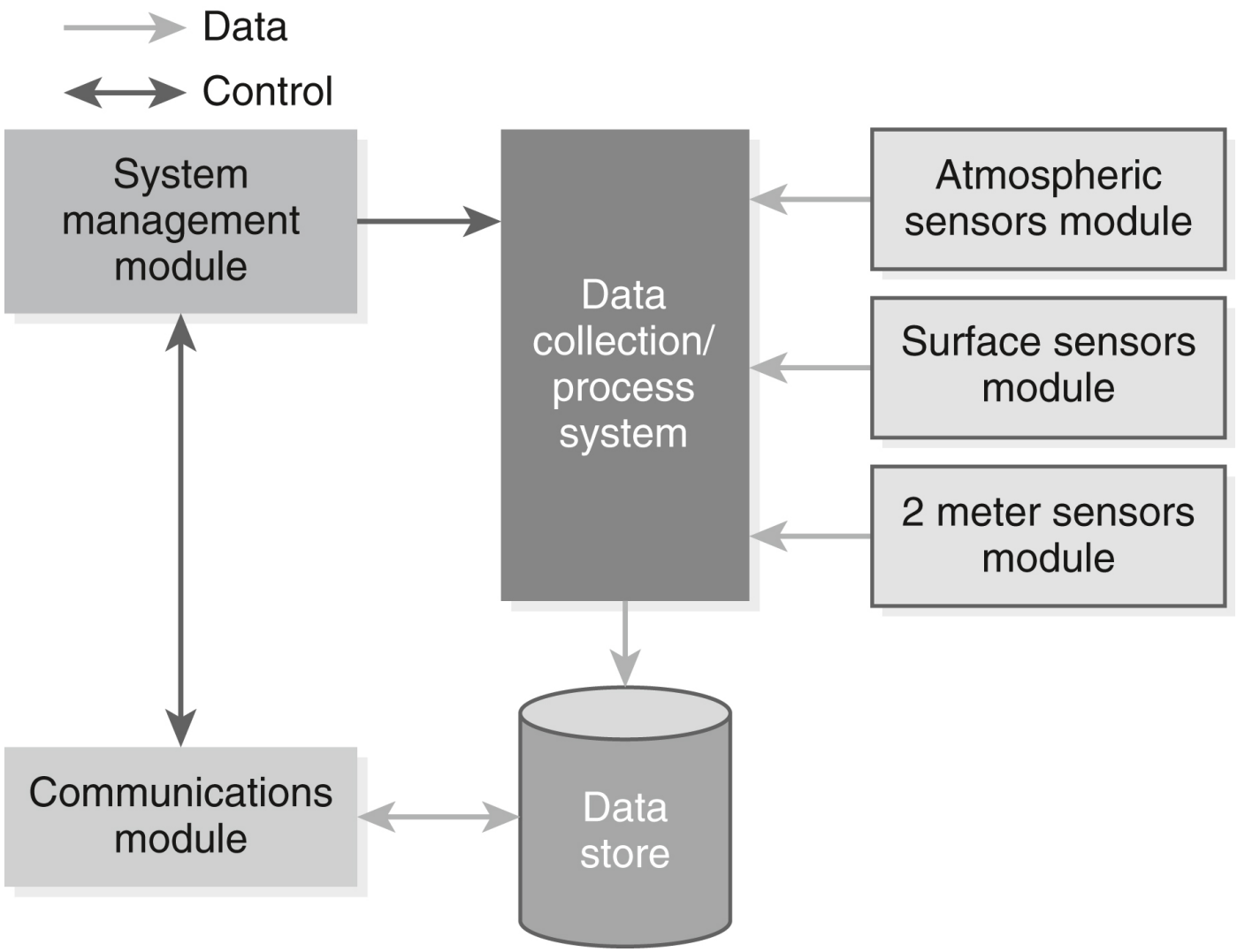


FIGURE 11.2 Logical Architecture for a Water Data Collection Buoy

Description

The buoys are used to gather data including temperature, salinity, opacity, insolation, and other factors. The buoys are moored in a specific location and visited periodically to have the data collected wirelessly. The data from multiple buoys can be used to evaluate conditions across a body of water.

Application Architecture

An **application architecture** is not always needed but is used when the system will be broken apart into distinct applications, some of which may be already developed or purchased separately. The application architecture defines the functionality that is provided by each of the component applications. It also specifies the communications pathways and what each application is responsible for providing to other applications.

The application view of the system would be very different from the business view, where web servers, databases, and other such background applications do not exist. An example of a software system that requires an application architecture is an ecommerce system. The system is going to have multiple application elements including web browsers, web servers, databases, and perhaps multiple applications implemented as applications accessed from the web server, some of which may be developed in house, such as a catalog manager, and some of which may be external, such as a credit card processor. **Figure 11.3** gives an application architecture of the overall system used with the water data collection buoys.

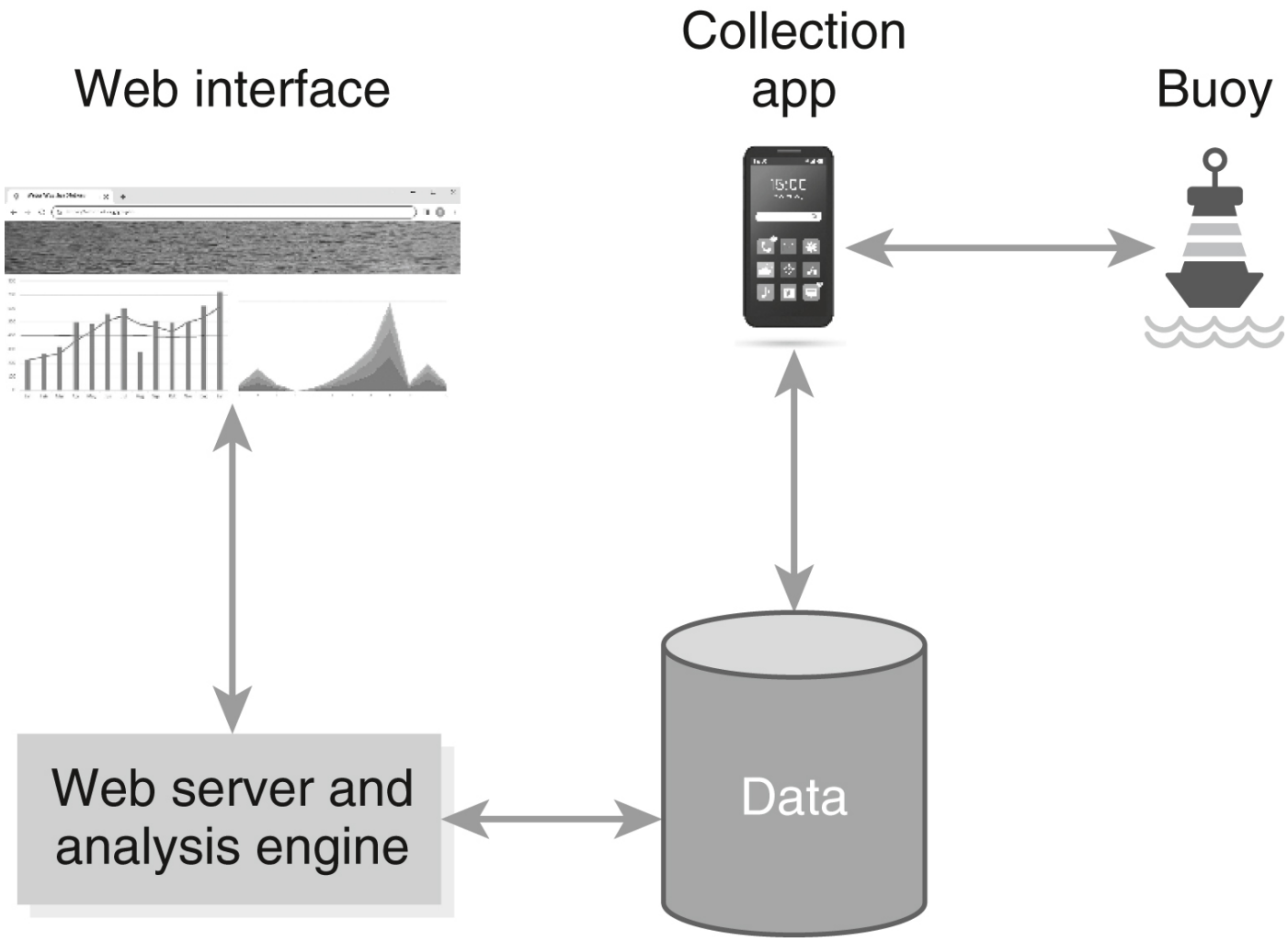


FIGURE 11.3 Application Architecture of the Water Data Collection System

© Pogorelova Olga/Shutterstock

Description

The applications given in this system include the buoy for which we saw the logical architecture; an Android application used to configure, collect data from

the buoys, and report the data to a database; the database; a server that performs graphing and other analysis while functioning as a web server; and the web client used to view data and manage the system. Just as there is a logical architecture for the buoy, there would also be one for the collection app and the analysis engine.

Data Architecture

Data architecture looks at the software system from the point of view of the data. It considers where the data are created, used, and stored. The data architecture considers the format and structure of the data. If there will be a database, the schema of the database is a portion of the data architecture. If the data are stored or used as an object model, the object model is included in the data architecture.

Designing the data architecture can require a different skillset than designing the software system that is going to use the data. The data architecture is going to be based on principles of information management. Information principles are concerned with the quality, accessibility, and utility of information. Basic principles include that every piece of data needs to have owners, custodians, and users; to have an authoritative source; to be actively verified for accuracy; to align with the business strategy (or have a business benefit); and to be appropriately protected.

Data owners are usually the business unit with the most interest in the data. For example, personnel data are owned by human resources, customer data are owned by marketing, and designs are owned by engineering. The data owner has the ultimate responsibility for making decisions about the data, such as how long it is kept, what level of protection the data requires, and who will have access to the data. The owners get to make the rules about the data.

Data custodians are those who store or manipulate data on behalf of the owners. Information technology is commonly a data custodian, but other groups in the organization are as well. Custodians have the responsibility to follow the rules set by the owners, to consult with the owners on issues that will impact the data, and to exercise due care in acting on behalf of the data owners.

The users of data are anyone who inputs or views the data. They also have the responsibility to follow the rules as established by the data owners. For example, HR users, even though they have access, would not be allowed to look up pay data on other employees without a specific business reason.

Authoritative sources are critical for proper information management. This prevents clashes between systems when each has conflicting versions of data.

Any changes to data should be made in the authoritative source and then propagated to secondary sources.

For example, an order entry system may give address A for a customer, and the customer relationship management (CRM) system may provide address B. If the correct address is the one in the order entry system, then there is a risk that as the order entry system has the address overwritten with the wrong address as updates are pushed out from the CRM system on a periodic basis. The correct process would have been to make changes to the address in the CRM system and then have the customer address pulled from that system. It seems to be a given that data accuracy is important, but the principle is often ignored. Accuracy needs to be actively maintained through the validation of incoming data and through ensuring data are unchanged during storage. One of the greatest challenges for accuracy of data is having a common understanding of what data actually are. Consider speed on a boat. If there is a data element called speed, is it the true speed of the boat relative to a fixed point on the surface, the speed of the boat relative to the water (which could also be moving at a particular speed), the apparent speed relative to the speed of another boat, or a speed calculated from GPS? Is the speed instantaneous or average over some time period? All of these different interpretations will give different results if used in navigation.

The principle of alignment requires that all data collected and stored have a specific business purpose and that the format for how that data are stored be aligned with the business purpose. Data are not collected just because they can be. In addition to periodically reviewing what data the organization possesses, the organization needs to evaluate the value of the data and ensure that appropriate protection controls are applied to the data. Data are classified; that is, the value of the data is ranked so that data can be properly protected.

Classifications will vary by organization. As long as the organization is not a governmental organization or does work for the government, no specific classification system is required. Most companies use labels such as “confidential,” “for official use,” “contains personal information,” or similar designations to specify levels of care. Personally, my favorite, which ought to be required of all companies, is “super-duper secretolicious.”

Sometimes data appear to be of little value or risk but when aggregated has significant value. Letting slip that the company bought a thousand pounds of a particular spice one month is insignificant, but when combined with purchases of other spices over time, that data could compromise the company’s secret recipe.

Alignment of data also needs to consider who owns and is the custodian of data. It needs to consider where the data are gathered, stored, and distributed. All of this is done with the business objectives in mind.

Physical and Technical Architectures

Physical or technical (systems, network, etc.) **architectures** provide perspectives of a system from the point of view of how everything is going to work together. These views of the system cover of much of the nonfunctional requirements. They provide the solid basis on which the whole system will be built. Often, people overlook these views as just details; the reality is that without a solid platform, nothing will perform well.

Sometimes it is hard to see how the logical architecture maps onto the physical architecture. In this case it is useful to have an overlay where a drawing of the logical architecture is superimposed over the physical architecture.

Physical and logical network architectures are particularly important for security. Vulnerability mapping on the network architecture can identify issues that would otherwise escape notice.

Enterprise Architecture

Enterprise architecture is the concept of designing an overarching architecture for the information systems in an enterprise. While conceptually, this would be a single architecture that handles all of the information processing needs of the organization, in practice it becomes more of a goal that moves all information systems toward more standardized approaches and greater compatibility.

The enterprise architecture works at multiple levels. First at the business level, it creates a vision of all systems in the organization working as a single system that handles all business requirements. At this level, all business requirements are identified and met through the application of the architecture. As new requirements are devised, they are evaluated as to how they fit within the unified system.

At the application level, the enterprise architecture maps the business functions to specific applications that will be used to implement those specific functions. The goal is to have each function performed by one, and only one, application. The application level will also specify how the applications are linked together to share the data and process the needs of the organization in a consistent and effective manner.

At the networking and hardware levels, the enterprise architecture will specify how everything works together. More importantly though, it will specify the types of systems, equipment, and configurations the enterprise prefers. This will set a standard to ensure interoperability and reduce the number of types and kinds of systems that need to be supported by the organization.

In the ideal, the enterprise architecture is realized as a single integrated set of systems that perform all business requirements of the enterprise effectively and exceptionally well. In reality, despite the claims of some vendor sales representatives, no such system exists or will ever exist. If such a system did exist, it would only prove to be adequate for a very brief period in time as business needs change and newer, better means of performing tasks are found. The other challenge is that the resources needed to obtain such an ideal system are beyond the financial capabilities of most organizations. Finally, there are always going to be specialized systems that perform some key or mission-critical function that the organization cannot live without, is only available in a system not in compliance with the enterprise architecture, and no, aligning the architecture to that key function won't help because then there is

this other key function, and … [of course there is an infinite chain of why things have to be the way they are, all because the organization started with a hodge-podge or systems rather than a well-designed system architecture and a well-managed portfolio].

The inability to ever actually achieve the desired enterprise architecture doesn't mean o simply throwing up one's hands and walking away from the concept. The real strengths of the enterprise architecture are not from achievement, but from the journey to get there. In having the architecture as the goal, systems are aligned and focused as they are designed, purchased, and implemented. Not everything is in perfect alignment, but the numbers and types of systems are reduced, and the overlaps and gaps in capabilities are reduced. Using the architecture, there is a basis for deciding to implement needed new functionality.

The enterprise architecture helps to provide a means for evaluating new systems being acquired by the organization. It is also useful as a tool for evaluating and prioritizing potential new systems, for planning for future needs, and in setting budgets.

A solid enterprise architecture is essential for solid cybersecurity. The consistency provided by following an enterprise architecture avoids having a total jumble of different systems, approaches, and hardware, all of which leads to a cascading anarchy of vulnerabilities. Even when systems fall outside of the architecture, the fact that they do so highlights the need for spending special attention on the unique security aspects of those systems.

Creating the Architecture

Creating the architecture for a system is not always an easy chore. For complicated systems, it can require years of experience, education, and practice to obtain an ideal architecture that is elegant in its simplicity and balances coverage of all concerns. Unfortunately, all too often, the developer by default becomes the architect without the benefits of experience, or worse yet, is put into a position where they need to impose an architecture on an unruly system begun with no strategy in mind. Useful in this situation as well as for extremely experienced architects is the use of design patterns.

As the architecture is developed, many concerns, principles, and security objectives need to be considered. These will affect the ability to accurately and quickly develop the system, and they will have a profound impact on the security of the system.

Architectural Concerns

Architectural concerns are factors that need to be considered for inclusion in a design. Not every concern applies to each system, but consideration of each needs to be done to judge the level of impact on a particular system.

BOX 11.1 Architectural Concerns

- Scalability
- Scope
- Security
- Performance
- Manageability
- Reliability
- Location/Locality
- Coupling
- Internationalization

The first concern is **scalability**. Scalability considers how much usage the system is going to need to be able to handle. Scalability can be measured in a number of different ways. For example, it may be the number of records that need to be maintained in the database. A simple database for tracking the employees of a small company may only need a few thousand entries (yes, there may only be a hundred of so employees at any given time, but the database will need to track previous employees over many years), while the database to track employees for a multinational conglomerate may be required to scale to millions of entries (those past employees again).

Scalability may also need to be applied to the number of transactions that a system needs to be able to handle at any given time. A web-based system for updating administrative details for an office may only need to be able to handle one user at a time. A web-based system for ecommerce, on the other hand, may need to handle hundreds or thousands of transactions at the same time. Scalability considerations can affect design criteria, such as how processing is done, or the number of systems resources, such as memory, storage, processors, and networking. Scalability may also drive how the system is broken into modules and whether processing needs to be parallelized or otherwise adapted.

Scope drives the architecture in several ways. The boundaries of the scope are going to drive the boundaries of the system. Scope will determine where there needs to be interfaces to other systems. It can be useful to group similar functions within a module, but only if they are all within scope. It is not uncommon for a system to require inputs that are out-of-scope from another system.

Scope can also require adding additional capabilities to a system to handle requirements that seemingly may not be directly related to what is expected but are included within the given scope. An example of this would be a requirement for looking up information. These seemingly innocuous little details can sometimes add a great deal of complexity or drive a need for a totally different architecture from what was expected.

Scope is often related to scalability, manageability, location/locality, internationalization, and security. The need for **security** is a given, but specific security needs, as identified in the vulnerability mapping process and threat assessment, can make some architectural approaches riskier than others. Systems designed to be used in-house and using low risk or open data will have fewer constraints to their architecture based on security needs.

Performance is a big factor driving the architecture. Many systems have no real performance challenges or requirements other than don't be slow. But that

can change with many factors. Systems that handle real-time transactions can have demanding performance requirements. In some cases that may even require that the system make use of a real-time operating system (RTOS). Other approaches to performance issues may be the same as are used for addressing scalability, including the introduction of parallelism, specific system resource requirements, and the structure of the components of the system. When performance is slow, people tend to reload/refresh. This cycle then repeats as more and more users refresh, making the system slower, eventually resulting in a denial-of-service (DoS). Systems can also behave unexpectedly when under extreme load and end up spilling confidential information through error messages.

Another means to address performance requirements is through the choice of languages used or requirements for specific processors, such as the video cards used for calculations for Bitcoin mining, or neural processors, which have been introduced for deep learning applications. Performance enhancements can also be made by profiling and optimizing segments of code or functions in the code. This can be done at a rough level at the architectural phase either by pencil and paper calculations or through simulations.

Architectural considerations for performance may also include specifying specific algorithms for implementation in the application. For example, rather than simply specifying that a sort is performed on data, the architect may specify that a bubble sort be used or that the data be stored in a balanced tree.

Manageability is the ability to control the system and how it operates. This includes tasks such as setting up users, making changes to work processes, or managing the timing and flow of transactions through the system. For some systems, manageability is not an issue. Users run the system when they need it, the system does its thing, and then it terminates.

Systems that are going to need to have multiple simultaneous users are going to need to give a great deal of thought as to how the users, user groups, and user privileges are managed. Often, there needs to be a whole subsystem related to user management. As this subsystem reaches into many areas of the system, it is often referred to as the management plane.

Systems that deal with sensitive data or financial transactions also need a higher degree of manageability. This includes being able to monitor what is happening in any part of the system at any time. Maintaining logging and ledgers is also a factor in manageability. Another aspect of manageability that is often overlooked is being able to determine how a specific transaction was processed and how decisions about a process were made. A place where this

could prove important is in ensuring that a bank is not red lining in approving loans.

Reliability is the ability of the system to continue to produce accurate and timely results. Reliability can be achieved in a number of different ways. One such way is through redundancy where there are multiple systems or multiple components. The redundancy can be applied to accuracy by having multiple components cross-check the results to ensure that they are accurate. This is done in traffic lights where one circuit or computer determines what the light settings will be while another checks to ensure that no output is even given that could result in green lights being given to two lanes of traffic where drivers might collide.

Redundancy can also be used to allow for failover and resiliency, where if a module fails, the load is handed to another, or multiple modules share the load to ensure that there are sufficient resources to enable the load to be handled. Resiliency and reliability can be achieved through load shedding, where when the demand is sufficiently high, lower priority tasks are shed to allow completion of those that require absolute reliability.

A good example of resiliency is with the Apollo Guidance Computer (AGC). These computers were used in the command modules and lunar landers for moon missions. Upon descending to the moon, the AGC of the Apollo 11 lunar lander was operating at normal load of about 85 percent. A docking radar system was placed on standby in case it was needed if the mission had to abort as the landing neared. This put the computer at close to maximum load. As the pilot, Buzz Aldrin, put in a request to display altitude differences between the landing radar and the calculated altitude, it caused the load to jump beyond 100 percent. Instead of a hard failure, the system used priority scheduling to shed less important tasks and was able to continue allowing the astronauts to make a successful landing.

Another means of obtaining reliability is with a multiphase commit. In this process, data are being entered into multiple places in a database or program. If the data are only entered into one of the locations, but not the others, the result can be an error (for example, money added to one account, but not deducted from the second) with significant results. To avoid a partial entry, the data are entered into the multiple locations, but only provisionally. If confirmation is not received from all locations that the entry was successful, the transaction is rolled back. Otherwise, a commit is issued, and the transaction is completed. Reliability is closely related to performance, scalability, and security.

Locality is concerned with how functions, data, and interfaces are grouped together. Locality can be obtained both spatially and temporally. Spatial locality would group data which are related or which will contribute to the same functions into the same or close proximity storage locations. This would minimize the need to transport data across a network or spend time assembling data for a function. Spatial proximity would also put the processing of data near where they are being stored—logically or physically. Logical proximity can be achieved by having fewer functions needed to prepare data for processing.

Temporal proximity is grouping data uses or processing functions together in time. Temporal proximity can be achieved by multiple methods. One is to handle data fewer times by processing as much as can be done whenever the data are accessed. For example, instead of going through a set of data to calculate an average, going through the data again to calculate a median, and then going through the data a third time to calculate the high and low values, all three could be calculated at one time. This is an obvious and simple example, but the principle of touching the data the fewest times applies. Another means of increasing temporal locality is to pass data directly from function to function rather than reading, calculating, writing to a database, and then going through the cycle again.

Locality can have an effect on the performance of a system. Every time data need to be gathered from across a system, there is a performance cost. There are also costs in security as the data are more exposed as they are in motion. Locality can also enhance reliability as the data are on hand and ready to process.

In the ideal world, all needed data are located together in one position where they are local to the processing that needs to be completed. Reality intrudes because this can only actually ever happen in relatively simple system and not in a system that is overly complex and likely poorly written. Instead, the best approach is to add as much locality as possible, especially in those areas where it can contribute to performance, reliability, and security. Sometimes too much locality can also have negative effects on performance, security, and reliability.

Coupling is a measure of how tightly the modules of a system are integrated. Tight coupling implies that the modules have very close connections with little delay through the connections and a high dependence on the reliability of the connections. In a loosely coupled system, the interfaces are designed such that the system can continue to function if the connection is not reliable.

The request across the interface is either repeated or replaced with another request to a different module.

Tightly coupled systems tend to have higher performance. In some circumstances tightly coupled systems can be more reliable, but that isn't always the case. The degree of coupling can have varied effects on security as well.

Rather than simple **internationalization**, there should perhaps be a broader term. The goal of internationalization is to think of the system in terms of not just where it is expected to be used, but where it could be used. Obviously, if a system is also going to be used in a country where the primary language is different from the country for which the system is primarily design, there will need to be consideration of how to support different languages.

The considerations need to reach well beyond language. There are differences in how business processes are run in different countries, even in the same industry, and even if the countries use the same language. Some of these differences are due to cultural differences; others can be ascribed to regulatory and compliance requirements. But sometimes things are done differently just because. This can apply within the same country as well. Two different companies in the same industry may do things differently. As shocking as this seems, systems need to be designed to accommodate this.

The designers also need to keep in mind that the system may even be used in an industry that is different from the one for which it was designed. There is nothing worse than being in the position of an integrator installing a system into an organization where the system tries to impose its worldview onto the organization. The result is doomed from the start to be unhappy, even if the install is successful.

Architectural Principles

Architectural principles provide the targets the architect is trying to reach in any design. Systems architects have worked out these principles over the years and determined that the effort put into achieving these targets is going to result in a superior system. Some important principles to keep in mind for both functional, technical and, in particular, security are discussed in this section.

BOX 11.2 Architectural Principles

- Single responsibility

- Don’t overload functionality
- Don’t repeat functionality (appear in one place only)
- Least knowledge
- Loose coupling
- Built to change
- Identify key engineering challenges first

Single responsibility is applied to functions and components in limiting the responsibilities of any given function or component. Another way of expressing it is that every single component should have a single task and should do it well.

When the single responsibility principle is ignored, we get modules with multiple intents or functions where it can become unclear which code controls which function and changing the code can have unexpected consequences. A good example of a failure of single responsibility is a car that I once owned. The car had a poor-quality radio with a nonfunctional AM band. I wanted to replace the radio. But upon further investigation, I discovered that the radio also contained the computer that was used to control other functions, such as the seat warmers, the air conditioning system, and even the cruise control function. Replacing the radio would have cost thousands of dollars, and the only replacements that would work were from the car manufacturer. The choices were not much better than the original. When the seat warmers in the rear seats stopped functioning, it was the same issue. Similar problems occur in other systems where the single responsibility principle is applied.

Related to single responsibility is the principle of “don’t overload functionality.” It differs from single responsibility in that it is not addressing unrelated functionality. Instead, it is focused on functionality that is not needed. Too often we add the ability to do things or control things because we can, not because we need to. Added functionality leads to added complexity, which reduces security. Any time someone can “play with” or “tweak” a parameter or setting, they will, even if they have no comprehension of what the “tweak” may have for an effect. Added functionality also adds costs and testing requirements.

Way back when component stereos were a big thing, the various manufacturers competed by adding all of the fancy dials, buttons, sliders, and other functions they could onto the amplifiers they built. The reality was that the

vast majority of the purchasers used two controls out of all of that functionality, the on/off switch and the volume control. The rest was simply show and extra cost. Even today I have an amplifier sitting on the shelf above my monitor with the treble and base knobs set to their default positions. I don't even use the volume control on this one but use the on-screen speaker icon instead.

Sometimes requirements specify that we need to include little used functionality. When that is the case, there are better ways to handle it than just shoving everything and the kitchen sink into the design. Whenever possible, most choices for setting options should be optional and include reasonable defaults. The users should also be informed of what options are available and their effects. Interfaces among components should accept the least amount of data needed to perform their function, again using reasonable or preset defaults whenever possible.

Also closely related to single responsibility is the principle don't repeat functionality. All functions should only appear in a single place. There are several reasons for this, the first of which is reliability. If a function is implemented more than one time, there is a greater chance of an error being made in the development. There is also a reduced chance that the error will be found in testing. The tests may cover one instance of the implementation, but not the other. If the tests appear good, the product is shipped with the error in place on the second implementation.

Even if all instances of a function are developed correctly, when it is time to change something in the function, it will be harder to identify where the changes need to be made, or even that multiple changes are made rather than one. Rebuilding functionality costs more than developing it a single time.

The concept behind **least knowledge** is that no module or user of a system should have access to anything not needed to perform the required function. The more frequent access there is to any information, the more opportunities there are for that information to be leaked, stolen, or abused. The principle of least knowledge dictates that we consider carefully whether data or information needs to be provided in its current form. Least knowledge is similar to the concept of least privilege, which is commonly used in cybersecurity. Under that concept, users are specifically limited to only the functionality they need to perform their job. Least knowledge differs in focusing on data and what users, interfaces, and components of the system need to know to perform a function.

Often, it is better to send summarized, processed, or higher-level data rather than providing raw data to a component. If for example, we need to know the age of an individual, but not the birth date, then only the computed age should be provided. The birthdate itself is personally identifying information (PII) and

can be abused more than the age can. In querying whether a user is allowed to perform a function or not, the proper response is a yes/no for the specific function queried rather than sending back a list of the functions that the user is allowed to access. The latter may provide an attacker a map of how to circumvent the access restrictions.

This same concept it going to drive how we handle error messages and error handling in the system. As a simple example, we can look at error messaging for login attempts. If when a user tries to login, but fails, the system sends either a “not authorized user” or a “failed password” error message, an attacker can try different forms of a user’s name. Most will fail with a “not authorized user” error. Once the error changes to “failed password,” the attacker has now confirmed the correct form of the username and can then check Facebook to find the appropriate clues to guess the password. In a similar manner, if the design does not thoughtfully handle errors and exceptions, valuable information can be exposed to attackers.

Failing to stick with the least knowledge principle can also harm the resilience of the system by creating dependencies that are not needed. If there isn’t a need for a piece of data to perform a process, requiring that data prevents the function when that data are not available.

In the previous section we discussed the degree of coupling as a concern. That leads to a principle of favoring **loose coupling** whenever possible. The principle of loose coupling needs to be applied in the broadest sense, not just in defining interfaces. This means taking the perspective of breaking apart a system into components that can operate independently and perform their specified tasks without depending on another component’s behavior.

Sometimes loose coupling can be achieved by providing a context as well as data inputs to allow greater independence between modules. Loose coupling is also aided by allowing out of order and parallel processing. This can develop into a service-like design where the component providing the service is free to internally order and handle the service requests in the most efficient manner rather than having to respond to each request in the order submitted. Following this principle, we look for how we plan our components to encourage loose coupling.

A basic architectural principle is that anything we design needs to be **built to change**. Even when we think the solution is a temporary stopgap measure that will be soon tossed on the trash heap, we need to build to change. It turns out that while there is a lot of code on the trash heap, much of what is not was code that originally was intended to be, but it worked good enough, especially if tweaked here, here, and here. Twenty years later, that code is still running,

and the maintainers are cursing the name of the original designer, wishing the system had been built to change.

When a system is designed from the beginning to be flexible and allow easy changes, it is far cheaper over the longer term to make those changes.

As we develop the architecture, we need to identify **key engineering challenges** first and let them drive the architecture. Key engineering challenges are those which are going to be the most difficult to meet and those which will have the most influence on other areas of the architecture. The key engineering challenges are going to have the fewest workable solutions, will fit the fewest design patterns, and are going to involve the most effort to implement. These challenges are the ones that are going to be the hardest to accomplish if a noncompatible design pattern is chosen. While they won't often be totally undoable, the extraordinary measures that will be required to make them fit will be costly and will almost certainly ensure that security vulnerabilities will arise.

Security Objectives

Security objectives are the security results that are expected from the developed architecture. They are overarching requirements that are not specifically requested but need to be met to the highest degree possible because of the architecture. How well they are met is a test of the quality of the design. The objectives also provide a means for comparing different architectural choices and designs against each other. Gauging the differences in how each potential design choice meets the objectives allows for objective comparisons of choices.

BOX 11.3 Architectural Security Objectives

- Protect information
- Reduce attack surfaces
- Improve resilience/robustness
- Create audit trail

The first objective for security is protecting information. Information needs to be protected at work, at rest, and in transit to the greatest extent feasible. Not all information is equal, and the protection levels need to be adjusted to meet

the levels required as determined by the sensitivity of the information. Any architecture that leaves information exposed by default is flawed.

The next security objective is to reduce attack surfaces. Attack surfaces are those points where threats potentially have access to a system. These include interfaces, between components, between the system and people, and between the system and other systems. Consolidating these surfaces to areas where they can be closely watched and managed is one useful method. Another is never exposing interfaces when they aren't strictly needed. For example, functions should be implemented as private functions when possible.

Attack surfaces are also found in the physical and logical infrastructures that support the systems. These need to be carefully architected to prevent them from becoming the vulnerable point in the system.

Attack surfaces can come in the data of a system. Data structures and storage that are poorly designed can lead to significant vulnerabilities. In addition, if the data are not used carefully, then the data itself can be used as an attack surface. One method in which this could be done is through embedding code into the data that gets executed when the data are accessed. Another method of attack is to use poorly designed metadata, such as indexes with encoded meanings to infer information about the data.

Another objective is to improve resilience and robustness of the system wherever possible. Doing so includes loose coupling as we have discussed previously, adding redundancy where possible, and preventing potential problems from ever occurring. Including resilience and robustness requires a lot of thought exercises, considering what could go wrong and how to prevent that from leading to an issue.

In designing software systems, it is important to create an audit trail. The reasoning behind decisions made in a software system need to be able to be clearly understood. Sometimes this will require storing information about why decisions were made. Sometimes merely recording information about the transactions are sufficient. Audit trails serve multiple purposes. They can be used to identify potential attacks, they can be used to identify potential deficiencies in the system, and they can be used to demonstrate that the system is working as it is required to.

The next principle is to secure interfaces. Because interfaces are known and exposed attack points, how they will be secured always need significant consideration.

Design Patterns

Most systems and their requirements are not totally unique. That makes life easier because instead of designing everything from whole cloth, we can reuse successful effective designs. Those effective designs that reoccur time after time are known as **design patterns**. These design patterns appear at the macro level as established architectural frameworks, and at the micro level they represent patterns for providing solutions to bounded problems.

There are several keys to effectively applying design patterns. The first is to be familiar with and be able to recognize the types of problems or circumstances that are related to specific patterns. Without broad knowledge of many different patterns, every problem becomes a “hammer” problem. That is the same thing that happens when the only tool a builder has in his or her toolbox is a hammer: “Nail—no problem, I have a hammer! Screw—no problem I have a hammer! Prying two boards apart—no problem, I have a hammer!” Just because a particular approach has been effective in one instance does not mean that it will be in every other instance.

Some system architects work with only a hammer in their toolbox. Unfortunately, they find enough hammer problems to be successful, obtain a degree of recognition, and then continue to apply hammer solutions to systems where such an approach is totally inappropriate. When the development ultimately fails, they place the blame on the developers, the management, or “misfortune” when the real reason for failure was their inability to recognize and apply an appropriate design pattern. Don’t be a “hammer” architect; search out, study, and learn about as many different design patterns as possible.

The next key to effectively applying a design pattern is to recognize that the pattern is only a guide. It is not a legally binding contract where deviation can lead to jail time and fines. Patterns are just that—a guide. Like Plato’s forms of things, the design pattern is not real itself, and it can only be implemented in an imperfect manner because of real-world obstacles. The form shows the qualities the architecture must have but not all of the ways in which those can be implemented.

What this means for us is that sticking too closely to previous implementations of a particular design pattern can limit our ability to successfully apply that pattern to the next solution. Instead of focusing on the details, the focus needs to be on the key qualities of the pattern that make it effective.

The other key to effectively using design patterns is to know when to break with the pattern. Again, problems are unique, and even when a system is being built to accomplish something for which completed systems already exist, unique circumstances and criteria will make this system unique. At those points where the uniqueness is evident, some items just won't fit within the design pattern. Although sometimes that could mean that there was a better design pattern that could have been chosen, at other times the only effectiveness is to bend or step out of the design pattern for pieces of the system. No one will put you in jail for doing so, although your professor may cut you a grade or two if you can't justify why it was necessary. No big deal though because a real boss would simply congratulate you on figuring out how to overcome the challenge.

Example Design Patterns

The following are brief overviews of a number of common architectural design patterns. The specific implementations of these patterns can vary significantly from the core. Often, multiple patterns are combined together in various ways to achieve the desired goals. The objective with design patterns is not to rebuild anything exactly, but to leverage the work done by others to fit a solution to a specific problem.

Terminal-based is the standard way of doing things. The user interacts with the software system through a terminal, adding inputs and receiving outputs. These two are very tightly coupled; even if the terminal is far from where the system is running and has all of the data displayed, nothing works if the connection isn't there. Sometimes we think of this as the old-fashioned way of doing things. But most of the web is terminal-based. The only difference between the browsers as we use them and the old 3270 or VT-100 terminals is the quality of the graphics.

The **client-server** model moves much of the processing off the server onto a capable user device. The server provides data and handles high resource demand processing needs as well as specialized functions to perform tasks such as user authorization or validating data provided by the client. The client off-loads processing as much as it is able given its capabilities. Clients that perform minimal local processing are often referred to as "thin" clients. Clients that perform significant amounts of local processing are "thick" clients.

Coupling between the client and server if implemented correctly is looser so if the client has collected its data and has no high resource processing needs, it can continue to process information on its own up to the point where it needs more from the server. The client can be designed to hold onto data it produces

until the server becomes available. Some web-based systems have begun to move more in this direction through client-side processing. In the client-server model, there is still a terminal, but the client takes the responsibility for determining what is sent to the terminal, how it is displayed, and what inputs are received.

N-tier, or **multi-tier** systems extend the client server model. These systems begin with a three-tier model: the data are stored on a separate database server; a server accesses the data and applies business logic to the data as well as performing high resource demand tasks; and a client-tier performs some processing and manages the interaction with the terminal. As additional specialized processing needs are added, additional layers are added to the system. These may be added in parallel to the existing layers, or in series. There are multi-tier web systems, although many that appear to be actually are the client-server model, with the database on the server, the web server acting as the client, and the web browser simply a terminal.

A **service-oriented architecture (SOA)** divides the functions that must be performed into services. The services are independent processing needs that are requested from a client. The client requests services from multiple components and then stitches together the results into what will be presented to the user. As it is implemented, the components that provide the services are reusable and interoperable. The specification defines a standard interface for the service. Usually, this is defined as an application programming interface (API).

The standardized interface and access allow services to be shared among applications or rapidly integrated into applications. The API, if well written, can allow loose coupling between the requester and the service. The architecture can also allow for flexibility in where the service is physically located with requests made across a network rather than as function calls on the same system. Services may also make use of other services.

There are many different implementations of service-oriented architectures, each with their own set of protocols and specifications. Each has a slightly different interpretation of how a service-oriented architecture should be built. Some implementations are web based; others are more traditional programming focused. Common protocols and approaches include SOAP, REST, WSDP, and WCF.

An extension of the service-oriented architecture approach is **micro-services**. In this approach, services are broken into their most elemental components, which are individually implemented. Larger services can be composed from groups of micro-services, or the micro-services can be used

directly by the client application. As with the service-oriented architecture, there are many different specifications and approaches. Most contain some sort of a management or orchestration module that is responsible for starting and stopping services, distributing the micro-services across computing platforms, and performing other management tasks, such as load balancing, resilience, and logging.

Another architecture related to the service-oriented architecture is the **message-based architecture**. In a message-based architecture, the components of the system are all connected to a shared messaging bus. The bus may be implemented in many ways, including locally on a single server or distributed across a network. The components provide different services or facilities.

Components of the message-based architecture include one or more display modules, data modules, and other functional modules. When a module requires data or the results of a service, the request is put onto the message bus. From there it is either distributed to an appropriate service model by a message broker or recognized by the service module as a message of a type it should respond to. The receiving module processes the request in the message appropriately and passes the results back through another message.

Some types of messages may simply include data or information that components listening on the bus may choose to use or respond to. Messages may also be assigned to queues to wait for their turn to be processed by one or more components. Queues can be used to distribute processing with multiple processing components picking up messages from the top of the queue as the component completes its previous task and is now available.

The message bus and mechanisms for selecting which message components respond to allows for the loose coupling among components.

The **event-driven architecture** builds on the concepts of the message-based architecture. In an event-driven architecture, all processing is done in response to events. Events can be keyboard strokes, mouse clicks, messages from other components, or timer based. The existence of an event is delivered as a message. Information about the nature of the event and event details may be included in the message, or it may be passed through a mailbox, shared memory location, or other mechanism.

Upon receiving the notice of the event, the component completes the appropriate processing, passes any messages needed, checks the queue for the next event, and if there is none either goes to a polling or interrupt driven waiting state until the next event arrives. In this architecture, usually an event manager or message manager ensures the messages are distributed to the

correct event handler components. The components register their existence and the events they can handle with the event handler. As an event occurs, the event manager will check the priority and load of the various event handlers using that and other information to determine which handler or handlers to pass the event to.

In some implementations, the event manager function is distributed with multiple event managers monitoring the message bus and determining if the event handlers they have responsibility for should handle a particular event. The Windows architecture is an event-driven architecture, as are many games.

The **model-view-controller (MVC) architecture** is based on separating data from how they are viewed and how they are manipulated. In MVC, the model is how the data are represented within the system, either in a database or in active data structures. This model is the base representation. Different views are created from the model for displays to users and to applications that will consume or use data from the system. The views are built to coincide with the needs of the viewer. Views can be selected data, summarized data, or calculated from the data (for example, ages calculated from birthdates) and can be textual or graphical in nature.

Changes to the data, and thereby results, are created by invoking the controller. This function set validates inputs, creates and manipulates the data, and then returns the data to the model. Separating the responsibilities for managing the model, providing the view, and control allows for a concentration of similar functions. It also allows for loose coupling, if done correctly.

The MVC pattern was originally created to support graphical user interfaces and has become popular for web applications as well. Many frameworks (Django, Rails, etc.) implement the MVC in structured manners and are popular for reducing development time by using prebuilt templates for the models, views, and controllers. As long as the problem space fits these templates, the paradigm works well. Once the level of complexity outgrows them, it gets challenging.

Another model base architecture is the **model-centered architecture**. This is also sometimes known as a shared data architecture. In the model-centered architecture, the model, as instantiated in the database, is shared by all components. The components function independently of each other, relying solely on the database for communication. Changes in the database are used to trigger or stimulate components that need to perform processing tasks. Other components function to generate or display data or to perform processing on the data based on user requests.

Building on the model-centered architecture is the **space-based architecture**. This architecture is designed to handle the challenge of scalability and variability of demand. It does this by eliminating the central database but sharing a distributed data store. The processing units each work on their local view of the data performing needed manipulations. Processing units may be specialized or general. A middleware layer is used to asynchronously replicate changed data to active nodes and distribute data to new nodes as they are activated. The middleware layer also handles any needed messaging about units, processing that needs to be done across units, and deployment of needed nodes. The middleware is responsible for initial routing of requests to processing units.

The space-based architecture works well for handling tasks that naturally require little coordination, especially when they are high volume. The space-based architecture is commonly used by major web service providers.

Another architectural model that distributes processing is the **peer-to-peer architecture**. In this model, each node in a network of computers acts as both a client and a server. As a client, the system provides the interface to the user and runs an application to perform the main function. The client reaches out to peers for specific services. The peers are other systems on the network that provide processing and storage services. Communication among the peers is protocol based. In the peer-to-peer system, there is no central controller, and the purpose of the system is accomplished through distributed cooperation.

A common example of a peer-to-peer system is a file sharing application. Using the client application, the user requests a specific file. The client then queries a distributed index maintained on the server portion of the system. One or more of the peers will have the location of the file indexed and return that information to the client. The client then uses the location information to request a download of the file from the peer server or servers that have stored the file or portions of the file. Because the work is distributed, no single server experiences an overwhelming load or becomes a bottleneck in the process.

The indexes and processing are also arranged with a high degree of redundancy so that if one or more of the peers drop out of the network, or become unreachable, there is no negative effect on performance. In some cases, even a peer that gets isolated can continue to function with its own local server, although with reduced capabilities.

In addition to file (and music) sharing, peer-to-peer architectures have been used for many distributed tasks, including searching, identifying potential alien signatures, telephony, digital currency, anonymous web browsing, and internet

commerce. The architecture is well suited for problems requiring redundancy and decentralization.

Some implementations of peer-to-peer systems fudge a bit and include centralized indexing or other controls. Usually, this is done to maintain control of the network of peers for commercial purposes.

Another model used for distributing processing is the **map-reduce architecture**. Under this model, a problem with massive data or massive processing or both is handled by mapping the task into multiple parallel jobs. Each job will perform a portion of the actual work obtaining partial results. If a task cannot be divided along specific boundaries, the partial results need to be distributed. These distributed partial results are reduced into a new set of job parts that are then either mapped into the next stage or combined to produce the final result. The number of map/reduce stages depends on the nature of the problem.

Implementations of the map-reduce architecture generally use some sort of a tracker/manager to oversee the process. In theory, this could all be done in a distributed manner, but in practice, this tracker/manager tends to be centralized.

Refinement Techniques

Developing a successful system architecture is never a simple once and done effort. Because of the need to balance so many different considerations at a time, it is essential to take an iterative approach, looking at as many different ideas as possible each time. As the design proceeds there are several approaches which can be used to help.

BOX 11.4 Refinement Techniques

- Layering
- Risk assessments
- KISS
- Examine trust relationships
- Model complex systems

Layering is breaking the problem domain into different layers and working through how those are implemented or supported. Sometimes these will become the layers of the architecture as discussed previously, but sometimes specific layers are used and considered in the design process even when they are later set aside.

Layering is also used in considering things at the most abstract level first. Then, as a solution begins to result, reach down to the next layer of abstraction, and the next until the design is completed.

Risk assessments, including vulnerability mapping, need to be performed continuously. Although formal risk assessments and vulnerability maps should be created periodically, that is not sufficient. Each time a drawing is completed or a solution is contemplated, that drawing or solution should have an informal risk assessment completed. This means that in addition to answering the question “Will this design element help solve the problem?” we also need to ask, “Will this design element add any vulnerabilities or risks to the system?” If we don’t ask and answer those questions, we risk ending up as a meme. The next refinement technique is to simplify, simplify, simplify, expressed as Keep It

Simple Silly—**KISS**. As the design process moves forward, we are going to inevitably add bits and pieces here and there in our process to handle specific cases, additional requirements, or details of the next level of abstraction. These bits and pieces slowly but surely begin to add complexity to what was an elegant solution. Periodically throughout the process we need to step back, review what has been completed and look for ways to simplify the solution. Remember, complexity adds risk.

Periodically we need to examine **trust relationships** and then reexamine them. Trust relationships are the expectations of levels of trust between different components of a system, or different systems. Typical examples of trust relationships may be to accept all data passed, to allow full use of functions, or to only use data passed after a thorough filter. A **trust zone** is a grouping of components and systems which share a common trust relationship. While we may have begun with some expectations of what the trust relationships would be, the boundaries of trust zones can stretch or erode as a design progresses. This can happen as we assign additional functionality to a component, as we extend the interfaces between one component and others, or as we add interfaces into our design. A design change may expose an area where we thought everything was working at the same level of trust.

Another refinement technique is to model complex systems. The work of creating the model can help guide us to potential solutions, or it can show us what we thought would be a solution is not feasible. Modeling the complex parts of the system also helps us in our efforts to create vulnerability maps and perform risk analysis.

Architectural Reviews

Architectural reviews are an important step in the design process. The architectural reviews can be both formal and informal. Formal reviews tend to be more structured, covering specific points with attendance by specific stakeholders and a scripted walk-through of the architecture. Their purpose is usually to obtain approvals or buy-in to the architecture. Formal reviews can require significant preparation time.

Informal reviews are used to provide waypoints in the development of the design, to inform others of the path the architecture is taking, and to obtain input from people who may be able to make valuable contributions. Informal reviews are a means of increasing the collaboration going into a design. The informal review may cover all or a part of the architecture. It can be tailored to focus on the areas of concern to the reviewers. Informal reviews can also be used to structure the layering process being used to refine the design.

SUMMARY

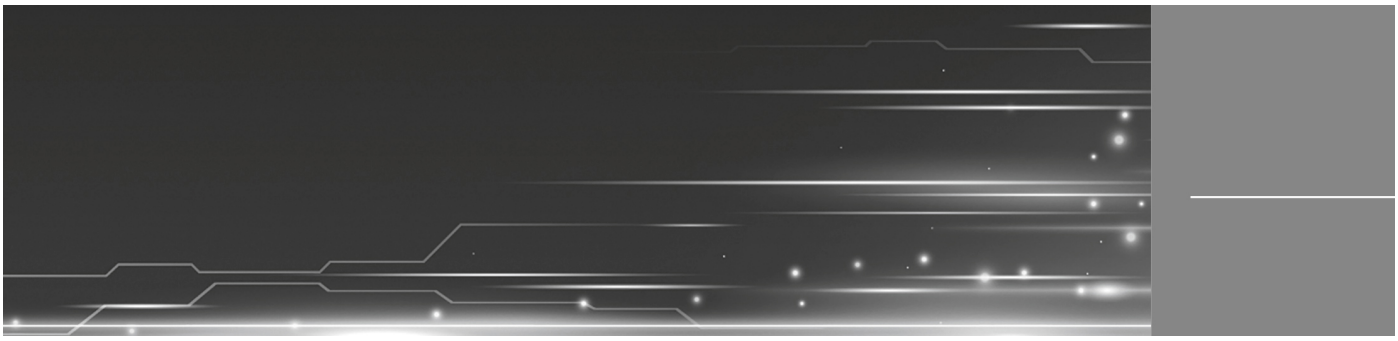
Establishing a solid architecture is essential for a secure software system. The solid architecture provides structure from which all of the features can be provided. Without the solid architecture, pieces get tacked on here and there, and soon the system is full of vulnerabilities. Fortunately, some solid foundations are available that have proven themselves over time. These design patterns provide a guide on which architectural concerns and principles can be applied to form the basis for a secure system. Recognizing which pattern to apply to a specific development effort depends on understanding the nature of the system being developed, being familiar with the different design patterns, and being able to apply the pattern to the problem.

Once a basic architecture is decided, it needs to be evaluated carefully to ensure that the architecture fits the problem and that no latent security issues need to be addressed. This is done in the architectural review.

KEY TERMS

application architecture
architectural concerns
architectural principles
architectural reviews
architectural views
authoritative source
built to change
business or functional architecture
client-server
coupling
data architecture
data custodian
data ownerdesign patterns
enterprise architecture
event-driven architecture
internationalization
key engineering challenges
KISS
layering
least knowledge
locality (location)
logical architecture
loose coupling
manageability
map-reduce architecture
message-based architecture

micro-services
model-centered architecture
model-view-controller (MVC) architecture
n-tier, or multi-tier
peer-to-peer architecture
performance
physical or technical architectures
reliability
risk assessments
scalability
scope
security
service-oriented architecture (SOA)
single responsibility
space-based architecture
terminal-based
trust relationships
trust zone



© d1sk/Shutterstock

CHAPTER 12

Vulnerability and Threat Assessment

CHAPTER OBJECTIVES

- Develop a vulnerability map using a system model.
- Identify methods to find system or service vulnerabilities in design, build, or any life cycle phase.
- Conduct risk management on identified vulnerabilities.
- Conduct a threat assessment exercise and develop a threat model.

Introduction

When we start to develop a system, we seldom start by imagining what can go wrong and who will want to attack our system. But we should. Beginning this type of analysis early is exactly what allows us to make our system secure. This analysis has two parts. First is figuring out who or what the threats are, and then second is anticipating where in the system or service design are vulnerabilities that can be exploited. In this chapter, we will look at both areas. However, as we do, it is important to remember that while we do these at the start of a development, it neither is a one-time effort nor stays static. Also, each effort informs the other and, used in an iterative manner, can improve results.

Vulnerability Assessment and Mapping

The goals of **vulnerability assessment** and **vulnerability mapping** are many:

- Identifying potential vulnerabilities of a system as early in the process as possible
- Influencing the design and development to minimize vulnerabilities that do appear in the system
- Determining the attack surface of a system
- Creating a vulnerability matrix
- Providing input into risk management and mitigation practices
- Driving security testing

BOX 12.1 Vulnerabilities

A **vulnerability** is a weakness in a system that has the potential to fail or otherwise be exploited thereby exposing the system to undesired performance, leakage of data, or other negative security problems.

The first two goals go hand in hand, as many potential vulnerabilities that are identified early can be designed or developed out of the system. Awareness of what can potentially become a vulnerability allows developers to either avoid the choices that could lead to that vulnerability altogether or develop robust safeguards for those vulnerabilities that cannot be entirely avoided. While not eliminating the vulnerabilities, the likelihood they will be exploited can be reduced, and if exploited, the impact can be reduced.

The **attack surface** of a system is where the system faces or is exposed to potential attacks. It includes areas where a system interacts with other systems or accepts inputs from people and things. Minimizing attack surfaces is a good security practice because it limits the number of pieces of a system where defenses need strengthening, and it limits the places, methods, and techniques that a potential attacker could attack.

A good illustration of reducing an attack surface is a moat around a castle. Without special equipment, the moat limits direct attacks to only the drawbridge. Likewise, limiting access to or from the internet to a single module of a system reduces the attack surface of the system for internet-based attacks. As with the castle, it does not entirely eliminate all attacks because there will still be Bad Guys with special abilities and other types of attacks, but it does greatly reduce where many common attacks can be made.

Identifying early in the system development what the attack surface will be is useful in molding it into a smaller attack surface as we continue the design and development of the system, in ensuring that those components and sections of components that are a part of the attack surface are appropriately hardened, and in creating a catalog of the surface that should be monitored when the system is in production. This could, for instance, lead to choices of specific logging requirements.

The results of the vulnerability assessment and mapping will be used to help drive the security testing for the system. Testing based on known potential vulnerabilities ensures that they have been handled properly according to plan. It tests areas that testing based solely on functionality or technical requirements will not.

The Process

Vulnerability mapping begins with a model or multiple models of the initial system concept. Having a model is efficient for beginning to identify potential vulnerabilities as it matches the way that our brains work. People are graphically oriented, and our brains have evolved to identify potential problems in images, whether that is a leopard hiding in a tree waiting to pounce or a pending buffer overflow. Having a graphical representation also aids in providing a systematic approach.

The modeling methodology used is not important. In fact, it can be useful to have several different models of the system, each with their own perspective. This gives broader insight and may make it easier to identify the full suite of potential vulnerabilities. One example of a vulnerability map is shown in **Figure 12.1**.

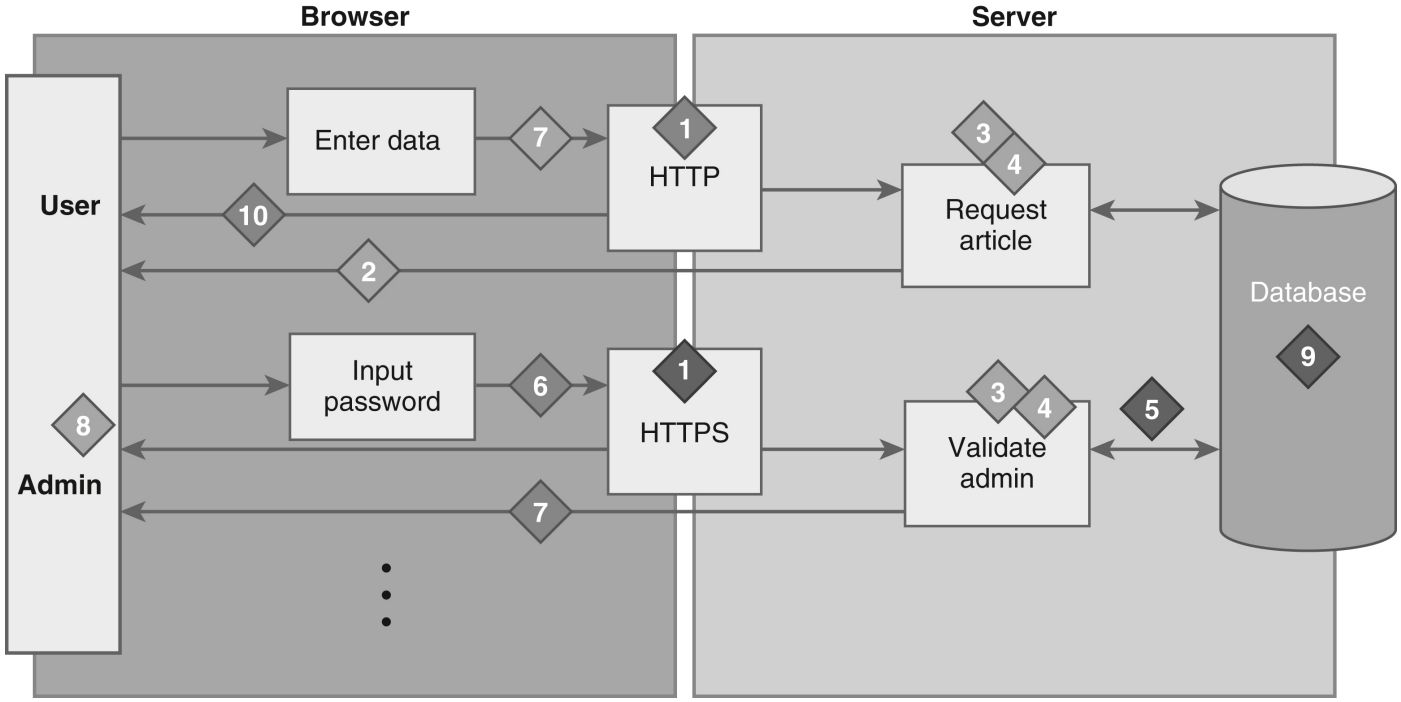


FIGURE 12.1 Example Vulnerability Map

Description

On this map, a model of a web-based news reader, the locations of potential vulnerabilities are indicated using colored diamonds where the color indicates the severity. The labels, descriptions, and so on are put into the vulnerability matrix.

Although there are many published specific vulnerability mapping methodologies, they all have the same core features. In all of them, the model is carefully and systematically examined by an experienced security expert. During the examination, the vulnerability analyst identifies each spot on the model where one or more vulnerabilities are likely to occur. The spot is marked and labeled with an identifier. Sometimes additional information about the severity, likelihood, or other aspect of the vulnerability is also included in the label.

The Vulnerability Matrix

The identified points of potential vulnerabilities are then added to the vulnerability matrix. Each vulnerability is then assessed and rated by various criteria. Any updates to the label on the mapped are noted.

Figure 12.2 and **12.3** provide examples of vulnerability matrices. A vulnerability matrix is a listing of all the potential and actual vulnerabilities identified for a system. Such a list is maintained throughout the life cycle of the

system. In addition to including information about the nature and location of the vulnerability, the matrix is also used to track the handling of the vulnerability, whether it has been eliminated through some method, is protected through a control, or simply left for future consideration. The matrix would generally include information about the level of danger associated with the vulnerability.

Threat	Impact	Vulnerability	Likelihood	Severity	Risk	Preventative measures
(1) DDOS attack	Servers could be shut down for unknown amount of time	Firewall is not up to date	Low	Moderate	Low	DDOS protection in the firewall and limited login attempts
(2) Malware filled news article website	User devices could be infected with malware	Virus's that are hidden within a website's links or images	Moderate	Moderate	Moderate	Security check of news source before allowing user access
(3) Server hardware exploit	Server information, including that of users, could be accessed and released	Hardware is out of date or has known exploits	Low	Severe	Moderate	Keep hardware up-to-date and ensure that security patches are implemented
(4) Server hardware damage	Hardware would be impacted and unable to be used until replaced	Environmental disaster/human interference	Low	High	Moderate	Server racks/room designed to enhance durability

FIGURE 12.2 Vulnerability Matrix—Example 1

Description

Index	Vulnerability	Severity	Likelihood	Description	Handling or remediation (include generated requirements)
6	Buffer overflow	**H**	**H**	Receives string from input.	Mitigate: **(R 2.15)** All input strings must be sanitized. **(R 2.16)** Use only safe string functions.
•••	•••	•••	•••	•••	•••
21	Man in the middle attack	**H**	**L**	Comm channel could be intercepted.	Mitigate: **(R 4.17)** Use public key validation of the comm channel.
•••	•••	•••	•••	•••	•••

FIGURE 12.3 Vulnerability Matrix—Example 2

Description

Even vulnerabilities that have been avoided (mitigated) or eliminated should remain in the matrix. One reason for this is that it documents changes that have been made to avoid or eliminate the vulnerability. Should someone later revisit the changes and wonder why they were made, or propose other changes, the reason for the changes is documented, and the opportunity to reopen vulnerabilities can be avoided.

Risk Assessment

The vulnerability matrix provides an input to the risk management and risk mitigation process for the system. Risk management recognizes that there is no perfect world without risk. It provides a methodology for proactive management of risk in a manner tuned to the organization.

The basis of **risk management** begins with developing the level of risk an organization is willing to accept, or the risk appetite. This can be a challenging process for an organization that has never done so before because it means questioning values, understanding the organizational goals, and aligning with the corporate culture. These take a certain awareness that is not always common in organizations. Unfortunately, no magic potion gives an immediate awareness of the values, goals, and culture within an organization and how those translate to a risk appetite, but as the organization makes the effort to do things deliberately and pays attention to challenging questions, the awareness will emerge.

The next step in risk management is to identify all the potential sources of risk. Many of them will be the identified vulnerabilities, but there will be others, such as development delay, loss of customer confidence, market risk, and others. Each potential risk is evaluated, either quantitatively or qualitatively, as to the likelihood of occurrence and the loss should the risk be realized or happen. These values can then be examined and ranked.

Each identified risk will be handled in one of four manners. The risk can be **avoided**. That may be as simple as choosing not to do something or choosing a different method without the risk.

The risk can be **mitigated**. Mitigation involves putting controls in place to reduce or eliminate the risk or the extent of harm caused by the risk. For example, adding two-factor authentication reduces the risk of stolen or hacked passwords because the password cannot be used successfully without the second factor. Salting the hashes of passwords in a password file reduces the risk of passwords being compromised if the file is exposed because the hash does not depend on the password alone.

The risk can be **transferred**. In this scenario, the risk becomes someone else's responsibility, either operationally, financially, or both. One example of this is the use of insurance to move the financial risk to another party. Another is using an outsourced security monitoring organization to operationally manage risk from network intrusions.

The risk can be **accepted**. Sometime the level of a risk is so low or the cost of addressing a risk is so high that it is simply accepted. Often as a part of that decision, the risk area is designated for monitoring and periodic reconsideration. Or it may be that some significant event informs us to reconsider the extent to which we accept a risk. For example, as we sit here, there is a risk that the super volcano in Yellowstone is going to blow. It is a low probability of occurrence, even though the consequences could be devastating. So, we accept the risk and don't do anything to reduce the impacts of the event. If, however, there were a series of earthquakes and an outbreak of new geysers, we would reconsider our evaluation and take steps to mitigate the consequences. For example, the park might be closed to visitors, and nearby residents might stock up on basic supplies.

Which of the four methods is applied to each source of potential risk will depend on a number of factors. Prime among those will be the risk appetite of the organization, the budget, and the technical feasibility of the different risk management strategies. Ideally, as well, the risk management and mitigation strategies will not be applied simply as a part of development planning for the current system but as a part of the overall security and corporate risk management process of the organization.

Additional Vulnerability Assessment Tools

Trust Boundaries

During the systemic analysis, several areas are closely assessed or used as assessment guidelines. For example, the different relationships among the items in the model can provide clues about vulnerabilities along the boundaries they share and their interfaces. A boundary type of particular concern would be a **trust boundary**. That is the boundary between modules with differing levels or domains of trust.

The most obvious type of trust boundary is the external trust boundary. This is the boundary between different systems or a system and external inputs. Neither side of an external trust boundary should trust the other. Limiting the places where trust boundaries are crossed or interface leads to limiting the attack surface for a system. For external trust boundaries, interfaces need to provide significant focus on authenticating and validating data and communications.

Internal trust boundaries are important as well. These are boundaries between different components or groups of components within a system. How vulnerable items cross these trust boundaries will depend on how and where the components are implemented. For example, two processes operating on the same CPU and communicating through shared memory or an operating system buffer will be less concerning than the same two processes operating on different processors and communicating over a network socket. Internal trust boundaries, like external trust boundaries generally, expect equivalent levels of trust. Assuming too much about what those levels of trust are can become a vulnerability. Internal interfaces will also require appropriate levels of authentication and validation.

A system can have multiple internal trust boundaries. For example, there would be a trust boundary around functions that control administrative actions. The same system may also establish a trust boundary around the components that communicate externally. Internal trust domains cannot be overlapping. In the case that they are, the full set becomes the trust domain with the level of isolation and trust of the least of the domains and potential access to everything within the expanded domain for anywhere within the expanded domain.

The third type of trust boundary is the client trust boundary. Unlike the external and internal trust boundaries where the level of distrust is an equal on either side of the interface, the client trust boundary is more of a lopsided type of trust. There may be an initial verification of the source, but thereafter the interface toward the client-side of the boundary does minimal authentication or validation of the data, instead implicitly trusting what it receives. The standards of authentication and validation are significantly higher for data traveling in the other direction.

An illustration of a client trust boundary would be a weather reporting system. The client consumes data about the current weather conditions from a server. The data are used primarily for display and are fleeting. The server does very little to check who the client is, let alone authenticate, because it really does not care. If the temperature is off by a few degrees, no big deal; it will get updated in a second or so. On the other hand, if the client tries to report weather readings to the server, the server is going to make sure it authenticates the client and that the client is allowed to report weather data. The server is also going to check that the data fall in an expected range. Additionally, the server may include checks to validate that the readings received do not deviate from the previous set provided beyond a reasonable range. In that way the server ensures that the weather it is reporting to other clients has a degree of accuracy.

Life of Data

Another type of analysis is a **life of data** or **n-scope map**. This analysis is based on the fact that data don't just exist in one place in cyberspace for its entire existence. Data are created or entered somewhere and then eventually find their way into an application where they are transferred, processed, and stored. Data also leave the application. As long as data remain within the scope of the application or a selected component of the application, they can be protected and secured. The component is built to provide the appropriate level of confidentiality, integrity, and availability (CIA) for each piece of data. Before and after the data are in the scope of the component, protection is no longer certain.

To evaluate vulnerabilities from where data come and go, we need to trace the life of that data. To do so graphically, we can use an n-scope (or n-order scope) map; a sample is shown in **Figure 12.4**.

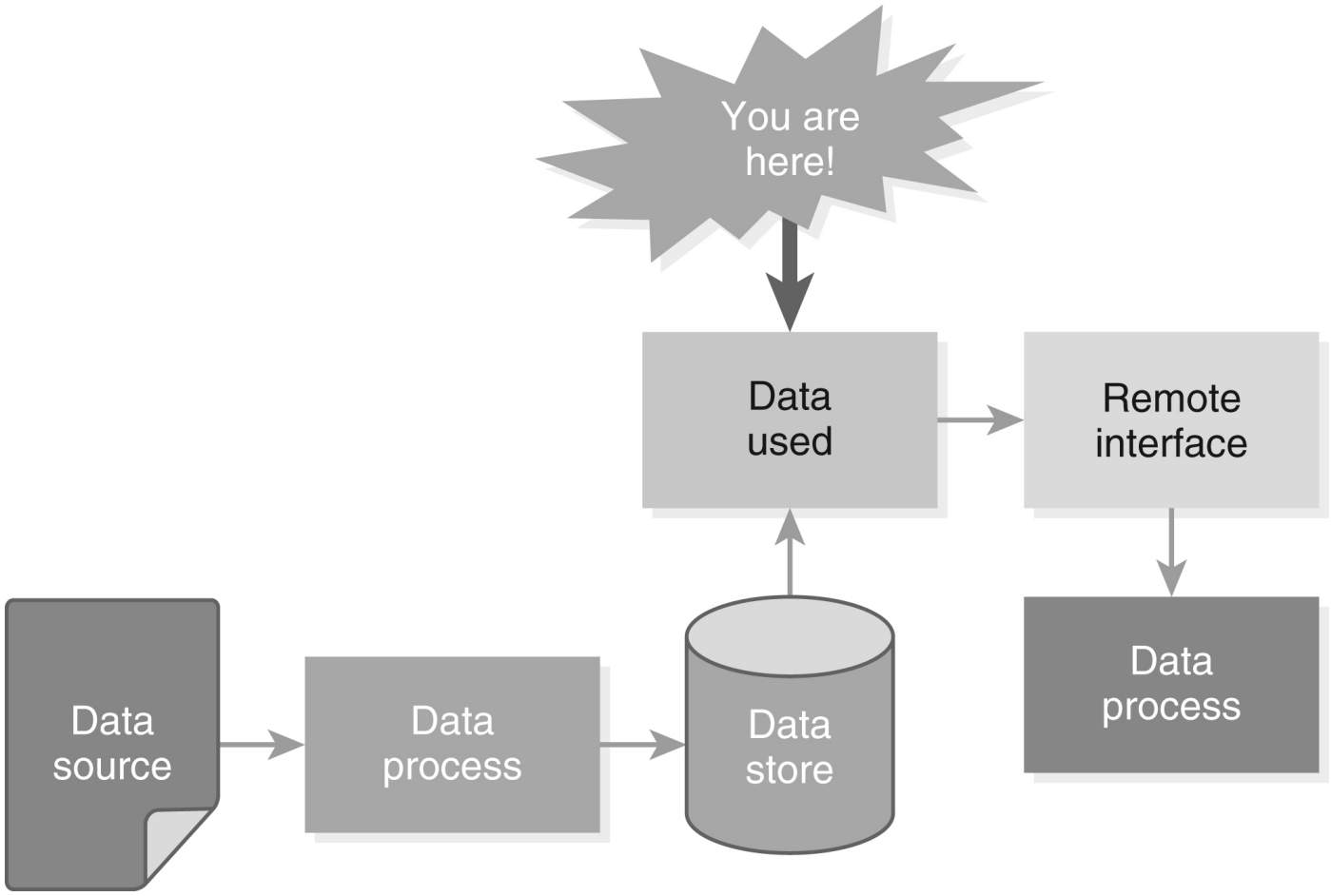


FIGURE 12.4 N-Order Scope Map

Description

To create the map, we begin drawing the current component. The next step is to draw lines to each of the immediate sources of data within that component. At each of those sources, we review what happens within the scope of that source to the data to determine if there is an opportunity for the data to be altered or otherwise compromised. Each opportunity is listed as a vulnerability. For each of the sources, we will repeat tracing where the data came from until we either reach all the original sources for the data or we have reached a predetermined number of levels.

Having traced the data backward, we now begin to trace the data forward to where it is sent when it leaves the scope of our application. Again, we can use arrows pointing toward data receivers. Here again, we will examine what occurs within the scope of the recipient. In addition to looking at immediate losses of control of the data, we will also want to examine the scope for possibilities of data leakage through inference, aggregation, or other means.

Other Sources for Analysis

A good source for discovering potential vulnerabilities in a planned system is to look at attack vectors used for similar or general systems. An **attack vector** is a commonly used means for launching or is a carrier of attacks, like how a mosquito is a vector for diseases. Certain practices, protocols, and applications are vectors for cyberattacks. A good example is email.

Emails can contain harmful attachments, nasty links, and other just plan no-good stuff. Any use of email in a system would then present a potential for vulnerabilities. Those vulnerabilities can be avoided or mitigated by how the system is designed and developed. A system that doesn't allow attachments on incoming emails is not vulnerable to attacks originating in email attachments. However other attack vectors are still available to attackers using email systems, And the users of the email system are not going to be happy. In this case, avoiding the vulnerability is not the best choice, instead, we will need to look for a way to mitigate the vulnerability by reducing the ways an attachment can cause problems and minimizing the harm that can be done when they do.

Attack trees are a similar source for discovering vulnerabilities. Attack trees are the patterns and combinations that attackers use to exploit a system. For example, they may start with a vulnerability scan, locate a poorly secured port, load a shell onto the port, and then download a rootkit. Tracing how known attack trees could be directed against a system allows the designers and developers to chop off the attack at the roots before it can be successfully exploited.

In addition, we can look to known classes of vulnerabilities and decide whether they apply to the system. Each class of vulnerability is associated with a service, type of service, or type of component typically found in software systems.

Threat Modeling

Threat analysis is the art of determining who or what are the most probable threats to our system. A **threat**, as we are aware, is an actor or action that is going to take advantage of a weakness or vulnerability to breach, incapacitate, or otherwise harm system security. That harm can come in any of the classic forms of confidentiality, integrity, or availability—the CIA triad.

BOX 12.2 Threats

A threat is a person, object, or situation that is likely to cause damage to our system.

- People include black hat hackers, rouge users, and innocent employees making simple mistakes.
- Objects include magnets next to disks, rocks, and bad software.
- Situations include fires, blackouts, and volcanos.

A threat model is the result of an analysis that provides us a snapshot of the expected threat picture, likely threats. Multiple threat models may result from our threat analysis. For example, we may develop both a current threat model for threats we are facing today as well as a future threat model for threats we expect to face at some point in the future. Threat models are updated and revised on a regular basis.

Threat models are used to help several ways. They help determine the current defensive posture. When the threat model tells us we are currently under a high level of threat, we can increase the defensive measures and monitoring we use. We can also determine from the threat model where to focus our defensive measures and resources. This allows more effective use of limited resources and increases the likelihood that our defenses will be effective.

Threat models can also be used in evaluating vulnerabilities in systems we are developing. In creating a threat model for a system under development, we create a tool that can focus our attention on critical places to look for and

address potential vulnerabilities in a system. This aids in developing more secure systems.

Purpose of the System

Different threats are inherent, attracted to, or can result from different system purposes. Systems that handle money (banking, wire transfers, ecommerce) attract threats that would like to have money. Systems that provide power or utilities (electricity, water, or pipelines) attract threats that would like to have political power by manipulating the critical infrastructure. Systems that manage private materials attract threats that would like to peer into that data and exploit it for their personal gain or fascination. Determining which threats will be interested in a specific system begins by identifying the purpose of the system.

The purpose of the system may be defined in multiple ways. Most obvious is by the industry or industrial sector in which the system will be deployed, but purpose can also be defined by the functional role the system will perform within the organization. These roles include standard types of functions, such as human resources management, billing, customer relationship management, and enterprise resource planning, as well as industry-focused systems such as policy management systems, insurance, industrial control systems, manufacturing, and automated visual inspection systems.

Classes of purposes that need special consideration are those that support critical infrastructure, and those that support third-party services. Critical infrastructures include areas that keep the economy functioning and safe, like utilities, transportation, and emergency services. In all, the Department of Homeland Security has identified 16 critical infrastructure sectors thar through their incapacitation or destruction would have a crippling effect on the economy, national security, health, or other vital interests of the country. Because of this risk, these sectors are especially attractive to many threats.

BOX 12.3 Roofing Threat

In one interesting example, a building was reroofed. As a part of the process, a leak developed immediately over a rack of network equipment in the top floor server room. The leak put water onto a firewall, a laptop, and an old CRT monitor. The CRT monitor simply steamed off the water and was not damaged. The laptop filled with about a cup and a half of water and was destroyed. In the firewall, the effect was a partial hardware failure that affected the processing but still allowed some unfiltered traffic to

continue. The firewall could not be properly restored and needed to be replaced.

Systems supporting third-party services are often deployed onto the networks and into the facilities of the clients of these services. When this is done, the systems are not always managed or monitored at the same levels as other corporate systems. Even when security practices and monitoring are in place for these systems, unexpected gaps may exist at the service interfaces. Combine this potential for vulnerabilities with the third-party systems being deployed into “interesting” places, where “interesting” is defined as in the vicinity of potential targets, and these systems become an attractive target for many threats.

Every created system attracts adversaries, antagonists, and nuisances that stand in the way of accomplishing the system’s purpose or actively opposes the system. The challenges are many and varied. Some are general challengers of any purpose, but others may be foes specifically directed at specific purposes. These adversaries, antagonists, and nuisances represent threats to the system. Threats are people, processes, conditions, environments, and things that actively or passively seek to do harm to a system. A black hat hacker on the internet is a potential active human threat. The extreme cold and cosmic radiation of space is a potential passive environmental threat. But these threats are only threats if the circumstances allow them to be. If our system is not connected to the internet, the black hat is not a threat. If our system is not being used in outer space, the cold and radiation is not a threat. When circumstances change, we need to reevaluate the potential threats. This happens if a system designed to be disconnected from the internet is used on a computer attached to the internet, or if a system is selected to be used on a mission to Mars.

Generally, we can classify threats into four categories: environmental threats, user threats, external threats, and the purposefully persistent. **Environmental threats** come in many forms. An environmental threat exists in the environment in which the system operates, regardless of where that environment is located. They may be natural environmental threats, such as earthquakes, volcanos, hurricanes, drought, and tornados, or special types due to where the system is deployed, such as deep space radiation, saltwater intrusion, or intense heat. These natural environmental threats can negatively affect the system’s ability to function, stress the system, and cause weaknesses to fail, thereby resulting in direct failure of the system or

susceptibility to other security issues. For example, an undersea security monitoring system that has its sensors disabled by saltwater intrusion may fail to detect a diver swimming past. Or a system in a data center compromised in an earthquake may reboot to a state where root access becomes available online.

Environmental threats also occur outside of nature in the buildings, offices, and other infrastructure. Common infrastructure threats can include fire, poor environmental conditioning, leaks, and location problems. Location problems may include complex challenges such as having a shared data center or simple challenges such as deploying a system where it cannot always be monitored, such as with point-of-sale equipment, informational kiosks, and remote sensing systems.

User threats come about because the users of systems are humans. Humans do all kinds to things that are not always considered to be ideal. Humans forget things, make mistakes, jump to conclusions, cheat, and betray our trust. User-related threats can vary from simple ignorance in not knowing how to use a system, to careless practices and habits, to deliberate insider attacks. The first two are perhaps more common, but the last can be much more destructive.

Insider attacks are deliberate attempts by employees, contractor, or other insiders to damage or take advantage of an organizations systems. Such attacks are made by insiders who are disgruntled, entitled, or criminal. Disgruntled employees see attacking an organization's systems as a way of getting revenge without "hurting" someone. Other insiders may feel they are justified because their compensation is too small so they are only getting what they are rightfully due when they misdirect funds from the organization to their own bank account or pursue similar activities. Other insiders are motivated by external compensation. This is a combined insider and external threat, where the insider is recruited, knowingly or unknowingly, to enable or participate in an attack by an outsider.

Because insiders have the opportunity to have significant knowledge about systems, processes, and targets of an organization and because they are often permitted broad access to those systems, insider threats have the potential to enable significant damage. These types of threats need to be seriously considered and carefully evaluated in completing a threat assessment. It is all too easy for insider threats to be quickly and simply dismissed as "we don't have that kind of people working here." That kind of attitude represents an insider threat in itself—deliberately blinding the organization to the potential.

The next category of threat is the **external threat**. These are deliberate attacks against systems from external adversaries. This is the black hat hacker or hacker organization going against a system or the latest versions of ransomware floating around on the internet. External threats represent the largest category of threats and generate the most attention.

A specific type of an external threat is the **attractive nuisance**. Attractive nuisances are features and characteristics of systems that encourage external attackers. These attackers are like moths drawn to a flame or neighborhood kids sneaking into an unfenced swimming pool. They appear because of characteristics of a system, not because they have an interest in the system itself. They are attracted to the type of system or to methods used in the system, such as a common port. The attackers often are the first to appear, usually as a swarm. They may suddenly disappear, only to be replaced shortly by a new mob responding to the next attraction of the day. While often not very sophisticated, the sheer numbers and constant bombardments can slip up the unwary. In developing our systems, it is important to eliminate attractive nuisances whenever possible.

The purposefully persistent are often categorized as the **advanced persistent threat (APT)**. These threats have an intent beyond pursuing the flavor of the month attack. Their motivation, be it money, ideological, or other, is specifically directed at certain targets. The targets are carefully chosen, under constant examination, and are subjected to continuing efforts. Simply defeating the first, third, or forty-seventh attack from the threat isn't enough to defeat the determined. The purposely persistent will continue until they have accomplished their purpose, the target is removed, or the threat disbanded or wiped like a scourge from the face of the earth.

Threat Intelligence

Threat intelligence is the process and the product of identifying, analyzing, and tracking threats. Threat intelligence can be assembled from a variety of sources. Some of these are more general resources, which provide intelligence about common threats, recent incidents, and relevant news. Other resources are going to be more specific about threats in an industry, against a specific application, or regionally seen threats. Threat intelligence sources can be "open source," where the intelligence is available to everyone without charge. Other sources are commercial.

Open-Source Intelligence (OSINT) makes use of openly available resources to preform threat analysis. While anyone can perform OSINT, some individuals and groups specialize in the activity. These producers often focus on a specialized area. They might look at threats to a specific industry or sector or track and trace specific threats or types of threats. Many of these groups openly share their results and analysis on the web, letting everyone benefit from their efforts.

Intelligence Sharing and Analysis Centers (ISACs) are industry or sector (financial services, utilities, health care, maritime, etc.) focused organizations supported by governments, industry coalitions, and other parties seeking to support the best interests of the industry. They research, collect, and provide intelligence specific to the industry or region they serve. ISACs have the advantage of knowing their industry or sector and are thus able to perform deeper research and provide more relevant information than general threat analysis.

Threat intelligence sources can provide data as a feed, a push stream from the provider. Often it is formatted for electronic consumption. This is especially true for tactical intelligence, intelligence meant for immediate action, such as a stream of known bad IP addresses meant to be added to firewall filters. Strategic intelligence, on the other hand, is intelligence that influences long-term thinking and planning. Sometimes it is less structured in format and may not be as easy to fit into a standardized format. Strategic intelligence may need an analyst to decode its significance and importance. Often intelligence can be used for both tactical and strategic purposes.

A common standard used for sharing threat information is the **Structured Threat Information eXpression (STIX)**. STIX structures messages to include threat information such as motivations, abilities, capabilities, and response and

has become an international standard. STIX messages are shared using **Trusted Automated eXchange of Intelligence Information (TAXII)**. The TAXII standard defines an API for sharing intelligence using multiple models. Commonly an ISAC will distribute categorized and refined intelligence from TAXII clients while accepting raw intelligence, which will be analyzed and shared with others.

Threat intelligence from news feeds is used to keep the analyst aware of the general environment, significant trends, and an awareness of new developments. A well-informed analyst is better able to place a potential emerging threat in the right context or to understand the potential of a threat in each environment. Even when not working specifically as an analyst, it is a good idea for every cybersecurity professional to monitor cyber news on a regular basis.

Some of the organizations that work with threat intelligence like to display pew-pew maps. These are graphical representations of current attacks underway around the world with arrows, flashes, and sound (hence the "pew-pew"). While impressive visually, these usually serve little purpose other than executive fluff. However, the tracking, data, and trending that goes on behind these maps is of great value. Trending is what allows analysists to predict what attacks are becoming more common and are more likely to be seen in the future. Trending analysis can sometimes detect the early beginnings of future trends.

Vulnerability databases, such as the CVE® list maintained by MITRE or the U.S. National Vulnerability Database (NVD) maintained by NIST, also provide a source of threat intelligence. While the focus of these is on threats, the information can also be used to understand where threats may be emerging, what vulnerabilities threats have taken advantage of, and other valuable threat intelligence.

Through threat intelligence we attempt to identify who the known bad actors are and who will become the next bad actor. For each bad actor, we want to understand their motivation. Bad actors may be criminals. Usually, their motivations are financial. Bad actors may be state actors, either military or civilian. Their motivations are to further some state political or economic goal. State-sponsored actors may work to further those same types of goals but may also have a secondary financial goal. Ideological actors are driven by political, theological, or other motives.

In understanding the actors, we will try to identify their known and projected capabilities, such as their coding competency, their abilities in other technical areas, their access to financial and other resources, and what type of support

systems they have in place. In analyzing their technical competencies, we can identify techniques that they are known to use, which of those are their signature techniques, and those most closely identified with their work. We can also identify the actors, their preferred targets, the sectors, systems, and areas they tend to attack.

Building the Threat Model

Building the **threat model** is a four-step process:

1. Create classes of potential threats.
2. Identify and list potential threats.
3. Specify known, favored vulnerabilities.
4. Update the model.

In creating **classes of potential threats**, we are developing criteria for determining which of the broad spectrum of threats are of specific interest for our system. Not every threat is going to go after every potential target. Threats, like everyone else, have limited resources and are going to focus the resources they have based on their motivation and skills. Threats are more likely to target systems and victims that are like those where they have had previous successes. Basically, they are identifying the market for their product. In creating classes of potential threats, we are trying to identify in which of those markets our system lies.

Classes of potential threats can be based on threats that target the industry our system supports. The threat class may include those who target a particular technology used in our system, similar systems, or systems using specific libraries or code bases. In building the classes, we need to consider every motivation that some threat may have to target the system we are developing rather than some other random system.

Potential threats are threats that have been evaluated and deemed to be relevant to our system and organization. With the classes of threats identified, we can then work out a list of known and emerging threats that fall within the classes we have identified. The list of threats can be further pared through an analysis of the potential threats looking for factors that would preclude them from needing to be considered. For example, if they operate on a regional basis where that region is outside of our area, we would be less inclined to worry about them, unless they showed signs of extending their operations. In

another case, a potential threat that targets our type of system but has shown ideological intent aligned with our organization may not be considered a significant threat. Once we have listed the identified potential threats, they are ranked by likelihood and level of ability or severity of the threat.

With the threats identified and ranked, the next step is to identify the specific vulnerabilities that are going to be **favored vulnerabilities** of those threats. The favored vulnerabilities are those that a given threat is known to have previously used, has shown signs of using, or that is similar to another vulnerability that the threat has exploited. The first stop for those vulnerabilities is the vulnerability map and matrix developed for the system. Each favored vulnerability the threat is known to exploit is compared to the matrix to see if that vulnerability has the potential to exist in the system. Those that are not found in the matrix are checked to see if that vulnerability could exist in the system. Perhaps it was one that was overlooked in the vulnerability analysis. No problem because the vulnerability map and matrix are not a one-time artifact carved in stone. New potential vulnerabilities discovered in this manner are simply added to the matrix.

With a list of favored vulnerabilities of identified threats, there is a very specific list of areas for priority attention. Vulnerabilities attractive to the worst threats receive the priority for ensuring that they never convert from a potential to an actual vulnerability. If the identified vulnerabilities cannot be entirely avoided, then they can receive the priority for mitigation and monitoring.

Naturally, the threats are going to change over time. New threats will emerge, and others will fade away. Existing threats will change their focus, tools, and techniques. For that reason, it is important to update the threat model on a regular basis. Besides periodic reviews, there are other indicators that the threat model should be updated, such as emerging threat intelligence, changes to the system and its surrounding environment, and occurrence of significant business, political, or ideological events.

SUMMARY

Two things need to be clearly understood in order to protect software systems. The first is the potential and present vulnerabilities of the system. The second is the threats that are going to try to exploit those vulnerabilities. By understanding the potential vulnerabilities, they can be dealt with using standard risk management activities: avoidance, mitigation, transfer, and acceptance. Once this is completed, there will be a remaining residual risk.

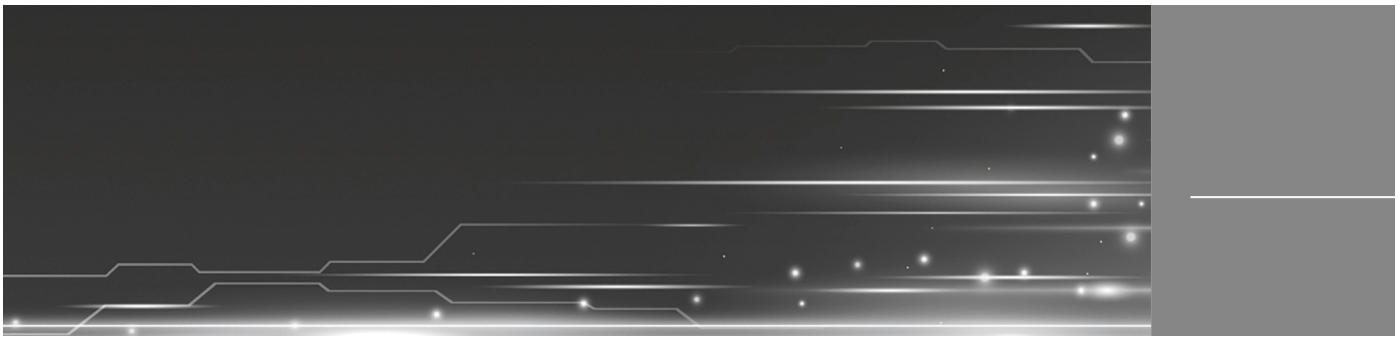
Threat modeling is a method to understand the threats that are real and relevant for a particular system in the context of the organization. While all organizations and systems face some common threats, others are more individual and focused. Understanding these threats and the vulnerabilities that are attractive to them provides a basis for determining how to treat the risks posed by vulnerabilities in the system and how much of a hazard the residual risk represents.

Threats and vulnerabilities change over time. This means the vulnerability and threat assessment efforts cannot be a one-time event. Instead, they need to become an ongoing effort based on current intelligence and knowledge.

KEY TERMS

advanced persistent threat (APT)
attack surface
attack vector
attractive nuisance
classes of potential threats
environmental threats
favored vulnerabilities
insider attacks
Intelligence Sharing and Analysis Centers (ISACs)
life of data, or n-scope map
Open Source Intelligence (OSINT)
potential threats
risk acceptance
risk avoidance
risk management
risk mitigation
risk transfer
Structured Threat Information eXpression (STIX)
threat analysis
threat intelligence
threat model
threats
trust boundary
Trusted Automated eXchange of Intelligence Information (TAXII)
user threats
vulnerabilities

vulnerability assessment
vulnerability mapping
vulnerability matrix
external threats



© d1sk/Shutterstock

CHAPTER 13

The Development Environment

CHAPTER OBJECTIVES

- Describe how the development environment impacts the security of software systems.
- Discuss how the selected development tools affect system security.
- Outline precautions to apply when using libraries or other external code.
- Explain how organizations establish standardized practices for system development.

Introduction

The environment in which we develop software systems is as important to the security of the systems we develop as the code we write. A well-managed and secure development environment is needed to produce secure, quality software systems. The development environment needs to be considered carefully as a major source of potential vulnerabilities and treated with the same degree of risk management we treat other vulnerabilities. Suppose for instance, we try to bake a cake in a broken oven, with a dirty pan, or a using an unordered recipe. The cake is bound to fail. It is easy to think of the development environment as benign; after all, it will go away when we release the system and send it to be deployed. But that is not the case in reality. The end product is a result of the environment in which it was produced and inextricably carries with it the stains of that environment.

Culture of Secure Development Practices

The first area of consideration of the development environment is the corporate **security culture**. While much is said about that in another chapter, the key here is focusing on how the culture applies to the development environment. It is not uncommon that a company has a culture that is supportive of the development of secure products, at least on the surface, but has little concern for security in the development environment itself. This is an attitude that "security is important for our customers, but we don't need to concern ourselves with it." This attitude allows bad practices to flourish in the development environment, which then work their way into the finished product.

Organizations with a relaxed security culture for development will send mixed signals by having well-secured, managed, and monitored production environments, but an unmonitored, unmanaged development environment. In the production environment, all changes to servers and firewalls will be rigidly controlled and documented by a central team. But in the development environment, the developers make changes as they like without verifying the changes ahead. This disparity in approaches to security reinforces in the developers and others in the organization that despite the window dressing of concern for security on the released product, security in not really valued in the organization as anything more than a sales point and the culture does not really value security.

Often the relaxed attitude to security is justified as "not interfering with the developers' ability to get their job done." But responsible development requires adherence to processes that ensure security is always considered. Thus, while the processes followed in "production" may not be followed exactly, there will be equivalent processes tuned toward allowing developers to make changes needed without compromising security. For example, while some developers may be given administrator access to their computers, they would be required to follow a specific process and document any software they load onto their computers. This both allows flexibility for developers to test new tools and other software in a timely manner while maintaining an accurate record of software in use.

Another dual culture challenge is when developers are treated different from other users of the network. An example of this happened when hackers broke into an Electronic Arts Slack channel using a stolen cookie. Inside the chat, they posed as an employee who had lost his phone and requested a multifactor

authentication token from IT (Cox 2021). Instead of requiring the employee go through a proper verification process, the request was granted, and the result was hackers successfully gaining a lot of valuable data.

Of course, one of the most insidious cases of cultural failures is when the organization avoiding the standard processes is the security organization itself. Monitoring security does not justify administrative access to everything. The security staff, as with any other group, needs to follow the principle of least privilege and only be allowed access to what they need to accomplish their mission. Security staff, just like other people in the organization, can go rogue, make mistakes, or take misinformed actions. The effects of these risks need to be limited as with any other risk.

Secure Development Tools

The selection of tools used in development is important for ensuring a secure product. Nothing is more insidious than trying to build secure software systems only to have the tools you are using insert secret backdoors into the code, allow universal access to your proprietary code, or allow a developer to overwrite and lose key code. The concept of **secure tools** requires that tools used in development need to be reviewed carefully to ensure that using the tool doesn't insert new vulnerabilities into the development process. These vulnerabilities may be intentional or unintentional and may appear either in the product developed or in the process of developing the product.

While tools are always designed to help in some manner with the development process, sometimes tools are created or bought without considering the effects they will have on security. For example, having an online tool to help track and manage projects can be really useful as the system can help with planning and scheduling as well as tracking progress. But if the tool is not well secured or if the team using the tool uses poor password practices, the whole development project is now accessible to anyone on the internet.

Sometimes the tool allows itself to become easily vulnerable by having **unsecure defaults**. Often, tool vendors have the defaults in their tools pre-set to the lowest levels of security because this makes it easier for users to learn and adopt their tools. There are no configuration challenges; the users just install the tool and begin using it. The problem is that these default configurations often involve leaving information open to external access, sending copies of information to places where it can be accessed by or sold to competitors, and default or easily guessable passwords. The first thing that needs to be done with any tool is a slow deliberate walk-through of the configuration settings with security in mind.

Naturally, a big concern for all software tools is usability. Tools that are complex or difficult to learn can discourage people from adopting the tool. Complex, difficult-to-master tools can also result in users making poor decisions about the security needs of these tools. For this reason, it is important to consider tools with usability in mind and to also ensure that users get proper training on the tool.

In addition to having vulnerabilities, development tools can be compromised or even be developed with malicious intent. For example, a compiler may add binaries for a backdoor into code it is compiling. This can be difficult to detect

as the exact size of the compiled code depends on many different factors other than the number of lines of code compiled. Even comparing a resulting binary to a binary from a different compiler would not be useful because each compiler naturally has differences in the outputs.

Naturally, no one would intentionally use a compiler that had been compromised, but unsuspecting developers can be lured into using a compromised compiler. For instance, a compromised version of a well-known compiler could be loaded onto a “free tool” website or put in as a replacement on a poorly secured distribution site. In this way, even though it is sourced from a “legitimate” distribution site, the copy received may be tainted. If the unwary user doesn’t compare the copy received to the distributed checksums, chaos may ensue.

The quality of the tools used in the development process directly affects the end results. If a tool doesn’t support needed features, can’t handle complexity, or severely limits the options of the user, it will result in lower quality code. Free is not always better; neither is cheap or expensive. Better comes from the care put into the design and development of the tool. Developers and their management need to be picky and selective in deciding which tools are going to be used. They need to carefully evaluate how well the tool performs the tasks it was designed to perform and the relevance of those tasks to the organization’s needs. The developers and managers need to evaluate the overall value of the tool, all costs and lost opportunities associated with the tool, and whether return on investment is sufficient.

The quality of tools and the results they produce are also directly related to the quality of the training on how to use those tools. Users can be self-taught, be mentored, or receive formal training on a tool. Simply handing developers a tool and telling them to use it, without providing the training or the time to train, is worse than having developers make do without the tool.

Users without sufficient training are also likely to misuse the tools they are given. This is not intentional. They simply have not been told how to properly use the tool. Sometimes they find one of the features of the tool and simply overuse applying it because they don’t realize that it inserts a vulnerability when not applied correctly. Sometimes they try a feature, and it gives them the response they wanted, so then every time they use the tool, they use the same feature. It was successful once; it should be so again. Other users may take the approach that if it doesn’t work with force, use more force (get a bigger hammer).

Software Sources

Software used in a system can be sourced from many places. Usually, systems involve software from multiple sources. Each of those sources has multiple advantages and disadvantages. As with everything else in secure systems development, the key is to do a careful assessment of what is appropriate for the specific project and to identify the potential vulnerabilities associated with the sourcing choices.

Commercial off-the-shelf (COTS) software has been developed by an organization and is sold to customers. Note the phrasing there. It is not always "developed *to be* sold to customers." It is sold to customers. Sometimes, software is developed for internal use or other reasons and then is subsequently sold to customers. Not all COTS software is of the same quality.

Generally, however, COTS software comes with a certain level of support, some, often limited, warranties, and the company selling the software is expected to continue to develop and maintain the software. None of these are certain, so customers need to be wary and do their due diligence, not only on the software, but also on the company selling the software. One solution if the company appears unstable is to have the code put into escrow with a third party so that if the company disappears or fails to maintain the software, the buyers, or a designated company, can access the code themselves.

The expectation with COTS software is that the company selling the software has done a significant amount of testing and that there are no known security or other flaws. But this is not always the case. There is often no way to know what testing has been done on COTS software. If there are issues, the company should be expected to fix them, although the purchaser may need to also purchase a maintenance package to receive the updates with the fixes. It is always important to carefully check the sales contract to know what fixes are included and what warrantees the vendor provides.

Buyers of COTS software can only find security issues with the software themselves through a black box approach to testing or by investigating CVE databases (discussed elsewhere) to identify issues discovered by others.

The biggest advantage to COTS software is that it can be cost effective. Because there are presumably a lot of users of the software, the development and maintenance costs of the software are spread across a broad base of users. The more specialized the use case, the less likely there is to be a cost-effective commercial product available. On the other hand, very few companies

develop their own word-processing software. Since virtually everybody needs to use that type of software, the cost of creating a company-specific system with the same level of functionality would be prohibitive. Instead, everybody purchases a COTS version or uses an open-source version.

There is only one place where there is a need for software to control a large hadron collider. There isn't going to be any COTS software available for that system. Some needs are in-between in the amount of demand. For these purposes, fully custom software is expensive, but the needs of similar installations are sufficiently distinct that a simple COTS system would not be a good match. In this case, some software is available as a **COTS platform**.

COTS platform software is written to perform common functions out-of-the-box but allows for extended functionality to be programmed or added to meet an organization's specific needs. The platform offers a balance of affordability and flexibility. Some platforms lean more to one side of the equation than the other.

Platform software has some of the same concerns and resolutions with security issues that other COTS software has. One of the more significant risks with platforms though is that the customizations written for them can introduce security vulnerabilities or may even bypass internal controls.

Not all off-the-shelf software is commercial. There are many **open-source software projects** where software is developed and maintained by volunteers or corporate-sponsored developers. Other times individuals give their time to make a product that they believe in. These developers may be unsupported or may get their financial support through donations. Sometimes open-source projects are a public version of a commercial product. Another model is for a company to support an open-source product in order to make money from installing, maintaining, and supporting the product. Any system may involve multiple funding models.

The funding model can have a significant impact on the time spent on the code and therefore the quality. It can also impact the quality through the way decisions are made about the product. It may take longer for issue solutions to get into poorly funded projects. An open-source project with a commercial version may not receive the same attention as the commercial version. In considering the use of an open-source project, it is useful to consider the project's funding model.

As open-source projects provide the software for free, the costs associated with using the software are in how the software is installed, supported, and maintained. These are important life-cycle costs for COTS software as well but are more extensive with open-source software.

Other than low initial costs, one of the advantages with open-source software is that the code is available and can be inspected. In an open-source project with many users and developers, this means that many eyes will be on the code and will have the opportunity to identify vulnerabilities before they are exploited. In less popular software, this advantage may not exist, but at least the code is available to review a vulnerability after it has been exploited and find a control to prevent it from being exploited again. Of course, from an adversarial perspective, the code is available to study and identify vulnerabilities that could be exploited.

Custom-developed or **bespoke software** is created for a specific customer. It can include a full software system, the integration of different commercial products with key custom components, or extensive software used to customize a platform. Normally, bespoke software is written as a work for hire and becomes the property of the company that paid for its development. Sometimes, the developer or developer's company retains the rights to the software and may use it as the basis for other new COTS products.

Bespoke software can be created in-house by developers and contractors who work directly for the company having the software developed, or it can be developed out-of-house under contract. The latter is referred to as outsourcing.

Outsourcing can move many of the tasks needed under the secure software development life cycle to become the responsibility of the contracted company. The more of those responsibilities that are moved, the greater risk the project involves for the contracting company. Unfortunately, when many companies approach outsourced development, they only consider the coding and perhaps testing as necessary elements and don't provide internal or outsourced coverage for the majority of the efforts needed. This makes low rates for offshore programmers particularly attractive when compared to not only the costs of local programmers, but also project management, security, operations involvement, and the many others involved in a well-run secure software development project.

The other big challenge with outsourcing is communications. Because everything needs to be defined carefully in a contract and changes to that contract can be expensive, good initial communications are essential. But continuing communications over the interpretations of the contract, clarifications of questions and issues, tracking of statuses, and other items are just as essential. Too often the approach has been to put the requirements in a ball, throw it over the wall, and hope that the perfect product comes back. That doesn't happen.

Off-shore development refers to having the majority of a product's development completed in another country. Off-shoring can be done internally within a company, but most often refers to off-shored outsourced developments. **On-shore development** is done locally within the country. Sometimes people also refer to **near-shore development**, where the development is done in another country, but that country shares cultural, language, time zone, or other aspects that make communication and project management easier to accomplish.

One type of code that is almost always bespoke but often overlooked when it is time for proper secure development management is **glue logic software**. This is the code that is written to stitch together a group of commercial products or commercial products and custom components into a coherent system. Since it is "simple" and "incidental," it is easy to ignore. Unfortunately, this becomes the vulnerable spot, the code may not be backed up, or an issue fix may be handed to a junior developer to handle it "however you see fit." Glue logic is every bit as important, if not more so, than any commercial or other customer piece of code.

Selecting the Development Language

One of the key items in development is selecting the software coding language. In some cases, the language to use is a given. The organization uses a specific language, and there is no option to choose another. At other times, there may be certain restrictions on the choice of language, but a there are several to choose from. Some problems are better addressed in some programming languages than others. Sometimes it is appropriate to choose different languages for different portions of a system. In web development, this is common, with one language used for client-side and another for server-side programming. This doesn't include the many special case languages that are also included in a project, such as SQL, HTML, and various scripting languages.

Three key issues to consider in selecting a language are functionality, efficiency, and security. Because different languages are designed with different problem domains in mind, some languages are better suited for some tasks than others. For example, some languages do a much better job of natively handling text. Text manipulations are built into their design. Other languages are specifically designed for efficient and accurate mathematical calculations. These languages are designed for strict adherence to standards for arithmetic such as IEEE 754 and will give accurate results to calculations without heroics on the part of the programmer. On the other hand, languages like C pay little heed to those standards and can often produce erroneous math results.

Programming language efficiency is another concern. It depends on both the language and the complier. Some languages will produce machine code that operates significantly faster than that of other languages. Systems written in interpreted languages are almost always going to run slower than compiled languages. Neither is inherently bad; sometimes runtime differences are insignificant for the particular system.

The choice and use of a language can also influence the security of a system. Some languages allow sloppy programmers to create vulnerabilities, such as buffer overruns, stack overflows, and memory leaks. Yet other languages introduce safeguards to prevent these issues from happening. Usually, some sort of a trade-off is involved in the design choices of the language.

One of the most important things to consider in selecting the system language(s) is the familiarity and proficiency of the developers in the languages chosen. Developers will normally be more productive in languages that they already know. Additionally, programmers are more likely to create errors and vulnerabilities in a new language where they are not yet familiar with the capabilities, weaknesses, and common usages of the language. However, a good training program can overcome that problem if it is an issue.

Using Libraries in the Development Process

One key programming practice is the use of **code or function libraries**. Libraries allow the reuse of code and keep the world from having to reinvent every needed function (such as the save or close functions on each form). Libraries allow programmers to use clever functions invented by people who are far smarter. Libraries allow programmers to share occasional strokes of genius with others. Libraries can be used to remove much of the tedium and intricacies of details and housekeeping from slowing programmers' ability to create masterful code. Libraries can be used to allow a programmer to write a particular function one time and then use it repeatedly, not just in the current development effort, but in any other system that needs the same functions. Libraries standardize how parts of code are built. This is all great, and libraries, as a useful invention, rate up there with sliced bread (which also makes life easier, removes the housekeeping of slicing, and standardizes the slices).

For all the reasons just discussed, libraries should be used in programming. But with that said, the indiscriminate use of libraries is one of the worst things a programmer can do, both in terms of producing poor quality code and in producing insecure systems. This happens because programmers are unaware of or don't care about vulnerabilities embedded in the libraries, vulnerabilities that arise from the combination of the library with other code, or other technical, commercial, or legal ramifications that may come from the use of a particular library. Unfortunately, it is usually not the beginner programmers who are solely prone to make mistakes in the application of programming libraries. Sometimes, the more senior programmers are the most grievous abusers. For this reason, it is important for libraries that will be used in the development of a system be identified up front, reviewed carefully, and rules developed prior to their use. Normally, this will be done by the systems architect or designer working in consultation with the security team. Others may be involved in the decision as well. Anyone who may be responsible for creating libraries needs to be aware of their positives and negatives.

Commercial Considerations

One of the first challenges with libraries is that they may have a commercial effect on the system that is being developed. Libraries, like all software, carry with them licensing and other legal entanglements. Not every library is free.

With commercial libraries, the inclusion of the library may require that a fee be paid every time the library is included with new software. While such a fee may not seem significant for a single license, for example for the developers, when a system is deployed and suddenly licensing fees need to be paid for every customer or for each of thousands of nodes across an organization, the "nominal" fees can quickly add up to quite a substantial amount.

Sometimes these licensing fees can be hidden and insidious. This isn't an uncommon issue with outsourced software development. The organization sends off a specific set of requirements, receives back completed code, and then continues on. At some point, an issue with the code comes to light and then suddenly it is revealed that a third-party software library was used in the production of the code. While there may not have been a licensing issue with the code as delivered, it may turn out the only way to correct the issue is through purchases of licenses, either from the original vendor or through a third party. For this reason, it is wise to include in outsourcing agreements that the vendor may not use third-party libraries without permission of the organization and that any vendor-owned libraries included in the development be accompanied by a perpetual, royalty-free, worldwide license for unrestricted use and modification without restriction on the right to sublicense or otherwise authorize third-party use. This prevents vendors from including hidden license bombs in the system by doing development using their internal proprietary code for which they will later extract royalties.

Even libraries that are not licensed for a fee can have significant commercial effects. Much of the open-source code, including libraries that are available, have restrictions attached their use. These restrictions can include a requirement that the source of code developed using the library must be made available or, in some cases, that any code developed using the library automatically assumes the licensing attached to the library. This is the case with the Gnu General Public License (GPL), often referred to as "copyleft." If the system includes proprietary company logic, this can be a significant issue. Open-source licensing can also come with patent issues, compliance obligations, the need to purchase potentially expensive indemnifications, or other legal entanglements. These all need to be checked before approving the use of an open-source license. Sometimes the cost of "free" libraries may prove to exceed that of paid libraries.

Excessive Library Dependence

Libraries are sometimes abused through excessive library dependence. A common way for this to occur is when the library functions are trivialized. Trivialized library functions happen, for instance, when library functions are called that turn out to be nothing more than nested calls to standard functions. Sometimes the trivial function is included to have a function that fits within a naming convention. At other times, the function may have previously contained significant code but now has been replaced by the single function call. Other functions may contain slightly more code but still represent trivial implementations of functionality (preset variables for example).

The challenge with the inclusion of trivial functions in a library is multifold. First, they increase overhead by requiring multiple function calls with their stack operations and context switching, which are not necessary. Second, they obfuscate the code, making it more difficult to read and to identify problems. Third, trivial functions represent a hiding place for security vulnerabilities. This is sometimes through being another hook that can be leveraged by malware. Other times, misconfigured variables or other coding vulnerabilities can be hidden in the function. These often get overlooked in a code review because they are contained within a library, whereas if they had been in the actual code the issue would have been readily apparent.

Excessive library dependence can also manifest itself when a developer uses library functions to implement code that could have been more elegantly and securely implemented using the language itself, without using a library. Often these unneeded uses of the library include some intricate code gymnastics to make the library function fit the problem needs.

Finally, excessive library dependence can become a problem when dependence on a specific library causes a developer, or designer, to overlook other libraries that may be better suited to the problem being addressed in system. One of the reasons that there are many different libraries is because different problems are better addressed in different manners. Limiting development to a single library limits the ability to apply appropriate tools. It is a balancing act, just the right amount of dependency on libraries but not too much.

Dependency Issues

Another challenge with libraries is sorting through dependencies. Often, libraries are created, especially open-source libraries, that are built upon other libraries. These in turn can be built on top of other libraries. In the initial evaluation of a library, it may not be readily apparent that there is such a

cascading dependency. It is only when the library is loaded or accessed that the issue is noticed. It is also possible that the dependencies may not even be noticed during development. Often, the package management system used to load a library has the ability to automatically calculate dependencies and load them as a part of a package update or installation. If no one is paying attention during the install, the dependency may totally escape notice. These are **hidden dependencies** (see **Figure 13.1**).

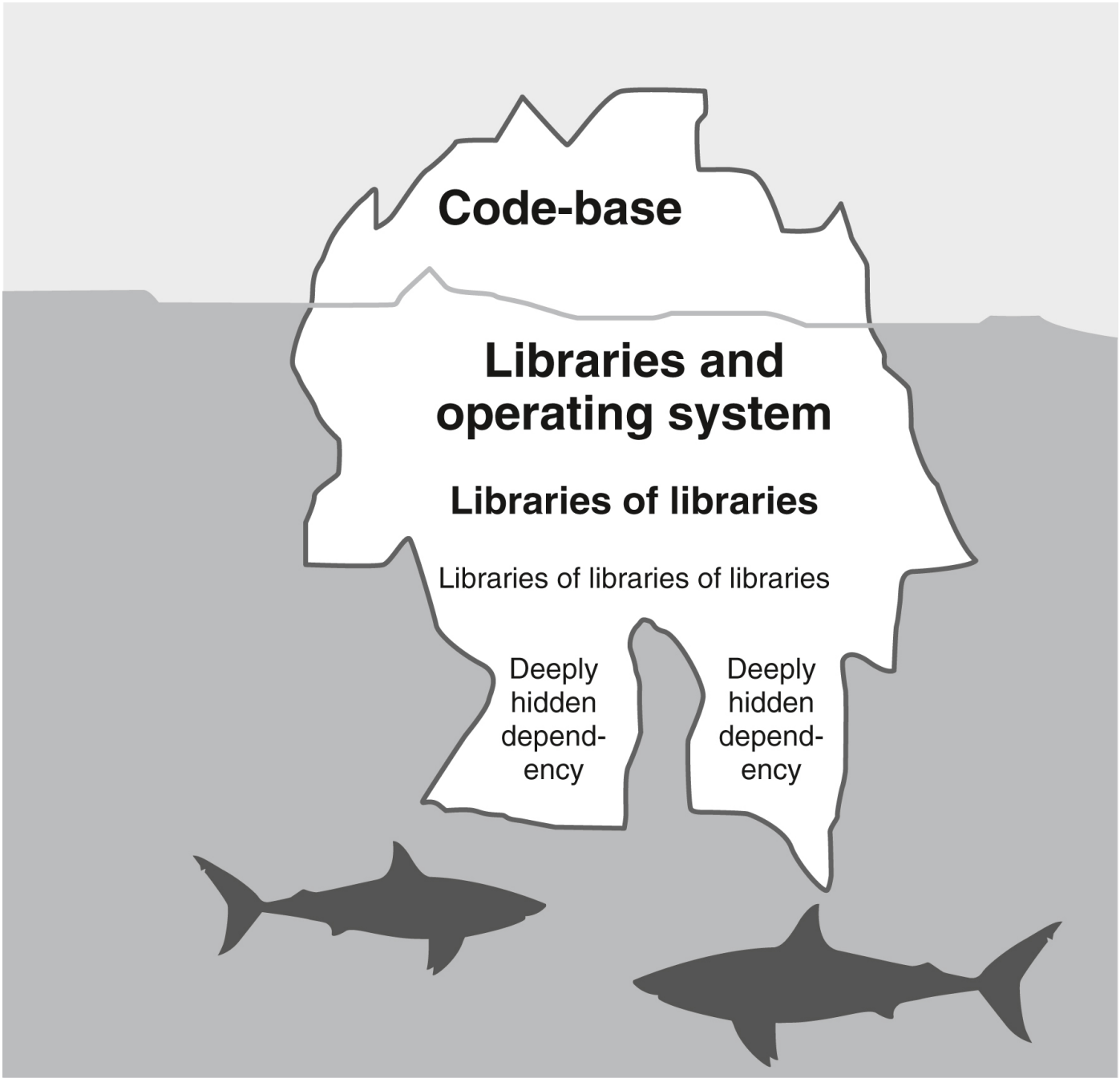


FIGURE 13.1 Lurking Dependency Issues Are Like an Iceberg

Description

The testers may also use a package manager to do the installation on their system, and it is only when the dependencies become a problem that they are discovered. By then, the problems have already begun. Not being aware of multiple layers of dependencies can also cause security issues—what we don't know may carry security risks.

Sometimes lurking dependencies bring licensing challenges; at other times, they simply bring unwanted behaviors and make troubleshooting difficult. Occasionally, circular dependencies appear, where the dependency chain loops around on itself.

Another common issue with dependencies is in managing dependency version control. Libraries, like any other software, are version controlled, and there are differences among versions. The versions need to be kept up to date, as with any other software, to ensure that the latest vulnerability and other fixes are in place. However, if a library with a dependency hasn't been updated, it can force the system to retain an outdated version of that dependency. This can be an especially challenging issue if the dependency software was updated because of a security vulnerability, but now as the library in between isn't updated, the vulnerability lives on in the system even after it was "fixed."

This can get to be a real problem if multiple libraries share the same dependence, with some updated to support the new version and others still matching an older version. It isn't unusual that a system is forced to maintain two sets of a library simultaneously to deal with version dependence issues. It can get tricky pointing each user of the library to the right version without confusion.

Another question is whether a hidden dependency has been updated will even be discovered. Because dependencies are hidden, they are not on a watch list of software versioning to be tracked by the configuration management system (see the next chapter). There is no way of checking what is unknown.

Inherited Vulnerabilities

One of the biggest challenges with libraries and their dependencies is that of **inherited vulnerabilities**. Any vulnerabilities that are present in a library are present in the software developed from that library. Depending on how the library is implemented, compiled, and linked, the inheritance of vulnerabilities can apply not just to the specific functions that were used, but to the entire library package.

Finding vulnerabilities in library functions can be challenging. While the standard techniques of vulnerability mapping and assessment can be helpful, not knowing exactly how everything is designed in the package can limit their effectiveness. Fortunately, other users of library packages can help to identify and flag vulnerabilities, but those efforts may not be timely, and communicating those issues can be slow and confusing.

With some libraries, testing can only be done using a black box method where the testers are unable to see how the code operates. Being able to see how code functions helps testers to design thorough test cases. Other libraries, especially open source, provide access to the code, which can help.

Another thing to remember with libraries is that if you can discover their vulnerabilities by inspecting the code and testing them, then so can everyone else. Despite all of these concerns, the use of libraries is still a good idea. The challenge is doing it carefully and correctly.

Operating Systems and Architecture

Operating systems are an important part of any software system. The choice of operating system to use for a particular development effort is often driven by considerations other than the operating system itself. Sometimes organizations have standard operating systems that they use for all their systems. This is done for operational reasons because it simplifies many operational issues. At other times, the choice of the operating system is driven by the choices of tools and subsystems selected. In a limited set of cases, the choice is actually made because of the capabilities of the operating system itself. Usually, this more applicable with real-time and embedded systems. Without any of these constraints, the choice of operating system devolves to the preferences of the designer or developer. Among the major operating systems, for most applications there are no significant reasons why one operating system need be chosen over any other. All offer advantages and present issues, none of which would demand or preclude their use.

For any operating system, the major concerns are vulnerabilities that are included with the operating system and the ability of the developers to properly use the operating system. Because operating systems are commonly used by many different people, solid mechanisms are set up for discovering, reporting, and patching vulnerabilities in the operating system. For vulnerabilities that cannot be patched or are of use, guides are available to explain countermeasures and controls for managing those vulnerabilities. Along with ensuring that these are followed, the best defense for weaknesses from the operating system is to have the developers well trained on the same operating system. Time and resources spent there can alleviate significant future concerns.

There is some thought that containerizing applications is a way to eliminate exposure to operating system vulnerabilities. While there is certainly merit in reducing the number of systems sharing an operating system and in simplifying the deployment of systems, containerization itself doesn't eliminate most issues. Additionally, methods for breaking out of containers are known. In addition, the major orchestration tools used with containers tend to be weak in accounting for security concerns and make mistakes, such as injecting security data in plain text, not supporting multiple security configurations, or failing to correctly set up secure access. While there are controls for these issues, the

area has significant work to be done in considering security as a part of containerization and orchestration.

Another big challenge that needs to be addressed with operating systems and development is that of keeping up with and supporting development across multiple versions of an operating system. The more an application becomes dependent on specific and proprietary features of an operating system, the larger the support tail is going to become for maintaining the system. With an in-house development, this can often be relieved by requiring a specific version of the operating system; with commercial systems and large organizations, this can quickly become impractical or expensive.

Cloud Environments

One of the biggest trends in systems is the development and deployment of systems in a cloud environment or using **cloud services**. Cloud services are defined as having access to shared IT resources such as network, servers, storage implementations, applications, or services on demand. Cloud offerings may be internal to a company, or they may be provided by an external vendor. Primarily the concept that is resources are available quickly and shared with others (internal or external). Sometimes cloud services are referred to as "using someone else's computer," even if that someone is an internal provider. Cloud services also introduce one or more layers of abstraction to the system, whether as virtual machines or as services.

Cloud computing services are provided by data centers with large numbers of servers (for computing power), large amounts of storage (for files), big network bandwidth (so many people can connect), and the ability to provide services of one of three different types to either in-house or external clients. (See Box 13.1.)

BOX 13.1 Cloud Service Models

- **Software as a service (SaaS)**: Clients purchase subscriptions to use specific software that is hosted and administered on an external provider's cloud infrastructure. Clients use the software but neither administer nor maintain any of the infrastructure or the software.
- **Platform as a service (PaaS)**: Clients contract for and utilize a provided service to construct and deploy software applications. These services can include data storage (relational and non-SQL), authentication, communications, computation, neural networks, and many more. Clients are responsible for developing and deploying their applications, configuring them, and maintaining them on the provided infrastructure. The applications may be built entirely on the cloud or may be built off the cloud using cloud-provided services. Clients will not have capability to manage or modify the underlying infrastructure such as servers, storage, or network resources.
- **Infrastructure as a service (IaaS)**: Clients contract for storage, servers (processors), and some network capabilities that they can

configure and maintain. The servers are provided as virtual machines accessible to the clients as if they were remote physical devices. Clients are not given access to the physical cloud infrastructure, which is likely to be shared with other clients.

Applications can be developed and deployed as a **cloud-hosted application**, in which case there is little difference between how the application would be hosted on basic servers. This is what is referred to as a "forklift" replacement, in which the same systems and software are replicated from physical servers and networks to a virtual network and servers in the cloud as a complete system. The system is then used and administered in the same manner in as it would have been outside of the cloud. The most significant differences would be to processes and responsibilities for security and management. From the developer perspective, little needs to be done differently as a result of the application being in the cloud. Cloud services often have server and other services deployment automation built in. However, such automation means less customization of configurations—customers get to choose from a list of available servers or services without much opportunity to customize.

Applications can also be created to be **cloud native**. Cloud-native applications are designed to take advantage of all of the capabilities that the cloud has to offer. They can be based on PaaS or IaaS types of cloud infrastructure. The applications will make full use of cloud-provided services, such as storage, authentication, as well as infrastructure services, such as spinning up additional servers on demand to balance the load. Cloud-native applications require the developer to have significant knowledge of cloud capabilities and application. The management and maintenance of cloud-native applications is going to a different experience as well.

Converting from an existing internal software and services IT infrastructure to a cloud-native service can take significant amounts of planning, time, and effort. It is no different than when companies need to streamline their multiple applications and services. There are many, many advantages of using a cloud infrastructure from decreased hardware, services, and personnel costs through increased availability of resources when needing to scale up systems and services. At the same time, any company needs to assess the risks and restrictions before jumping straight into "we are moving to the cloud." In addition, the organization needs to consider how customers, internal personnel,

any remaining internal applications, and any applications not managed by the company will communicate with the software applications that have been transitioned to the cloud.

Cloud Security Considerations

Regardless of when engaging an internal or external cloud provider, the security risks are more numerous than in a dedicated server infrastructure. Because cloud environments consist of large amounts of interconnected physical devices (servers, routers, firewall, and other physical devices), they can present a large number of potential attack surfaces. These large infrastructures are then partitioned into smaller services or sections to be contracted out to clients. In this way, there are many different client systems, many of which have external portals or access that attackers can use to attack either one client cloud section or even attack the underlying cloud infrastructure. How seriously any one client takes security of their virtualized area of the cloud infrastructure can impact other clients in their virtualized environments on the physical servers and services. A lack of security in one virtual compartment can lead to a cybersecurity breach, and from that point other virtual machines on the same set of physical machines could put at risk for a breach.

In addition to the security vulnerabilities due to clients who do not keep their cloud infrastructures or platforms secure, the cloud providers are faced with constantly evolving tasks to keep the underlying, large supporting infrastructure secure from attack. It is often true that the more complex something is, the more parts there are that can go wrong. Just one configuration set incorrectly, and data are now visible that should not be.

A significant number of security breaches in cloud environments have resulted from clients failing to apply proper configurations to their provided cloud environment. In some cases, this results from the cloud being used as a supplement to internal applications that require access to the cloud resources. Because this can be complicated in some cases, the defaults are left in place with the most open access made available. Unfortunately, this means access becomes available globally to anyone who looks.

Cloud Services Engagement

When considering moving to any type of cloud service, what the service provides, what the service requires of clients, and what maintenance, recovery, and security is available from the provider should be investigated. Just like with

any service being purchased, the purchaser needs to be careful about what is being bought.

Internal cloud services may be engaged either on an agreement-by-agreement basis where a business group within a company pays for time on cloud services. Alternatively, an internal cloud service may be available on demand for a business group and paid for as part of a central IT budget.

Engaging external cloud services involves a contractual agreement defining many criteria including what type of service is to be provided (SaaS, PaaS, IaaS), what size of services will be needed, how many people will need access, and what level of security is required. Contracting for an external cloud service also needs large amounts of legal and service-level agreement specific requirements. Clients have been put in disastrous positions when cloud providers shut down without notice, leaving their clients without business-critical IT services and no way to get their software or data back to set up their own systems again.

Details on recovery, security, maintenance, and other expectations must be covered in any agreement with an external cloud provider. Depending on what security requirements a business needs to meet (such as NIST 800-53 or NIST 800-171), some cloud providers may be unable to provide the required flexibility. How security items such as authentication and encryption are implemented may be limited in a cloud environment.

Requesting or ordering a standard virtual server with a standard operating system (such as Windows Server) means it can be deployed automatically and quickly by the majority of external vendors (such as Google, Microsoft, or Amazon). However, when using any cloud environment, some things can be done and others cannot. For example, the specific operating system that is desired may not be compatible with the cloud environment without a specific contract and a quite a bit of work including specific tools or even new software tools.

The cloud provider may require specific security settings or configurations in order to keep the overall infrastructure safe and secure from possible cyberattacks. Some of such settings may have an impact either to how a client wants to set up their application in an IAAS cloud offering. It may even make it not possible to run the client application on the cloud infrastructure without a great amount of software changes.

Cloud Development Practices

Doing software development for any cloud infrastructure (internal or external) follows the same processes as development of a single software application or service. A business need is identified, requirements (functional and nonfunctional) requirements are captured, a design is determined, the product is created, testing happens, and finally the product is given to clients to use. (More details in this process have been reviewed elsewhere in this book.)

Using a cloud infrastructure means cloud specific requirements will be part of the requirements gathering phase. These requirements will then be incorporated into the product design and final product creation (development).

Also programming and development practices need to be applied when developing for the cloud. Each cloud provider will have its own unique requirements for how to use its infrastructure. These requirements will need to be incorporated into and merged with the organization's coding practices.

Coding Practices

Good coding practices can contribute significantly to developing a secure system. Coding practices are the patterns used by developers in developing their code. Good coding practices include a broad range of habits from ensuring that the code is well documented to ensuring that the formatting of the code is consistent. Simple practices, such as determining whether a curly bracket that starts a set of statements goes on its own line or is at the end of the previous line with the conditional can be important. The choice of where the bracket goes is not important; what is important is that wherever it goes, it goes there consistently.

Good coding practices also require management of the ego and attitude of the developer. Some developers see the code that they write as an opportunity to demonstrate their prowess in applying all of the advanced and obscure features of a programming language. Other programmers refuse to document their code, believing that if a future maintainer cannot understand what the code it is doing, it is a failing of the education and intellectual abilities of the maintainer. In reality, if a programmer cannot explain their code in simple direct terms that can be understood by someone else, it is the programmer who has failed and perhaps doesn't really understand what he or she has managed to hobble together. Good code is clear, straightforward, and direct. It does not rely on layers of indirection. It is complete without extraneous statements.

Standing Operating Procedures (SOPs)

Standing operating procedures (SOPs) are written guides for how an organization completes its processes. As for every other process in the organization, it is important that processes related to software development and coding are explicitly written and not just "understood." Significant processes to include in the SOPs are how to originate a block of code, how to check out code from a repository to work on it, the unit testing process expected for a block of code, and how to check in or replace a block of code that has been worked on. The SOP should include checklists for the processes.

The SOP will include processes and tasks that the team performs on a daily basis as well as those that occur more infrequently. It would seem to some that there would be no need to write down and document daily tasks as performing them becomes second nature, and the routine is quickly memorized. However,

there are going to be new people on the team, including contractors and people from other groups within the organization, who may not be familiar with the way the team operates. More importantly, however, is that as procedures are followed, what was done “last time” becomes the routine of what is done forever after. If a step was missed last time, without a written procedure, that step will be routinely omitted, and only be recognized has having been missed when it becomes an issue. This is why airline pilots use checklists when doing their preflight checks.

The process of writing the SOP is also valuable in that it gives the organization an opportunity to question and examine what it is doing. During this process, programmers have the opportunity to recognize steps that should be included as well as to question the value of extraneous steps.

Programming Manuals or Style Guides

Programming manuals or **style guides** are used to codify the coding practices of the organization. These can be incorporated into the SOP or developed as a stand-alone document. Which specific items are included in the style guide and which in the SOP may vary from organization to organization, but in general, the SOP addresses process and language independent issues, while the style guide addresses issues specific to one or more languages. In some cases, the programming manual may have general sections that apply to all languages used in an organization but will also have specific sections that apply to each language. Often languages are developed to support the coding styles of one or many of the originators of the language. This style may not be explicitly stated or may not be specific enough for an organization. The programming manual addresses this.

Having a common programming style in an organization is important for security, functionality, and supportability. While some make the claim that allowing programmers to each develop with their own style increases productivity by releasing the programmers to work in a manner that most suits them, it just doesn’t. It may allow one programmer to be more productive, but overall it reduces the productivity of the organization.

Any code that is written is going to need to be read by multiple people. It will need to be read in code review, it will need to be read while being maintained, and it will need to be read by people who are developing compatible code or code that extends the functionality. Code written in nonstandard or a variety of styles is more difficult and takes more time to read. The different code styles can also lead to misunderstanding how the code functions. Obfuscated code

doesn't serve any purpose other than to attempt to show the prowess of the coder. Differing code styles can allow vulnerabilities to slip into code undetected.

The programming style guide should include a section on the documentation to go into the beginning of the file. This will generally include the title of the file, the purpose of the file, the author of the file, versioning information, and other information that the organization finds important and useful. An example of a file header comment section is provided in **Figure 13.2**.

```
/*****************************************************************************
*  FILENAME:  foveal.c
*
*  DECRIPTION:
*         This program takes an image file and performs a foveal transform
*         on the contents.  Each pixel is enhanced by positive feedback from
*         the immediately surrounding pixels and negative feedback from pixels
*         over a broader region.
*
*  ARGUEMENTS:
*           -f filename  : specified the filename for input.
*
*           -o filename  : optional output file. Default output will be to STDIO.
*
*           -r int, int  : specifies the width and height of the filter.  Default
*                          is 10, 6.
*
*  USE:
*        fovea -f filename -r 10, 6
*
*  AUTHOR:  Erik Fretheim
*
*  VERSION:
*           1.0 - 15 Feb 1989
*
*****************************************************************************/
```

FIGURE 13.2 Typical File Header Comment Section

The style guide will also specify how comments are to be formatted throughout the code. The manual should also provide guidance on when comments are appropriate or required. The consistency of comments aids in the readability and maintainability of the code.

Another significant item to be included in the programming manual is the naming conventions for variables, functions, and other artifacts. This would

include both the visual presentation of the names (see Box 13.1) and the composition of names.

BOX 13.2 Naming Presentation

Snake Case—Words or portions are separated by underscores:

variable_one
snake_case

Kebab Case—Words or portions are separated by dashes:

variable-one
kebab-case

Camel Case—Words or portions are distinguished by capitals in the middle of the name:

variableOne
camelCase

Pascal Case—Words or portions are distinguished by capitals, including the first word:

VariableOne
PascalCase

Yelling Case—Words or portions are in upper case with or without underscores:

VARIABLE_ONE
VARIABLEONE
YELLING_CASE

There are many considerations in the composition of names. In some cases, the programming language will dictate elements of name composition, put constraints on the type and number of characters and symbols which can be

used in names, have an associated “traditional” style, or prescribe the use of “namespaces” or other conventions. Organizational conventions for naming are placed on top of language constraints. Typical conventions include starting the name with an indicator of the use of the name, such as for a function or variable, starting the name with the data type, or starting the name with the section or package in which the name is used. Some conventions use an underscore or two to distinguish private variables and functions (_myVariable). Applying different style conventions to the naming of variables and items can help prevent programming issues, such as redefining a global variable or calling the wrong function because of different ordering of include functions.

The style guide should also address issues such as the use of white space, indentations, and the placement of curly braces. Sometime languages drive these conventions, such as with Python’s bizarre use of indentations to indicate code blocks, but usually these are left to the developer. In an organization they shouldn’t be. The organization’s coding conventions should dictate the standards for how much a block of code is indented and whether it is done with spaces or tabs as well as where braces are placed at the beginning of a block of code (see **Figure 13.3**).

```
for (i=0; i<bigNum; i++){
   bob[i] = getStuff();
}

for (i=0; i<bigNum; i++){
   bob[i] = getStuff();
   }

for (i=0; i<bigNum; i++)
{
   bob[i] = getStuff();
}
```

```
for (i=0; i<bigNum; i++)
   {
   bob[i] = getStuff();
   }

for (i=0; i<bigNum; i++)
   {
      bob[i] = getStuff();
   }
```

FIGURE 13.3 Options for the Placement of Braces

Description

Error handling, such as attempting to open a file when it is not present, should be done in a standard manner across an organization. The methods for handling errors need to also include considering how the program user should

be informed of the error. This has significant ramifications for security. For example, I recently attempted to pay a bill online. The system gave me the option to pay without logging into my account. I did need to enter the account number and zip code, but no password. The program failed, but rather than simply telling me that the service was unavailable, try again later, I was given an internal error screen from the database that not only showed the account number, but also other details that I should have only seen if I was logged in. A hacker could have used this flaw to enter random account numbers and obtain personal information about many of the customers. I informed the doctor's office about the issue, but never received a reply.

Many organizations also include conventions in the programming manuals about which functions and libraries should or should not be used. Many functions, such as the C function strcpy(), are recognized as inherently risky and prone to producing insecure code. The manual will list those functions that should not be used and provide acceptable alternatives or required methods of securing the functions if they absolutely can't be avoided.

Programming manuals specify acceptable code libraries. These are libraries that ideally have been vetted to be secure and to not have excessive nested dependencies. Even in cases where multiple secure libraries can accomplish the same functions, it is valuable to specify which one should be used in the organization. This simplifies maintenance, adds consistency to the applications, and prevents mix-ups when two different functions of the same name are called differently or provide different results depending on which package they come from. Overall, a smaller set of acceptable libraries is easier to maintain and support.

Fail-Safe Logic and Coding

One of the principles introduced in the architecture section was to design systems for graceful degradation and to fail in safe modes. These principles need to be carried over into the development and programming phases as well. Writing fail-safe code requires that the programmer consider what happens with unexpected inputs and values, even if it is "impossible" for them to occur.

An example of writing fail-safe code is including a default with a "switch" or a "case" statement. In these, after all expected cases have been accounted for and handled, a "default" needs to be set that is never expected to be used but provides a safe or graceful way to exit the code without enabling vulnerabilities or allowing errors if encountered. The same kinds of fail-safe thinking can be applied to nested if-statements.

An area with a large potential for failures is the interfaces between modules. In programming these interfaces, significant consideration needs to be given to how to respond to malformed messages or invalid data on the interfaces. There are several options. Invalid messages could be ignored. Invalid messages could cause a request to resend the message. Malformed messages could reset the connection. Each of these may be appropriate in different situations. Considering the proper response needs to be based on the potential harm that can result from the selected response.

Integration Issues

One of the biggest programming challenges and the source of many errors is the integration of code into a larger assembly. Anytime sections of code are written by multiple people, there are going to be differences of understanding, interpretations, and assumptions. These will lead to problems. The best means to reduce those problems to manageable levels and numbers is through good communications practices.

Clearly, communicating the interfaces between modules in the design is going to be the first step in reducing the opportunity for mismatched expectations. However, this is seldom sufficient. Issues will arise that the designer did not anticipate or that are based on cultural or other differences. The amount of communication between developers or teams working on different modules is going to directly impact the ease with which the pieces integrate. Instead of developing a module with an interface and waiting until integration time to see how well they work together, the developers can share their early implementations and notes about what they are doing. This can catch many situations, such as when one programmer is thinking in radians and the other in degrees. In many cases, the programmers may be able to share snippets of code used in implementing the interface between the modules. An organization with a collaborative environment is going to have significantly fewer integration issues.

Another issue to avoid is the big bang integration. In this approach, everyone develops their part of a system independently and simultaneously. One integration day, the parts are all matched together, and the test begins. The expectation is that because they were all well designed, with well-specified interfaces, everything is going to work correctly together. The bang is the noise of failure. But now, instead of finding that there is a module that doesn't work well with the others, there is only confusion. Nothing works, but everything is right, so where is the problem? By bringing everything together at once, the

ability to easily identify where the issues lie becomes quickly obscured. A better choice is to introduce the modules into the system individually and identify issues in a more isolated manner.

SUMMARY

The development environment plays a significant role in the ability of an organization to produce secure software systems. All of the aspects of the environment are factors, including the organizational culture, the tools used, the use of libraries and code repositories, and the standards and processes used by the organization. To counter environmental challenges, organizations develop SOPs and programing manuals or style guides, which ensure a degree of consistency across the organization and helps create secure code.

KEY TERMS

bespoke software
cascading dependency
cloud native
cloud services
cloud-hosted application
code or function libraries
commercial off-the-shelf (COTS)
COTS platform
glue logic software
good coding practices
hidden dependencies
infrastructure as a service (IaaS)
inherited vulnerabilities
near-shore development
off-shore development
on-shore development
open-source software projects
outsourcing
platform as a service (PaaS)
programming manuals or style guides
secure tools
security culture
software as a service (SaaS)
standing operating procedures (SOPs)
unsecure defaults

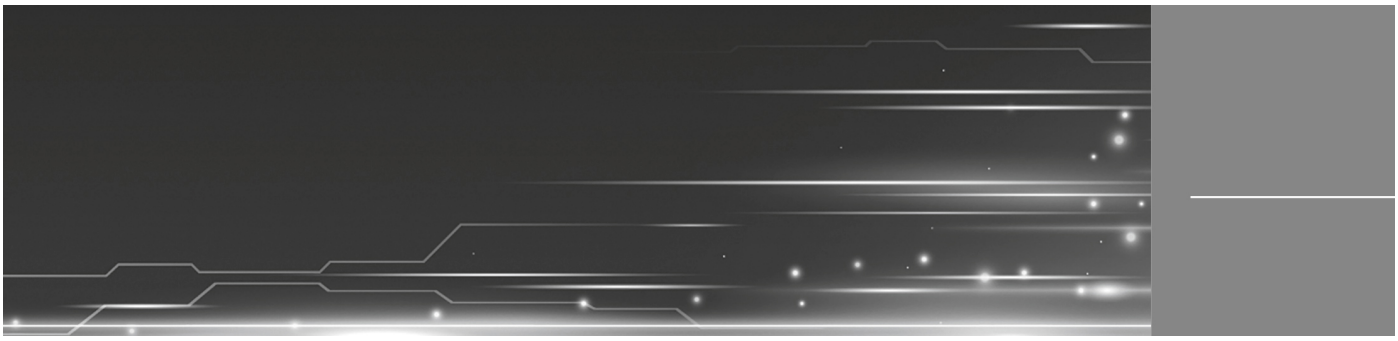
References

Cox, Joseph. 2021. “How Hackers Used Slack to Break into EA Games.” www.vice.com. June 11, 2021. https://www.vice.com/en/article/7kvkqb/how-ea-games-was-hacked-slack.

Mell, Peter, and Tim Grance. 2011. “The NIST Definition of Cloud Computing.” NIST. https://csrc.nist.gov/publications/detail/sp/800-145/final

ST. 2020. “Security and Privacy Controls for Information Systems and Organizations.” Csrc.nist.gov. September 23, 2020. https://csrc.nist.gov/publications/detail/sp/800-53/rev-5/final

Ross, Ron, Victoria Pillitteri, Kelley Dempsey, Mark Riddle, and Gary Guissanie. 2020. “Protecting Controlled Unclassified Information in Nonfederal Systems and Organizations.” Csrc.nist.gov. February 21, 2020. https://csrc.nist.gov/publications/detail/sp/800-171/rev-2/final



© d1sk/Shutterstock

CHAPTER 14
Configuration Management

CHAPTER OBJECTIVES

- Apply version management to software systems.
- Use configuration management to track the components of systems.
- Describe the use of configuration management in an operational system.

Introduction

Configuration management is an area that has changed and grown over the years because of the increasing complexity of systems and the environments in which they operate. Architects, developers, and operators of systems has increasingly recognized the value of tracking configurations in many ways and have demanded more from configuration management.

Initially, the concern of configuration management was ensuring that when a software system was being developed or sold, it was composed of specific versions of components. Configuration management was used to track the components through the development process and into the final product. This ensured that the versions of components assembled had been tested together and contained the intended functionality.

Over time, as systems have grown in complexity, organizations have recognized the value of configuration management, and its application in many organizations has expanded with expectations for additional capabilities. Configuration management is now applied not only to the software system itself, but also to the environment in which the system operates, including hardware, operating systems, configuration settings, and other elements. Every component version, operating system version, and type of hardware that a specific software relies on to run at peak efficiency is specified in configuration management documentation.

Configuration management is a method of product change control. Each component (part) of the final product will go through many changes while being created. A cohesive, documented, and communicated configuration management process is essential so each component and the final product's necessary configurations are captured. Configuration management is used in organizations to address the compatibility of different assemblies of the components of a system as well as tracking configurations that have been shipped to different customers.

On the user organization side, configuration management is applied to tracking the versions and components that have been deployed onto the network and systems as well as tracking what should be installed and the changes in what has been installed over time. Advanced configuration management systems provide tools to track the presence and state of installed systems.

A key element of configuration management is version control or version management. Version control is the low-level tracking of the different components involved in configuration management. In version control, labels, also called tags, identifiers, or numbers, are used to track the process of the component through the development and deployment process.

Version Control

Version control is the process of tracking a component through the development and deployment process. Version control is used to track the implementation, changes, testing, and documentation of a component. It is used to track who has worked with the component and to differentiate different instantiations or versions of the same component. The key to version control is the assignment of a version label (also called a tag, identifier, number, etc.) to each different instance of the component. The version label is changed each time the component is changed. Information about the component, such as who made the changes or testing results, is attached to the version label. This allows for tracking of the state of the component.

A component may have multiple **version labels** active at the same time. These may follow different labeling schemes as well. During development, it is not uncommon for multiple people to work on a component. Each developer's version of the component will have its own label. This label may be developer specific (Bob-12) or simply a unique serial id (#123). As the work of the different developers is brought together, the differences among the versions are checked, and the versions are merged. The labeling scheme applied to the merged code may be simply serial labeling, or it may be given a version label in a different format that will be used in configuration management.

Version control systems, also known as revision control or source control, are automated systems used to add management, consistency, and automation to the version control process. Examples include RCS, CVS, Visual Safe Source, Mercurial, GIT, and many others. These systems track the components and their changes. While many models are implemented in the different systems, most have features for adding code to the repository; checking it out by different users; comparing versions of code, including tracking differences; merging multiple versions together; and checking in changed versions. The version control systems track who checked out and checked in the code. In addition to the code, the version control system can be used to track documentation and other electronic artifacts associated with a component.

Configuration Management in Product Development

Configuration management relies on good version control or management. It is not possible to have configuration management unless there is information on the versions of the software products and each product part that all maintenance, operations, and information security people can use to make sure all the part versions are compatible. Initial compatibility information is provided by the development team, confirmed in the testing processes, and managed by the product managers. The compatibility information indicates which versions of elements of the software system are known to integrate successfully together. Using this version information, the maintenance team can confirm they received the right versions for the product they need. The operations team will use part version labels to watch for any configuration changes in the live system or server and to monitor whether a version changed in production. Lastly, when new patches containing updated versions of the software parts, the version numbers help in determining what version to take for the update. Needless to say, all this requires a complex tracking effort, and versioning is a central part to effective configuration management.

All of the actions involved in the configuration management life cycle are best tracked using one system, or at least a set of systems that interact. That way the part information (requirements, design, specifications) and any post release issue information are easily accessible for anyone who needs information on the part and product.

Configuration management systems (such as Team Foundation Server, IBM Rational, Ansible, Terraform, or even GIT with extensions) are able to do quite a number of tasks related to storing part information as well as other tasks. These tools either use their own source control system or function on top of another source control system, such as Git. In examining their use, it is often difficult to differentiate between which are source control functions and which are configuration management functions as the two are often tightly coupled.

For software systems development, configuration management tools in addition to the code have access to any type of software part document or artifacts as they are checked in through (registered in) the tool. This includes initial design documents in any form, design artifacts in any form, iterations of the software product, and test cases with test case results. Some configuration

management tools are even able to interface with other tools that can automatically run test cases.

To start, initial part information is entered into a configuration management system. Each new entry is recorded with who inserted the information, a date and time stamp, and a unique label for that version of the part number. You can image how helpful this is when trying to track down a particular change to a part.

Most commonly for software products, each file that makes up the final product is assigned a number as a label. That assigned number or label increments in a predefined way every time the part (software or otherwise) changes. As well as implementing a basic numbering system, in a configuration management system it is possible to create different branches of a part configuration. Particularly in software, we must support multiple different versions of a complete software product for an extended length of time. Every different set of a complete software product has its own branch or lower label designation. Any changes to a part or a set of parts after a complete set of parts making up the final product has been assigned to a branch will take on a label that relates to the base branch label. For example, the first release of a final product may be Rel1, and each part will have the same label or designation but with their unique name. When a second release is created with a large number of changes to Rel1, a branch is created with all parts being labeled as Rel2. Both branches need to be supported until one or the other branch is retired. So we have a Rel1 or Rel 2 branch. What if we have to fix a part on that branch? Then we have to again iterate the label such as Rel2.1 and so on. An example of multiple releases is noted in **Figure 14.1**.

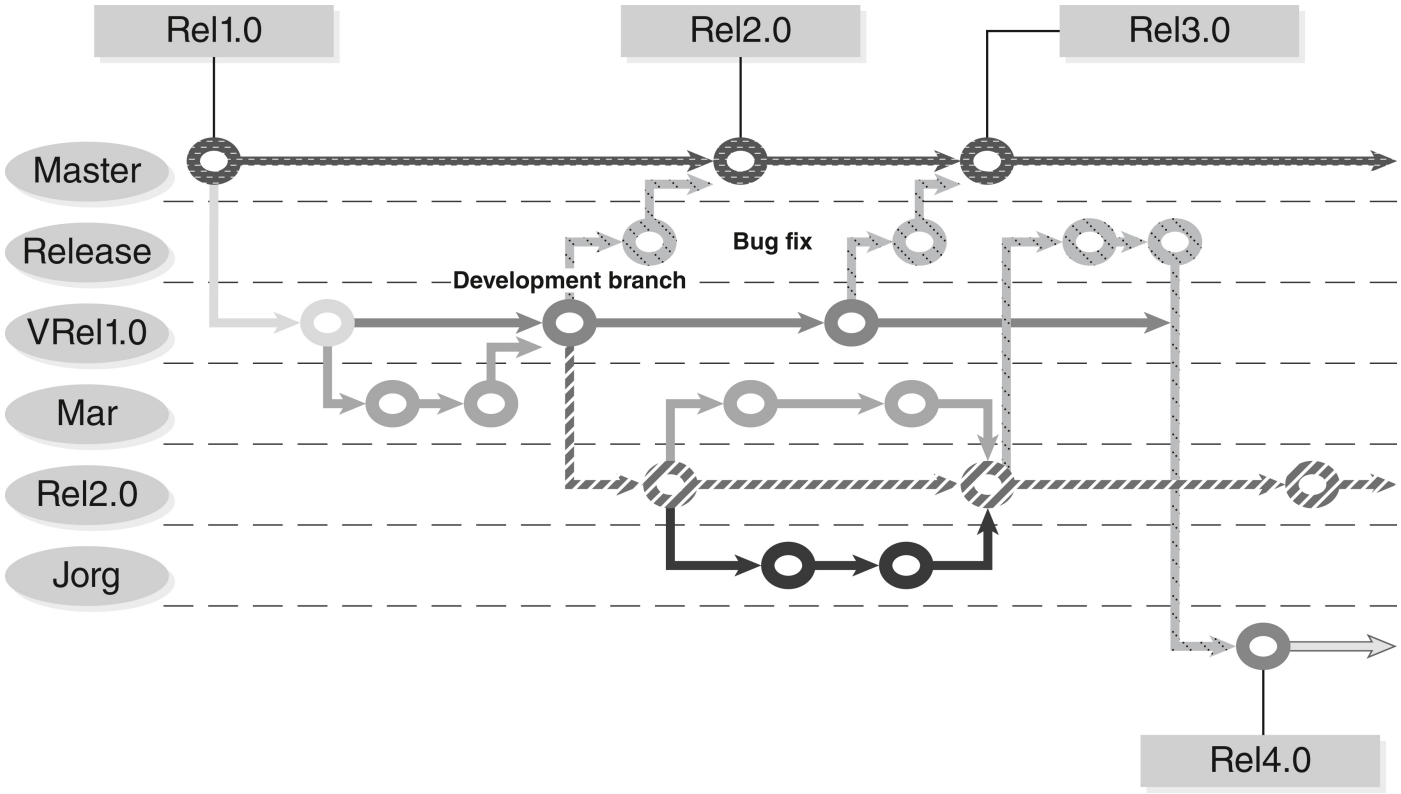


FIGURE 14.1 Source Code Versioning

Description

Using this type of system provides a number of advantages to being able to capture information on changes as they are made to a part. Such systems can be and usually are configured to perform specific tasks and require specific information when using them. One such task is the part labeling that also allows for identifying what parts make up the baseline for the full product. This is sometimes called the bill of materials.

The history of each part, including who created the original part, who made each and every change subsequent to the first part being saved in the configuration management system, and information on every change made, is something that all configuration management systems encourage. Granted the information provided is up to the human being creating or updating the part; however, that is where management auditing of developer input comes in. Having the history of a part and all its past changes, why changes were made, who made them, what changes were made is invaluable both when making a new change and performing root cause analysis for a security issue or any type of failure or issue with a part. Human auditors are responsible for reviewing data quality for part and product information recorded in the configuration management tool.

Configuration Management in Operations and Maintenance

A second major function that some configuration management tools can help with is monitoring and managing configurations on any electronic system (servers, workstations, routers, and other systems). Information regarding many types of software or system configurations on a system can be monitored and even remotely updated. This means there is no need to either have a human being physically go to a server or other device, log onto it, and manually check configurations. As for what types of products or configurations can be monitored, almost the everything on a server or workstation is visible. This provides companies with some wonderful advantages, such as being able to push the latest patches automatically and on specific schedules. In my personal experience, such systems also can be set to deploy patches to specific servers (not *all* servers at one time) on specific schedules. This flexibility helps manage network impact of large-scale deployments as well as allowing deployment to specific devices for testing purposes.

As such configuration management tools have real-time connection to devices, the tool can be used to monitor device health and configuration. This allows operations staff to monitor **configuration drift** and make immediate corrections. Configuration drift is what occurs as the configurations of software and settings changes over time as updates, fixes, and other activities occur without being properly entered into the configuration monitoring systems. Configuration drift may be intentional (done by a human) or unintentional (perhaps due to an unexpected impact from a recent update). Being able make the needed changes to push the configuration back to what is required, either through automated means or human intervention remotely, helps keep company device systems and services in the required configurations. This ability includes being able to push updates and change configurations on operating systems, software, or many applications; monitor device health; change user permissions; and even delete everything from a device. This last option is particularly useful if a device is lost or stolen.

The vast amount of data that gets collected by configuration management tools on company devices helps auditors perform human audits over time to assess many issues, including security issues such permissions, who or what has access, open ports, and software versions or levels. As well, operations,

maintenance, and audit staff have access to historical data that can be used for multiple purposes.

Given the power of configuration management tools that can remotely manage devices tool security is critically important. The principle of least privilege is always used when granting access to the tool. Which functions will be granted to which people must be tightly controlled as well.

Because configuration management tools have such power, their software application security during their development life cycle is even more critical. Often such tools are automatically updated by the supplier or vendor. Updates are pushed automatically to the tool's clients. This can cause a supply chain attack. The term is a little misleading because the tool supplier gets attacked and infected code then gets pushed to customers. Having access to a supplier's source code means an attacker can introduce multiple types of malware that will remain latent until being pushed to clients.

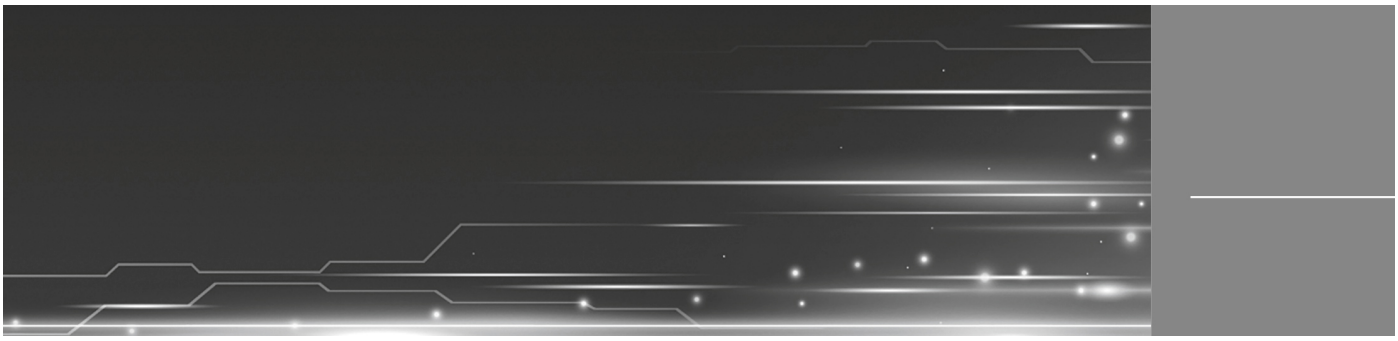
Configuration management tools can perform a wide variety of functions, including helping to manage software development processes, manage software and hardware of actual devices, perform many configuration management tasks automatically, and ensure system security and integrity.

SUMMARY

Software systems are built out of many components, all of which are subject to changes over time. This creates a significant degree of complexity because not all versions of all components will work correctly together in a system. Adding order to the potential chaos relies first on version management, where versions of software are tracked and managed. These managed versions can then be assembled into configurations where the components are known to be compatible and to provide desired capabilities. As the system becomes operational, the challenge of managing the configurations extends to knowing the configuration of each device and what that configuration needs to be.

KEY TERMS

configuration drift
configuration management
configuration management system
version control
version control system
version labels



© d1sk/Shutterstock

CHAPTER 15
Testing

CHAPTER OBJECTIVES

- Differentiate between verification (meets specific requirements) and validation (performs intended functions).
- Discuss test-driven development.
- Outline and describe the components of a test plan.
- Review key roles in software systems testing.
- Describe basic test processes and methods.

Introduction

Ultimately every software system is going to be tested. It can be done by jumping to the ultimate crucible of real-world use. Many programmers feel that the quality of the code they produce is such that this is the only test ever needed. These are delusional people doomed to failure. Testing is a critical element of the secure software development life cycle and needs to be included from the start and conducted throughout the process.

Testing is the process of verifying and validating that a software system or application meets the intended requirements, performs as expected, and contains no unknown defects, flaws, or errors, and no unmitigated vulnerabilities. **Verification** is the process of ensuring that the system meets the provided requirements. It answers the question “Did we build it correctly.” **Validation** is the process of ensuring that the system performs the functions it is intended to perform in an acceptable manner. It answers the question “Did we build what we were supposed to build?”

Both verification and validation will be performed throughout the secure development life cycle. Early in the cycle, a form of validation is performed in the review of the project charter, plan, and requirements. As the development proceeds, the focus then shifts to verification as the parts, pieces, and assemblies are checked to ensure that they meet the requirements as specified. Finally, as the system takes shape and approaches completion, the focus will shift back to validation. This change in focus over time is seen in the classic **V-model** depiction of testing as shown in **Figure 15.1**.

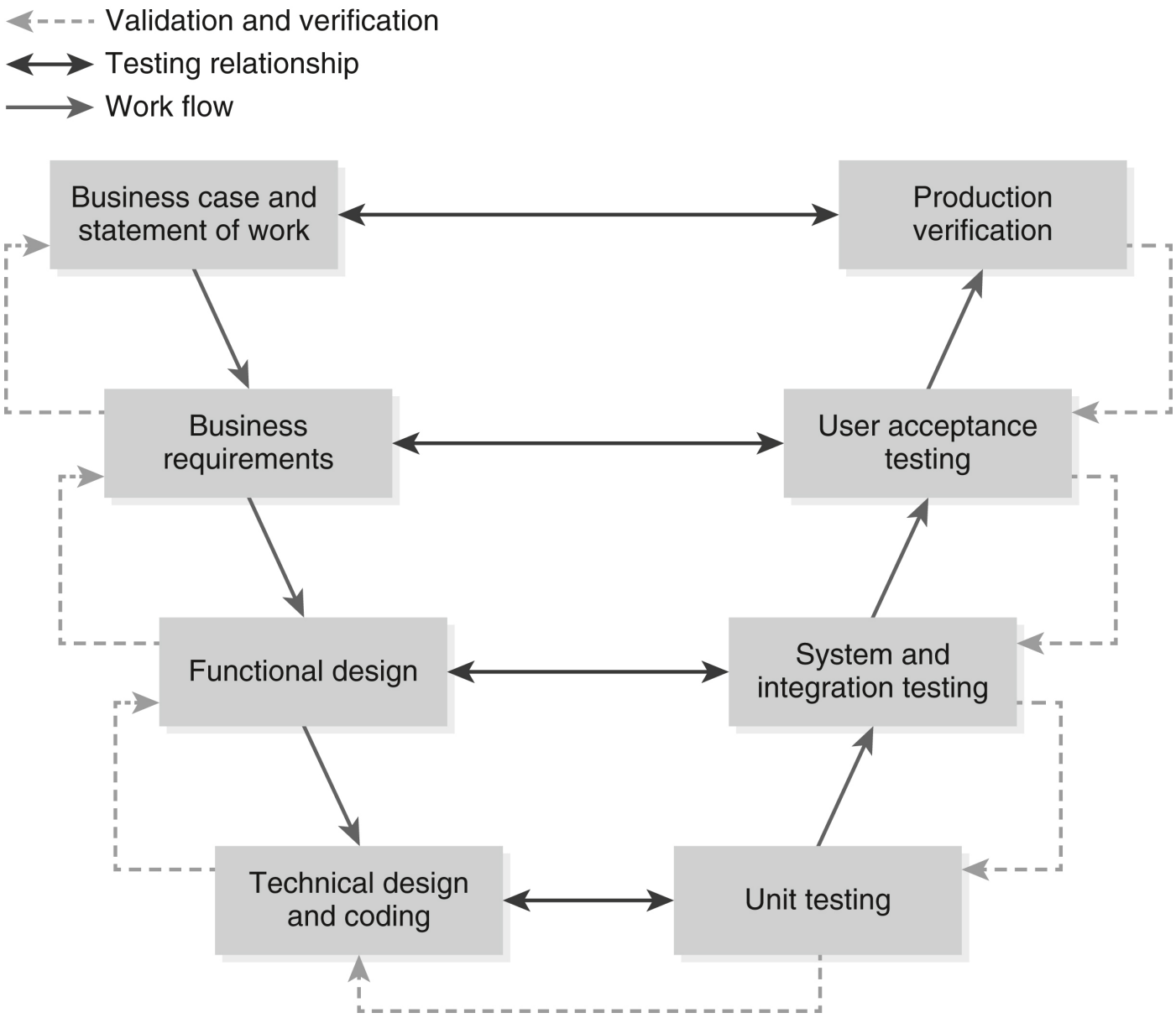


FIGURE 15.1 Testing V-Model

Description

In this model, the relationships between the documents on the left and the testing performed on the right at the top level are heavily focused on validation, while the tests at the lower levels are more focused on verification. Throughout the test process, verification and validation are performed against previous steps to ensure that the development efforts have not wandered astray. However, the same emphasis of validation at higher levels and verification at lower levels applies. In **Figure 15.2**, the secure software systems development life cycle (SSDLC) is imposed on the V-model to help illustrate the relationships within the model: testing, verification, and validation.

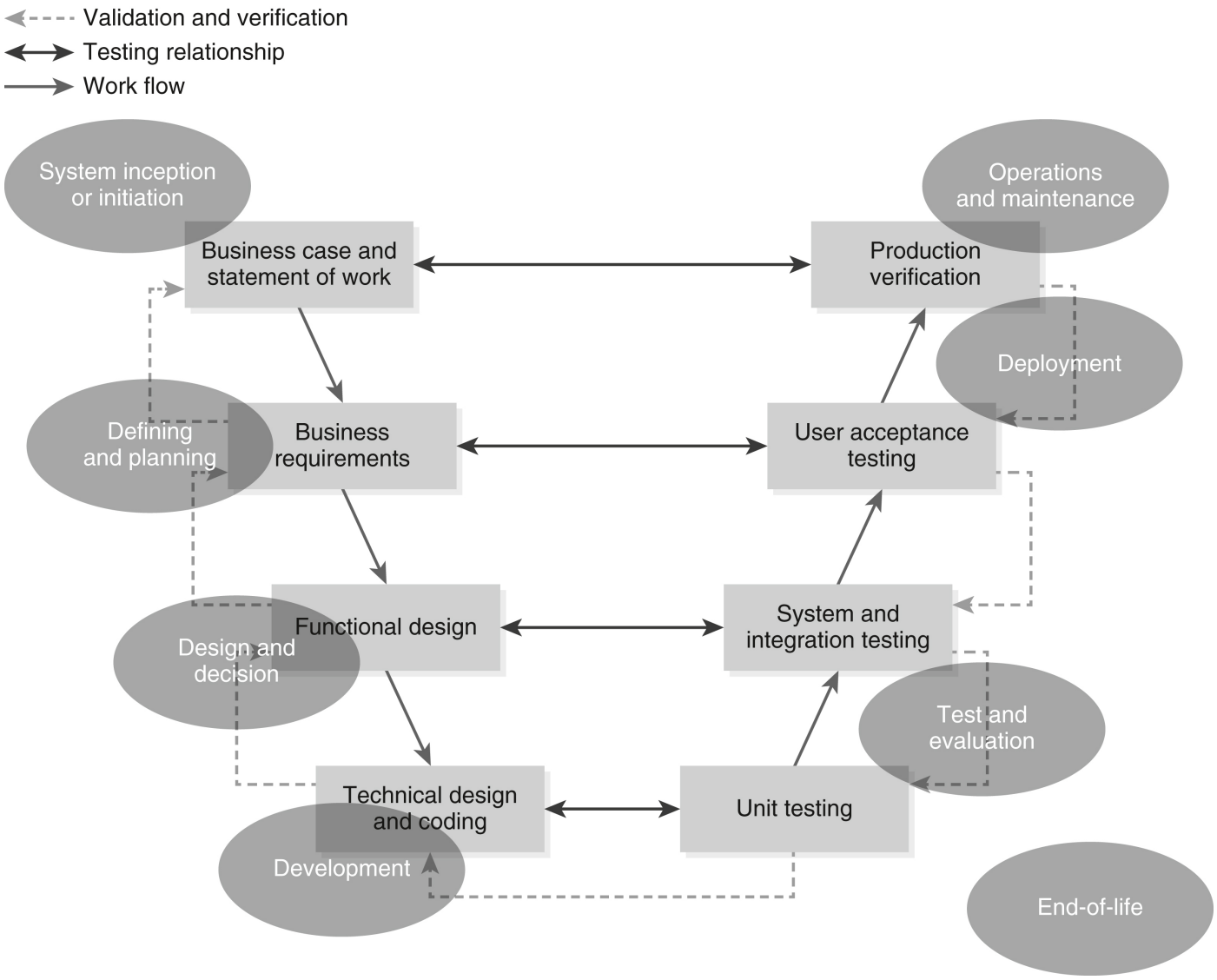


FIGURE 15.2 V-Model with SSDLC Imposed

Description

It is important to remember that even though it is shown as a simple path through the cycle with a specific test at each point, that is not reality, and the simplified mapping is only used to help guide the understanding of what needs to be accomplished.

One of the key realities of systems development is that problems will be found during both verification and validation. These will be errors of both omission and commission. Examples of errors of omission, things that should be done but aren't, would include failing to check inputs, forgetting potential cases or to add a default case, or leaving out a semicolon, which changes the meaning of a code statement. Examples of errors of commission, things that shouldn't be done but are, include adding a logical fallacy, such as "if (A and !A)," which can never evaluate to TRUE, implementing an incorrect calculation,

or adding extra spaces in a Python program to improve the layout but also changes the block of code. The earlier in the process these are discovered and resolved, the less costly that resolution will be. In order to ensure that this happens, considerable thought needs to be put into how testing is going to be accomplished. This means performing the right tests at the right times, having the right people completing the testing, and communicating the results of testing in the right manner. The key to accomplishing this is through the development and adherence to a comprehensive test plan.

To start planning for testing early allows for better understanding of the requirements through the testing lens. Starting early leads to higher probability of increased testing coverage through deeper understanding of what the product is supposed to do and any requirements changes that happen throughout the software development life cycle. Catching an issue during development is anywhere from 4 times to 10 times less expensive than catching an issue in production. Test early, test often is our motto.

BOX 15.1 Test Harness

A test harness is code written to surround a module that provides a range of specific inputs and accepts outputs from that module. The inputs are designed to provide good coverage. The outputs are assessed for correctness.

The test harness is used when the code with which the module will interact is not available or if the module needs intense testing which is hard to complete indirectly using the connected modules. In the latter case, the test harness can quickly and easily present a broad range of inputs and interactions with the tested module.

Having a test team as part of the product team early in the life cycle also has the added benefit of the test team being more integrated into the product team. Very often test teams are viewed as a necessary impediment because the test team has the unenviable job of pointing out planning and product alternative approaches. The job of test teams is to identify all potential issues of the developed software or, since identifying them all is impossible, at least identifying as many as is possible. For many, this part of the test team's job can be seen as judgmental and is negatively perceived by developers. Integrating the test team throughout all phases of the SDLC helps the

development and test teams become a better functioning unit as they develop increased respect for the contributions each makes to the product.

Test-Driven Development

Test-driven development is an important concept in secure software systems development. The key point of the concept is that testing becomes the focal point for development. In a purely test-driven environment, the tests that will be performed, the testing tools, and the test harnesses are developed and implemented before the design and development of the system is begun. Because the tests are developed directly from the requirements and extra care is taken to ensure that the testing thoroughly covers the requirements, the design and development are completed to satisfy the tests.

Test-driven development has the advantage of ensuring that testing is always considered and is done early and often. It can lead to significant reductions in development errors being found late in the process. This approach also ensures close cooperation between the developers and the test team.

In formal test-driven development, the tests are applied cyclically to the developing code.

1. The first test is the simplest test of basic functionality. Based on this test, the developer creates the simplest code possible that can successfully pass the initial test.
2. The developer then adds a test and runs all previous tests as well as the newly added test on the code that passed the previous test. This test should fail, as the functionality included in the test was not implemented (it was not part of the simplest solution). This test failure verifies that the testing is conducted correctly in that it actually identifies the missing functionality.
3. The developer then implements the functionality needed to pass the test in the simplest code possible. Each test is run in sequence until all tests are passed.
4. If there are more tests, return to step 2. Otherwise, move to the next module or set of tests.

As a part of the development, refactoring of the code may be needed to be able to enable needed functionality, to enhance performance, or to improve the readability and maintainability of the code.

There are many variations on the basic theme of test-driven development. Some are focused on larger scale tests driving the development, or on other aspects of the approach. The greatest value of a test-driven approach to development is that it emphasizes the importance of early involvement of testing.

There are also concerns with some approaches to test-driven development. One concern is the focusing on meeting specific early tests rather than designing a comprehensive effective solution for a module. Another is that overemphasis on making the tests work can lead to missing untested issues. Writing excessive numbers of tests and extensive refactoring can lead to longer development times. Finally, a trend that emphasizes unit testing over integration testing and a false sense of security based on all of the “successful” tests are also concerns.

The Test Plan

No matter what approach to testing will be used, every secure software systems development effort needs a test plan. The test plan contains the details of what to test, how to test, and when to test. The test plan also includes information about the standards for how tests will be performed, what inputs to use for tests (test cases), and what to expect from the results and how to interpret those results. The plan may come in the form of a document, or the details may be in an application where the plan, execution of the plan, and the plan results are captured. The goal of the test plan is to confirm that the final software product does what was requested and planned to do and does it properly.

BOX 15.2 Key Components of the Test Plan

- Test Strategy
- Test Goals
    - Release Requirements
- Test Team
    - Team Roles
- Test Tracking and Communication
- Test Schedule
    - Timeline and Priorities of Tests
        - Test Cases
        - Test Tools
        - Test Resources
        - Test Environment

Development of the test plan begins at the very beginning to the SSDLC. It is not something that is created at or for the test phase. The test plan is structured to include all of the key information needed to ensure proper testing.

The Risk-Based Test Strategy

The test plan begins with the **test strategy**. While it may seem to be the best possible test strategy is to do a comprehensive and thorough testing of all components of a software system in all possible contexts, in reality, such testing is impossible. Any software system simply has too many combinations and permutations of inputs, states, and activities in to test everything. For this reason, developing the test strategy begins with a risk analysis. The team will then carefully think through the optimum test strategy to ensure it will cover those areas indicated by the risk analysis as well as meeting other organizational needs and fitting within the organizational culture.

Some strategies include broad coverage where risks are diffused and as many areas as possible are covered. Early user testing is used when validation or customer acceptance risk is high. When specific areas are identified as high risk, the strategy may include concentrating resources on focus areas for in-depth testing and briefer coverage of broader areas. A strategy of repetitive regressions is used with many cyclical development models. There are many other potential strategies.

Many different types of risk analyses are performed throughout a software product's life cycle. Risk analysis at the beginning of any software development project is done to evaluate areas that may negatively impact the project. This could be possible timeline, quality, technical, or even people resource risks. In test planning, risk analysis is focused on issues that can be identified through testing.

The goal of risk management overall is to plan for risks in such a way that the risks do not become issues. This can be done by avoiding risks through design choices, mitigating risks through the use of controls, transferring risks, or accepting the risks. Ideally, every risk has been addressed in the most effective manner; however, there are never enough resources to eliminate all potential risks. As a result, once the risks are identified, each risk is reviewed for likelihood and severity of impact and ranked. The ranking of each risk determines how the risk is addressed and the resources applied to mitigating or avoiding that risk.

Even those risks that are avoided or mitigated will have residual risk. Residual risk remains even after the risk has been addressed. For example, if it was determined that using a particular software library risked adding potentially vulnerable code to a system and the risk management strategy was to not use that particular library, it is still possible that a developer didn't get the word and used the library anyway, or that someone used a differently named library that

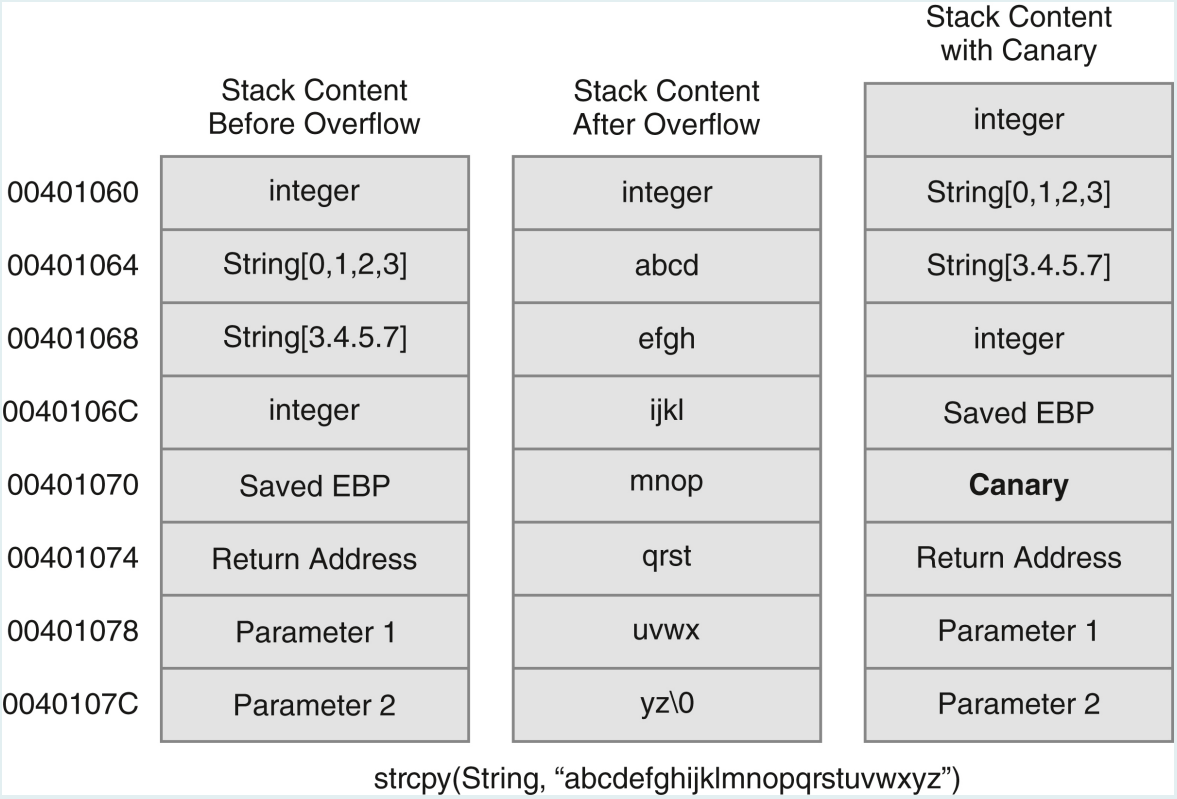
was derived from the first. It is also possible that a mitigation strategy, such as using canaries to detect stack overflows, was implemented incorrectly.

BOX 15.3 Using Canaries to Detect Stack Overflows

The use of canaries refers to the old practice of miners bringing canaries with them into the mine to detect hazardous gasses. Because the canaries were more sensitive than the miners to the gasses, if the canary died, it would give the miner in time to escape.

A stack overflow is created when a bad user tries to create a buffer overflow that extends an array stored in the stack beyond the stack frame, overwriting the stored return address, often with the address for a block of malicious code. The goal is that as the function returns, the address of the malicious code is passed to the program counter and executed.

The canary used to detect a stack overflow is pushed onto a stack just above the return address for the function in the previous stack frame. Prior to returning, the current function checks to see that the canary value remains unchanged. If it has been changed, the function calls a failure function; otherwise, the function returns and exits normally.



Description

In this example, the buffer overflow is done using the strcpy function, which doesn't check the string length prior to copying. Instead of the alphabet, the attacker would insert a string that included the target address.

Testing can determine the level of residual risk and whether or not the risk has been successfully mitigated. Testing is also used to cover areas where the risk has been accepted to ensure that those risks have become issues. The test planner will review all of these areas identified as needing testing and develop a test strategy that provides the greatest possible coverage of the identified areas of risk with the least effort.

Testing cannot cover all risks. Many, such as budget, personnel, and risks that are accepted but not be exposed until after deployment, are beyond the scope of testing. Those risks require other strategies, including management practices and monitoring and operational practices. Many types of risks, including new hardware being used for the project, not being able to get good testing people (or keep them), and something as obvious as not having sufficient time to perform all the necessary testing, can be checked with thoughtful test planning efforts.

As defects (or potential issues) are identified during testing, the tester will perform a quick risk analysis and assign the defect a level of risk. This categorization of discovered risk presents a quick assessment of the state of the system and allows for prioritization of efforts to address the discovered issues. The assignment of risk is primarily based on preestablished criteria. In some cases, it may be appropriate to complete a more in-depth assessment and potentially change the risk level to a more appropriate designation.

BOX 15.4 Levels of Defect Risk

- Critical: Defect prevents the system from functioning.
- Major: Defect causes the system to produce significantly wrong results for some functions. Significant requirements are not met.
- Minor: Defect causes the system to not function well or to not meet all requirements. Results produced are corrected in all but a few obscure cases.
- Informational: Defect does not conform precisely with requirements, or other issues that result in a less than perfect system.

Test Goals

The test plan needs to clearly lay out the test goals.

In general, the goal of testing is to make sure that what is released to the customer does what they expected and helps make the business more efficient. This very broad goal is made up of many more granular goals. By breaking the larger, general goal into smaller, more precise goals, it is more likely that a quality software application can be delivered that helps the business and customers can easily use.

The **test goals** section is where the test plan will specifically note the release requirements. These requirements include the amount of test coverage to be met and the established defect targets.

In order to produce a quality software product, functional goals, technical goals, performance goals, and security goals need to be met. Each of these may be laid out in their own section or combined as appropriate for the particular development effort.

Release Requirements

Release requirements specify the exact testing requirements and results that must be met before a software system can be released to customers. Alpha, beta, first commercial release, and full release may have distinct requirements.

The release requirements will specify the amount and types of testing that are required before the system is ready. This specification may include a reference to the section of the test plan listing specific tests to be performed, or it may be a listing of minimum specified tests.

BOX 15.5 Release Levels

Many software systems are released at different levels as they near completion. Different organizations use different designations for levels of release. Common designations are the following:

Alpha: System is distributed to a very limited number of users, often as a part of a testing effort or as a preview. Users expect limitations or issues involved with use. Not a revenue generating event. Use of the release may be halted at any time. Not ready for use in production.

Beta: System is distributed to a large number of users, often as a part of a testing effort or as a preview. Users expect minor issues, but the system should generally perform as expected. Not a revenue generating event. Use of the release may be halted at any time. Use as a production system is not advised.

First Commercial Release: System is distributed to a single or limited set of customers. Users expect testing has been completed and issues to mainly involve how to use the system. This prepares the organization for how to distribute or integrate for other customers. The customer pays for the system. The system continues in production.

Full Release: System is widely distributed commercially. All testing is complete. Processes surrounding distribution and integration are refined. The customers pay for the system. System is used in production.

The release requirement will also list the release **defect targets**. These targets specify the types, numbers, and severity of defects found in testing that are acceptable to include in a release.

These defect targets may be negotiated by the members of the project team, or they may be specified by management. If the targets are negotiated, the business representatives push for few, if any, defects to be allowed into production. The technical team (developers in particular) will argue that some defects are always probable because we are human beings and unexpected things arise, so the targets need to be as flexible as possible. The test team attempts to understand both perspectives and presents an approach for release defect targets that are between zero and what the technical team may propose.

In deciding which defects fall within a target, corrected defects are not included, but defects that have a work around may or may not be included. This often depends on the impact of the workaround to the customer perception of the product. This sometimes means that a workaround deemed to be trivial and simple to implement by a technical person may be deemed to be unacceptable from a business perspective because it makes the product appear to be “incomplete,” “unpolished,” or otherwise tainted in the eyes of the customer. Likewise, some defects deemed to be potential showstoppers from a technical perspective may not appear so to the business side.

Functional Test Goals

The goal of **functional testing** is to show that new software or new software changes meet the business requirements. Functional requirements are the details of how the software will perform specific tasks to meet business requirements. Testing for business requirements uniquely tests that the software is able to provide increased business value based on the pre-defined business requirements. Nonfunctional requirements can also contribute to whether the system will satisfy business requirements. After all, a system that does exactly what it is supposed to do, but does so incredibly slowly, is not commercially viable. By analyzing the results of *all* testing, management and the test team can determine whether the final software part or product meets the business requirements.

In establishing functional testing goals, the test plan sets objectives for the degree of consistency with the business requirements and objectives for the coverage of functional requirements. The functional testing goals will also serve to prioritize the areas that will receive the most attention. The functional testing goals may include a list of the types of tests that will be conducted to verify the functional requirements.

Regulatory, Legal, and Compliance Goals

The regulatory, legal, and other compliance requirements that the system being developed needs met, as well as any optional compliance requirements, are include in the test plan. Which areas will need special attention are clearly indicated. The inclusion of specific compliance requirements will also help indicate which specific tests or test documentation may be required. This is important for compliance; often, specific tests or test documentation is required to certify compliance. Failing to cover those tests or to document in the prescribed manner can mean needing to redo testing later solely to meet compliance requirements. This can be expensive, time consuming, and even lead to delays in bringing a product to market, generating lost sales opportunities.

For example, medical device manufacturing companies must meet Food and Drug Administration regulatory requirements. Even software used in a medical device or in a manner considered to be a medical device must meet all of the compliance requirements included in the regulations. Failing to do so can mean failing to receive approval to market the software. At times, this has led to some humorous testing. For example, the Combined Federal Regulations (CFR) required that all medical devices must undergo a drop test. For software devices, this meant copying the software onto a CD, repeatedly (100+ times) dropping the CD from 5 feet onto a hard surface, and recording the number of

“fails.” The CFR also required a remediation plan for the percentage of CDs that failed the test. The remediation plan was rather simple: send the customer a new CD. The real challenge in this test was keeping track of the number of times the CD was dropped. While the drop test for software is no longer required, it would be interesting to see how it would be applied to current software distribution systems. Would someone need to drop the cloud service provider’s data center or drop the workstation being used to download the data? Perhaps the method would be to print out the URL and drop the paper it is printed on.

Often regulatory or legal requirements are met by meeting specific certification requirements. Capturing test case results in a format that can be reviewed by certification granting authorities is more easily done using test software products where test cases are documented and often can be automated. The results are usually in some form of database that can be provided to the certifying agency (such as the Federal Aviation Administration) for their review. This type of approach to certification is quite a bit more efficient than performing audits manually at the software product development site. Such systems also do not fail to include all test cases, while manual test case execution and manual tracking (spreadsheets, other documents) can be unintentionally omitted when going through certification.

Security requirements include many items, such as who gets access, what access they may get, how the software application will protect nonapproved people from accessing the software code and the data in the database, and how the application software code is protected against possible unapproved tampering.

All types of requirements are documented in some form or another. There may be written documents, graphical representations in the form of drawings, or statistical results from test engines. Documentation confirms the product goals have been met. Again, this applied whether the part is for a car, a house, or for software. For our purposes we focus on software parts.

Technical Goals

The technical goals of the test plan specify areas to be covered in the test that are not included as a part of the functional testing but are still important for ensuring the system works correctly. An example would be to ensure that the system functioned properly on different types of networks or with different server configurations.

Performance Goals

The **performance goals** of the test plan provide the levels of performance that the software system is expected to meet. The goals will specify what performance levels the system needs to achieve to be considered successful. This should include the conditions in which the system is expected to obtain that performance. This will drive the amount and types of performance testing needed.

For example, if transactions are required to be completed within 35 milliseconds under normal load and 300 milliseconds when the system is heavily loaded, the definitions of normal and heavy load are required. The test team can then recreate those conditions and perform the appropriate tests.

Sometimes rather than specifying requirement performance that must be met, the goal of performance testing will be to discover the characteristics of the performance of the system in different situations. These characteristics may be related to the loading of the system, to the configuration of the system, for example, using different backend databases, or to other variable characteristics of the system or its environment. These characteristics may then be used to evaluate potential future problems or to develop deployment guidelines for the system.

Security Goals

The test plan security goals would specify the security environment in which the system is expected to operate and the types of security issues the system needs to be tested and prepared for. The security goals will be derived from the threat profile, itemizing test objectives needed to handle expected threats.

The Test Team

The test plan identifies all members of the test team, which includes members of an independent test team, if one exists, and all others who have testing responsibilities.

Organizations use several different approaches to determine who should perform testing. Some companies only require that the developers test their code; they do not need to use separate formal test teams. Many other companies embrace the need for dedicated test teams with the goal that they can maintain objectivity while testing. There are also many variations in between. The approach will vary based on a huge variety of factors, including the resources available in the organization, the organizational needs and culture, the particular products being developed, external requirements, and, sometimes it seems, the phase of the moon.

When no specific test team testing activities still need to be performed, test activities become the responsibility of the developers. Often, developers are supported by either commercial-off-the-shelf (COTS) or in-house developed testing tools. Developers also will cross test each other's work.

It is preferable to have a **dedicated test team**. This allows them to focus on verifying, validating, and just basically finding issues with the product. Additionally, it allows the test team independence from any pressures to finish coding and pass the product along. While those designing and those building (developing) the product are expected to test the output of their efforts to make sure they have met at least the known expectations for their part of the product, it is probable that they may not have the knowledge or time to test all functionality, much less look for the unexpected.

Often the skillset and mindset of a tester is significantly different from that of a developer. Many developers are not excited about learning how to deploy and use testing tools, are unable to focus on the details needed to apply extensive test cases, and may even find the idea that an error exists unfathomable and thus overlook it. When developers tests code, they test it from the perspective of how they built it. The code reads an integer, so they test it with integers, not characters or floating point. Some of these issues can be overcome with careful test planning and test cases, but not always.

In any case, having a dedicated test team does not mean that developers are removed from testing. Developer still have a role to perform by unit testing the code as it is being developed, initial system testing code as it is being integrated together, and in other areas identified in the test plan. Developers need to have a significant understanding of the testing that is going to be performed and need to be able to understand and respond to the results of the tests. There should never be a situation where testing is "throw it over the wall and see what comes back."

At the same time, an independent test team has drawbacks. Often **testers** come into conflict with developers. This is understandable because testers do need to point out issues with a product. To a sensitive developer, or from an insensitive tester, this can be seen as a personal attack. Developers may feel that the test team or a specific test person is deliberately focusing on a specific developer. Test team or test team members may feel that developers are not testing their own code sufficiently and relying on test team members to catch issues.

Because many test team members are sufficiently technical, they can often make suggestions on how to correct the issues they find; however, this is not the primary role of the test team. In situations where they do provide

suggestions, some developers may take offense at being told how to do their job.

What then can be done about such inter-team tensions? It takes time to build trust. Time is often in short supply on projects. The project team can work to get to know each other both through work and through nonwork experiences (such as going for coffee outside of the workplace together in small groups or people from all teams attending outside work events they are interested in) and by taking time at work to encourage respect for each other's work.

The number of people needed on the test team can vary, depending on the size of the project regulatory requirements, the project management methodology, or what phase a project is in. The larger and more complex the software application, the more test people are needed. No matter what type of project management methodology is being used, fewer hands-on test people are required during the initiation and the user acceptance testing phase. Test case creation begins in earnest during requirements gathering. Test execution happens as software is being written. This allows for starting a project with fewer test people and ramping up as the need for more people increases and again using fewer test people toward the end of the project.

Consider the differences in test staffing levels based on project management methods. In a waterfall managed project, the test team does not fully engage until after all software development (coding) is completed. This would be in phase 4, testing. Some members of the test team would have been required to write test cases during requirements gathering; however, in some cases even test cases are not written until well into the development phase. In an agile management project, test team members are required much earlier in the project. This is because the agile approach requires smaller pieces of software code to be released much more frequently than in a waterfall approach. Smaller pieces of software code are put into production earlier and more frequently than in a waterfall approach. Before any code is put into production, it must be tested. The goal of the agile project is for software to be put into production quite quickly. While prototyping is also trying to provide code quickly, robust, highly structured testing is not required because the goal is to quickly produce something that customers can review and provide input on rather than a fully functioning system.

Test teams can be made up of people with different skills. People who enjoy testing usually are good at and enjoy finding differences between what something was expected to do and what it does. Depending on what type of testing is required, the test team may have highly technical people trained in specific technical areas such as Java or databases. Some people may want to

break into software coding and see testing as a way to get there. It is common for test teams to have people with writing experience to help develop coherent test cases as well as perform the test cases. A subject matter expert on the test team for a specific application may be involved to provide continuous understanding of the overall goal(s) of the software application. Some members of the test team might understand software but have little experience with software development. Finally, all test teams will have specialists from areas such as network and information security available for consultation when writing test cases and to help execute area specific testing. Having test team members with different skills allows for greater testing breadth, and inter-team collaboration of different skills leads to better identification of outlier test cases.

Team Roles

The roles included in the test team will change over time. Even if there is an independent test team, many of those roles will be filled by people outside of that team. This is for several reasons. Some testing needs to be performed in stages before or after the system is given to an independent test team. Some test team members have distinct test functions, such as monitoring and tracking, coordination, designing and planning tests, and other roles. Involving multiple people in the test function helps to ensure that many different perspectives have a look at a service or system before it is given over to business customers for production use, which in turn leads to a more thorough check out or testing. Additionally, having people who are not dedicated testers look at a system or service helps to get their approval for any changes to a system or service before the change is put into production. Several roles are involved in testing:

Project Manager

The project manager is primarily making sure that testing is included and that enough time and attention (rigor) is paid to testing. The project manager is usually responsible for planning and funding staffing at each phase of any system change project.

The project manager is also responsible for providing information briefings on testing progress and findings for upper management and any relevant business people. The project manager doesn't create the information that goes into these briefings, but rather gathers it from the appropriate sources, translates it into an appropriate format, and provides the briefings as needed.

Test manager

The **test manager** works with the test personnel to familiarize everyone with testing expectations, make sure everyone is familiar with the system or service, familiarize testers with current company test processes, and works with the team to develop needed test processes. The test manager will also make sure everyone is a familiar with all the tools to be used during testing on a particular project.

The test manager also needs to manage the testers and escalate to the project manager if personnel issues arise. They will perform regular audits of test cases and test case results to confirm testing is being performed and documented to a high-quality level and are responsible for reporting test case results to the project manager and possibly to upper management.

Testers

The testers are directly responsible for test execution and test result documentation or recording. A tester's conscientiousness in applying tests and recording the results determine the final quality of the deployed product. Being a tester is not a temporary position or a part-time assignment. Quality testers bring a very meticulous outlook to the position and tend to be the type of people who gain a deep sense of personal satisfaction from being able to apply a rigorous, measured, and organized approach to testing.

It is interesting to watch the approach of some testers as they methodically work through the test sequence. At times they know there is a bug in the system waiting to be found. But rather than jumping ahead to the bug, they work their way through the entire sequence of tests that will eventually lead them to the issue. For them this careful hunt does two things. First it confirms that their methods were effective in uncovering the issue so that even in the case where it wasn't known, they are secure in the knowledge they would have found it. Second, it ensures they don't miss another issue simply because they were focused on the known issue.

Many testers come from a background in coding, while others have taken different pathways to the position. Sometimes developers make the mistake of disparaging testers who don't know how to code. Coding isn't their job. Testing what they have written and making sure it was done correctly is the job of the coder. In theory, if every coder did their job correctly, ignoring design errors, there would be nothing for the testers to find. It turns out though that there is almost always something for them to find.

Usually, the testers in a test team are generalists. They have expertise and experience testing a broad variety of areas. Sometimes, though, specialized skills are needed to complete an area of testing. For these, **subject matter**

expert testers are used. These subject matter experts may be internal or external resources. They plan and perform tests of specialized functions, specific types of modules, and of types in which the standard testing may not have the depth of experience needed to be highly effective. Examples of some types of testing generally done by subject matter expert testers include user interface testing, high precision computing testing, and penetration testing. Subject matter experts with a business focus may also be responsible for final testing during a deployment but before release to business users (customers).

Companies may choose to hire testers either onshore (within the same country) or offshore (outside their own country and usually on another continent such as India, China, Russia, or another country). Such outsourced resources are trained in software testing approaches and processes and may also be software program graduates. They will need to familiarize themselves with the system or service they will be testing. An advantage of outsourcing is that the outsourced testers are able to keep a level of objectivity that sometimes cannot be maintained by internal test teams. A disadvantage is that they are likely to be less familiar with the regulatory, legal, and internal political foundation for the requirements.

Developers

Developers are always the first line of testing. As they create each piece of code, they need to test it to ensure that it correctly performs the functions that they have written. This testing will not catch every possible issue, but it will provide a solid base of code to be handed to others for further integration and testing. At times developers do not perform a specific test because they created the code and know how it works. While they could perform the test, they may do so in the manner they built it and thus miss errors that would become obvious if someone tested the code with a different approach. Some of the best developers are people who started out as either functional testers or as release testers (install testers) and have experienced how a lack of testing delays product release while issues are reported and need to be fixed. They are aware of the types of issues that should be caught during code development.

Developers are often responsible for specific types of testing, including unit tests, in flow and outflow tests (testing that their part works with the parts before and just after their part), application programming interface (API) testing, and assisting with system integration testing. The developers may also be responsible for completing an installation test, confirming that the system installs cleanly. Developers also usually complete some level of regression

testing, making sure that the changes they have made do not cause new problems. The most immediate form of regression testing is called smoke testing. This comes from electrical engineering where the first check after working on any device is to plug it in and observe for blue smoke.

System Administrators

Systems administrators are important to the test team. Not only will they be asked to configure the networks, servers, and other components of the system correctly for the testing being completed, but they may be asked to complete portions of the testing themselves. In particular, the administrators will often be tasked with testing new processes for a system. They will check that the processes are easily understood and can be maintained by the operations team.

Another area commonly monitored by the systems administrators throughout the testing process is the system and service configuration stability. It may be that a system is successfully passing all of its functional and performance tests, but instabilities and abnormalities manifest themselves outside the test results. These may simply appear as glitches where the system needs frequent restarts, where ancillary services stop functioning, or as other obscure indicators. It is important that these be noted, included in the testing documentation, and investigated for their causes.

Test Tracking and Communication

Performing testing ensures that is delivered is what the agreed to requirements asked for and with expected quality. **Test tracking** is performed to ensure that each stage of testing has been accomplished using the processes, systems, configurations, and settings proscribed for the tests. Documenting and communicating the results of testing provides the first proof of the quality of a software part or product. If testing has not documented, then there is no confidence that it has actually been performed or that the results are satisfactory.

In the past, the test team would often be co-located with the developers. This is not generally true in today's interconnected world. Development teams (subject matter experts, developers, and testers) may all be spread across a country or, more commonly, around the world. When testing and deciding what to fix, effective communication and clear policies are even more important to make sure everyone understands what policies are being followed.

The testers need to document the performance and outcome of each test as it is performed in a process or system, which allows tracking of the testing and testing status. For tests with multiple steps or phases, the team records the conduct and result for each step or phase.

For every test case executed quite a bit of information is captured:

- Test identification: Indicates the specific test identified in the test plan that is being performed.
- Test execution date/time.
- Tester identification: This may be a different tester for each step or phase of a complex test. Sometimes a person's name is used. Other times it may be a different form of identification (Inspector #7). Using a name personalizes the results, which can be a good thing, or not. In the case where developers are sensitive about their code, it may make them feel a particular tester is on a vendetta. On the other hand, direct identification make allow a more collegial relationship to develop.
- Test environment: In the case of multiple test environments (systems) or a test environment is not specified, testers indicate the environment they used to execute the test. For example, a tester may have used a particular browser or browser version. This type of information can be critical in reproducing issues or identifying the source of issues if a test was passed, but an issue appears later.
- Version of software and possibly software part tested. This can be used later to confirm what was tested. This can be important when issues are discovered later and can confirm that the particular version in question was tested.
- Confirmation of inputs: This might just be a yes or checkmark to confirm the suggested inputs were used.
- Outputs received: Again, this may be a simple checklist of the expected outputs. It may also require copying the output received to the results report.

- Whether test passes or fails: Pass/fail criteria are in the test plan. The result might be a simple Y or N, or partial, provisional, or uncertain results may be possible.
- Bug, issue, or ticket IDs: Identifies where any bugs, deficiencies, issues, or recommendations were entered into the organizations tracking systems.
- Extra comments may be added to provide feedback to developers or to document items not indicated in the other areas. This is not used to record items that would affect the usability of the system, such as bugs or improvement suggestions, which are handled in the organizations issue tracking systems. These items may be focused on the process or other areas.

Test tracking can be performed in any tool. It can be done on paper (although that is less likely given how difficult paper is to communicate to many people across distances), through spreadsheets, documents, in-house databases, and COTS testing tracking systems. Some organizations choose to store results in their version control tool, which provides the added benefit of test results being stored with requirements, design documents, software code, and other project artifacts. Added to the benefit of having all project information in one place is the advantage that each test result iteration is assigned a unique version label. Some test tracking tools do a good job of collecting and formatting the required data. In some cases, a test management/tracking tool can be connected directly to the tools that are used to perform the tests and the results are populated automatically.

Regardless of which tools are used, all results need to be kept somewhere that they can be accessed by anyone in the company that would have a need to access the results and to provide a historical record. In some cases, such as the development of medical devices, the test tracking records are legal documents and are required to be organized and retained in a manner where they can be made available to auditors or other inspectors.

The results also need to be communicated effectively. Simply putting the results into a tracking system and expecting them to be communicated is totally ineffective. While in theory someone could poll the system on a frequent basis looking to see if any results have appeared for testing on a specific version of a piece of code, that is inefficient and isn't likely to go well. Instead, as results are completed, their existence and often a summary need to be communicated

to the right people. Even a short notice can provide a prompt to the developer to look in the tracking tool. Developers need to know the results of any testing performed on the software they have developed or helped develop. This is not only to allow them to fix any bugs in the software (sometimes they may not be allowed to) but also to enable them to learn from the results of the testing (perhaps a technique they were trying turned out buggy code, or a tool inserted hidden characters into strings).

Information about results also needs to be communicated promptly and accurately to the project manager as well as to other management who are keeping an eye on the project. Summarized information about the status of the testing should get sent to other interested parties. This information needs to be communicated proactively as well as be available in an easy to find location. The sooner test results and status are communicated, the quicker they can be acted upon.

Another important part of test results is to assess whether the test case needs to be updated or perhaps a new test case needs to be added to test an aspect of the application that was previously not considered. As much as possible, any test case updates or new test cases should be automated so that they can be run in the full regression suite.

Test Schedule

The **test schedule** provides timeline of the testing to be completed and when it is to be completed. The timeline will inform the test team of which tests need to be run and will be used to help schedule the resources needed for testing. The test timeline is linked to the schedule and milestones in the overall project plan. Each test in the timeline will be prioritized. This indicates where, if there are competing requirements, resources will be focused. It also helps when decisions need to be made about additional testing that needs to be completed.

In selecting tests to be included in the test plan, there are two primary considerations: the test goals and coverage of requirements through traceability. These, together with the approach to testing within the software development methodology being applied, will guide the test selection.

The strategic approach to **traceability** throughout a project is to make sure every requirement has one or more test identified with test methods and test cases created for it. As development proceeds, each module of code is linked to the requirements it implements. The module may have more than one requirement, or more than one module will be used to implement a specific

requirement. The appropriate test methods and cases for that requirement are linked to the code. This linkage is shown in **Figure 15.3**.

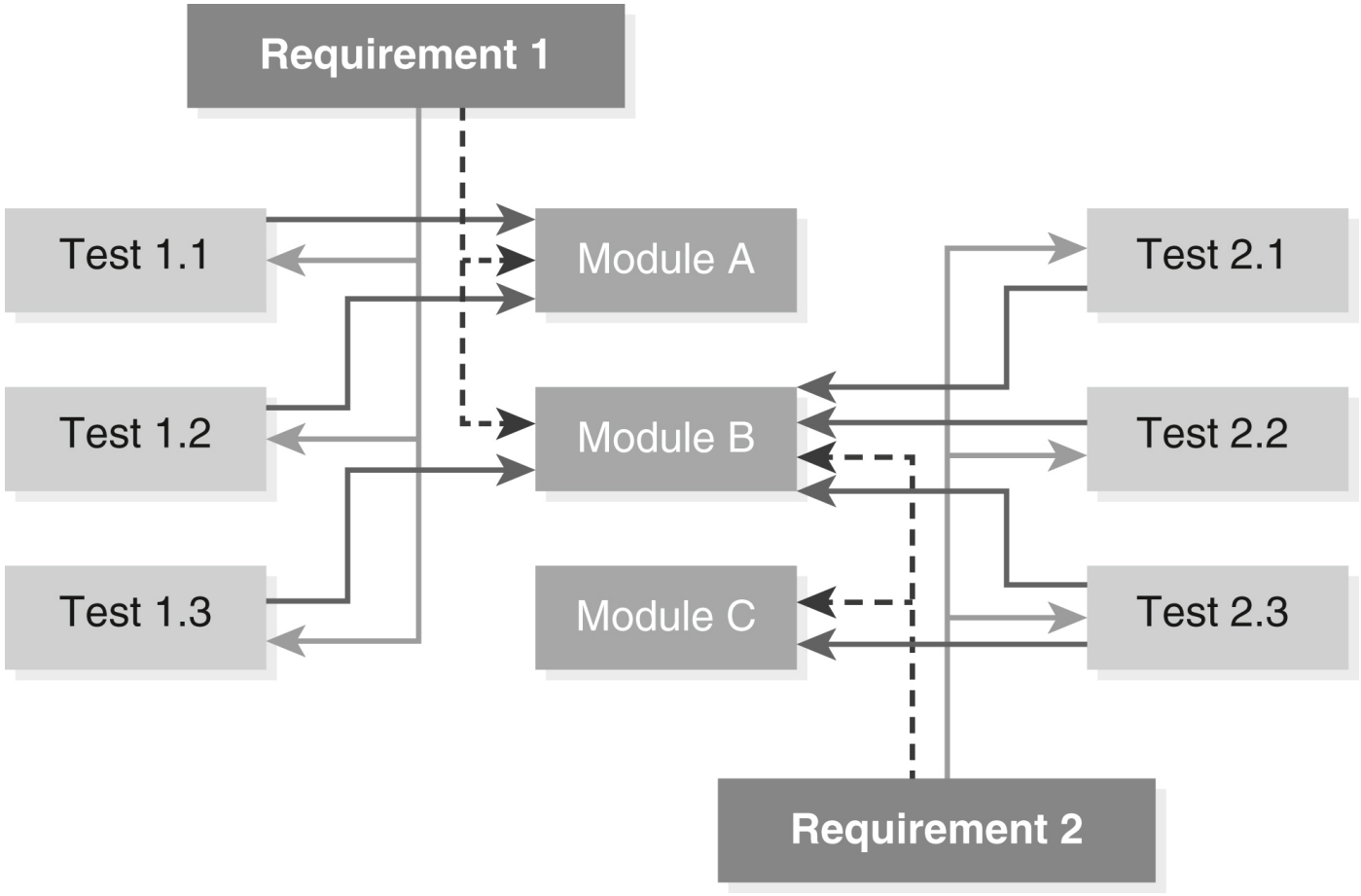


FIGURE 15.3 Requirements Traceability

Description

The effort to create the traceability throughout the entire software development project takes time and quite a bit of effort. Being able to trace testing and code to the initial requirements is the best known way to be sure the requirements have been met and tested for quality.

Another advantage to having traceability between test cases and requirements is that it provides the business users with confirmation that the original requirements have been met and perhaps exceeded. Beyond business requirements are technical and security requirements. These too will have links from the requirement specification, through design, build, and finally testing.

The software development methodology will not fundamentally affect the tests that will be performed, as these are dictated by the test goals and traceability. It will, however, affect the timing and frequency of when the tests are conducted. In the waterfall methodology, developers are expected to

perform testing on their work to make sure their code works as per the requirements received for their section. Once all parts of the project requirements are complete, the test team begins testing. This is often the first time that end-to-end integration testing is performed. This type of approach can cause problems if a major issue with an important software product part is found during testing. Finding a major issue could mean backtracking to repeat a lengthy development.

An agile methodology is similar to a waterfall method with smaller development iterations giving more frequent delivery of functionality to production to help business users. Testing is done as soon as a piece of code is completed but before it is released or deployed to production. Regression testing is performed for each iteration of release to production. This means that regression testing is performed many more times than in the waterfall methodology.

Test Cases

Each testing method selected for inclusion in the test plan will include a list of test cases to be covered. The **test case** specifies the inputs and actions for a particular iteration of a test as well as the expected results. The test cases will determine the thoroughness of the testing.

Test cases need to be developed that cover the standard expected inputs, interactions, and conditions for the item being tested. While full coverage of every possible value is not going to be possible, the test case should cover a statistically representative sample of possible values. In addition to the basic representative values, test cases should cover **corner cases** and cases covering the upper and lower bounds of allowed values as well as samples from outside of the boundaries. Corner cases are situations where the nature of the inputs change significantly. A classic example of a corner case is a zero crossing. Other corner cases would occur as the year or the century changes. One of the most famous corner cases was Y2K. As dates changed from 19XX to 20XX, software that assumed the century to be constant encountered all kinds of issues and errors. Corner cases are where we expose the unwarranted assumptions of the designers and developers.

In **boundary value testing**, it is important to use values that approach those the boundaries from both directions. That is, values from a little before the boundary and a little after the boundary should be used. This ensures that no hidden processing takes those values and crosses the boundaries. Boundary testing needs to be conducted for representational boundaries as well as for

range boundaries. An example of a representational boundary would be a change in how a number is presented as it grows from a simple floating-point number to a significant-sized number presented in scientific notation.

Special cases should also be tested whenever appropriate. This would include dates such as April 31 (never legitimate), February 29 (sometimes legitimate), or January 1 (a corner case at the start of the year). Negative values used in monetary entries may sometimes provide unexpected results. For example, what would happen if you filled out an electronic check to someone for a negative amount? Would it collect money from that other person? Other special cases would include NaNs (a legitimate floating-point number), infinity, or something out of the class. There are times when values such as 999 or 9999 have been used to signal special cases, but these are also legitimate Julian dates.

Test cases should be performed in combination and in different orders. It is surprising what doing test cases out of order or in random combinations will reveal.

The test case may include a reference to specific **test scripts** that will be used for the test. The test scripts specify the test as code and contain the instructions to drive the system using the values specified by the test case. Test scripts allow enhanced automation of the testing. These scripts are generally maintained in a code repository or other such system. One challenge with overly scripted testing is that it may result in ignoring the instincts of the tester and the arbitrary randomness that comes from testing done by people. This human touch has a lot of value because it often exposes the unexpected.

The test cases will also specify the **test environment**. The test environment will include any systems, networks, software, or other resources needed to complete the test. Additionally, the test cases will specify which test tools will be used for the tests.

The design of the test cases should cover all possible paths through every module. Paths are ways through the code that cover a specific set of code statements. No path will cover all statements because decisions (if statements for example) will cause some statements to be skipped and others run instead. **Full path coverage** is achieved when each possible path, and thereby each line of code, is run at least once. **Figure 15.4** gives examples of possible paths through a simple model of a program.

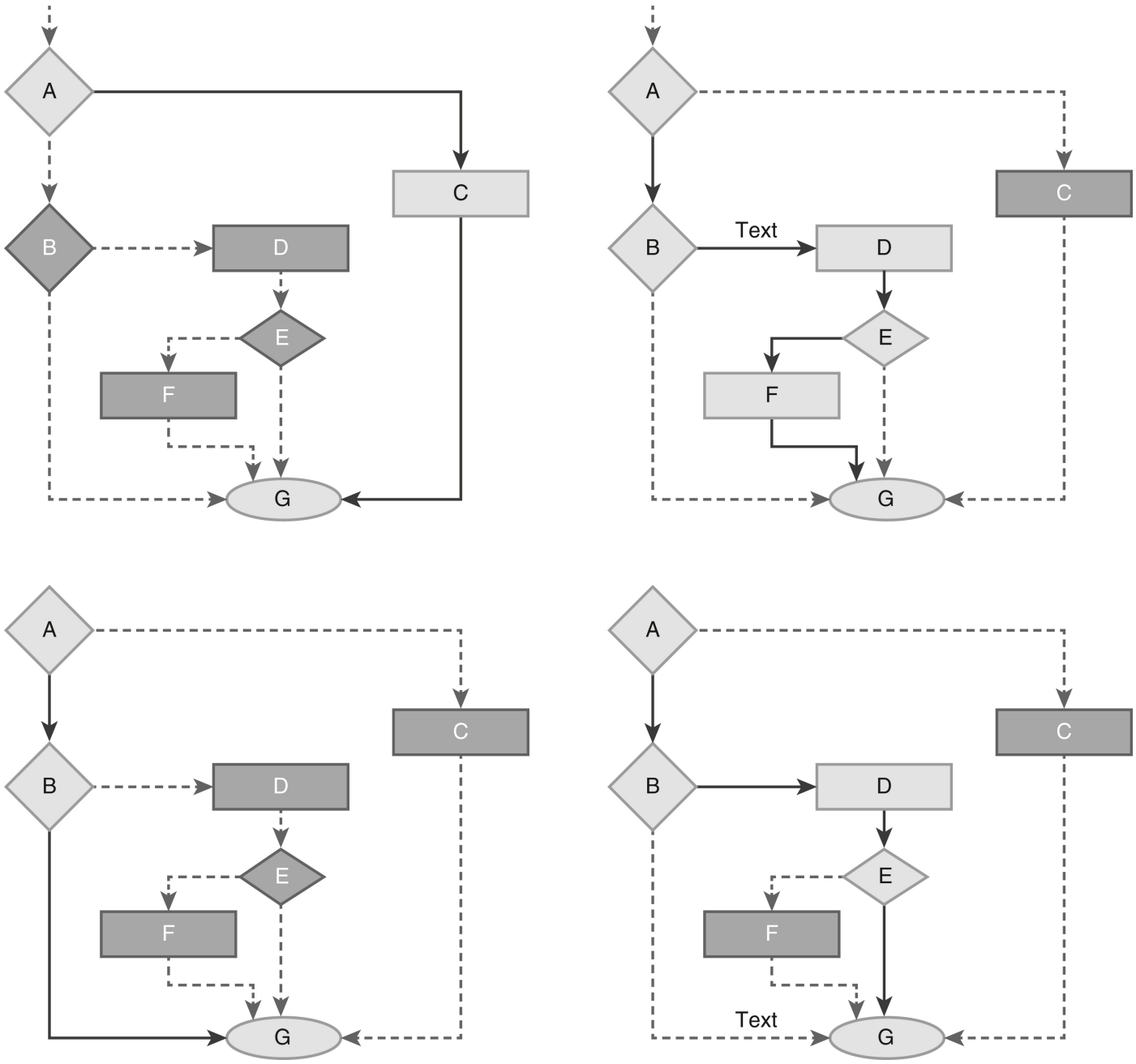


FIGURE 15.4 Dark Arrows Highlight Different Paths Through the Code

Description

While it is relatively simple to identify all of the paths through this simple model, the number of test cases needed to achieve full path coverage grows exponentially as the number of possible paths increase. This is especially true as loops and feedback are incorporated into the code. This means that in any code with a moderate level of complexity, the number of paths is going to become excessive to the point where the possible paths become hard to identify, and identifying test cases to cover each path is a real challenge.

Due to the challenge of obtaining full path coverage, a reasonable goal is **best path coverage**. This is achieved when as many divergent paths as possible, covering as many lines of code as possible, are followed given the constraints of time and resources. As a part of best path coverage, all known **critical paths** need to be covered. A critical path is a path through a section of code containing critical calculations or known to be subject to attempts at security breaches.

Another method used to aid in providing better path coverage is to select random paths using **fuzzing** or randomized small changes to inputs. The goal with fuzzing is to stumble across sections with issues. If you can't be thorough, be lucky. Although not ideal, given the lack of time and resources to systematically provide full path coverage and the inability of a test designer to accurately predict which paths other than know critical paths, this approach can be effective.

Some test cases are not going to be run unless there are specific indicators that the test case would be appropriate to run. These indicators are called **entry criteria**. Entry criteria are specified in the test plan.

Other test cases may be run for an indeterminate amount of time or until some measure is achieved. The indicators of when the test case is complete are called exit criteria and need to be listed in the test plan.

Testing Tools

Tests often need to be performed many times. Every time an issue is fixed, the test that uncovered the issue, a regression test, and potentially several related tests need to run. While it is unlikely that a single issue is going to be fixed and tested with each iteration, a large number of tests may still need to be repeated. Other tests are going to need an extensive number of inputs or conditions run as a part of the test. This volume of testing becomes a significant challenge when testing is performed manually. As a result, many tests are heavily automated.

While most test organizations have members skilled in scripting and potentially programming, writing test tools can be time consuming and expensive. Numerous commercial and open-source test tools are available. Many of them can be adapted to specific uses through configuration or scripting. This allows the team to rapidly assemble a good suite of testing tools suited to their projects. Some examples of applications that can perform testing include TestRail (integrates with GitHub), Functionize, Katalon, Lamda, and TestProjec.

The test team will still need to create the test case inputs and expected outputs. This does take time. Every time the software is changed, the test cases may need to be updated, but the effort to make these changes are insignificant compared to the time it would have taken to rerun the test manually. Because the tools allow more rapid and effective application of tests and test cases, testers can now focus on a more strategic approach to testing the product. They have more time to consider what is being tested currently, what is being missed, and how to improve the current testing.

Types of Testing

Tests can be characterized as being static or dynamic, white box or black box. **Static testing** is done when the software is not running or not actively in use. Often static testing is checklist-driven testing, checking to see that the system meets particular characteristics. At other times, static testing can be exploratory testing, for example using peer review of code to identify issues, readability, and other items. In some cases, static testing is performed after a system has been running to check that the outputs and expected artifacts have been generated.

Dynamic testing is done on running systems. Usually with dynamic testing, various inputs are selected and applied, and then either the outputs or intermediate results are examined to see whether they appear as expected. When the choices of inputs are actively managed, the testing is dynamically driven. If the inputs are randomly generated or accepted in some noncontrolled fashion, for example by beta users, the testing is passively driven.

Black box testing is where the tester has no access to the code. Because of this, the tester cannot determine what inner pathways can be taken through the system; this may result in less-than-optimal coverage. In some black box testing, published specifications for the system are supposed to meet and to which results can be compared. In other cases, black box testing is purely speculative with little known about expected outputs and sometimes little known about the types and values of potential inputs.

White box testing is when the tester has access to the system code and specifications. White box testing allows the tester to identify every branch within the software to ensure it is taken and tested. White box testing can also be used to step around failed tests and test deeper into the system in cases where a black box tester would be stymied. This can allow the identification of multiple issues simultaneously rather than having to identify them sequentially through the application of multiple tests.

It seems every few months a new type of testing is either defined or is needed. In some cases, previously known test types are broken down into smaller testing groups with new names. The amount and types of testing has increased dramatically over the past few decades as everyone works to increase quality. It also must be remembered that software usage has come a long way from being used for static information pages to being used for everything from helping run important airplane functions to controlling remote

surgical equipment. Uses such as these are life critical, making software quality and reliability critically importance.

Although the more commonly known test types were discussed, not every type of test was covered because a complete discussion of testing would take its own book. Software development is changing so quickly that any comprehensive list would be quickly outdated. It is probable that there are other test types for more specific types of software depending on a specific industry. Every piece of software does not need every type of testing, but given how widely software is used and how critical the functions of software can be, it is important to select the types of testing that will provide the most effective coverage. Be open to how to make sure your software product is the best it can be.

Standard Development Test Methods

A number of standard tests are or should be done for any type of development. These basic tests cover the core needs of testing and are applied in almost every test situation. The standard tests will be supplemented by other testing methods as needed.

Unit Testing

Unit testing is one of the first testing types done. Unit testing is white box, dynamic testing. It is usually done by a person doing the software coding for a specific requirement. Developers will test the piece of code they are working on across many criteria or categories of specifications. First functional testing makes sure the piece of code functionally works according to the defined requirements. Then developers will check that any data flows into and out of their piece of the code works properly and as expected. They do not go beyond the piece of code they are currently working on.

Because of the inherent limitations of users testing their own code, to increase test coverage, or for regulatory reasons, some organizations will conduct independent unit testing. This does not preclude developers from testing their own code; instead, it adds an additional layer of assurance.

Unit testing is often done using a **test harness**. A test harness is a piece of software that is built to run the software being tested either by a function call, an API, or other appropriate mechanism. The test harness provides inputs to the unit in the same manner it would receive them in the context of the full system and is able to accept the outputs of the unit. The text harness can be built to fully exercise a range of inputs, or it can serve as a tool for the tester to

enter specific inputs. Because the test harness accesses the unit in the same manner as the system would, it provides a better test than a programmer could by forcing the inputs using a debugger or other such tool.

Test harnesses can be created by the same developer who creates the unit, or by someone else who is either part of the development or test team. The advantage of having the test harness created by someone other than the developer is the independent interpretation of the requirements and expectations for inputs and outputs. The advantage of having the test harness created by the developer is that they are already familiar with the requirements and can build it faster. No matter who is developing the test harness, the specific required test cases will be detailed in the test plan, although the developer or tester may add additional cases based on their understanding of the unit requirements.

Well-performed unit testing has definitely decreased issues found later in development. Developers have also reported that while unit testing takes time away from coding, they learn to write better code through the testing.

Unit testing can be established as a requirement before checking code into a repository, or it can be done as a part of the check-in process.

Static Code Review

Static code review is one of the earliest types of testing during a software development project. Static code review checks for commonly known issues that can arise in development. Increasingly, it is common for some level of static code review to be included in editors or interactive design environments. While these included reviews are useful, they are seldom sufficient to cover all static code review concerns, so other reviews are needed. Often this type of review will be done both by an automated tool and through manual review. Automation helps look for known issues or weaknesses in the planned approach, while manual review will find things for which the automation has yet to be programmed. In addition, static code review tools often have a high level of false positives that need to be confirmed and noted.

Catching an issue as early as possible saves a great amount of time and money versus catching an issue further into the development cycle. Static code review has proven especially helpful in identifying and helping to eliminate many types of low-level security issues. Static code review is a white box static test.

Peer Review

Peer review of systems is a form of a static, white box review. In peer review, a fellow developer or a specifically identified team review completed code

parts for specific reasons. Peer reviews do not include functional testing or inflow and outflow testing and therefore are considered static testing. They do include someone reviewing another's code to check whether the code has followed company required writing standards, to check for proper functionality, and to check for known vulnerabilities. Code review often includes looking at security standards.

Other concerns that may be covered in a peer review include expected features and business rules, how sensitive data are managed (if sensitive data are to be manipulated in the code being reviewed), as well as how user roles, responsibilities, and security are managed. The unit of code is reviewed to make sure it has followed the overall coding design as was specified in the design phase.

Peer reviews should be performed using a checklist and all results are to be documented. They may be informal or done as a formal a walkthrough in a team meeting, checking the code for all to see and learn from. When performed using a group approach, less experienced developers have a chance to learn from feedback from group members as well as providing everyone with increased exposure to various pieces of the code base (this in turn helps everyone understand better how their code fits into the whole). Peer review can also be done in a paired fashion between two developers. Regardless of whether done as a group effort or in pairs, code reviewers begin to get a sense of possible pending issues with the code as they began to see trends in the code. Equally important to code quality is to assess in peer reviews whether the code meets any legal and regulatory requirements at the code level (potentially security, permission, or reporting requirements).

The documented results are saved in the project's configuration or version control system. Again, this documentation ensures an end-to-end record of the development process that can be traced throughout the project as well as providing a historical record for possible future review and process improvement. Such historical records build the body of knowledge.

Each company and possibly each project will have their own set of expectations for developers to follow. Those companies or projects that have documented the minimum to look for in peer reviews will have checklists. Using checklists and documenting the results has been shown to increase the final product quality. The historical record such documentation provides helps if an issue should arise later. Documentation is also used for audit purposes to help increase quality of future software development projects.

Peer reviews can either be performed before committing code to the project's configuration version control system or after commitment (checking

code in) or after the code has been committed. The best approach is to code in smaller chunks and commit code. Doing smaller units of coding and testing, each as quickly as is reasonable, will reveal issues sooner.

Integration and System Integration Testing

Integration testing is done anytime two or more units are combined. It can also be done as units are added to a system. **System integration testing** is done when the whole or a significant portion of the whole system is combined together. Fundamentally, system integration is no different than any other integration testing other than in scope. Integration testing is white box, dynamic testing. The focus of integration is one the interfaces between units and end-to-end results. As the units have already been fully tested, integration testing will use representative inputs and outputs for the various units rather than trying to run the full range. These representative samples are chosen carefully to provide the broadest coverage of different interface conditions possible.

Integration testing is critical to the successful development of software systems. This is where issues of differences in the interpretation of requirements between different developers is discovered. For example, one developer may base his or her calculations on the inputs to a function being given in radians, another on the same input representing degrees. This may seem to be a trivial issue, but a $125 million Mars Climate Orbiter was lost in 1999 because one unit was providing acceleration data in units of pound-seconds, while the receiving unit based its calculations on the units being newton-seconds (Hotz 1999).

Integration testing also covers testing in the environment in which the system is designed to be run. The modules or units combined for the integration may include operating systems, databases, external applications, or other elements that will combine to form the basic environment in which the system will perform. Later compatibility testing will be used to determine how well the system performs in different environment configurations.

Integration testing of systems that are combined with a new unit should always begin with a regression test of the existing functionality of the system. It doesn't do any good to add new functionality to a system if everything else is broken or existing functions are lost.

Integration testing is usually down by a combination of developers and testers. As with unit testing, test harnesses are often used in integration testing. These may be built to test particular combinations of units or combinations of functionality in the system. If they are testing specific functionality provided by a set of units, the test harness may bypass some of

the normal processing of the system to inject inputs and intercept results in an optimum location for observing the performance of the system. The test harnesses may be built by the test or development team. As with unit testing, a list of test cases and expected results is included in the test plan.

As with all testing, the results of integration testing are recorded and stored. The recorded results are used to correct the issues uncovered, and the testing is repeated until the system performs at an acceptable level.

Regression Testing

Regression testing is repeated throughout the development of a system to ensure that subsequent development and integration does not negatively impact existing functionality. Regression testing does *not* test new features or performance; it tests what has been provided in the past. Too often regression testing is overlooked, issues arise, and software updates receive a negative impression.

A regression test develops as a system is integrated together and tested. As each integration step is successful, representative test cases from the integration are added to the regression test. Over time this builds a test that is able to identify the loss of function even in very complex systems. Most software applications have more lines of code in than there are parts in a 747 airplane (yep, I did a comparison), all of which need to continue functioning correctly even other areas are changed.

Regression testing is often completed throughout the lifetime of a system, even when no further development is currently being done. It may seem counterintuitive to do testing without ongoing development, but software systems, as with most other systems, tend to have the classic bathtub shaped reliability curve as seen in **Figure 15.5**.

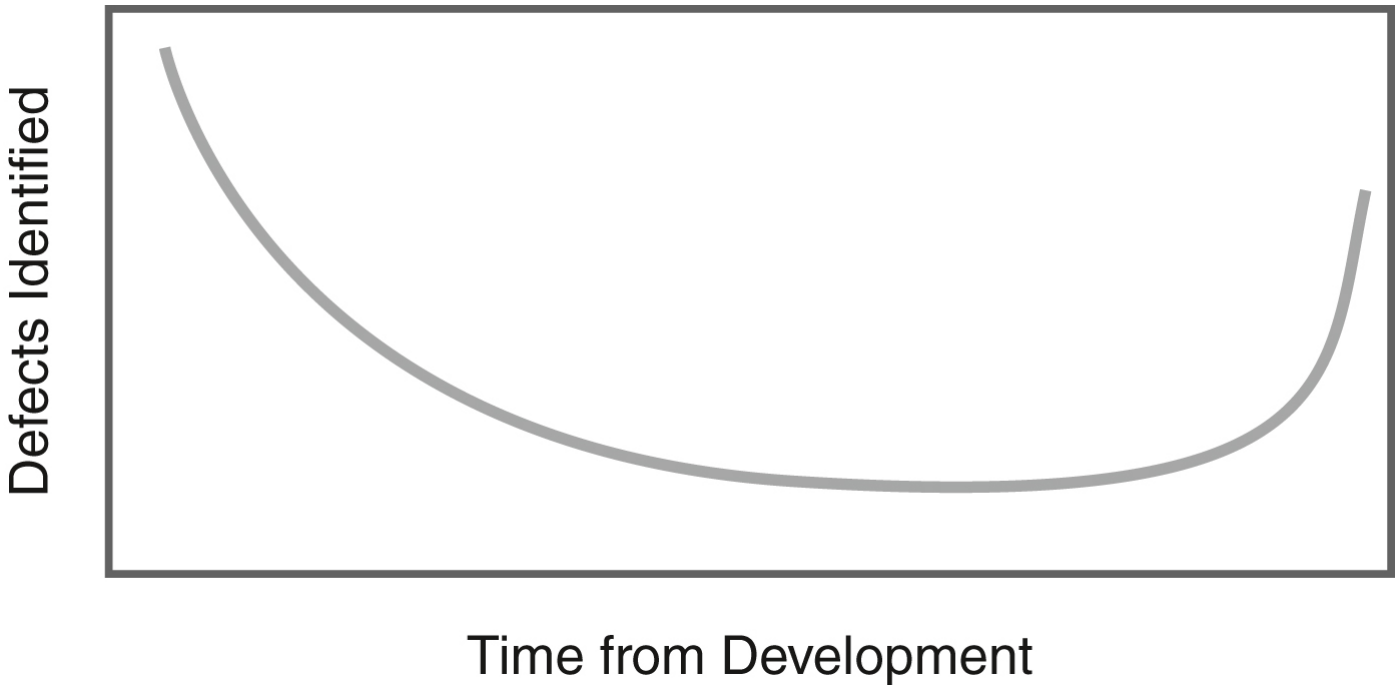


FIGURE 15.5 Classic Bathtub Reliability Curve

Description

Many defects are identified early in the life of the software as it is being tested and stressed for the first time in many areas. Over time, the obvious and easy to locate issues are identified and resolved. Then the system moves into a period where fewer defects are discovered, though they are often harder to replicate and resolve. This period continues for a relatively long period. But then the system gets to where the components start to fail with an increasing frequency. In physical systems, this is often due to wear and stress over time. In software systems, this is more often because of changes to the operating system, hardware, or other environmental factors that don't stay stable.

Security Testing

Security testing is primarily driven by the threat assessment as well as testing to general security issues. Vulnerability assessment tools and techniques are used to scan for known standard classes of security issues. This includes checking for open ports, checking how APIs are used, and checking the scanning and validation of inputs. Additional testing will test the limits of authentication, authorization, and functional segregation of responsibilities. The security tests will also cover any other areas indicated by the threat assessment. In some cases, additional security testing, such as penetration testing, is appropriate.

System Testing or Final Systems Test

System testing or final systems test is a special case of integration testing. While it is primarily a regression test, it also has a focus on the end-to-end correctness of results. Final systems tests are the last tests and are performed solely by development and test teams before user systems testing has begun (user testing of the interface and usability may have occurred prior to this point). Final systems testing may also include performance testing. Final system testing is also used to ensure that the issues identified and addressed during the testing have been correctly fixed and included in the system. During final system testing, the testers are going to ensure that all of the units included in the system and environment are those that have been cleared in previous tests and are unchanged from their state when they were tested.

Another goal is to find any and all issues (bugs) in the product that might not have been seen in all the previous testing. All issues are again documented, logged in the project issue tracking system, and reviewed for severity. While there isn't a need to fix *all* issues, the team will need to fix any issues that are greater than the severity level set out as acceptable when the project processes were defined. Low severity bugs may also have to be fixed if the number of low severity bugs are greater than what was stated as being allowable in the final product.

Final testing uses an environment specific to the test team. This nondevelopment, new environment is installed from a set of software that is expected to be released to customers. This makes sure that what is being tested will be the final product. Development environments tend to be constantly changing as developers work on the code. Final checks should not be done on software that may be changing as developers work on fixes or even the next set up updates. Development environments may also have special conditions that allow a system to function but do not exist in a production environment. For example, developers may have a special version of a library installed on their system that is not commonly found on user systems. Final system testing may also include compatibility testing, performance testing, or other specialty testing.

As many of the issues (bugs) found during this phase must be corrected, there are usually many rounds of system testing followed by fixes, followed by more testing. After each fix or a set of fixes, it would be great if a full set of system testing could be performed, but often there is not enough time in the schedule. The test teams then have to decide exactly which tests will be repeated after each issue or issues are fixed. At a minimum, a regression test

consisting of a full set of end-to-end functional tests, basic security testing, and some compatibility testing will be done.

After the completion of final system testing, the organization may move the system to release and to user testing. If the release will be an alpha or beta release, then testing will be the appropriate level of testing. Once those results are returned, the system will be prepared for its next release and go through another round of development testing. For a commercial release or internal general release, the next stage will be user acceptance testing or user integration testing. In either of these cases, the developing organization may provide testing support but is not responsible for conducting the testing nor does it expect to need another round of development testing for this release.

User Test Methods

User testing is a class of testing done by users or customers of a software system. This does not include testing done on users, such as user interface testing or usability testing. Some user testing is included in the test plan of the organization developing a software system. Other user testing is included in the test plan of an organization adopting or integrating the software system. Some tests appear in both.

Alpha Testing

Alpha testing is not done by the organization that developed the software system. It is done by a select group of users on a noncommercial release. The goal of the alpha release and testing is not to identify bugs in the software but rather to gauge user acceptance and use of particular features of the software, to discover how users will actually engage with and apply the system, and to discover unexpected usage cases. Often in an alpha test the software is not in its final form and may include features that may not appear in the final version.

A by-product of alpha testing is the discovery of unexpected errors and issues. Developers at this stage have completed their testing and are standing by to address any issues that may be found in this test phase. Some errors discovered in this stage may be irrelevant if the feature or this particular implementation of the feature will not be included in the next release. Other errors will not be so pressing that they will be fixed in the alpha release, but they will be fixed for the upcoming release.

Organizations may use internal users for their alpha testing, or they may have a limited pool of power users for alpha testing. Some companies use the terms “sneak peak,” “preview edition,” or similar titles for their alpha testing.

Beta Testing

Beta testing is similar to alpha testing in that it is done as a noncommercial release and is completed by the users rather than the developing organization. Unlike alpha testing, a primary goal of beta testing is to discover issues with the software system. In particular, beta testing focuses on compatibility, user interface, and performance and scaling issues. The developing organization may have concerns about limited resources for testing or about having so many variables that it is difficult to define a comprehensive set of tests. For these reasons, beta testing is generally conducted with a significantly larger base of users.

Some organizations abuse alpha or beta testing by using it to replace their own integration and final systems testing. This is a poor manner of conducting testing and leads to poor quality software with hidden issues because no user has yet tried some set of inputs that the company itself should have found in a reasonable testing regime.

The results of beta testing are used to make improvements for the next release. Generally, these are not major functionality changes, as can be expected with an alpha test, but rather targeted enhancements for performance and usability or bug fixes. A round of development testing is expected after a beta test is completed.

User Acceptance Testing

User acceptance testing is performed by the organization, which will be using the software system in production. The purpose of user acceptance testing is to ensure the users that the software is configured and performs in a manner that meets their needs. The majority of issues that are expected to be found are related to conformity with requirements (validation—does it do what we need it to do) and usability of the system. While it isn't uncommon to find verification issues (bugs), those are incidental and not the actual purpose of the testing.

The organization that developed the system does not perform user acceptance testing. Even in a dev-sec-ops type of environment, there still needs to be a certain distance between the developer and the acceptance tester. This maintains the objectivity of the test. The organization that did the development certainly has an interest in supporting the testing as needed, in the results, and in quickly addressing any issues that are discovered.

There will usually be multiple rounds of user acceptance testing. During the first round of user acceptance testing, it is very probable the user community will find either functional or performance issues that had not been found in all

the different types of testing done previously. Issues that exceed the acceptable severity or number of issues found in each round of user testing are fixed before the next round of user testing. Often user acceptance testing is so close to the expected date for production deployment that there is rarely enough time to re-perform all prior testing (including full regression) before moving to a second round of user acceptance testing.

The environment in which user acceptance testing is performed should be as close to the actual production environment as possible. If it is a new system, then the actual production environment would be used. If the system is an upgrade for or a replacement of an existing system, then the actual production environment can't be used. In this case, either as close to a replica of the production environment as can be afford is created for the testing, or in some cases, a duplicate is created that will be switched to become the actual production environment upon completion of the testing cycle.

User Integration Testing

User integration testing is the equivalent of user acceptance testing by an organization that will be integrating the software system into its own system. In this case, the current "users" of the software are the developers who will be doing the integration. Ultimately, another user organization will be performing user acceptance testing on the integrated product. The ultimate user organization may or may not know that the system incorporates third-party software. From their perspective, the whole system is the responsibility of the organization that delivered it.

The integrating organization needs to incorporate thorough testing of the delivered product prior to, during, and after integrating it into their own system. The testing prior to integrating the delivered system into their own product ensures that the delivered system performs as expected. It also identifies any quirks that must be accounted for during integration. Finally, it helps to identify issues that are endemic to the delivered system and did not develop over the course of the integration.

Continued testing during the integration uncovers issues quickly, helps to enable identifying the root source of the issue, and prevents major surprises after the integration is completed, or nearly complete. The motto "test early and often" applies.

Specialized Test Methods

Performance Testing

Performance testing is done when the product is first created, when making changes to a product, and on a regular basis to make sure there has been no performance erosion. This testing confirms that the software system or service meets the performance objectives and does what it is supposed to do when subjected to the specified and expected amount of work to be done through the system or service. Nothing is more frustrating to a user or business than a business application performing poorly or performing worse than it did before a change or update. If a new system performs worse than an old system, the business is not likely to be very happy with it. Performance testing looks at workload, stability, system response times, and whether it can scale up when necessary.

There are a few different specific types of performance testing. Not every type of test will be of interest to every system. Different factors, including the use cases of the system and the expected environment, help to determine the appropriate types of performance testing.

Load Testing

Load testing is a type of performance testing that determines whether the system or service can effectively manage expected user workloads or to characterize the expected performance at different load levels. Generally, in load testing, the goal is to identify the response times of the system at different loads.

Usually load testing begins with establishing a baseline at a low load level. This may be with a single user or a small number of users. Often this baseline is established with a load substantially lower than the normal expected load. The response times and other performance characteristics are recorded at the baseline.

Once a baseline has been established, the load is incrementally increased. At each increase in the load, the response times and other performance characteristics are recorded. Once the load has reached the design limit, the testing is complete. Further increases in load, beyond the design limit, may be used for scalability testing.

After the load testing is completed, the test manager can analyze the results. The first pass at analysis will determine whether the system met the performance goals. Deeper analysis will reveal the performance curves of the system. These can show the load levels at which performance becomes responsive to load as well as whether there are any inflections in the curve that would indicate an emerging weakness under load.

Often during load testing, it is useful to instrument the various components of the system and observe their performance as well as the overall performance of the system. These data are useful in determining which units are responsible for each portion of performance changes over varied loads as well as indicating which units may be reaching their performance capacity.

Scalability Testing

Scalability testing is a type of load testing used to determine the capacity limits of a software system. It is likely that usage of a good and useful software system will increase over time, not decrease. By developing an understanding of the degree to which the system can scale, it is possible to avoid having to completely redesign and rebuild a system or service every time more users are added, more data are being processed than was originally anticipated, or when the system will reach its limits and need to be replaced or supplemented.

One of the biggest challenges with scalability testing is figuring out how to place sufficient load on the system in order to reach the limits of the system's performance. For most testing efforts, there are simply not enough actual test users to create the required levels of load. Automation can provide a method of generating simulated user interaction, but automation has its limits in creating a representative load. In addition, even with automation, there may not be sufficient resources to complete the desired levels of scaling. In these cases, referring to load curves and inferring the projected scaling may be the most achievable option.

As with general load testing, instrumenting and collecting performance data from the individual components of the system can be useful.

Stress Testing

Stress testing induces specific conditions on the system to determine its potential breaking points. Naturally, scalability testing is a form of stress testing, but there are many others. Stress tests may be concerned not just with the maximum load on a system, but how the system the system responds to a significant load over a sustained period. Other stress conditions can be applied to the system as well. For example, the workload of the system can be changed to a mix that is heavy in functions known to be complex, time consuming, or with potential race conditions.

Some tests involve adding external stress to the system. For example, if the system is distributed across multiple platforms, stress testing may include increasing the load on the network, increasing communication delays, or

dropping connections intermittently. The goal in the stress test is then to see how these environmental factors affect the performance of the system.

Spike Testing

Spike testing is a type of stress testing that looks at rapid, high increases in workload or users. While stress testing is often focused on the performance and reliability of the system in the face of large, sustained loads, spike testing focuses on short-term responses. While a sustained load may result in obvious and sometimes spectacular system failures, spikes in workloads can produce unexpected transient issues that may not appear immediately or may not manifest themselves in obvious ways. For example, a part of a customer order may disappear, or the order may be there, but the cost of one item is extremely off. These types of issues may result from missing or jumbled transactions or other effects of the momentary extreme load.

User Interface Testing

User interface testing addresses whether the appearance and functionality of the system's user interface meets the defined requirements. User interface testing seeks to answer such questions as the following:

- Are all the fields present?
- Are the fields in the correct order or place?
- What input do the fields accept?
- What input do the fields reject? (This is particularly important when considering security at the forms level.)
- Is the order moving from field to field through the interface correct? I've seen forms that when tabbing go from the first field to the third to the sixth and then finally back to the first field, or where the use of the tab key did not work at all. At the same time, with the American Disabilities Act (ADA) requirements, many software applications today respond very effectively to verbal commands and provide verbal information as users interact with them.
- Are the colors of the form as specified in the requirements?

- When data are inputted into a field, is the action as specified in the requirements? Does any result of requesting data manipulation meet the functional expectations? For example, when entering 2 + 2 in the software form and asking it to perform the calculation is the answer 4 (a simplistic example that can be extended to any different functional action)? If the answer is yes, then the test passes; if no, then the test fails.
- Are *understandable* error messages displayed for pre-defined error conditions? For example, if data are entered into a field that is not allowed for that field, the error message needs to provide a message saying that, but the message better not reveal too much about the internal workings of the system. If too much is revealed about the internal workings of a system, an attacker can use the information to help attack the system.
- Perform functional system integration testing to make sure when moving through an application from one form or area to another that access and data continuity (as expected) is maintained. Something as simple as, did the cursor land on the first field in the next form or in the fourth field?
- As necessary, perform testing for ability enhanced features such as being able to navigate forms and fields verbally (ADA compliance).

Usability Testing

Usability testing is similar to user interface testing in that it is applied to the interface, but rather than being focused on whether the interface meets specific requirements, it is concerned with how well the interface supports the tasks that the users are trying to perform. It can also show whether the layout of the items on the interface is effective or not.

A common approach to usability testing is to sit users down at the interface and give them a task to complete. The tester observes the user, without providing guidance, and observes whether the user can complete the task, and how long, or how many mouse clicks, or how many data entries it takes the user to complete the task. Sometimes users are asked to describe what they are doing as they work their way through the task or asked to describe the choices they made after completing the task.

The data gathered from observing and listening to the users allows the tester to assess how supportive the interface is in supporting the users. The result of

the usability testing may be a redesign of the interface, or it may be a user manual to guide the users through challenging tasks. Another potential solution is a task “wizard” for those tasks that proved difficult.

Sometimes usability testing is done as A/B testing. Though done in many different manners, the basic premise of A/B testing is to present users with different potential methods for completing tasks. In comparing the measurements from the two methods, including feedback from the users, the testers can discover which of the methods or which elements of the methods are most effective.

Fuzz Testing

Fuzz testing, also called fuzzing, is a specific type of black box testing that, while it is assigned a unique name, is often incorporated in other testing types. Fuzzing works be using automation to make small, randomized changes to data entries and other application data being tested. The goal is to reveal application issues such as unexpected system or service crashes, slowdowns, memory leaks, and potential security vulnerabilities. Corner conditions, testing boundary conditions, or throwing random data at an application often reveals where a software application has not been developed to manage unexpected input well. Data the application expects are modified in ways that would not normally be expected.

Exploratory Testing

Exploratory testing is often used in complex systems where obtaining full coverage of all cases is impossible. The exploratory tester starts with one or several scripts for a scenario-based test. As the tester moves through the scenario, he or she will be aware of minor or curious deviations from the types of responses normally expected in this type of a scenario. At this point, the tester will then diverge from the prewritten test case to try to cause unexpected behavior and failures and to chase down any rabbit holes.

Chasing these often-ephemeral anomalies and seemingly insignificant glitches allows the tester to peek into the cracks and crannies of the system and discover issues that may otherwise may have been overlooked. As testers work their way through the way the system functions, as opposed to how it is supposed to function, they develop a deeper understanding of how the system was built. This in turn allows for further depth of exploration.

Being successful at exploratory testing usually requires a great deal of experience as well as persistence and the ability to think laterally.

What is interesting about exploratory testing is that it often reveals issues that no one had anticipated and could have a high impact to the system and the business. This type of testing is useful in any type of system, but particularly in complex systems (multicomponent, many codependent systems, gaming systems) because it is unlikely test scripts for every eventuality are possible.

Penetration Testing

Penetration testing is, as the name suggests, a deliberate, approved, attack on a software system or service. The goal is to figure out what types of attacks the product is susceptible to while the product is being developed rather than in production. Systems are made up of many smaller software parts joined together. It is the joining of these parts that often leads to weaknesses, either between the parts or even in some of the parts. Performing penetration testing means vulnerabilities that cause a software product to be susceptible to attack can be identified and fixed before a product with vulnerabilities gets put into production. Once in production, a vulnerability can be exploited by any number of attackers. As we keep seeing in the news, not catching vulnerabilities in the development life cycle leads to cyber criminals successfully compromising production systems and their data or services.

Penetration testing checks for potential vulnerabilities by simulating cyberattacks. It is different from vulnerability testing because vulnerability testing only looks for known vulnerabilities within the software code. A penetration test attempts to breach or penetrate a system or service.

Penetration testing is usually done by subject matter experts in the field of security. They attack a system manually and using software applications designed to perform testing by applying known techniques. GitHub has a module that can perform penetration testing; so do many other tools, including Nmap, Nessus, Wireshark, and John the Ripper. The penetration tester will apply many different tools, each of which may provide one or more ways to attack a system or service. Many different types of attack are used to test the system, including phishing, network access testing (internal and external), wireless access testing, and application access testing.

Penetration testers are likely to have access to information on the system or service that helps them more thoroughly plan some of their attacks. The vulnerability map or threat model created during system or service design helps when planning penetration testing. If testers don't have access to the vulnerability map, they will create their own.

To effectively perform penetration testing, the tester requires careful planning and execution. The most important part of the penetration test, however, is not

the penetration itself, but that the approaches and results are fully documented. All issues discovered in penetration testing are logged in the project bug-tracking system, but a significant amount of useful information can also be obtained by studying the attack techniques that were successfully thwarted.

Compatibility Testing

Compatibility testing is used to determine whether, and how well, a system interacts with other software systems. This is not system functional testing but testing of how the new system or an updated system works with all the other systems, subsystems, or services with which it will interact or that may be used in place of the integrated components of the system.

Compatibility testing gets done after a system is getting ready for release, after user acceptance testing, and before being deployed. All software systems or services will interact with other parts of the cyber ecology, including hardware, operating systems, devices in the network, the network itself, and other systems. Compatibility testing is done to make sure that these components do not have negative effects on each other, and when the systems interact, they do so successfully. Conformance to standards for interfaces with other systems will be done during the regular testing phases. However, although they have been thoroughly tested for compliance to the standard, it is not just possible but common that two systems adhering to the same standard have minor differences that, although they each fall within the standard, cause failures in the interface.

Compatibility testing also tests whether new software or changes to software are compatible with older cyber ecosystems (including hardware, different operating systems, or networks) and tries to determine whether the system will be compatible with future systems. Each system is thoroughly tested with the operating system, hardware, and other software components it was designed and built to work with, but it is inevitable that some of these will change. The operating system will release a new version, the database will need to be changed to one from a different vendor, or a client prefers using a different web browser. Each of these changes is going to need to be tested for compatibility with the system.

Compatibility testing is easiest if it can be automated, although it can be performed manually. Imagine having to test the same thing on many, many different possible cyber ecosystems. The possible combinations are mind boggling! Very large software companies such as Oracle have systems that automatically make any changes to a software product to make it compatible for specific types and versions of hardware and software.

SUMMARY

Testing is one of the most critical elements of developing secure software systems. Some would argue the most important. Without testing, code is merely a text file, perhaps with claims, but with no proof it meets any of those claims. Testing is complex. While testing may not require a specific test team, it must be performed in such a way that can confirm the product (new or an update to a software product) meets the highest level of quality. There is little point in producing a product that does not meet the business needs or, worse, makes business performance inferior to the previous product. Testing will both verify that the system works correctly and securely and validate that the software developed performs the functions intended.

The key to conducting effective testing is through creating and following a thorough, comprehensive test plan. This planning needs to begin at the very start of the secure software development life cycle. In developing the plan, the test manager needs to clearly identify the test strategy and goals and then select test methods and test cases that support them. The test plan is integrated with the overall project plan and clearly identifies the resources and time needed to test effectively. Done properly, the testing described in the plan will drive the development.

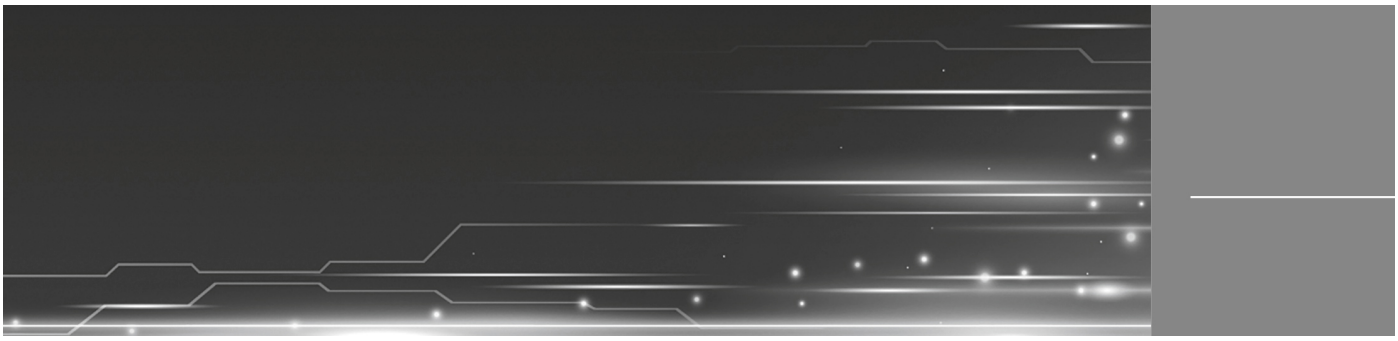
KEY TERMS

alpha testing
best path coverage
beta testing
black box
boundary value testing
compatibility testing
corner cases
critical paths
dedicated test team
defect targets
dynamic testing
entry criteria
exploratory testing
full path coverage
functional testing
fuzzing
integration testing
load testing
peer review
penetration testing
performance goals
performance testing
regression testing
release requirements
scalability testing
security testing

special cases
spike testing
static code review
static testing
stress testing
subject matter expert testers
system integration testing
system testing or final systems test
test case
test environment
test goals
test harness
test manager
test schedule
test scripts
test strategy
test tracking
test-driven development
testers
testing
traceability
unit testing
usability testing
user acceptance testing
user integration testing
user interface testing
V-model
validation
verification
white box

Reference

Hotz, Robert Lee. “Mars Probe Lost Due to Simple Math Error.” *Los Angeles Times*. October 1, 1999. https://www.latimes.com/archives/la-xpm-1999-oct-01-mn-17288-story.html.



© d1sk/Shutterstock

CHAPTER 16

Product Release and Deployment

CHAPTER OBJECTIVES

- Become familiar with the various parts of a release process for either custom software or commercial-off-the-shelf (COTS) software.
- Review why and when accreditation is necessary or useful.
- Identify the need for training on new systems or services, when to train, who to train, and training methods.
- Identify planning needed before deploying software or systems for business client use.
- Review communication methods and details in preparation for deployment.

Introduction

Finally, all the hard work of designing, coding, and testing the software is finished! Now to get the system or service out there for people to use. This is when release and deployment happens. Two distinct steps occur, even if they are commonly lumped together as is done here. Release is the acknowledgement that at system is ready to be used by clients for day-to-day work. It often involves formal certifications and approvals. Deployment is the actual installation of a system into a “production” or active system for use by the business.

Release can happen more than once. Commercial-off-the-shelf (COTS) systems often go through multiple releases. The first release will come as the product is readied for release to customers and first commercial use. This release is completed by the company that produces the product. In this case, the release does not mean that the next step should be for the system to be deployed on the customer network. It means that the developing company has readied the product to the point where it can be turned over to the customer and that the product has met all internal requirements of the developing company, and possibly some external certifications if needed.

Once a COTS product has been released to the customer, it is delivered. The path to delivery can be complex, including being sold, distributed, and otherwise moved through channels until it arrives in the hands of a customer. If the particular release is for an update to a previously sold or distributed system, it will still need to be distributed to the customers through some means. Sales and distribution methods and channels are a question for product management, as we discussed in chapter 2, so we will assume that it happens in some sort of a secure manner.

The delivered COTS software system, even though it has been “released” by the vendor, is not ready for deployment. The COTS system will need to go through further validations confirming that it is compatible with the purchasing company’s cyber environment. The client company, being aware that it needs to make sure that the system operates correctly and securely on its system, will go through rigorous set of tests of the released system, including a test install on an internal environment.

If, on the other hand, the delivered software system is being incorporated within or integrated with another system that in turn is being resold to further customers, the system will go through an intermediate release. For example,

database software being used as a part of an application will be updated and released to the application development company. The application development company will test and ensure the database works with its product and release the database for use with its application. The customer of the application company will then test and release the database prior to deploying it.

Preparing for Release

Preparation for release does not begin when the testers think the system is ready. Preparation begins well before through creating a **release plan**. This may be a separate formal plan, or it may be incorporated into the other planning documents for the system. The key elements of the release plan include determining what the composition of the release will be, as we saw in configuration management, the release timeline, the standards, approvals and certifications needed for release, release reviews, and the release authority.

While there are some differences between the release activities for COTS products and in-house developed products, similar activities are also done.

The **release authority** is the person or people who are authorized to make a determination that a system is ready for release. That authority may be divided among multiple people, with one person with the authority to approve the functionality meets requirements, a person with authority for security, and a person with authority for technical requirements. The release authority may also be vested in a committee or other group that has a standard for determining how the release decision is made.

The **release criteria** or standards will include the criteria for what must accompany the release, including documentation, media, and other materials, what level of completeness the system must meet in terms of functionality, and what level of quality the release needs to obtain. Other organizational specific requirements may be included as well. In addition, the release criteria will lay out any specific approvals and certifications (such as UL, CE, Food and Drug Administration clearance, or others) that the system needs to obtain prior to release.

The release review is often a formal part of the release and is used to ensure that the system meets all criteria. A release review includes many parts, including final security reviews, reviews of open issues (functional, technical and security), and a review of accreditation requirements.

Release Review

In preparation for release, all products that are created will go through last checks and approvals. This is the **release review**. When building a house, a last review is made to ensure nothing has been missed before handing it over to the owners to move in. The same is true for computer and cyber products. The release review is used as a formal step to ensure that the criteria for release have been met and that the system is in fact ready for release. The review may take place in several steps.

A perfect product (house or computer product) would take far too long to create, so long that the product would be out of date before it was released. This includes a house, car, or other human-made product. There will always be issues that are not resolved before declaring a product ready to be released. These are called open issues.

Final Security Review

A **final security review** of the product is performed to make sure there are no known vulnerabilities. It is probable that new vulnerabilities have been found since the product was developed and tested. A final security review is performed that includes looking at the types of security testing that were performed during product development, when a last security test was done, and what the results were. New vulnerabilities as reports on various security websites (such as CVE) or by the vendors are looked at to see if any of them affect the current product and will need to be fixed before the product can go into production.

Any new vulnerabilities found would need to be evaluated to how critical they are and whether they need to be fixed before production or can be delayed until after production release.

As a part of any final security review, any remaining open security issues and new vulnerabilities are evaluated for how much of a risk they are (criticality/severity), and any high impact issues will need to be fixed before the product is released into production to avoid leaving the product open to attack by bad guys. Security issues affecting data, access to data or systems, or hardware must be fixed before release can be approved.

Open Issues

While any product is being developed, a list of requirements as well as a list of issues is documented and tracked. Any issues that cannot be resolved before the product is declared complete are open issues. At the beginning of a project and as part of the working processes, definitions, the number, and criticality of allowable open issues before product release are identified. At time of release, all open issues are reviewed. The product be released only if the remaining open issues are of a low criticality/priority. Any open issues that have a greater criticality/priority than agreed to will cause the release to be delayed until the issue is resolved. Any remaining issues that the product is released with are actively tracked and must be fixed within a specific (short) time after product release.

The remaining issues do not disappear, but rather they are kept open so that their fixes can be included in future releases. Remaining security issues, in particular, are carefully reviewed to be sure they will not leave the product open to possible attack. If security issues could give a potential attacker a way to breach a system product, then release and deployment is delayed until the issues can be fixed. Security issues include front-end vulnerabilities (open to attacks such as SQL injection and other types of attacks), recently discovered database vulnerabilities, or vulnerabilities in the coding language being used. Every tier or layer of a software product is reviewed for new vulnerabilities, including the hardware and operating system on which the new product is being installed.

Accreditation for Release

Many systems today perform important functions and are involved in businesses that must meet legal or regulatory requirements. **Accreditation** is the process of assuring and certifying that the system meets the legal and regulatory requirements. Accreditation may be done internally or externally.

Internal Accreditation

Internal accreditation is done by people in the organization who have knowledge and experience in the particular area. The accreditors are independent from the development and verification teams. Larger corporations usually have accreditation experts in the fields for which they develop software. The in-house experts are independent of the development process and review the product before release to check that everything works as expected based on the test results, that all documentation is complete, and that all standards and legal requirements have been met.

The internal accreditation may be the only accreditation step required, or it may be the predecessor to additional external accreditations. It is not unusual that internal accreditation efforts are periodically reviewed by independent auditors or agencies.

Government System Accreditation

It will come as no surprise that systems used by the U.S. government need to be designed and built to stringent specifications, which means they will need to be assessed and accredited. There are a number of different accreditation programs: the Federal Risk and Authorization Management Program (FedRAMP) is used for cloud systems; Federal Information Processing Standard (FIPS) specifies cryptographic requirements; **Authorization to Operate (ATO)** authorizes a product for use and has several steps; American Disabilities Act (ADA) ensures that all U.S. systems are accessible to people with disabilities; and the Risk Management Framework (RMF) covers entire software development process with the inclusion of cybersecurity requirements.

ATO has to be achieved before any system can be put into use in any part of the federal government. A certification authority (external) reviews the system for security, e-authentication, and business continuity planning. Each federal

agency may have unique requirements, which makes sense because different agencies are responsible for different types of business (Food and Drug Administration versus Federal Aviation Administration, for example). A system provider has to have an ATO before the system can be either sold or installed for production use. ATO has three levels: initial, interim, and authorization to operate approval. Every 3 years systems are reviewed for ATO renewal based on updated requirements. The details to obtain approval are not discussed here because they are comprehensive and can be reviewed through various online sites. The ATO covers many cybersecurity topics, including access control, authentication, how media is protected, physical security, training, and configuration management.

Once the relevant accreditation has been received the system can be released for sale to customers (for COTS systems) or, in the case of in-house created systems, can be deployed to production. We are one step closer to getting into production and being able to use the system to help make the business more effective.

Deployment

Deployment comes after a system or service is declared to be ready to be put into production (released), to be used by the business. This is done after all product development has been done and the required release steps have been met. At this point, the new system or new updates can be used to increase business efficiency. However, deployment is not as simple as clicking on an icon and beginning to use a system.

A significant amount of careful planning needs to be done to ensure a successful deployment. This is true whether deploying a new system or service or updating an existing one. Doing deployment correctly can avoid significant issues that can quickly become highly visible. In order to ensure success, it is important to create and follow a **deployment plan**.

The deployment plan can be a stand-alone document or incorporated as a part of other planning documents. The plan needs to consider all aspects of deployment, including the roles of people involved, training needs for people in those roles, the timelines for deployment, the requirements that must be met during the deployment, and measures of success of the deployment.

Deployment Roles

Several roles need to be filled for a deployment. The roles may vary, based on the needs of the system and the organization. The first of these is the deployment project manager. This person is responsible for monitoring, guiding, and tracking the deployment to ensure that the system is launched as needed.

Another role is that of the deployer, who will be performing the actual deployment tasks. The skills, capabilities, and responsibilities of the deployer will be driven by the needs of the system. In many cases, the deployer may need to be a system administrator or have system administration credentials.

The end users are those who will be using the system. There may be different categories of end users based on how the system operates and the needs of the organization. These may be users with different levels of privileges on the system, such as in an accounting system where there are entry roles and approval roles, or the users may be segmented between standard users and power users. Power users are defined as those who will be expected to use more functions of the system, use the more advanced features of the system, or be expected to guide and assist the normal users.

The system may have administrators, who are responsible for managing aspects of the system, such as enabling users or granting permissions. Other types of administrators may be responsible for administering the technology, including computers and networks, on which the system is running.

All of these different roles are going to need to be considered and addressed in creating the deployment plan.

Training before Deployment

As we get closer to either releasing a COTS system or deploying an in-house developed system, **training** is needed for all the people who will support, use, or maintain the system. Training is the task of making sure everyone on the team knows their role on the system and how to perform it. Very often training is either not budgeted for or is minimally done. Lack of training for anyone using or supporting a system means they will not be able to take full advantage of the system and may even inadvertently cause a system to be vulnerable to attack! There are many levels and different types of training.

User Training

Anyone using or managing the system needs to be given training on new features and functionality, security features relevant to the user, what *not* to do in a system, and many other system and system security specifics. Training should always include the consequences of *not* following or doing what they have been trained to do to keep the system and system's data secure. Training for the user is tailored to the needs of each position.

At the executive level, training should be brief—perhaps just a high-level overview of the capabilities of the system along with how to perform the few actions in the application that may be needed to meet any regulatory or legal requirements. This will ensure that executives' expectations of what the system can do are aligned with reality.

Managers need to know how to do the specific actions for which they are responsible. In addition, managers need to know what will be expected of all of the different employees whom they manage. This does not always require that managers know the details of how front-line people perform their activities, but managers do need to know what those activities are and have enough understanding to evaluate employee performance and be able to provide assistance for employees when needed.

Managers and executives must understand the security built into the system and the consequences of not meeting security actions. Managers and

executives must be aware of how they can stay in compliance as well as what expectations are of their personnel as well. Support for security as well as the application comes from the management and executive level. They will create and socialize business security expectations, policies, and procedures.

Administrator Training

Everyone whose job is to make sure the system performs as expected and stays safe from attack (i.e., administrators, support people) needs to receive training on how they can succeed at that task. The training should include how the system needs to be configured to provide maximum performance, operating system settings, database sizing and settings, any hardware configuration settings, and other day-to-day operating requirements. Training should include information on the frequency of checks and testing to keep the system at peak performance.

The administration staff should be trained on how the system needs to be configured for maximum security. Training should cover what multi-factor authentication to use as well as system configuration to make sure the product works securely and efficiently within the business network architecture. Training may also be needed for implementing necessary encryption and identifying which data elements need to be encrypted. Administrators need to know where system logs are stored and what is in each log file as well as how often logs should to be reviewed for issues (access, security, and system performance).

In addition to the specific items, training should include how to access references, including frequently asked questions (FAQs), covering known functionality, performance issues, security challenges, and other areas. This training provides a resource for the administrators rather than forcing them to discover everything for themselves.

Deployment Communications

Once any required accreditation has been attained and training completed, a system can finally be deployed for use! We don't just install a system, but plan carefully to make sure it all goes well and business isn't negatively impacted. An install rehearsal, migration window, backout plan, communication plan, and other details are required during migration planning.

Let's first talk about the **deployment communication plan**. Frequent and multilayered communication would have been done throughout the project; however, the closer we get to deployment, the more important communication becomes. Communication pre-deployment lets people know the expected

timeline. Pre-deployment communication starts months before deployment and is done many times. It's always better to tell people many times over a few months to make sure no one is surprised when the actual deployment happens. The communication will also let users know of any actions they may need to take before deployment (such as backing up any data or even modifying specific data to get a system ready for an upgrade). For a new application, users may need to double check the information they provided for access to the new system as well as review the training they went through to be ready for when the system goes live.

The communication plan has many different parts, and communication is done many different ways. That way everyone in the organization who needs to know when deployment will occur is notified, has the appropriate information, and is prepared. Organizations use multiple communication methods (called multimodality) because not everyone takes notice of every type of communication. The organization needs to make use of every communication method available, including emails, meetings, websites, internal news sites, and any other methods. A good method is to set up communication contact "trees" where information gets reviewed by management with their staff and perhaps even pre-identified key communicators who will help spread the information to their colleagues.

Communications need to be repeated periodically. That would include informing everyone at least 4 weeks, then 2 weeks, and finally, days before deployment. While it may seem repetitious, some people require and pay attention to early notifications, but others only respond to messaging when the expected event occurs within a shorter time horizon. The repeated communications are not only reminders but also match different communication styles.

Deployment or Migration Planning

So now that the product has been built, thoroughly tested, and assessed ready for release to clients and deployment (install) we can install it on a production environment or system for people to use in the business. Usually when we are deploying a system, it is not going into a "green fields" environment where no other application has been in use before, but rather it is replacing an existing system. This means that we are going to need to do a **migration** of the users, data, processes, etc. from the existing system to the new system. A migration is a carefully planned and executed transition from an existing system to a new one or an upgrade of the existing one.

We cannot just load the system onto the new hardware, or in the case of an upgrade, overwrite the current system without first carefully planning the deployment. Some really terrible things can happen without planning. In one case, we needed to update 20,000 servers with a required monitoring agent. The software package was small and had been tested in the lab. However, the deployment team installed it on one particular set of development, test, and production servers at the *same* time. The patch caused a fatal failure on this particular set of servers with a critical business application, causing the application to completely fail. It was 2 days before the root cause could be identified and the system rolled back to a prior working state. The financial impact was high.

Therefore, careful deployment/migration planning is needed. This includes any and all necessary software (target software as well as pre-requisites) must be known and documented, all people with the necessary accesses and knowledge for each part of the deployment need to be available for the deployment time frame (window), and it needs to be confirmed they will be present during deployment. To test that the deployment was successful as well as did not have any unintended impacts, both technical and business people need to be booked for the deployment time frame.

Once the deployment is completed, closure activities performed by the appropriate people document how the deployment went and any necessary follow up.

Planning for Deployment Failure

When deploying any product (a new bridge, new tv, or new software product), we plan for possible product deployment failure. A lot of careful planning went into the product from when it was first thought of (envisioned), through design, building it, and finally testing it; however, the product could still fail on final deployment when being put into use.

The most important part of deployment is planning what to do if a deployment fails. How will we put the system back to what it was just before deployment so that the business can keep working? This is the role for the **backout plan**. The backout (or rollback) plan is a plan for how to return a system, or systems, to the state prior to the beginning of an upgrade or other deployment. The backout plan is invoked when it becomes obvious that the upgrade is not going to be successful or is not likely to be successful. Deploying upgrades to systems without a backout plan is like doing trapeze tricks without a safety net. Sure, everything may go right and be completed successfully, but if something goes wrong, the results can be catastrophic.

The most important roll of the backout plan is to make sure we can “fall back” to the system as it was working before the new deployment was attempted. The backout plan starts with a solid understanding of the system prior to the upgrade. That understanding includes knowing the systems involved, the configurations of all systems and software, any communications to external systems, and other key information. The next step is to ensure that a full back up is made of the original system before doing any updates of either system files or configurations. While the organization should already have backup processes in place that can be leveraged, a special rollback backup is often needed; the timing of the backups may only indicate a partial backup at the point of the upgrade, or the organization of the backups may not be ideal for supporting a rollback. This additional backup will take quite a bit of storage space, but it will ensure the system can be rolled back to a working system. The rollback backup needs to be kept for a length of time after deployment *just in case* something shows up in production that was not found during development or testing that requires the system be rolled back. In addition, new data being fed into the upgraded system needs to be retained in a manner that allows it to be fed into the restored system if necessary.

Part of any backup is noting all the supporting system configurations that could possibly change during deployment. Will new security options need to have support system changes (such as operating system changes or even possibly hardware changes) that affect new access controls or encryption requirements? If so, the current configurations of those supporting systems need to be captured in some way. Capture can be through backup of the supporting system, or it may be either screen captures or downloads of the configurations. The information should be saved in the version control system so that it does not get “lost” on someone’s personal computer. Also, by saving such information in the version control system, the information will be there years from now should it be necessary to go back that far during an investigation.

Rehearsals

Once all steps for the deployment have been agreed to, a deployment practice run or **rehearsal** should be performed. This helps identify any tasks or people missing in the plan. A rehearsal is, by necessity, done either on a similar environment (has similar hardware and supporting systems) or if the needed resources are not available, then it may be done virtually through a number of collaborative meetings where all people involved in the deployment are present. This type of virtual walk-through lets everyone involved think about their role,

the timing of each task, and any other tasks or people who might be needed rather than being in the middle of the deployment and calling an emergency change to the deployment plan.

Multiple rehearsals are always a good idea. Start rehearsals well in advance of the scheduled deployment. There needs to be time to identify and resolve any issues that are identified in the rehearsals. If the issues can't be resolved in time, the deployment may need to be delayed.

Specific roles or experts need to be involved in both rehearsals and deployment: network, database, supporting application(s), application developer, and information security experts. Needless to say, it is often difficult to get the needed time commitment for the planning and rehearsal sessions as people are so busy with their day-to-day work.

Deployment/Migration Window

Finally, the day and time of installing the new application or upgrading an existing system arrives: It is deployment day. The team (business and technical) will have determined on what day to start the deployment. The **deployment window** is the specific period during which the system will be installed or migrated. There is a specific start to the window, before which the existing system needs to be fully functional, and all transactions conducted on the system need to be complete and successfully captured. There is a specific end to the deployment window by which time the new system needs to be installed and fully functional, or the deployment needs to be called to a halt, and the old system restored and made fully functional.

Often a number of activities can be started before the deployment window, including getting equipment, software, and materials into place and starting the capture or conversion of existing data.

As the deployment window approaches, one last communication is sent to users to let them know that the system is going "down" and will be unavailable for the scheduled time. Users are expected to have completed their work and logged off before this last communication. In some cases (well, many cases), users will need to be removed from the system. All systems have this capability built in, and if they don't, it should be a capability for the next system revision. Now that only administrative people have access, deployment can truly begin.

One person (often the project manager) is responsible for setting up a virtual meeting, as was detailed in the communication plan, on the deployment day, at a time that starts before the deployment execution. There is nothing like setting up a meeting with no room for people to be delayed; this can stall tasks and in

turn delay getting the system back online. In today's world of virtual workers and teams that are not co-located, conference calls (which may include cameras to be able to see each other or may be only voice call) are used. A conference call (with or without video) is started and will be ongoing until the effort is completed or the deployment is abandoned due to failure. The first part of the deployment window focuses on technical tasks. Expected to be present at the deployment call are database, application, network, and development representation. Developers are present *just in case* there is a misunderstanding of the deployment instructions or if something does unexpectedly fail during deployment.

As was noted in the deployment planning section, a full backup of the application as well as backups of co-dependent system configurations is done. That backup of the working system needs to be saved to an online, reliable, easily available storage space, just in case there is a need to restore the system. Be sure to name the backup with an easily recognizable name, including date and time taken. Be sure to encrypt the database backup if there are sensitive data in it! If a new system is being installed, a backup is not done because there is no current system to backup.

Now to deploy the new system or current system changes. Preferentially, deployment comes in the form of either one or a couple of files that support staff can execute to apply or install the new files, make configuration changes, and even double check that the install went (technically) successfully.

Once technical install has been completed, user experts are invited to test the new system or system changes to be sure everything works as expected. Part of testing includes confirming all the security expectations are met in the new or updated system. As mentioned in the testing section of this book, it is helpful if basic functionality testing is automated so that users can perform automated testing and concentrate on functionality not covered in automated test cases and possible corner cases.

As the deployment and deployment testing is proceeding, someone needs to be keeping an eye on the clock. They are watching for an impending deadline. That deadline is based on the time needed to restore the existing system before the end of the deployment window. Before the time reaches that point, either the migration needs to be confirmed as successful, or someone with the authority to do so needs to make a call to backout the migration and return to the old system or take a risk as to whether the migration can be completed successfully,

SUMMARY

It is incorrect to assume that as soon as a system or service has been created and tested it gets pushed to production. Some companies with noncritical products can take such a risk; however, most products supporting businesses and critical infrastructure use structured release and deployment processes. Such processes have proven to decrease production release failures or impacts.

Multiple team members, including development team, client functional experts, management, and security experts (to name a few), review testing results and other factors. A review of remaining open items and newly discovered security items is performed to determine whether a system or service meets necessary compliance and quality.

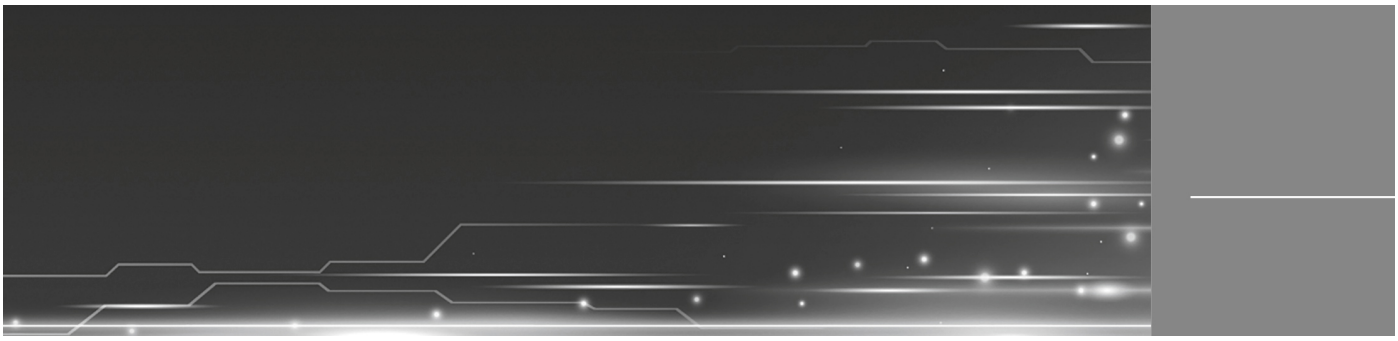
Some systems may require accreditation before being put into production. Project teams make sure the product meets accreditation requirements and goes through the accreditation processes. Accreditation is another way to confirm a product meets specific compliance requirements.

As part of pre-deployment preparation, those who will be using or supporting the product need to be trained. Without training, users are unable to use the product as fully as intended, and support personnel do not know what support tasks are necessary to keep the product in the best running order.

Finally, an ordered deployment plan is created, including a timeline for deployment, approved date for deployment, and a backout plan in case of deployment failure.

KEY TERMS

accreditation
Authorization to Operate (ATO)
backout plan
deployment communication plan
deployment plan
deployment window
final security review
migration
rehearsal
release authority
release criteria
release plan
release review
training



© d1sk/Shutterstock

CHAPTER 17
Operations and Maintenance

CHAPTER OBJECTIVES

- Examine the roles of operations in deployed software systems.
- Outline the planning and execution of incident management processes.
- Describe periodic and responsive maintenance tasks.
- Discuss the importance of addressing environmental concerns during operations.

Introduction

The life cycle of a secure software system does not end with the deployment of the system. Any system is going to need support throughout the time it is deployed. That support will include administrative tasks, such as adding users and permissions, assisting system users, monitoring the system to ensure it is functioning properly and securely, and performing periodic and one-time maintenance tasks.

Like cars, software systems have operational tasks that need to be performed. Besides the act of driving itself, cars require operational tasks such as buying gas, charging the battery, navigating, and securing the car (locking the doors) when it isn't +occupied. Drivers also need to monitor the car's instrumentation. The instruments provide critical information for other operational tasks (speed, fuel level, etc.) as well as warnings about potential problems (check engine and other warning lights). In addition to the operational tasks, periodic routine maintenance of your car or house is done to proactively fix something that is a risk of becoming a problem, likely at an inconvenient time. Such preventive maintenance includes oil changes, replacing the air filter, and washing the car. Other maintenance tasks are completed on a reactive basis, such as replacing burned out lights or collision repairs.

Just like a car (or a house), software services and systems need operations, monitoring, and maintenance on a routine basis. Day-to-day operations focus on ensuring the system or service continues to run as the business expects, just as it ran when first installed, or as the latest updates were installed. Such activities include monitoring for storage space issues, noting any network impacts or slowdowns, and reviewing software application logs for reports of impending issues. Activities falling under operations are performed by technical staff specifically trained on the system or service. In addition, there will be a designated maintenance team (which may be the developers). Some routine maintenance tasks and specific reactive maintenance tasks may be completed by the operations team, but more complex tasks will be the responsibility of the maintenance team.

The operations of software systems, just like the operations of cars, can have effects on the earth's environment. Choices about systems and how systems are or will be operated can increase or mitigate these environmental effects. Obviously, many of those considerations need to be addressed during the design and development phases. Consideration of how those will be

affected during operations is going to drive requirements and concerns being brought into those phases. Other environmental concerns will need to be addressed in ongoing operational processes.

Some operational or maintenance tasks may result in changes in the configuration or environment. These changes have to be applied carefully and thoughtfully and tracked. As with anything else, change not done properly, can introduce problems that are worse than those that drove the change. To avoid these problems, robust change management processes are used. The change management process will be discussed in this chapter in the Change Management section.

Operations

Operations is the process of managing, monitoring, and supporting systems to ensure they are functioning properly and securely, that users are able to perform their needed tasks, and that potential issues are identified as rapidly as possible, hopefully before they cause problems. Operations are usually performed by a dedicated group in an organization. However, in small organizations, operations tasks may be an extra duty.

The care and feeding (metaphorically) or operations of any software system or service requires people who enjoy working with such systems or services daily. Processes telling the team what to keep an eye on are provided with a new system install or an update to an existing system. The initial operations processes are provided by the development or maintenance team working together with the operations team. The development or maintenance team (in-house or vendor development team) has more initial in-depth knowledge of the technical aspects of the system than the operations team does. On the other hand, the operations team generally has more expertise on the organization's standard operational practices, the types of issues users encounter, and monitoring and reporting requirements. Determining what operations and monitoring processes need to be is an important part of the development or maintenance process and should have the involvement of the operations team during those phases of the cycle. Remember, many security vulnerabilities were evaluated and determined to be best covered through procedure rather than code. Addressing these vulnerabilities needs to be incorporated into the operational processes.

These initial processes define information on such things as what logs to check, on what regular basis, and what to look for in the logs. The operations team may have daily activities such as status checking, updating of queues, and other tasks. Other processes may include regular verification of file permissions (which groups of users get to do what with which files) unique to each system or service, how to securely set up the system, and what to watch for in the system configuration for indicators of possible **security drift** (see **Box 17.1**). Processes are also created which describe how to set up system or service users so that both the system and the users are as secure as possible from being breached and sensitive data copied.

BOX 17.1 Security Drift

Many system configuration changes that expose vulnerabilities are not one-time events, but a rather a series of configuration changes made over time that together gradually move, or *drift*, the security posture of the system into an unsecure state. Usually, no single change by itself is sufficient to compromise the system security, but the cumulative effects of multiple seemingly innocuous changes leave the system vulnerable.

The chapter on configuration management discusses the importance of version control. The operations team must also pay attention to the version of the software files (parts) that are on their systems that make up the products the business is so reliant on. As noted, all final products are labelled with a specific version label as are all parts that make up the product. Just one part with the wrong label (which means the software part is not confirmed to be compatible with all the other software parts making up the product) and the product is very likely not to behave functionally as the business expects nor as the product was designed to do. This could be a small difference or a critically impacting difference in behavior.. It may be a difference that has a large impact on the security of the system, such as a security bug or issue that was fixed in the current version but was not fixed in a prior version and a prior version is on the production system. In other words, wrong part versions can cause a multitude of issues from high visibility and high impact to low visibility and low impact.

Pre-deployment, the operations team does an assessment to understand the processes provided by the developers and to determine which recommended or additional, specific processes will need to be followed in the given environment. In this assessment, they will evaluate if upstream or downstream co-dependent systems have to be configured or managed in a specific way to make sure the new software works seamlessly within the entire cyber ecosystem. If so, the operations team adds the needed processes. The operations team also adds processes needing to be done on supporting systems (operating system, network, hardware, or other supports for the system).

As the operations team observes how the system functions over time, they will discover other needed processes. For example, we once had a Unix system with a poorly written process that failed to properly handle closing connections. When a connection to a system is stopped/terminated within a

process without proper closure, the Unix operating system takes over the connection. Eventually thousands of these can be present, which drastically slows down a system. As a result, the server itself had to be rebooted weekly to get rid of the orphan connections. Unfortunately, this type of experience isn't uncommon, and we have experienced quite a few of these systems over the years. In these cases, a process is written covering when and how to reboot the servers to get rid of the orphans.

Part of establishing operations is deciding on whether to use a formal, pre-defined, framework (such as COBIT or ITIL), a combination of two or more frameworks, or to develop a new operational framework. The basis for the organization's processes come from the framework that it decides to use. Using a pre-defined framework makes process development easier as the framework has pre-defined processes. We're now finding that combining more than one framework is best. This is because everything in one framework does not fit everything a business operations team might need. So, yes, we can pick and choose which processes to use from which frameworks that best fit the business and the software systems being use.

Specific procedures, such as when to do backups, where to store backups, or what type of multi-factor authentication is to be used in the company, are not defined in the framework. The organization will establish an internal framework for these types of procedures to ensure common approaches across all operational systems. Based on this internal framework, the operations team, with input from the business teams, will establish specific procedures for each system. The business users need to be involved to make sure processes cause minimal impact to the business while still being able to keep the systems and services running at peak performance. Some issues that should be considered are how frequently and when a system should be rebooted, hours of direct support, frequency of monitoring and checks, service level agreements for responding to user issues, and when maintenance windows will be available.

The operations team has processes on how to engage co-dependent teams. There may be regular meetings with other teams or there might be an indirect way to get co-dependent teams to check their system settings. There may be formal methods such as logged tickets in work management systems, defined email processes using group emails, or there may be understood and loosely defined contact processes such as "shoulder tapping" where people speak directly with one another. This last process is a problem because there is no record of what was asked and what work was performed on the co-dependent system. When mistakes are made (an incorrect configuration or perhaps an

incorrect password change), there is no audit trail to be able to quickly locate and correct the issue.

An important part of what the operations team does is to make sure the system or service is running as expected. Often systems look like they are 100 percent as expected on the technical end, but functionally they are not behaving as expected. Software systems are so complex anymore that it is almost impossible to know exactly how all the lines of code interact with each other. The vendor or internal software development team will perform functional testing when making changes. Unfortunately, frequently the timeline to get a product or update into production does not allow sufficient time to perform a set of full functional tests. This has changed as automated testing increases as well as clients demand increased quality. Once a new system or system update has been installed, the operations team will check with an identified key user who is familiar with the expected system functionality for any anomalies.

As part of regular operations, the operations team should have processes, preferably automated, that will perform some key user function testing on a regular basis and report back success or failure, much like a regression test. In this way, the operations team will hopefully identify an issue before the user community sees it, and the team can resolve it before business is affected. This type of operational excellence is what makes the operations team very valuable. However, often in businesses, if there are no issues, the nontechnical businesspeople start to think the systems are so stable that no regular operations are necessary. Nothing is further from the truth!

Monitoring

Another set of duties for the operations team is **systems monitoring**. Systems monitoring involves the periodic or continuous checking of system parameters to ensure that everything is functioning as it should be and to spot issues as they arise. Sometimes monitoring involves logging into and manually checking individual systems and processes. In most organizations, an operational monitoring system will be in place. These systems automate the process of checking parameters and receiving error messages from systems. Usually, the systems provide charts and graphs to help the operations staff through visualization. In addition, the systems can be configured to provide notification in the event of special error messages or if the indicators being monitored fall outside of normal parameters. Increasingly, these monitoring systems are adding artificial intelligence elements to help in the monitoring and alerting tasks. Often operations staff will write their own scripts to automate elements

of monitoring or other operations tasks or to adapt a process or system to work with the automated monitoring system.

Monitoring can also involve reviewing system logs manually or through automation. **System logs** are used to record significant events in a system for later review. Most systems allow logging at different levels. As the levels change, the number and types of events that are logged change, or the amount of detail in the information logged changes. Choosing the level of logging to use for any particular system requires a bit of judgment. While higher logging levels provide more events to review and more information about those events, higher logging levels also require more storage, can impact system performance, and may make ferreting out useful information from the logs more difficult.

Ideally, logs are not maintained on the same servers as the system itself. This can keep the logs accessible in the event of a problem on the server. It can also prevent the logs from being compromised if the server is compromised. This will result in a greater chance to detect a compromise from inspecting the logs as well as a great opportunity to reconstruct the compromise and study its affects. Segregated logs also contribute to successful recovery efforts.

The review of log files should be automated whenever possible. This should be done for several reasons. First, it is a *very* boring job to manually check the huge amount of system logs and other system indicators. Bored people tend to get distracted and miss things or find themselves in state where they zone out and fail to pay attention. Second, it takes more time than the operations team has to check these indicators and logs manually. And finally, users are prone to miss details and specifics, especially when they have been working on the same task for an extended length of time. Automating the review of logs can show us when certain specific types of situations arise, such as a failure or weakness in some part of the system or service.

Figuring out what to look for in the logs prior to automating their monitoring is the job of the operations staff. They are able to leverage their ability to spot patterns and anomalies to identify what checks can be automated. The operations staff should also periodically review the logs that have been checked by automated systems. This helps to identify new emerging patterns and issues that the automated system may not have been programmed to catch.

Monitoring activities should be recorded to provide a historical record of system activity and performance and to document the processes.

Security Monitoring

A specific form of monitoring is **security monitoring**. Security monitoring specifically focuses on events and activities that indicate the active presence of a malicious attack, a system compromise, or another security event. Security monitoring is distinctive from other monitoring activities both because of the high degree of risk associated with system compromises and under the principle of segregation of duties. A security threat may be internal.

While large businesses may have their own security teams that perform such monitoring, small and mid-sized companies often rely on their operational teams to watch for attempts to get into their systems by those nasty attackers. The benefit of separating the responsibility for security monitoring from the routine operations team is twofold. First, a separate team provides a double check, which can be useful in identifying internal threats or missed operational tasks. Second, the security team has a specific focus, which can prevent them from being distracted by other operational tasks. In addition, security team members are generally more educated on security specific issues and better equipped to recognize emerging security issues.

Organizations can also outsource much of their security monitoring. This allows them to gain many of the benefits of a separate security monitoring team, but at a greatly reduced expense. Different vendors offer different levels of monitoring and employ different techniques, so it is important to thoroughly investigate the offering before engaging any particular company.

If an organization is not able to either establish an independent internal security monitoring team or cover the expense of an outsourced solution, then the organization should establish processes that cross check the monitoring activities.

User Support

A part of the operations team is likely to provide **user support**. User support is direct or indirect assistance to the user in resolving issues the user may experience with the system. Often these issues are only indirectly related to the system itself or may be a result of challenges with training on or understanding of the system.

This part of the operations team may work directly within the overall operations team, or for particularly large businesses, there may be a dedicated systems and services support team. This part of the operations team is familiar with how the system (software) is supposed to act functionally as well as having some basic information on the technical and business operations. Having

a high-level understanding of functionality as well as being able to troubleshoot technical issues lets this important teams assist users when they call for help.

Support personnel help people use the system to the best advantage of the business, to use it as it was intended to be used, and to provide information on how to efficiently use the system. Often they are the first to hear of potential issues as they answer calls from the possibly tens of thousands of users. Because of this intermediary position between the business users and the technical team, support personnel are in a unique position to help increase communication between both groups. After all, communication is so important when trying to keep a business performing well, which includes business critical electronic systems and services.

Support also helps inform the users what new features are available or upcoming or of upcoming planned system outages. Support teams need well-written information on the system or service to be able to troubleshoot/help any callers. Part of the information they need is information on established processes. Support needs to know what the process is when someone calls for a password reset, including details on how to authenticate the person calling is who they say are and are authorized to be on the system. When an issue arises that the support team is approved to resolve, they need defined processes for resolving the issue in a structured, established way. This approach means repeatability, consistency, and decreased time to issue resolution because everyone knows their role in an issue as well as steps to follow. Instructions and the process where, when, and how to apply the instructions are required for support people to follow.

Not all members of the support team are going to have the same level of knowledge and skills. As a result, support teams are often divided into levels. Level 1, or basic support, is the first point of contact for a user. In addition to the immediate interface to the user, the level 1 support team begins the task of recording the support interaction. Whenever possible, the level 1 support team attempts to resolve the user issue.

Recording all support requests is important for several reasons. First, as each request is recorded and its handling is tracked, a record is established that can be used to ensure that each request is completed. Second, the actions taken to complete each request are recorded so that when similar requests are made, the actions can be repeated and consistently provided to all users. Finally, the records provide a trail of where users most frequently encounter issues. This trail provides inputs to revisions and updates of the system and its processes. Organizations implement trouble- or support-ticketing systems to support the recording and management of requests.

For those items that are beyond the capabilities of level 1 support, there is level 2 or intermediate support. This team handles many of the more complex situations or support requests that require management authority. While a level 1 support person may know about a limited extent of a system, or only about one specific system, the level 2 support team is going to have both broader and more specialized knowledge. The level 2 support team also has the option of bringing in additional resources. Escalation of an issue to level 2 can be done either by the level 1 team or by the user if the user is not satisfied by the level 1 response.

A support team may also have a third level of support for highly technical or specialized issues. Often this is an external group that provides level 3 support on an as needed basis. In some cases, this may be a developer, or group of developers, from the part of the organization that developed the system. Level 3 support may also be provided externally by the creators of a system or through a specialized vendor.

Generally, the support team has a specific service level they need to meet. This may be a part of the overall information system's **Service Level Agreement (SLA)** or a specific support SLA. The SLA will prescribe how rapidly the support team responds to different types of requests, the escalation processes that are followed in the event an issue takes longer than expected to resolve, the user is not satisfied with the resolution, or the issue cannot be solved within the support team. The SLA helps to determine the size, composition, and processes of the support team.

Backups

One of the responsibilities of the operations team is to make and restore **backups**. This may be the responsibility of a special team within operations, or it may be a part of the responsibilities of the system administrators, or another group. Backups are a copy of data and configuration of a system. Note that both are essential. The objective of a backup is to restore lost data to be able to resume operations. Data could be lost or unusable for a variety of reasons. Naturally, a disk crash or failure should be considered, and while not uncommon, it is not the only reason for needing a backup. Sometimes data get lost because of a poorly done system setting or a user or administrator accidentally deleted or began an operation that deletes data. Lately, a lot of data has been lost due to ransomware attacks, where the data are there but encrypted and lost to use. Other cyber attacks may simply delete data to cause harm. Finally, data can be lost due to manmade or natural disaster—all

of the exciting stuff like hurricanes, tornados, earthquakes, volcanoes, and air raids.

Backup strategy and planning begins early in the development cycle. Backups need to be considered as a part of the requirements. In the planning and design phase, the best approaches to backups for a secure software system ought to be included in the data architecture. Thought put into how the design of the data can enhance or hinder backups can yield a significant advantage when it comes time to put the operational processes into place. Designed carefully, a system can avoid needing downtime to create backups. For example, a system may distribute data both to the main data store and to a process that creates a backup of the data. A system may have a process that runs in the background, pulling a continuous or periodic backup that is synchronized to avoid missing data locked by other processes.

How the system gathers, uses, and stores its configuration information can also either help or hinder backing up the system. For example, a system that allows reading and entry of configuration data only through a graphical user interface (GUI) is going to be very challenging to back up and restore, requiring significant manual effort. On the other hand, a system with centrally located, file-based configuration data management is much easier to back up and restore.

At the early phases, planning for backups needs to consider the organization's specific approach to backing up systems. If this is a custom development, the backup approach will be tailored to the organization. For developing systems that will be distributed to multiple customers, the design should consider flexibility and an ability to easily integrate with standardized backup systems. For products, a solid approach to backups combined with flexibility to meet differing organizational needs can be a strong sales feature.

The backup plan is going to need an encryption strategy. Backups have the potential to be vulnerable because backups may be transmitted or transported to other locations for storage, or they may be maintained for lengthy periods on systems and in places that are not heavily surveilled. Data from backups are just as valuable as data from live systems. Encryption does add some complexity to the backup and restore process but is usually worth the effort. One of the biggest challenges of encrypting backups is key management. Because the data may be stored for some time, ensuring that the key isn't lost and is available when needed requires careful consideration. Putting a sticky note on the backup tape with the encryption key is not a good choice for managing this challenge. The channels used to maintain and manage the

encryption keys need to be distinct from the channels used to distribute and manage the backup data.

Determining what to backup is, at least on the surface, fairly easy. Anything that will need to be accessed again at any time in the future needs to be backed up. But that means that some kinds of data may not need to be backed up. For example, data used to calculate real-time information, but nothing else, can be ignored because, once it is restored, it is no longer real time. On the other hand, sometimes we don't think about data that will be needed in the future. That is where configuration information comes in. How a system is configured is vital in making it work properly and often essential in interpreting what the data restored actually mean. More than once, I have seen organizations that successfully backed up and restored their data, struggle because the restored system isn't properly configured and the data are unusable or corrupted.

The next concern is how frequently to back up the data. Backing up a lot of information can take a considerable amount of time. The process of making a backup can also put a significant load on a system. Sometimes, organizations use redundant systems to help with the issue by having data stored at the same time on one system used for processing, and another solely to be accessed for creating backups.

Another approach is to perform incremental backups. With this technique, a full back-up is made periodically, for example once a month. A weekly, or daily backup is then made that only contains the differences between the full backup and the current state. If a restoration is needed then both, or multiple, backups are applied in their proper sequence.

Organizations often store backups from multiple periods. This is in case one or more of the backups are corrupted or unavailable or if an event happened before the latest backup but was not yet detected. This allows the organization to reach back further to find a "good" backup. The challenge is then how to handle everything that happened with the data between when the backup was made and recovered.

Another issue is where to store the backups. Ideally, they are stored as close as possible to where they will be needed, but as far away as is needed to keep them safe. Some organizations find it convenient to use on-line storage options for backups. The challenge with that is if the organization can reach the backup on-line, so can a threat who is determined to corrupt or destroy the data. Several layers of security can lower the odds, but it is challenging to be perfectly certain.

Generally speaking, it is safest if the data can be stored in a place that cannot be reached by any connected computer. That may mean a hard drive that is removed once written or a tape system with tapes stored away somewhere. The location where everything is stored is going to depend on various threats the organization faces. Best practices generally call for keeping the backups off-site in case of fire or other damage to the location.

Sometimes it may be prudent to even have the data locked away in a vault on the other side of the state or country. But, before doing so, it is important to consider the actual circumstances of the company. Consider, for example, a company chooses to store backups on the other side of a state because the side they are on is subject to potential earthquakes and tsunamis. In the event one happens, would the organization even be able to get an employee to the storage location to restore the data? And if restored at the distant location, would the data be useful if the company operates locally but can't get to the data about its local customers?

Planning and preparation also needs to go into the restoration process. Some think it is sufficient to simply check that the data on the backup are good. That is necessary, but not sufficient. If an organization has not been through the process of doing a full restore of its systems and data, it will be hard pressed doing the restore in the middle of a contingency operation. The best approach is for the organization to rehearse the process. Not everything all at once though. Start with a simple talking walk-through over how everything would be done. Then scale up to a more complete exercise rehearsing individual sets of data. Continue scaling the exercises until reaching the point where the organization can perform a complete restoral. Participants in these exercises will be amazed when they discover the number of small details that make the difference between success and failure in the restoration process.

Incident Management

Regardless of how much planning and testing happens during development and pre-deployment, issues (incidents) do show up in production. These production issues are called incidents because they are unanticipated, a surprise, and cause more problems for the business than something that might have been found during the development process. We have to keep systems and services up and running to the highest level of functionality and performance to make sure the advantages provided by the systems and services help keep the business running optimally. Any impact to the business potentially put business income at risk as well as staff employment at risk. **Incident management** is the business process of responding to issues that disrupt routine operations.

As can be anticipated, incident management requires careful planning and involving the right people. The support team usually is the first to hear of an incident. They will then engage operations. Some incidents may have been anticipated as being a possible situation identified by risk assessments. The operations team determines whether the incident is known and if there is a **mitigation plan**. A mitigation plan are processes applied in the event of an incident to ensure the incident results in the least harm and disruption possible. Using the existing mitigation plan, operations can resolve the incident quickly. In that case, the usual incident management processes of communication and meetings are followed; however, they are likely to be brief and informational only.

Incident Response Plan

When an incident has not been anticipated and there is no known mitigation plan, a more in-depth incident management exercise is needed. This is the role of the **incident response plan**. The incident response plan provides the organization a road map of how to deal with unanticipated incidents. Many different types of incidents need to be planned for. Think about database incidents, application incidents, hardware incidents, or the worst—security incidents.

Organizations need to have an overall incident response plan with specific sections or subplans for each of the systems in the organization. These plans need to be developed before any system is first deployed. The plans need to be periodically reviewed, tested, and updated. An incident plan must be

documented and communicated with everyone and anyone who may need to be involved to help resolve an incident.

An incident plan requires many parts, including the definition of what constitutes an incident, assessment, personnel, communications, processes, and procedures. The incident definition section of the place provides criteria to allow the responsible party or parties to declare an incident and officially begin the incident response process. This section would include guidance on how to recognize an incident has occurred or is occurring. Not every event is an incident. A script kiddy doing some port knocking that is stopped by the firewall is not an incident. Network-level scans are a constant and regular occurrence for any internet-connected and internet-facing device such as a server. These scans look for the presence of open ports or other potential vulnerabilities and are only a concern if allowed through the firewall (or if the firewall itself is vulnerable). Indications that a server has been compromised and someone is exfiltrating data is an incident.

After an incident has been identified, it needs an **incident assessment**. For any system or service covered in the incident plan, criteria are defined so that an incident can be evaluated for its impact to the business. The plan will specify the authority to conduct the initial assessment. The bigger the impact, the more critical the incident and the more resources are likely to be pulled in to help resolve the incident. The incident assessment will attempt to both understand the nature of the incident and the extent of disruption due to the incident. In most cases, an initial assessment will be followed by updates as the extent and nature of the incident are more fully understood.

Based on the initial assessment, the assessing authority will activate the appropriate incident response team. What each expert or team involved in a particular incident type is expected to contribute to an incident is defined in the plan. As well, some incidents can be resolved at the frontline level, while other more impactful incidents will require upper-level management involvement. It may be necessary to pull in outside assistance (internet service providers, electricians, perhaps even police in the case of a cyber attack) and define who will engage such external sources.

For anything more than a simple incident, the response team will be divided into subteams. The first team is the **incident manager**, or incident commander, and their advisors. The level of authority of the incident manager will depend on the seriousness of the incident. It may be a line manager, or it may be an executive in the organization. The incident manager is authorized to make decisions related to the incident. The advisory team provides input and may be called upon to assist in decision-making. Members of the advisory team may

include representatives from legal, finance, human resources, public relations, and business units. The team members advise in areas of their expertise as well as engage with their organizational units to implement portions of the response.

The response team includes a **communication team**. The communication team may include both internal and external communication specialists. The team will operate according to a communication plan. The plan will include methods for internal communication, such as telephone fan outs (where specific people are identified as primary contacts who are then responsible for communicating the update to a set of people), electronic bulletin boards, text messages to all corporate phones, recorded messages on support help lines (perhaps help desk). The plan will also cover when external communications to customers, news media, the public, and others are needed and the methods employed for those communications.

The communication plan specifies who, which team, what type of communication (meetings, email, other communication methods) and how often communication is performed. How communication is documented, where it is stored, and for long is also part of the communication plan. This allows for real-time capture of notes both during incident resolution as well as for future review after an incident has been resolved.

The response team will include a reaction team and an investigation team. The **reaction team** is responsible for mitigation, remediation, and restoration. This team works to mitigate damage to the business and assets caused by the incident, remediate the systems and processes to remove continuing threats and problems, and to restore the systems, data, and business processes and functions.

Experts across multiple specialties are included in the reaction team to troubleshoot where the incident could be arising from, stop further damage or degradation of services, remediate those damages that have occurred, and restore services and functionality. This team is tailored to the needs of the specific incident. While similar groups of people are called in to help resolve each incident, specialists will be needed for specific types of issues. For example, a network incident may not need a database expert.

Once an issue has been identified, the team will determine the best manner to apply a fix to the situation. In some cases, the team may apply a preplanned decision tree. Having thought through potential scenarios and developed contingencies for those anticipated both allows more rapid response and ensures those responses are more likely to be successful because the team had time to think through the response when they were not under pressure.

The proposed solution is reviewed by the incident manager and appropriate members of the management team.

A fix may follow the usual processes, or the situation may be so impactful that emergency procedures need to be followed to get the fix into production as quickly as possible. A temporary fix to get a system or service up and running immediately might be used to decrease downtime and business impact. Even in the case of an emergency fix, the team follows specific processes to decrease possible production impacts and to identify when, how, and who will make the emergency fix. When the team applies a temporary quick fix, they will also develop a longer term plan to complete the complete solution. Applying the longer term solution may be the responsibility of the reaction team, but often the solution is passed to the appropriate teams in the organization for application using standard business procedures.

Once a fix has been put into production, the technical team performs an initial validation and certifies the system ready for use. Validation of the solution will include operational testing. With the solution in place and initial validation completed, the reaction team will communicate a service restoration plan to the users of the system. Service can be restored all at once, or the team may choose to begin staged restoration with selected functions or users. As service restoration proceeds, the team will closely monitor the systems to ensure that there are no surprises, hidden problems, or other emerging issues.

After confirming that the restoration was successful, the reaction team will document their response to the incident, any system changes implemented, and other useful information. As they complete their documentation, the team will hand over the system to standard operations. The team will participate in the incident review.

The **investigation team** is separate from the reaction team. The investigation team has the responsibility to investigate the cause of the incident, to gather and preserve evidence if the incident may involve a crime or has the potential to generate civil legal actions, and to investigate potential related incidents and anomalies. The investigation team in not focused on service restoral or limiting damage. In fact, in some cases, those kinds of actions have the potential to destroy evidence, hide the cause of an incident, or cover other suspicious activities.

The conflict of interest between the needs of the investigative team and the reaction team is why there is a separation of duties between the two. The investigative team needs to be focused on what actions most benefit the investigation, not service restoral. That is not to say that they don’t care about service restoral, but sometimes long-term concerns may be more pressing. For

example, if a system is becoming unresponsive frequently, spending more time investigating the causes may result in less total downtime for the system over the long term. In other cases, when an intruder is detected in the organization's networks, the investigation may want to track the intruder and learn what methods the intruder used to gain access rather than just immediately locking him or her out of the system.

When there is a conflict between requirements for service restoration and the needs of the investigation, the incident manager will need to make the decision. The investigations team and the reaction team will both make their case to the incident manager, who will weigh the arguments and make a decision in the best interests of the organization. In some cases, this decision may be influenced by external factors, such as subpoenas from legal authorities.

The investigation team remains in place for the duration of the incident. If the investigation is completed early, then the team can be on standby in case additional evidence emerges. If the team has not yet completed its investigation by the time service has been restored, the team may be continued on, or the investigation may be handed over to another team under the organization's standard procedures. In some cases, the investigation may require substantial efforts to determine not just what happened in an incident, but what the root causes were.

Incident Root Cause Analysis

A **root cause analysis** is a tool to understanding not just the immediate cause of an incident, but the full set of circumstances and situations that lead to the incident. Root cause analysis is a methodical investigation of all of the circumstances surrounding the incident and is important because it can uncover systematic issues and prevent future incidents from occurring. Sometimes even seemingly unrelated incidents stem from the same root causes.

The root cause analysis, while initiated as the result of an incident, is not a part of the incident response. The team completing the root cause analysis needs to take a measured and methodological approach to their investigation. This requires time, and it also requires the authority to pursue the investigation wherever it may lead. At times this may become politically unpopular within the organization. The team also needs to investigate beyond the immediate incident and identify other related problems.

Root causes may be the result of technical issues, or they may initially appear to be the result of technical issues. Under the surface though, those technical challenges may in reality be the result of process faults or other

causes. Sometimes, technical issues provide the patterns that point toward the nontechnical root causes of an issue. For example, if buffer overflows are being identified as the immediate cause of a number of incidents, that could be an indication that either the development team has not been sufficiently educated on secure coding processes or that the tools the team uses (compiler in particular) have not been properly configured.

Incident Tracking

Incident tracking is an important part of incident response. Each incident needs to be tracked in a central application. Details regarding the incident get recorded in this central application. Details always include what system or service appears to be impacted, what its importance to the business is, when the incident was first reported, what symptoms were first noted, and who reported the issue(s). As the incident response team works through the issue, notes are captured for review during root cause analysis. How the incident was resolved is recorded in the application along with dates for further follow up and a final fix to be completed (if necessary).

The documentation of the incident in the tracking system and associated locations is important for multiple reasons. The documentation proves invaluable in many ways, both when trying to resolve a previously unknown issue and when reviewing the incident to determine whether the fix put in place is permanent or a temporary patch. The documentation is also valuable if another similar incident occurs. If the fix was a temporary patch, then post-incident analysis using information from the captured documentation will help teams to arrive at a more permanent resolution to ensure the issue does not happen again.

Disaster Recovery

Some incidents will rise to the level of a disaster. This happens when the incident is of a magnitude and duration that it represents an existential threat to the organization or the community where the organization is located. These cases are addressed in a **disaster recovery** section of the incident response plan. The disaster recovery section is created during planning based on worst-case scenarios. As these scenarios are imagined, the planning team develops potential methods for mitigating the impact of the disaster or methods to aid the organization in recovery. The organization applies a risk assessment strategy and determines which of these are viable for implementation. For

some strategies, such as process-based strategies, the cost is low. Others, such as a backup location, may require a significant expenditure.

Preparation and Practice

Any incident response or disaster recovery plan is likely to be highly ineffective if it is only paper sitting on a shelf or electrons spinning in cyberspace. To be effective, teams need to prepare and practice. **Preparation** means that the teams need to have read and understood the plan, know their role in the plan, know what is expected of them, and have the tools ready and available that they will need to execute the plan. When the power goes out, it is too late to go buy a generator. Even if a capable generator could be located, there will be challenges in getting it on-site, getting it fueled, and getting it connected to the load. Likewise, getting specific software tools may be possible, but not very valuable if no one knows how to use the tool.

After becoming prepared, the team needs practice. Tasks that are written on paper but have never been performed may not work, or there may not be sufficient information on how to perform the task. The plan may have critical data missing. In addition, it is important for the people on the team to know each other and know how to work together. The only way to identify the missing information and to get the team working together is through practice.

Practice comes from implementing and following the plan. But it is never a good idea to practice by implementing the plan for the first time for an actual incident. A better approach is to use **drills**. Drills are implementations of the plan based on simulated, or notional, incidents. Drills usually begin as simply as possible. Often this is done as a walk-through.

In a walk-through, the members of the team are gathered and presented a notional incident. Based on this incident, the team members go through the actions they are expected to take according to the plan. At first this is done only by talking their way through the plan. As the team progresses in their abilities and responses, the physical actions are introduced and followed. Eventually, the drill becomes a full-scale response to the notional incident.

As the drills progress, the team will identify any weaknesses, missing components, or conflicts in the plan. The team addresses these issues, adjusts the plan, and then drills using the improved plan. Eventually, the team reaches a state where it has full confidence in the plan and in each other. At this point, if an actual incident occurs, the team is ready and responds quickly and effectively.

Post-Incident Review

A key part of the incidence response process is the **post-incident review**. After each incident, the incident manager and the rest of the team will review the response. This will include a critical assessment of the conduct and effectiveness of the response. The goal is not to apply blame for any action but rather to allow the team to learn from what occurred and thereby to provide an opportunity for improved responses in the future.

The review needs to include performing a risk assessment based on the incident information as well as the original risk assessment information. By performing a post-incident risk assessment, we are more likely to see risks not previously seen and that will feed into a final resolution to stop a similar incident in the future. After an incident, we need to assess whether the incident was negatively impacted by other systems or services. Without this information, it may not be possible to arrive at the best resolution for the experienced incident.

Maintenance

Maintenance involves the tasks taken to anticipate, respond to, and apply changes, modifications, and improvements to a system. Maintenance may be periodic and routine, or it may be applied in response to specific events and conditions. Maintenance may be performed by an organization that uses a system, by the organization that develops a system, or by both. Maintenance of systems focuses on increasing reliability, performance, and security.

While routine monitoring and operations may be performed by the day-to-day technical operations staff, often larger or specialized maintenance chores are performed by specifically assigned staff who focus on longer term, proactive changes, or possible unintended, unanticipated issues in the system or service. These maintenance people will perform proactive preventative activities that are focused on removing potential issues before they become a problem. For example, older data may be compressed and archived to help ensure that there is sufficient storage space for new data, periodic tests may be performed on network connections to ensure there remains sufficient bandwidth, or assessments may be performed from different points in software processing to ensure that there are no missed or inappropriate data. Once processes and procedures have been developed, the maintenance team may hand off execution to the operations team for day-to-day implementation

Maintenance staff look at software patches or fixes for each system and decide whether the patch is needed for their particular system. The team will also evaluate new security patches and new security approaches to system and service configurations to determine new processes for the operations team to follow.

Increasingly, maintenance teams are taking proactive risk management strategies to increase their effectiveness. Rather than simply waiting for the next system to fail, the team monitors performance indicators and begins to anticipate possible future problems and mitigate or resolve them *before* any problems can impact smooth running of the business-necessary systems or services. Resolution (mitigation) of the risks found from a risk assessment may require proactive action, such as replacing hardware, replacing an existing software system or service, or updating an existing software or service. Each of these options will result in one more system or service being impacted, possibly to the point of needing to have an outage. Businesses are not very

accepting of outages because they almost always affect business operations to some degree.

The maintenance team also looks at optimizing hardware and software. They focus on tuning software that supports business applications or any of the many systems and services that support the important business applications. Tuning may involve configuration changes or possibly even a software patch installation. As a part of the process, when considering any type of change to a system or service, the team will launch an investigation into whether the configuration change or the patch version are compatible with the current environment configurations, and whether it will be expected to have the desired results.

If there are one or more copies of the same system or service (to support redundancy and eliminate system or service outages for any reason) all environments must have exactly the same configurations. These configurations are recorded in whichever configuration management tool the company is using. Note that there is likely to be different configuration management tools for different purposes.

All good maintenance engineers are constantly investigating how to increase system performance, such as speed of throughput, optimal bandwidth, and efficient data packet routing as well as reliability. They evaluate which new hardware on the market has been shown to be more reliable, what type of hardware upgrades are needed, or if there is a need for redundancy. The team will also evaluate hardware for security and how it will affect the probability of cyber attacks.

Hardware reliability over the past 30 to 40 years has increased dramatically. Yet hardware still fails, and when trying to support upward of tens of thousands with minimal staff streamlined and automated hardware checks become even more important. The maintenance team will define hardware expectation parameters as well as how to monitor for when hardware acts outside of the prescribed parameters. These hardware parameters are updated on a regular basis. Either maintenance or operations will do the ongoing monitoring.

Automated hardware monitoring is *great*, but only when it is installed and set up. If all servers are expected to have a remote monitoring card installed and set up but don't, it can lead to surprise hardware failures. A number of different software packages will also monitor hardware state remotely. Whether an extra piece of hardware or some software is installed on hardware, as long as it is reachable remotely, staff will be notified of failure of any part of the hardware. This allows hardware to be placed where space and electricity are less expensive.

Configuration and Software Updates

Before any changes can be made to a system or service, the proposed change should go through the established change management process. When the operations, maintenance, or information security teams become aware of needed software or configuration updates, those changes would go through the change management process for approval. Once through the change management process and approved for the change to proceed, the team(s) then work through the established process to schedule and communicate the change date and time.

After approval, the process to make a change to any system or service follows many steps (phases). The change process helps avoid bad outcomes. All changes to production systems are first done on pre-production systems. If there is a development environment, the change/update is done there first. After being applied to the development environment, both technical and functional people perform testing to be sure there are no issues or errors. Although changes were tested when they were made either by the vendor creating the product or the in-house team who developed the product, issues may still arise. No system is exactly the same. The systems may be on different hardware or the target environment may have slightly different configurations. As well, because these systems are so complicated, we cannot fully know how a change in some part of the system will influence other parts of the system.

After applying to a development environment and waiting some time to see if issues arise, the same change is done to a pre-production test environment with more testing. Next, we test and wait to see if any unknown issues come up. *Finally*, if no complications have surfaced from the change or update, it is applied to production systems. Performing configuration or software updates in this type of sequential and organized manner decreases the probability of a business-impacting problem showing up in production.

Any change that causes a previously working part of the system to fail is called a **regression error**. These errors can occur when there is not enough time to completely test all the previous working aspects (technical and functional) of the system. Sometimes we find issues that only occur in specific types of environments during this final round of changes and testing.

Functional Fixes and Patches

Functional fixes include advised configuration changes as well as software patches that affect the front-end functionality important to the business.

Patches are changes to system software provided by either the vendor of the product or by group in the organization responsible for maintaining or updating the product. These changes follow the same change management process as any other change. The primary difference is who will be involved in approving the patch for install as well as approving that the new functionality meets expectations once installed.

For technical patches with no functional impact, the technical (maintenance) team will propose and approve a new patch or configuration change, while a functional patch will be proposed by the functional team and approved for install by the functional team. The technical team reviews any technical impacts to provide technical feasibility to install the patch; however, the driving force is the functional team. Functional changes should have full sets of regression testing done just to make sure no previous functionality has been affected. The new functionality is also thoroughly tested before the patch is applied to a production environment and released for use.

An important part of managing a system or service is deciding on what changes need to be made to all parts (tiers) of any system or service. The need for change in a system is going to be driven by a lot of different interests. The need for change may be the result of business process changes or changes in the organizational mission and objectives. However, much of the need for specific changes is generated from observing the day-to-day operations, the challenges that users face, the issues that occur when the system doesn't quite manage to perform as it should (or could), and any unnecessary constraints the system imposes on users and managers.

Security Configuration and Software Updates

Security changes such as a configuration change or software update may be performed by the standard maintenance team or by a specific security team. Security changes may only affect the back end of systems, such as network firewalls, in-transit encryption, in-transit malware checking, or other security related items, *or* they may impact front end or functional activities. When functional activities are impacted by security changes, more lead time, increased communication, and training are required. If a security change will impact business users, then coordination regarding communication and training need more time.

Security updates such as anti-malware software updates, are done on a frequent basis. New viruses are created and released every day by attackers, so antivirus companies provide frequent malware signature updates as soon as

possible to all customers. Companies then distribute the new anti-malware updates to internal systems. Prior to installation, the team will always test the update just in case the new updates cause a previously unknown issue.

Those performing security maintenance will assist the incident response team with resolving or cleaning up a security breach. They will determine, with input, the procedures to be used when malware is found on a system or service. Removing malware from a large server or storage system takes a careful plan with multiple steps. While entire courses cover computer security and forensics, few approaches to maintenance will use to resolve malware. The first and easiest is to allow the anti-malware being used to clean up any malware. When a new, previously unknown malware surfaces, the anti-malware vendor is immediately notified so it can begin working on a fix. The issue is also reported to national reporting databases, such as Common Vulnerabilities and Exposures (CVE).

Once malware is detected within a company and a fix deployed, a project team is assigned to follow up on all computer systems to make sure the malware is eradicated throughout the company. The project team may reverse engineer the malware to determine its impacts and possible fixes. Only when the project team determines the malware threat has been brought under control is the team disbanded.

Patches specific to a particular application or service follow the same change management process as any other software update or patch; however, they may need to be expedited to mitigate a recently revealed vulnerability.

Vulnerability Management

One large area of either maintenance or information security, given the ever-increasing number of cyber attackers, is cybersecurity maintenance. While there are frameworks such as NIST 800-53 or NIST 800-171, they are always changing with increased security requirements to keep up with the attackers. Maintenance or information security teams are constantly needing to keep up with the latest security requirements, investigate how to implement the new requirements, and plan for the new requirement implementations. They pass new processes and products specific to security to the operations team for daily support.

Either the operations team or a dedicated cybersecurity team will be responsible for monitoring systems and services for attempted (or successful) cyber attacks. All types of malware need to be monitored for and cleaned from systems. Either maintenance or information security teams will propose

security software to be used on corporate systems as well as propose any updates as new updates are released. This is a never-ending job, which is only increasing in breadth and complexity with the ever-increasing number of attackers and attack methods.

A significant part of security maintenance is maintaining the vulnerability matrix and threat model. These tools are initially created early in the secure software systems development cycle and have been updated during the various steps of the cycle. Once the system enters the operations and maintenance phase, the value of these tools does not diminish. If anything, their importance becomes even more critical as they are the method by which we become aware of changes in the security environment and can take action to preempt pending challenges. As a result, there needs to be a scheduled periodic process to ensure these tools remain current.

Change Management

Change management is the process of making sure that any changes applied to systems are done in a systematic and disciplined manner with proper authorizations and complete documentation. Changes to any system or service must follow a defined procedure that includes reviewing how the change affects the system and the business as well as approval requirements before the production system is altered.

Often organizations will implement **change boards** as a method of reviewing and approving changes. The change board is composed of appropriate personnel and given the authority within the organization to approve designated changes. There may be multiple different change boards, each dedicated to specific types of changes, or there may be one change board for each particular type of system or service.

Organizations establish processes to ensure that the change boards act in a consistent and organized manner. These processes include a documented and communicated method of how to capture suggested changes, expectations of meeting frequency, how the change board will communicate within the change board as well as with other teams. Details such as meeting type and frequency along with communication methods with internal members and external teams is defined as part of the change board processes.

On occasion there will be more than one change board for a particular system or service: one for needed technical changes (for tech insertion), one for functional changes, and potentially one for security focused changes. Alternatively, there may be one change board for a specific system or similar

set of systems covering all requirements, technical, security, and functional. One common type of change board focused on functional changes is a customer change board (CCB). The CCB may play an advisory role in approving functional changes, or the organization may give it full authority.

Change boards may be multidiscipline with technical, functional, and security members, or they may be made up on a specific discipline. This composition depends on the types of changes being captured. Functional changes need to be looked at to be sure they will sustain or improve the system or service's business contributions. Technical change board representatives focus on evaluating any proposed change from a technical perspective, including feasibility, effort to implement, and possible other technical impacts. Security representation is even more important because it must provide input into any possible security effects of a change.

Change boards are developed by an organization based on its specific needs. An authority within the organization appoints the members of the board. A change board may be assigned the procedures and processes they will be governed by, or they may develop the processes and procedures themselves. A change board performs many tasks. The appointing authority determines the scope of change board activities, including the types of changes the board reviews and the type of decisions they make. Change boards typically evaluate need for technical, configuration, or functional (to increase business performance) changes or patches. Implementation security changes needed to fix vulnerabilities also are reviewed by change boards before any action is taken.

Organizations establish methods to track suggested changes so that all changes are documented, tracked, and reviewed in a similar manner. Applications are available to track requested or proposed changes. Sometimes spreadsheets are used, but they can become a little difficult as the scale of changes grows. The early period of tracking via spreadsheet can be useful in defining the metadata used by the more robust type of system. This can be difficult when just setting up the change process.

Any change must be evaluated by members of the board and possibly other company members. The evaluation process is defined and includes a change approval method, such as how voting is performed (through email, formal board meeting) and captured.

Just as with any system or service update, any approved change must be tested. The change board may specify the amount and rigor expected for an upcoming change. All changes go through testing. Testing is done on at least one pre-production system to test whether the update can be technically and

successfully made (installed) as well as confirming that new functionality works along with the old functionality.

Companies may create a separate, specific change board for security issues. Such a change board could be made up only of information security experts. In this case, the change board's mandate is to keep up with new threats and vulnerabilities. An information-security-only change board also leads efforts to assess when and how to apply necessary fixes to software vulnerabilities. The board will coordinate with business users for the best time to apply fixes if applying a software fix requires a system impact (usually an outage). If a fix does not require a system outage, it can be done (after adequate testing in test environments) at almost any time. If there is no business impact.

BOX 17.2 Patching Cautions

- Patches do not always work the same in all environments.
- Changes to systems may require changes to databases, filesystem, and configurations.
- Access controls may need to be removed and then reinstated after an update. Missing the second step is a common critical problem.
- Configurations may conflict with each other.

If there is not a complete, separate security change board, likely a subset of a technical or functional change board will primarily examine security issues. Business users have to be part of the decision process on when, where, and how to implement a security issue fix if such a fix means an affect the business.

Finally, after any configuration changes or patches (software updates), all documentation must be updated. This means updating the architecture diagrams, threat model, and information sources regarding the application (on what patch level the application is and what the latest changes were for).

Maintenance Windows

Once any type of change to a system or service has been determined to be necessary, it must now actually be done in the production environment. We cannot just arbitrarily pick a date and time to make the change and slap it on.

Always, the goal is to minimize impact to the business. The business therefore makes the final decision on when a software product change would be of least impact. A **maintenance window** is the time allotted for a system to be unavailable or operate in a limited availability mode while changes are made to the software and systems. Maintenance windows can be from a short time to quite a long time, depending on the size of the change being implemented.

The maintenance window is scheduled based on the organization's operational needs. The scheduling is done as far in advance as possible to allow for planning both for the technical teams and the affected users. Ideally, maintenance windows are scheduled during periods when they will have little if any effect on most users or business operations. This is why many updates and installations take place at night, on weekends, and over holidays.

The best periods for maintenance windows will vary significantly from organization to organization. For example, many retail businesses prohibit all systems changes, except urgent updates, from September through December. This is their busy period where holiday purchasing represents a significant portion of their annual revenue. The retailers are hesitant to allow anything that could interfere with sales during this period. Schools and universities often try to schedule their maintenance windows during the breaks between terms. Many manufacturers try to schedule their maintenance windows during a summer shutdown period when other tooling in their factories is also being refurbished.

The maintenance window length is often determined by the technical team. However, in some cases, the maintenance window length may be determined by the business and the technical team because they need to figure out a way to accommodate any updates within that window. Some updates, software or configuration, are quite fast to apply and test, and other software installations take longer and involve more technical specialists, such as network, information security.

The technical team decides how long an install, update, or configuration change will take. Once again, representatives from any nonproject technical team that would need to make changes to their system or service are part of the planning and will be part of the actual deployment. The time needed for the deployment is based on past experience during the various testing exercises they followed, as close to the same procedures as possible on testing environments of the same or *very* similar structure and configuration. Not testing on an environment similar to production can be bad. In the largest software company on earth, an update was deployed to a worldwide business-to-business portal and application. Unfortunately, the testing environment was

much larger (larger servers, more network bandwidth). The update went well on the testing environments but failed horribly in production. As a result, the portal was inaccessible for days, costing the company tens of millions of dollars. All the testing should give the team a close estimate to the time necessary for the same activities to occur in production during deployment.

Included in deployment tasks is a technical smoke test or check out of the production system *before* it is handed over to user subject matter experts for final functional smoke testing/sanity check. Only after numerous validations and verifications have been done in the just installed or just upgraded production environment can it be released for all approved business customers to use. We never want to release something and then have support receive a large number of customer calls stating the production system is not working.

Any good project manager will take the technical team's assessment and add extra time to allow for possible unforeseen issues (which frequently arise). These issues can be anything from a power outage somewhere critical to a team member being ill at the last minute and requiring a less experienced team member to participate in deployment. Generally, the technical activities assessment time frame will be anywhere from an hour to a full 24 hours, depending on complexity. Once the technical length of time is known and some extra time has been added to it, the project manager will speak with the subject matter expert testers to get their assessment of how long it will take to perform functional testing—for them to feel comfortable to release the production environment after the install or update is done.

Next, the most important deployment date and time approvals are negotiated —with the business users and customers. Business users can be worldwide, which can leave very few days and hours of the day that a critical system or service is perceived to be able to be taken offline—made inaccessible—for any length of time. Often the change board that approved the change will work with the larger business community to determine an acceptable day given the length of time needed to do the install or update. Some organizations have preplanned scheduled maintenance windows when all upgrades and improvements are expected to be applied.

Once the date and time for the deployment or maintenance window have been agreed to by everyone, it is communicated across the company. Once again, early and effective communication is necessary to increase the likelihood of minimal business impact, increase coordination across all teams, and be more likely to have a positive update reception. Put another way, if everyone knows as far ahead of time as possible that an update is coming and when the update is going to take place, it is far more likely to go well, and the customers

are less likely to be inconvenienced. Communication is done through as many different methods as available within any particular company. This includes internal newsletters, email distribution lists, announcements at various meetings, and a general fan out through management layers of the company. Flyers may even be posted in central areas or messages added to computer sign-on screens (this last is particularly effective). Start communicating early and do it often.

As the maintenance window approaches, the team performing the updates will prepare by staging any hardware or other items needed, with as much of the software systems components as possible already installed in the vicinity of where the installation or update will take place. The team also completes any tasks that can be completed without affecting the service performance of the system. This may include performing backups or loading software into staging directories on the system. The goal is to have as much completed prior to the window as possible.

Once the actual window begins, the team will drop and lock out any users who will not be participating in the upgrade. The team will then conduct a quick review of the system to identify any unanticipated anomalies. This could include discovering malware on the system or identifying unauthorized internal changes to the system that have not been reported. The checks should also include a quick review of hardware function. Any anomalies spotted during these checks need to be accounted for because they could negatively affect the outcome of the changes.

Having made their checks, the team will then proceed with their maintenance tasks following a checklist. As they proceed, the team will need to keep a careful eye on the clock. Because of the potential for severe negative consequences of overrunning a maintenance window, the process may reach a **critical decision point**. The critical decision point is the last point at which there is sufficient time remaining in the maintenance window to execute the backout plan. At this critical decision point, there are two options: (1) proceed with the risk that if anything else takes longer than anticipated on the timeline, the tasks will exceed the maintenance window, and the team and organization must be prepared to accept the consequences, or 2) execute the backout plan.

The **backout plan** is a plan to remove any installed changes and restore the system to the state that it was in prior to the start of an upgrade. This would include time needed to restore previous versions of software, set the configurations to their previous settings, and restore any data. From the functional perspective, the system should be exactly as it was prior to the maintenance window. Sometimes, the backout plan is executed after the

maintenance window if issues with the upgrade emerge over time that were not spotted during the testing performed in the maintenance window. The only thing worse than having to execute a backout plan during these circumstances is not having one.

Provided the upgrade is proceeding according to plan, once all of the changes have been put into place, the technical team will conduct checks of all technical aspects of the system. When sufficiently satisfied, the technical team will have the business team perform functional checks of the system. After a final check out, the teams release the system again for all business users and customers.

Changes in how systems and services are designed and architected can make business impacts (taking a system offline) unnecessary. Business critical systems or services are expected to have redundancy so that one copy of a system or service can be taken down and updated while the other copy is still up and running. These types of design not only add to business stability through system redundancy, but they also make maintenance scheduling quite a bit easier because there is minimal, if any, impact to the business. However, even the most redundant systems need the occasional maintenance window.

Environmental Issues

An ever increasingly important part of planning, developing, and operating systems is the environmental impact of the system. Computers, computerized devices, and the networks connecting them are consuming ever increasing amounts of electricity as well as other scarce natural resources used for their construction. Most computers and technology are only used for a short time and then are discarded. This e-waste contributes to another set of problems and environmental degradations. In addition to the power and resource demands, software systems have other significant effects on the environments in which we live and work.

Choices that are made throughout the secure software systems development life cycle are going to determine how these software systems will affect the environment. Those choices are all too often made with little if any consideration or awareness of the consequences. Taking positive control of simple choices can make a significant difference in how systems affect the environment.

Paying for electricity to run tens of thousands of servers along with other devices necessary to support applications and our current interconnected world is expensive. More important is the environmental impact of how electricity is produced. Much of the world's electrical production is not good for the environment, whether that is using dams or burning fossil fuels to produce electricity. Recent electronic devices have evolved to become increasingly operationally efficient. They are faster and use less electricity, but there are more of them. And some computing, such as Bitcoin mining, takes such huge amounts of electricity that countries have banned their use.

Power Consumption

One of the most immediate concerns about systems is the power they consume. The power is most obvious when we plug computers into a circuit and feed power into the processors. However, computers draw power in other ways as well. We need to connect to the internet or some other cabling system to link all company sites or provide external customers the capability to do business electronically with a company. This interconnection includes routers, modems, servers, and a myriad of other physical devices. Also the power used for cooling the systems needs to be considered.

The first steps in reducing the power consumption for computers are done it the design and development stages. Here the choices are made about which systems and components are going to put into the system. Here also are the choices about how to implement algorithms in energy efficient manners. But just because it is too late to influence those choices when we are at the operations and maintenance stage doesn't mean we can have a significant influence on the amount of power our system consumes.

A significant influence on power consumption is in ensuring that our system servers operate efficiently. Systems that are overpowered consume electricity needlessly, but systems that are underpowered consume even more excess power. Systems running within their design constraints are able to work within a more efficient power level. In addition, using a cloud type of architecture, whether it is in house or remote, allows for the efficient sharing of resources.

While not yet common, a number of companies are evaluating the use of green electrical supplies to provide their necessary electrical needs. This is done by using electrical companies that have invested in green power generation from sources such as solar or wind. As well, very large companies that build and run their own data centers are beginning to look at incorporating green energy production into their buildings—adding solar panels or evaluating the use of windmills to produce electricity for their data centers.

The design and operations of a data center can also have a significant effect on the amount of power used for cooling. Efficiency in the airflow and the use of ambient air when possible reduces cooling needs. In many cases, the excess heat from a server room can be reused to warm other locations in a building. A significant amount of research has been conducted on designing cooling efficiencies. Organizations need to consult this when they create and operate their data centers.

E-Waste

As well as electricity needs, all physical devices are made up of materials such as steel, plastic, and scarce materials. Many of the materials in a device and how the device is made have environmental impacts, such as pollution during mining (such as lithium) as well as the production process. During production, many other chemicals and materials are needed that are also air and water polluting. We *are* getting better as we pay more attention to the environment; however, there will always be some impact to the environment in the production of any product. How much is up to us as we recognize the environmental impacts and work to decrease them as much as possible.

E-waste is the term for the discarded bits of electronics and computers that contain plastics, electronics, and hazardous substances. The simplest way to prevent a device from later becoming a part of the e-waste problem is to not buy the device. Just because something can be done with a device doesn't mean that it should. Organizations need to make better choices about what they are going to deploy, and when they do choose to deploy something, they need to make sure it is a long lasting, useful device with minimal residual impact.

Another method to reduce e-waste is to repurpose devices once they are no longer serving their original purpose. This means looking for methods within the organization to meet current needs with currently owned equipment. It doesn't mean extending the life of a device by donating it to a school in Africa so that you don't need to deal with it yourself. I once spent some time in Africa working volunteering at a school. Much of my time was spent going through donated equipment to find and fix those few donated computers that were worthwhile. Most of it was total junk and disposing of it properly is a lot harder in Africa where there are fewer resources. The only donated items of less value to those Tanzanian students were the snowmobiling manuals and the 50 copies of the history of a small church in upstate Minnesota that we had to cull from the library donations. If you wouldn't use it yourself, don't donate it.

Once a device is no longer needed, the physical device has to be disposed of. Disposal of a physical device should be done in an environmentally aware manner, but we know that frequently does not happen. Our world is littered with electronic devices that have reached the end of their life or are broken. Such devices are not compostable (steel, plastic, and other materials), and few agencies or companies take the devices apart to recover and recycle the materials.

Major companies have begun to consider the environmental impacts of their outdated physical devices. Such companies seek out recycling companies and specify in their contracts how physical devices are to be disposed of.

SUMMARY

Day-to-day management of IT systems and services must be performed on a regular basis. Dedicated operations and maintenance teams perform such day-to-day management to keep systems and services running as expected. These dedicated teams use defined processes and procedures as well as their skills to monitor systems and services.

In addition to day-to-day management, systems and services require regular updates for many reasons, including changes to functionality, needed technical changes (which might include hardware changes), or changes to increase security. Change boards consisting of representatives from different disciplines review needed changes using established processes and procedures.

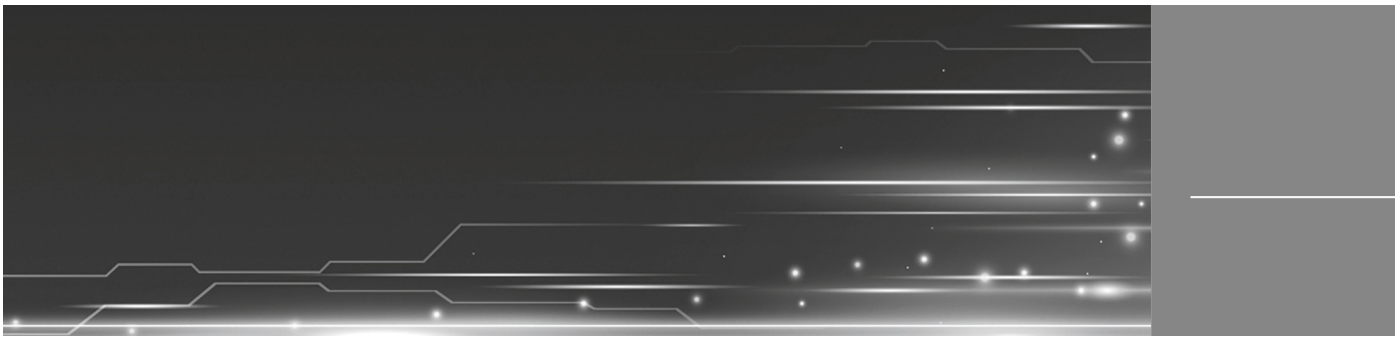
Management of incidents is clearly defined, which enables teams of the most appropriate experts to get together should any type of incident that impacts a business system or service happen. When situations occur, incident management as well as which relevant experts should be involved should already be defined. Administrative details such as communication and documentation processes are defined. The solution to the incident is applied using defined processes.

Updates to systems and services are needed on a regular basis. To increase the probability of success of such updates and decrease any unexpected impacts, a maintenance update process is documented. It includes details including how to apply the update, when to apply it, and how to plan for how long it will take.

KEY TERMS

backout plan
backups
change boards
change management
communication team
critical decision point
disaster recovery
drills
e-waste
functional fixes
incident assessment
incident management
incident manager
incident response plan
incident tracking
investigation team
maintenance
maintenance window
mitigation plan
operations
patches
post-incident review
practice
preparation
reaction team
regression error

root cause analysis
security drift
security monitoring
Service Level Agreement (SLA)
system logs
systems monitoring
user support



© d1sk/Shutterstock

CHAPTER 18

Retirement or End-of-Life

CHAPTER OBJECTIVES

- Outline the planning needs of end-of-life for a system.
- Describe strategies an organization can take to minimize the impact when a system it uses faces end-of-life.
- Examine potential security issues associated with end-of-life and retirement of systems.
- Outline the planning needs for retiring a system.
- Describe the decommissioning process for servers and devices.

Introduction

Eventually, every software system is going to reach the end of its useful life. This can happen because the system is outdated and will replaced with a better system, because the business shifts its business focus and there is no longer a need for the particular system, or because the capabilities of the system are met elsewhere with a system more aligned with the organization's enterprise architecture or service portfolio. Beyond business needs, all systems reach a point that technologically they need to be replaced with newer systems or services. For software providers, the cause may be lagging profitability, new market opportunities, or that it is time to replace the system with something new.

When an organization determines that it is time to stop using a software system, the system is prepared for **retirement**. Retirement is the removal of the system from active use by the organization. If the system is an in-house developed system, it may at the same time be considered for **end-of-life**. End-of-life for a system is when it is no longer being developed or supported by the developing organization. A commercially produced system may be end-of-life from the developers, but still not put into retirement by the using organization. Likewise, frequently when systems are put into retirement, they still remain available to other customers.

While putting a system into retirement or end-of-life seems as simple as taking ones hands off of the keyboard, dropping the mouse, and flipping the power switch, significantly more work needs to be done, more challenges to consider, and a potential for some significant risk. All of this requires planning to ensure that everything that needs to happen does, with the minimum of risk.

End-of-Life

The decision to stop building and supporting a software system is usually more of a business decision than a technical decision. If upgrades to technology always dictated changing to new systems using the new technology, there would not be millions and millions of lines of COBOL applications still running, still supported, and still, in some cases, being sold. Being old, or old-fashioned, is not the equivalent of being unprofitable or undesired. If that were the case, there would not be such a strong demand for handcrafted furniture or other goods made by hand by the Amish and other artisans. In fact, I am convinced that there will soon be a market for "hand-crafted COBOL programs written by *greybeards-in-sandals.*" On the other hand, we also find cases where the perfect technical system is no longer commercially viable, or at least not desirable, for the business, and it is time for end-of-life.

Planning End-of-Life

Whatever the reason, once the business has determined it is time for end-of-life for a system, it is time to begin planning. The plan needs to address when the system will reach end-of-life, when and how users or customers of the product will be informed, how the system will be supported as it approaches end-of-life, contingencies for migrating customers off the system, and the archiving of the system. **Figure 18.1** gives the end-of-life timeline.

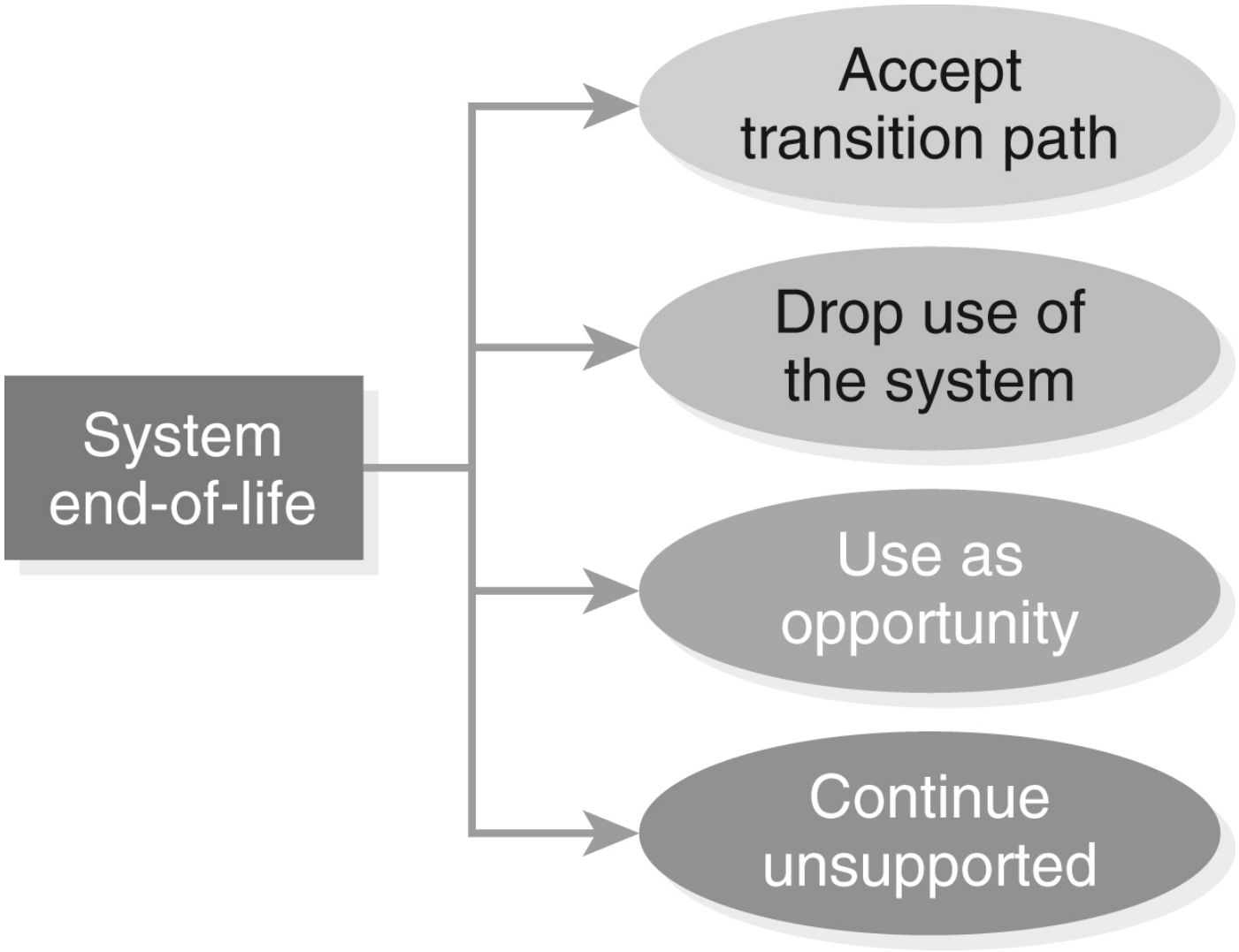


FIGURE 18.1 The End-of-Life Timeline

Description

Determining the specific date for the end-of-life of the system is also a business decision but may require some input from the development and support teams. If the intention is for the customers to migrate to a new system, then there will need to be sufficient time from the announcement until the final date to allow customers to make that migration. The time that customers require may be significantly different from the time that it takes to simply make the technical transition. Customers will need to complete their planning, testing, and validation processes, to have resources available, and to fit the transition into their operational schedule. Hurried transitions with the end-of-life of a product, even if the intention is to provide a substantially better new product, can push customers to choose products from other vendors.

Providing an extended **transition period**, the amount of time from the announcement until the end-of-life date, can increase the expenses to the

organization as they are both supporting the old product and the new one for an extended period. In some cases, this can divert resources away from improving the new product. In other cases, if the extended support includes producing upgrades to the current product, it can make providing equivalent features on the new product a moving target. For this reason, organizations that are planning to end-of-life a system will often freeze feature improvements prior to the point where they stop providing other support, such as security updates and bug fixes.

In determining the transition period then, one of the key considerations is the plan for what to do with existing customers. Sometimes it doesn't really matter. The system is meeting end-of-life because the organization is going out of business, because there are no more existing customers for the system, or because switching to an alternative is quick and simple. However, often the goal is not to simply get rid of the customers, but to move them to another product. To do this well, the organization is going to need to create a **transition path** for the customers. A transition path is a plan for moving from the current system to a replacement.

A well-thought-through transition path is going to not only include plans and tools for converting data from the old system to the new one, but also may include financial incentives for the customers to make the transition. At times customers may have more than one viable transition path, perhaps with a choice of alternate successor systems. In these cases, it is best to be able to walk the customer through the alternatives, providing guidance on what is best in this specific case, both from a technical and a business perspective. Left on their own, customers may make a less than optimal choice, which could negatively affect their perceptions of the new system and the company.

Direct customers are not the only ones who can be affected by an end-of-life. Many software systems develop significant eco-systems of integrators, third-party support, interconnected systems, distributors, and other stakeholders in the systems. Planning for transitions is going to need to account for these stakeholders, both in terms of the technical aspects as well as business relationships. Some of these parties may be involved in the planning and development of the successor systems, such as ensuring that the APIs function properly or are extended for interconnected systems.

The next concern of end-of-life planning is the support plan for the transition period. This is a critical period for support because when word of the pending action gets out, there is likely to be a short-term significant increase in support requests. These requests will come both from customers who are preparing themselves to transition and from customers who are planning to stay with an

unsupported system but want to ensure all of their languishing and anticipated issues are taken care of.

The support plan may include changes in what types of support will be provided during phases of the transition plan. Considering that a transition plan may take years to complete, immediately dropping all support is unlikely to be a good choice. The transition may begin with a period where limited updates and fixes are still made to the system. Then a second period where only priority issues are addressed. That may be followed by a final stage where support is provided as answers to questions, trouble shooting, and other actions that do not result in changes to code. Finally, at the end-of-life date, all support is withdrawn. Well, almost. Sometimes a special customer may require or contract for additional support beyond the product end-of-life. At that point, the product becomes a custom product for that special customer, the terms of which are dictated by the specific agreement.

With the dates, the transition paths, and support plans known, it is time to craft communications to the customers, stakeholders, and to other interested parties. While some stakeholders will be aware that an end-of-life event is in the works, others may not. Many stakeholders are in a position where they are direct influencers of customers, or they provide direct support or distribution. It is important to craft the communications plan in a manner that is going to consider their needs. The last thing that a support group wants to hear is that their customers are asking about how the group will support a transition for a product when the group had no previous knowledge and no time to prepare a response. Their response is likely to be to transition the customer to a new “supportive” vendor.

Communication to customers needs to be carefully crafted and well thought through. Sometimes the best path to communicate the end-of-life event to the customers is through distribution or support channels. In other cases, a direct message is a better approach. Whatever the means, the message needs to include the key information, such as dates, a reason for the event, and information about the transition plan. This provides customers with what they need to complete their own planning.

In addition to stakeholders and customers, the communication plan needs to consider any appropriate media communications. Reaching out through the media is a method to reach customers who avoid normal channels of contact. In addition, the media can help shape the perceptions of the end-of-life event and the organization. This can be useful in convincing customers to follow the transition plan.

Archiving

Once a software system has reached its end-of-life, the system needs to be archived. **Archiving** is the process whereby a system is put into storage in a manner designed to preserve key aspects, to prevent loss, and to protect the system from loss, theft, or other issues. It may seem curious to need to preserve a system that is no longer supported, and it would seem far easier to just erase the bits and be done with it. However, we need to remember that end-of-life does not equate to the retirement of all instances of a system and gone is not forgotten. For instance, a popular trend is the revival of "vintage" computer games. Other reasons to put an old system from the archives would be to support legacy applications that ran the system unsupported but now are in a position where they are willing to pay large sums for support, or legal issues may require accessing the system. Business changes may mean that a system that went end-of-life last year is again viable as a product. A system may even be archived in preparation for sending it to a museum for historical purposes.

As with any phase of the life cycle, end-of-life archiving requires careful consideration. Key issues in archiving include what to archive, how to archive it, and where to archive. In addressing what to archive, the simple answer is "everything." But everything is not appropriate. Naturally, the code base needs to be considered for archive as well as perhaps a final executable. But many times the code base alone is not sufficient for preserving the system.

Systems require tools to build. Some of those tools we can assume will still be available in 20 years, such as a basic C compiler. But other tools, such as those built-in house, are unlikely to still be available. Even some tools that will be around may change to a point where they are incompatible with the code base. Some tools may need to be stored with the system. Even when the tools are not stored with the system, the settings and configurations used on the tools should be.

Systems are also built using libraries. These systems, like the tools, may change over time, so it may be wise to store the current version with the system. Of the libraries are not being preserved, then information needs to be sufficient to be able to reconstruct needed dependencies. Operating systems are similar to libraries. Systems are dependent on particular versions and configurations of operating systems. The archive needs to include copies or configurations. Even if the revived system is not going to run on the same operating system it used to, having the older version and configuration available will ease the task of bringing the system up on the new operating system.

Another challenge of archiving is determining what to do about the hardware components of the system. Having a warehouse full of ancient computers is not an ideal solution. Most hardware components are commodity items, and it is probably safe to assume that there will be sufficient preserved items, equivalently functioning items, or emulators to be able to reconstruct a physical or virtual hardware environment. However, at times key pieces of hardware are rare or unique, such as special function cards, and may be worth preserving.

Naturally, in addition to all of the components, tools, libraries, and operating systems, it is important to preserve the system documentation, including design documents, specifications, and user manuals.

The format for the preserved materials is important. Media formats and file formats change. We no longer use 5.25-inch floppy discs. While a search may be able to locate a drive that can read them, that is only the start because when a drive is found, we will not be able to be plug it into our computer anyway. The 5.25 drive is going to be easier to find than an old Zip drive. Choosing the media to use to preserve the system is more important than simply choice of what media provides sufficient storage space.

The question of where to store the archive is another one of those seemingly simple questions where the answer turns out to be more complex and involve more considerations than it would first appear. The easy answer is, “we’ll just leave it in our GitHub account.” But GitHub and other code repositories are not designed for archival purposes, and the account structure may not be long-term stable. Business issues are also in play. Where does the organization want to system to reside? If the system was developed in a branch, is it to be archived in the branch or centrally. What if the branch is sold? Depending on the terms of the sale, the archived system may not transfer with the branch. In either case, if the archive isn’t where it remains in the possession of the owning party, the system may need to be moved. Likewise, what happens if the location of the archive is later closed?

Strategies When Faced with End-of-Life

Sometimes organizations find themselves in the position where the systems they use are going to be discontinued. The first thing that the organization needs to do it to complete an **impact assessment**. The impact assessment is a detailed study of the impact of the event on the organization. The assessment will look at the role of the system in the organization. Is it a primary system? Is it a secondary system—useful, but not essential? Is it integrated into another organizational or third-party system? The assessment needs to consider how tightly the impacted system is connected to other systems the organization uses.

The impact assessment will look at the ongoing costs of operating the system. That includes both direct costs for licensing, servers, and support, and indirect costs for training, processes, and other activities. The assessment will evaluate how the system fits into the organizations processes. Sometimes it is also useful to look at the history of the system in the organization.

With an impact assessment complete, the organization next will need to develop alternative courses of action. These are different approaches the organization can take to managing its response to the end-of-life event. Each alternative needs to be developed out to the point where the team can assess the potential costs of the alternatives, the potential effects in the organization, both negative and positive, and the time and effort it will take to implement the alternative. Based on these studies, there will be a decision of one or more of the alternatives to be pursued. (See **Figure 18.2**.)

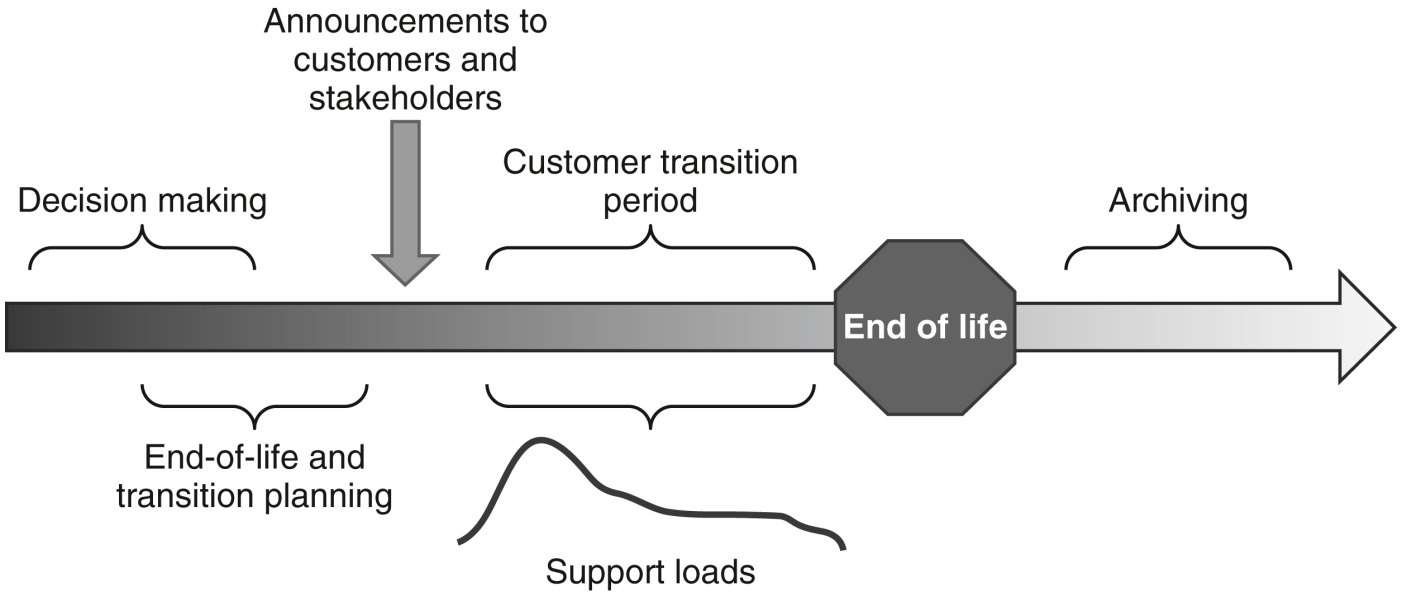


FIGURE 18.2 Alternatives When Facing System End-of-Life

Description

The first alternative is usually to accept the transition path provided by the system vendor. Reviewing this is going to require addressing concerns about technical capabilities, functionality, and compatibility with the organization's business processes. Additionally, vendors don't change products because they want to make less revenue, so the customer organization needs to take a close look at the financial impacts of the transition.

Another potential alternative is to simply drop the use of the product. It may be that the function the product performed was either nonessential to the organization and not worth the costs of a replacement for the system. It could also be that the organization already has another system that either performs or is capable of performing the functions of the soon to be discontinued system. The assessment will need to review any costs associated with transitioning users.

An end-of-life event can also be used as an opportunity. A commonly considered option is the possibility of using alternative systems supplied by other vendors. These may provide comparable or enhanced capabilities. Especially in the case of enhanced capabilities, because the organization will be going through a transition anyway, it may be a favorable time to switch systems. Naturally, this is not going to be as simple as a transition where the vendor has already defined the transition path, so the assessment will need to dig deeply into the technical feasibility as well as all associated costs and risks.

Sometimes, the best plan is not to transition, but to simply continue using the system without support from the vendor. Certainly, risk factors and potential costs are associated with this approach. For example, support needs to be transitioned in-house. However, factors can mitigate the risk and may make this the best option. It may be too expensive to replace a system that is tightly integrated into others, or the systems it is integrated with may themselves be reaching end-of-life or retirement, and there is no point in updating them all individually.

At times, there may be little or no risk in keeping an unsupported system active. For example, a computer was running an early version of MS-DOS and operated a tire-balancing system in an auto shop. After 30 years out-of-support, it was still running, still balancing tires. It wasn't connected to the internet. A new computer and operating system would have required buying a

new balancing system at great expense to the shop while adding no needed functionality.

In-house support can be easier if the code is available either by agreement with the vendor or if it has been open-sourced. It may be hard to justify replacing a system that is a part of an antiquated process no longer being used much in an organization. If an organization anticipates there is a possibility a product from a vendor may become unsupported, either voluntarily or because the vendor is financially unsecure and may disappear, a good option may be to have the system code put in escrow. This is done by providing a copy of the code to a third party, the escrow agent. The code in escrow is kept updated. If the terms of the escrow are ever met, the vendor goes out of business or drops the product, the code is then released to the customer.

Retirement

Eventually, an organization is going to come to a point where it determines that a system is no longer needed or viable or is going to be replace be a new system. Now comes the point to retire the system. Retirement is the process of taking a system permanently out of service. The retirement process is not as simple as turning off the switch and shutting down the system. (See **Figure 18.3**.)

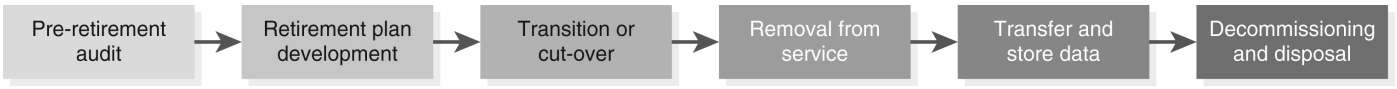


FIGURE 18.3 Progressing through Retirement

Description

The retirement process involves functional personnel and all relevant technical teams to be sure to retire any software system safely and securely.

Retiring a system is not as simple as shutting it down, deleting the data and files, and walking away. Sometimes business reasons may decide we need to reinstate the system at a later point. Perhaps this is due to a failure of the replace system, or a there is a new processing demand that can only be met by the retired system. In addition, there may be legal or financial reasons to reach into an old system at some point in the future. Even if the system is never to be used again, retiring it in an orderly fashion can ease the transition and prevent future problems from arising.

The approach taken to manage and handle the data from the system is a particular concern. Data should be retired in a manner that allows future retrieval and needs to be carefully transitions from a retiring to a replacement system.

With custom-built systems and highly customized systems, the retirement event is also an end-of-life event, and the planning and execution of the two need to be integrated together.

Communication

Just as with all system or service development and support, communication throughout the retirement phase is important to decreasing retirement impacts and increasing smooth execution. Communication with business users as well

as the technical systems and services communities is the first task when retiring a system or service. All companies have numerous internal communication channels. The planned retirement of a system needs to be communicated through formal channels, such as email distribution lists and internal newsletters, rather than simply being passed along by word of mouth.

Communication is repeated multiple times over a long period leading up to the actual retirement. This ensures that the message is heard and understood. Widespread and repeated communications help provide a greater likelihood of locating an unknown user or systems that are tied to the system being retired. This situation has come up more than once when retiring systems and services.

In addition to the broad communications, there needs to be specific communications to key stakeholders. While these should have been involved in the retirement decision, situations arise where a key stakeholder wasn't paying attention to the decision-making process or perhaps was left out of it for some reason. The project manager for the retirement project will obtain a confirmation from the key stakeholders that they are aware of the impending retirement. This is their opportunity to raise issues and objections either to the project manager or up through their business channels.

Once we get confirmation from the business users that the application is no longer required and any system or service that is connected to the system to be retired has confirmed they are ready for the retirement, we can proceed to performing the next set of retirement tasks.

Communication will continue throughout the retirement process. As the system is closed to users, the team will provide instructions on new methods for performing processes and the new systems to use. The team will also provide a contact point for anyone with questions or concerns during the process. Many teams find it is helpful to publish a frequently asked questions (FAQ) section during the process. An important part of the communication during this period is reminding the people of the value provided by retiring the system. This encourages people to focus on long-term benefits rather than short-term inconveniences.

The final communication will come once the system is retired. In this communication, the team should take the time to thank everyone and celebrate the project's success.

Developing a Retirement Plan

As with any project, a retiring a system requires a project manager and a retirement plan. The plan needs to carefully and thoughtfully address all the

actions needed to successfully retire a system. The **retirement plan** will be developed as the result of a pre-retirement audit and consider all the findings of that audit. In addition, it will address the timeline, transition plans, handling of data, archiving, decommissioning of hardware, and other topics as necessary.

Pre-Retirement Audit

The **pre-retirement audit** is an audit of the system that is conducted prior to the retirement of the system. The purpose of the audit is to verify the state of the system and to identify any issues that need to be addressed during the retirement process. Representatives of the technical and functional business teams will work together to perform the audit. Each team has perspectives and knowledge that contribute to the success of the audit. The audit needs to address the system components and functions, the users, files and data, and interconnections.

In auditing the system, the team will identify all components, such as libraries, objects, and keys, and devices, such as computers, routers, networks, and drives, that are used by the system. These need to be split into those that are unique to the retiring system and those that are shared by other systems. Items that are unique to the system may be relatively simple to decommission or remove. Items that are shared with other systems can be more of a challenge.

If a shared component, such as a library, is a part of the system rather than a separate system or a part of a separate system, then there will need to be planning on how to either replace the component (find a new equivalent library), operate without the component, or separate the component from the retiring system. Separating the component may relatively simple, such as establishing it in an independent code base or acquiring a license for the library, or it could be complex requiring a rewrite of parts of the code.

The pre-retirement audit team will also conduct a functional audit. The functional audit verifies all of the functional uses of the system within an organization. Ideally, this is a verification of the functionality planning that the organization has already completed as a part of its portfolio planning process. The audit team will check that the active functionality is either accounted for in a transition plan to other systems and processes or that the functions are no longer needed in the organization. If a key function is found that if lost will be disruptive, there will need to be a plan for how to continue or replace the function. Sometimes these kinds of interdependencies stop or delay the retirement of a system or lead to a partial retirement.

The user audit requires researching information on who has ever used the system and who is currently using the system. While the need for current user information is obvious, information about past users is useful in identifying mystery processes or resolving other issues discovered during retirement. Knowing who to talk to is important. Some systems also have users who only access the system infrequently but for important reasons, such as annual audits, quarterly reviews, or other special cases. Capturing only “current” users may miss these.

Capturing user information takes time and is done through several methods. First the list of known users is usually documented either in some online file somewhere or in a system or service information database. Second, we can get the information from the system directory by looking at who has accessed the system within a specific length of time (make it a long length of time). Often nonsystem (manually created) records are not up to date, which is why multiple sources are necessary. *Hopefully* that should flush out all the possible people and business processes that may be impacted.

The pre-retirement audit will include a file and configuration audit. The goal of this audit is to identify all of the files and other configuration artifacts associated with the system. This would include register keys, entries into firewall rules, routing table or DNS entries, and any other place where there are references to the system. The capture of these allows the plan to include provisions to remove, alter, or update these to clear all lingering references to the system. Sometimes this audit identifies odd and unexpected entries or files that require further examination. These may be part of another system, they may be evidence of improprieties, or they may turn out to be harmless artifacts.

The data audit portion of the audit will identify all data that are stored in the system, all data that are fed into the system, and all data that are delivered from the system. This will include stale as well as active data. Stale data are data that were acquired by the system at some point during its lifetime but are no longer actively used. Stale data may include specific fields in a database table or files maintained in the system. Sometimes stale data can be challenging to identify because a field may have been used for one purpose in the past, but the field has been redesignated to serve a different purpose later.

All data in the system to be retired will need to be evaluated for what to do with it. The evaluation needs to consider the data retention policies of the organization, legal requirements for data handling, and the potential future value of the data. The data audit also provides an opportunity to verify that all data that will be needed in the future have a home in another place. Sometimes data

that aren't going to be moved to new systems may need to be kept accessible on-line. At other times, data may be kept off-line in whole or in summary.

The pre-retirement audit includes an interconnections audit. Systems can feed systems. The audit includes reviewing and evaluating what other systems connect to the retiring system. If data are still being sent to a system downstream, the team will review what data are sent, to what other systems, and how often. By using an internal company list of applications and their details, we can often track down what systems or services connect to the retiring system. This is not always the case because over time and through careless processes organizational records are not always accurate, so the team will need to study logs of the application to be retired to gather information on system-to-system connections.

Connection information is needed because any system connecting to or sending information to the system being retired needs to stop or be redirected, otherwise connection error messages are likely to be generated in the sending system or service. Such disconnects and failures often impact system performance when a connected system tries to handle or process the disconnect. In the worst-case scenario, if we do not work with any receiving system so that the team can update their system to modify it to get what is needed elsewhere or change their system so that the information is no longer needed, their dependent system may fail!

While the audit can take time, it is essential, but even with that, because systems are complex and information gets lost over time, there are still opportunities to miss vital connections. As an example, when shutting down a large data center, we had a situation where a rather old Unix server was audited, and the applications were known (we thought). No one had accessed any of the applications in years; in fact, almost all the known users were no longer at the company. Rather than go through the process of deleting the applications, the hard drives were pulled and thrown into a pile for disposal. The disposal process was good with all hard drives being shredded (metal pieces left) and even the servers themselves being sent to be recycled, not to be used again. Five days after the hard drives were pulled, a key company application started alarming. One single small application on the hard drive now in a disposal bin sent a specific data point to the key company application once a quarter, and no one knew, and no one caught it. You can imagine the vision of a person being upside down in a bin looking for a specific hard drive while hoping once the hard drive was reinserted into an obsolete Unix box that the system would start back up. It did and we recovered, thank goodness.

Once the pre-retirement audit is completed, the team will analyze the results and ensure that all details uncovered in the audit are accounted for in the retirement plan. The longer a system has been in use, the more likely it is that the audit will uncover something unexpected. Sometimes that is something that may have a major impact for the organization or one of its customers. For example, we once had a system that provided informational performance data for a customer network on a webpage. There was no known connection to any external system. The expectation was that the data was just there as a courtesy and to assure the customer they were receiving quality network services. Because the business side felt there was no longer a need to provide the data via the webpage, it was discontinued. That was when we discovered that a major customer had built a web scraper to collect its data and feed it through an algorithm to provide a real-time estimate of its sales volume from all its locations. The customer was in a panic because that in turn fed several decision-making processes. Fortunately, we were able to quickly restore the service.

Timeline

The **retirement timeline** represents the schedule for all of the events and activities that need to occur to complete the retirement. The timeline will include key events, such as the last day users are on the system, the last day data are captured from the system, and the date when systems are shutdown. Often the timeline is influenced by outside factors such as end-of-life hardware, needing to return hardware, pending security vulnerabilities, or need to sunset the system or service to decrease corporate costs. The timeline will also be influenced by the availability of maintenance windows, resource availability, and other scheduling constraints.

If the system is being retired as a part of a project to introduce a replacement system, then the retirement timeline is going the need to be carefully crafted to match the deployment timeline of the new system. If this is the case, the change over from one system to another is going to affect the timeline. The transition may be gradual or abrupt. In a gradual transition, both the retiring system and the new system are kept functioning with processes, data, and users migrated over time from one system to the other. In these cases, it may take quite some time from when the retirement process begins until the retirement can be complete. A gradual transition may also require special processes and tools to maintain consistency between the systems during the transition.

In the case of a hard cutover, more initial work may need to be done prior to the cutover to ensure that everything is prepared. Pulling data off of the working system to get it ready to be converted to a new format and other similar work needs to be planned and executed carefully as the additional load, or changes needed to do the prep work, have the potential to create an instability in the system. Once the hard cutover is completed, the system is ready for the shutdown process; however, it may be a good idea to wait for a brief period to ensure the cutover was successful and the backout plan doesn't need to be executed.

Sometimes instead of a gradual transition or a complete hard cutoff with the old system shut down, the organization may decide to complete a hard cutoff, but leave the old system available for a period to allow limited, generally read-only, access by chosen people. This can preclude the need to immediately transition historical data. In these cases, the team will make adjustments to the retiring system to put in place the desired limits.

Timelines often end up being more complicated than initially expected as the team learns more and more information about the system. Complicated timelines can be difficult to implement. If possible, the team should consider ways to simplify the timeline.

The completed timeline needs to be communicated to all involved parties. When possible, a redacted or highlighted timeline for specific groups, such as users and executives, can provide clearer communication. The team will also need to communicate any subsequent changes to the timeline.

Removal from Service

A key milestone in retiring a system is **removal from service**. This is the point where the system is made unavailable to users and stops providing services. At this point the system administrators disable access to the system for all remaining users who will not be participating in further retirement actions. Usually, this is all but the administrators themselves, although it may also include management, users collecting data and statistics, and auditors.

With the users removed and the system out of service, the team will begin removing all pointers, links, and dependencies in other systems. This is completed before the system is brought off-line as failure to do so may cause disruptions, error conditions, or service failures in the other systems. The systems also need to be removed from access lists and firewall rulesets. This process includes closing any ports on firewalls, blocking any internal or external access to the system, removing access from single sign on systems, and

ensuring all DNS servers get updated appropriately. The full list of what needs to be disconnected comes from the pre-retirement audit. Sometimes the ordering of addressing the items is important. For example, removing DNS entries prior to removing URLs or other links causes errors.

Transferring and Storing Data

Once the system has be removed from service and isolated, the team will handle any data that are still on the system. Retired data may be saved in summary or complete protected off-line, or they may be completely deleted because they are no longer needed. Not all data need to be or should be swept up in a block and archived. The team needs to separate data with different retention policy requirements to allow the specific handling required for each type. The organization will also pay particular attention to cleaning and segregating sensitive data to receive proper handling. Stored sensitive data are encrypted for security.

If data are going to need to continue to be accessed or moved into the new system, but haven't been yet, the team will migrate the data to the new location. As a part of that process, there may be a process to scrub the data to remove any data errors and ensure that only correct, usable data get migrated. Scrubbing data is surprisingly hard. Part of that is the nature of most bad data. Most bad data appears like good data but aren't. For example, a date of 30 Feb 2022 matches the format of a date and, if it were any month but February, would be a valid date. Combing carefully through thousands of potential subtle errors in millions of lines of data takes a lot of time. There are tools that can help, but even there it can take significant manual intervention to set up the process. This means that if the data aren't needed, it is best to avoid a costly time-consuming data scrubbing process. But, at the same time, if the data will be reused, it is better to clean the data now rather than send corrupted data off to the new system.

While archived data are rarely going to be accessed, if there is a need for access, it is often for a critical reason. Organizations need to give thought to how they are going to access and read the data they archive. Often data from applications are in a form where the meaning and interpretation of data are not readily apparent. The organization needs to plan to have a means for reading the data. That can be done by keeping a copy of the system or a portion of the system available to be able to read the data when needed. Another method is to have sufficient format and schema information available to either manually read the data or to build a quick tool to read the data. In the worst case, the

organization may find itself needing to reverse engineer the data interpretations and structure. This can be time consuming and expensive.

If there is a possibility that the system may ever be brought out of retirement, then the team needs to copy and archive the system configuration. The system itself can usually be restored from media or a copy obtained from the manufacturer. For in-house systems, the end-of-life process will be important.

Decommissioning the Hardware

With the users, connections, and data accounted for, the system is shut down as it normally would in an orderly fashion. The before running around and hitting off switches, the system administrators are going to remove the application software off of any shared hardware. Ideally, the drives where the system software and data were stored are wiped and then written over multiple times to ensure there is no readable residual image.

The administrators will check all physical hardware, virtual machines, and specialized hardware components that were catalogued in the pre-retirement audit, ensuring that all media in the components are identified. Hard drives and other physical media are going to be removed and destroyed, but some devices, such as routers, have internal memory-based storage that needs to be cleared. After the system administrators have checked all of the devices, they are shut down and powered off.

The administrators pull the devices from the racks and prepare them for disposal. They send the hard drives to be crushed or shredded according to protocol. There are almost daily horror stories about devices sold with their hard drives intact, which contained proprietary, classified, or otherwise sensitive data.

Based on their value and utility, the administrators may repurpose the remainder of the devices, sell them as excess, or dispose of them in an environmentally sound manner. If the hardware was rented or leased, it returns to the vendor. Before deciding whether to repurpose the devices, the owners will consider several factors. First is whether the device will still function for its intended or a needed purpose. For older devices, while the device may still be working and functional, the power consumption of the device itself may make reusing the device noneconomical. At one point, when replacing an external hard drive with one of significantly greater capacity, we calculated that the cost of the replacement drive was less than the cost of six months of electricity for the drive being replaced. In that case, there was no justifiable reason to retain

the old drive. Sometimes, however, while an older device may not be economical as a long-term solution, it may be a good spare to have on hand in case of a short-term emergency need. Older less capable systems may also serve well in a test or development capacity.

Disposing equipment in an environmentally friendly manner doesn't mean donating junk to impoverished schools in developing countries. In some cases, this is a good solution, if the equipment is still usable. The challenge is that junk in one country is junk in another, and proper disposal is more difficult in a country that does not have the infrastructure to support it. I spent two weeks in Tanzania going through a container of donated computers, sorting out what was usable and fixable. At the end of the time, there were around 30 viable computers (out of more than one hundred) and only a half dozen monitors worth keeping. Most of the donated monitors were old CRT monitors, and the school already had sufficient flat-screen monitors for all the working computers.

The biggest challenge was then how to dispose of the junk. The first instinct of the school officials was to burn and bury it, as was common for trash. We finally managed to find a place in a nearby city that would take the computers and use them for parts. We had to pay. Other donations to the school with little value included a snowmobile manual (in Africa) and 50 copies of the history of a church in Northern Minnesota. Those did get burned. Please be thoughtful and responsible about what you donate.

Licensing Adjustments

As systems are removed from service, the organization needs to check the licensing status of the commercial components. The project team will inform whomever the system or the licensed components of the system that the previously purchased licenses are no longer needed or may be amended to adjust for fewer users. It may also be that the vendor has licenses for limited usage (read only) and the license type may be switched. In that way, there is no risk of being charged for licenses that are no longer being used or needed. Any licensing agreements define the length of the license term and how to cancel a license.

Catching the changes in licensing is especially important for subscription-based licensing. If the manager of the subscription isn't notified of the changing needs, the organization could continue paying for licensing long after the system is retired and gone. Often these kinds of subscriptions have automatic renewal clauses where, if the license is not cancelled significantly (60 to 90

days) ahead of the renewal date, the customer is committed for another renewal period.

The hardware components of the system may also have licensing and support contracts associated with them that need to be terminated as well. There is no need for annual upgrade and maintenance for a router that is out of service. Server and desktop operating system upgrades and licenses may be accounted for individually or may be included in an organization-wide agreement. With individual licenses ongoing costs may need to be terminated. For organization-wide agreements, accounting for the retired devices may alter the costs of the agreement.

If a device is being sold or transferred to another organization, the license may need to be transferred along with the device. Some types of licenses transfer automatically with the device, but others require specific licensing by the obtaining party. Routers are notorious for being tied to an organizational agreement with a significant fee required of any purchaser to establish the router onto a new agreement. This kind of an issue can significantly reduce the resale value of some devices.

Finalizing the Retirement

As with all other projects, the final step in a retirement project is to review that everything was completed, document what was done, and file the results. A portion of the paperwork is providing notice to the stakeholders that the retirement is complete. Some organizations choose to have a retirement party for the system, especially if it was retired after a long period of service. This allows people who may have resisted the change to accept it and move forward.

SUMMARY

Eventually any system is going to reach a point where it is no longer commercially viable or desirable, no longer needed by an organization, or is being replaced. At this point, the system may go into end-of-life mode where the system is no longer actively developed or supported, or it may go into retirement where it is removed from active use. End-of-life and retirement may happen together or independently. Either milestone in the life cycle is carefully planned and executed as a project.

Without sufficient planning for end-of-life and retirement events, an organization can face some significant consequences. This could include the loss or disclosure of data and down time in the organization.

Many organizations find themselves in a situation where systems they depend upon are approaching end-of-life. These organizations need to carefully weigh how they will react to the event and prepare ahead of the event so that it doesn't prove disruptive. Maintaining business continuity is the first consideration, so an alternative must be in place for the same functions to be performed, or a plan for how to maintain the services on an unsupported system must be made.

When a system is being retired, once business considerations are met, the organization will complete technical tasks to retire the system in an orderly manner. These technical tasks are done to make sure any data or internally developed system software is securely backed up off-line just in case it should be needed in the future. Finally, the servers and other support systems are decommissioned. Physical hardware may be recycled using secure procedures. Virtual systems will be deleted. Licenses are also either reused, or the vendors selling the licenses are notified of the intent to cancel the license.

KEY TERMS

archiving
end-of-life
impact assessment
pre-retirement audit
removal from service
retirement
retirement plan
retirement timeline
transition path
transition period

© will/iStock/Getty Images.

Glossary

accreditation The process of assuring and certifying that a system meets legal and regulatory requirements.

ad hoc testing Testing done without pre-written test cases.

adherence The degree to which a law, regulation, standard, or framework must be or is followed.

advanced persistent threat (APT) Threats with significant capabilities, resources, and specific goals that are continue over a long period.

after-action review (AAR) A review with all involved members following the completion of a project focused on what went well and what did not go well.

Agile A movement toward a more flexible, developer-centered approach to building systems as described in the Agile Manifesto. Often the term is mis-applied to denote particular practices.

Agile process management Application of the Agile Manifesto principles to process management.

alpha testing A small, select team of users perform initial testing after development is completed to assess product features and functions.

Americans with Disabilities Act (ADA) Prohibits discrimination against people with disabilities in several areas, including employment, transportation, public accommodations, communications, and access to state and local government' programs and services.

anomaly A deviation from the expectations as set by requirements or an unexpected system behavior.

application architecture Defines the functionality that is provided by each of the component applications.

application programming interface (API) A defined set, or library, of program functions used to interact with a system.

AppSec Security measures applied to an application.

architectural concerns Factors that need to be considered for inclusion in the design of a system architecture.

architectural principles Provide the targets the architect is trying to reach in any design.

architectural reviews A formal or informal review of an architecture to identify issues, obtain approvals, or for other purposes.

architectural scope The boundaries of what will be included in a system.

architectural views Used to focus attention on key elements of the architecture, often as seen from different perspectives.

architecture The high-level design of the structure of an application or system.

archiving The process whereby a system is put into storage in a manner designed to preserve key aspects, to prevent loss, and to protect the system from loss, theft, or other issues.

artifact Any type of documentation or representation that provides information on project progress and the final product.

attack surface All the areas of a system where the system faces or is exposed to potential attacks.

attack vector A commonly used means for launching an attack or is a carrier of attacks.

attractive nuisance Features and characteristics of systems that encourage external attackers.

auditing Reactive process that identifies where companies may be failing to implement the laws, regulations, standards, or identified framework used to conduct the audit.

authoritative sources (golden source) Source of information that is designated as the correct source. Any changes to data should be made in the authoritative source and then propagated to secondary sources.

authority The ability to make decisions and take action in an area.

Authorization to Operate (ATO) Formal U.S. government approval that a system has met security requirements and may be put into production within the government.

awareness Ability to recognize potential security challenges, apply general security principles, and find help and guidance when encountering an issue.

backout plan Plan to revert to a previously successful version of the product in case of issues with a new change or update.

backups Copies of data and software that are maintained to be used in the case of loss or corruption of the originals.

bespoke software Custom software created for a specific customer.

best path coverage Testing following many divergent paths through the system as possible, covering as many lines of code as possible given the constraints of time and resources.

beta testing A larger user audience is given access to the system for pre-release testing that focuses on compatibility, user interface, and performance and scaling issues.

black box testing Testing completed where the tester has no access to the code, configuration or design.

boundary A transition from one group of values, inputs, functions, or trust areas to another.

boundary value testing Uses values that approach the boundaries from both directions.

buffer overflow A condition where an application tries to write beyond pre-allocated memory or buffer space.

bug Refers to any failure, defect, or issue found in the software product. May be found by anyone: users, developers, and testers.

built-to-change Code designed from the beginning to be easy to change and update.

business case Brings together the product idea with information on how the project will increase company value, the project's feasibility, the project's criticality, and other factors.

business or functional architecture Provides the view of how the system is put together and functions from the point of view of the users.

business strategy How the company intends to conduct its business, the goals of the organization, and how those will be achieved.

buy-in The degree to which the members of the organization accept security as a key and fundamental value of the organization.

capability maturity model (CMM) A process and behavior model to guide organizations to improving the delivery of projects, particularly software projects.

capability maturity model integration (CMMI) A process and behavior model to guide organizations to improving the delivery of integrated systems.

cascading dependency One piece of software depends on another which depends on another, thereby creating a cascading effect.

CE A mark that affirms conformity with European health, safety, and environmental protection standards. Required for goods sold in the European Economic Area. Letters are abbreviation for Comformité Européenne (French: European Conformity).

change board or change control board (CCB) Composed of appropriate personnel and given the authority within the organization to approve designated changes.

change management The systematic management of changes that will affect the scope, deliverables, or other factors. Includes structured methods and organization to manage proposed changes.

change process The processes and procedures used to manage changes in an organization.

change request A change that either the user community, maintenance, or development community wants. It is not a bug.

CIA (confidentiality, integrity, availability) traditionally recognized goals of cybersecurity.

chief information officer (CIO) Executive in an organization with oversight of all information systems and information management.

chief information security officer (CISO) Executive in an organization with oversight of all information security matters.

classes of potential threats Criteria-based categorization of threats so they can be addressed in a systematic manner.

client-server An architectural pattern in which the workload is divided between a shared central system (server) and an individual system that makes requests of the server.

cloud-hosted application An application deployed in a cloud-computing environment.

cloud native An application specifically designed to take advantage of unique cloud computing capabilities.

cloud services Shared resources such as software, networks, storage, and other IT resources provided on an as-needed basis over a network. May be internal or external to a company.

code or function libraries Collections of functions which are shared to allow the reuse of code and keep the world from having to reinvent every needed function.

common vulnerabilities and exposures (CVE) Databases that log device, application, and network vulnerabilities, often shared publicly.

compatibility testing Tests used to determine whether, and how well, a system interacts with other software systems.

competitive analysis A business process to understand competitive products and services and how they compare.

compliance The act of ensuring that systems, processes, and deliverables meet specified requirements, especially with respect to externally applied requirements.

compliance measurement Checking through defined and documented monitoring and auditing processes and procedures to confirm the degree to which systems, services, and personnel meet the requirements.

Computer Fraud and Abuse Act (CFAA) U.S. law to address cyber-based crimes.

configuration drift Uncontrolled changes to the pre-determined configuration that happen over time. Can be due to changes to settings, human error, or system changes made through patching.

configuration management Documentation and management of the configurations required for all parts or components of a system.

configuration management system Automated tool for the management of the configurations of systems and components.

Control Objectives for Information and related Technology (COBIT) A framework to aid organizations in the development, implementation, monitoring, and governance of information technology systems.

corner cases Test cases of those situations where the nature of the inputs or values change significantly, such as zero crossings, value boundaries, century changes, etc.

corrective action Measures taken to address issues identified in a root cause analysis, audit, or assessment.

Commercial-Off-the-Shelf (COTS) Software that has been developed by an organization and is sold to multiple customers without significant customization.

COTS platform Software written to perform common functions out-of-the-box but allows for extended functionality to be programmed or added to meet an organization's specific needs.

coupling A measure of how tightly the modules of a system are integrated.

critical path, projects The ordered combination of tasks in a project which represent the longest time from beginning to end to complete. Represents the least time to get a project completed.

critical path, testing A path through a section of code containing critical calculations or known to be subject to attempts at security breaches.

customer relationship management system (CRMS or CRM) Systems designed to store, track, and process information about customers.

data architecture Architectural view of the software system from the point of view of the data that considers where the data are created, used, and stored.

data custodians Those who store or manipulate data on behalf of the owners.

data owner Has the ultimate responsibility for making decisions about the data, such as how long it is kept, what level of protection the data requires, and who will have access to the data.

dedicated test team Test team with no duties other than testing, allows independence from pressures to finish testing and pass the code along.

defect Error in software that needs correction (See *bug*).

defect targets Specify the types, numbers, and severity of defects found in testing that are acceptable to include in a release.

defining and planning Stage of the secure system life cycle development model where the system moves from a general concept to more fully developed plans.

denial of service (DOS) A cyber attack that focuses on disruption and preventing the use of systems.

deployment The phase of the secure software systems development life cycle where the system is moved into an operational mode.

deployment communication plan The part of the deployment plan that covers internal and external communications.

deployment plan Considers all aspects of deployment, including the roles of people involved, training needs for people in those roles, the timelines for deployment, the requirements that must be met during the deployment, and measures of success of the deployment.

deployment window A period, selected to meet operational needs, for a system to be put into active use.

design and decision The stage of the secure system life cycle development model where requirements drive high-level designs and refined plans.

design and plan Stage of a project that moves from a general idea to concrete plans and designs.

design patterns Effective designs that reoccur time after time that can be recognized and reused.

development The phase of the secure software systems development lifecycle where the system is created.

development methodologies A standardized process or management method for developing software systems. A particular methodology may cover a portion of, or the whole development lifecycle.

disaster recovery Section of the incident response plan covering responses to worst-case scenarios.

distributed denial of service (DDOS) A denial of service attack that uses multiple systems coordinated for the attack.

DevOps A development methodology that combines software development and software operations teams throughout the cycle.

DevSecOps A development methodology that combines software development, security, and software operations teams throughout the cycle.

dynamic analysis Analysis of systems while in operation to identify, isolate, and investigate potential and observed issues.

dynamic process analysis Analysis of a process completed while observing a process in action or a simulation of a process.

dynamic testing Testing completed on active systems.

ecosystem management Management of activities created by the presence of a product in a market.

Electronic Communications Privacy Act (ECPA) U.S. law concerning interception of communications and related matters.

end-of-life A system that is no longer being developed or supported by the developing organization.

entity–relationship (E-R) diagrams A modeling tool that focuses on sets of data (entities) and the relationship between data elements. Often used to design databases.

entry criteria Specific indicators that the test case would be appropriate to run.

enterprise resource planning (ERP) A system that manages a broad variety of organizational functions including inventory, production, sales, and others.

event-driven architecture Processing in a system is based on events. Events can be keyboard strokes, mouse clicks, messages from other components, interactions with other systems, or timer based.

enterprise architecture The concept of designing an overarching architecture for the information systems in an enterprise.

environmental threats Threats that exist based on the environment in which the system operates, regardless of where that environment is located. They may be natural environmental threats, with common types such as earthquakes, volcanos, hurricanes, drought, and tornados, or special types due to where the system is deployed, such as deep space radiation, saltwater intrusion, or intense heat.

ethical integrity Being consistent with the ethics professed and actions taken.

e-waste Electronic waste, a plague on the environment if not properly disposed of.

exploratory testing Nonstandard software testing where the tester begins with a script but deviates to investigate minor or curious deviations in system behaviors.

external threat Deliberate attacks against systems from external adversaries.

Fair and Accurate Credit Transactions Act (FACTA) U.S. law protecting users of consumer credit.

favored vulnerabilities Vulnerabilities that a given threat is known to have previously used, has shown signs of using, or that are similar to another vulnerability that the threat has exploited.

final security review Review of a system just prior to release to ensure all security issues have been identified and addressed.

final systems test A special case of integration testing. While it is primarily a regression test, it also has a focus on the end-to-end correctness of results. See *system testing*.

first-in first-out (FIFO) A queue where the first arrival is served first.

first-line supervisors The lowest level of management and oversight in an organization.

flow chart Low-level model of the function of a system. Often uses standard symbols.

frameworks Provide broad guidance, which may be implemented in a variety of ways, for building, managing, or maintaining a system or a set of processes.

full path coverage Testing where each possible path through a system, and thereby each line of code, is run at least once.

function libraries Allow the reuse of code and keep the world from having to reinvent every needed function.

functional architecture See *business or functional architecture.*

functional fixes Configuration changes as well as software patches that affect the front-end functionality important to the business.

functional requirements System behaviors needed to accomplish the purpose of the system.

functional security requirements Used to describe the security that must be supported by the system as a part of its mission.

functional testing Testing to evaluate that new software or new software changes meet the business or functional requirements.

fuzzing The use of random paths or small randomized changes to inputs in testing to provide more complete coverage.

Gantt chart Graphical detailed breakdown of project tasks, including dependencies, time to complete, responsibility for tasks, and other project details.

General Data Protection Regulation (GDPR) European Union framework that sets guidelines for the collection and processing of personal information.

glue logic software Code that that is written to stitch together a group of commercial products, or commercial products and custom components into a coherent system.

good coding practices The patterns used by developers in developing their code including a broad range of habits from ensuring that the code is well documented to ensuring that the formatting of the code is consistent.

governing body The organization or entity that is responsible for determining the content and requirements of a compliance source.

GNU General Public License (GPU) Copyright concept that allows free use of the licensed software provided changes in the software are made freely available to the public.

graphical user interface (GUI) An interface for users that is visually oriented.

gray box testing Testing conducted on a system when the tester has some, but limited access to the internals of the system.

Health Information Technology for Economic and Clinical Health (HITECH) U.S. law creating incentives for the use and protection of healthcare information technology, including electronic health records (EHR).

Health Insurance Portability and Accountability Act (HIPAA) U.S. law that established national standards to protect sensitive patient health information.

hidden dependencies Created when a system or package depends on another in an unobvious manner or one that has not been disclosed.

highest priority first A queue type where servicing order is based on transaction priority.

industrial control system (ICS) Systems designed to control industrial processes.

information and communications technology (ICT) Term that covers computer and networking systems.

impact assessment (end-of-life) A detailed study of the impact on the organization of the end-of-life of a system.

impedance The degree to which changes, ideas, and values are slowed in their implementation and conversion to action.

implementation/execution Phase in a project where results are created.

in-scope Items that will be included in the current project.

incident When a production system's behavior is different than what is expected in a manner that potentially disrupts operations or needs to be investigated and addressed.

incident assessment Evaluation of an incident to identify the actual or potential impact to the business.

incident management The business process of responding to incidents that disrupt routine operations.

incident manager or incident commander Person designated to lead the response to an incident.

incident response plan Provides the organization a road map of how to deal with unanticipated incidents. See also *mitigation plan.*

incident tracking Details regarding an incident get recorded in an application.

information management Managing the people, processes, and technology that provide control over the structure, processing, and use of information in an organization.

Information Technology Infrastructure Library (ITIL) An information technology service management framework.

infrastructure as a service (IaaS) Clients contract for storage, servers (processors), and some network capabilities that they can configure and maintain.

inherited vulnerabilities Vulnerabilities that are present in a library appear in the software developed from that library.

initiation The first phase in a project or SSDLC where the determination is made to begin and the first description is completed.

insider attacks Deliberate attempts by employees, contractor, or other insiders to damage or take advantage of an organizations systems.

integration testing Testing completed done anytime two or more units are combined.

Intelligence Sharing and Analysis Centers (ISACs) Industry or sector (financial services, utilities, health care, maritime, etc.) focused organizations supported by governments, industry coalitions, and other parties seeking to support the best interests of the industry. They research, collect, and provide intelligence specific to the industry or region they serve.

intended security practices Security practice goals for organizations using a system.

interfaces Places where systems interact with other systems or users.

International Organization for Standardization (ISO) An independent nongovernmental international organization that promulgates and governs standards.

internationalization The ability to adapt to operating in different countries, including the use of different languages, currencies, and business models.

investigation team People assigned to investigate an incident.

ISO/IEC 27001 ISO directive that establishes best practices for information technology as well as information security.

issue A problem or a concern with a system that may or may not have been anticipated.

issue management The process of managing issues related to the product or project in a disciplined manner.

iteration One round of a set of phases to develop a small part of software (as in Agile methodology).

joint application design (JAD) A development methodology where users and the technical team work together to discuss requirements.

Keep It Simple Silly (KISS) Eliminating unnecessary complexity from systems.

key engineering challenges Challenges that are going to be the most difficult to meet and those that will have the most influence on other areas of the architecture.

key functions What the system needs for it to perform its function at the basic level.

last-in first-out (LIFO) A queue where the last arrival is served first. Also known as a stack.

laws and regulations Provide directions from a government on specific topics.

layering Breaking the problem domain into different layers and working through how those are implemented or supported.

Lean A process improvement methodology focused on decreasing waste and unnecessary costs.

least knowledge No module or user of a system should have access to anything not needed to perform the required function.

least privilege People or applications run at lowest level of permissions and access needed.

leadership Providing the vision, motivation, resources, and energy to move the organization in a direction.

lessons learned Document and determine improvements for future projects based on information from the AAR.

life of data Analysis of data from the time it is created until it is retired or destroyed.

load testing Determines whether the system or service can effectively manage expected user workloads or to characterize the expected performance at different load levels.

locality Grouping together of functions, data, and interfaces physically or temporally for optimal system operation.

logical architecture Groups the implementation requirements together into where the functions will be performed rather than how they are used.

long-term thinking Considers not only the immediate consequences of actions and the systems, but also future consequences.

loose coupling Components that can operate independently and perform their specified tasks without depending on another component's behavior.

manageability The ability to control the system and how it operates.

maintenance window Specific time frame (periodic or episodic) established to perform maintenance (apply patches, update configurations, take backups, and other maintenance) on operational systems.

map-reduce architecture A problem with massive data or massive processing or both is handled by mapping the task into multiple parallel jobs. Each job will perform a portion of the actual work obtaining partial results. These distributed partial results are reduced into a new set of job parts that are then either mapped into the next stage or combined to produce the final result.

market opportunities Where there are gaps in the offerings in the markets, when there are imbalances in Porter's forces, or an opportunity for creating a new market or for introducing an effective substitute for a product.

market share The percentage of the market represented by a product.

market strategy How to position our product in the market compared to all other competitive products.

marketing All activities involved in the management, advertisement, production, sale, and delivery of a product or service.

matrixed organization Organization with two (or more) distinct reporting structures.

message-based architecture Components of the system are all connected to a shared messaging bus.

metrics Measurable values that are used to monitor and evaluate a project.

micro-services An architectural pattern where services are broken into their most elemental components, which are individually implemented.

milestone chart(bird-on-a-wire chart) Graphical timeline of dates of high-level deliverables and goals.

mitigation Eliminating or reducing the potential harm of an incident or risk.

mitigation plan A plan and processes applied in the event of an anticipated incident to ensure the incident results in the least harm and disruption possible. See also *incident response plan*.

model-centered architecture An architectural pattern where the model, as instantiated in the database, is shared by all components. The components function independently of each other, relying solely on the database for communication.

modeling A graphical representation of a system, component, or the function of a system or component, used to gain deeper understanding.

modeling techniques Systematic methods that can be applied to create models.

modeling tools Applications designed to assist in the creation and manipulation of models.

model-view controller (MVC) An architectural pattern based on separating data from how they are viewed and manipulated.

monitor and control Phase in a project which ensures the project is working to plan. Simultaneous with implementation phase.

monitoring, compliance Proactive periodic or continuous activities collecting information and checking compliance.

monitoring, systems Proactive periodic or continuous review of systems, including logs, to find issues in real time.

multi-tier See *n-tier or multi-tier*.

National Institution of Standards and Technology (NIST) U.S. government agency that promotes standards, science, and technology including those for cybersecurity.

near-shore development Uses development teams within same geographical area, but not the same country. Often presumed to be outsourced but may be in-house teams.

nonfunctional requirements Requirements that qualify or constrain the way the system is developed, system performance, or how it completes a functional requirement.

nonfunctional testing Focuses on testing technical and other nonfunctional items (quality, performance, and others).

nonrepudiation The ability, through the use of electronic signatures, to ensure that an action has been taken or an electronic document produced which cannot be denied.

n-scope map Diagrams the flow of data through a system to identify sources, uses, and sinks of data, including potential risks to and abuses of the data.

n-tier or multi-tier Architectural pattern where multiple layers of software are used to provide the full user functionality (front end, middleware, business layer, database).

object model A modeling technique which groups attributes and actions together into objects while identifying relationships between objects.

off-shore development Uses development teams that are geographically dispersed across different countries. Often presumed to be outsourced but may be in-house teams.

on-shore development Uses development teams that are geographically dispersed within same country. Often presumed to be outsourced but may be in-house teams.

open-source software projects Software developed and maintained by volunteers or corporate-sponsored developers with a license that makes it freely available to others.

Open-source Intelligence (OSINT) Makes use of openly available resources to preform threat analysis.

open-standards Standards and frameworks to which anyone with an interest in the subject area is free to contribute.

operating rhythm The consistent patterns a team follows during a development process.

operations The process of managing, monitoring, and supporting systems to ensure they are functioning properly and securely, that users are able to perform their needed tasks, and that potential issues are identified as rapidly as possible.

operations and maintenance The phase of the secure software systems development life cycle where the system is in active use.

outsourcing Hiring external teams do perform work.

organizational culture A set of norms and expectations commonly held by the majority of people within an organization.

out-of-scope Items that will not be included in the current project.

patch Changes to system software provided by either the vendor of the product, or by group in the organization responsible for maintaining or updating the product.

Payment Card Industry Data Security Standard (PCI DSS) Industry standards, policies, and procedures for the protection of users of credit cards.

peer review Static form of testing where a developer, or a group of developers, reviews another developer's code for issues, style, or educational purposes.

peer-to-peer architecture An architectural pattern each node in a network of computers acts as both a client and a server. Clients reach out to peers for specific services.

penetration testing A test methodology to uncover product vulnerabilities through a deliberate, approved, attack on a software system or service.

performance Architectural concern regarding the speed and other factors of the systems operation.

performance goals Metrics that establish the levels of performance that a system is expected to meet.

performance testing Confirms that the software system or service meets the performance objectives.

Personal Information Protection and Electronic Documents Act (PIPEDA) Canadian Law regarding the collection, use, and disclosure of personal information.

personally identifying information (PII) Information that can be used to specifically identify an individual.

PERT (program evaluation review technique) charts Chart used to show the relationship between tasks in a project.

phishing Sending emails with a goal to persuade people to reveal personal information or to perform other compromising actions.

physical or technical architecture Architectural view that describes the physical or technical components and how they are interconnected.

platform as a service (PaaS) A cloud service where clients contract for and utilize a provided services (databases, compute engines, services, etc.) to construct and deploy software applications.

portfolio Set of software applications that together provide capabilities to businesses or customers to meet information technology needs.

portfolio management A business process to properly manage the portfolio of systems.

point-of-sale (POS) Systems that track purchases and payments.

positioning How a product is presented to the market.

post-incident review Review of causes, situation, and actions taken during an incident.

potential threats Threats which that have been evaluated and deemed to be relevant to a system or organization.

preemptive queue A queue type where the servicing of a transaction may be suspended to service a transaction with a higher priority.

pre-retirement audit An audit of a system and its environment that is conducted prior to the retirement of the system.

privilege levels Limits access by specifying the permissions and access to software using pre-defined levels.

procedure Detailed instructions on how to implement a process.

process Anything an organization or system does on a planned repetitive basis.

process diagram/process flow diagrams Model of the processes and the flow of work through the processes implemented in a system.

process flow The ordered steps a transaction takes in moving through a process.

process improvement Steps to raise quality, shorten processing times, or to make another quantitative or qualitative improvement in the process.

process management The management of people, resources, and flow in order to optimize the effectiveness, efficiency, and quality of processes.

process quality Measure of the performance of a process. Often focused on defects produced during the process or output quantities of a process.

product A software system to be built as a part of a development effort and offered for sale or use.

product definition Determining the functional features and capabilities of the product, user interface and experience (UX/UI) for the product and its target markets as well as consumer product benefits.

product management A business process to offer targeted customers software systems with the right capabilities at the right time to maximize profitability or otherwise support the business objectives.

product planning Detailed plan for how a product will be produced and managed.

product roadmap A representation that traces the planned evolution of the product over time.

product strategy Plan for creating a product to match a market strategy.

product vision A story about how the product is going to fit into the market.

professionalism Ensuring that our interactions with others are clear, concise, and that we have done our best to be effective.

program An organized a set of projects that focus on meeting specific business goals.

program management A strategic activity focused more on the broader management of resources, planning, and coordination across a program rather than the direct accomplishment of specific tasks.

program management office (PMO) A corporate function to manage programs and projects.

programming manuals Codify the coding practices of the organization. These can be incorporated into the SOP or developed as a stand-alone document.

project artifacts Documents, records, code, and other tangible items produced in completing a project.

project charter Document created at the inception of a project to set out goals, intent, and boundaries of the project.

project close Final phase of a project, includes delivery and after delivery tasks.

project management A disciplined approach to completing projects.

project plan Provides clear guidance to all stakeholders of the project on the details of how the project will be completed, managed, and reported.

protected health information (PHI) Patient information protected under HIPAA.

prototyping development model A methodology where at an early stage a quick implementation of an example of a product, or key portions of a product, are produced to evaluate what works and what doesn't.

quality assurance (QA) Processes and tools used to ensure quality is baked into products throughout the secure software systems development life cycle.

quality control (QC) Focuses on the product and includes processes and procedures that directly measure and ensure that quality goals are being met.

quality expectations The level of quality expected from customers, clients, and users of the product or service.

quality measures or metrics Specific measurable requirements selected to evaluate the quality of the product.

queueing theory A mathematical discipline that quantitatively describes the actions of transactions in queues.

queues Where the tasks or transactions in a process wait for an action to be performed or a decision to be made.

rapid application development (RAD) A development methodology that focuses on quickly providing a minimally functional product with quick successive incremental increases in functionality.

reaction team Team responsible for mitigation, remediation, and restoration in the event of an incident.

regression error Error introduced into a previously tested system due to new changes to the system.

regression testing Testing repeated throughout the development of a system to ensure that subsequent development and integration does not negatively impact existing functionality.

rehearsal A practice system deployment used to uncover potential issues in the deployment plan.

release When a software version is made available for deployment to users.

release authority The person or people who are authorized to make a determination that a system is ready for release.

release criteria The criteria that must be met prior to release, including what must accompany the release (documentation, media, and other materials), the required completeness of functionality, and the level of quality.

release plan Document detailing how the system will be released, including the composition of the release, the release timeline, the standards, approvals and certifications needed for release, release reviews, and the release authority.

release review A formal step to ensure that the criteria for release have been met, and in fact the system is ready for release.

release requirements See *release criteria.*

reliability The ability of the system to continue to produce accurate and timely results.

removal from service The point where the system is made unavailable to users and stops providing services.

request for proposal (RFP) A document that is generated, based on requirements, when a vendor or multiple vendors are going to asked to submit proposals for a pre- or custom-built system.

requirements Specifications of what must be included in a software system. Will include functional and nonfunctional requirements.

requirements engineering Categorizing, clarifying, deconflicting, and analyzing all requirements.

requirements solicitation Efforts to collect requirements for a project from stakeholders.

requirements tracking Requirements is recorded and traced throughout the development process.

residual risk The risk remaining after risk mitigation, avoidance, and transfer efforts.

responsibility Tasked with and held accountable if things do not go right.

responsibility boundaries Where responsibility for providing, monitoring, and managing security transitions from one organization or organizational component to another.

Representational State Transfer (REST) An architectural pattern that focuses on uniform stateless interfaces between distributed systems.

retirement The phase of the secure software systems development life cycle where a system is removed from service, or the activity of removing a system from service.

retirement plan A document that addresses the timeline, transition plans, handling of data, archiving, decommissioning of hardware, and other topics as necessary.

retirement timeline The schedule for all events and activities that need to occur during retirement.

risk The probability of a vulnerability being exploited multiplied by the cost of occurrence.

risk acceptance Acknowledging a risk exists but not otherwise addressing it, often because the level of a risk is low or the cost of addressing a risk is too high.

risk assessments The process of identifying and evaluating risks.

risk appetite The level of risk an organization is willing (and able) to tolerate.

risk avoidance Taking steps to ensure that the risk is not encountered.

risk management A systematic approach to dealing with risks. Risks are identified, evaluated, and addressed by being avoided, mitigated, transferred, or accepted.

risk mitigation Putting controls in place to reduce or eliminate the exploitation of a vulnerability or to reduce the extent of harm caused by the incident.

risk transfer The use of insurance or other means to transfer a risk to become another's responsibility.

risk management framework (RMF) A framework of security management practices developed by the National Institutes of Science and Technology.

risk-based decision-making Methodology for making decisions that considers risks.

root cause analysis Looking behind issues identified in testing, monitoring, and auditing to identify not the specific issue itself, but rather the fundamental reason that the issue occurred.

software as a platform (SaaP) A cloud service that provides access to software platforms that organizations customize to meet their needs.

software as a service (SaaS) A cloud service where clients purchase subscriptions to use specific software that is hosted and administered on a cloud infrastructure.

sanity test Informal minimal set of testing to make sure system appears to be behaving as expected.

Sarbanes-Oxley Act (SOX) U.S. law to improve auditing and public disclosure.

scalability Measure of the ability of a system to manage large loads (data, users, throughput)

scalability testing Testing used to determine the capacity limits of a software system.

scope Definition of what will and will not be included in a system or project.

scope boundaries Define for a project what functionality will be added, what portions of the system will be touched or left alone, and the level of effort that will be available to develop the system.

scope creep Additional requirements for a product added after initial requirements are agreed upon.

scrum An Agile-inspired software development methodology focused on incremental and collaborative delivery development.

secure functional requirement Functional requirements that have been assessed and supplemented to ensure the function is implemented securely.

Secure Software Development Framework (SSDF) A framework developed by the National Institute for Science and Technology to focus on initiatives and processes to improve the development of secure software.

secure software systems development life cycle (SSSDLC) A description of the processes, activities, and tools needed to securely develop software systems in a structured manner.

secure tools Tools used in development reviewed carefully to ensure that using the tool doesn't insert new vulnerabilities into the development process.

security An architectural concern.

security culture The attitude and approach to security within an organization.

security enhanced five (six) forces model A model that shows the influences on the positioning of a product.

security monitoring Continuous or periodic inspection of a system that specifically focuses on events and activities that indicate the active presence of a malicious attack, a system compromise, or another security event.

security requirements Protections to be put in place in the system, as well as those to be put in place during the development of the system.

security testing Primarily driven by the threat assessment as well as testing to general security issues.

separation of duties Related high-risk tasks are divided among multiple people to ensure no single person has sole authority in a process.

sequence diagram A model used to track the order of synchronized interactions among elements of a system.

service discipline The algorithm that determines how transactions are selected for servicing in a queue.

Service Level Agreement (SLA) An agreement that prescribes how rapidly the support team responds to different types of requests, the escalation processes that are followed in the event an issue takes longer than expected to resolve, the user is not satisfied with the resolution, or the issue cannot be solved within the support team.

service-oriented architecture (SOA) An architectural pattern that divides the functions that must be performed into services. The client requests needed services from multiple components and then stitches together the results into what will be presented to the user.

severity An evaluation the criticality of an issue.

Simple Object Access Protocol (SOAP) A web-based implementation of a service-oriented architecture.

single responsibility Every single component should have a single task and should do it well.

Six Sigma A methodology that uses metrics and analysis of software issues/errors to help improve product quality.

subject matter expert (SME) Someone who is an expert in a particular field. Often used for consultation.

smoke test informal test to make sure nothing significantly bad happens when a system is started. See *sanity test.*

Software/Systems Requirements Specification (SRS) Written format used to document the requirements in a structured manner and obtain buy-in and approvals for the requirements.

space-based architecture An architectural pattern designed to handle the challenge of scalability and variability of demand. It does this by eliminating the central database but sharing a distributed data store.

spear phishing Phishing directed at specific people.

spiral model A development methodology where a system is built in phases in which each includes a complete pass through all processes prior to starting the next cycle.

special cases In testing, unusual or significantly different cases.

spike testing Subjecting a product to rapid and high increases in users and workloads to test performance and reliability.

stakeholder Anyone that may have an influence, be influenced, or a direct interest in the outcome of a project.

stakeholder expectations Perceptions of the stakeholder that need to be managed.

stakeholder influence The ability of a stakeholder to affect the project.

stakeholder interest The degree of attention a stakeholder applies to a project.

standards Guidelines, recommendations, or requirements that provide specific details about the subject of the standard.

standing operating procedures (SOPs) Written guides for how an organization completes its processes.

state diagram Modeling tool for understanding and documenting process-based computing and other computing processes where distinct sets of values or states are connected through pathways that are traversed based on specific conditions.

Static Application Security Testing (SAST) Analyzing software code for security vulnerabilities.

static code review Review software code when not running.

static process analysis Analysis of a process based on a static diagram or process description.

static testing Testing completed when software is not running or not actively in use.

structure chart A modeling technique where a system or problem is progressively broken into smaller problems.

stress testing A test that induces specific challenging conditions on the system to determine its potential breaking points.

STRIDE Acronym used to refer to a set of cyberattack vectors: spoofing, tampering, repudiation, information disclosure, denial of service, elevation of privilege.

structure chart A model that breaks a project or problem into smaller pieces displaying them from largest part to smallest part

Structured Threat Information eXpression (STIX) A common standard used for sharing threat information.

style guide See *programming manual*.

subject matter expert testers Testers who specialize in specific types of testing or the testing of specific kinds of systems or components, often those which require extensive depth of knowledge.

Supervisory Control and Data Acquisition (SCADA) Computer control of machines (such as manufacturing machines). See information and communications technology (ICT).

supply chain attack An attack directed toward an organization through its supply chain, for example, malware is introduced through software suppliers, which then infect customers as they consume software updates

swim-lanes Within a process diagram, the tasks are organized into columns that define who will perform the task.

system boundaries Specify what functions will be performed within the system.

system integration testing Testing completed when the whole or a significant portion of the whole system is combined.

system logs Used to record significant events in a system for later review.

system testing A special case of integration testing. While it is primarily a regression test, it also has a focus on the end-to-end correctness of results. See also *final systems test*.

systems monitoring The periodic or continuous checking of system parameters to ensure that everything is functioning as it should be and to spot issues as they arise.

tasks The basic unit of work used in a project.

terminal-based A user interface where the user interacts with the software system through a terminal, adding inputs and receiving outputs. The terminal does no processing.

test and evaluation The phase of the secure software systems development life cycle where the software and systems are tested. Usually overlaps with the development phase.

test case Specifies the inputs and actions for a particular iteration of a test as well as the expected results.

test driven development Test focused development. Test cases that will be run during coding are developed before coding begins.

test environment Includes any systems, networks, software, or other resources needed to complete the test.

test goals The types or tests to be applied and the amount of test coverage to be met.

test harness A piece of software that is built to run the software being tested either by a function call, an API, or other appropriate mechanism.

test manager Person assigned the responsibility for overseeing testing.

test schedule Provides timeline of the testing to be completed and when it is to be completed.

test scripts Specify the test as code and contain the instructions to drive the system using the values specified by the test case.

test strategy Plan for testing of all components of a software system and the types of tests to be performed to maximize coverage.

test tracking Performed to ensure that each stage of testing has been accomplished using the processes, systems, configurations, and settings proscribed for the tests.

testers Personnel assigned the responsibility for performing testing.

testing The process of verifying and validating that a software system or application meets the intended requirements, performs as expected, and contains no unknown defects, flaws, or errors, and no unmitigated vulnerabilities.

thick client Client-server architecture where most of the processing is done on a local machine.

thin client Client-server architecture where minimal processing is done on the local machine.

threat An actor or action that is going to take advantage of a weakness or vulnerability to breach, incapacitate, or otherwise harm system security.

threat analysis Art of determining who or what are the most probable threats to a system.

threat assessment Process followed to identify, evaluate, and rate threats.

threat intelligence The process and the product of gathering information to aid in identifying, analyzing, and tracking threats.

threat mapping Visual annotation applied to a model to indicate different types of actual or potential threats.

threat model A description of the potential expected threats against a system.

total quality management (TQM) A process improvement methodology focused on involving all members of the organization in generating and applying improvements.

traceability The ability to track a requirement from solicitation to implementation and testing. Being able to track a project requirement through every stage of development to final release. Used to confirm all requirements have been coded, tested, and accepted (by the client).

transaction Task, item, or group of things that traverse the flow in a process as a single entity.

transaction behavior Description of how a transaction acts in a process.

transition path Plans, tools, and incentives to move from an existing system to a new one.

transition period The time from the announcement of retirement or end-of-life for a system until it is removed from service or no longer supported.

trust boundary Boundary between modules or systems with differing levels or domains of trust.

trust relationships The expectations of levels of trust between different components of a system, or different systems.

trust zone A grouping of components and systems that share a common trust relationship.

Trusted Automated eXchange of Intelligence Information (TAXII) Defines an API for sharing intelligence using multiple models.

Underwriters' laboratory (UL) An independent safety testing organization that grants tested products the right to include a UL designation.

unit testing White box, dynamic testing. Often done by a person doing the software coding for a specific requirement.

unordered queues A queue where no specific criteria are enforced in the selection of the next transaction to process.

unsecure defaults Defaults in systems pre-set to the lowest levels of security.

usability test Testing concerned with how well the interface supports the tasks that the users are trying to perform.

use model Sometimes called a user story or use case, is a model developed formally or informally of how a particular product will be used.

use case Requirements of how a product will be used expressed using specific wording. Provided by the people who will use the product.

use case model Used to model interactions between users and a system. The model is used to document the actors in a system and the cases in which they will use the system.

user acceptance testing Performed by the organization that will be using the software system in production. The purpose of user acceptance testing is to ensure the users that the software is configured and performs in a manner which that meets their needs.

user interface (UI) The method with which users interact with a system.

user interface testing Addresses whether the appearance and functionality of the system's user interface meets the defined requirements.

user integration testing The equivalent of user acceptance testing by an organization that will be integrating the software system into its own system.

user support Direct or indirect assistance to the user in resolving issues the user may experience with the system.

user threats Threats resulting from users of the system, either through deliberate action or negligence.

validation The process of ensuring that the system performs the functions it is intended to perform in an acceptable manner.

verification The process of ensuring that the system meets the provided requirements.

version control The process of tracking a component through the development and deployment process.

version control system Automated system used to add management, consistency, and automation to the version control process.

version labels An identifier assigned to each different instance of a component used to track information about the instance and component.

version management Includes version labels, version control, and other processes to manage the required versions of every part that a system needs to work. Includes versions of hardware, operating system, network, and software packages.

v-model A view of testing processes that maps testing to phases of development.

vulnerability A weakness in a system that has the potential to fail or otherwise be exploited, thereby exposing the system to undesired performance, leakage of data, or other negative security problems.

vulnerability assessment Review of a system to identify and evaluate potential vulnerabilities conducted multiple times throughout the SSDLC.

vulnerability mapping Identification of potential vulnerabilities of a system based on inspection of models of the system. Usually graphically displayed.

vulnerability matrix A listing of all the potential and actual vulnerabilities identified for a system along with key information about the vulnerability.

waterfall A development methodology where development falls from one stage to the other, without loops or feedback, toward an inexorable outcome.

work breakdown structure (WBS) The designation of units of work and their relationships in a Gantt chart.

whaling Type of phishing focusing on high value victims.

white box testing Tester has visibility of code and evaluates paths the code takes in executing functionality.

work breakdown The lists of all tasks to be completed in a project.

workflow system Provides an automated means of routing transactions in processes from one step to the next.

© will/iStock/Getty Images.

Index

NOTE: Page numbers followed by *f, t and b* indicate material in figures, tables and box respectively.

A

AAR. *See* after-action review
acceptance testing, 9
accreditation
  definition of, 290
  government system, 290–291
  internal, 290
adherence, 138
administrator training, 292–293
advanced persistent threat (APT), 216
after-action review (AAR), 64–65
Agile movement, 93–94, 93*b*
Agile process management, 87
Agile programming, 13–14
alpha testing, 278
Amazon Web Services (AWS), 123
American Disabilities Act (ADA), 290
analysis phase, of project management, 41–47
APIs. *See* application programming interfaces
Apollo Guidance Computer (AGC), 190
application architecture, 183–184, 184*f*
application programming interfaces (APIs), 2, 95, 198
APT. *See* advanced persistent threat
architectural concerns, 188, 188*b*
  coupling, 191
  internationalization, 191–192
  locality, 190–191
  manageability, 189–190
  performance, 189
  reliability, 190
  scalability, 188

scope, 189
security, 189
architectural principles, 192, 192*b*
built to change, 194
don't overload functionality, 193
don't repeat functionality, 193
key engineering challenges, 195
least knowledge, 193–194
loose coupling, 194
single responsibility, 192–193
architectural reviews, 202
architectural views, 182
application architecture, 183–184, 184*f*
business or functional architecture, 182–183, 182*f*
data architecture, 184–186
logical architecture, 183, 183*f*
physical/technical architectures, 186
architecture
architectural concerns, 188–192
description of, 181–182
design patterns, 196–201
enterprise architecture, 186–187
formal and informal reviews, 201–202
principles, 192–195
refinement techniques, 201–202, 201*b*
security objectives, 195–196, 195*b*
views/models *See* (architectural views)
archiving process, 330–331
attack surfaces, 195–196, 206
attack vector, 212
attractive nuisance, 216
auditing
of compliance, 138, 151–152
quality assurance, 158
audit trail, 196
authentication and authorization systems, 125
authoritative sources, 185
authority, 102
Authorization to Operate (ATO), 290–291
awareness training, 110
refreshes over time, 111, 111*f*
Azure, 123

B

back-of-the-napkin design, 162*b*
backout plan, 294–295, 322
backups, 306–308
bespoke software, 226
best path coverage, 271
beta testing, 9, 278–279
BitCoin ATM, 174–175, 174*f*

black box testing, 272
boundary value testing, 269
brainstorming groups, 122
built to change, architectural principle, 194
business architecture, 182–183, 182*f*
business case, 24, 43–45, 43*b*
business strategy, 33–35
buyer power, Porter's model, 22
buy-in, organizational culture, 99

C

Canadian regulations and laws, 143–144
capability maturity modeling (CMM), 85–86
Capability Maturity Model Integration (CMMI) model, 148, 149*f*
cascading dependency, 230
CCB. *See* customer change board
CFAA. *See* Computer Fraud and Abuse Act
change board, 318–319
change management, 94–96, 95*b*, 318–320
change process, 96
chief information security officer (CISO), 101–102, 105
  reporting structures, 102*f*
China regulations and laws, 145
CISO. *See* chief information security officer
client-server model, 197–198
client trust boundary, 210–211
cloud environments
  cloud services, 232–234
  development practices, 235
  security risks, 234
  services engagement, 234–235
cloud-hosted application, 233
cloud-native applications, 233–234
cloud security, 234
cloud services, 232–234, 233*b*
CMM. *See* capability maturity modeling
CMMI model. *See* Capability Maturity Model Integration model
COBIT. *See* Control Objectives for Information and related Technology
code/function libraries, 227–228
coding practices
  fail-safe code, 240
  good coding practices, 236
  integration issues, 240–241
  programming manuals/style guides, 237–239, 237*f*, 238*b*, 239*f*
  standing operating procedures, 236
Combined Federal Regulations (CFR), 259
commercial-off-the-shelf (COTS) software, 150, 224–225, 287–288
communication, in retirement, 334–335
communication plan, project management, 59–61
communication team, 310
compatibility testing, 284–285

competitive analysis, 22–23
competitive rivalry, Porter's model, 21
compliance
  adherence, 138
  auditing of, 138, 151–152
  certification of, 137
  definition of, 135
  frameworks, 137
  governing body, 137–138
  laws and regulations, 136, 139–140
    Canada, 143–144
    China, 145
    data privacy, implications of, 140
    European Union, 144
    U.S., 140–143
  models/frameworks, 145–146
    CMMI, 148, 149*f*
    COBIT, 146–147, 146*f*
    ISO/IEC 27001, 147
    ITIL, 147
    SSDF, 148–150
  monitoring, 138, 150–151
  recognition, 139
  sources, 136, 136*b*
  standards, 137
compliance measurement, 138
Computer Fraud and Abuse Act (CFAA), 141–142
conception phase, of project management, 41–47
configuration drift, 247
configuration management, 245
  description of, 243–244
  operations and maintenance, 247–248
  in product development, 245–247, 246*f*
  version control, 244–245
configuration management systems, 245–246
Control Objectives for Information and related Technology (COBIT), 146–147, 146*f*
copyleft, 229
corner cases, 269
corrective actions, 159–160
COTS platform software, 225
COTS software. *See* commercial-off-the-shelf software
coupling, 191
critical decision point, 322
critical paths, 271
custom-developed software, 226
customer change board (CCB), 318–319
customer relationship management (CRM) system, 185
CVE® list, 217
cybersecurity
  leadership sets tone for, 101
  threats to, 22
  training, 111, 112

D

data architecture, 184–186
data custodians, 185
data owners, 185
dedicated test team, 261
defect risk, levels of, 256–257, 257*b*
defect targets, 257–258
define, measure, analyze, design, verify (DMADV), 86
define, measure, analyze, improve, control (DMAIC), 86
deployment, 8–9, 291
  communications, 293–294
  deployment plan, 291
  deployment window, 296–297
  failure, planning for, 294–295
  migration planning, 294
  rehearsal, 295–296
  roles of, 291–292
  training before, 292–293
deployment communication plan, 293–294
deployment plan, 291
deployment window, 296–297
design and plan phase, project management, 47–51
design patterns, of architecture, 196–201
development environment, 221
  cloud environments
    cloud services, 232–234
    development practices, 235
    security risks, 234
    services engagement, 234–235
  coding practices, 236
    fail-safe code, 240
    good coding practices, 236
    integration issues, 240–241
    programming manuals/style guides, 237–239, 237*f*, 238*b*, 239*f*
    standing operating procedures, 236
  language selection, 227
  libraries using in
    code/function libraries, 227–228
    commercial effect in, 228–229
    dependency issues, 230–231
    excessive library dependence, 229
    inherited vulnerabilities, 231
  operating systems and architecture, 231–232
  secure tools, 222–224
  security culture, 221–222
  software sources, 224–226
development methodologies, 91–92
dev-ops, 9
disaster recovery, 313
documentation, 2–3
drills, 313
dynamic process analysis, 81–82

dynamic testing, 272

E

ecosystem management, 31–32
ECPA. *See* Electronic Communications Privacy Act
EHR. *See* electronic health records
Electronic Communications Privacy Act (ECPA), 141
electronic health records (EHR), 142
email queues, 76*b*
end-of-life, 10, 328. *See also* retirement
  alternatives, 332–333, 332*f*
  archiving process, 330–331
  definition of, 327
  impact assessment, 332
  in-house support, 333
  planning, 328–330
  timeline, 328, 328*f*
end user license agreements (EULA), 141
enterprise architecture, 186–187
entity-relationship (E-R) diagrams, 169–171, 170*f*, 171*f*
entry criteria, 271
environmental issues, 323
  e-waste, 324–325
  power consumption, 323–324
environmental studies, 44
environmental threats, 215
E-R diagrams. *See* entity-relationship diagrams
error handling, 239
errors of commission, 250, 252
errors of omission, 250
ethical integrity, 107–108
EULA. *See* end user license agreements
European Union (EU) regulations and laws, 144
event-driven architecture, 199
e-waste, 324–325
excessive library dependence, 229
exploratory testing, 283–284
external threats, 215–216
external trust boundary, 210
Extreme programming, 13–14

F

FACTA. *See* Fair and Accurate Credit Transactions Act
fail-safe code, 240
Fair and Accurate Credit Transactions Act (FACTA), 142–143
Federal Information Processing Standard (FIPS), 290
Federal Risk and Authorization Management Program (FedRAMP), 290
file header comment section, 237, 237*f*
final security review, 289
final systems test, 277–278
first-in first-out (FIFO) queue, 77

first-line supervisors, 100
five forces model, 21–22, 21*f*
five Ps (process, program, project, product, and portfolio), 14–16
  combining with secure software systems life cycle, 16–17, 18*f*
flow charts, 166, 167*f*, 168
focus groups, 122
formal discussions, 42–44
frameworks, 137
full path coverage, 270, 270*f*
functional architecture, 182–183
functional fixes, 316
functional requirements, 115
functional security requirements, 116, 116*b*
functional testing goals, 258–259
fuzzing/fuzz testing, 271, 283

G

Gantt chart, 54–55, 54*f*
GDPR. *See* General Data Protection Regulation
GDPR European Directive, 144
General Data Protection Regulation (GDPR), 140
glue logic software, 226
Gnu General Public License (GPL), 229
good coding practices, 236
governing body, 137–138
government system accreditation, 290–291
graphical user interface (GUI), 306

H

Health Information Technology for Economic and Clinical Health (HITECH) Act, 142
Health Insurance Portability and Accountability Act (HIPAA), 142
hidden dependencies, 230, 230*f*
highest priority first queue, 77
HIPAA. *See* Health Insurance Portability and Accountability Act
HITECH Act. *See* Health Information Technology for Economic and Clinical Health Act

I

IBM, 138
IETF. *See* Internet Engineering Task Force
impact assessment, 332
impedance, of organization, 99
implementation/execution phase, 58–61
incident assessment, 309
incident management
  definition of, 308
  disaster recovery, 313
  drills, 313
  incident response plan, 309–312
  mitigation plan, 308–309
  post-incident review, 314

preparation and practice, 313
regression error, 316
root cause analysis, 312
tracking system, 312
incident manager/commander, 309–310
incident response plan, 309–312
incident tracking, 312
influence, of stakeholder, 120
informal discussions, 42
information management, 36
information protection, 195
Information Technology Infrastructure Library (ITIL), 147
infrastructure as a service (IaaS), 233*b*
inherited vulnerabilities, 231
initiation phase, of project management, 41–47
in-scope boundaries, 118
insider attacks, 215
integration testing, 275–276
Intelligence Sharing and Analysis Centers (ISACs), 216
intended security practices, 125–126
interest, of stakeholder, 119
internal accreditation, 290
internal trust boundaries, 210
internationalization, 191–192
International Organization for Standardization/International Electrotechnical Commission (ISO/IEC 27001), 147
Internet Engineering Task Force (IETF), 138
investigation team, 311–312
ISO 9001, 84–86
ISO/IEC 27001. *See* International Organization for Standardization/International Electrotechnical Commission
issue management, 96–97
ITIL. *See* Information Technology Infrastructure Library

J

joint application design (JAD) session, 123, 123*f*
jumpers/switches, 25*b*

K

key engineering challenges, 195
key functions, 117
KISS (Keep It Simple Silly), 202

L

lasers, 25*b*
last-in first-out (LIFO) queue, 77
laws and regulations, of compliance, 136, 139–140
Canada, 143–144
China, 145
data privacy, implications of, 140

European Union, 144
United States, 140–143
layering, 201
leadership, 101
Lean process improvement technique, 87
least knowledge principle, 193–194
leave off connections, 25*b*
licensing, 25*b*
life of data, 211–212
load testing, 280–281
locality, 190–191
logical architecture, 183, 183*f*
logical proximity, 191
long-term thinking, 108–109
loose coupling principle, 194

M

maintenance, 9–10
change management, 318–320
configuration and software updates, 315–316
description of, 314–315
functional fixes, 316
maintenance window, 320–323
patches, 316
security configuration, 317
software updates, 317
tasks, 299, 300
vulnerability management, 317–318
maintenance window, 320–323
manageability, 189–190
map-reduce architecture, 201
marketing communications, 20, 20*f*, 31
market opportunities, 23
market share, for product, 29
market strategy, 23
matrixed organization, 69–71, 70*f*
mere marketing, 25–26
message-based architecture, 198–199
metrics, 55–56, 56*f*, 159
micro-services, 198
Microsoft OneNote, 59–60
migration planning, 294
migration window, 296–297
milestone chart, 55, 55*f*
mitigation plan, 308–309
model-centered architecture, 200
modeling
definition of, 161
process flow diagrams, 162
selection of, 163–164
techniques *See* (modeling techniques)

tools, 164–165
vulnerability mapping, 163
modeling techniques, 166
E-R diagram, 169–171, 170*f*, 171*f*
flow chart, 166, 167*f*, 168
object model, 172–174, 173*f*
sequence diagram, 175–176, 175*f*, 176*f*
state diagrams, 176–178, 177*f*
structure charts, 168–169, 169*f*
use case modeling, 174–175, 174*f*
modeling tools, 164–165
model-view-controller (MVC) architecture, 199–200
monitor and control, project management, 61
finances, 63
operating rhythm, 62
quality and conformance, 62–63
recording and resolving issues, 63
risks, 63
schedule maintanance, 61
scope creep, 63
stakeholder expectations, 63–64
task tracking, 62
monitoring
of compliance, 138, 150–151
quality assurance, 158
multi-tier systems, 198

N

naming coventions, 238
National Marine Electronics Association (NMEA), 138
National Vulnerability Database (NVD), 217
near-shore development, 226
NMEA. *See* National Marine Electronics Association
nonfunctional requirements, 115
n-scope map, 211–212, 211*f*
n-tier systems, 198

O

object model, 172–174, 173*f*
observation method, 121–122
off-shore development, 226
on-shore development, 226
Open-Source Intelligence (OSINT), 216
open-source software projects, 225
open-standards, 138
operating rhythm, 48
operations, 9–10
backups, 306–308
definition of, 300
description of, 299–300
engage co-dependent teams, 302

frameworks, 302
monitoring, 303–304
pre-deployment, 301
security drift, 301, 301*b*
security monitoring, 304
service level agreement, 306
user support, 304–306
organizational culture, 99–101. *See also* responsibility; security culture; training
Original Equipment Manufacturer (OEM), 26
out-of-scope boundaries, 118
outsourcing, 226

P

patches, 316, 319, 319*b*
Payment Card Industry Data Security Standard (PCI DSS), 143
peer review, 274–275
peer-to-peer architecture, 200–201
penetration testing, 284
performance, architecture, 189
performance goals, of test plan, 260
performance testing, 280
periodic portfolio review process, 18*f*
personal computer (PC) standards, 138, 139
Personal Information Protection and Electronic Documents Act (PIPEDA), 143–144
personally identifiable information (PII), 142, 194
PERT charts. *See* program evaluation review technique charts
“phake-phishing” campaign, 112
PHI. *See* protected health information
physical architecture, 186
PII. *See* personally identifiable information
PIPEDA. *See* Personal Information Protection and Electronic Documents Act
platform as a service (PaaS), 233*b*
PMO. *See* program management office
Porter’s five forces model, 21–22, 21*f*
portfolio management, 14, 16–17, 19
business strategy, 33–35
definition of, 32
goals of, 32
information management, 36
post-incident review, 314
potential threats, 218
potential vulnerabilities, 206, 207
practice, incident management, 313
preemptive queue, 77
preparation, incident management, 313
Prepare the Organization (PO), 148
pre-retirement audit, 335–337
primary stakeholders, 119
privilege levels, 125
process diagram, 78–79, 80*f*
processes. *See also* process management

components of, 79*f*
definition of, 75
for developed system, 88–89
and security, 88
process flow, 78
process flow diagrams, 162
process improvement, 86–88
process management, 16, 17, 18*f*, 75
documentation, 77–78
dynamic process analysis, 81–82
identification, 75–76
process diagram, 78–79, 80*f*
process improvement, 86–88
process quality, 83–86
queueing theory, 82–83
security, 88
static process analysis, 80–81, 81*f*
understanding, 76–77
workflow system, 83
process quality, 83–86
Produce Well-Secured Software (PW), 149
product definition, 26
delivery strategy, 26–28
pricing model, 28
sales and distribution strategy, 28
sourcing, 26
product development, configuration management in, 245–247, 246*f*
product management, 14–17, 19
ecosystem management, 31–32
identifying opportunities, 23–26, 25*b*
marketing communications, 20, 20*f*, 31
Porter's five forces model, 21–22, 21*f*
positioning and differentiation, 26
product definition, 26–28
product planning, 29–31
security-enhanced five (six) forces model, 22–23, 23*f*
product planning, 29–31
product quadrants, 29, 29*f*
product release
accreditation
definition of, 290
government system, 290–291
internal, 290
COTS system, 287–288
preparation for, 288
release review, 288–290
product roadmap, 29–30
product strategy, 23–24
product vision, 25
professionalism, 108
program evaluation review technique (PERT) charts, 56–57, 57*f*, 79
program management, 15–1]7, 39

description of, 66–67
program management office, 68–72, 70*f*–72*f*
program manager, 67
vs. project management, 40*f*
sponsorship, 67
strategic focus of, 67
program management office (PMO), 68–69
assigned model, 71–72, 71*f*
embedded model, 72, 72*f*
matrixed organization, 69–71, 70*f*
program manager, 67
programming manuals, 237–239, 237*f*, 238*b*, 239*f*
program sponsorship, 67
project artifacts, 45–46, 46*b*
project charter, 46–47
project close, 64–65
project dashboards, 58
project management, 15, 17, 18*f*, 39
definition of, 40
design and plan phase, 47–51
implementation/execution phase, 58
communication, 59–61
tasks, 58–59
initiation/conception/analysis, 41–47
monitor and control, 61
finances, 63
operating rhythm, 62
quality and conformance, 62–63
recording and resolving issues, 63
risks, 63
schedule maintanance, 61
scope creep, 63
stakeholder expectations, 63–64
task tracking, 62
phases of, 41, 66–67, 67*f*
vs. program management, 40*f*
project close, 64–65
project plan *See* (project plan)
Project Management Institute, 40*b*
project manager, 263
project plan
definition of, 51
features, 53
metrics, 55–56, 56*f*
milestone chart, 55, 55*f*
outline, 51*b*–52*b*
PERT charts, 56–57, 57*f*
project dashboards, 58
risks and assumptions, 52–53
work breakdown and Gantt chart, 53–55, 54*f*
protected health information (PHI), 142, 144
Protect the Software (PS), 149

prototyping development model, 12–13, 13*f*

Q

QA. *See* quality assurance
QC. *See* quality control
quality assurance (QA), 84, 155
  auditing and monitoring processes, 158
  life cycle, 156*f*
  processes, 156–157
  task expectations, 158
  tools, 157
  training, 158
quality control (QC), 155, 158–160
quality expectations, 156
quality management, 155
  quality assurance, 156–158
  quality control, 158–160
  quality organizations, 160
quality measures, 159
quality organizations, 160
queueing theory, 82–83
queues, 77–79, 82–83

R

reaction team, 310–311
refinement techniques, of architecture, 201–202, 201*b*
regression error, 316
regression testing, 276, 276*f*
regulations and laws. *See* laws and regulations, of compliance
rehearsal, 295–296
release. *See* product release
release authority, 288
release criteria, 288
release plan, 288
release requirements, 257–258, 258*b*
release review, 288–290
reliability, 190
removal from service, 338–339
repeatability, 2
request for proposal (RFP), 129, 132–133, 133*b*
requirements, 115
  formatting, for sharing, 129
    request for proposals, 132–133, 133*b*
    SRS documents, 129–132, 132*f*
  functional, 115
  functional security, 116, 116*b*
  nonfunctional, 115
  requirements engineering, 126–128
  risk assessments, 127–128
  scope boundaries, 118
  secure functional, 116, 116*b*

security, 116–117
sources of, 118–119
intended security practices, 125–126
outsourcing and cloud services, 126
stakeholders *See* (stakeholders)
system definition, 117
traceability, 128–129, 129*f*
requirements engineering, 126–128
requirements solicitation, 121–122
requirements tracking, 94, 94*f*
resilience/robustness, of system, 196
Respond to Vulnerabilities (RV), 150
responsibility, 101, 107
ethical integrity, 107–108
long-term thinking, 108–109
professionalism, 108
responsibility boundaries, 126
retirement, 10
communication, 334–335
decommissioning hardware, 340–341
definition of, 327
final step in, 341
licensing adjustments, 341
pre-retirement audit, 335–337
progressing through, 333*f*
removal from service, 338–339
retirement plan, 335
timeline, 337–338
transferring and storing data, 339
retirement plan, 335
retirement timeline, 337–338
revision control. *See* version control systems
RFP. *See* request for proposal
risk acceptance, 209–210
risk appetite, of organization, 99–101
risk assessments, 202
risk avoidance, 209
risk-based decision-making, 84–85
risk management, 209–210
Risk Management Framework (RMF), 290
risk mitigation, 209
risk transfer, 209
roofing threat, 214*b*
root cause analysis, 159, 312

S

Sarbanes-Oxley (SOX) Act, 141
scalability, architectural concern, 188
scalability testing, 281
scatter analysis, of stakeholders, 119–121, 120*f*
scope

architectural concern, 189
of project, 47
scope boundaries, 118
scope creep, 47–48, 63
secondary stakeholders, 119
secure functional requirement, 116, 116*b*
Secure Software Development Framework (SSDF), 148–150
secure software development life cycle (SSDLC), 5*f*, 11*f*
boundaries of, 3–4
defining and planning stage, 6
deployment, of system, 8–9
design and decision stage, 6–7
development stage, 7
end-of-life stage, 10
five Ps combining with, 16–17, 18*f*
management
Agile movement, 93–94, 93*b*
change management, 94–96, 95*b*
development methodologies, 91–92
issue management, 96–97
overlapping areas, 91, 92*f*
requirements tracking, 94, 94*f*
spiral model, 93
waterfall model, 92–93
operations and maintenance, 9–10
retirement, 10
system inception/initiation, 5–6
test and evaluation stage, 7–8
understanding model, 5
unsecure faulty development cycle, 11*f*
variations in, 10–14
development methodologies, 13–14
prototyping development model, 12–13, 13*f*
spiral development model, 12, 12*f*
unsecure faulty development cycle, 11*f*
V-model, imposed on, 250, 251*f*
secure tools, 222–224
security, 189
security configuration, 317
security culture, 221–222
in all products, 103, 103*t*, 104*f*
among programmers and systems administrators, 107
customers, discussed and shared with, 104
leadership, 101
maturity, 105–106, 105*b*
policies and procedures, 106–107
responsibility and authority, 101–102
roles and responsibilities, 103
value on, 104–105
security drift, 301, 301*b*
security-enhanced five (six) forces model, 22–23, 23*f*
security goals, of test plan, 260

security monitoring, 304
security objectives, architecture, 195–196, 195*b*
security requirements, 116–117. *See also* requirements
security testing, 276–277
security theater, 31*b*
security training. *See* training
separation of duties, 125
sequence diagram, 175–176, 175*f*, 176*f*
service discipline, 82–83
service-level agreement (SLA), 126, 306
service-oriented architecture (SOA), 198
shadow IT organizations, 35*b*
single responsibility principle, 192–193
Six Sigma, 86–87
SLA. *See* service-level agreement
SOA. *See* service-oriented architecture
software-as-a-platform (SaaP), 27
software-as-a-service (SaaS), 27, 233*b*
software rots, 2, 2*f*
software sources, in development environment, 224–226
Software/Systems Requirements Specification (SRS) document, 129–132, 132*f*
software updates, 25*b*, 317
SolarWinds, 150*b*
SOPs. *See* standing operating procedures
source control. *See* version control systems
sourcing, 26
SOX Act. *See* Sarbanes-Oxley Act
space-based architecture, 200
spatial proximity, 191
special cases, testing, 269
specialized training, 110
spike testing, 281–282
spiral model, 12, 12*f* , 93
SRS document. *See* Software/Systems Requirements Specification document
SSDF. *See* Secure Software Development Framework
SSDLC. *See* secure software development life cycle
stack. *See* last-in first-out (LIFO) queue
stakeholder expectations, 63–64
stakeholders, 119
  JAD phases, 123, 123*f*
  primary, 119
  requirements solicitation, 121–122
  scatter analysis of, 119–121, 120*f*
  secondary, 119
  working with, 123–124
standards, 137
standing operating procedures (SOPs), 78, 236
state diagrams, 176–178, 177*f*
static code review, 273–274
static process analysis, 80–81, 81*f*
static testing, 272
stress testing, 281

structure charts, 168–169, 169*f*
Structured Threat Information eXpression (STIX), 217
style guides, 237–239, 237*f*, 238*b*, 239*f*
subject matter expert testers, 264
supplier power, Porter's model, 22
swim lanes, 79
swivel-chair interface, 34*f*
system boundaries, 117
system definition, 117
system integration testing, 275–276
system logs, 303–304
systems monitoring, 303
system testing, 277–278

T

tasks, in project, 53–54
technical architecture, 186
technical goals, of test plan, 260
technology-related compliance regulations and laws, 140
temporal proximity, 191
terminal-based design pattern, 197
test cases, 269–271, 270*f*
test-driven development, 252–253
test environment, 269
testers, 261, 263–264
test goals, 257
test harness, 252*b*, 253, 273
testing
  alpha, 278
  beta, 278–279
  black box, 272
  compatibility, 284–285
  definition of, 249
  dynamic, 272
  errors of commission, 250, 252
  errors of omission, 250
  exploratory, 283–284
  final systems test, 277–278
  fuzz, 283
  integration, 275–276
  load, 280–281
  peer review, 274–275
  penetration, 284
  performance, 280
  quality control, 159
  regression, 276, 276*f*
  scalability, 281
  security, 276–277
  spike, 281–282
  static, 272
  static code review, 273–274

stress, 281
system, 277–278
system integration, 275–276
test-driven development, 252–253
test plan *See* (test plan)
test team, 252
unit, 273
usability, 282–283
user, 278
user acceptance, 279
user integration, 279–280
user interface, 282
V-model, 250, 250*f*, 251*f*
white box, 272
test manager, 263
test plan, 253–254
components of, 254*b*
developers, 264–265
functional testing goals, 258–259
performance goals, 260
project manager, 263
regulatory, legal, and compliance goals, 259–260
release requirements, 257–258, 258*b*
risk-based test strategy, 254–257, 255*b*–257*b*
security goals, 260
systems administrators, 265
team roles, 263
technical goals, 260
test cases, 269–271, 270*f*
testers, 263–264
test goals, 257
testing tools, 271
test manager, 263
test schedule, 267–268
test team, 261–263
test tracking and communication, 265–267
test schedule, 267–268
test scripts, 269
test strategy
defect risk, levels of, 256–257, 257*b*
risk analysis, 254–255
stack overflows, using canaries to detect, 255, 255*b*–256*b*
test team, 62, 261–263
test tracking, 265–267
threat analysis, 212, 213
threat intelligence, 216–219
threat model, 213, 218
classes of potential threats, 218
favored vulnerabilities, 218–219
potential threats, 218
threat of new entry, Porter’s model, 21
threat of substitution, Porter’s model, 21–22

threats, 212, 213*b*
  environmental, 213–216
  external, 215–216
  intelligence, 216–219
  purpose of the system, 213–216
  user, 215
traceability, 128–129, 129*f*, 268, 268*f*
training, 100, 292
  administrator, 292–293
  awareness over time, 111, 111*f*
  description of, 109
  importance of, 109–110
  quality assurance, 158
  style, 111–112
  user, 292
transaction behavior, 82
transactions, 76
transition path, 329, 330
transition period, 329
trust boundary, 210–211
Trusted Automated eXchange of Intelligence Information (TAXII), 217
trust relationships, 202
trust zone, 202

U

Unified Modeling Language (UML), 173, 174
unit testing, 273
Unix operating system, 301–302
unordered queue, 77
unsecure defaults, 223
unsecure faulty development cycle, 11*f*
usability testing, 282–283
use case, 49
use case modeling, 174–175, 174*f*
use model, 8*b*
user acceptance testing, 279
user integration testing, 279–280
user interface testing, 282
user support, 304–306
user testing, 278
user threats, 215
user training, 292
U.S. regulations and laws, 140–143

V

validation, definition of, 249. *See also* testing
verification, efinition of, 249. *See also* testing
version control, 244–245
version control systems, 244–245
version labels, 244
V-model, 250, 250*f*

with SSDLC imposed, 250, 251*f*
vulnerabilities, 2
definition of, 206*b*
development tools, 223
inherited, 231
management, 317–318
potential, 206, 207
risk management, 209–210
vulnerability assessment
attack vector, 212
goals of, 205–206
life of data, 211–212
trust boundary, 210–211
vulnerability databases, 217
vulnerability mapping
goals of, 205–206
process of, 206–207, 207*f*
vulnerability matrix, 207–209, 208*f*

W

waterfall model, 92–93
WBS. *See* work breakdown structure
white box testing, 272
work breakdown structure (WBS), 50–51, 54, 54*f*
workflow system, 83

An instance of software rot. Windows displays a message indicating that a particular app cannot run on the P C. This is due to the fact that the app was built for an earlier version of the P C.

Back to Figure

The series of steps in the secure software development life cycle model. System inception or initiation. Defining and planning. Design and decision. Low level design and development. Test and evaluation. Deployment. Operations and maintenance. Back to defining and planning or exit with End-of-life.

Back to Figure

The order of steps in the secure development cycle as it often happens. The steps in the secure software development life cycle model is indicated as follows. System inception or initiation. Defining and planning. Design and decision. Low level design and development. Test and evaluation. Deployment. Operations and maintenance. Back to defining and planning or exit with End-of-life. This order may not be followed in reality and many of steps may need to revisited. This is indicated by arrows that criss-cross across the steps in the model.

Back to Figure

The steps in the unsecure faulty development cycle. Development. Text and evaluation. Design and decision. Defining and planning. Re-development. End-of-life. The following are two steps defined individually and separately. Deployment. Operations and maintenance.

Back to Figure

The steps in the spiral development model. System inception or initiation. Defining and planning. Design and decision. Development. Test and evaluation. Design and decision. Development. Test and evaluation. Design and decision. Development. Test and evaluation. Design and decision. Development. Test and evaluation. Design and decision. Development. Test and evaluation. Deployment. Operations and maintenance. End-of-life.

Back to Figure

The steps in the prototyping development model. System inception or initiation. Defining and planning. Development. Back to defining and planning. Design and decision. Development. Test and evaluation. Deployment. Operations and maintenance. Back to Defining and planning. End-of-life.

Back to Figure

The steps in project and processes. The project to develop a new system has the following steps. Initiation. Defining. Testing. Operations. Initiation involves the project initiation process. Defining involves the requirement gathering process. Testing involves test processes. Operations involves general operations processes. These four steps involve project management and reporting processes. They are standard processes used to accomplish a project. The defining step involves the following as well. General concept of operations and user and administrator guides. These are processes created for a specific system. The operations step involves system specific operations processes.

Back to Figure

The steps in the periodic portfolio review process. System needs assessment. Portfolio capabilities. Shortfalls and gaps review. System development process.

Back to Figure

The Ps of marketing are Product, Price, Positioning, Place, People, and Promotion.

Back to Figure

The five forces model of Porter. The model shows Competitive rivalry in the center. Contributing to competitive rivalry are the following. Threat of new entries. Threat of substitution. Supplier power. Buyer power.

Back to Figure

The security enhanced five (six) forces model. The model shows Competitive rivalry in the center. Contributing to competitive rivalry are the following. Threat of new entries. Threat of substitution. Supplier power. Threats of cyber security. Buyer power.

Back to Figure

In the product quadrant, the product’s market share and the market’s growth rate are plotted into four quadrants.

Back to Figure

A swivel-chair with a stop sign on it. This refers to the infamous swivel-chair interface which is not preferred.

Back to Figure

The relationship between programs and projects. The program involves projects 1 to 5 which extend to different time periods.

Back to Figure

An example of a Gantt chart. A Microsoft Excel sheet is pictured containing a Gantt chart.

Back to Figure

An example of a milestone chart. The milestone chart features a horizontal timeline. The timeline indicates the following dates and events. October 22, 2017: Meet with stakeholders to vote on project, Tuesday, October 10, 2017. November 5, 2017. November 19, 2017: Gain requirements approval, Wednesday, 22 November, 2017. December 3, 2017. December 17, 2017. December 31, 2017: Complete execution: Wednesday, 3 January, 2018. January 14, 2018: Finish, Wednesday, 17 January, 2018. Close project: Wednesday, 17 January, 2018.

Back to Figure

A metric example. A bar chart with a trend line for requirements burndown. The horizontal axis marks the total and months from January to next year, January. The vertical axis measures the number of requirements remaining from 0 to 1,800 in intervals of 200. The following are the details captured in the graph. Total: 1589. January: 1575. February: 1541. March: 1487. April: 1470. May: 1314. June: 1146. August: 933. September: 750. October: 446. November: 306. December: 239. January: 40. The trend line is a straight line that slopes downwards from 1589 requirements to 40 requirements.

Back to Figure

A PERT chart example. A flowchart is indicated with the following steps. Step 1: Build business case: January to March. Step 2: Plan Project: March to April. Step 3: Lay house foundation: May 10 to June 10. Step 4: Build house frame: June 15 to July 20. Step 4 goes to Step 5 and Step 6 side by side. Step 6: Electrical panel and wiring: July 15 to July 30. Step 6: Build plumbing: July 10 to July 30. Steps 5 and 6 ends with Step 7. Step 7: Complete house: August 30.

Back to Figure

The phases in project management. Initiation: Present idea. Research feasibility. Define: Gather requirements. Document requirements. Gain requirements. And Plan: Use requirements to decide approach. Implementation and execution: Create product. Test product. Monitor and control: Monitor and control progress in implementation. Mitigate any risks. Mitigate any issues. Close: Release product to client. Perform ongoing maintenance.

Back to Figure

A matrix of P M O in the organization. Under the P M O are three P Ms. One of the P Ms is a direct report to a Project Owner. There are specialized technical teams. With a technical manager and team members. The P Ms have some team members with them. The team members are directs reports to technical managers. The team members are direct reports to their P Ms.

Back to Figure

Assigned P M O in the organization. Under the P M O are three P Ms. A Project Owner directly reports to three Technical Managers. Each

Technical Manager indirectly reports to a P M. There are varied technical team members who directly report to the Technical Managers.

Back to Figure

Embedded P M O in the organization. Under the P M O are three P M who are indirect reports. A Project Owner directly reports to three Technical Managers. A Technical Manager directly reports to a P M. There are varied technical team members who directly report to the Technical Managers.

Back to Figure

The components of processes. Steps are organized into flows. A step is a single discrete action, set of actions, procedure, or set of procedures. Rectangle with rounded side is used for start or end states. A rectangle is used for actions. A flow is the organization and ordering of the steps of a process. A line with an arrow is used for transitions. Transactions. Transactions are tasks, steps, or groups which traverse the flow, or portions of a flow, as an entity. Transactions are split or combined with are represented by relevant shapes. Decisions. Decisions are points where the flow changes for individual transactions based on intrinsic or extrinsic factors. A diamond shape is used for decisions. Queues. Queues are where transactions wait for an action to be performed or a decision to be taken. A rectangle shape with partitions is used for a queue.

Back to Figure

A typical process diagram. The following details are shown in the diagram. An employee initiates a vacation request with a manager. When he approves it, the H R checks the vacation balance available

and transfers the process to finance. Finance adjusts the payroll and the process is complete. If H R finds that vacation balance is not available, then H R requests override with manager. If manager approves it, then the process goes to Finance who adjusts the payroll and the process is complete. If approval is not given by manager for override request then the employee may try again with the manager In the initial case when the vacation request goes to the manager and he does not approve it, the employee may try again. Then, the employee goes back to work.

Back to Figure

A diagram on adding parallel capacity. Two diagrams are shown. In the first one, there are three requests in queue and one lengthy process is initiated. In the second one, there are three requests in queue and three lengthy processes are initiated.

Back to Figure

Steps in the secure life cycle with supporting concepts. Overlapping ovals are indicated in the diagram. These ovals represent process management, project and program management, product management, and portfolio management. The steps in the secure life cycle are the following. 1. System initiation. 2. Defining and planning. 3. Design and decision. 4. Low level design and development. 5. Test and evaluation. 6. Deployment. 7. Operations and maintenance. 8. Back to Defining and planning. 9. End-of-life. Steps 1, 2, 8, and 9 portfolio management steps. Steps 3 and 6 are project/program management and product management steps. Steps 4 and 5 are project and program management steps.

Back to Figure

The steps in requirement tracking through development. A requirement includes the following three things. High level design. Test plan. Documentation. High level design involves a low level design and unit dev. Unit dev goes on to testing and integration steps. Integration goes on to a testing step. Test plans go through testing steps. Integration is based on documentation.

Back to Figure

Some C I S O reporting approaches. The following are two good approaches. First good approach. There are the C I S O and C I O who report to the C E O. The C E O reports to the B o D. Second good approach. The C I S O reports to the Corporate Secretary of Audit. The C I O reports to the C E O. The Corporate Secretary and C E O report to the B o D. The following are two bad approaches. First bad approach. The C I S O reports to the V P. The V P reports to the C I O. The C I O reports to the C E O. Second bad approach. The C I S O reports to the C I O and the C I O reports to the C E O.

Back to Figure

The areas to consider security. The following list is a series of steps. Portfolio planning. Requirements. Design. Development. Deployment. Support.

Back to Figure

Training refreshes awareness over time. The training time is shown as a line with crests and troughs. The crests and troughs flatten out from left to right.

Back to Figure

A scatter analysis of stakeholders. A graph is indicated plotting interest against influence. The graph is divided into four quadrants. The first quadrant shows primary sources. The second quadrant shows sources of ideas. The third quadrant shows limited input. The fourth quadrant shows sources of support.

Back to Figure

J A D phases are the following. Define objectives. Session preparation. Session conduct. Documentation.

Back to Figure

The steps in requirements traceability. Step 1: Requirements: Gathered from multiple sources. Step 2: Requirements: Sorted, assessed, engineered. Step 2 flows on to Steps 3 and 4. Step 3: Requirements: Included in design, assigned to modules. Step 4: Requirements: Drive test plans. Step 5: Requirement: Implemented in modules. Step 4 and Step 5 flow on to Step 6: Requirements: Validated, verified. Step 7: Requirements: Met in deployed system.

Back to Figure

The following are the documents that are present in the early life cycle. Project charter. Project plan. Project plan follows with Schedule and Budget. Project plan follows with Requirements repository. Requirements repository follows with S R S and R F P. S R S follows with Architecture high level design and Test plan. Architecture high level design also follows with Test plan, Component S R S and Component R F P.

Back to Figure

COBIT 5 principles are the following. 1. Meeting stakeholder needs. 2. Covering the enterprise end-to-end. 3. Applying a single integrated framework. 4. Enabling a holistic approach. 5. Separating governance from management.

Back to Figure

The C M M I framework progression. The diagram shows a pyramid with the C M M I progression listed from bottom to top. Level 1 – Initial. Provides services and products. Reactive responses. Level 2 – Managed. Add project management. Level 3 – Defined. Proactive. Identified standards for I T. Level 4 – Quantitatively managed. Quantitative performance goals. These goals align to business needs. Level 5 – Optimizing. Continuous improvement but flexible and innovative organization.

Back to Figure

The components in the Q A life cycle are plan, execute. Verify, and update process.

Back to Figure

Examples of symbols that are used in flowcharts. The symbol for Start and End symbol is a rectangle with rounded edges. The symbol for Decision is a diamond shape with arrows for true and false decisions. The symbol for Sequence or action is a rectangle. The symbol for iteration is a hexagon with an arrow mark leading to itself and an arrow leading away from the iteration.

Back to Figure

Example flowchart for a bubble sort. Step 1: Start. Step 2: b array equals the following elements. 4, 6, 2, 8, 3, and 1. n equals 6. Step 3: for i equals i to n minus 1. Go to Step 4 and Step 7. Step 4: for j equals n minus 1 to 1. Got to Step 3 and Step 5. Step 5: Decision: Check if b of j less than b of j minus 1. If True, then Step 6. If False, then Step 4. Step 6: temp equals b of j. b of j equals b of j minus 1. b of j minus 1 equals temp. Go to Step 4. Step 7: End.

Back to Figure

The structure chart for car purchase analysis program. Step 1: Car purchase analysis. Step 1 flows on to Steps 2, 3, and 4. Step 3: Capture inputs. Step 3: Calculate payments and financial information. Step 4: Display results. Step 2 flows on to Steps 5 and 6. Step 5: Capture inputs. Step 6: Validate inputs. Step 3 flows on to Steps 7 and 8. Step 7: Determine calculations. Step 8: Calculate financials. Step 8 flows on to Steps 9, 10, 11, and 12. Step 9: Calculate payment. Step 10: Calculate interest rate. Step 11: Calculate down payment. Step 12: Calculate total payment.

Back to Figure

Relationship types in E R diagrams. A car has an engine: A one to one relationship. A car has many tires: A one to many relationship. Many cars have many drivers: A many to many relationship.

Back to Figure

An example E R diagram. The diagram shows many boxes which are class boxes with the name of the class and their attributes defined in them. The boxes are connected to each other with lines which indicate the relationship types between the classes.

Back to Figure

An object diagram overview. The diagram shows instance specifications, value specifications, slots, and links.

Back to Figure

A use case model for a Bitcoin A T M. The actors in the use case model are the end user, maintainer, exchange, and bank or financial institution. The following are the use cases indicated. An end user can do the following. Make an exchange rate enquiry. Buy bitcoin. Sell bitcoin. Deposit cash. Collect cash. Transfer money to and from bank. A maintainer puts cash in the machine. An exchange does the following. Make an exchange rate enquiry. Validate a wallet. A bank or financial institution can validate an account and transfer money to and from a bank.

Back to Figure

A sequence diagram example for a web-based image system. The diagram shows the following systems. Web client. Image service. Authenticator. Info server. Image server. A web client makes a login request to the image service. An authentication request goes from the image service to the authenticator. The authenticator returns an authentication status to the image service which sends it to the web client. The web client sends an image request to the image service. The image request is passed on by the image service to the info server. The info server sends related information back to the web client. The info server also passes on the image request to the image server. The image server sends the image back to the web client. The same kind of requests are repeated for more images requested by

the web client. The web client finally sends a logout request to the image service.

Back to Figure

A diagram indicating details with gap in sequences. A section of the sequence diagram is shown. The sequences have gaps in them.

Back to Figure

A state diagram showing the various states and transitions in a simple traffic light controller. In the initial state, the traffic lights are all blinking red. Once the self checks are successful, the traffic light enters the first state in which the east-west or E W lights are green and the north-south or N S lights are red. The light remains in this state for the time in Timer 1. In the next state, the E W lights turns yellow and the N S lights remain red. The time of the yellow light is determined by Timer 2. Once this timer finishes, the next state is entered in which the E W lights are red, and the N S lights are green. The next transition happens after Timer 3. The next transition happens after Timer 2. The controller meanwhile has come to the second state. The transitions continue as described above until a fault occurs, or the system is shut down.

Back to Figure

Business architecture showing relationships among systems. A C R M computer system is indicated. The C R M system takes care of order entry, manufacture, and shipping. Manufacture and shipping details are processed in inventory management. Order entry and inventory management are stored in a data warehouse.

Back to Figure

Logical architecture for a water data collection buoy. System management module controls the data collection and process system. The system management module and communications module control each other. Data goes back and forth between the communications module and data store. Data from the data collection and process system is stored in the data store. The following sensors also store data in the data collection and process system. Atmospheric sensors module. Surface sensors module. Two meter sensors module.

Back to Figure

Application architecture of the water data collection system. Data is exchanged between the units indicated below. Buoy. Collection app. Data. Web server and analysis engine. Web interface.

Back to Figure

An example vulnerability map for a web-based news reader. The map shows the interactions occurring between the user of the browser app and the news database server using labels and descriptions. The potential vulnerabilities are indicated using numbered diamonds.

Back to Figure

An example of a vulnerability matrix. A table listing the potential and actual vulnerabilities identified for a system. The matrix indicates the nature and location of the vulnerabilities along with details of likelihood, severity, risk, and preventative measures.

Back to Figure

Another example of a vulnerability matrix. A table detailing the vulnerability, severity, likelihood, description, and handling and remediation details.

Back to Figure

An N-order scope map. A series of steps are shown as listed below. Data source. Data process. Data store. Data used. Remote interface. Data process. A call out indicates that the use is in the data used process.

Back to Figure

Lurking dependency issues which are like an iceberg. An iceberg is shown. The part of the iceberg above the water line is the code-base. The part under the water line includes the libraries and operating system, libraries of libraries, libraries of libraries of libraries, and deeply hidden dependencies.

Back to Figure

The different options for placement of braces in code blocks. The examples indicated contain different ways to place the opening brace and closing brace used in a for loop.

Back to Figure

Source code versioning. The example shows the different branches of code in development that has gone on to be released as release versions 1.0, 2.0, 3.0, and 4.0. Six branches of code are shown named l o r g, Release 2, Mar, V Release 1.0, Release, and Master.

Back to Figure

An example of a testing V-model. The following steps are shown in four levels on the left side of the V. Business case and statement of work. Business requirements. Functional design. Technical design and coding. The work flow is from top to bottom. The following steps are shown in four levels on the right side of the V. Production verification. User acceptance testing. System and integration testing. Unit testing. The work flow is from the bottom to top on this side. There is a testing relationship between the items listed on the four levels of both sides of the V. Validation and verification is done on the items listed on both sides of the V.

Back to Figure

An example of a V-model with S S D L C imposed on it. The following steps are shown in four levels on the left side of the V. Business case and statement of work. Business requirements. Functional design. Technical design and coding. The work flow is from top to bottom. The following S S D L C steps are also indicated. System inception or initiation. Defining and planning. Design and decision. Development. The following steps are shown in four levels on the right side of the V. Production verification. User acceptance testing. System and integration testing. Unit testing. The work flow is from the bottom to top on this side. The following S S D L C steps are also indicated. Operations and maintenance. Deployment. Test and evaluation. End-of-life. There is a testing relationship between the items listed on the four levels of both sides of the V. Validation and verification is done on the items listed on both sides of the V.

Back to Figure

A diagram that shows the content of a stack when canaries are used to detect stack overflows. The function that is executed is s t r c p y of

the string that contains the letters from a to z. The stack content before overflow is the following from bottom to top. 0040107 C: Parameter 2. 00401078: Parameter 1. 00401074: Return address. 00401070: Saved E B P. 0040106C: integer. 00401068: String of 3, 4, 5, 7. 00401064: String of 0, 1, 2, 3. 00401060: integer. The stack content after overflow is the following from bottom to top. 0040107 C: y z backslash 0. 00401078: u v w x. 00401074: q r s t. 00401070: m n o p. 0040106C: i j k l. 00401068: e f g h. 00401064: a b c d. 00401060: integer. The stack content with canary is the following from bottom to top. 0040107 C: Parameter 2. 00401078: Parameter 1. 00401074: Return address. 00401070: Canary. 0040106C: Saved E B P. 00401068: integer. 00401064: String of 3, 4, 5, 7. 00401060: String of 0, 1, 2, 3. integer.

Back to Figure

A requirements traceability diagram. A flowchart like diagram shows requirements 1 and 2. Requirement 1 needs Modules A and B. It includes Tests 1.1, 1.2, and 1.3. Tests 1.1 and 1.2 are done on Module A. Test 1.3 is done on Module B. Requirement 2 needs Modules C and B. It includes Tests 2.1, 2.2, and 2.3 which are done on Module B.

Back to Figure

Some flowcharts that show examples of possible paths through a simple model of a program. The exhaustive steps in the flowchart are from A to G. The first flowchart shows the following sub path. B to D, E, F, and then G. The second flowchart shows the sub path A to C and then G. The third flowchart shows the sub path A to C and then G and also D to E, F, and then G. The fourth flowchart shows the sub path A to C and G and also E to F and G.

Back to Figure

A line graph depicting the classic bathtub reliability curve. The graph plots time from development in the horizontal axis against defects identified in the vertical axis. The graph shows a curve that has a classic bathtub outline.

Back to Figure

The following is the timeline for system end-of-life. Accept transition path. Drop use of the system. Use as opportunity. Continue unsupported.

Back to Figure

The alternatives when facing system end-of-life. A diagram with a timeline is indicated on which end of life is indicated. Before end of life, we have the following. Decision making. End of life and transition planning. Announcements to customers and stakeholders. Customer transition period. Support loads. After end of life, we have archiving.

Back to Figure

The series of steps that are followed when progressing through retirement. Pre-retirement audit. Retirement plan development. Transition or cut-over. Removal from service. Transfer and store data. Decommissioning and disposal.

Back to Figure